LEDGER：理解与计算分离的可追溯 A 股投研多智能体系统

LLM-driven Equity Research with Deterministic Grounding, Evaluation and Reporting

完整技术报告

陈沫樵[1,*,†]　许冰洁[1,*]　赵千淳[2,*]　刘盈羽[1,*]

[1] 对外经济贸易大学　[2] 中国人民大学

mochiaochen@gmail.com

代码：https://github.com/MochiaoChen/finagent

摘要

大语言模型（LLM）擅长读懂长文档、组织论证与写作，却在投研场景中有三类不可接受的失败：编造或记错财务数字、在增速与估值上心算出错、凭参数记忆补写没有出处的外部信息。改写环节还会引入新的数字错误，而研报质量本身缺乏可复核的评价，使“改进”难以度量。本文完整介绍 LEDGER（**L**LM-driven **E**quity Research with **D**eterministic **G**rounding, **E**valuation and **R**eporting）——一个面向 A 股上市公司的可追溯投研多智能体系统。它围绕财报分析、自动化估值建模与研究报告质量评估三个主选题构建，并覆盖数据提取、产业链分析与研究报告纠错核查三个子选题；它能把一份年报加工为 25 ~ 32 页、60 个以上图表的卖方体例深度研报与公式联动的估值工作簿。

系统的核心原则是**理解与计算分离**：模型负责读、抄、写、判断，确定性程序负责校验、计算、拦截。我们把这一原则落实为一组可测试的不变量与对应机制：（1）**证据约束的双通道抽取**——模型抄录的每个数字必须在所写页码的原文中按数字边界找到，规则通道独立取数交叉校验，冲突不静默取舍而是阻断估值；（2）**单一真值表**驱动的分析与估值链——分部量价预测、三表联动、三情景 FCFF 折现、逐项股权价值桥、反向 DCF 与可比公司，评级由程序给出；（3）按研报**展示精度**（半个末位）设定容差的纠错与护栏，误报控制优先于召回；（4）**数字锁**，保证模型改写只能删减、不能新增或改动数字；（5）由确定性编排器驱动、五个上下文隔离的无头 Claude Code 会话（研究助理、分析师、主笔、复核编辑、排版设计师）与 **11 道程序闸门**构成的研报流水线，先搜集外部资料、后分析写作；（6）四层评分标准与**对生成方不可见**的盲评闭环，并用重复评审、程序制造缺陷与人工复核三组证据**校准评审模型本身**。

在三家公司的真实年报上，模型抄录并通过证据校验的 301 个数值中，规则通道独立取到且一致的有 280 个（93.0%），冲突为 0，勾稽 56/56 全部通过；流水线引入先搜集与深度专题后，两家公司深度研报

* 共同第一作者。　† 通讯作者。

2026 北京市大学生金融人工智能竞赛参赛作品（主题：金融投研智能体构建）。文中估值结论均依赖所列假设，不构成投资建议。

的质检得分由 74、67 分提升至 90、88 分（同一评分标准、单次盲评）；评审模型重复盲评的逐条一致率为 86%～91%，判出问题经人工确认的精确率为 84%。本文逐一拆解每个模块的动机、形式化定义、算法、实现与设计权衡，讨论系统的演进史与失败的尝试，并如实报告证据的规模与局限。所有程序计算均可在保存的输入上离线逐位复算。

目录

1 引言 **9**

1.1 投研场景与卖方研报的生产过程 . . . 9
1.2 让 LLM 直接读年报写研报：六类失败模式 . . . 10
1.3 本文立场：模型读、抄、写、判断；程序校验、计算、拦截 . . . 12
1.4 系统概览 . . . 13
1.5 一个贯穿全文的例子：一个数字的一生 . . . 15
1.6 贡献 . . . 16
1.7 结果概览 . . . 17
1.8 范围与非目标 . . . 17
1.9 全文导读 . . . 18

2 背景与相关工作 **19**

2.1 领域背景：A 股定期报告 . . . 19
2.2 领域背景：卖方研报体例与估值方法 . . . 22
2.3 金融数值推理与财报问答 . . . 23
2.4 程序辅助推理与工具使用 . . . 24
2.5 金融大模型与金融智能体 . . . 24
2.6 多智能体框架 . . . 25
2.7 自我改进与反思 . . . 25
2.8 幻觉与可追溯生成 . . . 26
2.9 LLM 评审与基于评分细则的评测 . . . 26
2.10 编码智能体 harness . . . 27
2.11 对比与定位 . . . 27
2.12 小结 . . . 28

3 问题形式化 **28**

3.1 记号总览 . . . 29
3.2 文档、页与证据 . . . 29
3.3 指标、期间与事实 . . . 30
3.4 双通道抽取与真值表 . . . 31
3.5 任务一：数据提取与财报分析 . . . 32
3.6 任务二：自动化估值建模 . . . 33
3.7 任务三：研究报告纠错核查 . . . 35
3.8 系统性质 . . . 36
3.9 单位与口径约定 . . . 38
3.10 威胁模型与系统不变量 . . . 39
3.11 研报生成任务的形式化 . . . 41
3.12 评价问题的形式化 . . . 42
3.13 小结 . . . 44

4 设计原则与系统不变量 **44**
4.1 原则的分类与执行强度 . . . 44
4.2 P1 LLM 不做算术 . . . 45
4.3 P2 每个数字可追溯 . . . 47
4.4 P3 双通道不静默取舍 . . . 48
4.5 P4 一张真值表 . . . 49
4.6 P5 所有读写走 tracing . . . 50
4.7 P6 单位约定 . . . 51
4.8 P7 误报控制优先于召回 . . . 52
4.9 P8 事实 → 推论 → 观点 . . . 53
4.10 P9 模型输出加护栏 . . . 54
4.11 P10 会话隔离 . . . 54
4.12 P11 不预设行业框架，文风不做程序拦截 . . . 55
4.13 执行路线的收敛：从内嵌模型调用到无头 Claude Code . . . 56
4.14 原则之间的张力与优先级 . . . 57
4.15 原则—机制—测试总表 . . . 58
4.16 小结 . . . 59

5 系统架构 **60**
5.1 四层结构 . . . 60
5.2 模型与引擎的唯一接口：文件与命令行 . . . 60
5.3 计算引擎的内部结构 . . . 63
5.4 三条流水线 . . . 65
5.5 命令行、产物目录与环境自检 . . . 66
5.6 终端界面 . . . 67
5.7 跨平台 . . . 68
5.8 依赖最小化 . . . 68
5.9 可复现性的两层定义 . . . 69
5.10 golden 快照与参照对照：自研引擎与原插件的关系 . . . 70
5.11 测试体系 . . . 71
5.12 小结 . . . 73

6 文档解析与证据约束抽取 **73**
6.1 问题与总体设计 . . . 73
6.2 文档模型：分页、chunk 与表格 . . . 75
6.3 关键页定位：`locate` . . . 76
6.4 指标字典 . . . 77
6.5 表头期间识别 . . . 77
6.6 规则通道与真实年报版式难点 . . . 78
6.7 数字解析与规范化 . . . 82
6.8 模型通道：`facts.json` 与证据校验 . . . 82
6.9 双通道合并 . . . 84
6.10 勾稽校验在抽取中的角色 . . . 85
6.11 经营数据表：`extras` . . . 85

6.12 季报与补充文件 . . . 86
6.13 verify-facts 与 verify-extra 的完整流程 . . . 87
6.14 真实年报上的结果 . . . 88
6.15 失败案例与边界情况 . . . 90
6.16 小结 . . . 90

7 财报分析 91
7.1 流水线与产物 . . . 91
7.2 真值表的构造 . . . 91
7.3 同比与环比 . . . 92
7.4 单季拆分与分季度勾稽 . . . 93
7.5 勾稽校验 . . . 94
7.6 异常信号 . . . 95
7.7 利润变动归因桥 . . . 97
7.8 实际 vs 预期、业绩预告与 EPS 质量 . . . 98
7.9 经营数据在分析中的作用 . . . 100
7.10 报告输出 . . . 100
7.11 样例与真实公司上的结果 . . . 101
7.12 失败案例与边界情况 . . . 103
7.13 小结 . . . 104

8 估值建模 104
8.1 估值链总览 . . . 104
8.2 分部量价预测 . . . 106
8.3 三表联动模型 . . . 108
8.4 FCFF DCF . . . 112
8.5 情景、敏感性表与流动性压力测试 . . . 114
8.6 反向 DCF：参考股价隐含了什么 . . . 115
8.7 可比公司估值 . . . 116
8.8 盈利预测摘要与程序评级 . . . 117
8.9 公式联动工作簿 . . . 118
8.10 与参照引擎的口径一致性 . . . 120
8.11 端到端数值示例：星河智造 . . . 120
8.12 真实公司：茅台、宁德时代、招商轮船 . . . 121
8.13 局限与小结 . . . 122

9 研报纠错 123
9.1 输出契约：七类判定与一条发现的结构 . . . 123
9.2 断言抽取：规则通道 . . . 124
9.3 断言抽取：模型通道与合并 . . . 127
9.4 真值与按展示精度的容差 . . . 128
9.5 错误类型诊断：搜索空间、排序与巧合误诊 . . . 130
9.6 修改建议的生成 . . . 131
9.7 误报控制的决策论解释 . . . 132

9.8 研报内部一致性 . . . 133
9.9 样例纠错结果与变异实验 . . . 136
9.10 失败模式汇总 . . . 137
9.11 与数字护栏的关系 . . . 137
9.12 小结 . . . 138

10 数字护栏、数字锁与来源校验 138
10.1 威胁与检查的对应 . . . 139
10.2 出处检查 . . . 139
10.3 巧合放行：为什么只在结论性产物里找出处 . . . 142
10.4 指标与期间绑定 . . . 143
10.5 外部来源登记 . . . 145
10.6 前后矛盾 . . . 146
10.7 出处分类：事实、推论与观点 . . . 147
10.8 数字锁 . . . 148
10.9 渲染凭证 . . . 149
10.10 为什么不对文风做程序拦截 . . . 151
10.11 在流水线与质检中的位置 . . . 151
10.12 小结 . . . 152

11 多智能体研报流水线 152
11.1 问题与设计目标 . . . 153
11.2 执行器：无头 Claude Code . . . 153
11.3 `build`：仓库之外的隔离工作区 . . . 156
11.4 五个生产角色与盲评质检员 . . . 159
11.5 十一道闸门：`status` . . . 162
11.6 先搜集、后分析：资料包与深度专题 . . . 166
11.7 `refine`：盲评驱动的返工闭环 . . . 169
11.8 `collect` 与 `scorecard` . . . 171
11.9 四个 skill：方法与约束，而不是行业模板 . . . 172
11.10 排版：reportkit 与渲染实测 . . . 174
11.11 失败案例 . . . 176
11.12 权衡汇总 . . . 177
11.13 端到端运行实例：贵州茅台 1005 批次 . . . 178
11.14 小结 . . . 180

12 评测体系与评审校准 180
12.1 问题与设计目标 . . . 180
12.2 评分标准的来源与四层结构 . . . 182
12.3 程序检查：L0 硬闸门与 L1 数据闭合 . . . 184
12.4 盲评会话 . . . 187
12.5 计分、交付标准与问题单 . . . 189
12.6 评分标准的版本演进 . . . 191
12.7 评审校准 . . . 192

12.8 信息隔离 . . . 197
12.9 辅助工具：命令、查看包、人工质检页与工作簿审计 . . . 197
12.10 使用结果与讨论 . . . 199

13 实验 **200**
13.1 实验设置 . . . 201
13.2 RQ1：真实年报上的双通道抽取 . . . 202
13.3 RQ2：财报分析的确定性部分 . . . 204
13.4 RQ3：研报纠错与容差策略 . . . 205
13.5 RQ4：估值引擎的一致性、闭合与耗时 . . . 208
13.6 RQ5：研报质量 . . . 209
13.7 RQ6：流水线组件消融（初步） . . . 211
13.8 RQ7：评审是否可信 . . . 212
13.9 成本与耗时 . . . 213
13.10 案例研究 . . . 214
13.11 测试套件 . . . 216
13.12 有效性威胁 . . . 216
13.13 跨实验的错误谱系 . . . 216
13.14 小结 . . . 217

14 讨论：设计取舍、演进与教训 **217**
14.1 模型与程序的分工边界 . . . 218
14.2 系统演进：每一步由哪个失败驱动 . . . 219
14.3 为什么用编码智能体做 harness . . . 220
14.4 闸门、评分与 Goodhart 定律 . . . 221
14.5 误报的不对称代价贯穿全系统 . . . 222
14.6 不预设行业框架的代价与收益 . . . 222
14.7 没有成功的尝试 . . . 222
14.8 可迁移性 . . . 223

15 局限、风险与合规 **223**
15.1 评测证据的局限 . . . 223
15.2 计算口径的简化 . . . 224
15.3 工程约束 . . . 224
15.4 使用风险与合规 . . . 224

16 结论与后续工作 **225**

附录 A 评分标准全文 **243**

附录 B 劣化算子定义 **246**

附录 C 数据结构 **247**

附录 D 十一道闸门 **248**

附录 E 角色启动提示要点 250

附录 F 第三方组件 252

附录 G 复现说明 253

1 引言

一份 A 股上市公司的年度报告通常有两百到三百页，其中三张合并报表、母公司报表、附注、管理层讨论与分析、分季度数据、非经常性损益表散落在不同章节，单位在“元”“万元”“百万元”之间切换，表格跨页续排、科目名称折行。卖方分析师从这样一份文件出发，加上年报之后的季报、行情、行业数据和同业估值，写出一篇二三十页的深度研究报告，给出盈利预测、估值区间和投资评级。这件事在券商研究所里是一个由多人、多轮、多道复核组成的生产过程。大语言模型（large language model，LLM）读长文档、归纳论点、写中文长文的能力已经足以覆盖其中的大部分环节，直接让一个模型“读年报、写研报”看起来也确实能产出一篇像样的文稿。

问题在于“像样”与“可信”之间的距离。研报的价值取决于其中每一个数字是否经得起追问：这个营业收入取自哪一页、是合并口径还是母公司口径、同比增速是谁算的、行业规模出自哪家机构、目标价能否由模型复算。我们在构建 LEDGER 的过程中反复观察到，即便是当前最强的模型，在不加约束地写研报时，也会以稳定的频率犯下几类错误，而且这些错误恰好集中在读者最难自行核对的地方。本文报告我们为此设计的系统：一个 ** 理解与计算分离 ** 的多智能体投研系统，面向 2026 北京市大学生金融人工智能竞赛，主选题为**财报分析、自动化估值建模与研究报告质量评估**，并涉及**数据提取、产业链分析与研究报告纠错核查**三个子选题。LEDGER 是 **L**LM-driven **E**quity Research with **D**eterministic **G**rounding, **E**valuation and **R**eporting 的缩写：模型驱动研究，确定性程序负责数字的落地（grounding），并配套评估与报告生成；代码仓库与 Python 包名为 `finagent`。系统产出一份卖方体例的深度研报 PDF 与一本公式联动的估值工作簿。

本章先描述投研场景与研报的生产过程（§1.1），再把“让 LLM 直接写研报”的失败模式归为六类并逐一举例（§1.2），由此引出本文的基本立场（§1.3）、系统概览（§1.4）、贡献（§1.6）与结果概览（§1.7），最后给出全文导读（§1.9）。

1.1 投研场景与卖方研报的生产过程

一份深度研报包含什么。 卖方深度研报（initiating coverage / deep-dive report）面向买方基金经理与研究员。按国内券商的通行体例，一篇“年报点评暨深度研究”通常包含：首页（评级、目标价或估值区间、三四条投资要点、基本数据与盈利预测表）、业绩回顾（收入与利润的同比、单季走势、利润变动归因）、业务拆分（分产品、分渠道、分地区的量价结构）、盈利能力与现金流、行业与竞争格局、若干深度专题、盈利预测与关键假设、估值（绝对估值与相对估值的交叉验证、情景与敏感性）、风险提示，以及财务报表预测附录与免责声明。在这些部分中，数字的来源有本质区别：

- **披露事实**：营业收入、归母净利润、资产负债表科目等，直接取自定期报告；
- **计算推论**：同比、环比、毛利率、单季拆分、净现比、利润归因各项等，由披露事实按确定公式算出；
- **外部事实**：行业规模、市场份额、同业倍数、一致预期、股价等，来自年报之外的机构或行情；
- **分析观点**：收入增速与利润率假设、WACC 参数、情景设定，以及由它们推出的预测值、每股价值与评级。

这四类数字的可信度、核对方式与出错后果完全不同。监管与行业自律对“区分事实、推论与观点”的要求，在竞赛规则中也被明确写成硬约束：估值和投资判断必须写明假设、适用范围与风险，并区分事实、推论与观点。

研究所里的生产过程。 在一家典型的研究所里，一篇深度报告的生产大致经过以下环节：研究助理收集最新定期报告、行情、卖方一致预期与行业数据，整理成资料包；分析师读年报、搭财务模型（历史报表、分部预测、三表联动、DCF 与可比公司）、形成投资逻辑；分析师执笔写正文；质控（合规）编辑通读全文，删改不当措辞、核对前后数字、确认风险提示与免责声明；排版人员出版面；最后由研究所负责人或审核委员会签发。这里有两个值得注意的结构性特点：其一，** 建模与写作是分开的 **——数字在 Excel 模型里由公式算出，文字只是引用模型；其二，** 写作者与复核者是不同的人 **，复核者没有参与写作，不继承写作者的思路，因而能看出写作者看不出的问题。LEDGER 的多角色设计在很大程度上就是把这两个特点移植到 LLM 系统中（§1.4）。

为什么这是一个好的 LLM 系统问题。 从机器学习系统的角度看，研报生产同时具备几个难点：输入是长且版式复杂的半结构化文档（表格、跨页、多单位）；输出是数万字的长文，内部有大量必须彼此一致的数字；任务链很长（抽取 → 分析 → 预测 → 估值 → 写作 → 复核 → 排版），任一环节的错误都会沿链条传播且越来越难定位；质量评价既有可以程序判定的部分（数字是否闭合），也有只能由专家判断的部分（预期差是否成立、风险是否落到假设）。这使它成为检验“LLM 作为核心推理引擎、但不独自承担全部责任”这一系统设计思想的合适场景。

1.2 让 LLM 直接读年报写研报：六类失败模式

我们最早的原型是“一个模型会话读年报 PDF、搜网页、直接写研报”。下面把在原型与后续迭代中观察到的错误归为六类。每类给出一个具体示例：凡是能在仓库技能文档、校准报告或人工质检记录中找到原文的，按原样引用；为说明机制而构造的示例细节用橙色标出（如此）。

1.2.1 F1：数字幻觉与口径混用

年报中合并报表与母公司报表的科目名几乎完全相同（“营业收入”“净利润”“货币资金”），二者常常相隔几十页。模型在长上下文中定位数字时，可能取到母公司报表的同名科目，可能把“营业总收入”与“营业收入”混为一谈（含金融子公司利息收入的公司二者不同），可能把“净利润”当作“归属于母公司股东的净利润”，也可能在找不到某个数时用“看起来合理”的值补齐。

失败案例：取到母公司报表

某制造业公司 2025 年报第 98 页为合并利润表、第 112 页为母公司利润表。模型在写“营业收入同比增长”时引用了第 112 页的“营业收入 186.4 亿元”，而合并口径应为 342.7 亿元；随后它基于这个数算出的“毛利率”和“销售费用率”全部错位，文稿读起来完全通顺。这类错误的危险在于：数字本身确实出现在年报里，人工抽查“这个数在不在年报中”会放过它。

LEDGER 的对策是证据约束（§6）：模型抄录的每个数字必须写明页码，程序在该页原文中按数字边界逐字查找，同时拒收非合并口径的条目；规则通道从合并报表独立取数交叉比对，二者不一致即阻断下游估值。

1.2.2 F2：心算与“约数”错误

LLM 的逐位算术不可靠，这已是共识 [8, 20]。研报里的计算远不止加减乘除：同比增速在基期为负时要用绝对值作分母，单季数据要由累计数相减得到，比率类指标的变动要用“个百分点”而不是百分比，DCF 要年中折现并处理终值。更隐蔽的是“约数”——“约三倍”“超过四成”“接近翻番”这类措辞看似无须精确，实际上是模型心算后的近

似，同样会出错。

失败案例：环比心算取整

仓库的写作技能文档记录了一次真实拦截：某季度营业收入环比为 −23.3445%，模型在正文中写成“环比下降 23.35%”（正确的两位小数应为 23.34%）。这个差异肉眼几乎无法察觉，由数字护栏按展示精度核对后拦下（§10）。

LEDGER 的对策是“LLM 不做算术”：所有派生数（增速、占比、差值、单价、单季、估值）都由确定性 Python 计算并写入运行产物；文稿中的每个数字必须能在产物中按展示精度找到出处，连“约三倍”也要求写成程序算出的倍数。

1.2.3 F3：外部信息无出处

年报只覆盖公司自身。研报需要的行业规模、竞争格局、同业倍数、一致预期、年报之后的事件都在年报之外。模型对这些信息有大量“记忆”，但记忆有时效、有错误、无法追溯，而且模型倾向于用记忆补全检索不到的空白。

失败案例：凭记忆写行业数据

在一篇光模块公司的原型研报中，模型写道“2025 年全球 800G 光模块出货量约 2,000 万只，公司份额超过 40%”，没有任何出处；事后检索发现，公开的第三方机构估计口径各不相同，区间跨度超过一倍。 校准实验中我们把类似的一段无来源市场规模与份额数字注入一篇已达标的研报，盲评质检员判出“无来源的外部数据”，总分由 81 降至 40（封顶规则生效，见 §12）。

LEDGER 的对策是“先搜集、后分析”与来源登记：年报之外的事实只能来自研究助理检索后登记的 `sources.json`，每条来源有网址、发布与访问日期和不超过 160 字的原文摘句，所登记的数字必须原样出现在摘句中；正文中只能在外部来源里找到出处的数字，所在句子必须带对应的 `[S 编号]`（§10）。

1.2.4 F4：数字与指标、期间的错配，以及聚合数的构成错误

即使每个数字都“有出处”，它也可能被配给了错误的指标、期间或对象。这一类比 F1 更难发现，因为单看数字是对的。

失败案例：股权价值桥的构成写错

这是仓库技能文档记为“最严重的一次”的真实案例：贵州茅台研报正文把股权价值桥的加回项 1,789.15 亿元写成“拆出资金、货币资金等”，把扣减项 95.55 亿元写成“财务公司吸收的存款”。实际上加回 = 货币资金 + 非经营性资产净额（已扣吸收存款），扣减 = 少数股东权益 + 租赁等债务。两个数字都通过了出处核对，归属全错，读者对不上附表。

这类错误推动我们做了三件事：数字护栏增加“指标 + 期间绑定”核对，把能识别的表述交给研报纠错引擎按真值表判定；写作技能要求“写任何汇总数的构成之前，先在产物里把它拆开”；评分标准增加负向条目“聚合数构成描述错误”，由盲评质检员判定（§12）。它也提醒我们，程序护栏证明的是“存在性”而不是“语义正确性”——这一差距在第 3 节的威胁模型中被明确写出。

1.2.5 F5：改写引入新错

让模型“润色”或“复核”自己或他人的文稿，是常见的质量提升手段 [39]。但改写是一个高风险操作：模型在改措辞时会顺手“修正”数字、合并两个数、把 58.62 写成 58.6、把一个外部来源编号换成另一个。初稿上做过的全部核对，在改写后都失效。

失败案例：润色时改了数字

在引入数字锁之前的一次复核中，复核会话把“2026E 归母净利润 133.73 亿元，为预测期峰值”改写为“2026 年归母净利润约 134 亿元，达到峰值”，并把相邻一句中的“同比增长 18.17%”合并成“增长近两成”。三个数字中两个不再能在产物中找到原样出处，一个变成了心算约数。

LEDGER 的对策是数字锁（numlock）：任何让模型改写报告的环节，改写稿中的每个“数字 + 单位”和每个来源编号都必须原样出现在改写前的稿件中，只允许减少、不允许新增或改动；改写后再重跑数字护栏（§10）。

1.2.6 F6：质量无法客观评价，评价过程本身被污染

前五类是“对不对”的问题，第六类是“好不好”以及“怎么知道好不好”的问题。研报的分析质量（是否有预期差、驱动拆解是否成环、风险是否落到假设）没有标准答案，只能由专家或模型判断；而一旦用模型来判断，又会出现新的问题：

- **上下文污染**：若复核者继承写作者的对话，会沿用写作者的思路，看不出写作者的盲点；若评审看到写作过程，会因“知道作者有多努力”而变得宽容。
- **评分标准泄露**：若生成方能读到评分标准，它会针对条目字面“应试”，而不是提升质量（Goodhart 效应）。在早期版本中，工作区就在仓库目录树内，模型可以读到仓库里的评分标准与其他公司的文稿。
- **评审本身不可靠**：LLM 评审存在位置偏差、冗长偏好与自我偏好 [43, 73]，同一份文稿重复评审可能给出不同结论；不知道评审有多可靠，就无法用它驱动改进。

失败案例：评审过严与评审不稳定

人工质检记录（`eval/calibration/report.md`）中，三家公司早期研报的 33 条“评审判不通过”有 5 条被人工判为误报，例如“报告出现几次‘不是……而是……’以及程序说明，就算机器味太重”被认为过严。对同一份研报重复三次盲评，总分分别为 85、80、78，极差 7 分；逐条全一致的比例为 91%。

LEDGER 的对策是：每个角色是一次独立的无头会话，只靠文件交接；工作区建在仓库之外，不含评分标准与其他公司的产物；盲评在只含交付物的第三个目录中进行；回传给生成方的问题单只含理由与证据，不含条目原文与分值；评审模型本身用重复评审稳定性、程序劣化判出率和人工精确率三组证据校准（§11、§12）。

要点

六类失败有一个共同结构：模型的输出在 ** 局部看起来完全合理 **，错误只在与外部真值（年报原文、计算结果、来源原文、初稿）对照时才暴露。因此单靠更好的提示词或更强的模型无法根除——需要一个在模型之外、能够执行这种对照的确定性组件，并且让模型的每个输出都必须经过它。

1.3 本文立场：模型读、抄、写、判断；程序校验、计算、拦截

由上述观察，我们采取一个简单但贯彻到底的分工：

模型读、抄、写、判断；程序校验、计算、拦截。

模型做什么。 **读**：按页读年报 PDF、季报、网页与资料包，理解管理层讨论、附注与行业信息；**抄**：把数字按原文写法连同页码抄进结构化文件（`facts.json`、`extras.json`、`claims.json`、`sources.json`）；**写**：推导分析框架、写深度专题、写研报正文、写交付说明；

表 1 六类失败模式、其后果与 LEDGER 中的对应机制。

失败模式	典型表现	主要机制	章节
F1 数字幻觉	取母公司报表、混用营业总收入与营业收入、编造补齐	逐字证据校验、合并口径约束、双通道交叉比对、冲突阻断	§6
F2 心算错误	增速、单季、约数、DCF 心算	确定性引擎计算全部派生数；单一真值表；按展示精度的出处核对	§7、§8
F3 外部无出处	行业数据、份额、一致预期凭记忆	先搜集后分析；来源登记与摘句含数；`[S 编号]` 强制引用	§10、§11
F4 错配	数字配错指标、期间、对象；聚合构成写错	指标—期间绑定核对；内部一致性检查；负向评分条目	§9、§10
F5 改写引错	润色时改数、合并、取整	数字锁：改写稿数字与来源编号是初稿的子集	§10
F6 评价失效	上下文污染、评分标准泄露、评审不稳	会话隔离、仓库外工作区、盲评目录、问题单脱敏、评审校准	§11、§12

判断：起草估值假设（增速、利润率、WACC 参数、情景）、判断故事线与论证链、复核措辞与结构、按评分标准盲评分析质量。这些都是模型擅长而程序无法替代的工作。

程序做什么。 **校验**：模型抄的每个数字在所写页码原文中逐字查找；外部摘句必须包含所登记的数字；规则通道独立取数比对；勾稽关系逐项核对。**计算**：同比、环比、单季拆分、派生比率、利润归因、三表联动、三情景 FCFF DCF、敏感性、反向 DCF、可比公司、评级。**拦截**：数字护栏拒绝无出处、错绑定、缺引用、前后矛盾的文稿；数字锁拒绝引入新数字的改写；11 道闸门拒绝不完整或过期的阶段产物；编排器在闸门未过时不启动下一个角色。

这个立场不是什么。 它不是“少用模型”：LEDGER 中模型承担了全部的阅读、写作与判断，五个生产角色加一个盲评角色都是完整的 Claude Code 会话，可以联网检索、读写文件、调用引擎命令。它也不是“程序替模型写”：程序不生成任何一句分析性文字，也不预设任何行业的分析框架（分析框架由模型从年报与资料包推导）。它更接近于把模型放进一个 ** 会计意义上的内部控制体系 **：模型是有能力、但其输出不被默认信任的经办人；程序是不懂业务、但永不疲倦的复核与记账员。

权衡（Trade-off）：为什么不让模型调用计算器工具了事

程序辅助推理（PAL、PoT）[8, 20] 与工具使用 [50, 70] 让模型在需要计算时调用解释器，这解决了 F2 的一部分。但它仍由模型决定“算什么”“把结果抄到哪里”，结果的取用与转写环节仍可能出错，而且无法约束模型在不调用工具时直接写出数字。LEDGER 反过来：计算不是模型可选调用的工具，而是一条固定的确定性流水线，文稿只能引用其产物；模型可以自由写作，但写下的任何数字都必须在产物中找到出处，否则文稿不被接受。代价是灵活性下降——模型想要一个产物里没有的派生数，必须先把它登记为 `extras.json` 中的比率交给程序算，而不能当场算出。我们认为在研报这种“每个数都可能被追问”的场景中，这个代价是值得的。

1.4 系统概览

图 1 给出 LEDGER 的整体结构。系统由四层组成，自下而上为：

1. **计算引擎**（`finagent/`，不含 `evaluation/`）：确定性 Python，只用标准库与 pdfplumber，

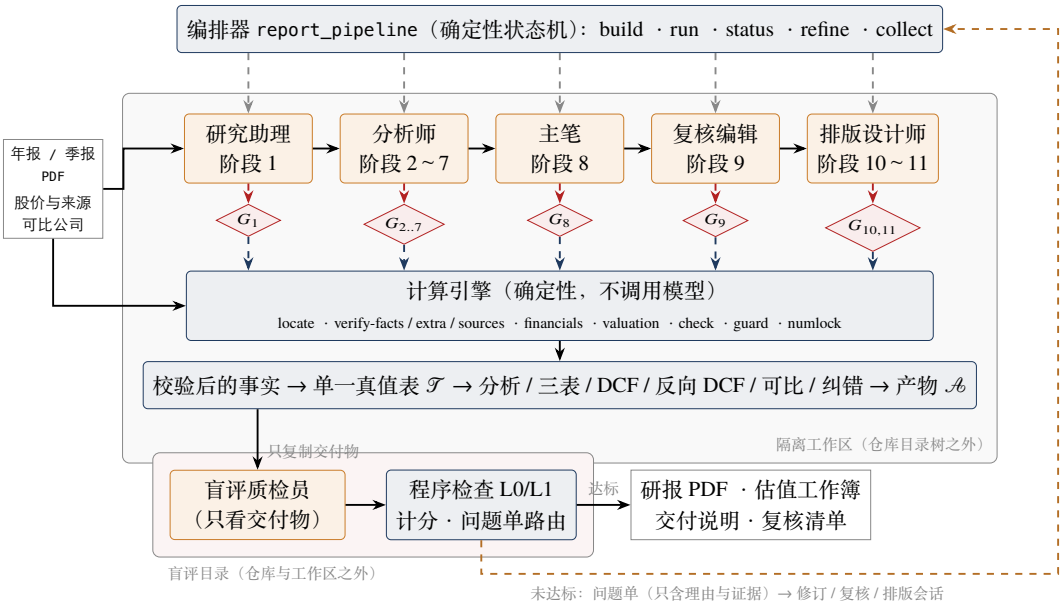


图 1　LEDGER 总览。橙色为模型会话（无头 Claude Code，每个角色一次独立会话），蓝色为确定性程序，红色菱形为程序判定的闸门。会话之间不共享上下文，只通过工作区文件交接；每个会话写下的数字、外部来源与文稿都必须经计算引擎校验（虚线）。编排器按闸门决定是否启动下一个会话；全部闸门通过后，盲评质检员在只含交付物的目录里评审，程序计分并把脱敏的问题单路由回相应角色。

不调用任何模型。提供 `locate`（定位关键页）、`verify-facts` / `verify-extra` / `verify-sources`（三类校验）、`financials` / `valuation` / `check`（财报分析、估值建模、纠错核查的计算流水线）、`guard`（数字护栏）与 `numlock`（数字锁）。全部文件读取与产物写入经追踪器记录 SHA-256，每次运行生成独立目录与事件流。

2. **角色说明**：四个技能（skill）文档与编排器为每个工作区生成的 `CLAUDE.md`，写明 11 个阶段各自读什么、不读什么、运行什么命令、闸门是什么。技能只给方法与约束，不给行业框架。
3. **多智能体编排**（`finagent/report_pipeline.py`）：确定性 Python 状态机。`build` 在仓库目录树之外为单家公司搭隔离工作区；`run` 依次起五个无头 Claude Code 会话——研究助理（阶段 1）、分析师（阶段 2～7）、主笔（阶段 8）、复核编辑（阶段 9）、排版设计师（阶段 10～11）；`status` 逐项判定 11 道闸门；`collect` 把产物收回仓库。
4. **质检闭环**（`refine` 与 `finagent/evaluation/`）：闸门全过后，在仓库与工作区之外只含交付物的目录里起盲评质检员会话，程序检查（L0/L1）与盲评（L2/L3/负向）合并计分；未达交付标准时把脱敏的问题单按类型路由回修订会话、复核编辑或排版设计师，最多三轮，回滚到得分最高的一轮。

这一结构中有三条贯穿全文的“数据流”：

- **数字流**：年报原文 → 模型抄录 `facts.json` → 逐字校验与双通道合并 → 真值表 $\mathcal{T}$ → 分析、三表、DCF、纠错产物 $\mathcal{A}$ → 文稿 $\mathcal{N}$ 中的数字（经护栏核对）→ PDF 中的图表（由脚本从产物读出）。每一步都可以反向追溯到上一步，最终到原文的 (文件, 页, chunk, 行, 列)。
- **证据流**：网页 → 研究助理登记的来源 $\mathcal{S}$（摘句含数）→ 资料包与专题笔记 → 正文中带 `[S 编号]` 的句子。
- **控制流**：编排器 → 会话 → 闸门 → 下一会话；质检 → 问题单 → 修订会话 → 重新过

闸门与盲评。

数字流与证据流保证“每个数可追溯”，控制流保证“没有经过校验的东西不会流到下游”。

权衡（Trade-off）：单会话长上下文 vs 多会话文件交接

最直接的做法是一个会话从头做到尾：上下文里有年报、资料、模型输出与全部草稿，模型可以随时回看。我们早期也这样做，并观察到三个问题：(1) 上下文越来越长，后期阶段对前期材料的利用变差（与长上下文中“中段遗忘”的现象一致）；(2) 复核阶段继承了写作阶段的全部思路，几乎从不推翻自己的结论；(3) 单个会话失败（超时、权限、费用上限）时整条链要从头再来。多会话的代价是交接成本：每个角色都要先读前一角色留下的文件，交接文件的格式需要设计和校验，跨角色的“隐性知识”会丢失。我们用两点缓解：交接文件本身就是结构化的、带闸门校验的产物（不是聊天摘要）；工作区 `CLAUDE.md` 为每个阶段明确列出必须先读的文件。

1.5 一个贯穿全文的例子：一个数字的一生

为了让后文的机制有一个具体的落点，我们跟踪仓库样例中虚构公司“星河智造”2024 年年报里的一个数字——归属于上市公司股东的净利润（下称归母净利润）——从年报到研报的全过程。样例年报、样例研报草稿与金标准都在仓库 `data/samples/` 与 `eval/` 下，下面的数字全部取自其中。

第 1 步：原文。 样例年报第 6 页“主要会计数据和财务指标”表中，归母净利润一行写作 `612,380,000.00`（本期）、`518,210,000.00`（上期），“本年比上年增减”列写作 `18.17%`；表头注明单位为元。第 7 页“分季度主要财务指标”表中，第四季度归母净利润写作 `211,720,000.00`。

第 2 步：抄录与校验。 分析师会话把这一行抄进 `facts.json`：指标 `np_parent`、期间 `2024`、原文写法 `612,380,000.00`、页码 6、单位元、口径合并。`verify-facts` 在第 6 页的 chunk 中去掉千分位后按数字边界查找 `612380000.00`，定位到具体表格 chunk；规则通道从同一张表（以及第 106 页起的合并利润表）独立取到同一个值，两通道一致，该事实的抽取方式记为 `rule+llm`。若模型误写为 `612,830,000.00`，查找失败，条目被拒收并附理由“未在第 6 页原文中找到”；若模型从母公司利润表抄了另一个数并标为合并口径，规则通道的值与之不一致，记为冲突并阻断估值。

第 3 步：进入真值表并派生。 `analyze()` 把它登记为真值表单元格 $(\texttt{np_parent}, 2024)$，并据此派生：同比增速 $(612{,}380{,}000 - 518{,}210{,}000)/|518{,}210{,}000| = 18.17\%$（与年报列示的同比在 $\frac{1}{2}$ ulp 容差内一致，构成一项勾稽）；非经常性损益 = 归母净利润 − 扣非归母净利润；若提供了三季报，则第四季度单季 = 全年 − 前三季度累计，并与第 7 页列示的单季值互相核对。每个派生单元格都记录公式与输入单元格的来源。

第 4 步：被估值与纠错使用。 估值模块只从真值表读取它：盈利预测摘要中的“2024A 归母净利润”、按少数股东损益占比把三表净利润预测换算为归母口径的比例，都来自这一单元格。研报纠错模块在样例研报草稿中读到“实现归母净利润 6.46 亿元，同比增长 25.30%”，拆为两条断言：前者与真值 6.12 亿元不符，但与同组指标“净利润”的真值 6.46 亿元在展示精度内一致，判为**口径错误**；后者与真值 18.17% 不符、也找不到任何精确匹配的替代解释，判为**数值错误**。两条结论都附带真值、来源页码与修改建议。

第 5 步：进入文稿并被护栏核对。 主笔在研报正文中写“2024 年实现归母净利润 6.12 亿元，同比增长 18.17%”。`guard` 把“6.12 亿元”换算为 6.12×10^8 元，在结论性产物中找到 612,380,000（按两位小数的半个末位容差），出处归为“报告披露”即事实；把“18.17%”在分析产物中找到，出处归为“程序计算”即推论；再把这句话交给纠错引擎按指标与期间绑定核对，确认指标与期间都对。若主笔写成“归母净利润约 6.1 亿元，同比增长近两

成”，前者可以通过（一位小数的容差更宽），后者没有数字可核对、但“近两成”是心算约数，按写作规范应改为程序算出的值——这类表述由复核编辑处理，而不是由程序拦截。

第 6 步：改写、排版与评审。 复核编辑改写这句话时，数字锁要求“6.12 亿元”“18.17%”原样保留或被删除，不得变成“6.1 亿元”；排版脚本从 `analysis.json` 读出同一个值画进“归母净利润与增速”图；盲评质检员看到的是 PDF 与文稿，它不知道这个数经过了哪些校验，只按评分标准判断“历史业绩归因是否落到科目并量化”。

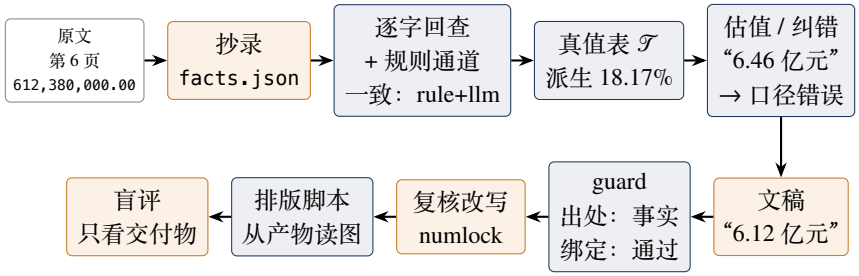


图 2　样例公司归母净利润从原文到研报的路径。橙色步骤由模型完成，蓝色步骤由程序完成；每一步的产物都可以反向追溯到上一步。

这个例子展示了本文想要的性质：数字在每个环节都有唯一的来源，模型参与的每个环节之后都紧跟一个程序环节，而程序环节从不需要“理解”这个数字的经济含义——它只检查字符串、页码、单位、精度与等式。理解是模型的事，记账是程序的事。

1.6　贡献

本文的主要贡献如下。

C1 **理解与计算分离的投研系统架构。**我们把“模型读、抄、写、判断；程序校验、计算、拦截”作为系统不变量贯彻到所有环节：引擎不调用任何模型，模型产出的每个数字都必须经过程序校验后才进入下游，文稿中的每个数字都必须在本次运行的结论性产物或外部来源登记中按展示精度找到出处。我们把这一分工形式化为一组不变量（第 3 节），并说明每个不变量由哪个机制强制执行（第 4 节）。

C2 **证据约束的双通道抽取。**模型通道按页读 PDF 抄录数字，程序在所写页码原文中按数字边界逐字回查并定位到 chunk；规则通道从合并报表表格独立取数，处理稀疏网格、跨页续表与科目折行。两通道不一致时不静默取舍，而是记为问题并阻断估值。在贵州茅台、宁德时代、招商轮船三份真实年报上，模型抄录并通过证据校验的 301 个数值中，规则通道独立取到且一致的有 280 个（93.0%），冲突为 0，勾稽 56/56 全部通过（第 6 节）。

C3 **单一真值表与可审计的派生计算。**抽取结果经 `analyze()` 闭包为唯一的真值表 $\mathcal{T}$：披露值与派生值（单季拆分、比率、非经常性损益）同表登记，派生值记录公式与输入单元格；估值、纠错、护栏都只读这张表，下游不重新抽取或重算基础数字。单季拆分时记录每个单元格依赖的“披露单元格集合”，避免勾稽核对退化为“自己和自己比”（第 7 节）。

C4 **三表驱动的 DCF 与反向 DCF。**分部量价预测驱动三表联动模型，DCF 的收入增速、EBIT 利润率、折旧摊销、资本开支、营运资本与税率逐期取自三表模型同一情景，保证 DCF 与三表同口径；股权价值桥逐项列示，非经营性资产逐项回查原文；评级由程序按上行空间阈值给出。反向 DCF 以网格扫描加二分法单变量求解

“参考股价隐含的增速、利润率、折现率与终值参数”，并处理非单调情形。自研引擎与参照引擎的产出逐位一致，由 golden 快照测试守护（第 8 节）。

C5 按展示精度的纠错容差与误报优先的错误诊断。研报纠错把容差定义为研报数字展示精度的半个末位（$\frac{1}{2}$ ulp），避免把四舍五入判为错误；预测性表述与缺少真值的断言判“无法核实”而不计为错误；单位、期间、口径、方向错误只有在找到精确匹配的替代解释时才下结论，且口径诊断只在“容易混用”的指标组内进行。样例集上 7 处埋错全部判出、错误类型全部正确、误报为 0（第 9 节）。

C6 数字护栏 guard 与数字锁 numlock。 guard 对文稿做四项检查：数字出处（按单位与展示精度，只在结论性产物中找，研报原文与中间网格不算出处）、指标—期间绑定、外部数字的 `[S 编号]` 引用、文稿内部前后矛盾，并把每个数字的出处归为事实、推论或观点。numlock 要求任何改写稿中的数字与来源编号是初稿的子集。二者把“模型写的文字”纳入与“模型抄的数字”同等强度的约束（第 10 节）。

C7 隔离工作区中的五角色无头 Claude Code 流水线与 11 道闸门。编排器是确定性状态机而不是模型；每个角色一次独立的 `claude -p` 会话，不共享上下文，只靠工作区文件交接；工作区建在仓库之外，不含评分标准与其他公司产物，权限配置禁止读取仓库、禁止修改引擎与技能；11 道闸门按“前序未过则后续一律不过”和“输入晚于产物则判过期”两条规则判定（第 11 节）。

C8 评分标准对生成方不可见的盲评闭环与评审校准。四层评分标准（L0 硬闸门、L1 数据闭合由程序判；L2 分析质量、L3 表达与合规由盲评判；另有负向条目）由程序计分；盲评在只含交付物的目录中进行，问题单只含理由与证据并按类型路由；我们用同稿重复盲评的稳定性、七种程序劣化的判出率与连带改判、人工复核的精确率三组证据校准评审模型本身（第 12 节）。

其中 C1、C6、C7、C8 是系统层面的设计贡献，C2 ~ C5 是面向 A 股财报与估值场景的方法贡献。我们不声称提出了新的模型或训练方法；本文的全部能力都建立在现成的 Claude 模型与 Claude Code 之上，贡献在于如何把一个强大但不可信的组件组织进一个可追溯、可复现、可评价的系统。

1.7 结果概览

表 2 汇总了截至本文写作时的主要结果，详细实验见第 13 节。需要强调几点：样例数据是我们构造的虚构公司，内部完全自洽，样例上的满分只说明流程正确；研报质检分数来自单次盲评，重复盲评的总分极差为 7 ~ 8 分，版本之间不是严格的对照实验；评审校准的劣化判出率偏低，主要原因是“整节删除”类劣化在深度研报中删不干净（同一论点在摘要、专题与图表中重复出现），我们在第 12 节逐条分析。

从这些结果可以读出两条与设计相关的信号。第一，真实年报上规则通道与模型通道“错配为 0”并不意味着模型从不读错——它意味着读错的那些条目在校验阶段就被拒收或标为冲突，由分析师会话回到原文修正后才通过闸门，最终进入真值表的值两通道一致。这正是双通道设计的意图：不是让某一通道永远正确，而是让两通道的分歧无法被静默掩盖。第二，研报质量的提升主要来自流程结构的变化（先搜集、独立复核、深度调研），而不是提示词措辞的调整：v0 → v1 → v2 的三次跃升分别对应三次流程改动，L2 分析质量的得分从 15 ~ 21（满分 45）上升到 40 ~ 42。这一观察支持本文的核心主张：对 LLM 投研系统而言，“怎样组织模型”与“用哪个模型”同样重要。

1.8 范围与非目标

为避免误读，我们明确本文不做什么。

表 2　结果概览。除特别标注外，数字均可在仓库测试、`eval/` 与 `runs/` 产物中复现。

维度	结果	说明
工程正确性	173 个测试中 167 个通过、6 个跳过（浏览器打印）；自研三表、DCF、情景表、业绩分析引擎与参照引擎产出逐位一致（golden 快照）	本地有参照插件时逐项对照测试一并通过
数据提取（样例）	35 / 35 个金标准字段正确	虚构样例，规则通道
数据提取（真实年报）	茅台、宁德、招商 301 个核验值，规则通道取到且一致 280 个（93.0%），冲突 0	用于交叉校验模型抽取
纠错核查（样例）	7 处埋错全部判出，错误类型全部正确，误报 0	含口径、单位、期间与数值错误
研报质量（评分标准 v0）	旧流程三家 46 ~ 49 分；加入先搜集与独立复核后 74 / 67；再加深度专题调研与篇幅闸门后 90 / 88，达到交付标准	中际旭创 / 宇树科技；单次盲评
评审稳定性	同稿三次盲评逐条全一致 86% ~ 91%，总分极差 7 ~ 8	中际旭创、宇树科技
评审精确率	评审判“不通过”的 31 条（不含待查）中人工确认 26 条，精确率 84%	三家早期研报人工质检
劣化判出率	有效劣化变体判出 33%（中际旭创）、50%（宇树科技）	整节删除类劣化删不彻底，见 §12
运行成本	一份研报五个会话约 30 ~ 60 分钟；一次盲评约 3 分钟、约 1 美元等值用量	生成侧单份研报约 $25 ~ 40 等值用量，见 §13

- **选题范围**。主选题为财报分析、自动化估值建模与研究报告质量评估；子选题为数据提取（第 6 节）、产业链分析（资料搜集与深度专题，第 11 节）与研究报告纠错核查（第 9 节）。六者共享同一数据底座（真值表），并以“深度研报”作为集成出口；其他选题不在本作品范围内。
- **不做投资建议**。系统产出的评级由程序按预设规则从估值结果机械给出，估值依赖所列假设；本文与系统产出的全部估值结论均不构成投资建议。
- **不做 OCR**。扫描页没有文本层时直接拒收，不做光学识别；真实年报中这类页面极少，但招股书等文件可能出现。
- **不训练或微调模型**。全部模型能力来自现成的 Claude 模型；我们的工作在系统与流程层面。
- **不在程序内调用模型 API**。历史上曾有在引擎内调用模型 API 的版本与另一种执行器，已全部删除；模型只以无头 Claude Code 会话的形式出现，计算引擎保持纯确定性，便于评审离线复现。
- **不用程序拦截文风**。“机器腔”等文风问题交给独立的复核编辑会话，不用正则规则拦截——程序拦截文风既容易误杀，也会诱导模型用同义改写规避规则。

1.9　全文导读

表 3 给出各章内容与阅读建议。只关心系统设计思想的读者可以读第 3、4、5 节后跳到第 10、11、12 节；关心金融方法的读者可以重点读第 6 ~ 9 节；希望复现结果的读者可直接看附录 G。

关于记号：第 3 节定义的记号在全文统一使用——年报 $\mathscr{D}$、页集合 $\mathscr{P}$、事实 f =

表 3 全文导读。

章节	标题	内容
§2	背景与相关工作	A 股年报结构、卖方研报体例与估值方法；财报问答、程序辅助推理、金融智能体、多智能体框架、自我改进、可追溯生成、LLM 评审与编码智能体 harness
§3	问题形式化	统一记号；三项任务的输入、输出与正确性标准；可追溯性、数字闭合、可复现性；威胁模型与系统不变量；研报生成与评价的形式化
§4	设计原则	十一条设计规范的动机、不变量与执行机制
§5	系统架构	四层结构、模块划分、追踪与复现
§6	证据约束的双通道抽取	文档解析、关键页定位、规则通道、模型通道、逐字证据校验、合并与阻断
§7	财报分析	真值表闭包、单季拆分、勾稽、异常信号、利润归因、一致预期与业绩预告、EPS 质量
§8	估值建模	分部预测、三表联动、三表驱动 DCF、股权价值桥、敏感性、反向 DCF、可比公司、评级、工作簿
§9	研报纠错	断言抽取、按展示精度的容差、错误类型诊断、内部一致性
§10	数字护栏与数字锁	guard 的四项检查与出处分类、外部来源登记、numlock
§11	多智能体研报流水线	隔离工作区、五个角色、11 道闸门、交接文件、refine 闭环
§12	评测体系	四层评分标准、盲评、计分、问题单路由、评审校准
§13	实验	抽取、纠错、估值一致性、研报质量、评审校准、成本
§14 ~ 16	讨论、局限、结论	设计取舍的回顾、已知问题与风险、后续工作
附录		复现说明（G）、评分标准全文（A）、闸门定义（D）、数据格式（C）、劣化算子（B）、角色提示（E）、第三方组件（F）

(m, k, v, u, σ)、真值表 $\mathcal{T}$、研报断言集 $\mathcal{C}$、外部来源登记 $\mathcal{S}$、文稿 $\mathcal{N}$、本次运行的结论性产物 $\mathcal{A}$、闸门 G_i。关于单位：财务事实内部一律以“元”存储；估值模块（三表、DCF、可比公司）以“百万元”计，每股以“元”计；比率存为百分数值（14.49 表示 14.49%）。文中引用的代码位置均相对于仓库根目录。

2 背景与相关工作

本章分两部分。前半部分（§2.1、§2.2）为不熟悉中国资本市场的读者补充领域背景：A 股定期报告的结构与口径、卖方研报的体例与常用估值方法。这些细节不是装饰——LEDGER 中相当一部分设计（只取合并报表、营业收入优先于营业总收入、季报表头的期间识别、单季拆分规则、扣非口径、EBIT 定义）都直接来自这些口径上的陷阱。后半部分（§2.3 ~ §2.10）按八个方向回顾相关工作，每个方向说明本文与之的差异；§2.11 用一张对比表总结定位。

2.1 领域背景：A 股定期报告

2.1.1 准则体系与年报结构

A 股上市公司按《企业会计准则》[75] 编制财务报表，按中国证监会的信息披露内容与格式准则 [76] 编制年度报告。年报的章节顺序相对固定，表 4 列出与本文相关的部分。对系统设计而言，最重要的事实是：**同一个指标会在年报中出现多次**，口径未必相同——“主要会计数据和财务指标”中的营业收入、合并利润表中的营业收入、母公司利

润表中的营业收入、附注中按产品拆分后的营业收入合计、管理层讨论与分析中的营业收入，前两者应相等，第三者通常不同，第四者可能与前者有“其他业务收入”之差，第五者可能四舍五入为亿元。

表 4 A 股年度报告中与本文相关的章节（按通行顺序）。页码为一份约 250 页年报的典型位置，仅作示意。

章节	典型页码	对本文的用途
公司简介和主要财务指标	5 ~ 10	“主要会计数据和财务指标”表（本期、上期、增减、上上期）；“分季度主要财务指标”表；非经常性损益项目及金额表
管理层讨论与分析	10 ~ 60	主营业务、分产品 / 分地区 / 分渠道收入与毛利率、产销量、成本构成、费用、研发、现金流、资产负债变动、未来展望；经营数据（`extras.json`）的主要来源
公司治理、环境与社会	60 ~ 100	一般不取数
股份变动及股东情况	100 ~ 115	总股本、股本变动
财务报告：审计报告	115 ~ 120	审计意见类型
财务报告：合并报表	120 ~ 135	合并资产负债表、合并利润表、合并现金流量表、合并所有者权益变动表；真值表的主要来源
财务报告：母公司报表	135 ~ 150	与合并报表科目同名，**必须排除**
财务报告：附注	150 ~ 250	会计政策、科目明细、现金流量表补充资料（折旧摊销）、分部信息、关联方；明细表与报表科目同名但口径不同

2.1.2 合并报表与母公司报表

合并报表把母公司与其控制的子公司视为一个经济主体编制，母公司报表只反映母公司法人主体。对控股型公司（例如经营在子公司、母公司主要持有股权的集团），母公司报表的营业收入可能远小于合并报表，净利润则可能主要由子公司分红构成的投资收益决定。两套报表的科目名称几乎完全相同，在年报中先后出现，一个只看“科目名 + 数字”的抽取器有相当概率取错。分析与估值只应使用合并报表，这是 LEDGER 抽取环节的第一条硬约束：模型抽取条目必须声明口径为合并（`scope=consolidated`），规则通道则按表格标题排除含“母公司”的表（第 6 节）。`locate` 命令会专门输出母公司报表所在页（`parent_only_pages`），提示模型避开。

2.1.3 收入与利润的口径

营业总收入与营业收入。 一般企业的利润表第一行为“一、营业总收入”，其下“其中：营业收入”；对多数公司二者相等。但若集团内有财务公司、保险或类金融业务，营业总收入还包括“利息收入”“已赚保费”“手续费及佣金收入”，此时二者不等。研报中“营收”一般指营业收入。LEDGER 的指标字典把二者都列为 `revenue` 的别名，但规则通道在同一张表同时出现二者时只取“营业收入”行（`extract.py` 中的 `has_op_rev` 判断）。

净利润、归母净利润与扣非归母净利润。 利润表中的“净利润”包含少数股东损益；“归属于母公司所有者的净利润”（年报关键指标表中写作“归属于上市公司股东的净利润”，简称归母净利润）扣除少数股东损益，是每股收益与估值的基础；“归属于上市公司股东的扣除非经常性损益的净利润”（简称扣非归母净利润或扣非净利润）再扣除非经常性损益。非经常性损益的认定依据证监会的解释性公告 [77]，包括非流动资产处置损益、计入当期损益的政府补助（与正常经营密切相关、按标准持续享有的除外）、金融资产公允价值变动与处置损益等。研报口语中“净利润”常被用来指归母净利润，这是纠错任

务中“口径错误”的主要来源（§1.5 的样例研报把含少数股东损益的净利润 6.46 亿元写成了归母净利润）。

EBIT 的口径。 A 股利润表不列示“息税前利润”，且把投资收益、公允价值变动收益、资产处置收益、其他收益、信用与资产减值、营业外收支都放在利润总额之前。估值所需的“经营性 EBIT”因此没有唯一定义。LEDGER 采用

$$\text{EBIT} = \text{利润总额} + \text{财务费用} - \text{投资收益} - \text{公允价值变动收益}, \tag{1}$$

即剔除融资成本与投资类损益，保留资产处置、其他收益、减值与营业外收支（简化）。被剔除的投资类资产需要在股权价值桥中作为非经营性资产加回，否则估值偏保守。这一口径在三表模型、DCF 与盈利预测摘要中保持一致（第 3、8 节）。

2.1.4 定期报告的期间口径

A 股公司每年披露四份定期报告：一季报（1～3 月）、半年报（1～6 月）、三季报（1～9 月与 7～9 月）、年报（全年）。其中**利润表与现金流量表只披露累计数**，单季数要靠相减得到；三季报的“主要会计数据”表通常同时有“本报告期”（第三季度单季）与“年初至报告期末”（前三季度累计）两列，且列标题不写年份；年报另有“分季度主要财务指标”表，直接列示四个单季的营业收入、归母净利润、扣非归母净利润与经营活动现金流量净额。资产负债表是时点数，不能相减；每股收益与比率也不能相减。表 5 总结了这些期间之间的关系。

表 5 定期报告的期间与单季拆分关系。“发生额”指利润表、现金流量表与非经常性损益类指标；时点数、每股收益、比率不拆分。

报告	披露期间（本文记号）	表头常见写法	可推出的单季
一季报	*Y*Q1	本报告期	Q1 即单季
半年报	*Y*H1	本报告期 / 本期	Q2 = H1 − Q1
三季报	*Y*Q3、*Y*Q1-3	本报告期 / 年初至报告期末	Q3 = Q1-3 − H1（与列示的 Q3 互核）
年报	*Y*；分季度表 Q1～Q4	本期 / 上期；第一～四季度	Q4 = 全年 − Q1-3（与列示的 Q4 互核）

这里有两个系统层面的陷阱。第一，“本报告期”在年报中指全年，在三季报中指第三季度单季，同一个词在不同文件中含义不同；LEDGER 的规则通道无法可靠区分，因此季报（通过 `--supplement` 传入）只接受模型抽取、不参与规则通道比对，单季拆分由 `analyze()` 按表 5 的规则完成并登记公式。第二，当单季值既有列示又能由累计数推出时，二者构成一项勾稽；但若某个累计数本身是由该单季推出的，再拿来核对就是“自己和自己比”。`analyze()` 为每个单元格维护其依赖的“列示单元格集合”，据此排除这种退化核对（第 7 节）。

2.1.5 业绩预告、业绩快报与一致预期

沪深交易所规则要求净利润为负、扭亏或大幅变动的公司在规定时限内发布业绩预告，通常给出归母净利润与扣非归母净利润的区间；部分公司在年报前发布业绩快报。卖方“一致预期”是各家券商对同一指标预测值的汇总（均值或中位数），由数据服务商发布。“实际 vs 一致预期”（超预期 / 低于预期，beat / miss）与“实际落在预告区间的位置”是年报点评的常规内容，对应财报分析选题中的业绩分析部分。LEDGER 不内置任何数据服务商接口：一致预期与业绩预告由人或研究助理会话以 CSV 形式提供，后者需登记来源（第 7、10 节）。

2.1.6 版式层面的困难

即便口径正确，从 PDF 中取数本身也有困难。表 6 列出我们在真实年报中遇到并处理的版式问题，详细处理见第 6 节。

表 6 真实 A 股年报 PDF 的版式问题与 LEDGER 的处理。

问题	表现	处理
单位切换	同一份年报中“元”“千元”“万元”“百万元”并存，单位写在表上方的“单位：”行	按页、按表识别单位提示；事实内部一律换算为元
括号负数与占位符	“(1,234.56)”表示负数；“—”“-”“不适用”表示无数	统一数字解析；占位符解析为缺失
跨页续表	合并报表跨两三页，续页没有表头	续页沿用首页表头的期间列，并记录 `continued_from`
稀疏网格	pdfplumber 抽出的表格有大量空列、错位列	去除空列后再按表头对齐
科目折行	长科目名（如政府补助的完整说明）被折成多行	合并折行后再匹配别名
填列说明	“投资收益（损失以“- ”号填列）”	别名匹配前去除填列说明
同名明细表	附注中受限资产、费用明细、关联方往来等表的行名与报表科目同名	规则通道只从主要会计数据、分季度、非经常性损益、三大报表与补充资料取数
追溯调整	表格同时列示“调整前 / 调整后”	记为“追溯调整”问题，提示同比应使用调整后数据
扫描页	无文本层	直接拒收，不做 OCR

2.2 领域背景：卖方研报体例与估值方法

2.2.1 研报体例与评级

国内券商的深度报告在结构上高度趋同（§1.1），首页的“投资评级”通常采用五档：买入、增持、中性、减持、卖出，定义为未来 6 ~ 12 个月相对基准的预期超额收益区间，各家阈值略有差异。LEDGER 的评级是一个确定性函数：以基准情景 DCF 每股价值相对参考股价的上行空间 u 为输入，$u \geq 20\%$ 买入、$u \geq 5\%$ 增持、$u \geq -5\%$ 中性、$u \geq -20\%$ 减持、其余卖出（阈值可在假设文件中覆盖）。研报只引用程序给出的评级，不自行判断——这既避免了评级与估值的矛盾，也把“观点”的来源明确限定在估值假设上。

2.2.2 现金流折现：FCFF DCF

企业自由现金流（free cash flow to the firm，FCFF）折现是卖方深度报告最常用的绝对估值方法 [12, 32]。其核心关系为

$$\mathrm{FCFF}_t = \mathrm{EBIT}_t(1-\tau_t) + \mathrm{D\&A}_t - \mathrm{CapEx}_t - \Delta\mathrm{NWC}_t, \tag{2}$$

$$\mathrm{EV} = \sum_{t=1}^{N} \frac{\mathrm{FCFF}_t}{(1+\mathrm{WACC})^{t-0.5}} + \frac{\mathrm{TV}_N}{(1+\mathrm{WACC})^{N}}, \qquad \mathrm{TV}_N = \frac{\mathrm{FCFF}_N(1+g)}{\mathrm{WACC}-g}, \tag{3}$$

其中 τ_t 为税率，D&A 为折旧摊销，NWC 为营运资本，g 为永续增长率，预测期现金流按年中折现（折现期 $t-0.5$）、终值按第 N 年末折现；终值也可用退出倍数法 $\mathrm{TV}_N = \mathrm{EBITDA}_N \times$ 倍数。WACC 由资本资产定价模型给出的股权成本与税后债务成本

按目标资本结构加权。企业价值经股权价值桥（加现金、非经营性资产、联营投资，减有息负债、租赁负债、少数股东权益等）得到股权价值，再除以摊薄股本得到每股价值。DCF 的结论对少数几个参数极其敏感（终值往往占企业价值的一半以上），因此卖方报告通常给出 WACC × 永续增长率的敏感性矩阵与多情景结果。

2.2.3 三表联动

DCF 中的 EBIT、折旧摊销、资本开支与营运资本变动本应来自一个完整的财务预测模型：利润表、资产负债表、现金流量表三表联动，资产负债表每期平衡、现金流量表的期末现金与资产负债表现金一致、留存收益按净利润与分红滚动、利息费用与债务余额相互依赖（通常需要迭代求解）。很多简化的 DCF 实现直接假设"EBIT 利润率""资本开支占收入比"等向量，跳过三表模型；这使 DCF 与盈利预测表可能互不一致（例如盈利预测表中的净利润与 DCF 隐含的 NOPAT 对不上）。LEDGER 让 DCF 的全部经营向量逐期取自三表模型的同一情景，并把"DCF 与三表同口径"作为评分标准的硬闸门（第 8、12 节）。

2.2.4 可比公司估值

可比公司法（comparable companies，comps）用同业的估值倍数（P/E、EV/EBITDA、EV/Sales、P/B 等）推算目标公司价值 [36]。它的难点在于可比公司的选取、倍数的口径配对（EV 类倍数配 EV 类分母）、亏损或负值分母的处理（记为 NM，不参与统计）以及离群值处理（常用四分位距规则）。在卖方报告中，可比公司法通常作为 DCF 的交叉验证，并解释目标公司相对同业溢价或折价的原因。

2.2.5 反向 DCF

反向 DCF（reverse DCF）由 Rappaport 与 Mauboussin 的"预期投资"框架推广 [46]：与其从假设推出价值，不如固定价格、反推"当前股价隐含了什么样的增长与盈利能力"，再判断这些隐含预期是否现实。它直接服务于研报中的"预期差"论证——如果股价隐含的收入复合增速远高于历史与行业水平，看多就需要更强的理由。LEDGER 的反向 DCF 固定其余假设为基准情景，每次只动一个变量（收入增速、EBIT 利润率、WACC、永续增长率或退出倍数），用网格扫描找变号区间再二分，求出"每股价值恰好等于参考股价"时该变量的取值，并按历史区间给出可行性分档（第 8 节）。

2.3 金融数值推理与财报问答

财报问答是最早系统研究"LLM 在财务数字上推理"的方向。FinQA[9] 从标普 500 公司的财报中构造了约八千个需要多步数值推理的问答对，并为每题标注了由加减乘除、比较等运算组成的推理程序；ConvFinQA[10] 把它扩展为多轮对话，考察推理链在对话中的延续；TAT-QA[74] 关注表格与文本混合的上下文，答案需要从表格中取数、从文本中取条件；FinanceBench[27] 以真实公开文件为语料构造开卷问答，报告了带检索的强模型在其样本上仍有大量问题答错或拒答；DocFinQA[48] 把 FinQA 的上下文从摘录扩展到完整文件，长度达到十万词量级；BizBench[33] 则把商业与金融定量推理整理为包含程序合成的基准。PIXIU[65] 与 FinBen[66] 把这些任务连同情感分析、信息抽取、预测等任务汇总成金融大模型的综合评测。

与本文的关系。 这些工作确立了两个本文赖以出发的观察：模型在财务数字上的多步推理不可靠；把推理写成可执行程序能显著提升正确率。但它们的设定与研报生产有三点根本不同。(1) **评价单位不同**：财报问答的单位是"一个问题一个答案"，研报的单位是"一篇数万字、含数百个相互依赖数字的文稿"，错误可以出现在任何一个数字上，也可

以出现在数字之间的关系上（前后矛盾、构成描述）。(2) **输出约束不同**：问答只要求答案正确，研报还要求每个数字可追溯到来源、区分事实推论观点、改写后不变。(3) **语言与口径不同**：上述基准以美股英文财报为主，A 股中文年报的口径陷阱（合并与母公司、营业总收入、扣非、季报表头）与版式问题（单位切换、跨页续表）不在其覆盖范围内。本文的数据提取、财报分析与纠错核查可以看作“面向 A 股的、以证据定位为必要条件的财报问答与核查”，但我们不构造问答基准，而是在系统中把“答对”强化为“能证明答对”。

2.4 程序辅助推理与工具使用

思维链（chain-of-thought）提示 [60] 与自洽性解码 [59] 提升了模型的多步推理，但算术仍由模型完成。PAL[20] 与 Program of Thoughts（PoT）[8] 让模型生成程序、由解释器执行，把“推理”与“计算”分开，在数学与金融数值推理上大幅优于纯文本推理。Toolformer[50] 让模型自监督地学会何时调用计算器、检索等工具；ReAct[70] 把推理与行动交错，形成“思考—调用工具—观察”的循环，成为后来大多数智能体框架的基本模式。

与本文的关系。 LEDGER 继承了 PAL / PoT 的核心洞见——计算应交给确定性执行器——但在两个方向上走得更远。**其一，计算不是模型可选调用的工具，而是一条固定的流水线。**在 PAL 中，模型决定写什么程序、程序算什么；在 LEDGER 中，算什么是预先确定的（三条流水线的步骤由 `Pipeline.step` 注册），模型只提供输入（抄录的数字与估值假设），不编写计算代码。这消除了“程序写错”这一新的失败面，也让计算结果在不同运行之间可比、可复现。**其二，约束模型的输出而不仅是模型的计算。**工具使用框架只保证“调用了工具的那部分”是对的，对模型在不调用工具时直接写下的数字没有约束；LEDGER 的数字护栏对文稿中的每个数字做出处核对，不论它是怎么产生的。换言之，PAL 回答的是“模型如何正确地算”，本文回答的是“如何保证模型写下的数都是算出来或抄对的”。

权衡（Trade-off）：让模型写计算代码 vs 固定计算流水线

让分析师会话直接写 Python 计算增速与估值，灵活性最高：遇到新的派生指标无须改引擎。我们没有这样做，原因有三：(1) 模型写的计算代码本身需要审计，而审计一段每次都不同的代码比审计一个固定引擎困难得多；(2) 评审复现要求“用保存的输入文件逐位复算”，若计算代码由模型生成，复现就依赖模型输出；(3) 估值口径（EBIT 定义、年中折现、股权价值桥的项目）需要在全部公司之间一致，固定引擎天然保证这一点。作为折中，`extras.json` 允许模型声明“我需要某两个已核验数字的增速或差值”（`ratios` 中的 `op: growth / diff`），由程序计算并登记出处——模型可以决定“算什么”，但不能决定“怎么算”和“结果是多少”。

2.5 金融大模型与金融智能体

金融大模型。 BloombergGPT[63] 以大规模金融语料与通用语料混合训练了一个约五百亿参数的模型，在金融 NLP 任务上优于同规模通用模型；FinGPT[67] 走开源、数据中心的路线，强调低成本的轻量适配与持续更新的金融数据管道。这一方向的目标是让模型本身“更懂金融”。本文不训练模型；我们的经验是，对研报生产而言，现成通用模型的金融知识已经足够，瓶颈在于数字的可靠性与可追溯性，而这不是通过领域预训练能解决的问题——一个更懂金融的模型同样可能取到母公司报表的数字。

金融智能体。 FinMem[71] 为交易智能体设计了分层记忆与角色设定；Zhang 等人的 FinAgent[72] 是一个面向交易的多模态基础智能体，结合工具增强与多样化的市场信

息[1]；TradingAgents[64] 模拟交易公司的组织结构，设置基本面、情绪、新闻、技术分析师与多空研究员、交易员、风控等角色，通过辩论形成交易决策；FinRobot[68] 是一个开源的金融智能体平台，提供分层的智能体与工具组织，其示例包括基于公司年报生成个股分析报告。

与本文的关系。 这些系统大多以**交易决策**为目标，评价指标是回测收益、夏普比率等；本文以**研究产出**为目标，评价指标是数字闭合、分析质量与表达合规，不做任何交易或收益声明。即便是与本文最接近的研报生成类示例，其关注点也在于"能否生成一篇结构完整的报告"，对"报告中每个数字从哪里来、错了能否被发现"缺少系统性的约束——数字通常由模型从检索结果或工具返回中直接转写进文本，既没有逐字回查，也没有对最终文稿的出处核对。此外，现有金融智能体中的多角色通常在同一进程、同一对话历史中协作（辩论、讨论），而本文刻意让角色之间**不共享上下文**，以获得独立复核的效果（§2.6）。

2.6 多智能体框架

AutoGen[62] 提供可对话智能体的抽象，智能体之间通过消息对话协作，并可接入工具与人类；CAMEL[35] 通过角色扮演与"起始提示"让两个智能体自主协作完成任务；MetaGPT[25] 把软件公司的标准作业流程（SOP）编码进多智能体系统，各角色产出结构化文档（需求、设计、接口）而不是自由对话，以减少级联幻觉；ChatDev[45] 以"聊天链"组织设计、编码、测试等阶段；Generative Agents[44] 研究了带记忆与反思的智能体社会模拟；多智能体辩论 [16] 让多个模型实例相互质疑以提升事实性与推理。

与本文的关系。 LEDGER 与 MetaGPT 在精神上最接近：都把一个人类组织的生产流程映射为多个角色，都以结构化产物而不是对话作为交接物。差异在于三点。**(1) 编排器不是模型。**上述框架中，任务的推进、角色的切换、何时结束通常由某个模型（或模型之间的对话）决定；LEDGER 的编排器是确定性 Python 状态机，起哪个会话、闸门是否通过、问题单路由给谁、何时停止都写成代码，行为可预期、可单元测试。**(2) 交接物受程序闸门约束。** MetaGPT 的结构化文档提高了交接质量，但文档内容是否正确仍由下游模型判断；LEDGER 的每个交接文件都有程序判定的闸门（数字逐字回查、来源摘句含数、护栏通过、数字锁通过、页数与图表数下限），不通过就不会启动下一个角色。**(3) 上下文隔离是设计目标而不是实现细节。**多数框架让角色共享对话历史或共享记忆，以便协作；我们观察到共享上下文会让复核者继承写作者的盲点、让评审者因看到过程而宽容，因此每个角色是一次独立的无头会话，连工作区都与仓库物理隔离，盲评更在第三个目录中进行。多智能体辩论依赖"不同实例给出不同观点"，本文则依赖"不同角色看到不同信息"——复核编辑只看到文稿与产物，看不到写作过程；盲评只看到交付物，看不到任何过程。

2.7 自我改进与反思

Self-Refine[39] 让同一模型对自己的输出给出反馈并迭代改写；Reflexion[52] 让智能体把失败经验写成语言反思存入记忆，在后续尝试中利用；Chain-of-Verification[15] 让模型先起草、再生成验证问题、独立回答后修正初稿。另一方面，Huang 等人 [26] 指出，在没有外部反馈的情况下，模型的"自我纠正"往往不能提升推理正确率，甚至会把对的改错。

[1] 与本系统的代码仓库名 `finagent` 同名，二者无关。

与本文的关系。 LEDGER 的 `refine` 闭环在形式上是一个“生成—评价—修订”的迭代，但它在三个方面区别于自我改进：(1) **反馈来自外部且部分是程序**——L0/L1 由程序判定，L2/L3 由一个看不到写作过程的不同会话判定，这与 Huang 等人的结论一致：有效的改进需要外部信号；(2) **修订受数字锁约束**——Self-Refine 式的改写可以任意修改输出，而本文中复核与修订会话的改写必须保持数字不增不改，需要改数字的问题被路由回分析师重跑数据阶段，由引擎重新计算；(3) **有停止与回滚规则**——迭代满三轮、连续两轮提升不足 2 分或无可回传问题时停止，结束时回滚到得分最高的一轮，避免“越改越差”。Reflexion 的跨回合记忆在本文中没有对应物：每轮修订会话都是新会话，只读脱敏的问题单，不积累对评分标准的“经验”，这是有意为之，以减少对评分标准的过拟合。

2.8 幻觉与可追溯生成

幻觉（hallucination）是生成模型的系统性问题 [28]。检索增强生成（RAG）[34] 把外部文档作为条件，Self-RAG[3] 让模型自行决定何时检索并对生成内容自我评判。在“可追溯”方向，ALCE[21] 建立了带引用文本生成的基准，用引用召回与引用精确率衡量引用是否支撑陈述；Attributed QA[7] 与 AIS 框架 [47] 形式化了“可归因于来源”的判定；Liu 等人 [37] 评测了生成式搜索引擎，发现相当比例的陈述没有被其引用完全支撑；FActScore[41] 把长文拆成原子事实逐条核验；RARR[19] 在生成后检索证据并修订不被支撑的内容。

与本文的关系。 本文的“数字可追溯”可以看作可归因生成在数值领域的一个强化版本，有三点不同。**(1) 判定是确定性的。** ALCE、AIS、FActScore 判断“引用是否支撑陈述”依赖自然语言推理模型或人工；LEDGER 只对数字做判定，判定规则是字符串与数值比较：抄录的数字必须按数字边界出现在所写页码的原文中，外部登记的数字必须原样出现在摘句中，文稿中的数字必须按展示精度等于某个产物值。这种判定无法覆盖非数值陈述，但在其覆盖范围内没有判定误差。**(2) 来源有层级。** 一般的引用生成只有“文档—陈述”一层；本文的数字有“原文—事实—真值表—派生—产物—文稿”多层，每层都可以反向追溯，派生数的出处是公式与输入单元格而不是某段原文。**(3) 出处类型被显式区分。** 本文按出处把文稿中的每个数字归为事实（报告披露、外部来源）、推论（程序计算）或观点（模型预测、估值假设），这对应竞赛与行业对“区分事实、推论与观点”的要求，是一般可归因生成框架没有的维度。另一方面，我们的出处核对只证明“存在一个匹配的值”，不证明“该值配给了正确的指标”——后者由指标—期间绑定核对部分覆盖，其余依赖复核与盲评。这一局限在第 3 节的威胁模型中明确列出。

2.9 LLM 评审与基于评分细则的评测

用强模型作为评审（LLM-as-a-judge）已成为开放式生成评测的主流做法。Zheng 等人 [73] 以 MT-Bench 与 Chatbot Arena 系统研究了 LLM 评审与人类偏好的一致性，并指出位置偏差、冗长偏好与自我增强偏差；Wang 等人 [57] 进一步展示了仅调换候选顺序即可改变评审结论；Panickssery 等人 [43] 发现评审模型能识别并偏好自己的生成。G-Eval[38] 用思维链与表单填写提升评审与人类的一致性；Prometheus[31] 训练了以细粒度评分细则（rubric）为条件的开源评审模型；HealthBench[2] 以大量医生撰写的、针对具体对话的评分条目评测医疗对话，把“开放式质量”拆成可逐条判定的二元条目，再由程序加权计分。

与本文的关系。 本文的评测体系在结构上借鉴了 HealthBench 一类的"逐条二元判定 + 程序计分"，在此基础上做了四点调整。**(1) 程序判与模型判分层。**能由程序确定判定的条目（数据闭合、数字前后矛盾）一律由程序判，模型只判程序判不了的分析质量与表达；v2 评分标准把"数字前后矛盾"从模型判改为程序判，正是因为观察到评审对该条目的判断随措辞波动。**(2) 评分标准对生成方不可见。**生成方的工作区不含评分标准与质检代码，问题单只含理由与证据，不含条目原文与分值，以减轻 Goodhart 效应 [40]。**(3) 评审看不到过程与身份。**盲评目录只含交付物，评审不知道写作过程与生成模型名，以减轻自我偏好与"过程宽容"。**(4) 评审本身被校准。**我们不假定评审可靠，而是用三组证据度量它：同稿重复评审的逐条一致率与总分极差；程序制造已知缺陷（劣化算子）后的判出率与对非目标条目的连带改判；人工复核评审判出问题的精确率。这些度量的结果并不完美（§1.7），但它们使"评审有多可靠"成为一个可报告的数字，而不是默认假设。

2.10 编码智能体 harness

近两年，面向软件工程的编码智能体发展迅速。SWE-bench[30] 以真实 GitHub 问题评测模型修复代码的能力；SWE-agent[69] 提出"智能体—计算机接口"（agent-computer interface），说明为模型设计合适的命令与反馈格式能显著提升任务完成率；OpenHands[58] 提供了开放的通用编码智能体平台。Claude Code[1] 是 Anthropic 发布的命令行编码智能体，支持读写文件、执行命令、联网检索、技能（skill）文档与项目级 `CLAUDE.md` 指令，并提供无头模式（`claude -p`）以 JSON 事件流输出完整的工具调用记录，以及按工具与路径的权限白名单 / 黑名单。

与本文的关系。 本文把一个通用编码智能体用作投研多智能体系统的 harness，而不是从头编写智能体循环。这一选择的理由是：(1) 研报生产中模型要做的事——读 PDF、读写 JSON 与 Markdown、运行命令行校验、根据报错修改文件、联网检索、写 Python 排版脚本——恰好是编码智能体最擅长的工作模式；(2) SWE-agent 的经验表明，接口设计比智能体循环本身更重要，我们因此把精力放在"给模型什么命令、命令返回什么反馈"上：每个引擎命令都返回结构化的拒收理由与下一步建议，`status` 命令直接告诉会话"下一道闸门是什么、差什么"；(3) 无头模式的事件流天然满足竞赛"全程记录工具调用、可追溯"的要求，编排器把它原样落盘到工作区 `.pipeline/agents/`；(4) 权限系统让我们能在操作系统层面之上再加一层约束：禁止读取仓库目录、禁止修改引擎与技能。与 SWE-bench 一类工作不同，我们不评测 harness 本身的能力，而是把它当作一个可替换的执行器——编排器只依赖"启动一个会话、传入提示与工具白名单、读回事件流"这一最小接口。

2.11 对比与定位

表 7 把代表性工作与本文在八个能力维度上做对比。需要说明：这些工作的目标各不相同，多数并不以表中维度为设计目标，表格只用于说明本文的定位，不构成优劣评价；"◦"表示部分具备或依配置而定。

从表中可以看出本文的定位：本文没有在任何单一维度上提出全新的技术原语——确定性计算、引用核验、多角色、LLM 评审都已有大量前人工作；本文的贡献在于**把这些原语以"不变量 + 强制机制"的方式组合进同一个系统**，使它们在研报生产这一长链条任务中同时成立，并且对每一条不变量都给出执行它的具体代码位置与测试。这种组合并不平凡：例如数字锁（D）只有在数字护栏（A、B、C）已经保证初稿数字有出处的前提下才有意义；评分标准不可见（F）只有在工作区物理隔离、问题单脱敏的前提

表 7　与代表性工作的能力对比。A：计算由固定的确定性流水线完成（而非模型心算或模型临时编写的代码）；B：模型给出的数字逐字回查原文页面；C：外部信息登记来源且正文强制引用；D：改写受数字锁约束；E：角色之间上下文隔离；F：评分标准对生成方不可见；G：评审本身被校准；H：面向 A 股年报到研报的端到端产出。✓ 具备，◦ 部分具备，✗ 不具备或不涉及。

工作	A	B	C	D	E	F	G	H
FinQA / TAT-QA / FinanceBench[9, 27, 74]	✗	✗	✗	✗	✗	✗	✗	✗
PAL / PoT[8, 20]	◦	✗	✗	✗	✗	✗	✗	✗
Toolformer / ReAct[50, 70]	◦	✗	◦	✗	✗	✗	✗	✗
BloombergGPT / FinGPT[63, 67]	✗	✗	✗	✗	✗	✗	✗	✗
FinMem / FinAgent（交易）[71, 72]	◦	✗	✗	✗	✗	✗	✗	✗
TradingAgents[64]	◦	✗	✗	✗	✗	✗	✗	✗
FinRobot[68]	◦	✗	◦	✗	✗	✗	✗	◦
AutoGen / CAMEL[35, 62]	✗	✗	✗	✗	✗	✗	✗	✗
MetaGPT / ChatDev[25, 45]	✗	✗	✗	✗	◦	✗	✗	✗
Self-Refine / Reflexion[39, 52]	✗	✗	✗	✗	✗	✗	✗	✗
ALCE / Self-RAG / RARR[3, 19, 21]	✗	◦	✓	✗	✗	✗	✗	✗
MT-Bench / Prometheus / HealthBench[2, 31, 73]	✗	✗	✗	✗	✗	◦	◦	✗
SWE-agent / OpenHands / Claude Code[1, 58, 69]	✗	✗	✗	✗	◦	✗	✗	✗
LEDGER（本文）	✓	✓	✓	✓	✓	✓	✓	✓

下才能成立；评审校准（G）需要能够程序化制造已知缺陷的劣化算子，而这又依赖研报纠错引擎（纠错核查子选题）对文稿结构的解析能力。各机制之间的依赖关系在第 4 节展开。

2.12　小结

本章补充了理解后文所需的两类背景。在领域侧，A 股定期报告的口径（合并与母公司、营业总收入与营业收入、归母与扣非、季报表头的期间含义、单季拆分）与版式（单位切换、跨页续表、同名明细表）共同决定了“从年报取对一个数”远比看上去困难；卖方研报的估值方法（三表联动、FCFF DCF、可比公司、反向 DCF）对少数参数高度敏感，且各部分之间必须口径一致。在方法侧，财报问答与程序辅助推理证明了“计算应交给执行器”，可归因生成证明了“陈述应有可核验的来源”，多智能体框架证明了“按组织流程分工”有效，LLM 评审研究提醒我们“评审本身有偏差、需要度量”。本文在这些工作之上，针对研报生产这一长链条、高数字密度、需要可追溯与可复现的任务，把上述思想组合为一组可强制执行的系统不变量。下一章给出这些不变量的形式化定义。

3　问题形式化

本章给出全文统一使用的记号，并把三项计算任务（财报分析、估值建模、纠错核查；数据提取贯穿其中）、系统要保证的性质、威胁模型、研报生成过程与评价过程分别形式化。形式化的目的不是追求数学上的完备，而是把“可追溯”“数字闭合”“LLM 不做算术”这类口号变成可以逐条检查、可以写成测试的命题，并把每个命题对应到代码中执行它的具体位置。凡是形式化与实现有差异（例如实现做了简化或只覆盖一部分情形），我们都在正文中明确指出。

本章结构如下：§3.1 汇总记号；§3.2 ~ §3.4 定义文档、证据、事实与真值表；§3.5 ~

§3.7 定义三项任务；§3.8 定义系统性质；§3.9 给出单位与口径约定；§3.10 给出威胁模型与系统不变量；§3.11 与 §3.12 分别形式化研报生成与评价。

3.1 记号总览

表 8 全文记号。代码中的对应物指仓库中实现该概念的主要数据结构或文件。

记号	含义	代码中的对应物
$\mathcal{D}$	一份定期报告（年报、季报或招股书），解析为 chunk 序列	`parsing.document.Document`
$\mathcal{P}$	$\mathcal{D}$ 的页集合 $\{1,\dots,\lvert\mathcal{P}\rvert\}$	`Chunk.page`
c	一个 chunk：$(\mathrm{id}, \mathrm{source}, p, \kappa, \mathrm{text}, \mathrm{rows})$	`Chunk`
$\mathcal{M}$	指标集合（可抽取指标与派生指标）	`schema.METRICS`
$\mathcal{K}$	期间集合（年度、半年度、单季、前三季度累计）	`external.PERIOD_RE`
$f = (m,k,v,u,\sigma)$	事实：指标、期间、数值、单位、来源定位	`extract.Fact`
σ	来源定位 $(\mathrm{source}, \mathrm{page}, \mathrm{chunk_id}, \mathrm{row}, \mathrm{col})$	`Fact.source`
F_R, F_L	规则通道、模型通道的事实集合	`extract_rule`、`ingest`
$\mathcal{T}$	真值表：$\mathcal{M}\times\mathcal{K}$ 上的部分函数（含派生单元格）	`analysis.json` 的 `truth_table`
θ	估值假设（情景向量、WACC 参数、终值参数、分部量价等）	`valuation_assumptions.json`
$\mathcal{C}$	研报断言集合	`claims.Claim`、`claims.json`
$\mathcal{S}$	外部来源登记	`research/sources.json`
$\mathcal{N}$	文稿（研报正文 Markdown）	`report/narrative.md`
$\mathcal{N}^0$	复核改写前的初稿	`report/narrative.draft.md`
$\mathcal{A}$	本次运行的结论性产物集合	`guard.TRUTH_FILES` 所列文件
$\mathcal{W}$	工作区状态（文件集合及其内容与修改时间）	工作区目录
G_i	第 i 道闸门（$i=1,\dots,11$），$\mathcal{W}\to\{0,1\}$	`report_pipeline.status`
$\mathcal{R}$	评分标准（条目集合）	`eval/rubrics/report_v2.json`
$\mathrm{ulp}(r)$, $\frac{1}{2}\mathrm{ulp}(r)$	原文写法 r 末位的单位及其一半	`numbers.half_ulp`
YoY, QoQ	同比、环比算子	`analyze.prior_periods`

3.2 文档、页与证据

定义 3.1 (文档与 chunk)**.** 一份文档 $\mathcal{D}$ 是 *chunk* 的有限序列 $\mathcal{D}=(c_1,\dots,c_n)$。每个 *chunk* 为 $c=(id, source, p, \kappa, text, rows, unit, caption)$，其中 *source* 是文件名，$p\in\mathcal{P}$ 是页码，$\kappa\in\{text, table\}$ 是类型，*text* 是该 *chunk* 的文本，*rows* 是表格 *chunk* 的单元格矩阵（文本 *chunk* 为空），*unit* 是从“单位：”行识别的单位提示，*caption* 是表格标题。记 $\mathcal{D}[p]=\{c\in\mathcal{D}: c.p=p\}$ 为第 p 页的全部 *chunk*。文档另有内容哈希 $h(\mathcal{D})$（*SHA-256*）。

跨页续表的续页 chunk 另带“候选表头”与指向上一页表格的引用（`continued_from`）；扫描页（无文本层）的 $\mathcal{D}[p]$ 为空，任何声明该页为来源的数字都无法通过下面的证据谓词，相当于直接拒收。

数字的原文写法。我们区分一个数字的*原文写法* r（字符串，如 `"612,380,000.00"`、`"(1,234.56)"`、`"14.49%"`）与其*数值* $\mathrm{parse}(r)\in\mathbb{R}\cup\{\perp\}$。parse 去除空白、千分位与百分号（百分号不除

以 100），把括号解释为负号，把“—”“-”“不适用”等占位符解析为 $\perp$。展示精度由 r 的小数位数 $d(r)$ 决定：

$$\mathrm{ulp}(r) = 10^{-d(r)}, \qquad \tfrac{1}{2}\,\mathrm{ulp}(r) = \tfrac{1}{2}\cdot 10^{-d(r)}. \tag{4}$$

例如 r = `"58.62"`（亿元）时 $\frac{1}{2}\,\mathrm{ulp}(r) = 0.005$ 亿元，即 50 万元。

定义 3.2 (数字边界匹配)**.** *对原文写法 r，令 $\mathrm{core}(r)$ 为去掉千分位、括号、正负号与百分号后的数字串（须形如 `\d+(\.\d+)?`，否则无定义）。称 r 按数字边界出现在文本 s 中，记 $r \sqsubset s$，当且仅当在去掉千分位的 s 中存在 $\mathrm{core}(r)$ 的一次出现，其左侧紧邻字符不是数字或小数点，右侧紧邻字符不是数字。*

数字边界的作用是避免子串误配：`"12.3"` 不会匹配到 `"112.34"` 中，`"612380000"` 不会匹配到 `"6123800001"` 中。实现中保留单元格之间的空白，以防删除空白后相邻两个数字被拼接成一个（例如 `"235 3884.59"` 被误拼为 `"2353884.59"`），这是一个在真实年报上踩到的问题，对应测试 `tests/test_evidence_spacing.py`。

定义 3.3 (证据谓词)**.** *给定文档 $\mathscr{D}$、页码 p 与原文写法 r，定义*

$$\mathrm{Ev}(\mathscr{D}, p, r) = \begin{cases} c^{\star} & \text{若存在} c \in \mathscr{D}[p] \text{ 使} r \sqsubset c.\mathit{text};\ c^{\star} \text{ 取满足条件的第一个 } \mathit{chunk},\ \text{表格优先} \\ \perp & \text{否则。} \end{cases}$$

Ev 的返回值即该数字的 *chunk* 级定位。实现为 `external.find_evidence`。

证据谓词刻画的是“这个写法在这一页出现过”，而不是“这个写法在这一页表示这个指标”。前者是可判定的字符串性质，后者是语义性质。LEDGER 用证据谓词保证模型没有编造或改写数字，用规则通道的交叉比对（§3.4）部分覆盖语义性质。

3.3 指标、期间与事实

指标。 指标集合 $\mathscr{M}$ 由指标字典给出（`finagent/tasks/financials/schema.py`）。每个指标 m 有类型 $\mathrm{kind}(m) \in \{\mathrm{amount}, \mathrm{per_share}, \mathrm{pct}, \mathrm{shares}, \mathrm{ratio}\}$ 与所属报表 $\mathrm{stmt}(m) \in \{\mathrm{key_data}, \mathrm{income}, \mathrm{balance}, \mathrm{cashflow}, \mathrm{non_recurring}, \mathrm{derived}\}$。$\mathscr{M} = \mathscr{M}_{\mathrm{ext}} \cup \mathscr{M}_{\mathrm{der}}$，其中 $\mathscr{M}_{\mathrm{ext}}$ 是可抽取指标（营业收入、营业成本、各项费用、投资收益、利润总额、净利润、归母净利润、扣非归母净利润、非经常性损益、经营活动现金流量净额、资本开支、折旧摊销、资产负债表主要科目、每股收益、ROE、总股本等），$\mathscr{M}_{\mathrm{der}}$ 是只能由程序派生的指标（毛利率、净利率、资产负债率、各项费用率、净现比）。称 m 为*发生额指标*，当且仅当 $\mathrm{kind}(m) = \mathrm{amount}$ 且 $\mathrm{stmt}(m) \notin \{\mathrm{balance}, \mathrm{derived}\}$；只有发生额指标可以跨期相减。每个指标还有两组别名：财报科目别名 $A_{\mathrm{fs}}(m)$（规则通道匹配表格行名用）与研报说法别名 $A_{\mathrm{cl}}(m)$（纠错时匹配研报措辞用）。

期间。 期间集合由文法

$$k \ ::= \ Y \mid Y\mathrm{H1} \mid Y\mathrm{Q}q \mid Y\mathrm{Q1\text{-}3}, \qquad Y \in \{2000, \ldots, 2099\},\ q \in \{1,2,3,4\}$$

生成（与 `external.PERIOD_RE` 一致）。定义同比基期与环比基期算子：

$$\beta_{\mathrm{YoY}}(Ys) = (Y-1)s, \qquad \beta_{\mathrm{QoQ}}(Y\mathrm{Q}q) = \begin{cases} Y\mathrm{Q}(q-1) & q > 1 \\ (Y-1)\mathrm{Q4} & q = 1 \end{cases},$$

β_{QoQ} 只对单季期间有定义。对数值 a, b，变动率定义为

$$\Delta\%(a,b) = \frac{a-b}{|b|} \times 100, \qquad b \neq 0, \tag{5}$$

分母取绝对值是 A 股惯例：基期为负时，“亏损收窄”表现为正增长。比率类指标的变动用百分点差 $a-b$，不用变动率。

定义 3.4 (事实)**.** 一条事实是五元组 $f=(m,k,v,u,\sigma)$，附带原文写法 r 与抽取方式 $\mu\in\{rule, llm, rule{+}llm, derived\}$。其中 $m\in\mathcal{M}$，$k\in\mathcal{K}$，u 为原文单位，v 为规范化数值：

$$v=\begin{cases}\phi(u)\cdot \mathrm{parse}(r) & \mathrm{kind}(m)\in\{amount, shares\}\\ \mathrm{parse}(r) & \text{否则}\end{cases},$$

ϕ 为单位换算系数（元 $=1$，千元 $=10^3$，万元 $=10^4$，百万元 $=10^6$，亿元 $=10^8$；股、万股、亿股同理）。来源定位 $\sigma=(source, page, chunk_id, row, col)$；对模型通道的事实，$row, col$ 可缺省，$chunk$ 由证据谓词给出；对派生事实，σ 替换为公式与输入单元格列表。

定义 3.5 (可接受的模型抽取条目)**.** 模型通道提交的一条抽取条目 $x=(m,k,r,u,p,scope,file)$（$file$ 即 `source_file`）被接受，当且仅当以下条件全部成立：*(i)* $m\in\mathcal{M}_{ext}$；*(ii)* k 符合期间文法；*(iii)* $scope=$ 合并；*(iv)* $\mathrm{parse}(r)\neq\bot$；*(v)* $file$ 是主文件或已传入的补充文件之一，记其文档为 $\mathcal{D}'$；*(vi)* $\mathrm{Ev}(\mathcal{D}',p,r)\neq\bot$；*(vii)* 若 $\mathrm{kind}(m)=amount$，则 u 属于可换算单位。不满足的条目进入拒收集合 Rej，附带具体理由。实现为 `external._to_fact` 与 `ingest`。

3.4 双通道抽取与真值表

定义 3.6 (两个通道与一致性)**.** 规则通道 $R(\mathcal{D})$ 从主文件的合并报表表格中按科目别名与表头期间取数，得到事实集 F_R；模型通道把模型提交的条目按定义 3.5 过滤，得到事实集 F_L。对键 $\kappa=(m,k)$，记 $F(\kappa)$ 为通道中该键的事实。两条事实 f, f' 称为一致，当且仅当 $|f.v-f'.v|\le\max(10^{-6}|f'.v|,\,0.01)$。冲突集合定义为

$$\mathrm{Conf}=\{\kappa:\exists f\in F_R(\kappa), f'\in F_L(\kappa), f, f'\text{ 不一致}\}\cup\{\kappa:\text{同一通道内}\kappa\text{ 的两处取值不一致}\}.$$

合并规则是：一致时保留一条，抽取方式记为 `rule+llm` 并把另一条的来源记为佐证；只有一个通道有值时保留该值；冲突时**不静默取舍**——在流水线中暂时保留模型通道的值（它来自模型逐页读原文，且已过证据谓词），但把该键记入 Conf，并在 $\mathrm{Conf}\neq\emptyset$ 时阻断估值（`pipelines._p_analyze`）。季报补充文件不进入规则通道（§2.1），其事实只来自模型通道。

定义 3.7 (派生规则与真值表)**.** 派生规则是形如 $\rho:(m,k)\leftarrow g((m_1,k_1),\dots,(m_j,k_j))$ 的确定性等式，g 为初等函数。本文使用三类规则：

1. **季度规则**（只作用于发生额指标 m，对每个年度 Y）：
$$(m,YH1)\leftarrow(m,YQ1)+(m,YQ2),\quad (m,YQ1\text{-}3)\leftarrow(m,YH1)+(m,YQ3),$$
$$(m,YQ2)\leftarrow(m,YH1)-(m,YQ1),\quad (m,YQ3)\leftarrow(m,YQ1\text{-}3)-(m,YH1),$$
$$(m,YQ4)\leftarrow(m,Y)-(m,YQ1\text{-}3);$$
2. **比率规则**（每个期间 k）：毛利率 $=$ (营业收入 $-$ 营业成本)/营业收入 $\times 100$，净利率、各项费用率、资产负债率同理，净现比 $=$ 经营活动现金流量净额 / 归母净利润；
3. **恒等规则**：非经常性损益 $=$ 归母净利润 $-$ 扣非归母净利润（仅在未列示时派生）；折旧摊销 $=$ 固定资产折旧 $+$ 无形资产摊销。

给定合并后的事实集 F，真值表 $\mathcal{T}=\mathrm{cl}(F)$ 是从 F 出发反复应用派生规则、直到不再产生新单元格为止得到的部分函数 $\mathcal{T}:\mathcal{M}\times\mathcal{K}\rightharpoonup\mathbb{R}$。规则只在目标单元格不存在且全部输入单元格存在时触发，从不覆盖已有单元格；派生单元格记录公式与输入单元格的来源。实现为 `analyze._split_quarters` 与 `_derive`。

命题 3.1 (闭包的终止性与条件唯一性)**.** *(a)* $\mathrm{cl}(F)$ 的计算必然终止。*(b)* 若 F 满足全部季度等式（即凡等式两侧单元格均为列示值时，等式精确成立），则 $\mathrm{cl}(F)$ 与规则的应用顺序无关。

证明. (a) 涉及的键集合有限（F 中出现的指标与年度只有有限个，每个年度的期间只有七种），每次触发规则都使已定义单元格数严格增加且从不删除，故步数有限。(b) 季度规则构成线性方程组 {H1 = Q1 + Q2, Q1-3 = H1 + Q3, 年 = Q1-3 + Q4}，每条规则只是其中某个等式对某个未知量的求解。若某个目标可经两条不同路径派生，两条路径的结果之差可以写成列示单元格上若干等式残差的线性组合；当列示值满足全部等式时残差为零，两路径结果相同。比率与恒等规则的输入在其触发时已确定，不受顺序影响。 □

命题 3.1(b) 的前提在真实数据中未必成立（例如年报分季度表中的 Q4 与"全年 − 三季报累计"因追溯调整而不等）。此时闭包结果依赖顺序，正因如此，`analyze()` 对"列示单季 vs 累计相减"逐项做勾稽，把不满足前提的情形显式暴露为勾稽失败。为避免"自己和自己比"，实现为每个单元格 κ 维护其列示依赖集 $B(\kappa) \subseteq \mathrm{dom}(F)$：列示单元格 $B(\kappa) = \{\kappa\}$，派生单元格 $B(\kappa) = \bigcup_i B(\kappa_i)$。一项季度勾稽 $\kappa = \kappa_a - \kappa_b$ 只在 $B(\kappa) = \{\kappa\}$（目标本身是列示值）且 $\kappa \notin B(\kappa_a) \cup B(\kappa_b)$ 时才执行。

定义 3.8 (勾稽)**.** 勾稽是一组等式 $E = \{e_1, \ldots, e_q\}$，每个 e_j 形如 $\ell_j(\mathcal{T}) = r_j(\mathcal{T})$，带相对容差 ϵ_j。对期间 k，e_j 的状态为：输入缺失时"不可评估"；$|\ell_j - r_j| \le \max(\epsilon_j |r_j|, 1\text{元})$ 时"通过"；否则"失败"。本文的 E 包括：资产总计 = 负债和所有者权益总计；负债和所有者权益总计 = 负债合计 + 所有者权益合计；所有者权益合计 = 归母权益 + 少数股东权益；列示的非经常性损益 = 归母净利润 − 扣非归母净利润；年报列示的同比增减 *vs* 程序计算的同比（容差 $\frac{1}{2}\,\mathrm{ulp}(r) + 0.01$）；列示单季 *vs* 累计相减（满足依赖条件时）。

定义 3.9 (抽取通过)**.** 称一次抽取通过，记 $\mathrm{Pass}_{ext} = 1$，当且仅当 $\mathrm{Rej} = \emptyset$、$\mathrm{Conf} = \emptyset$、全部勾稽无"失败"，且本期核心指标（营业收入、营业成本、营业利润、利润总额、净利润、归母净利润、扣非归母净利润、资产总计、负债合计、所有者权益合计、货币资金、经营活动现金流量净额、资本开支、基本每股收益）无缺失。只有 $\mathrm{Pass}_{ext} = 1$ 时真值表才可被估值使用（实现为 `verify_facts` 的 `ok` 字段与流水线中的阻断检查）。

3.5 任务一：数据提取与财报分析

输入。 主文件 $\mathcal{D}$（年报，或招股书），可选补充文件 $\mathcal{D}_1, \ldots, \mathcal{D}_s$（季报），报告年度 Y，模型抽取文件 X（可缺省，缺省时只走规则通道），可选的经营数据抄录 X_{extra}、一致预期表 $\mathcal{C}$ 与业绩预告表 $\mathcal{G}$。

输出。 分析产物 $A_2 = (\mathcal{T}, \mathrm{Chg}, \mathrm{Tie}, \mathrm{Sig}, \mathrm{Qtr}, \mathrm{Earn}, \mathrm{Ext})$：

- $\mathcal{T}$：真值表（定义 3.7），每个单元格带来源或公式；
- Chg：关键指标在期间 k 的水平值、同比（Δ% 或百分点差）、环比（单季期间）；
- Tie：勾稽结果（定义 3.8）；
- Sig：异常信号集合，每个信号 $\sigma = (\text{id}, \text{类型}, \text{严重度}, \text{事实陈述}, \text{推论}, \text{证据})$，由阈值规则触发（非经常性损益占比、归母与扣非增速差、净现比、利润与现金流方向背离、应收与存货增速快于收入、毛利率大幅变动、会计口径变化披露）；
- Qtr：单季表，逐季标注"列示"或"累计相减推出"及公式；
- Earn：实际值 vs 一致预期（超预期 / 符合 / 低于）、业绩预告区间对比、EPS 质量筛查、利润变动归因桥；

- Ext：经营数据表（逐格回查原文后的分产品、分渠道、产销量等）与程序计算的比率。

信号把“事实陈述”与“推论”分成两个字段，对应研报“区分事实与推论”的要求；信号阈值是确定性的，不由模型判断。

利润变动归因的闭合约束。 利润变动归因桥把归母净利润的同比变化分解为

$$\begin{aligned}\Delta \mathrm{NP}_{\text{parent}} = \underbrace{\Delta \mathrm{Rev} \cdot \mathrm{GM}_{k-1}}_{\text{收入规模效应}} + \underbrace{\mathrm{Rev}_k \cdot \Delta \mathrm{GM}}_{\text{毛利率效应}} - \sum_e \Delta \mathrm{Exp}_e + \Delta\text{其他经营损益} \\ + \Delta\text{营业外收支} - \Delta\text{所得税} - \Delta\text{少数股东损益},\end{aligned} \tag{6}$$

其中 e 遍历销售、管理、研发、财务费用。我们要求等式**严格闭合**：“其他经营损益”作为轧差项吸收税金及附加、投资收益、减值等未单列项目，从而各项之和精确等于归母净利润的变化，正文对归因项的任何引用都能与桥表对账。

正确性标准与指标。 设金标准 $\mathcal{G}^\star = \{(m,k) \mapsto v^\star\}$。抽取正确率（`eval/run_eval.py`）为

$$\mathrm{Acc} = \frac{1}{|\mathcal{G}^\star|} \sum_{(m,k)} \mathbb{1}\big[\mathcal{T}(m,k) \text{ 有定义且} |\mathcal{T}(m,k) - v^\star| \le 10^{-9}|v^\star| + 10^{-6}\big].$$

在没有金标准的真实年报上，我们报告*规则通道覆盖率*（模型抽取并通过校验的核验值中规则通道独立取到的比例）与*错配数*（两通道都有值但不一致、最终仍未解决的键数）。勾稽通过率与信号的事实正确性作为辅助指标。

3.6 任务二：自动化估值建模

输入。 真值表 $\mathcal{T}$（要求 $\mathrm{Pass}_{\text{ext}} = 1$）、估值假设 θ、可选经营数据 Ext、可比公司表 $\mathcal{P}$、参考股价 P_0 及其来源。θ 包含：三个情景 $s \in \{\text{base}, \text{downside}, \text{upside}\}$ 的收入增速向量（或分部量价设定）、资本开支率、终值参数与 WACC 调整；三表模型参数（毛利率偏移、周转天数偏移、分红率等）；WACC 参数（无风险利率 r_f、β、股权风险溢价 ERP、规模 / 特定 / 国别溢价、税前债务成本 k_d、边际税率 t、目标资本结构权重）；非经营性资产明细；评级规则（可缺省）。

分部预测。 若 θ 含分部设定，对分部 j、预测年 t、情景 s：

$$\mathrm{Vol}_{j,t} = \mathrm{Vol}_{j,t-1}(1 + g^{\text{vol}}_{j,t,s}), \quad \mathrm{Price}_{j,t} = \mathrm{Price}_{j,t-1}(1 + g^{\text{price}}_{j,t,s}), \quad \mathrm{Rev}_{j,t} = \mathrm{Vol}_{j,t} \cdot \mathrm{Price}_{j,t},$$

公司收入增速、综合毛利率与经营费用率由分部加总得到，覆盖 θ 中对应的情景向量。基期（$t = 0$）的分部收入之和须与真值表营业收入闭合（评分条目 L1-02）。

三表模型。 三表模型是一个映射 $\mathrm{TS} : (\mathcal{T}, \theta, s) \mapsto (\mathrm{IS}_t, \mathrm{BS}_t, \mathrm{CF}_t)_{t=1}^{N}$，单位百万元，满足以下约束（勾稽容差 0.05 百万元）：

$$\text{资产}_t = \text{负债}_t + \text{权益}_t, \tag{7}$$

$$\text{现金}_t = \text{现金}_{t-1} + \mathrm{CFO}_t + \mathrm{CFI}_t + \mathrm{CFF}_t, \tag{8}$$

$$\text{权益}_t = \text{权益}_{t-1} + \mathrm{NI}_t - \mathrm{Div}_t, \tag{9}$$

$$\mathrm{Interest}_t = f(\mathrm{Debt}_t, \mathrm{Cash}_t),\ \mathrm{Debt}_t, \mathrm{Cash}_t \text{ 依赖} \mathrm{NI}_t\text{（不动点迭代求解）}. \tag{10}$$

历史期 EBIT 采用式 (1) 的口径；经营费用（不含折旧摊销）为轧差项 毛利 − EBIT − D&A，从而历史期模型 EBIT 与 DCF 的 EBIT 同口径。

三表驱动的 DCF。 记 $\mathrm{drv}(\mathrm{TS}, s)$ 为从三表模型情景 s 导出的 DCF 驱动向量：收入增速 g_t、EBIT 利润率 m_t、税率 τ_t、折旧摊销率 δ_t、资本开支率 κ_t、营运资本占收入比 w_t。DCF 定义为

$$\mathrm{Rev}_t = \mathrm{Rev}_{t-1}(1+g_t), \quad \mathrm{EBIT}_t = m_t \mathrm{Rev}_t, \tag{11}$$

$$\mathrm{FCFF}_t = \mathrm{EBIT}_t(1-\tau_t) + \delta_t \mathrm{Rev}_t - \kappa_t \mathrm{Rev}_t - (w_t \mathrm{Rev}_t - w_{t-1}\mathrm{Rev}_{t-1}), \tag{12}$$

$$K_e = r_f + \beta \cdot \mathrm{ERP} + \pi_{\text{size}} + \pi_{\text{spec}} + \pi_{\text{country}}, \tag{13}$$

$$\mathrm{WACC}_s = E\% \cdot K_e + D\% \cdot k_d(1-t) + P\% \cdot K_p + \Delta_s, \tag{14}$$

企业价值按式 (3)（年中折现、终值按第 N 年末折现，永续增长法或退出倍数法）。股权价值桥为

$$\mathrm{EqV}_s = \mathrm{EV}_s + \underbrace{(\text{现金} + \text{非经营性资产} + \text{联营投资})}_{\text{加回}} - \underbrace{(\text{有息负债} + \text{租赁} + \text{少数股东权益} + \text{养老金} + \text{优先股} + \text{期权} + \text{其他类债务})}_{\text{扣减}}, \tag{15}$$

每股价值 $V_s = \mathrm{EqV}_s$/摊薄股本（元）。非经营性资产若以明细给出，每项须过证据谓词，由程序换算加总（`build_plan.resolve_non_operating_items`）。

敏感性、评级与反向 DCF。 敏感性矩阵 $S[i,j] = V_{\text{base}}(\mathrm{WACC} + \Delta_i,\ g + \Delta_j)$。评级为上行空间 $u = V_{\text{base}}/P_0 - 1$ 的阶梯函数

$$\mathrm{rating}(u) = \begin{cases} \text{买入} & u \geq 0.20 \\ \text{增持} & 0.05 \leq u < 0.20 \\ \text{中性} & -0.05 \leq u < 0.05 \\ \text{减持} & -0.20 \leq u < -0.05 \\ \text{卖出} & u < -0.20 \end{cases} \tag{16}$$

反向 DCF 对单个变量 $x \in \{g(\text{整体平移}),\ m(\text{整体平移}),\ \mathrm{WACC},\ g_\infty \text{ 或退出倍数}\}$ 求解

$$x^\star = \arg\min_{x \in \mathcal{X}^\star} |x - x_{\text{base}}|, \qquad \mathcal{X}^\star = \{x : V_{\text{base}}(x) = P_0\}, \tag{17}$$

即在所有使每股价值等于参考股价的取值中取离基准最近者；其余变量固定为基准情景。由于 $V(x)$ 对增速不一定单调（增长越快，营运资本与资本开支占用越多），实现先在区间上做 400 点网格扫描找全部变号区间，取离基准最近的一个再二分（精度 10^{-6} 元）；无解时报告网格上的最大可达每股价值。隐含值按“不高于基准 / 高于基准但在历史区间内 / 超出历史最好水平 / 无历史可比”分档。

可比公司。 对可比公司集合 $\mathcal{P}$ 与倍数 $\mu \in \{\mathrm{P/E}, \mathrm{EV/EBITDA}, \mathrm{EV/Sales}, \mathrm{P/B}\}$，分母非正时记为 NM 并剔除，按四分位距规则剔除离群值后取 P25、中位数、P75，乘以目标公司对应分母得到隐含价值区间，再经股权价值桥换算为每股价值。

正确性标准。 估值没有“真值”，其正确性是内部闭合与实现一致：

(V1) 三表勾稽：式 (7) ~ (9) 在全部预测期、全部情景成立；

(V2) DCF 与三表同口径：$\forall s, t$，DCF 使用的驱动向量与 $\mathrm{drv}(\mathrm{TS}, s)$ 逐项相等；

(V3) 股权价值桥闭合：式 (15) 各项逐项列示且加总等于股权价值；每股价值可由股权价值与股本复算；

(V4) 情景排序：$V_{\text{downside}} \le V_{\text{base}} \le V_{\text{upside}}$；敏感性方向：$\partial V/\partial \text{WACC} < 0$、$\partial V/\partial g_\infty > 0$（在矩阵上逐格检查）；

(V5) 评级一致：正文评级 = rating(u)；

(V6) 实现一致：在相同输入上，自研引擎与参照引擎的输出逐位相等（三表、情景表、业绩分析）或相对误差 $< 10^{-9}$（DCF）。

(V1)～(V5) 对应评分标准 L0-02、L0-03、L0-05 与 L1-03～L1-06；(V6) 由 golden 快照测试与参照对照测试守护。此外我们报告终值占企业价值比例与预测首年相对历史的衔接（毛利率与增速的跳变），作为合理性提示而非正确性标准。

3.7 任务三：研究报告纠错核查

输入与断言。 输入为一篇研报 $\mathscr{D}_R$（Markdown 或 PDF）与真值表 $\mathscr{T}$。断言是一个元组

$$\gamma = (m,\ k,\ \tau,\ r,\ x,\ u,\ \text{quote},\ \text{loc},\ \text{fc}),$$

其中 $\tau \in \{\text{level}, \text{yoy}, \text{ratio}, \text{pp_change}\}$ 为断言类型（水平值、同比、比率、百分点变动），r 为原文写法，x 为带方向的数值（“下降 5%”记为 -5），u 为单位，quote 为原文片段，loc 为位置（页、chunk、句序号），$\text{fc} \in \{0, 1\}$ 标记预测性表述（含“预计”“预测”“有望”“目标”或晚于报告期的年份）。断言集 $\mathscr{C}$ 由规则通道（中文数字模式与最长别名匹配）与模型通道（模型拆出、要求 quote 为原文连续片段且包含该数字）合并去重得到。

真值函数。 对断言类型 τ 定义真值 $T_\tau(m,k)$：

$$T_{\text{level}} = T_{\text{ratio}} = \mathscr{T}(m,k),$$
$$T_{\text{yoy}} = \Delta\%(\mathscr{T}(m,k), \mathscr{T}(m, \beta_{\text{YoY}}(k))), \qquad T_{\text{pp}} = \mathscr{T}(m,k) - \mathscr{T}(m, \beta_{\text{YoY}}(k)),$$

其中 T_{pp} 对应 `pp_change` 类型。断言的规范化数值为 $\hat{x} = x \cdot \phi(u)$（水平值）或 $\hat{x} = x$（其余）。

定义 3.10 (按展示精度的容差)**.** *断言 γ 的容差为*

$$\text{tol}(\gamma) = \begin{cases} \frac{1}{2}\,\text{ulp}(r) \cdot \phi(u) + 10^{-6}\,|T| & \tau = \textit{level} \\ \frac{1}{2}\,\text{ulp}(r) + 0.005 & \text{否则} \end{cases}$$

称 γ 与值 y 相符，记 $\gamma \approx y$，当且仅当 $|\hat{x} - y| \le \text{tol}(\gamma)$。

定义 3.11 (判定函数)**.** *判定集合 $\mathscr{V} =$ {正确，数值错误，单位错误，期间错配，口径错误，方向错误，无法核实}。判定函数 $\text{verdict}(\gamma, \mathscr{T})$ 按以下顺序取第一个成立的分支：*

1. *若 $fc = 1$ 或 $T_\tau(m,k)$ 无定义，判“无法核实”；*
2. *若 $\gamma \approx T_\tau(m,k)$，判“正确”；*
3. *若 $\tau = level$ 且存在替代单位 $u' \ne u$ 使 $|x\phi(u') - T| \le \frac{1}{2}\,\text{ulp}(r)\phi(u') + 10^{-6}|T|$，判“单位错误”；*
4. *若 $\tau \in \{yoy, pp_change\}$ 且 $-\hat{x}$ 与 T 相符，判“方向错误”；*
5. *若存在期间 $k' \ne k$ 使 $\gamma \approx T_\tau(m,k')$，判“期间错配”；*
6. *若存在同组指标 $m' \ne m$（m, m' 属于同一“易混口径组”）使 $\gamma \approx T_\tau(m',k)$，判“口径错误”；*

7. 否则判“数值错误”。

易混口径组为：{净利润，归母净利润，扣非归母净利润，利润总额，营业利润}、{营业收入，营业成本}、{毛利率，净利率}、{资产总计，负债合计，所有者权益合计，归母权益}、{折旧，摊销，折旧摊销}、{经营现金流，资本开支}。每个错误判定附真值、来源（派生值附公式与输入单元格）与修改建议。实现为 `verify.verify_claim`。

命题 3.2 (四舍五入安全)**.** *设真值为 T，研报作者把 T（换算到单位 u 后）按四舍五入保留 d 位小数写成 r，且 $fc = 0$、T 有定义。则* $\mathrm{verdict}(\gamma, \mathscr{T}) =$ 正确。

证明. 四舍五入保证 $|\,\mathrm{parse}(r) - T/\phi(u)| \le \frac{1}{2}10^{-d} = \frac{1}{2}\,\mathrm{ulp}(r)$，两边乘以 $\phi(u)$ 得 $|\hat{x} - T| \le \frac{1}{2}\,\mathrm{ulp}(r)\phi(u) \le \mathrm{tol}(\gamma)$，故分支 2 成立。对非水平值类型同理，额外的 0.005 只放宽容差。 □

命题 3.2 说明“把四舍五入判为错误”这一最常见的误报在定义上不可能发生。反方向的代价是：低于展示精度的真实错误（例如真值 58.624 亿元写成 58.62 亿元在任何意义上都不算错，但真值 58.626 写成 58.62 是“舍入方向错误”）被视为正确。我们认为这是合理的——研报读者无法从 58.62 分辨这两种情形。

误报控制的形式化。 判定函数的设计目标是在固定召回率下最小化误报。三条规则共同实现这一点：(1) 分支 1 把预测与缺真值的断言排除在“错误”之外；(2) 分支 3 ~ 6 的错误类型诊断要求*精确匹配*一个替代解释，找不到就退回“数值错误”，而不是猜测最可能的类型；(3) 口径诊断只在易混组内搜索，避免跨指标的巧合匹配（指标越多、期间越多，随机命中某个替代值的概率越高）。

内部一致性。 另定义研报自身的内部一致性：对评级、各情景每股价值、预测年度的公司整体收入 / 归母净利润 / 扣非归母净利润 / 净利润 / 每股收益 / 毛利率及其增速，若同一（指标、期间、情景）键在文中出现两个数值 a, b 且 $|a - b| > \frac{1}{2}\,\mathrm{ulp}(r_a) + \frac{1}{2}\,\mathrm{ulp}(r_b)$，记为一处矛盾。为保证“命中即确认”，引用外部来源的句子、条件句、敏感性与区间表述、分部口径的句子不进入比较（`consistency.py`）。

评价指标。 设标注错误集合 $\mathscr{L}$（每个标注有匹配片段与期望判定），系统判为错误的断言集合 $\widehat{\mathscr{L}}$。召回率 $= |\{\ell \in \mathscr{L} : \ell \text{ 被判为错误}\}|/|\mathscr{L}|$；精确率 $=$ 真阳性 $/|\widehat{\mathscr{L}}|$；误报数 $= |\widehat{\mathscr{L}}| -$ 真阳性；类型准确率 = 判出且类型正确的数目 / 判出数目（`eval_check`）。

3.8 系统性质

本节定义系统层面要保证的性质。它们刻画的不是某一个任务的正确性，而是“整个系统的输出可以被信任到什么程度”。

按展示精度的数值匹配。 对文稿中的一个数字 (r, u)（原文写法与紧随的单位），定义其候选换算因子集合 $\Phi(u) = \{10^{-8}\}$（$u =$ 亿元或亿）、$\{10^{-4}\}$（$u =$ 万元或万）、$\{1, 100\}$（$u =$ % 或个百分点，允许以小数存储的比例）、$\{1\}$（其余）。称 (r, u) *在值集 V 中有出处*，记 $(r, u) \models V$，当且仅当

$$\exists v \in V,\ \exists \varphi \in \Phi(u):\quad \big|\,|v| \cdot \varphi - |\,\mathrm{parse}(r)|\,\big| \le \tfrac{1}{2}\,\mathrm{ulp}(r)(1 + 10^{-9}) + 10^{-12}. \tag{18}$$

比较取绝对值：文稿写“下降 5%”时数字本身不带符号。年份、一位整数序号、页码（前有“第”、后有“页 / 年 / 月 / 日 / 节”）不参与检查。

定义 3.12 (结论性产物与值集)**.** *本次运行的结论性产物 $\mathscr{A}$ 是运行目录中以下文件的集合：财报分析（含真值表）、业绩分析、抽取结果、经营数据、DCF 结果、可比公司结*

果、非经营性资产明细、盈利预测摘要、分部预测、反向 *DCF*、估值输入假设（*WACC* 与终值参数）、纠错结论中的正确值。记 $\mathrm{val}(\mathcal{A})$ 为从这些文件中递归收集的全部数值（含字符串中“*58.62 亿元*”式写法换算后的值；以百万元计的块另按 $\times 10^6$ 换算）。研报原文中的断言数字、三表与敏感性等中间网格不属于 $\mathcal{A}$。外部来源值集 $\mathrm{val}(\mathcal{S})$ 为 $\mathcal{S}$ 中各条摘句所登记的数值。

把中间网格排除在 $\mathcal{A}$ 之外是有意的：网格中的数值数量巨大（三情景 × 多年 × 数十个科目，加上敏感性矩阵），值集越大，编造的数字“碰巧”落入某个容差窗口的概率越高（见 §3.10 的分析）。研报原文的断言数字被排除，是因为研报里的错误数字不能成为出处。

定义 3.13 (数字闭合)**.** 文稿 $\mathcal{N}$ 相对 $(\mathcal{A},\mathcal{S})$ 数字闭合，当且仅当 $\mathcal{N}$ 中每个参与检查的数字 (r,u) 满足 $(r,u)\models \mathrm{val}(\mathcal{A})\cup\mathrm{val}(\mathcal{S})$。

定义 3.14 (引用完备)**.** 文稿 $\mathcal{N}$ 相对 $\mathcal{S}$ 引用完备，当且仅当对每个句子 ς：*(i)* ς 引用的每个编号 `[Si]` 都在 $\mathcal{S}$ 中登记；*(ii)* 若 ς 中某个数字 $(r,u)\not\models \mathrm{val}(\mathcal{A})$ 但 $(r,u)\models\mathrm{val}(\mathcal{S})$（即只能在外部来源中找到出处），则 ς 至少引用了一个其值集包含该数字出处的来源编号。

定义 3.15 (绑定正确（部分）)**.** 把文稿当作一篇研报，用研报纠错的断言抽取与判定函数处理；记 $\mathscr{C}_{bind}\subseteq\mathscr{C}(\mathcal{N})$ 为口径明确的历史断言（排除预测、季度、跨年序列、分部、现金流明细等，规则见 `guard._binding_candidate`）。文稿绑定正确，当且仅当 $\forall\gamma\in\mathscr{C}_{bind}$，$\mathrm{verdict}(\gamma,\mathcal{T})\in\{\text{正确},\text{无法核实}\}$，或 γ 能按分部数据核对通过。

绑定正确是一个部分性质：它只覆盖断言抽取器能识别的表述。§1.2 中股权价值桥构成写错的案例就落在它的覆盖范围之外。

定义 3.16 (可追溯性)**.** 一个数字是可追溯的，当且仅当存在一条有限的出处链终止于原文或外部摘句：*(a)* 真值表的列示单元格追溯到 $(\mathit{source},\mathit{page},\mathit{chunk_id}[,\mathit{row},\mathit{col}])$，且证据谓词成立；*(b)* 派生单元格追溯到公式与输入单元格，输入单元格递归可追溯；*(c)* 估值产物中的数值追溯到其计算函数、输入单元格与假设文件中的具体字段；*(d)* 外部数值追溯到 $\mathcal{S}$ 中的一条来源（网址、发布与访问日期、摘句），且数值原样出现在摘句中；*(e)* 文稿中的数字通过式 (18) 追溯到 *(a)*∼*(d)* 中的某个值。系统可追溯，当且仅当 $\mathcal{T}$、$\mathrm{val}(\mathcal{A})$ 与 $\mathcal{N}$ 中的全部数字都可追溯。

定义 3.17 (出处类型)**.** 对 $\mathcal{N}$ 中有出处的数字，按顺序“报告披露 ≻ 外部来源 ≻ 程序计算 ≻ 估值假设 ≻ 模型预测”取第一个能找到出处的值集，定义其出处类型；报告披露与外部来源归为事实，程序计算归为推论，估值假设与模型预测归为观点。

先认事实、再认推论、最后认观点的顺序意味着：一个数若同时是披露值和某个预测值（巧合相等），按事实计。这一偏向使“观点”计数成为下界，适合用来提示“正文中至少有多少数字依赖我们的判断”。

定义 3.18 (数字锁)**.** 记 $\mathrm{tok}(\mathcal{N})$ 为文稿去掉注释与来源标记后提取的“数字 + 单位”记号多重集（去千分位，年份与一位序号除外），$\mathrm{src}(\mathcal{N})$ 为其引用的来源编号集合。改写 $\mathcal{N}^0\to\mathcal{N}$ 满足数字锁，当且仅当

$$\mathrm{set}(\mathrm{tok}(\mathcal{N}))\subseteq\mathrm{set}(\mathrm{tok}(\mathcal{N}^0))\quad\text{且}\quad\mathrm{src}(\mathcal{N})\subseteq\mathrm{src}(\mathcal{N}^0).\tag{19}$$

命题 3.3 (数字锁保持数字闭合)**.** 若 $\mathcal{N}^0$ 相对 $(\mathcal{A},\mathcal{S})$ 数字闭合，且 $\mathcal{N}^0\to\mathcal{N}$ 满足数字锁，则 $\mathcal{N}$ 相对 $(\mathcal{A},\mathcal{S})$ 数字闭合。

证明. 数字闭合只依赖每个数字记号 (r,u) 本身。$\mathcal{N}$ 中每个记号都以相同写法与单位出现在 $\mathcal{N}^0$ 中，而 $\mathcal{N}^0$ 中该记号有出处，故 $\mathcal{N}$ 中该记号也有出处。 □

注意数字锁不保持绑定正确与引用完备：改写可能把一个数字挪到另一个指标旁边，或把带 [s*i*] 的句子拆成两句使数字与编号分离。因此流水线要求改写后重跑完整的护栏（§10），数字锁只是第一道、最便宜的检查。

定义 3.19(可复现性)**.** 引擎运行是一个确定性函数 $E:(\mathcal{D},\{\mathcal{D}_i\},X,X_{extra},\theta,\mathcal{P},\mathcal{C},\mathcal{G},argv)\mapsto\mathcal{A}$。一次运行可复现，当且仅当其清单（*manifest*）记录了全部输入文件的 *SHA-256* 与命令行参数，且在任意环境中以哈希相同的输入重跑 E 得到逐字节（数值逐位）相同的 $\mathcal{A}$。

系统分三层讨论可复现性：引擎层满足定义 3.19（模型写下的抽取文件随代码提交，评审无需模型即可复算）；会话层只满足可审计——事件流完整记录每次工具调用与文件读写，但模型输出不能离线重放；质检层的计分可由保存的逐条判定重算，重新盲评需要模型。

3.9 单位与口径约定

表 9 汇总全系统的单位与口径约定。这些约定在各模块之间是强制一致的：跨模块传递的数值从不隐式换算单位，换算只发生在明确标注的边界（抽取入表、估值取数、护栏比对）。

表 9 单位与口径约定。

对象	约定	说明
财务事实（金额）	元	抽取时按原文单位换算：$v=\phi(u)\,\mathrm{parse}(r)$；原文写法 r 与单位 u 同时保留
每股收益	元 / 股	不换算
比率	百分数值	14.49 表示 14.49%；比率变动用百分点差
股本	股	万股、亿股按 ϕ 换算
估值模块	百万元	三表、DCF、可比公司；每股价值为元；产物中以 _mm 结尾的键与特定块按百万元计
DCF 与反向 DCF 中的比率	小数	0.08 表示 8%（与财务事实的百分数值不同，在边界处换算）
经营数据	按列单位保存	金额列须能换算为元；其余计量单位（吨、台、元 / 吨）原值保存；外币与少见量级拒收
报表口径	合并报表	母公司报表一律排除
营业收入	“营业收入”行	同表同时出现营业总收入时取营业收入
EBIT	式 (1)	利润总额 + 财务费用 − 投资收益 − 公允价值变动收益；资产处置、其他收益、减值、营业外收支留在 EBIT 内
非经营性资产	股权价值桥加回	被 EBIT 剔除的投资类资产在此加回；未设置时报告提示估值偏保守
归母净利润预测	三表净利润 × 最近实际年度归母占比	假设少数股东损益占比不变（简化）
同比基期为负	分母取绝对值	式 (5)

例 3.1 (单位边界上的一个错误类别)**.** 估值产物 *valuation.json* 中企业价值以百万元存为 *1748372.0*，正文写"*17,483.72* 亿元"。护栏若只按"亿元"换算 10^{-8}，会在值集中找 1.748372×10^{-2} 亿元而失败；因此 *guard* 对以百万元计的块额外生成 $\times 10^{6}$ 的元值。反过来，若把全部数值都同时按元与百万元两种解释放入值集，值集翻倍、巧合匹配概率上升。实现只对明确标注为百万元的块做双重解释，正是为了在两者之间取平衡。

3.10 威胁模型与系统不变量

3.10.1 把 LLM 当作不可信但有用的组件

我们借用安全工程中"威胁模型"的说法，但对象不是外部攻击者，而是系统内部的模型会话。模型被视为一个不可信但有用的组件：它的输出往往正确且有价值，但系统不默认信任任何一条输出，所有进入下游的内容都必须经过确定性检查。

能力假设。 在工作区内，一个模型会话可以：读取工作区内的任意文件（年报、技能、引擎源码、上游产物）；联网检索与抓取网页；在 `data/` 下创建与修改任意文件；以白名单方式调用引擎命令与编排器命令。它不能（由权限配置强制）：读取仓库所在目录及其上级目录；修改引擎（`finagent/`）、技能（`.claude/`）、年报原件、工作区元数据与 `CLAUDE.md`；调用白名单之外的命令。盲评会话只能读盲评目录、写判定文件，不能联网。

故障模型。 我们不假设模型是恶意的，但假设它的错误可以是任意的，并且错误在局部看起来合理（§1.2）。具体地，模型可能：(T1) 写出原文中不存在的数字；(T2) 写出存在但属于其他口径、期间或对象的数字；(T3) 写出心算的派生数；(T4) 写出无出处的外部数字或错误的来源编号；(T5) 在改写中改动数字；(T6) 在两通道冲突时选择其一并不报告；(T7) 跳过阶段、使用过期产物或声称完成了未完成的工作；(T8) 读取并迎合评分标准；(T9) 评审时受到过程信息或自身偏好的影响。

信任边界。 图 3 给出信任边界：橙色区域内的一切都是模型可写的，因而不可信；蓝色区域是只读的确定性程序与数据，是可信计算基（trusted computing base）。每条跨越边界、从橙色流向下游的数据，都必须经过一个红色检查点。

3.10.2 系统不变量

在上述故障模型下，系统保证以下不变量。每条不变量后给出强制执行的机制；第 4 节逐条论证。

不变量 1 (证据)**.** $\forall f \in \mathcal{T}$，若 $f.\mu \in \{llm, rule{+}llm\}$，则 $\mathrm{Ev}(\mathcal{D}_{f.source}, f.page, f.r) \neq \bot$ 且 f 取自合并口径。机制：定义 *3.5*，*verify-facts*。应对：*T1*，部分 *T2*。

不变量 2 (不静默取舍)**.** 若 $\mathrm{Conf} \neq \emptyset$ 或 $\mathrm{Rej} \neq \emptyset$ 或存在失败勾稽或核心缺失，则估值不运行。机制：流水线阻断与阶段 *2* 闸门。应对：*T2*、*T6*。

不变量 3 (单一真值)**.** 估值、纠错与护栏使用的全部历史财务数字都是 $\mathcal{T}$ 的单元格；下游不重新抽取或重算基础数字。机制：模块接口只接受 *truth_table*；代码评审与测试。应对：防止同一数在不同模块中取值不同。

不变量 4 (计算外置)**.** $\mathrm{val}(\mathcal{A})$ 中的每个派生数由引擎计算；模型不写入 $\mathcal{A}$。机制：$\mathcal{A}$ 只由引擎写入运行目录，运行目录由追踪器记录哈希；模型只能写 *data/* 下的输入文件。应对：*T3*。

不变量 5 (数字闭合与引用完备)**.** 进入排版阶段的文稿 $\mathcal{N}$ 相对 $(\mathcal{A}, \mathcal{S})$ 数字闭合（定义 *3.13*）、引用完备（定义 *3.14*）、绑定正确（定义 *3.15*），且无内部矛盾；外部登记的每

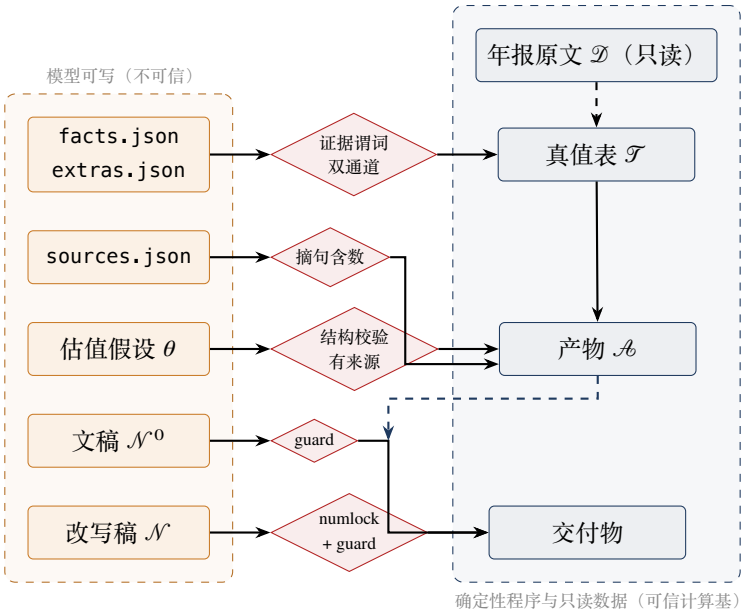


图 3　信任边界。模型可写的文件（左）只能经过程序检查点（中，红色）进入可信区域（右）；护栏检查文稿时以产物 $\mathcal{A}$ 为参照（蓝色虚线）。引擎、技能与年报原件对模型只读。

个数值原样出现在其摘句中。机制：`guard --sources`、`verify-sources`、阶段 *1*、*4*、*8* 闸门。应对：*T1*、*T3*、*T4*，部分 *T2*。

不变量 6 (改写不增数)**.** 任何由模型执行的改写满足数字锁（定义 *3.18*），且改写后重新满足不变量 *5*。机制：阶段 *9* 闸门、`refine` 中复核与修订会话。应对：*T5*。

不变量 7 (阶段单调与新鲜)**.** 第 i 阶段判为通过，当且仅当 $G_1, \ldots, G_i$ 均成立，且第 i 阶段的产物晚于其全部输入。机制：`status` 的前序阻断与修改时间比较。应对：*T7*。

不变量 8 (信息隔离)**.** 生成方会话可读的文件集合与评分标准、质检代码、评测结果、其他公司的产物不相交；评审会话可读的文件集合只含交付物与脱敏的评审条目。机制：仓库外工作区、权限配置、引擎复制时排除 `evaluation/`、盲评目录、问题单脱敏。应对：*T8*、*T9*。

不变量 9 (可审计)**.** 引擎的每次文件读取与产物写入都带 *SHA-256* 记录；每个会话的完整事件流落盘。机制：`tracing.read_file`、`write_artifact`、`.pipeline/agents/*.jsonl`。应对：事后追查任何一类故障。

3.10.3　不保证什么：残余风险

不变量刻画了系统能证明的东西，也就划定了它不能证明的东西。我们明确列出残余风险：

(R1) **语义错配**。证据谓词与数字闭合都是“存在性”判定：一个数在原文所写页出现、在产物中有匹配值，并不说明它被配给了正确的指标。规则通道比对与绑定核对覆盖了可识别的部分，聚合数构成描述、分部与整体混淆等仍依赖复核与盲评（负向条目 N-03）。

(R2) **巧合匹配**。编造的数字可能碰巧落入某个产物值的容差窗口。设文稿中一个两位小数、以亿元计的编造数字近似均匀地落在长度为 L（亿元）的区间内，值集中有 n 个落

在该区间、彼此间距大于窗口宽度的值，则巧合通过概率约为 $n \cdot 2\frac{1}{2}\,\mathrm{ulp}\,/L = n \cdot 0.01/L$。例如 $n = 300$、$L = 1000$ 亿元时约为 0.3%；把中间网格排除在 $\mathcal{A}$ 之外正是为了控制 n。对一位小数或整数写法，窗口变宽，风险相应上升。

(R3) **规则通道的盲区。**季报不进规则通道；股本变动表、政府补助口径、部分扣非季度别名规则通道取不到。这些键上只有单通道，冲突检测失效，只剩证据谓词。

(R4) **假设的合理性。**估值假设 θ 由模型起草，程序只检查结构、有无来源引用与闭合，不判断增速或 WACC 是否合理；合理性由盲评条目（预测假设有依据、情景与敏感性）与人工复核清单兜底。

(R5) **外部来源的真实性。**`verify-sources` 检查摘句含数与结构，不检查网页是否真实存在、摘句是否确实来自该网页（不在线复查）。模型可以在形式上伪造一条来源；我们依赖检索记录（事件流中的 WebSearch / WebFetch 调用）事后审计。

(R6) **评审误差。**盲评的稳定性与精确率是被度量而非被保证的（§1.7）；整节删除类缺陷的判出率尤其低。

权衡（Trade-off）：存在性判定 vs 语义判定

一个自然的问题是：为什么不让另一个模型核对“数字是否配给了正确的指标”，从而覆盖 R1？我们的回答是分层：能确定判定的部分（存在性、精度、绑定中可识别的部分、内部矛盾）一律由程序做，因为它没有判定误差、可复现、可单元测试；只能由语义理解判定的部分交给与写作者隔离的复核编辑与盲评，并通过校准度量其误差。把语义核对也交给模型并作为硬闸门，会把一个有误差的判定放进阻断路径，导致流水线在误判时卡死或在漏判时给出虚假的安全感。

3.11 研报生成任务的形式化

工作区与角色。 工作区状态 $\mathcal{W}$ 是一个文件系统快照：路径到（内容，修改时间）的映射。角色集合 $\mathcal{Q} = \{\text{研究助理}, \text{分析师}, \text{主笔}, \text{复核编辑}, \text{排版设计师}\}$，每个角色 q 负责连续的一段阶段 $I_q \subseteq \{1, \ldots, 11\}$：研究助理 $\{1\}$、分析师 $\{2, \ldots, 7\}$、主笔 $\{8\}$、复核编辑 $\{9\}$、排版设计师 $\{10, 11\}$。一次会话是一个（非确定性的）状态转移 $\mathcal{W} \xrightarrow{q} \mathcal{W}'$，其写集合受权限限制：$\mathrm{dom}(\mathcal{W}') \setminus \mathrm{dom}(\mathcal{W})$ 与被修改的文件都在 `data/` 与运行目录下。会话之间没有共享状态，除 $\mathcal{W}$ 之外不传递任何信息。

阶段与闸门。 第 i 阶段有读集合 In_i、产物集合 Out_i 与闸门谓词 $G_i : \mathcal{W} \to \{0, 1\}$（表 10）。闸门由三类原子条件组合而成：*存在性*（文件存在）、*校验性*（引擎命令在该文件上返回通过，如证据校验、护栏、数字锁）、*数量下限*（检索次数、专题数、字数、页数、图表数）。另有*新鲜性*条件：$\mathrm{mtime}(\mathrm{Out}_i) \geq \max \mathrm{mtime}(\mathrm{In}_i)$，例如阶段 2 要求校验结果晚于抽取文件，阶段 7 要求估值运行晚于抽取、经营数据、假设与可比公司表，阶段 10 要求 PDF 晚于文稿与排版脚本。

定义 3.20 (阶段状态)**.** *第 i 阶段的状态为 $\mathrm{ok}_i(\mathcal{W}) = G_i(\mathcal{W}) \wedge \mathrm{ok}_{i-1}(\mathcal{W})$，$\mathrm{ok}_0 \equiv 1$。工作区完成当且仅当 $\mathrm{ok}_{11}(\mathcal{W}) = 1$。下一个待办阶段为 $\min\{i : \mathrm{ok}_i = 0\}$。*

前序阻断使 ok 关于阶段号单调不增：一个阶段即便自身文件齐全，只要前序阶段未过（例如上游抽取被修改后重新校验失败），也判为未通过。这避免了“拿过期产物当成果”——不变量 7。

运行过程。 `run` 按角色顺序执行：对角色 q，若 $\mathrm{ok}_{\max I_q}(\mathcal{W}) = 1$ 则跳过，否则启动一次会话；会话结束后重新计算阶段状态，若 $\mathrm{ok}_{\max I_q} = 0$ 则停止并把卡住的闸门与下一步建议交给人。于是 `run` 是一个确定性的外层循环包着若干非确定性的会话：

表 10 11 道闸门的形式化摘要（阈值取自 `report_pipeline.py`；完整定义见附录 D）。

i	阶段	$G_i(\mathscr{W}) = 1$ 当且仅当
1	资料搜集	来源登记与资料包存在；七个类别全覆盖；资料搜集检索 ≥ 20 次；近 90 天发布的来源 ≥ 3 个；`verify-sources` 通过
2	财务抽取	校验结果晚于抽取文件，且 $\mathrm{Pass}_{\mathrm{ext}} = 1$
3	分析框架	框架文件含“外部调研清单”节与 ≥ 3 个深度专题
4	深度调研	专题笔记 ≥ 3 份，每份 ≥ 1500 字、≥ 3 个独立来源；补充检索问题 ≥ 8 个；来源登记晚于框架；`verify-sources` 通过
5	经营数据	经营数据逐格回查拒收 0 格
6	估值假设	有参考股价及其来源；依据中引用了 `[S 编号]`
7	估值运行	存在成功的估值运行，晚于全部输入，且带上经营数据与可比公司表
8	文稿	对当前文稿、以当前运行目录与来源登记跑过护栏且通过；正文 ≥ 15,000 字，深度研究占比 ≥ 60%
9	复核改写	初稿备份、修改记录、复核清单存在；数字锁通过
10	排版	PDF 晚于文稿与脚本；25 ~ 32 页；图表 ≥ 60；已命名且与主文件一致；渲染检查凭证有效
11	交付说明	交付说明存在

交付物。 完成的工作区产出交付物集合 $\mathscr{D} = \{$ 研报 PDF，估值工作簿，$\mathscr{N}$，$\mathscr{A}$，$\mathscr{S}$，复核清单，交付说明 $\}$。研报 PDF 中的每个图表由排版脚本从 $\mathscr{A}$ 读数绘制，正文取自 $\mathscr{N}$；因此 PDF 中的数字继承 $\mathscr{A}$ 与 $\mathscr{N}$ 的可追溯性。排版脚本本身由模型编写（属于不可信区），其“只从产物读数”是写作规范而非程序保证——这是残余风险 R1 的另一种表现，由渲染检查与盲评部分覆盖。

3.12 评价问题的形式化

评分标准。 评分标准是条目集合 $\mathscr{R} = \{\rho_1, \ldots, \rho_n\}$，每个条目 $\rho = (\mathrm{id}, \ell, j, w, \mathrm{cap}, \mathrm{crit}, \mathrm{route})$，其中层级 $\ell \in \{$L0，L1，L2，L3，N$\}$，判定者 $j \in \{\mathrm{program}, \mathrm{llm}\}$，分值 w（L0 为 0，N 为负），可选封顶 cap，判定标准与反例文本，问题单路由。v2 评分标准有 L0 5 条、L1 13 条（合计 35 分）、L2 12 条（45 分）、L3 7 条（20 分）、N 4 条（$-8, -8, -6, -8$）。

判定。 程序条目的判定 $\mathrm{res}(\rho) \in \{0, 1\}$ 由确定性检查给出。模型条目由评审会话给出，评审的信息集为

$$\mathscr{T}_{\mathrm{judge}} = (\mathscr{D}_{\mathrm{blind}},\ \{(\mathrm{id}, \mathrm{criterion}, \mathrm{fail_examples}) : \rho \in \mathscr{R}, j(\rho) = \mathrm{llm}\}), \tag{20}$$

即只含交付物副本与条目的编号、判定标准和反例——不含分值、层级权重、写作过程、生成模型名。评审对每个条目输出 $(\mathrm{passed}, \mathrm{evidence}, \mathrm{reason})$，规则为“拿不准判不通过”。负向条目的 passed = 1 表示“命中”。

计分。 记 $\mathrm{res}(\rho) \in \{0, 1\}$，缺判定按 0 计。层得分 $S_\ell = \sum_{\rho:\ell(\rho)=\ell} w_\rho \,\mathrm{res}(\rho)$，满分 $M_\ell = \sum_{\rho:\ell(\rho)=\ell} w_\rho$（$\ell \neq$ N）。扣分 $D = \max\left(\sum_{\rho\in\mathrm{N}} w_\rho \,\mathrm{res}(\rho),\ D_{\min}\right)$，$D_{\min} = -20$。总分

$$\mathrm{Total} = \min\Big(\max\left(0,\ S_{\mathrm{L1}} + S_{\mathrm{L2}} + S_{\mathrm{L3}} + D\right),\ \min C\Big), \tag{21}$$

其中封顶集合 C 包含：若任一 L0 条目未通过，则含 40；若命中带封顶的负向条目（无来源外部数据封顶 70），则含其封顶值；C 为空时 $\min C = +\infty$。

算法 1 研报生成的外层编排（`report_pipeline.run_agent` 的抽象）

function RUN($\mathcal{W}$)
  for q **in** (研究助理, 分析师, 主笔, 复核编辑, 排版设计师) **do**
    if $\mathrm{ok}_{\max I_q}(\mathcal{W}) = 1$ **then**
      continue ▷ 已通过的阶段跳过，可断点续跑
    end if
    $\mathcal{W} \leftarrow \textsc{Session}(q, \mathrm{model}(q), \mathrm{prompt}(q), \mathrm{allowlist})(\mathcal{W})$ ▷ 独立无头会话
    if $\mathrm{ok}_{\max I_q}(\mathcal{W}) = 0$ **then**
      return (未完成, $\min\{i : \mathrm{ok}_i = 0\}$)
    end if
  end for
  return (完成, $\perp$)
end function

定义 3.21 (交付标准)**.** *研报可交付，当且仅当 L0 全部通过、Total* ≥ 75、$S_{L1}/M_{L1} \geq 0.8$、$S_{L2}/M_{L2} \geq 0.6$、$S_{L3}/M_{L3} \geq 0.6$，*且未命中任何负向条目。*

计分与交付判定完全由程序完成（`evaluation.rubric.score`）；改权重或门槛后可用保存的逐条判定重算，不必重跑评审。

闭环。 设第 n 轮的评测结果为 $(\mathrm{Total}_n, \mathrm{deliverable}_n, \mathcal{J}_n)$，$\mathcal{J}_n$ 为问题单。问题单只保留每个未通过条目的理由、证据与路由，不含条目原文与分值。路由函数把问题分到三类：数据与假设类 → 修订会话（之后删除初稿备份，复核与排版从头再来）；措辞与结构类 → 复核编辑；版面类（阶段 10）→ 排版设计师。停止规则为

$$\mathrm{stop}_n \iff \mathrm{deliverable}_n \vee n \geq 3 \vee (\mathrm{Total}_n - \mathrm{Total}_{n-1} < 2 \wedge \mathrm{Total}_{n-1} - \mathrm{Total}_{n-2} < 2) \vee \mathcal{J}_n^{\mathrm{route}} = \emptyset,$$

结束时回滚到 $n^\star = \arg\max_n \mathrm{Total}_n$ 的快照；若该轮仍不可交付，把剩余问题追加进复核清单交人工。另外，若某轮开始时闸门未全过（$\mathrm{ok}_{11} = 0$），不做盲评而直接停止——交付物不全时盲评没有意义。

评审校准。 评审是一个随机判定器 $J : \mathcal{T}_{\mathrm{judge}} \to \{0,1\}^{|\mathcal{R}_{\mathrm{llm}}|}$。我们用三组量刻画它：

- **稳定性**：对同一份交付物独立评审 K 次，得到判定向量 $\mathrm{y}^{(1)}, \ldots, \mathrm{y}^{(K)}$。逐条全一致率 $= \frac{1}{|\mathcal{R}_{\mathrm{llm}}|} \sum_\rho \mathbb{1}[y_\rho^{(1)} = \cdots = y_\rho^{(K)}]$；两两不一致率 $= \binom{K}{2}^{-1} \sum_{a<b} \frac{1}{|\mathcal{R}_{\mathrm{llm}}|} \sum_\rho \mathbb{1}[y_\rho^{(a)} \neq y_\rho^{(b)}]$；总分极差 $= \max_a \mathrm{Total}^{(a)} - \min_a \mathrm{Total}^{(a)}$。
- **劣化判出率**：劣化算子 δ 把交付物 $\mathcal{D}$ 变成 $\delta(\mathcal{D})$，制造一个已知缺陷，其目标条目集合为 $\mathcal{R}_\delta$。若原稿已被多数评审判出该缺陷，则该变体*无效*（命中不说明灵敏）；对有效变体，判出 $\iff$ 评审在 $\delta(\mathcal{D})$ 上对 $\mathcal{R}_\delta$ 中某条给出“有缺陷”的判定。连带改判 = 非目标条目中与原稿多数判定不同的条目数，应与稳定性中的两两不一致率相当，否则说明劣化干扰了无关判断。
- **精确率**：人工复核评审判“不通过”（或负向“命中”）的条目，精确率 = 人工确认数 /（确认数 + 误报数），待查不计。

这三组量分别对应评审的方差、灵敏度（召回的代理）与特异度（精确率）。由于人工只复核评审判出的问题，我们没有评审“漏判”的直接估计，劣化判出率是它的代理——这一限制在第 12 节讨论。

3.13 小结

本章把 LEDGER 要解决的问题分解为三层。**任务层**：财报分析、估值建模与纠错核查分别是“从文档构造带出处的真值表并派生分析”、“在真值表与假设上构造内部闭合的估值模型”和“在真值表上按展示精度判定研报断言”，各有明确的输入、输出与正确性标准。**性质层**：数字闭合、引用完备、绑定正确（部分）、可追溯、出处类型、数字锁与可复现，刻画了“系统输出可以被信任到什么程度”；其中命题 3.2 与命题 3.3 给出了误报控制与改写安全的简单保证。**过程层**：在把模型视为不可信但有用的组件的威胁模型下，九条不变量通过证据谓词、阻断、单一真值表、计算外置、护栏、数字锁、单调闸门、信息隔离与追踪来强制执行；残余风险（语义错配、巧合匹配、规则通道盲区、假设合理性、来源真实性、评审误差）被明确列出，由复核编辑、盲评与人工复核清单兜底。后续各章按这一框架展开实现与实验。

4 设计原则与系统不变量

LEDGER 的代码量不大（引擎约一万两千行 Python），但它在几个方面比通常的“LLM + 工具”系统严格：模型抄下来的数字要在原文里逐字找到，文稿里的每个数字要在本次运行的产物里按展示精度找到，两条抽取通道不一致时整条估值链停下，复核编辑改写时数字只许减少。这些约束不是零散的工程习惯，而是十一条写进仓库开发指南（`CLAUDE.md`“设计规范”一节）的原则；所有模块、skill 和测试都围绕它们组织。

本章把这十一条原则逐条形式化为*系统不变量*（system invariant），即在任意一次合法运行结束时对产物集合 $\mathcal{A}$、真值表 $\mathcal{T}$、文稿 $\mathcal{N}$、外部来源登记 $\mathcal{S}$ 成立的谓词，并给出：(i) 维持它的执行机制，具体到函数；(ii) 守护它的测试，均为 `tests/` 下真实存在的测试名；(iii) 违反时系统的行为；(iv) 设计动机，即开发过程中遇到的真实问题；(v) 考虑过但放弃的替代方案。第 4.14 节讨论原则之间的张力和优先级，第 4.15 节给出原则—机制—测试总表。后续各章（抽取 §6、估值 §8、纠错 §9、护栏 §10、多 Agent 流水线 §11）讲每个机制的细节，本章只讲“为什么必须如此”和“如何保证如此”。

4.1 原则的分类与执行强度

十一条原则针对的失败模式各不相同。按它们保护的对象，可以分成四组（图 4）：

- **数值正确性**：P1 LLM 不做算术、P4 一张真值表、P6 单位约定。保护“数是对的”。
- **可追溯性**：P2 每个数字可追溯、P5 所有读写走 tracing。保护“数从哪来、过程是什么”可以事后查验。
- **冲突与误判处理**：P3 双通道不静默取舍、P7 误报控制优先于召回。规定系统拿不准时怎么办。
- **生成约束与组织**：P8 事实 → 推论 → 观点、P9 模型输出加护栏、P10 会话隔离、P11 不预设行业框架、文风不做程序拦截。规定模型写东西时的边界，以及多个模型会话之间如何分工。

执行强度。 同样叫“原则”，被执行的方式差别很大。我们区分四个强度等级，后文每条原则都标注它的等级：

定义 4.1 (执行强度)**.** *设 ϕ 为一条不变量。*

E1 **构造性**（*by construction*）：*违反 ϕ 的状态在代码里不可表达，例如下游函数的签名只接受真值表，根本拿不到原始文档。*

E2 **拦截性**（*gated*）：*违反 ϕ 可以发生，但会被某个确定性检查发现，检查返回非零退出码或抛出异常，阻断后续步骤或闸门。*

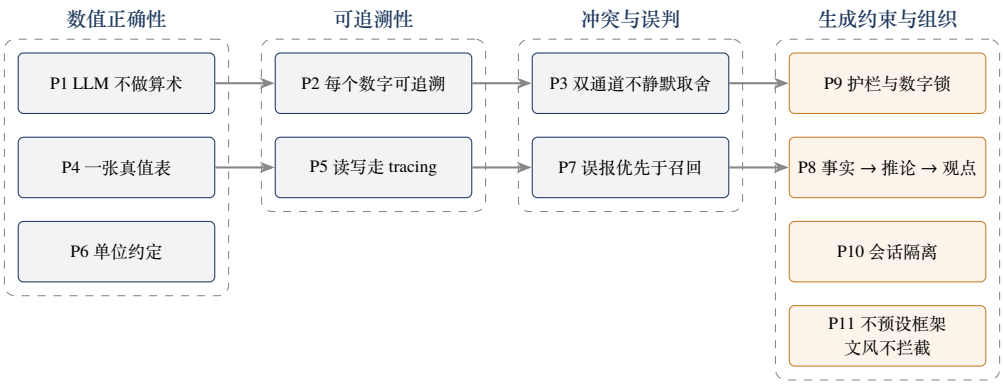


图 4　十一条设计原则的分组。蓝色组主要由确定性程序执行，橙色组约束模型会话的行为。箭头表示“前者是后者成立的前提”：例如 P9 的护栏只有在 P1（数字全部来自程序产物）与 P2（产物数字带出处）成立时才有意义；P7 的容差要以 P4 的单一真值为比较对象。

E3 **检测性**（*tested*）：运行时不检查，但有回归测试保证代码路径满足 ϕ；对代码修改有效，对模型行为无效。

E4 **约定性**（*by convention*）：只写在 *skill* 或开发指南里，由模型会话或人工评审遵守，没有程序检查。

强度越高，对模型越不信任。LEDGER 的总体倾向是：凡是能做成 E1/E2 的，就不留在 E4；做不成的（例如“不心算约数”“不套行业模板”）如实标为 E4，并在第 4.14 节讨论它们的残余风险。这一点与常见的“在系统提示里写清楚要求”的做法不同：提示词是 E4，模型在长任务里会遗忘或绕过它，我们在开发中多次观察到（见各节失败案例）。

记号。 沿用第 3 节的记号。另设：对一段文本 x，$\mathrm{Tok}(x)$ 为其中经正则 `NUM_UNIT_RE` 识别出的“数字原文 r + 单位 u”记号集合（年份、一位整数序号、页码除外）；对一次运行目录，$\mathsf{V}(\mathcal{A})$ 为结论性产物中全部数值换算到基本单位后的集合；$\mathcal{I}$ 为抽取与分析阶段的 issue 集合，$\mathcal{R}$ 为拒收集合。

4.2　P1　LLM 不做算术

不变量 10 (无模型算术)**.** *设 $\mathcal{N}$ 为模型写下的文稿，$\mathcal{S}$ 为外部来源登记。对任意 $(r,u) \in \mathrm{Tok}(\mathcal{N})$，存在 $v \in \mathsf{V}(\mathcal{A}) \cup \mathsf{V}(\mathcal{S})$ 与单位换算因子 $s \in S(u)$，使*

$$\big|\,|v| \cdot s - |r|\,\big| \;\le\; \tfrac{1}{2}\,\mathrm{ulp}(r)\,(1 + 10^{-9}) + 10^{-12}, \tag{22}$$

其中 $\frac{1}{2}\,\mathrm{ulp}(r) = \frac{1}{2} \cdot 10^{-d(r)}$，$d(r)$ 为 r 的小数位数；$S(\text{亿元}) = \{10^{-8}\}$、$S(\text{万元}) = \{10^{-4}\}$、$S(\%) = S(\text{个百分点}) = \{1, 100\}$，其余为 $\{1\}$。并且 $\mathsf{V}(\mathcal{A})$ 中的派生数值全部由确定性代码计算。

式 (22) 就是 `output/guard.py` 中 `_found` 的判定条件。它把“模型不做算术”转写为一个可检查的命题：模型写下的数，要么是从原文抄来的（进入真值表时已过 P2 的证据校验），要么是程序算好写进产物的，要么登记在外部来源里；模型自己算出来的数，只要它与程序结果在展示精度上不一致，就找不到出处。

执行机制（E2 + E1）。

- **派生指标由引擎计算并登记公式**：`analyze.FactTable.add_derived` 为每个派生值（毛利率、同比、单季拆分 Q4 = 全年 − 前三季度等）写下 `formula` 与 `derived_from`，`method="derived"`。

- **经营数据的比率由程序算**：分析师在 `extras.json` 里只写比率的定义（分子、分母引用哪一格），`extras.verify_extras` 内的 `_ratios` 计算数值；skill 明确要求“增速、占比、差值、单价这类派生数不要自己算”，连“约三倍”“超过四成”这类心算约数也不许。
- **估值全链在程序内**：分部量价预测（`segment_forecast`）、三表、DCF、反向 DCF、可比公司、评级（`forecast_summary.rating`）都是确定性函数。
- **出口检查**：`guard` 命令（`output/sources.run_guard`）对文稿逐数执行式 (22)，找不到出处即返回 1；阶段 8 的闸门要求 guard 通过（见 §11.5）。
- **界面也不重算**：终端界面的渲染模块 `finagent_tui/render.py` 在模块说明中写明“只做展示，不做任何计算”，数值全部来自引擎产物。

守护测试。 `test_extras_verified_and_ratios_computed`（比率由程序算出并与原文核对）、`test_guard_flags_invented_number`（编造的数字被 guard 拦下）、`test_guard_passes_correct_commentary`（正确的解读放行）、`test_guard_accepts_reverse_dcf_numbers`（反向 DCF 产物可作出处）、`test_guard_valuation_mm_aliases_and_bridge`（估值产物的百万元数值换算后可作出处）。

违反时的后果。 guard 返回 `unverified_numbers` 列表与非零退出码；主笔会话只能回到产物抄写展示值，或让程序补算（写进 `extras.json` 的 `ratios` 后重跑）。

失败案例：心算取整

skill 易错点清单记录的一个真实案例：主笔会话把产物里的 Q2 环比 −23.3445% 心算取整写成 23.35%（应为 23.34%，且丢了符号的语义）。guard 按两位小数的展示精度在产物里找不到 23.35，拦下了这句话。它说明的问题是：即使是“抄写 + 四舍五入”这样最轻的算术，模型也会出错，而且错误恰好落在人眼难以察觉的末位。

设计动机。 LEDGER 最早的原型（仓库首个提交）就把算术放在程序里，原因来自数值推理文献的共识：大模型在多步算术上的错误率远高于在“读懂题意、写出公式”上的错误率，把计算外包给解释器能显著提高正确率 [8, 20]。金融研报的特点放大了这一点：一份 25 ~ 32 页的深度报告有数百个数字，其中大量是增速、占比、单季拆分这类派生数；任何一个算错都会在复核时损害全文可信度，而逐个人工复算的成本又极高。

权衡（Trade-off）：让模型算、程序复核，还是完全不让模型算

备选 A：模型自由计算，程序事后复核。这是“工具增强”系统的常见做法：模型可以在文稿里写自己算的数，程序抽样或全量复算。问题在于复核需要知道模型用的是哪个公式、哪两个输入。文稿里的“毛利率提升 1.2 个百分点”既可能是本期减上期，也可能是单季对比，复核程序要先猜公式，猜错就会把对的判成错的（误报），或把错的放过。要让复核可靠，就得要求模型声明公式与输入——而一旦公式与输入都声明了，直接让程序计算更简单。

备选 B：模型调用计算工具（如 Python 解释器），把结果写进文稿。这比 A 好，但计算过程留在会话日志里，不进入运行产物，guard 无法区分“调用工具算的”和“心算的”；而且同一个派生数在不同会话里可能被算出不同的舍入版本。

选择：完全不让模型算（P1）。派生数一律以“定义”的形式交给引擎（`extras.json` 的 `ratios`、假设文件的驱动参数），结果进入产物，文稿只抄产物的展示值。代价是写作会话要多走一步“先登记、再运行、再抄写”；连“约三倍”这类口语化约数也得先让程序算出比值。这在 v1 研报里导致部分段落读起来更“数字化”，后由复核编辑（P11）处理措辞。

残余风险。 式 (22) 只能检查以阿拉伯数字出现的数。用汉字写的约数（“近三成”“翻一番”）不经过 `NUM_UNIT_RE`，对这类表述 P1 的执行强度降为 E4，由 skill 约束与复核编辑把关。另一个风险是*巧合放行*：模型心算出的数恰好等于产物中某个无关数值，guard

只检查“有出处”，不检查“出处对应的是同一个指标”，后者由 P9 的指标—期间绑定检查（`check_binding`）部分补上，见 §4.14。

4.3 P2 每个数字可追溯

不变量 11 (证据定位)**.** 对真值表中每条非派生事实 $f = (m, k, v, u, \sigma) \in \mathcal{T}$，$\sigma = (source, page, chunk_id, \ldots)$ 指向文档 $\mathcal{D}$ 的一个 *chunk* c，且

$$c.page = \sigma.page \quad \wedge \quad \pi(f.raw) \text{ 在 } \mathrm{norm}(c.text) \text{ 中按数字边界匹配成功}, \tag{23}$$

其中 $\pi(r)$ 为去掉千分位、括号、百分号与负号后的数字核，匹配模式为 `(?<![\d.])`$\pi(r)$`(?![\d])`。对派生事实，$\sigma.derived_from$ 列出全部输入事实的定位，递归地满足本不变量。对外部来源登记 $\mathcal{S}$ 中的每条事实 (q, v, u)（q 为原文摘句），v 的某种书写形式（整数、去尾零小数、千分位）原样出现在 q 中。

执行机制（E2）。

- `tasks/financials/external.find_evidence(doc, page, raw)`：只在模型声明的那一页的 chunk 里找，表格 chunk 优先；找不到返回 `None`，`_to_fact` 生成拒收理由“未在第 p 页原文中找到”。
- `_norm` 只去掉千分位逗号、保留空白。早期实现删除全部空白，导致表格相邻两格“235 3884.59”被拼成“2353884.59”，两个本来正确的数都被拒收；修复后由 `test_adjacent_table_numbers_keep_their_boundaries` 守护。
- `_to_fact` 同时检查指标合法、期间格式（`2024`、`2024H1`、`2024Q3`、`2024Q1-3`）、合并口径（`scope` 为母公司报表直接拒收）与单位合法。
- 研报断言（`claims.json`）：`claims.ingest_claims` 要求 `quote` 是研报原文的连续片段且包含该数值，否则拒收。
- 外部来源：`output/sources.verify_sources` 要求每条来源有网址、发布方、访问日期，每条事实有不超过 160 字的摘句，且数值出现在摘句中；guard 的 `check_citations` 再要求“只能从外部来源找到出处的数字”所在句子带对应的 `[S 编号]`。

守护测试。 `test_fabricated_number_rejected`、`test_wrong_page_rejected`、`test_partial_number_not_accepted_as_evidence`（“1.23”不能在“11.23”中匹配）、`test_parent_company_scope_rejected`、`test_claims_fake_quote_rejected`、`test_skill_run_truth_table_and_provenance`、`test_verify_sources`、`test_external_numbers_need_citation`、`test_continuation_inherits_header_and_keeps_its_own_page`（跨页续表沿用首页表头，但每行页码仍指向实际所在页）。

违反时的后果。 被拒收的条目进入 `rejected`，`verify-facts` 返回 1；流水线的 `analyze` 步骤在带模型抽取文件时检查 $\mathcal{R}_6 \neq \emptyset$ 并抛出异常，估值不会开始（与 P3 共用同一个阻断点，见式 (24)）。

为什么是“页 + 数字边界”，而不是更细的定位。 证据校验只确认“这个数在这一页出现过”，不确认“它在哪一行哪一列”。这是有意的分层：页级定位对模型友好（模型读 PDF 时天然以页为单位），而且足以拦住凭记忆编造和错页；行级错误（数字确在该页，但抄成了相邻行）交给 P3 的双通道比对与勾稽校验。`test_wrong_row_number_caught_as_conflict` 正是这个分层的回归测试：它把营业收入改成同页出现的“1.23”（“增加 1.23 个百分点”中的数），证据校验放行，随后被“同一指标多处取值不一致”与“规则与 LLM 抽取不一致”两个 issue 拦下。

权衡（Trade-off）：证据粒度与匹配方式

要求模型给出单元格坐标或包围盒。最精确，但模型读 PDF 截图时给出的坐标不稳定，大量正确抽取会因坐标偏差被拒；而且 `pdfplumber` 的表格切分本身在真实年报上就会错列（宁德时代年报的稀疏网格）。模糊匹配（编辑距离、数值容差）。能容忍 OCR 误差，但会让"5,862.34"匹配"5,862.43"，等于放弃了"逐字可定位"的意义。外部来源只要求网址。模型可以登记一个真实网址，却把记忆里的数字挂在它名下；要求摘句包含数字后，这种"挂靠"需要模型同时伪造摘句，成本高且在人工抽查时一眼可见。选择：页级 + 精确数字边界匹配 + 摘句含数；扫描页直接拒收，不做 OCR（OCR 的识别错误会以"有证据"的身份进入真值表，比拒收更危险）。

4.4 P3 双通道不静默取舍

LEDGER 的财务抽取有两条通道：规则通道（`extract.extract_rule`，基于表头期间识别与科目别名的确定性解析）与模型通道（分析师会话读 PDF 写下的 `facts.json`）。两者独立产生 $(m,k) \mapsto v$ 的映射。

不变量 12 (冲突不被吞掉)**.** 设 F_R、F_L 分别为规则通道与模型通道的事实集合，$\mathrm{close}(a,b) \equiv |a-b| \le \max(10^{-6}\max(|a|,|b|), 0.01)$。对任意键 (m,k)，若 F_R 与 F_L 都有取值且 $\neg\,\mathrm{close}(v_R, v_L)$，则 $\mathcal{I}$ 中存在类型为"规则与 *LLM* 抽取不一致"的 *issue*；同一通道内同一键的多处取值不一致时，存在类型为"同一指标多处取值不一致"的 *issue*。并且

$$\text{Block} \Longleftarrow \mathcal{R} \neq \emptyset \ \vee\ \exists\, i \in \mathcal{I} : i.type \in \{\text{冲突类}\} \ \vee\ \exists\, \text{勾稽失败} \ \vee\ \exists\, \text{核心指标本期缺失}, \tag{24}$$

其中 Block 表示估值流水线不得开始。

执行机制（E2）。 `extract.merge(rule_facts, llm_facts, prefer)` 合并两通道：一致时记 `method="rule+llm"` 并把另一方的来源加入 `corroborated_by`；不一致时无论保留哪一方都记 issue。流水线以 `prefer="llm"` 调用，即暂时保留模型读原文的结果用于展示，但阻断照常生效。阻断点有两处，判定逻辑相同：命令行 `verify-facts`（返回 1，并写出 `facts.verified.json` 供闸门检查）与流水线 `pipelines._p_analyze`（带 `--facts` 时检查式 (24)，抛出 `ValueError`）。核心指标清单 `external.CORE_METRICS` 共 14 项（营业收入、营业成本、归母净利润、扣非归母净利润、总资产、经营现金流、资本开支、基本每股收益等），本期缺失阻断、上期缺失只警示。

为什么保留模型值而不是规则值。 在真实年报上，规则通道的主要失效形式是"取不到"（稀疏网格错列、跨页续表、科目折行），而不是"取错"：茅台、宁德、招商三家 301 个核验值中，规则通道取到且一致的有 280 个，冲突 0。模型读页面时能看到版式，抄错的形式多为错行、错期间。二者冲突时，"谁对"无法事先判定，因此保留哪一方只影响展示，不影响阻断；真正的裁决由分析师会话回到原文完成。

不对称：缺失不是冲突。 当规则通道没有取到某个键时，模型值直接进入真值表（`confidence="medium"`），不视为冲突。否则在规则通道覆盖率低的文档（如招股说明书、季报）上，系统将完全无法运行。这意味着对这些键 P3 退化为 P2 单通道保护。季报就是典型：规则通道不识别"本报告期 / 年初至报告期末"表头，所以 `--supplement` 的条目只走模型通道，规则通道不参与比对（`test_supplement_requires_skill_facts`）。

守护测试。 `test_model_extraction_run_matches_rule_run`（样例上两通道产出逐项一致）、`test_wrong_row_number_caught_as_conflict`、`test_skill_pipeline_stops_before_valuation_on_conflict`、`test_swapped_period_is_flagged_and_model_value_kept`（期间看反被记为冲突且保

留模型值）、`test_missing_core_metric_blocks`、`test_verify_cli_exit_codes`、`test_pdf_model_extraction_verifies_and_matches_rule_run`。

权衡（Trade-off）：冲突时自动采信，还是交回原文裁决

备选 A：固定采信一方。简单，但会把另一方发现的错误静默吞掉。规则通道的“错列”与模型的“错行”都是系统性的，固定采信一方等于固定接受一类错误。**备选 B：置信度加权或多数投票。**只有两条通道，投票没有意义；引入第三条通道（例如另一个模型会话）作仲裁，成本翻倍，而且仲裁者同样可能错，最终仍需一个“仲裁者错了怎么办”的规则。**备选 C：程序按勾稽关系自动选一个让报表平衡的值。**看似聪明，但勾稽只约束少数恒等式（资产 = 负债 + 权益等），多数指标不在任何恒等式中；而且这会把“让数平衡”变成目标，掩盖真实的抄录错误。**选择（P3）：**冲突一律记 issue 并阻断估值，由分析师会话带着两个来源（页码、chunk）回到原文裁决。代价是一次冲突就要多一轮会话；但在真实运行中冲突很少：三家公司的最终抽取（`data/extractions/*/facts.verified.json`）拒收、冲突、勾稽失败均为 0，以招商轮船为例，规则 97 条、模型 97 条、合并后 115 条。少而硬的阻断比多而软的提示更容易被会话真正处理。

4.5 P4 一张真值表

不变量 13（单一数据源）**.** 设 $\mathcal{T} = \mathit{analyze}(\cdot)[\mathit{truth_table}]$。估值流水线、研报核查流水线、文稿护栏中的指标绑定检查所使用的全部历史财务数值都取自 $\mathcal{T}$；没有任何下游模块重新解析 $\mathcal{D}$ 或重新抽取基础指标。估值内部，*DCF* 各情景的经营预测逐期等于三表模型同一情景的输出：

$$\max_{s,t,j} \left| DCF_{s,t,j} - TS_{s,t,j} \right| \le 10^{-6} \text{ 百万元}, \tag{25}$$

其中 s 为情景，t 为预测年度，j 取遍 *TIE_KEYS*（收入、*EBIT*、折旧摊销、资本开支、营运资本变动等）。

执行机制（E1 + E2）。 这是十一条原则中构造性最强的一条。`pipelines.py` 中估值步骤（`_v_three_statement`、`_v_plan`、`_v_reverse_dcf`、`_v_scenarios`、`_v_forecast_summary`、`_v_comps`）与核查步骤（`_c_verify`）的数据入口都是 `ctx["analysis"]["truth_table"]`；guard 的 `check_binding` 读的是运行目录下 `financials/analysis.json` 的 `truth_table`。式 (25) 由 `tasks/valuation/run._tie_to_three_statement` 检查，失败时 `run_dcf` 返回 `model_status="blocked"`。需要原文的估值环节（非经营性资产逐项加回）同样要过 P2：`build_plan.resolve_non_operating_items` 逐项按原文核对后由程序加总。

守护测试。 `test_valuation_reads_analysis_truth_table` 在财报流水线跑完后把 `ctx["extract"]["facts"]` 清空，再跑估值流水线，要求估值照常成功——如果有任何估值步骤绕过真值表去读抽取结果，这个测试就会失败。`test_dcf_forecast_comes_from_three_statement` 与 `test_dcf_blocked_when_three_statement_disagrees`（把三表输出的基准情景第一年 EBIT 加 1.0 百万元，DCF 必须判为 blocked）守护式 (25)。`test_deterministic` 保证同一输入两次运行的核查结论与估值区间相同。

设计动机。 财报分析、估值建模、纠错核查分开做时，最自然的实现是每个任务各自抽取所需数字。这在早期导致过一类隐蔽的不一致：财报分析报告里的归母净利润与估值模型里的基期净利润来自同一年报的不同页（摘要页与利润表页），四舍五入方式不同，最终研报正文与附录差了末位。统一为一张表之后，三个任务的口径自动一致，纠错核查的“真值”与估值建模的“基期”是同一个对象。提交 `d321358`“统一下游真值表并阻断未校验的 Skill 抽取”就是这一原则落地的节点。

权衡（Trade-off）：每个任务按需抽取，还是共用一张表

按需抽取的好处是灵活：研报纠错遇到真值表没有的指标时，可以临时去原文找。我们放弃了这种灵活性：真值表缺的指标，纠错一律判"无法核实"（P7），而不是临时抽取。原因是临时抽取绕过了 P2/P3 的全部校验，等于在系统里开了一个不受保护的后门；扩大覆盖的正确方式是在指标字典 `tasks/financials/schema.py` 里增加指标，让它进入同一套校验。另一个代价是 DCF 不再接受假设文件中的 `ebit_margin` 等字段（这些字段"不再生效"，开发指南已注明），因为情景向量必须全部来自三表模型。

4.6 P5 所有读写走 tracing

不变量 14 (读写留痕)**.** *对引擎的任意一次流水线运行 ρ（运行目录 `runs/<run_id>/`），设 $\mathrm{In}(\rho)$ 为该运行读取的输入文件集合，$\mathrm{Out}(\rho)$ 为写出的产物集合，则*

$$\forall\, p \in \mathrm{In}(\rho):\quad (p,\ \mathrm{sha256}(p)) \in \texttt{manifest.inputs},\qquad (26)$$

$$\exists\, e \in \texttt{events}:\ e.\mathit{kind} = \texttt{file_read} \ \wedge\ e.\mathit{path} = p,$$

$$\forall\, a \in \mathrm{Out}(\rho):\quad (\mathrm{rel}(a),\ \mathrm{sha256}(a)) \in \texttt{manifest.artifacts},\qquad (27)$$

且事件按 `seq` 严格递增，每个 `Pipeline.step` 在事件流中有成对的 `step_start`/`step_end`（或 `step_error`）。对模型会话，事件流原样保存在工作区 `.pipeline/agents/<` 时间 `>-claude-<` 角色 `>.jsonl`。

执行机制（E1 + E3）。 `core/tracing.py` 提供四个入口：`read_file`（读取并记录哈希）、`RunTracer.write_artifact/write_json`（写出并登记哈希）、`register_artifact`（登记由第三方函数写出的文件，如 xlsx）、`traced` 装饰器（记录工具函数的入参摘要、出参摘要、耗时与异常）。当前追踪器存放在 `ContextVar` 中，模块代码无需传递 tracer 对象，在没有活动追踪器时 `read_file` 退化为普通读取，因此同一份代码既能在流水线里留痕，也能被测试直接调用。`parsing/document.load_document` 经 `tracing.read_file` 读入年报并记录 `document_parsed` 事件；引擎中 9 个核心工具函数（`extract_financials`、`analyze_financials`、`earnings_deep_dive`、`run_three_statement`、`run_dcf`、`run_scenario_tables`、`run_comps`、`extract_claims`、`verify_claims`）用 `@tracing.traced` 注册。实现细节见 §5.3.4。

守护测试。 `test_run_artifacts_and_manifest` 要求一次 `all` 运行的 manifest 状态为 `ok`、登记了财报报告、估值工作簿与核查结论，事件种类至少包含 `file_read`、`step_start`、`tool_end`、`artifact`、`run_end`；`test_deterministic` 保证同一输入重跑结论相同；`test_render_record_binds_artifacts_and_renderer` 把研报 PDF 的渲染记录绑定到产物与排版脚本的哈希上。

一次真实运行的留痕规模。 仓库内样例全流程运行 `runs/20261006-002204-all/`：88 条事件，其中 7 次 `file_read`、18 对步骤事件、9 对工具事件、24 次 `artifact` 事件（`check/report.md` 先后被核查与一致性两步写入，manifest 中去重后为 23 个产物），6 个输入文件哈希；有 18 条事件因载荷超过 2000 字符被截断为"哈希 + 预览"。

执行范围的边界（如实说明）。 不变量 14 覆盖的是*生成运行目录的流水线*。供模型会话调用的校验命令（`verify-facts`、`guard`、`numlock`、`verify-sources`、`verify-extra`）按设计不生成运行目录，读取文件时直接用 `Path.read_text`；它们的调用记录在会话事件流里（工具调用及其输出），结论文件（`facts.verified.json`、`commentary_guard.json`）写到工作区。因此这些命令的输入哈希不在任何 manifest 中，可追溯性依赖会话日志与闸门的"校验晚于修改"时间戳检查（§11.5），强度低于 E1。

表 11　单位约定。左列为数值所在的域，右列为执行换算的位置。

域	内部单位	换算位置
财务事实（真值表）	元；股数为股	`numbers.UNIT_FACTORS`、`extract._scale`、`external._to_fact`
比率类（毛利率、同比）	百分数值（14.49 表示 14.49%）	`numbers.parse_number` 去掉百分号但不除以 100
估值（DCF、三表、comps）	百万元；每股为元	`build_plan.MM` $= 10^6$、`three_statement.MM`
经营数据（`extras.json`）	按列声明单位原值保存，金额列换算为元	`extras.verify_extras`、`guard._extras_scaled_numbers`
文稿	亿元、万元、元、%、个百分点、倍	`guard._found` 的换算因子集 $S(u)$

权衡（Trade-off）：日志框架与截断策略

通用可观测性框架（OpenTelemetry、MLflow 等）。功能完整，但引入服务端或重依赖，与“引擎只用标准库”（§5.8）冲突，评审复现时还要多装一套组件。我们选择 JSONL 事件流 + JSON manifest：任何文本工具都能读，终端界面的“追溯”页直接回放它。**全量记录每步输入输出**。可追溯性最好，但单次运行的真值表、三表明细动辄数百 KB，重复写进事件流会让日志膨胀数十倍。**选择**：事件载荷超过 `MAX_LOG_CHARS` = 2000 字符时只记 `{"_truncated": true, "sha256", "preview"}`，完整内容由产物文件保存且产物本身登记哈希；因此截断不损失可验证性：任何人都可以用产物文件重算哈希来确认日志与产物对应。

4.7　P6　单位约定

不变量 15 (单位域)**.** *真值表中所有金额以元存储，所有比率以百分数值存储；估值域内所有金额以百万元存储；跨域换算只发生在表 11 所列的边界函数中，不发生在模型会话中。*

执行机制（E1 + E3）。 换算集中在 `core/numbers.py`（`UNIT_FACTORS`、`to_yuan`、`half_ulp`、`pct_change`）与估值入口（`build_plan`）。护栏在比较时同时尝试两种表示：`guard._mm_numbers` 把 `valuation.json` 中 `forecast_base`、`pv_base`、`bridge_base` 块与以 `_mm` 结尾的键按 $\times 10^6$ 换回元，供“17,483.72 亿元”这类表述核对；百分数允许因子 $\{1, 100\}$，以接受以小数存储的比例。`pct_change` 在基期为负时按 A 股惯例用 |基期| 作分母（`test_pct_change_negative_base`）。

守护测试。 `test_parse_number`、`test_half_ulp`、`test_guard_valuation_mm_aliases_and_bridge`、`test_operating_table_units_are_scaled_for_source_check`、`test_default_units_are_tons`、`test_custom_units_flow_to_workbook`。

权衡（Trade-off）：全程一个单位，还是两个域

全程用元。最简单，但 DCF 与三表的数量级在 $10^{10}\sim10^{12}$，与原参照引擎（以百万元为单位）的输出逐位比对（§5.10）将无法进行，工作簿里的数字也不符合卖方模型的阅读习惯。**带单位的数值类型（如物理量库）。**类型安全，但要求整个引擎、产物 JSON 与工作簿写出都携带单位对象，与标准库约束冲突，序列化也复杂。**选择：**两个域 + 少数边界函数 + 测试。代价是护栏必须同时认两种表示（因子集 $S(u)$ 变大），轻微增加了巧合放行的概率（§4.14）。百分数存"14.49"而不是"0.1449"，是因为年报原文、研报与模型抄写都是前者，减少一次模型侧的心理换算——这本身也是 P1 的延伸。

4.8 P7 误报控制优先于召回

这条原则主要约束研报纠错核查，也约束护栏的指标绑定与内部一致性检查。纠错的结论类型为 {正确, 数值错误, 单位错误, 期间错配, 口径错误, 方向错误, 无法核实}，其中前五种错误类集合记为 E。

不变量 16 (误报控制)**.** *对一条研报断言 c（原文数值 r、单位 u、类型 $\tau \in \{level, yoy, ratio, pp_change\}$），设真值为 t，*

$$\mathrm{tol}(c) = \tfrac{1}{2}\,\mathrm{ulp}(r)\cdot s_u + \begin{cases} 10^{-6}|t| & \tau = level \\ 0.005 & \text{否则} \end{cases} \tag{28}$$

其中 s_u 为金额单位换算因子。则：(a) c 被判为错误仅当 $|rs_u - t| > \mathrm{tol}(c)$；(b) 预测性表述或真值缺失时判"无法核实"，不计入错误；(c) 判为单位、期间、口径、方向错误，当且仅当存在一个精确的替代解释（换一个单位、期间、同组相关指标或符号后落入容差），否则归为"数值错误"。

执行机制（E1 + E3）。 容差按展示精度计算（`numbers.half_ulp`）：研报写"58.62 亿元"，只要真值在 58.62 ± 0.005 亿元之内即判正确，不会把四舍五入当成错误。口径诊断只在 `CALIBER_GROUPS` 定义的六组易混指标内进行（如 {净利润, 归母净利润, 扣非归母净利润, 利润总额, 营业利润}），避免"恰好等于某个无关指标"的巧合匹配被诊断为口径错误。同一原则也用于护栏：`guard.check_binding` 跳过预测句（`_is_forecast`）与分部口径的表述（`_is_segment_claim`），只对"口径明确"的断言做真值绑定；内部一致性检查（`report_check/consistency.py`）只在实体、口径、期间、情景都相同时才判矛盾。

守护测试。 `test_half_ulp`、`test_report_check`、`test_display_precision_tolerated`、`test_not_comparable_contexts_are_skipped`、`test_guard_skips_forecast_sentences_but_checks_history`、`test_binding_candidate_avoids_quarter_segment_and_cash_flow_aliases`、`test_rating_rule_text_and_others_ratings_ignored`（文中引述的评级规则与他家评级不算本报告评级）。

命题 4.1 (诊断的保守性)**.** *若算法 2 输出某个具体错误类型 $e \in E \setminus \{\text{数值错误}\}$，则存在对 c 的单一修改（换单位、换期间、换同组指标或取反）使修改后的断言被判为"正确"。*

证明. 逐分支检查：单位错误分支的条件即"以 u' 为单位时落入容差"；方向错误分支即"取反后落入容差"；期间、口径分支即"换 k' 或 m' 后落入容差"，且所用容差不大于式 (28)。因此对应修改后的断言满足"正确"分支的条件。 □

命题 4.1 的意义在于：系统给出的每个具体错误类型都带一个可执行、可验证的修改建议（`suggestion` 字段），而不是凭"看起来像"下结论。反过来，系统可能把一个真实

的期间错配归为“数值错误”（当错配的期间不在真值表中时），这是有意接受的召回损失。

权衡（Trade-off）：召回优先还是误报优先

召回优先，交人工过滤。在审计场景下常见：宁可多报。我们没有这样做，原因有两个。其一，纠错核查的读者是研报作者，对一篇正确的研报报出错误，比漏报一个错误更伤害系统的可信度，作者会很快学会无视所有提示。其二，更重要的是*闭环效应*：同一套核查引擎被护栏复用（P9），用来检查我们自己生成的文稿。护栏误报会迫使写作会话去“修正”本来正确的数字，或者为了绕过检查改写成更含糊的表述，两者都让报告变差。**选择：**容差按展示精度、预测与缺真值判无法核实、错误类型要求精确替代解释。召回的损失由两处补偿：样例评测集上纠错召回仍为满分（§13），真实场景的漏报由盲评质检（§12）中的人工复核清单承接。

4.9 P8 事实 → 推论 → 观点

大赛要求估值与投资判断“区分事实、推论与观点”。LEDGER 把这条要求从写作规范落实为程序可计算的*数字出处分类*。

定义 4.2 (出处类型与认识论类别)**.** *对文稿中每个有出处的数字，按 `guard.EVIDENCE` 的顺序取第一个能找到出处的类型：报告披露 → 外部来源 → 程序计算 → 估值假设 → 模型预测；并映射到类别：报告披露、外部来源为事实，程序计算为推论，估值假设与模型预测为观点。*

不变量 17 (可分类与结构)**.** *(a) 文稿中每个通过 guard 的数字都有唯一的类别（事实、推论或观点）；(b) 评级由程序按“基准情景每股价值相对参考股价的上行空间”确定（≥ 20% 买入、≥ 5% 增持、≥ −5% 中性、≥ −20% 减持、其余卖出），文稿只能引用，不能自定；(c) 估值报告包含假设、口径简化说明与“不构成投资建议”。*

执行机制（E2 + E3）。 `guard.tagged_truth_values` 把运行产物按文件与抽取方式分到五个池：真值表中 `method="derived"` 的数进“程序计算”，其余进“报告披露”；`valuation.json`、`forecast_summary.json` 等进“模型预测”；`plan.json` 中的 WACC 与终值参数进“估值假设”；可比公司原始行进“外部来源”，倍数统计进“程序计算”。`classify_text` 对文稿逐数分类，结果写在 guard 输出的 `provenance` 字段，排版时由 reportkit 的 `provenance_note` 写进研报附录（“可溯源的 n 个数字中，事实 a 个……”）。评级由 `forecast_summary.rating` 计算，质检 L0 的 `evaluation/checks.check_rating` 复算评级并检查正文出现的评级与之一致；`check_disclaimer` 检查免责声明。

守护测试。 `test_tags_partition_truth_values`（五个池的并集等于全部真值）、`test_classify_priority_and_classes`、`test_guard_reports_provenance_and_appendix_note`（DCF 基准每股价值被归为“观点（模型预测）”并写进附录说明）、`test_rating_rule_thresholds`。

权衡（Trade-off）：让模型给句子打标签，还是程序按出处分类

让模型在每句后标注“【事实】【推论】【观点】”是最直接的做法，但标签本身不可验证：模型可以把自己的预测标成事实。按出处分类把问题转成“这个数是从哪个文件来的”，答案由文件决定。代价有二：只能分类数字，不能分类无数字的定性判断（后者交给 L3 盲评条目“事实/推论/观点”）；同一个数同时出现在多个池时按“先认事实”的顺序归类，可能把一个恰好与披露值相等的预测值归成事实。我们接受这个偏差，因为它的方向是保守的（不会把事实降级为观点）。

4.10 P9 模型输出加护栏

不变量 18 (护栏与数字锁)**.** 设 $\mathcal{N}_0$ 为主笔初稿（*narrative.draft.md*），$\mathcal{N}_1$ 为复核改写稿（*narrative.md*）。交付前同时成立：

$$guard(\mathcal{N}_1, \mathcal{A}, \mathcal{S}).ok = true, \tag{29}$$

$$\mathrm{Tok}(\mathcal{N}_1) \subseteq \mathrm{Tok}(\mathcal{N}_0) \quad \wedge \quad \mathrm{Sid}(\mathcal{N}_1) \subseteq \mathrm{Sid}(\mathcal{N}_0), \tag{30}$$

其中 Sid 为文中 *[S 编号]* 集合。*guard* 的 *ok* 等价于四个集合同时为空：无出处数字、指标—期间绑定不符、未标注来源的外部数字、前后矛盾。

执行机制（E2）。 `output/sources.run_guard` 组合四项检查（`check_text`、`check_binding`、`check_citations`、`_contradictions`）；`output/numlock.numlock` 计算 `added` 与 `new_sources`，允许数字减少（删句）、不允许新增或改动；`report_pipeline.status` 的阶段 8 闸门要求 guard 通过且核对的是当前稿，阶段 9 要求数字锁通过（§11.5、附录 D）。数字与指标的配对是否被改写打乱，由改写后重跑 guard 的绑定检查兜底。

守护测试。 `test_guard_flags_every_planted_report_error`：样例研报草稿植入 7 处错误，把每处错误句写进解读，guard 都必须返回 1；`test_numlock_allows_deletion_blocks_new_numbers`；`test_review_stage_numlock`（改写稿多出“同比增长 9.99%”时闸门判“数字锁未通过”并列出该数）；`test_external_numbers_need_citation`；`test_guard_accepts_segment_figures_and_flags_wrong_ones`。

权衡（Trade-off）：数字锁的形式

逐字 diff 或编辑距离上限。会把正常的调序、合并段落判为违规，复核编辑就无法完成“理顺故事线”的本职工作。**让另一个模型判断“改写是否忠实”。**不可复现，且判官本身会漏看末位数字。**多重集包含（数字出现次数也不能增加）。**更严格，但复核常把一个数从正文移到小结再引用一次，这是合理的。**选择：**集合包含 + 来源编号包含 + 改写后重跑 guard。它的已知盲区是“同一个数换到另一个指标上”，由 guard 的绑定检查覆盖可识别的那部分。

4.11 P10 会话隔离

不变量 19 (隔离边界)**.** 设 $\mathcal{G}$ 为仓库所在目录的父目录树（仓库与参照插件副本都在其下），$\mathcal{W}$ 为一家公司的工作区，$\mathcal{B}$ 为盲评目录。则 *(a)* $\mathcal{W} \cap \mathcal{G} = \emptyset$ 且 $\mathcal{W}$ 的祖先目录（家目录除外）不含 *CLAUDE.md*；*(b) finagent/evaluation/* 与评分标准文件 $\notin \mathcal{W}$；*(c)* 五个生产会话之间没有共享的对话上下文，只通过 $\mathcal{W}$ 内的文件交接；*(d)* 回传给生产会话的问题单只含理由与证据，不含条目原文与分值；*(e)* $\mathcal{B}$ 只含交付物与不带分值的评审条目，位于 $\mathcal{G}$ 与 $\mathcal{W}$ 之外。

执行机制（E2 + E1）。 `report_pipeline._check_location` 拒绝在 $\mathcal{G}$ 内或祖先含 `CLAUDE.md` 的位置建工作区（Claude Code 会沿父目录加载 `CLAUDE.md`）；`build` 复制引擎时用 `ENGINE_EXCLUDE` = {`__pycache__`, `evaluation`} 排除质检模块；工作区 `.claude/settings.json` 禁止 `Read(/< 父目录 >/**)`，禁止修改引擎、skill、年报原件、`sample.json` 与 `CLAUDE.md`；同一份白名单 `ALLOWED_TOOLS` 经 `--allowedTools` 传入无头会话。每个角色由 `_session` 起一次独立的 `claude -p` 进程。盲评目录由 `evaluation/judge.prepare` 准备，评测结果写在工作区之外的 `~/finagent-evals/`。本章只陈述不变量，边界的完整论证见 §11.3 与 §12.8。

守护测试。 `test_workspace_contains_only_skills_engine_and_one_company`（工作区只有一份 PDF，不含其他公司的抽取、测试、评测）、`test_workspace_guide_and_permissions`、`test_`

build_guards（在仓库 runs/ 下建工作区被拒）、test_workspace_excludes_evaluation、test_prepare_blind_dir（评审条目不含 points，在仓库内准备盲评目录被拒）、test_refine_loop_routes_issues_and_restores_best、test_run_agent_chains_write_then_review。

失败案例：权限白名单在无头模式下失效

第一次无头运行时，工作区 settings.json 中的 allow 规则全部不生效，会话的每条命令都被拒。原因是 Claude Code 对未被信任的目录忽略 allow（deny 照常生效）。修复方式是不去改用户的全局信任配置，而是把同一份白名单再经 --allowedTools 传入（提交 1dab4ca）。第二个坑是通配写法：旧式 前缀:* 只在词边界匹配，data/extractions/:* 匹配不到 data/extractions/< 公司>/...，改为 Bash(cp data/extractions/*) 这类通配规则（提交 6d9a436）。第三个坑是命令前缀：会话写 cd ws && ./py ... 时前缀匹配失败，因此每个角色提示都要求"命令直接以 ./py 开头，不加 cd、不串联"。这些都说明：隔离边界必须由程序生成并测试，不能靠人工维护配置。

权衡（Trade-off）：单 Agent 长上下文，还是多会话文件交接

单个长会话完成全部 11 个阶段。上下文连续，不必重读文件，token 成本更低。v0 批次（旧流程，没有独立的资料搜集与复核会话）接近这种形态，观察到三个问题：写作阶段会引用调研阶段读到但未登记的数字（上下文里"记得"，来源表里没有）；上下文压缩后会丢失前序阶段的约束；自己写自己审，复核几乎不改东西。v0 三家研报的质检分为 46～49 分。**多 Agent 对话框架（AutoGen、MetaGPT 式的角色对话）**[25, 62]。角色之间通过消息交流，便于协作，但消息本身就是共享上下文，写作者的自我辩护会传给复核者；评审者若看到生成过程，也更容易偏向生成方 [43]。**会话内 subagent。**父会话会把自己的上下文摘要带给子会话，隔离不彻底。**选择（P10）**：五个独立的无头会话 + 文件交接 + 程序闸门。交接文件就是接口契约，闸门判定的是文件而不是会话的自述。代价是每个会话冷启动要重读工作区（成本见 §11.13），以及"交接文件写什么"需要精心设计（§5.2）。

4.12 P11 不预设行业框架，文风不做程序拦截

这条原则与前十条方向相反：前十条都在收紧模型的自由度，P11 在两个方面有意不收紧。

不变量 20 (负向不变量)**.** *(a) skill 与工作区说明中不含任何特定行业的分析模板（"白酒看吨价""航运看运价"之类），只含方法与约束；分析框架由会话读完年报与资料包后写入 framework.md；(b) report_pipeline.status 的任何闸门都不依赖对文稿措辞的模式匹配（如禁用词表、句式正则）。闸门只检查结构（是否有外部调研清单、是否有 ≥ 3 个深度专题）与数量（字数、来源数）。*

执行机制（E4 为主）。 (a) 写在 finagent-report/SKILL.md（"分析框架自己推导。本 skill 不预设任何行业该看什么"）与 references/analysis.md 开头；(b) 文风由复核编辑会话（finagent-review skill）负责：以"没有参与写作的研究所质控编辑"身份通读，删去自证、圆场、程序口吻，改写受数字锁约束（P9）。两者都没有直接的程序检查，属于约定性原则。间接的测试是 reportkit 模板的通用性：test_template_on_another_company 与 test_template_survives_loss_no_quarterly_no_comps 保证排版模板不依赖某家公司或某个行业的数据形态；test_skill_docs_in_sync_with_code 保证 skill 里引用的命令与指标清单与代码一致。

设计动机。 (a) 的来源是框架污染：仓库里存有别家公司的分析框架与文稿，模型会照抄（docs/architecture.md 隔离工作区一节的理由）。例如为宁德时代出报告的会话读到贵州茅台的框架后，照搬了"量价拆分 + 渠道结构"的骨架，而对电池企业更关键的产能利用率、海外工厂与储能业务写得很薄。隔离工作区（P10）解决了"读到别家文稿"，P11

表 12　执行路线的演变（据 `git log`）。

阶段	代表提交	形态
内嵌 API	`dbcb501`（9 月 22 日）	程序内 `core/llm.py` 调用 OpenAI 兼容接口（DeepSeek、通义千问、智谱），`core/tools.py` 工具注册表 + `ask` 工具循环；请求哈希作缓存键，`LLM_MODE=replay` 可离线重放
Skill 版本	`70cdd0d`	Claude Code 直接读 PDF，程序负责 `locate`、`verify-facts`、`guard`；与内嵌版本共用引擎
多执行器	`1dab4ca`、PR #16	研报流水线支持 Claude Code 与 Codex 两种 harness；PR #16 增加 DeepSeek 执行器
单一路线	`4e8fe66`、`d9ae074`（10 月 5 日）	删除 DeepSeek 执行器、程序内 LLM 客户端、`ask`、prompt 模板、`--llm` 参数与 Codex 执行器；只保留无头 Claude Code

解决“skill 本身就是别家框架”。(b) 来自 PR #16（DeepSeek 执行器）的经验：那条路线加了正则文风闸门，命中“值得注意的是”等短语即拒收，结果模型学会同义替换，文风没有变好，闸门却让会话在措辞上反复打转。合并该 PR 时（提交 `4e8fe66`）明确“不采用正则文风闸门（文风由复核会话把关）”，这条决定后被写进开发指南。

权衡（Trade-off）：行业模板 vs 模型自推导；正则拦截文风 vs 编辑会话

行业模板。可以保证每份报告覆盖“该行业该看的指标”，评审也容易对照。但模板是对行业的静态先验，对跨行业公司（中际旭创：光模块 + 数据中心周期）与新业态（宇树科技：招股书阶段的机器人公司）会给出错误的框架；而且模板一旦写进 skill，模型会填空而不是思考。**模型自推导**的风险是遗漏，由两道机制兜底：阶段 3 闸门要求框架含外部调研清单与至少 3 个深度专题；盲评 L2 条目检查“驱动拆解”“预期差”等通用问题（§12.2）。**正则拦截文风**的问题如上；**编辑会话**的代价是不可复现、每轮多一个 Sonnet 会话，且复核质量依赖模型能力。我们接受这个代价，因为文风是“读起来如何”的问题，它的评判本来就需要读者，而不是模式匹配。相关讨论另见 §14.6。

4.13　执行路线的收敛：从内嵌模型调用到无头 Claude Code

十一条原则之外，开发指南还有一条架构层面的决定：“只有一条路线”。它不是一条不变量，但决定了上述原则由谁执行，所以在此一并说明。仓库共 93 个提交，路线经历了四个阶段（表 12）。

权衡（Trade-off）：国产模型 API 内嵌调用 vs 无头 Claude Code harness

内嵌调用的优点是真实的：大赛优先国产模型；`temperature=0` + 固定 seed + 响应缓存使模型输出可以离线重放（可复现性更强，见 §5.9）；不依赖外部命令行工具。放弃它的理由按重要性排列：(1) *重复实现 harness*。读 PDF 页面图像、联网检索、文件读写权限、长任务的上下文管理，内嵌路线都要自己实现，而且实现得不如成熟 harness；`ask` 工具循环越做越像一个简化版 Claude Code。(2) *两条抽取路径分叉*。内嵌版的 LLM 抽取补漏与 skill 版的 `facts.json` 是两套代码，行为细节不同，测试与文档都要双份维护，P4“一张真值表”的入口也随之变成两个。(3) *能力与任务长度*。深度研报需要几十到上百轮工具调用、联网检索与自我检查（§11.13），这是 agent harness 的强项，而不是一次 Chat Completions 调用的强项。(4) *原则执行不依赖模型来源*。P1 ~ P9 的执行全部在确定性引擎里，与哪个模型、哪个 harness 无关；保留单一路线反而让这些检查只有一个入口。代价：模型输出不再能离线重放（可复现性降为两层定义，§5.9）；与“优先国产模型”的赛题偏好不一致，在计划书中单独说明理由；对一个外部商业工具形成依赖。为降低后者，编排器把 harness 封装在 `agent_command` 与 `_session` 两个函数里，引擎与闸门对 harness 一无所知，换执行器只需改这两处（另见 §14.3）。

4.14 原则之间的张力与优先级

十一条原则并不总是同向。下面讨论开发中实际遇到的五对张力，以及系统给出的处理规则。

张力一：召回 vs 误报（P7 对 P1、P9）。 P7 要求护栏宽容（按展示精度、跳过预测句、只绑定口径明确的断言），P1 与 P9 要求护栏严格（不放过任何编造数字）。护栏的放行条件式 (22) 是“存在某个出处”，所以它有一个结构性的*巧合放行率*。设某单位下产物可作出处的数值集合为 V_u，文稿数字 r 在其展示精度下的取值区间长度为 L（例如两位小数、0 ~ 100 的百分数，$L = 100$），若编造的数在区间内近似均匀分布，则被巧合放行的概率约为

$$P_{\text{coinc}} \;\approx\; \min\Big(1,\; \frac{|\mathsf{V}_u| \cdot 2 \cdot \tfrac{1}{2}\,\mathrm{ulp}(r) \cdot |S(u)|}{L}\Big). \tag{31}$$

对两位小数的百分数，$\frac{1}{2}\,\mathrm{ulp} = 0.005$，实测五份交付研报的结论性产物中 0 ~ 100 区间内约有 460 ~ 600 个不同的百分数值，取 $|S(\%)| = 2$ 时上式给出 $P_{\text{coinc}} \approx 0.1$；实测的末位扰动放行率为 3% ~ 5%（第 10.3 节），低于均匀假设，因为真实数值并不均匀覆盖整个区间；对“亿元”两位小数、区间数千亿元的金额，概率低一到两个数量级。也就是说，guard 对*小区间、低精度*的数（如一位小数的百分数）保护较弱。系统用两种方式收窄这个缺口：其一，`check_binding` 对能识别指标与期间的表述做真值绑定，此时比较对象只有一个真值而不是整个 V；其二，guard 的真值池只取结论性产物（真值表、DCF 结果、comps 等），不含三表明细与敏感性网格等中间结果，以控制 $|\mathsf{V}_u|$。我们没有进一步收紧（例如要求每个数都能绑定指标），因为那会把大量合法的叙述性数字（分部、经营数据、外部数据）判为违规，违反 P7。

张力二：深度 vs 可验证（P1、P2 对报告质量）。 深度研报需要大量外部信息与多环定量推导，而 P1、P2 要求每个数都有出处、每个派生数都由程序算。直接后果是写作成本上升：外部数字要先检索、登记摘句（`verify-sources`），派生数要先写进 `extras.json` 的 `ratios`。v1 批次的研报因此显得“数字多、分析薄”。v2 的处理不是放松 P1/P2，而是在流程上给深度留出空间：阶段 4 深度调研闸门（≥ 3 份专题笔记、每份 ≥ 1500 字与 ≥ 3 个独立来源）、阶段 8 篇幅闸门（正文 ≥ 15,000 字、深度研究占比 ≥ 60%），让“可验证”与“深度”在不同阶段各自被满足。质检分从 v1 的 74/67 升到 v2 的 90/88（单次盲评，非对照实验）。

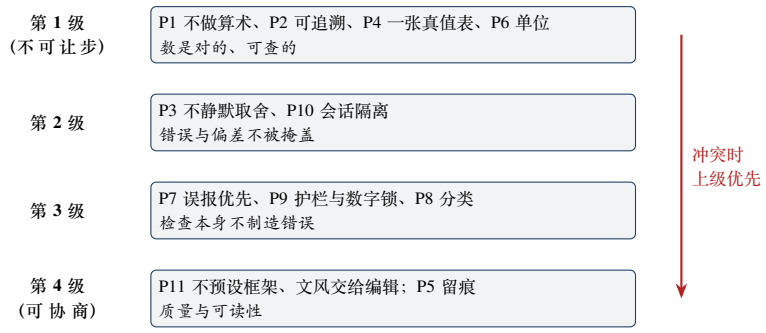


图 5　原则的优先级（字典序）。上级原则与下级原则冲突时，修改下级原则的实现，而不是放松上级原则。例如为提高可读性（第 4 级）而希望写“约三倍”，正确做法是让程序算出比值（第 1 级）后由编辑决定如何表述；为降低误报（第 3 级）而放宽护栏，不能放宽到允许无出处数字（第 1 级）。

张力三：自动化 vs 人工复核（P3、P9 对端到端自动化）。 P3 的阻断与 P9 的拒收都会让流水线停下。LEDGER 的规则是：*程序能判定的，阻断并交回会话修复；程序判不了但重要的，写进人工复核清单*（`review.md`），而不是由模型自行决定放行。`refine` 在闸门未通过时不做盲评、直接停下交给人；三轮未达标时回滚到最佳一轮，并把剩余问题追加进 `review.md`（§11.7）。自动化的边界因此是明确的：系统保证“交出去的东西过了全部程序检查”，不保证“全部检查都会自动通过”。

张力四：隔离 vs 效率（P10 对成本）。 五个会话各自冷启动，重读年报定位文件、资料包与运行产物；盲评会话还要在全新目录里重读交付物。隔离让每次运行的 token 成本显著高于单会话（§11.13）。我们保持隔离，原因是 v0 的单会话流程在质量上的损失远大于成本节约；降低成本的手段是让每个会话*少读*（工作区 `CLAUDE.md` 按阶段写明“读什么、不读什么”，上下文被压缩时“先跑 status，不要重读已完成阶段的文件”），而不是共享上下文。

张力五：一致性 vs 修正（参照一致对 P4 与估值口径）。 自研引擎要与原参照插件逐位一致（§5.10），这与“发现口径问题就修”有冲突。例如流动性压力测试沿用原插件口径 FCF × (1 + 冲击)，FCF 为负时方向相反；为保持一致没有改，只在报告中注明。反之，EBIT 剔除投资收益与公允价值变动、三表经营费用改为轧差项、DCF 改由三表驱动，这些是*有意*的口径修改，修改时同步重新生成 golden 快照并在 `THIRD_PARTY.md` 中说明。规则是：引擎内部算法保持与参照一致，口径映射层可以改，但必须显式记录。

优先级。 把上述处理规则归纳为图 5 的字典序。它的含义是可操作的：在代码评审中，任何为了下级目标而削弱上级不变量的修改都应被拒绝。有一个例外值得说明：P5（留痕）被放在第 4 级，不是因为它不重要，而是因为它几乎从不与其他原则冲突，且它的削弱（如截断日志）可以被产物哈希补偿。

4.15　原则—机制—测试总表

表 13 汇总十一条原则。“强度”列见定义 4.1；测试列只列最有代表性的一到三个，完整对应关系可在 `tests/` 中按函数名检索。

表 13 原则—机制—测试总表。

原则	不变量（简述）	执行机制	代表测试	强度
P1 不做算术	文稿每个数在产物/来源中按展示精度可找到；派生数全部由程序算	`FactTable.add_derived`、`extras._ratios`、`guard.check_text`	`test_extras_verified_and_ratios_computed`、`test_guard_flags_invented_number`	E2
P2 可追溯	事实带 σ，数字核在所写页按数字边界匹配；外部数字在摘句中	`external.find_evidence`、`verify_sources`、`check_citations`	`test_fabricated_number_rejected`、`test_wrong_page_rejected`、`test_verify_sources`	E2
P3 不静默取舍	两通道冲突必记 issue，并阻断估值	`extract.merge`、`_p_analyze`、`verify-facts`	`test_wrong_row_number_caught_as_conflict`、`test_skill_pipeline_stops_before_valuation_on_conflict`	E2
P4 一张真值表	下游只读 `truth_table`；DCF 与三表逐期相等	`pipelines.py` 数据入口、`_tie_to_three_statement`	`test_valuation_reads_analysis_truth_table`、`test_dcf_blocked_when_three_statement_disagrees`	E1
P5 留痕	输入与产物哈希入 manifest，步骤成对记录	`tracing.read_file`、`RunTracer`、`Pipeline.step`、`@traced`	`test_run_artifacts_and_manifest`、`test_deterministic`	E1/E3
P6 单位	事实为元、估值为百万元、比率为百分数值	`numbers.UNIT_FACTORS`、`build_plan.MM`、`guard._mm_numbers`	`test_parse_number`、`test_guard_valuation_mm_aliases_and_bridge`	E1/E3
P7 误报优先	容差 = $\frac{1}{2}$ ulp；预测判无法核实；错误类型需精确替代	`verify_claim`、`CALIBER_GROUPS`、`_is_forecast`	`test_half_ulp`、`test_display_precision_tolerated`	E1/E3
P8 事实/推论/观点	每个数有唯一类别；评级由程序定	`tagged_truth_values`、`classify_text`、`forecast_summary.rating`	`test_classify_priority_and_classes`、`test_rating_rule_thresholds`	E2/E3
P9 护栏与数字锁	guard ok；改写稿数字与来源编号 $\subseteq$ 初稿	`run_guard`、`numlock`、阶段 8/9 闸门	`test_guard_flags_every_planted_report_error`、`test_review_stage_numlock`	E2
P10 会话隔离	工作区在仓库树外、无评测代码；会话只靠文件交接	`_check_location`、`ENGINE_EXCLUDE`、`settings`、`judge.prepare`	`test_workspace_contains_only_skills_engine_and_one_company`、`test_prepare_blind_dir`	E2
P11 不预设框架	skill 无行业模板；闸门不匹配措辞	skill 文本、复核编辑会话	`test_template_on_another_company`（间接）	E4

4.16 小结

本章把 LEDGER 的十一条设计原则写成了可检查的不变量。它们有一个共同的结构：模型负责读、抄、写、判断，程序负责校验、计算、拦截，而二者之间的每一条边界

都落在一个具体函数和一组具体测试上。十一条中，P1 ~ P10 的执行强度为 E1 ~ E3，即使模型不遵守 skill 中的要求，违反也会被程序发现；只有 P11 是有意保留的约定性原则，因为它约束的对象（行业判断与文风）不适合用程序判定。原则之间的冲突按图 5 的字典序处理：数值正确与可追溯不可让步，误报控制与可读性在其下协商。下一章描述承载这些不变量的系统架构。

5 系统架构

第 4 节把设计原则写成了不变量；本章描述承载这些不变量的软件结构。重点是*计算引擎*这一层：它如何与模型会话交接、内部如何组织步骤与留痕、如何对外提供命令行与界面、如何保证跨平台与可复现，以及靠什么测试体系维持上述性质。多 Agent 编排（五个会话、十一道闸门、`refine` 循环）与质检体系另有专章（§11、§12），本章只交代它们在整体中的位置和接口。

5.1 四层结构

LEDGER 由四层组成（图 6）。自下而上：

1. **计算引擎**（`finagent/`，不含 `evaluation/`）：确定性 Python，*不调用任何模型*。负责文档解析与定位、抽取校验、财务分析、估值、研报核查、文稿护栏与数字锁，全程留痕。依赖仅为标准库与 `pdfplumber`。
2. **技能层**（`.claude/skills/` 四个 skill + 编排器为每个工作区生成的 `CLAUDE.md`）：用自然语言写给模型会话的方法与约束，规定每个角色读什么、写什么、跑哪条命令、闸门是什么。技能层不含代码逻辑，只引用引擎的命令。
3. **多 Agent 编排**（`finagent/report_pipeline.py`）：确定性 Python 编排器。`build` 为一家公司搭隔离工作区，`run` 依次起五个无头 Claude Code 会话，`status` 判 11 道闸门，`collect` 收回产物，`scorecard` 对比多个工作区。
4. **质检循环**（`finagent/evaluation/` + `refine`）：程序检查（L0/L1）+ 盲评会话（L2/L3/负向）+ Python 计分 + 问题单路由与回滚。

分工可以用一句话概括：*模型读、抄、写、判断；程序校验、计算、拦截*。图中橙色节点是模型会话，蓝色节点是确定性程序，红色菱形是程序闸门。所有跨层交互都经过两种媒介之一：工作区里的文件，或引擎的命令行（及其退出码）。没有任何一层通过共享内存或对话上下文与另一层交互。

为什么是四层而不是“一个 agent + 一组工具”。 这种分层直接来自第 4 节的原则：P1/P4 要求计算集中在一个不调用模型的层；P10 要求会话之间只靠文件交接，因此需要一个确定性编排器负责“谁先谁后、何时放行”；P11 要求方法与约束以自然语言给出而不预设行业框架，于是有独立的技能层；评分标准对生成方不可见（§12.8），于是质检单独成层并物理隔离。每一层可以单独替换：换 harness 只动编排器中的两个函数，换评分标准只动 `eval/rubrics/`，引擎对二者一无所知。

5.2 模型与引擎的唯一接口：文件与命令行

定义 5.1 (引擎接口)**.** *引擎对模型会话暴露的接口是一个二元组* $(\mathcal{F}, \mathcal{H})$：$\mathcal{F}$ *为交接文件集合（模型写、引擎读），* $\mathcal{H}$ *为命令集合*

$$\mathcal{H} = \{\mathit{locate}, \mathit{schema}, \mathit{verify\text{-}facts}, \mathit{verify\text{-}extra}, \mathit{verify\text{-}sources}, \mathit{guard}, \mathit{numlock}\} \cup \{\mathit{financials}, \mathit{valuation}, \mathit{check}, \mathit{all}\}.$$

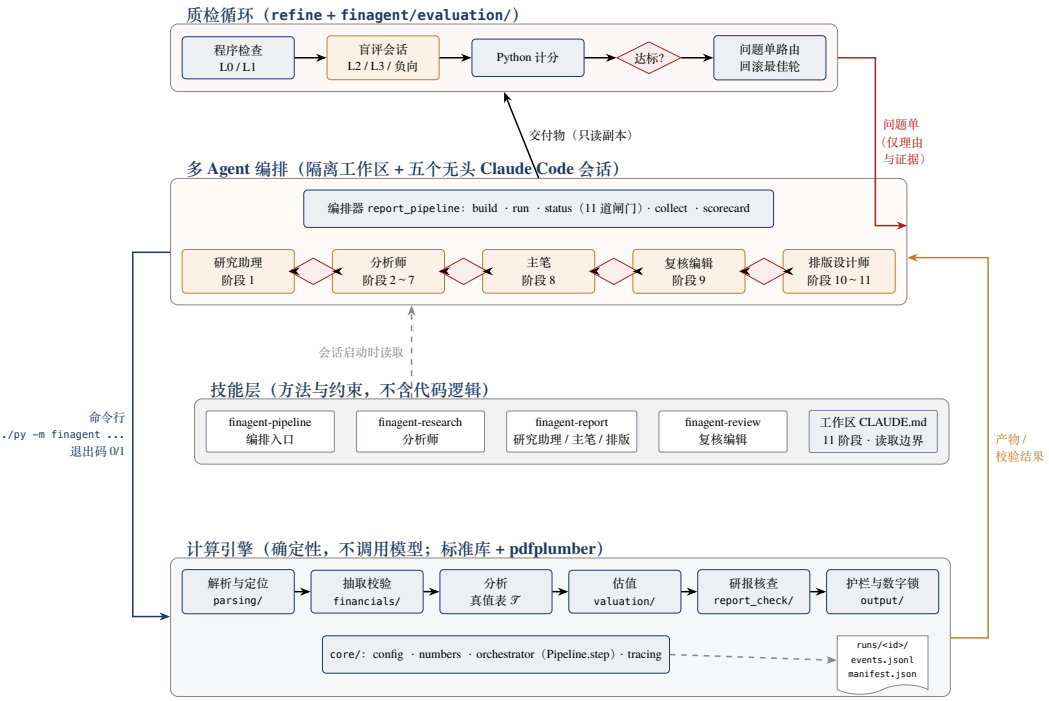


图 6　LEDGER 的四层结构。蓝色：确定性程序；橙色：模型会话；红色菱形：程序闸门。会话之间不共享上下文，只靠工作区文件交接；模型与引擎的唯一接口是文件与命令行（§5.2）。质检层的评分标准与完整结果不进入工作区，回传的只有问题单。

前七个为校验命令：不生成运行目录，结果打印为 JSON，退出码 0 表示通过、1 表示存在阻断项；后四个为流水线任务：生成 runs/<run_id>/ 并完整留痕。

模型会话只能以 `./py -m finagent < 命令 >` 的形式调用引擎（工作区权限白名单只放行这一前缀，§11.3），不能 import 引擎模块、不能修改引擎代码。因此“模型能对计算施加的影响”被严格限定为它写下的交接文件内容；而交接文件的每一个字段，都会在进入计算之前经过 $\mathcal{H}$ 中的某个校验命令。退出码是这一接口的关键：闸门（§11.5）与模型都能以同一个比特判断“是否可以继续”，不需要解析自然语言输出。

六个模型交接文件。 表 14 列出模型写给引擎的六个文件。它们的共同设计是：模型只写“读到了什么”和“假设是什么”，不写“算出了什么”。完整 JSON 模式见附录 C。

表 14　模型写、引擎读的六个交接文件。

文件	写入者	内容与约束	校验 / 接入
`facts.json`	分析师	逐条 {metric_id, period, raw_value, unit, page, scope, label, table, source_file}；raw_value 照抄原文，不换算；metric_id 取自 schema 命令给出的指标清单，派生指标不抄	verify-facts（证据、口径、双通道、勾稽、核心缺失）；流水线 --facts

文件	写入者	内容与约束	校验 / 接入
extras.json	分析师	经营数据表（分产品、分渠道、产销量等）：每表声明列与列单位、逐行 values 与页码；ratios 只写比率的定义（分子、分母引用的格），不写数值	verify-extra 逐格回查；_ratios 由程序计算；流水线 --extras
sources.json	研究助理（分析师追加）	questions（检索问题、类别、检索词、状态）与 sources（网址、发布方、发布与访问日期、摘句 ≤ 160 字、数值）；数值须原样出现在摘句中	verify-sources；guard --sources；阶段 1 覆盖度闸门
claims.json	分析师	研报断言 {metric_id, period, claim_type, value, unit, quote, is_forecast}；quote 必须是研报原文连续片段且含该数值	claims.ingest_claims；流水线 --claims
valuation_assumptions.json	分析师	情景向量（收入增速、资本开支率、终值增长、WACC 调整）、WACC 参数、三表驱动、可选分部量价 segment_forecast、非经营性资产明细、current_price 与 price_source、依据（引用 [S 编号]）	三表与 DCF 引擎的结构校验；非经营性资产逐项回查原文
narrative.md	主笔 → 复核编辑	研报文稿，按 <!-- section: 名称 --> 分节；数字只能抄产物展示值或带 [S 编号] 的外部数	guard（阶段 8）、numlock（阶段 9）

引擎写给模型的文件同样有约定：locate.json（关键页与单位，build 时预先生成放进工作区）、facts.verified.json（校验结论，闸门据此判断）、运行目录中的结论性产物（analysis.json、valuation.json、forecast_summary.json 等，主笔从中抄数），以及 commentary_guard.json。会话之间的交接（资料包、分析框架、专题笔记、复核清单等）属于编排层，见 §11.4 与表 80。

一个 **facts.json** 条目。 下面是样例公司一条营业收入抽取（中文字段值以 LaTeX 转义插入，原文件为 UTF-8 JSON）：

```
{"metric_id": "revenue", "period": "2024", "raw_value": "5,862,340,125.36",
 "unit": "元", "page": 6, "scope": "consolidated",
 "label": "营业收入", "table": "主要会计数据和财务指标"}
```

引擎对它依次执行：指标合法性（revenue 在指标字典中且非派生）、期间格式、口径（consolidated）、数值解析、页码存在、find_evidence(doc, 6, "5,862,340,125.36") 在第 6 页按数字边界找到、单位换算为元，最后与规则通道同键结果比对（§6）。

表 15　finagent/ 的模块组织（行数为 2026-10-05 版本，含注释）。

目录 / 模块	职责	行数
core/	配置、数字与单位、步骤编排、追踪	341
parsing/	PDF / Markdown 切块（带页码、chunk 锚点、单位、表题、续表）；关键页定位	396
tasks/financials/	指标字典、规则抽取、模型抽取接入、经营数据、分析与真值表、业绩对比	1,367
tasks/valuation/	分部预测、三表引擎、DCF、反向 DCF、情景表、可比公司、工作簿	3,931
tasks/report_check/	断言抽取、核查、内部一致性	560
output/	报告渲染、xlsx 写出、护栏、来源登记、数字锁、渲染检查	1,462
pipelines.py、__main__.py、cli.py	三条流水线、命令行入口、统一命令行	672
catalog.py、doctor.py	产物目录（命令行与界面共用）、环境自检	292
report_pipeline.py	多 Agent 编排器（第 11 节）	1,139
evaluation/	质检：程序检查、盲评、计分、校准（第 12 节；不进工作区）	1,838

权衡（Trade-off）：接口形式

函数调用接口（MCP 服务或 function calling）。结构化程度更高，参数可做模式校验。但它把引擎绑定到某一种 harness 协议上；而命令行 + JSON 文件可以被任何能执行 shell 的 agent 使用，也可以被人直接在终端里复现。另一个考虑是*留痕*：文件是持久的，闸门可以比较“校验结果文件的修改时间是否晚于被校验文件”，从而发现“改了输入却没重新校验”的情况（§11.5）；函数调用的返回值只存在于会话日志里。**让模型直接 import 引擎写 Python 脚本**。灵活，但模型可以绕过校验直接构造真值表，P2/P3 就失效了。工作区权限只允许 `./py -m finagent` 与少数只读命令，`./py -c` 虽被允许（用于排版脚本），但工作区禁止写 finagent/，且流水线只从交接文件读模型输入。**选择**：文件 + 命令行 + 退出码。代价是模型需要学会一组命令与文件格式，这由 skill 文档承担，并由 test_skill_docs_in_sync_with_code 保证文档与代码同步。

5.3　计算引擎的内部结构

表 15 给出模块组织，合计约一万两千行。下面依次说明 core/ 的四个模块，它们是第 4 节多条不变量的直接载体。

5.3.1　core/config.py：只来自环境变量的配置

配置只有一个字段 runs_dir（运行目录根，默认仓库 runs/），全部来自环境变量或仓库根目录 .env。_load_dotenv 用 os.environ.setdefault 读取 .env，所以*显式环境变量优先于文件*，评审可以在不改文件的情况下覆盖配置。Settings.public() 返回写入 manifest 的配置副本。历史上这里还有模型名、温度、随机种子、缓存目录等字段（内嵌模型路线），在单一路线清理时一并删除（§4.13）；现在与模型有关的配置（FINAGENT_GEN_MODEL、FINAGENT_QC_MODEL）只存在于编排器，引擎配置中没有任何模型相关项——这本身就是 P1“引擎不调用模型”的一个构造性体现。

5.3.2　core/numbers.py：中文财报的数字与单位

所有数字解析经过 parse_number：接受千分位（含全角逗号）、括号负数“(1,234.50)”、多种负号（-、–、−）、百分号（去掉但不除以 100），把“—”“不适用”“N/A”等识别为

缺失。`half_ulp(raw, factor)` 按原文小数位数给出舍入容差，是 P7 与护栏的共同基础；`pct_change` 在基期为负时用绝对值作分母；`UNIT_FACTORS` 定义元、千元、万元、百万元、亿元、万亿元与股、万股、亿股的换算。这些函数都是纯函数，由 `test_parse_number`、`test_half_ulp`、`test_pct_change_negative_base` 覆盖。

5.3.3 `core/orchestrator.py`：步骤注册与执行

引擎内部的流水线是一个极简的“步骤表”：

```
@dataclass
class Pipeline:
    name: str
    steps: list[Step] = field(default_factory=list)

    def step(self, name, output_key=None):          # decorator: register a step
        def deco(fn):
            self.steps.append(Step(name, fn, output_key or name))
            return fn
        return deco

    def run(self, ctx):
        for i, st in enumerate(self.steps, 1):
            tracing.log("step_start", pipeline=self.name, step=st.name, index=i, total=len(self.steps))
            try:
                ctx[st.output_key] = st.fn(ctx)        # each step reads ctx, writes one key
            except Exception as exc:
                tracing.log("step_error", pipeline=self.name, step=st.name, error=...)
                raise
            tracing.log("step_end", pipeline=self.name, step=st.name, elapsed_ms=...)
        return ctx
```

每个步骤是一个 `ctx -> 值`的函数，返回值写入 `ctx[output_key]`；步骤按注册顺序执行，任何异常都先记 `step_error` 再向上抛出，不吞异常。这个设计有意没有做依赖解析、并行或缓存：三条流水线总共 18 个步骤，样例全流程在 0.06 秒内完成（真实年报的耗时主要在 PDF 解析），复杂调度带来的收益远小于它给“可读、可复现”带来的成本。阻断（P3、P4）通过步骤内抛出 `ValueError` 或返回 `status="blocked"` 实现，异常会使 `RunTracer` 以 `status="error"` 结束并写出 manifest，失败的运行同样留痕。

5.3.4 `core/tracing.py`：留痕

`RunTracer` 是一个上下文管理器。进入时生成 `run_id = <YYYYMMDD-HHMMSS>-<task>`，创建运行目录，打开 `events.jsonl`，并把自己放进 `ContextVar`；退出时写 `run_end` 事件与 `manifest.json`，异常退出时先写 `run_error`。模块代码通过模块级函数 `tracing.log`、`tracing.read_file`、`tracing.current()` 访问当前追踪器，不需要层层传参；没有活动追踪器时这些函数退化为空操作或普通读取，因此同一份分析代码可以被单元测试直接调用。

manifest。 `manifest.json` 汇总一次运行：`run_id`、任务、状态、开始时间、耗时、Python 版本、平台、配置（含完整 argv 与抽取方式 `"model+rule"` 或 `"rule"`）、输入文件哈希表、产物哈希表、事件总数。输入哈希来自 `read_file`，产物哈希来自 `register_artifact`；二者共同构成运行的“指纹”，复现时比对指纹即可确认输入与产出是否一致（§5.9）。

哈希与截断。 所有哈希为 SHA-256。事件载荷经 `_summarize` 处理：先序列化为 JSON，若长度不超过 `MAX_LOG_CHARS = 2000` 字符则原样记录，否则记为 `{"_truncated": true, "sha256": < 全文哈希 >, "preview": < 前 2000 字符 >}`。完整内容由产物文件保存且产物

表 16　events.jsonl 的事件种类。每条事件含递增的 seq、毫秒级时间戳 ts 与 kind。

kind	产生位置	载荷
run_start / run_end / run_error	RunTracer	任务名、配置（含 argv）；状态与耗时；异常类型与信息
step_start / step_end / step_error	Pipeline.run	流水线名、步骤名、序号/总数；耗时（毫秒）；异常
tool_start / tool_end / tool_error	@traced	工具名、入参摘要；出参摘要与耗时；异常与耗时
file_read	read_file	路径、字节数、SHA-256
document_parsed	load_document	来源、哈希、chunk 数、表格数、页数
artifact	register_artifact	运行目录内相对路径（POSIX 形式）、SHA-256、字节数

本身登记了哈希，所以截断不损失可验证性。下面是样例全流程运行的前几条事件（节选，哈希截短）：

```
{"seq": 2, "kind": "step_start", "pipeline": "financials", "step": "parse", "index": 1, "total": 6}
{"seq": 3, "kind": "file_read", "path": "data/samples/annual_report_2024.md",
 "bytes": 5898, "sha256": "a77370e5...97468"}
{"seq": 4, "kind": "document_parsed", "chunks": 29, "tables": 6, "pages": 110, ...}
{"seq": 7, "kind": "tool_start", "tool": "extract_financials",
 "args": {"_truncated": true, "sha256": "f2fe6405...1d04f", "preview": "..."}}
{"seq": 9, "kind": "artifact", "path": "financials/facts.json",
 "sha256": "6808a385...c9160", "bytes": 60730}
```

跨平台路径。 产物键用运行目录内的相对路径并统一为 POSIX 形式（as_posix()），否则 Windows 上会写出反斜杠，同一运行在两个平台上的 manifest 不可比（提交 861ab84）。

要点：留痕要在 manifest 上测

comps 步骤的公式工作簿 comps_formula_workbook.xlsx 与估值报告步骤的 model.xlsx 由 xlsx 写出函数直接落盘，不经过 write_artifact，因此两处都显式调用 register_artifact 补登记。只检查“文件存在”的测试发现不了漏登记，所以 test_run_artifacts_and_manifest 直接断言 manifest 中有 valuation/model.xlsx：留痕的完整性要在 manifest 上测，而不是在文件系统上测。

5.4　三条流水线

pipelines.py 用 Pipeline.step 注册三条流水线，对应财报分析、估值建模与纠错核查（图 7）。入口 pipelines.run(task, ctx) 总是先跑财报流水线，再按任务名追加估值与核查：financials 只跑第一条，valuation 跑前两条，check 跑第一与第三条，all 三条全跑。这保证了 P4：估值与核查永远建立在同一次分析产出的真值表上。

财报流水线（6 步）。 parse 调 load_document 切块；extract 在有 facts.json 时调 external.ingest（证据校验 + 与规则通道合并），否则只走规则通道 extract_financials；analyze 计算派生指标、同比、勾稽、异常信号、单季拆分并产出真值表，带模型抽取时在此检查阻断条件；extras 逐格核对经营数据并由程序计算比率，有拒收即停止；earnings 做实际 vs 一致预期 / 业绩预告与 EPS 质量；report 渲染财报分析报告。

估值流水线（8 步）。 three_statement 读假设文件，若有 segment_forecast 则先由分部量价生成收入增速、毛利率与费用率向量（要求提供 --extras），若非经营性资产以明细给

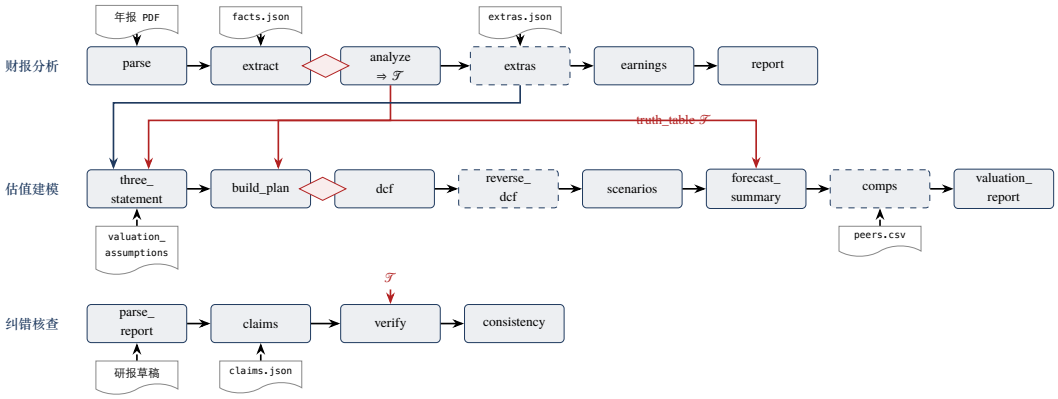


图 7　引擎三条流水线的步骤（实线框）与数据依赖。虚线框为可选步骤（无对应输入时返回 None），虚线箭头为模型交接文件或外部输入，红色为真值表 $\mathcal{T}$ 的扇出（核查流水线的 verify 同样读取 $\mathcal{T}$），蓝色细线为经营数据驱动分部预测，小菱形为阻断点：analyze 之前（P2/P3：拒收、冲突、勾稽失败、核心缺失），build_plan 与 dcf 之间（P4：DCF 与三表逐期核对）。

表 17　上下文字典 ctx 的主要键。步骤之间只通过这些键传递数据。

键	写入步骤	读取步骤
doc	parse	extract、extras、three_statement（非经营性资产回查）
extract	extract	analyze（只用于阻断判定与 issue）
analysis（含 truth_table）	analyze	extras、earnings、全部估值步骤、verify、consistency
extras	extras	three_statement（分部预测）、valuation_report
three_statement	three_statement	build_plan、dcf、forecast_summary、valuation_report
plan / valuation	build_plan / dcf	reverse_dcf、scenarios、comps、valuation_report
claims / check	claims / verify	verify / consistency

出则逐项回查原文后加总，然后构建并运行三表模型；build_plan 把真值表、假设与三表输出组装为 DCF 引擎的 plan（百万元）；dcf 运行三情景 FCFF DCF 与敏感性，并按式 (25) 与三表逐期核对；reverse_dcf 在有参考股价时逐个反解隐含参数；scenarios 生成情景与敏感性表格；forecast_summary 汇总预测并给出程序评级；comps 在有同业文件时做可比公司估值并写公式工作簿；valuation_report 渲染估值报告并写出联动工作簿 model.xlsx。

核查流水线（4 步）。 parse_report 切块研报草稿；claims 接入 claims.json（引用须为原文片段）或用规则抽取断言；verify 按算法 2 逐条核对真值表；consistency 检查研报内部一致性（评级、各情景每股价值、预测年度公司整体收入与利润），对 PDF 研报把行号换算回页码。

表 17 说明：extract 的结果在 analyze 之后不再被任何步骤读取（test_valuation_reads_analysis_truth_table 清空它后估值照常成功），这就是不变量 13 在代码层面的样子。

5.5　命令行、产物目录与环境自检

两级入口。 finagent/__main__.py 定义选题命令与校验命令（定义 5.1），用一个 HELPERS 字典把七个校验命令分派到各自的函数，它们不创建 RunTracer；四个流水线任务在 RunTracer 中执行，结束时写 summary.json 并打印摘要。--supplement 必须与 --facts 同时给

表 18　统一命令行 finagent < 命令组 > 的九个命令组。

命令组	作用	原入口
research	财报 / 估值 / 核查流水线与校验、护栏命令	python -m finagent
report	研报流水线：build、run、status、refine、collect、scorecard、template	finagent.report_pipeline
eval	研报评测：盲评、重算分数、对比、质检包	finagent.evaluation
calibrate	评审校准：重复评审、劣化命中、人工一致	evaluation.calibrate
bench	样例评测集：抽取准确率与纠错召回/准确率	eval/run_eval.py
test	运行全部测试	pytest -q tests
doctor	运行环境自检	finagent.doctor
ls	列出研报工作区与运行目录（含闸门进度与最近质检分）	新增
tui	终端界面	新增

出，因为规则通道不识别季报表头（开发指南“已知问题”）。finagent/cli.py 是统一命令行，把原来分散的入口收在一处（表 18）；旧入口（python -m finagent all ...、python -m finagent.report_pipeline ...）照旧可用，以 finagent financials 这类旧任务名开头的调用自动转发到 research 组。

免安装启动脚本。 bin/finagent（POSIX shell）与 bin/finagent.cmd（Windows）优先使用仓库 .venv 的解释器，找不到时退回系统 Python，并把仓库根加入 PYTHONPATH，因此评审克隆仓库后不必 pip install 也能运行；pyproject.toml 同时注册了 finagent = finagent.cli:main 入口，可选依赖分为 report（matplotlib、pillow）、tui（textual）、dev（pytest、fpdf2）三组。

产物目录 catalog.py。 命令行 ls 与终端界面共用一个只读的产物目录：workspaces() 扫描工作区根目录（默认 ~/finagent-samples），以含 sample.json 为准；runs() 列出仓库 runs/ 中含财务分析产物的运行（排除样例评测运行）；gates() 调用编排器的 status() 取闸门进度，工作区不完整时返回 None 而不是报错；scores() 把 refine 各轮、eval/results/ 与 eval/calibration/ 中评的是同一运行的结果按评测时间排序；events()、agent_runs()、agent_log() 读取引擎事件流与会话事件流。目录只读文件、不改任何东西，因此可以在流水线运行时安全使用。

环境自检 doctor.py。 finagent doctor 逐项检查并以 ✓（就绪）、!（可选功能缺失）、✗（必须修复）标注：Python 版本（≥ 3.10，建议 3.12）；pdfplumber、matplotlib（必需）与 PIL、pytest（可选）；Claude Code 是否安装、是否登录（ANTHROPIC_API_KEY 或 claude auth status）；生成与质检模型分工；排版所需的浏览器（HTML → PDF）；matplotlib 可见的中文字体（楷体、宋体、黑体等十种候选）；.env 是否存在。--live 额外向生成与质检模型各发一次最小请求（claude -p " 只回复 OK" --max-turns 1），确认模型真的可用。任何一项 ✗ 时退出码为 1，且从不打印密钥。自检把“环境问题”与“系统问题”分开：评审复现失败时，先跑 doctor 就能区分是缺浏览器还是代码出错。

5.6　终端界面

finagent_tui/（约 720 行，Textual 实现）是产物的只读看板。左侧树列出研报工作区与运行目录，右侧七页：

- **概览**：11 道闸门状态、估值区间条、程序评级、最近质检分；

- **真值表**：指标 × 期间表格，选中一个数显示其来源（文件、页码、表题、chunk、是否续表）、抽取方式（规则 / 模型 / 二者一致）与派生公式；
- **估值**：三情景每股价值、预测摘要、敏感性矩阵热力表；
- **质检**：逐条判定、证据与理由，可切换不同轮次或评测；
- **追溯**：按 `seq` 回放 `events.jsonl`，可按关键词过滤（读了哪些文件、调用了哪些工具、写了哪些产物）；
- **Agent**：各会话的模型、轮数、耗时、token 与费用，以及事件流摘要；
- **命令**：在界面内运行任意 `finagent` 命令并流式显示输出。

界面遵守 P1：渲染模块 `render.py` 只做格式化（金额按亿/万展示、比率加百分号），不做任何计算；数值全部来自引擎产物。Textual 是可选依赖，`finagent/` 本身不依赖它；未安装时 `finagent tui` 提示安装命令并返回 2，而不是抛出导入错误。`test_tui_renders_every_tab` 在无头模式下渲染全部七页。选择 TUI 而非网页前端的理由（与流水线"文件即状态"的形态吻合、同语言直接调用 `status()`、可经 `textual serve` 发布到浏览器）记录在 `docs/design/2026-10-04-frontend-tui.md`；网页前端由其他成员另行开发。

5.7 跨平台

开发在 macOS 上进行，但评审可能使用 Windows。跨平台问题集中在四处，均有对应的代码与测试：

1. **解释器包装**。工作区生成两个包装器：`py`（POSIX shell，设置 `MPLCONFIGDIR` 后 exec 解释器）与 `py.cmd`（Windows 批处理）。编排器用 `PY_WRAPPER = "py.cmd" if os.name == "nt" else "./py"` 生成会话提示与权限白名单，使同一份工作区说明在两个平台上都可执行。
2. **路径表示**。manifest 的产物键统一为 POSIX 形式（§5.3.4）；工作区元数据中的年报路径用 `as_posix()` 写入。
3. **编码**。所有文本读写显式指定 `encoding="utf-8"`；Windows CI 设置 `PYTHONUTF8=1`，因为 GitHub Windows Runner 可能把重定向输出设为 cp1252，估值评测打印中文时会出错（提交 `c69a047`）。
4. **外部程序定位**。浏览器与中文字体不依赖 macOS 固定目录：`FINAGENT_CHROME`、`FINAGENT_CJK_FONT` 环境变量优先。

`.github/workflows/windows.yml` 在 `windows-latest`、Python 3.12 上安装依赖、运行全部测试与样例评测，并做一次冒烟测试：在 Runner 临时目录建贵州茅台工作区，用 `py.cmd` 运行 `status`，断言新工作区闸门未通过时退出码为 1（这一断言本身曾因 PowerShell 把非零退出码传成步骤失败而需要显式 `exit 0`，提交 `289c818`）。`tests/test_windows_compat.py` 的三个测试检查：字体环境变量可接受 Windows 风格路径、浏览器环境变量跨平台生效、README 与工作流文件中写明了 Windows 虚拟环境路径与 UTF-8 设置。

5.8 依赖最小化

引擎只依赖标准库与 `pdfplumber`（表 19）。在去除原插件依赖的提交（`2300cfc`）中，`requirements.txt` 去掉了 pandas、openpyxl 与 XlsxWriter：三表引擎与情景表格改为纯 Python 实现，xlsx 由 `output/xlsx.py` 用 `zipfile` 直接写 OOXML。

表 19　各部分的依赖。“必需”指运行该部分不可缺少。

部分	依赖	性质
计算引擎 `finagent/`	Python 3.12 标准库；`pdfplumber`≥0.11（仅 PDF 解析）	必需
xlsx 写出	标准库 `zipfile` + 手写 OOXML（`output/xlsx.py`，含公式单元格与缓存值）	无第三方
研报排版 reportkit	matplotlib、pillow、本机 Chrome / Chromium / Edge	研报流水线必需
终端界面	Textual（附带 Rich）	可选
测试与开发	pytest；fpdf2（仅重建样例 PDF）	开发时
多 Agent 执行	Claude Code 命令行（用户自行安装、登录）	研报流水线必需

权衡（Trade-off）：依赖最小化的理由与代价

理由。(1) 评审复现：赛题要求评审能复现，依赖越少，安装失败的概率越低；引擎在任何有 Python 3.12 的机器上 `pip install pdfplumber` 即可运行。(2) 工作区复制：编排器把整个引擎复制进每个隔离工作区，引擎越自包含，工作区越小、越不受外部环境影响。(3) 可审计：数值计算全部在可读的 Python 代码里，没有隐藏在 DataFrame 运算或第三方库的默认行为中（例如 pandas 的缺失值传播与舍入规则）。(4) 逐位一致：与参照引擎逐位比对时，减少一层库版本带来的浮点差异。
代价。(1) 自己写 OOXML：公式单元格、样式、数字格式、图表都要手写，`xlsx.py` 与两个工作簿模块合计约一千六百行；工作簿审计（`evaluation/workbook_audit.py`）也要自己解析 XML。(2) 没有向量化，三表与 DCF 都是显式循环；对单家公司 5 ~ 10 年预测的规模而言耗时可忽略。(3) PDF 解析能力受 `pdfplumber` 限制：稀疏网格错列、跨页续表、科目折行都需要自研后处理（`compact_columns`、`link_continuation`、`merge_wrapped_labels`，见 §6），扫描页无法处理。我们认为在比赛规模下理由明显占优；如果引擎需要处理批量公司或高频更新，引入 DataFrame 与专业 PDF 解析器是自然的扩展。

5.9　可复现性的两层定义

赛题要求“可追溯、可复现”。对一个以大模型为核心推理引擎的系统，“可复现”需要分层定义，否则要么承诺过度（声称模型输出可复现），要么承诺不足（因为模型不可复现就放弃整体复现）。

定义 5.2 (两层可复现)**.** ***A* 层（逐位复算）**：给定一次运行的输入文件（哈希与 *manifest* 一致）、交接文件（*facts.json*、*extras.json*、*claims.json*、假设文件）、命令行参数与代码版本，在任何受支持的平台上重新运行引擎，所有结论性产物的内容逐位相同（时间戳与运行编号除外）。***B* 层（可审计但不可重放）**：模型会话的完整事件流（每次工具调用、读取的文件、写下的内容、轮数、*token*、费用）被原样保存，任何人可以逐步查看模型做了什么；但在没有模型的环境下无法重新生成相同的交接文件与文稿。

命题 5.1 (引擎确定性)**.** 引擎的流水线任务不读取时钟、随机数或网络，不调用模型；除运行编号、时间戳与耗时字段外，其输出是输入文件字节与命令行参数的函数。

论证. 按模块检查：`core/` 中只有 `RunTracer` 读取时钟，且只写入 `run_id`、`ts`、`started_at`、`elapsed`；引擎中没有对 `random` 的使用（随机输入只出现在对照测试 `test_parity_reference.py` 中，且固定种子 20260924）；网络访问只发生在模型会话（WebSearch / WebFetch）中，不在引擎中。集合与字典的迭代顺序在 CPython 3.7+ 中按插入顺序确定，产物 JSON 中的列表按 `(metric_id, period)` 等键显式排序。`test_deterministic` 对同一输入运行两次，断言核查结论与估值区间相同；Windows CI 运行

表 20　各层记录与可复现程度。

层	记录	可复现程度
计算引擎	runs/<id>/events.jsonl、manifest.json（输入与产物哈希、argv、Python 版本、平台）	A 层
校验命令	结论文件（facts.verified.json、commentary_guard.json）+ 会话事件流中的调用	A 层（给定输入）
模型会话	.pipeline/agents/*.jsonl 原始事件流、agent_runs.json（模型、轮数、耗时、token、费用）	B 层
工作区	sample.json（年报 SHA-256、仓库提交号 repo_commit、创建时间、外部输入）	定位代码与输入版本
质检	judgments.json、score.json、issues.json、loop.json	计分 A 层；盲评 B 层

与 macOS 相同的 golden 测试，要求产物与快照完全相等，从而在两个平台上检验了浮点结果的一致性。 □

为什么不追求模型输出的 A 层复现。 内嵌模型路线曾经提供过：core/llm.py 以请求体的 SHA-256 为键缓存响应，LLM_MODE=replay 时只读缓存、缓存缺失即报错，因此整条链路可离线重放（§4.13）。无头 Claude Code 的会话是多轮工具调用，环境（文件系统、联网检索结果）随时间变化，缓存单次请求不足以重放整个会话。我们的处理是：(1) 让模型输出尽早落地为文件（交接文件、文稿），一旦落地，其后的所有计算都是 A 层；(2) 收回仓库时（collect）把抽取文件、调研资料与文稿复制到 data/extractions/< 公司 >_< 年度 >/，评审可以跳过模型、直接用这些文件逐位复算全部产物；(3) 会话事件流原样保存，作为 B 层审计材料。换言之，系统承诺的是“给定模型交出的文件，后面每个数都能复算”，而不是“模型每次交出同样的文件”。复现步骤见附录 G。

5.10　golden 快照与参照对照：自研引擎与原插件的关系

LEDGER 的若干估值与业绩分析引擎在计算规则上参照了 OpenAI Public Equity Investing 插件（0.1.31，许可证标注为 Proprietary −Internal Use Only）。仓库最早通过 third_party/ 原样引入并调用该插件（vendor.py），在提交 2300cfc 中全部删除，改为自研实现；仓库现在不包含原插件的任何代码，THIRD_PARTY.md 如实列出参照关系（表 21，完整版见附录 F）。

两类测试共同维持“自研 = 参照”这一性质：

golden 快照（tests/test_golden.py，无需原插件）。 tests/golden/ 下的四个 JSON（three_statement.json、scenarios.json、comps.json、earnings.json）是 2026-09-24 用原插件引擎跑样例数据得到的产物。test_matches_original_engine 按这四个文件参数化，要求自研引擎的对应产物与快照完全相等（Python 对象 ==，即逐字段、浮点逐位）。另外三个测试检查可比公司公式工作簿：估值区间缓存值与 comps.json 一致、敏感性表的 0% 列等于隐含股价、来源台账覆盖目标公司与全部同业、摘要页含两张原生 Excel 图表。快照的作用是：即使仓库不再保留原插件，任何对引擎的修改只要改变了样例产物就会被发现。

参照对照（tests/test_parity_reference.py，需本地原插件）。 原插件不入库；tests/_reference.py 按环境变量 FINAGENT_REFERENCE_PLUGIN 或仓库上级目录查找，找不到则整组跳过。找到时按文件路径加载原插件脚本（多个 skill 有同名模块 skill_core.py，

表 21　参照原插件的自研引擎与一致性校验方式（据 THIRD_PARTY.md）。

参照内容	自研实现	一致性
三表联动、营运资本、PP&E、债务与现金归集、勾稽、模型状态	ts_engine.py	浮点逐位相等
三情景目标价与偏度、各类敏感性、跟踪触发表	scenario_tables.py	输出逐字节相等
beat/miss、业绩预告中值对比、EPS 质量规则	earnings.py	逐字段相等
FCFF DCF	dcf.py	相对误差 $< 10^{-9}$
EV 桥、倍数配对、NM、IQR 离群值、P25/P75 区间	comps.py	按文档规则自研
反解思路、重复数字对账、论点框架、工作簿控制、证据标签	reverse_dcf.py、consistency.py 等	仅借方法

因此赋予唯一模块名并临时调整 sys.path），用样例、边界值与随机输入逐项对照五个方面：三表模型（样例 + 边界 + 60 组随机 plan，覆盖收入/成本/营运资本/PP&E 各口径、亏损与可抵扣亏损、循环贷与现金归集、分红回购增发等）、三表 plan 能通过原插件自己的校验器、情景表格（180 组随机输入）、业绩规则（300 组随机输入）、DCF（20 组随机情景）。随机输入使用固定种子，失败可复现。在本文写作时的环境中这五个测试全部通过。

有意修改口径的流程。 2026-09-28 我们有意修改了三处口径：EBIT 改为利润总额 + 财务费用 − 投资收益 − 公允价值变动收益；三表经营费用改为轧差项（毛利 − EBIT − 折旧摊销），以消除原口径中折旧摊销被重复扣减的问题；DCF 改由三表驱动。修改发生在*口径映射层*（three_statement.py、build_plan.py），引擎本身（ts_engine.py 等）未改，因此参照对照仍逐项相等；golden 快照因输入映射变化而重新生成，变更原因记录在 tests/golden/README.md 与 THIRD_PARTY.md。这条流程把“与参照一致”和“修正口径”分到两层：引擎算法保持一致，口径映射可以改，但每次改都留下记录（另见 §4.14 张力五）。

权衡（Trade-off）：保留原插件调用、复刻代码，还是自研并对照

直接调用原插件。实现最快，早期版本即如此；但原插件许可证为内部使用，参赛提交不能包含其代码，且其依赖（pandas、openpyxl 等）与引擎最小依赖目标冲突。**复制修改原代码。**同样有许可问题，且会让“哪些是自研”变得模糊，违反赛题“不得冒充自研”的要求。**选择：**按计算规则自研，用 golden 快照 + 本地参照对照保证结果一致，在 THIRD_PARTY.md 逐项说明参照关系与校验方式。代价是需要维护两套测试，并接受“为保持一致而保留原口径中的不合理之处”（如流动性压力测试在 FCF 为负时方向相反，已在报告中注明）。

5.11　测试体系

tests/ 下共 27 个测试文件、约 3,100 行，收集到 173 个测试用例（含参数化）。在本文写作的环境中（macOS、Python 3.12.14、本地有原插件），运行 python -m pytest -q tests 的结果为 **167 通过、6 跳过**；跳过的 6 个是 test_report_render.py 中需要真实 Chrome 打印的回归测试（需设置 FINAGENT_BROWSER_TESTS=1）。本地没有原插件时，test_parity_reference.py 的 5 个测试也会跳过。表 22 按覆盖面分组。

表 22 测试按覆盖面分组（括号内为收集到的用例数）。

覆盖面	测试文件	用例
解析与数字	test_pdf_tables（7）、test_evidence_spacing（1）、test_numbers_and_schema（5）、test_resources（1）	14
抽取接入与护栏（skill 路线）	test_skill（26）、test_real_report（14）	40
财务分析	test_quarterly_ebit（13）、test_financial_model_todos（2）、test_pipeline（6）	21
估值	test_new_modules（4）、test_reverse_dcf（6）、test_segment_units（2）、test_workbook_audit（3）	15
参照一致	test_golden（7）、test_parity_reference（5）	12
纠错、一致性与出处	test_consistency（11）、test_guard_caliber（2）、test_provenance（3）、test_sources_numlock（3）	19
多 Agent 流水线与排版	test_report_pipeline（10）、test_report_kit（6）、test_report_render（8）、test_review_docs（4）	28
质检与校准	test_evaluation（12）、test_calibrate（4）	16
命令行、界面与平台	test_cli（5）、test_windows_compat（3）	8
合计	27 个文件	173

测试的四种角色。 按它们守护的对象，测试分为四类：(1) *不变量测试*：直接对应第 4 节的原则，如 test_valuation_reads_analysis_truth_table（P4）、test_wrong_row_number_caught_as_conflict（P3）、test_workspace_excludes_evaluation（P10），总表见表 13；(2) *对抗测试*：构造模型可能犯的错误并要求被拦截，如编造数字、错页、母公司口径、只抄了半个数字、期间看反、伪造研报引用、植入 7 处错误的研报草稿；(3) *真实数据回归*：test_real_report.py 在仓库内的贵州茅台年报 PDF 上跑规则通道、模型抽取校验、非经营性资产回查、经营数据核对、联动工作簿公式，保证真实版式上的改进（稀疏网格、跨页续表、科目折行）不退化；(4) *一致性测试*：golden 快照、参照对照、文档与代码同步（test_skill_docs_in_sync_with_code：skill 中提到的每条命令都存在、抽取清单与指标字典一致、核心指标标记一致）。

测试替身。 多 Agent 流水线的测试不调用真实模型：test_report_pipeline.py 与 test_evaluation.py 用模拟会话（monkeypatch _session，在工作区写入预设文件）检验会话串联、闸门判定、问题单路由与回滚；盲评的计分与重算用预设的 judgments.json。因此全部测试可以离线、在半分钟内跑完（本环境约 24 ~ 31 秒）。

样例评测。 测试之外还有 eval/run_eval.py（finagent bench）：在虚构样例公司“星河智造”上计算抽取准确率（对照 eval/gold_financials.json）与纠错的准确率、召回率、误报（对照 eval/labels.json 中的埋错标签）。开发指南规定“改完代码必须跑测试和评测，样例评测指标下降视为回归”；样例数据内部完全自洽，真实数据效果会明显下降，真实数据上的实验见 §13。

表 23　三份真实年报的解析规模（`load_document` 在本机实测；“续表”指 `continued_from` 非空的表格 chunk）。

年报（2025 年度）	页数	表格 chunk	续表	解析耗时（秒）
贵州茅台（600519）	143	270	42	5.5
宁德时代（300750）	232	655	59	10.0
招商轮船（601872）	246	325	82	9.4

权衡（Trade-off）：测试覆盖什么

测试只覆盖确定性部分与编排逻辑，不覆盖模型输出质量。后者由质检循环（§12）衡量，二者职责分开：测试回答“如果模型交出了这些文件，系统是否正确处理”，质检回答“模型交出的东西好不好”。我们没有把真实模型调用放进测试（例如录制一次会话做回放测试），因为会话依赖联网检索与 harness 版本，回放极易因无关变化而失败，维护成本高于收益。

5.12　小结

LEDGER 的架构可以归纳为三点。第一，*计算引擎是一个不调用模型的确定性程序*，与模型的唯一接口是文件和命令行，每个交接文件进入计算前都要过校验命令，退出码让闸门与模型用同一个比特判断能否继续。第二，*引擎内部刻意简单*：18 个按顺序执行的步骤、一个上下文字典、一个基于 `ContextVar` 的追踪器，换来的是可读、可复算、全程留痕；依赖只有标准库与 `pdfplumber`。第三，*可复现被分为两层*：引擎逐位复算（由确定性、golden 快照与跨平台 CI 保证），模型会话可审计但不可重放（由原始事件流保证）；系统的承诺是“给定模型交出的文件，后面每个数都能复算”。在此之上，多 Agent 编排与质检循环分别在第 11 节与第 12 节展开。

6　文档解析与证据约束抽取

本章讲 LEDGER 怎样从一份上百页的 A 股年报 PDF 得到一张“每个数字都能指回原文”的事实表。第 3 节已经给出证据谓词 Ev（定义 3.3）、数字边界匹配（定义 3.2）、可接受条目（定义 3.5）、两通道一致性与冲突集（定义 3.6）和抽取通过条件（定义 3.9）。这里不重复这些定义，而是展开它们背后的算法、实现与真实年报上的版式难点：文档被切成什么样的单元（§6.2），模型该读哪几页（§6.3），指标与期间怎样识别（§6.4、§6.5），规则通道怎样应付稀疏网格、跨页续表、科目折行和同页多单位（§6.6），模型通道写下的 `facts.json` 怎样逐条过证据校验（§6.8），两个通道怎样合并而不静默取舍（§6.9），经营数据表与季报的接入（§6.11、§6.12），以及 `verify-facts` / `verify-extra` 两个命令的完整流程（§6.13）。最后报告三家真实年报上的结果（§6.14）并讨论失败案例（§6.15）。

6.1　问题与总体设计

真实输入是什么样子。 表 23 列出本章用到的三份真实年报在解析后的规模（本文在当前代码上重新解析得到）。一份年报 143 ~ 246 页，pdfplumber 识别出 270 ~ 655 张表格，其中 42 ~ 82 张是跨页续表。需要抽取的核心数字分布在“主要会计数据和财务指标”“分季度主要财务指标”“非经常性损益项目及金额”、三张合并报表及现金流量表补充资料中，合计约 10 ~ 15 页；其余页面里有大量与报表科目同名、口径不同的附注明细表（受限资产、费用明细、关联方往来、母公司报表），是误取的主要来源。

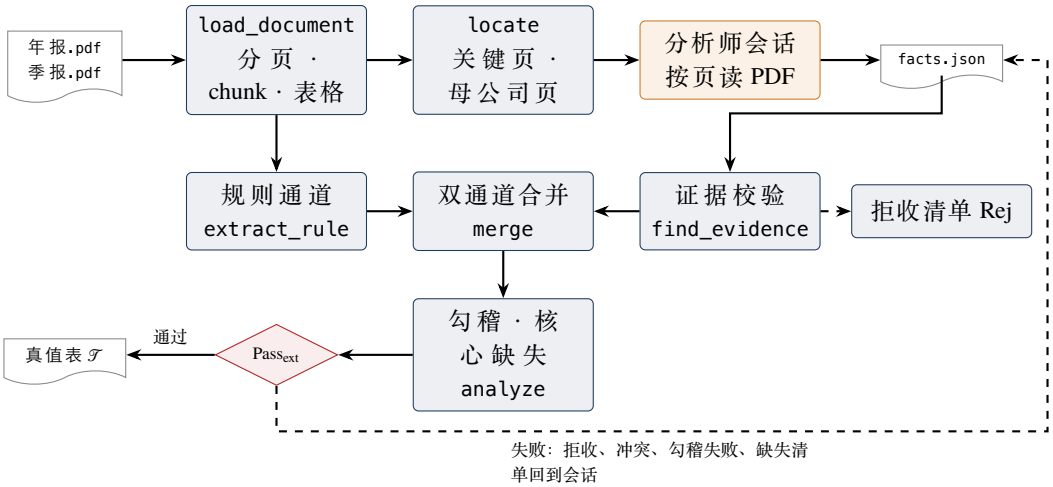


图 **8**　证据约束抽取的数据流。蓝色为确定性程序，橙色为模型会话，红色菱形为闸门。模型只写 `facts.json`，其余步骤都不调用模型；闸门不通过时，`verify-facts` 的拒收、冲突、勾稽失败与缺失清单回到分析师会话。

两种直觉方案各自的问题。 第一种是*纯规则*：按科目名在表格里查数。它确定、可复现，但真实 PDF 的表格结构经常被解析器切坏——宁德时代的报表在解析后科目和数字错开一列，规则通道在修复前一个数都取不到（表 32）。第二种是*让模型读 PDF 直接给出数字*。模型对版式的鲁棒性好得多，但它可能抄错一位、看错一列、把母公司报表当成合并报表，或者干脆写出原文没有的数字 [28]。金融场景中这些错误代价很高，又很难事后发现。

设计。 LEDGER 让两种方案同时运行、互相约束，并把模型的输出限制在“可以被程序逐字核对”的形式上（图 8）：

1. 程序先把 PDF 切成带页码的 chunk，并用 `locate` 告诉模型该读哪几页、哪些页是母公司报表、哪些页没有文本层；
2. 分析师会话（模型）按页读 PDF，把数字按原文写法抄进 `facts.json`，并写明页码、单位与口径；它不做任何换算或计算；
3. 程序对每一条做证据校验：原文写法必须按数字边界出现在所写页码的文本中，否则拒收；
4. 规则通道独立地从同一份 PDF 取数，与模型通道逐键比对；一致记为相互佐证，冲突记为 issue 并阻断估值；
5. 合并后的事实进入真值表，经勾稽校验与核心缺失检查后才能被下游使用。

这里模型承担的是“读”和“抄”，程序承担的是“找证据”“比对”“计算”。这种分工与程序辅助推理 [8, 20] 的思路一致：不信任模型做算术；但我们更进一步，连模型抄下来的数字也不直接信任，而是要求它带着页码交给程序回查，接近“带引用生成” [21] 在数字层面的严格版本。

要点

本章所有组件都服从同一条约束：*进入真值表的每个数字，要么由规则通道从某个表格单元格取得（带行列坐标），要么由模型抄写且在所写页码的原文中按数字边界找到，要么由程序从前两类数字派生（带公式与输入来源）。*第 3 节不变量 1 是这条约束的形式化版本。

6.2 文档模型：分页、chunk 与表格

Chunk。 解析器（`finagent/parsing/document.py`）把文档变成 chunk 序列。每个 chunk 带 `chunk_id`（按出现顺序编号 `c0001`、`c0002`……）、`source`（文件路径）、`page`（PDF 物理页码，从 1 开始）、`kind`（`text` 或 `table`）、`text`（文本，表格则为其 Markdown 渲染）、`rows`（单元格矩阵）、`unit_hint`（单位提示）、`caption`（表格标题），以及续表专用的 `header_rows` 与 `continued_from`。文档整体带 SHA-256 哈希，读文件经 `tracing.read_file()`，哈希写入运行目录的 `events.jsonl`。

页码选物理页而不是年报印刷页码，是因为模型会话用 Claude Code 的 Read 工具读 PDF 时看到的就是物理页；两边用同一个页码，模型写下的 `page` 才能被程序直接回查。抽取规范（`references/extraction.md`）明确要求“PDF 页码（从 1 开始，Read 工具的页码），不是年报印刷页码”。

PDF 解析流程。 对每一页，`_parse_pdf` 依次做：

1. 用 `page.extract_text()` 取整页文本，用正则 `UNIT_RE`（“单位：[人民币] 元/千元/万元/百万元/亿元”）找出本页全部单位声明；
2. 用 `page.find_tables()` 找出表格，逐张抽取单元格并去掉全空行；
3. 为每张表找标题（`_caption_above`）、定单位（见 §6.6.5）、判断是否为上一页表格的续表（见 §6.6.4）；不是续表时做稀疏列合并与折行合并（§6.6.2、§6.6.3）；
4. 记录页末标题（表格从下一页开始的情形），供下一页首张表使用；
5. 按空行切分整页文本，生成文本 chunk。

文本 chunk 与表格 chunk 并存：同一个数字会同时出现在表格 chunk 的 Markdown 和文本 chunk 里。证据校验对表格优先（§6.8），这样定位尽量落在结构化的表格上。

表格标题。 标题决定规则通道是否从这张表取数（§6.6.1），也是区分合并报表与母公司报表的主要依据。`_caption_above` 在表格上方、上一张表底部以下的区域里自下而上最多看 6 行：跳过“单位：”行和 `SKIP_RE` 匹配的行（“编制单位”、报表日期如“2025 年 12 月 31 日”、“√ 适用 不适用”、币种、页眉“年度报告”）；第一个长度不超过 40 字、含“表”字或形如“一、”“（一）”“1、”“(1)”的编号标题即为标题；遇到超过 40 字的行（正文段落）就放弃。找不到标题的表格沿用上一张表的标题——这对续表正确，对“同一节下的第二张小表”也大体正确。

Markdown 与 TXT 输入。 样例数据与测试用 Markdown（`_parse_markdown`）：`<!-- page: N -->` 注释模拟 PDF 页码，管道表格按表格解析，最近的“#”标题作为表格标题，“单位：”行更新后续表格的单位。这让全部样例回归和大部分单元测试不依赖 PDF 解析器，也让我们能用虚构但内部自洽的样例（`data/samples/build_samples.py`）精确构造勾稽与季度拆分场景。

扫描页。 扫描页没有文本层，`extract_text()` 返回空，该页不产生任何 chunk。`locate` 把这些页列在 `no_text_layer_pages`；任何声明来自这些页的数字都会因 $\mathrm{Ev} = \bot$ 被拒收。我们不做 OCR。

表 **24** `locate` 的节与关键词（`SECTIONS`），以及在贵州茅台 2025 年报上的命中页（`report/resources/贵州茅台/locate.json`）。

节	关键词（任一命中即可）	茅台命中页
`key_data`	主要会计数据和财务指标;主要会计数据;主要财务指标	4, 5, 6
`quarterly`	分季度主要财务指标；分季度主要财务数据；分季度主要会计数据	7, 8
`non_recurring`	非经常性损益项目及金额；非经常性损益项目；非经常性损益明细	7, 8, 111, 142, 143
`balance`	合并资产负债表	56
`income`	合并利润表	61
`cashflow`	合并现金流量表	64
`cashflow_supplement`	现金流量表补充资料；补充资料	114, 115, 142
`share_capital`	股本变动情况；股份变动情况；普通股股份总数	47, 48
建议阅读（三张主表向后延伸 2 页后合并为区间）		4–8, 47–48, 56–58, 61–66, 111, 114–115, 142–143

权衡（Trade-off）：扫描页拒收而不做 OCR

OCR 能让扫描页的数字“可抽取”，但 OCR 的输出本身是模型推断，用它来核对另一个模型抄下的数字，等于用不可靠的证据证明不可靠的结论；OCR 错把“8”认成“3”时，证据校验反而会把模型的正确抄写判为拒收，或把同样认错的抄写判为通过。沪深交易所要求的定期报告基本是带文本层的 PDF，扫描页罕见；遇到时 skill 要求分析师会话“告诉用户，由用户决定是否人工录入”。我们宁可让流程停下来，也不让无法逐字核对的数字进入真值表。

6.3 关键页定位：`locate`

动机。 把整本年报送进模型上下文既贵又容易出错：模型会在 200 页里找到多个“营业收入”，其中有母公司报表的、有分部明细的、有附注的。`locate`（`finagent/parsing/locate.py`）用章节标题关键词告诉模型“该读哪几页”，把阅读范围压到 10 ~ 20 页。

算法。 先把每页所有 chunk 的标题与文本拼起来并去掉空白，得到页文本 s_p。对表 24 的每个节，命中页为 $\{p : \exists \text{关键词} \in s_p\}$。三张主表（资产负债表、利润表、现金流量表）常跨 2 ~ 3 页，建议阅读范围从命中页向后延伸 `SPAN=2` 页；其余节只读命中页。另外：

- `parent_only_pages`：含“母公司资产负债表 / 利润表 / 现金流量表”的页，提示模型这些页的数字不要抄；
- `unit_hints`：建议阅读页上的第一个单位声明；
- `no_text_layer_pages`：总页数（由 pdfplumber 给出）中没有任何 chunk 的页。

延伸页可能同时包含合并报表末尾与母公司报表开头（茅台第 58 页就是这种情况），`locate` 照样建议阅读，口径由 `parent_only_pages` 提示与条目的 `scope` 字段把关。

茅台年报 143 页，建议阅读 25 页，约为全书的 17%。关键词检索是粗粒度的：“补充资料”会命中附注里其他补充资料，“非经常性损益”会命中附录。这些误命中只会多读几页，不会让错误数字进入真值表；漏命中（关键词写法不同）则由模型自行翻页补足。

权衡（Trade-off）：关键词定位 vs. 版面分析模型

版面分析与表格结构识别有成熟的学习方法 [4, 42, 54]，它们能给出更准确的表格边界与单元格结构。我们没有采用，原因有三：(1) `locate` 的产物只是“建议阅读页”，读错一页的代价是多花一点 token，不影响正确性，不值得引入重模型依赖；(2) 计算引擎约定只用标准库加 pdfplumber，便于评审在任意机器上复现；(3) 表格结构错误由规则通道的稀疏列合并与续表算法处理，并且最终由模型通道交叉兜底。若后续需要处理大量非标准版式（如港股年报），可以在不改动证据校验的前提下替换解析器。

6.4 指标字典

指标字典（`finagent/tasks/financials/schema.py`）共 49 个指标：42 个可抽取（利润表 13、资产负债表 18、主要会计数据 5、现金流量表 4、非经常性损益 2），7 个派生（毛利率、净利率、资产负债率、销售/管理/研发费用率、净现比）。每个指标是一个不可变 `Metric`：`id`、中文名 `name`、所属报表 `statement`、类型 `kind`（39 个金额、7 个比率、每股、股数、倍数各 1 个）、财报科目别名 `aliases` 与研报说法别名 `claim_aliases`。前者建成 58 项的别名索引供规则通道匹配表格行名，后者共 56 个说法供研报纠错（第 9 节）识别“营收”“归母净利润”“经营现金流”等口语化写法。其中 14 个是核心指标（`external.CORE_METRICS`），本期缺失即阻断估值，上期缺失只作警示。

行名清洗与精确匹配。 报表行名带大量装饰：编号前缀（“一、”“三、”）、连接词前缀（“其中：”“加：”“减：”）、单位后缀（“（元 / 股）”“（%）”）、填列说明（“（损失以“- ”号填列）”“（亏损以“-”号填列）”）、全角空格与 PDF 折行产生的换行符。`clean_label` 依次去空白与换行、去前缀（`_PREFIX_RE`）、去填列说明（`_SIGN_NOTE_RE`，要求括号内同时含“以”与“号填列”）；`match_label` 先查清洗后的整串，再查去掉单位后缀的串。两次都是精确查表，不做子串或模糊匹配。

例 6.1 (精确匹配避免的误配)**.** “归属于母公司股东的净利润”包含子串“净利润”，“其中：对联营企业和合营企业的投资收益”包含子串“投资收益”。子串匹配会把前者配成 *net_profit*、把后者配成 *investment_income*，两者都是真实年报中会出现的错取。精确匹配下前者命中 *np_parent* 的别名，后者不命中任何指标（*tests/test_quarterly_ebit.py* 中 *test_sign_note_suffix_matches_metric*）。

作用域别名。 “合计”是非经常性损益表的合计行，但年报里有上百张表都有“合计”。`SCOPED_ALIASES` 把“合计”限定为只在标题含“非经常性损益”的表中生效：`match_label(" 合计")` 为空，`match_label(" 合计", " 九、非经常性损益项目及金额")` 为 `non_recurring`。

发生额与时点数。 `is_flow(m)` 判断指标能否跨期相减：金额类且不属于资产负债表或派生指标。它决定季度拆分（第 7 节）作用于哪些指标；资产负债表科目、每股收益、比率、股本永远不拆分。

6.5 表头期间识别

规则通道需要知道每一列对应哪个期间。`parse_period(cell, fiscal_year)` 把表头单元格映射为期间标识或特殊标记，规则见表 25。

几个细节来自真实年报：

- **表头折行。**PDF 中“本期比上年同期增减”常被折成“本期比上年同期增\n 减 (%)”，所以先去掉全部空白再判断（`test_parse_period_skips_ratio_columns_and_wrapped_headers`）。
- **结构比列。**资产负债表分析表的“本期期末数占总资产的比例（%）”含“期末”，若不先排除会被识别成本期数值列，把占比当成金额。规则 2 放在规则 5 之前。

表 25　表头期间识别规则（extract.parse_period），按判断顺序排列。Y 为报告年度。

顺序	条件（去掉全部空白后）	输出	例
1	含“增减”“变动”“同比”“变化”	chg	本年比上年增减 (%)
2	含“比例”“比重”“占”	无（结构比列）	本期期末数占总资产的比例
3	含“调整前”	skip（并记追溯调整 issue）	2024 年（调整前）
4	含 20xx 年份	该年份	2025 年 12 月 31 日
5	无年份，含“本期/本年/期末/本报告期”	Y	期末余额
6	无年份，含“上期/上年/期初/上年同期”	$Y-1$	期初余额
7	无年份，含季度关键词	Y	第一季度（1-3 月份）
8	年份确定后再查季度/半年/前三季度关键词	加后缀 Q1⋯Q4、H1、Q1-3	2024 年 1-9 月

- **变动列先于年份。**“2025 年比 2024 年增减”同时含年份与“增减”，必须先判为变动列。
- **“期末”作为季度关键词。**季度关键词表 _Q 中“3 月 31 日”“6 月 30 日”“9 月 30 日”分别对应 Q1、H1、Q1-3，用于识别季报资产负债表的日期表头。

表头行由 _header 在表格前 3 行中寻找：第一行中除首列外存在一个合法期间（不是 chg、skip、空）的行即为表头。找不到表头且没有续表表头的表格整张跳过。

财报自带的“本年比上年增减”列不丢弃：规则通道把它记为 < 指标 >.reported_yoy 伪指标（期间取表头中最新的期间），供第 7 节的“列示同比 vs 计算同比”勾稽使用。这是一个廉价但有效的抽取自检：如果两列期间识别反了，或者单位错了一个量级，列示同比与计算同比会立刻不一致。

6.6　规则通道与真实年报版式难点

规则通道 extract_rule(doc, fiscal_year) 的主循环很短（算法 3），真正的工作量在解析阶段对表格结构的修复。本节先给主循环，再逐个讲版式难点。

规则通道产生的每条事实都有完整的 $\sigma = (\text{source}, \text{page}, \text{chunk_id}, \text{row}, \text{col})$，是全系统定位最细的事实。续表上的事实还带 continued_from，指向表头所在的上一页表格。

6.6.1　表格白名单与合并/母公司区分

_is_source_table 只接受三类表：(1) 标题含“主要会计数据”“主要财务指标”“分季度”“非经常性损益”“资产负债表”“利润表”“现金流量表”“补充资料”之一；(2) 没有标题（解析器没找到时不贸然丢弃）；同时排除 (3) 标题含“母公司”的表，以及 (4) 首行任一单元格含“关联方”的表（行名是科目、列是关联方，结构完全不同）。

白名单是在真实年报上调出来的。附注里的受限资产表、费用明细表、存货分类表都有“货币资金”“销售费用”“存货”这样的行名，其数值是明细或期末某一分类，与报表科目口径不同。在白名单引入之前，这些表让同一键出现多个不同取值，规则通道内部就会产生大量“同一指标多处取值不一致”的冲突。

标题识别对合并与母公司的区分至关重要。茅台年报中合并资产负债表在第 56 页、母公司资产负债表在第 59 页，合并利润表在第 61 页、母公司利润表在第 63 页，两组表科目完全同名。test_statement_captions_distinguish_parent_and_consolidated 断言解析出的标题正确区分了四张表。

营业收入与营业总收入。 茅台合并利润表有“一、营业总收入”与“其中：营业收入”两行：营业总收入包含集团财务公司的利息收入，而“主要会计数据”和“分季度数据”里的营业收入是不含的口径。若两行都配到 revenue，同一键就有两个值。规则通道的处理是：同表中两者都出现时只取“营业收入”。茅台 2025 年 revenue 的取值因此唯一，为 168,838,102,514.79 元（test_rule_channel_on_real_pdf）。

补充资料中的调节项。 现金流量表补充资料把净利润调节为经营现金流，其中有“财务费用（收益以“-”号填列）”“投资损失（收益以“-”号填列）”等行。它们与利润表科目同名（清洗后都是“财务费用”），但数值是现金流口径的调节额。规则通道跳过行名含“收益以”的行；注意利润表上的“投资收益（损失以“-”号填列）”是“损失以”，不受影响。

6.6.2 稀疏网格：compact_columns

现象。 pdfplumber 依据线条与文字位置推断单元格。部分年报（宁德时代最典型）的报表线条稀疏，解析结果把一个逻辑列切成几列：科目落在第 2 列、表头“期末余额”比下面的数字向右错开一列、其余列全空。下面是按宁德时代版式构造的片段（test_compact_columns_aligns_header_and_numbers）：

	项目			期末余额			期初余额	
	货币资金		333,512,927			303,511,993		
投资活动产生的现金流量净额			-79,421,912			-19,213,430		

按列号直接对齐，“期末余额”（第 5 列）下面没有数字，数字所在的第 4 列没有表头，规则通道一个数都取不到；第三行的科目名甚至跑到了第 1 列。

算法。 观察到：同一个逻辑列被切成的几个物理列中，任意一行至多一个非空。据此做贪心的从左到右分组（算法 4）。

上例中第 1、2、3 列在每一行至多一个非空，合成一组（科目列）；第 4、5、6 列合成一组（期末余额：表头在第 5 列，数字在第 4 列）；第 7、8、9 列合成一组（期初余额）。结果正好是规整的三列表：

项目	期末余额	期初余额
货币资金	333,512,927	303,511,993
投资活动产生的现金流量净额	-79,421,912	-19,213,430

命题 6.1 (规整表格不变；合并不丢信息)**.** *(a) 若 G 不含全空列且 force 为假，*Compact *返回同一对象（test_compact_columns_leaves_regular_tables_alone 断言 is）。(b) 任意情况下，G 的每个非空单元格恰好出现在 G' 的一个单元格中，且同一行的非空单元格的左右顺序保持不变。*

证明. (a) 由第 3 行直接得出。(b) 分组是对列下标 $1..w$ 的有序划分，每列恰属于一组；并入条件保证组内每行至多一个非空，故拼接不会把两个值粘在一起；组按列下标递增产生，行内顺序保持。删除的只是全空组。 □

性质 (a) 很重要：它保证修复只作用于“病态”表格，不会改变已经正确的表格。触发条件选“存在全空列”，是因为规整的报表不会有整列为空的情况（附注列在续页上全空是例外，由续表算法用 force 处理）。贪心分组不是唯一可行的划分，但在“表头错开一列”这种版式下，它恰好把表头与数字并到一起；若某行在两列都有值，分组会在这里断开，宁可多一列也不把两个值合并。

6.6.3 科目折行：`merge_wrapped_labels`

现象。 科目名长于列宽时，PDF 把它折成多行，每行成为表格的一行；数字只在第一行（或最后一行）。常见形态：

项目	2025 年	2024 年
归属于上市公司股东	72,201,282	50,744,682
的扣除非经常性损益		
的净利润		
购建固定资产、无形资产和其他长	42,344,558	31,179,943
期资产支付的现金		
流动负债：		

不处理时，第二行的行名“归属于上市公司股东”配到的是……什么也配不到（别名表里没有这个截断形式）；更糟的情况是截断形式恰好是另一个科目的别名。

判据。 `_is_fragment(row, prev_label, limit)` 判断一行是否是上一行科目名的折行尾巴。前提：本行只有标签、没有任何数值；本行不以冒号结尾（“流动负债：”是小节标题）；上一行有标签。满足前提后，以下任一成立即视为折行：

1. 上一行括号未闭合（“（”与“(”的个数多于“）”与“)”）；
2. 上一行最后一个物理行写满了列宽：limit ≥ 8 且上一行末行长度 ≥ limit − 1，其中 limit 为表内科目名单行最长字数（`_label_width`）；
3. 本行以不会作为科目开头的字开头：“的”“）”“)”“率”“号”“列”“股）”“流量”“填列”“益）”“”””“以“”。

`merge_wrapped_labels` 自上而下扫描，把满足判据的行用换行符接到上一行的标签后面。保留换行符是有意的：判据 2 要看“上一行最后一个物理行”的长度，连续折成三行时，第三行的判断要基于第二行的长度而不是拼接后的总长。匹配别名前 `clean_label` 会删掉换行符。

上例经处理后得到“归属于上市公司股东的扣除非经常性损益的净利润”“购建固定资产、无形资产和其他长期资产支付的现金”“流动负债：”三行（`test_wrapped_labels_rejoin_the_row_with_numbers`），前两行分别命中 `np_parent_deducted` 与 `capex`。

权衡（Trade-off）：启发式判据 vs. 版面坐标

另一种做法是用字符的纵坐标与行距判断两行是否属于同一单元格。pdfplumber 的 `extract_table` 已经丢掉了单元格内部的行信息，重新从字符层恢复需要自己做表格结构识别，复杂度高很多。三条判据都是“该接却没接”方向保守的：判据 3 的开头字表是在真实年报上逐个收集的，不在表中的折行（例如“期资产支付的现金”以“期”开头）靠判据 2 的“写满列宽”兜住。误接的风险在于把一个独立科目接到上一行，但独立科目行通常有数字，而前提要求本行没有数值，这排除了绝大多数误接。

6.6.4 跨页续表：`link_continuation`

现象。 合并资产负债表、利润表、现金流量表都跨 2 ~ 3 页。续页的表格*没有表头*：第 57 页有“项目 / 附注 / 2025 年 12 月 31 日 / 2024 年 12 月 31 日”，第 58 页直接从“应付账款”开始。规则通道找不到表头就整张跳过，续页上的全部科目都取不到。修复前，茅台的营业利润、存货、资产总计、经营现金流、净利润都在续页上，规则通道一律缺失（`test_rule_channel_on_real_pdf` 中的注释记录了这一点）。

更麻烦的是两页的列切分可能不同：续页的附注列全空、或续页把“2024 年度”一列劈成两列。下面是按茅台利润表版式构造的例子（`test_continuation_with_different_cell_split`）：

```
第 61 页（首页，4 列）：  [项目 | 附注 | 2025 年度 | 2024 年度]，[信用减值损失（损失以“-”| 54 | 17.00 | -23.00]
第 62 页（续页，5 列）：  [号填列）| | | | ]
                          [四、利润总额（亏损总额以“-”号填列）| | 120.00 | 110.00 | ]
                          [五、净利润（净亏损以“-”号填列）| | 90.00 | | 85.00]
```

续页第一行“号填列）”是上一页末行科目名的折行尾巴；2024 年的数字在第 4 列或第 5 列，取决于这一行被怎么切。

何时认作续表。 解析时，一张表被尝试作为续表，当且仅当同时满足：它是本页第一张表；它上方没有自己的标题；上一页末尾没有“页末标题”（有页末标题说明下一页首张表是新表）；上一张表恰好在前一页。满足这些前提后调用 `link_continuation`（算法 5）。

算法的关键在第 4 行：把上一页的原始表头和本页的原始数据*放在一起*做稀疏列合并。这样两页共享同一个列分组，表头的“2024 年度”与续页劈开的两列会被并入同一组。只有联合合并失败（列数与上一张表不一致）时，才退回到各自合并；各自合并的列数也必须与上一张表一致。上例中联合合并把续页 5 列压成 4 列，与首页一致，“号填列）”接回首页末行，得到“信用减值损失（损失以“-”\n 号填列）”；2024 年的利润总额 110.00 与净利润 85.00 都落在“2024 年度”列。

性质。

- **只记关系，不合并行。**续页的行留在续页 chunk 中，每条事实的 `page` 是它实际所在的页（`test_continuation_inherits_header_and_keeps_its_own_page`：应付账款取自第 58 页，`continued_from=c1`）。这保证规则通道的定位与模型通道的页码可以直接比较，也保证证据校验查的是正确的页。
- **链式续表。**三页的表，第三页的 prev 是第二页的续表 chunk，head 沿用第一页的原始表头（解析循环中 `last` 的第二个分量在链接成功时保持不变）。
- **拒绝错位。**列数对不上时返回 False（`test_continuation_rejected_when_columns_do_not_line_up`），续页按普通表处理；它没有表头，规则通道会跳过它。漏取的科目由模型通道补齐。

权衡（Trade-off）：续表上漏取 vs. 错配

续表链接中最危险的错误不是“取不到”，而是“取错列”：如果把 2024 年的数字放在 2025 年的列下，规则通道会给出一个看起来合理、数值却属于另一期间的事实。如果模型通道恰好也抄错了同一格，两条通道会“一致地错”；如果模型通道是对的，就会出现冲突并阻断估值，需要人来排查。因此算法的每一个出口都偏向“不认作续表”：列数必须严格相等，不允许“差一列也凑合”。规则通道覆盖率因此低于理论上限，但在三家真实年报的全部重叠键上没有出现一次错配（§6.14）。

6.6.5 同页多单位声明

现象。 年报中同一页可能有两张表用不同单位：上表“单位：亿元”的经营概况，下表“单位：元”的主要会计数据。若整页只取第一个单位，下表的全部金额会被放大 10^8 倍。

算法。 当一页的单位声明多于一种时，`_unit_marks` 用 `page.search` 取出每个“单位：”声明的纵坐标，每张表取*表格上边界以上最近的*声明；上方没有声明的表沿用上一页结束时的单位。只有一种单位时整页统一，不调用坐标检索（旧版 pdfplumber 无 `search` 时也退回整页单位）。跨页时，上一页最后一个单位声明延续到下一页没有声明的表格，这对续表是正确的。

单位只影响规则通道的换算（$\phi(u)$）。模型通道的单位来自条目自己的 `unit` 字段，缺省时才用证据 chunk 的 `unit_hint`——这一点的风险见 §6.15。

6.6.6 已知问题：季报表头

季报的表头写“本报告期”“年初至报告期末”“上年同期”。按表 25，“本报告期”命中规则 5，被识别为年度 Y，“年初至报告期末”同样含“报告期末”。结果是三季报的单季数和前三季度累计数都会被当成全年数，与年报的全年数冲突。规则通道没有可靠的办法从表头本身区分单季与累计（要结合报告类型与列的位置），因此我们选择*不让季报进入规则通道*：季报只能作为 `--supplement` 补充文件传入，并且命令行要求必须同时有 `--facts`（`__main__.main` 中的 `ap.error("--supplement 需配合 --facts")`）。季报的数字只来自模型通道，由模型按抽取规范把“本报告期”写成 `2024Q3`、“年初至报告期末”写成 `2024Q1-3`。年报的“分季度主要财务指标”表不受影响：它的表头是“第一季度（1-3 月份）”这样的形式，由规则 7、8 正确识别。

6.7 数字解析与规范化

`finagent/core/numbers.py` 是全系统共享的数字工具，内部金额统一为元（单位约定见表 9）。

`parse_number`。 输入可以是数值或字符串。字符串依次：去半角与全角空格；占位符（空、“-”“—”“–”“不适用”“N/A”“/”）返回 `None`；全角逗号转半角，Unicode 减号“−”与短横“–”转 ASCII 减号；用正则

```
^\(?[-]?\d[\d,]*(\.\d+)?\)?%?$
```

整串校验，不符合（如“增加 1.23 个百分点”）返回 `None`；括号包裹视为负数；去括号、去百分号、去千分位后转浮点。百分号*不除以 100*：14.49% 存为 14.49，与比率的内部约定一致。

整串校验是刻意的：一个单元格里如果混了文字，宁可解析失败让这一格缺失，也不从中“抠出”一个数字。年报中“增加 1.23 个百分点”这类单元格在 ROE 的变动列里很常见，它们不会被误当作 1.23% 的相对增速。

其他工具。 `UNIT_FACTORS` 给出元、千元、万元、百万元、亿元、万亿元与股、万股、亿股的换算系数；`decimals_of` 与 `half_ulp` 按原文写法的小数位数计算展示精度（式 4），供勾稽与研报纠错使用；`pct_change` 实现式 5，基期为 0 或缺失时返回 `None`；`close(a,b,rel,abs)` 是 `math.isclose` 的薄封装，两通道一致性用 `rel=1e-6, abs=0.01`。

6.8 模型通道：`facts.json` 与证据校验

6.8.1 分析师会话写什么

模型通道由分析师会话（第 11 节阶段 2）完成：它运行 `locate` 拿到建议阅读页，用 Read 工具逐页读 PDF，按 `schema` 命令给出的指标清单把数字抄进 `facts.json`。每一条是一个对象：

```
{"metric_id": "revenue", "period": "2024", "raw_value": "5,862,340,125.36",
 "unit": "yuan", "page": 106, "label": "...", "table": "...", "scope": "consolidated"}
```

（实际文件中 `unit` 写中文单位“元”，`label` 与 `table` 写原文科目名与表名；此处为避免代码清单中的中文排版问题改写。）

抽取规范对模型提出的要求都指向同一个目标——让程序能核对：`raw_value` 与原文逐字一致（保留千分位与小数位，括号负数照写）；`page` 写 PDF 物理页；`unit` 按该表的“单位：”声明写；`scope` 固定为合并口径；毛利率等派生指标不要抽取；不要换算、不

表 26　拒收理由（external._to_fact 与 _pick_doc），按检查顺序排列。

检查	拒收理由模板
文件	source_file 未作为主文件或补充文件（--supplement）传入
指标	未知或不需抽取的指标（派生指标也在此拒收）
期间	期间格式不合法（应为 2024 / 2024H1 / 2024Q3 / 2024Q1-3）
口径	取自非合并报表（scope=…），只接受合并报表口径
数值	数值无法解析
页码	缺少页码
证据	< 指标 > < 期间 >=< 原文写法 > 未在第 p 页原文中找到
单位	单位不合法（金额类单位必须可换算为元）

要相减；公司没有的科目不写，不要填 0。最后一条防止“填 0”混入真值表后在勾稽或估值中被当成真实数值。

6.8.2　条目校验 _to_fact

external.ingest 对每个条目依次检查定义 3.5 的七个条件，任一失败即拒收并给出中文理由（表 26）。理由写得具体，是因为它们会原样回到分析师会话，作为修正指令。

通过的条目成为 method="llm"、confidence="medium" 的事实，来源记录证据 chunk 的页码、chunk_id、标题、模型声明的表名以及抽取器标识（环境变量 FINAGENT_EXTRACTOR，默认 claude-code-skill）。模型通道内部，同一键出现多次时：一致则记为佐证，不一致记 high 级 issue“同一指标多处取值不一致”。年报里同一数字常在“主要会计数据”和报表中各出现一次，模型两处都抄是允许的，而且两处一致本身就是一次核对。

6.8.3　证据校验 find_evidence

find_evidence(doc, page, raw) 实现证据谓词 Ev：

1. _num_pattern(raw)：去空格、全角逗号转半角、去千分位、去首尾括号、去末尾百分号、统一减号后去掉前导负号，得到数字核 core(r)；核不是 \d+(\.\d+)? 时返回 None（直接拒收）；否则编译正则 (?<![\d.])core(?![\d])；
2. 取该页全部 chunk，表格在前、文本在后；
3. 对每个 chunk 的文本做 _norm（只删千分位逗号，保留空白），用正则搜索，返回第一个命中的 chunk。

表 27 列出若干写法在一个单元格文本 | 货币资金 | 3,884.59 | -1,234.56 | (77.10) | 上的行为（本文实测）。

为什么保留空白。 早期实现为了容忍 PDF 中数字内部偶尔出现的空格，在匹配前删除了全部空白。在招商轮船年报的船队表上，这导致相邻两格“235”与“3884.59”被拼成“2353884.59”，两个正确的抄写都因右边界不成立而被拒收。修复后 _norm 只删逗号，并以 tests/test_evidence_spacing.py 固定：五个相邻数字逐一命中，拼接串 2353884.59 不命中。

为什么符号不参与匹配。 年报对负数有三种写法：“-1,234.56”“(1,234.56)”“- 1,234.56”，模型抄写时可能统一成其中一种。符号若参与匹配，会拒收大量正确抄写；符号的正确性由数值层面保证：parse_number 按模型写的符号取值，再与规则通道和勾稽比对。若模型把负数抄成正数，证据校验会通过，但该键若同时被规则通道取到就会冲突；若只在模型通道（如政府补助），则依赖勾稽与人工复核（§6.15）。

表 27　证据匹配的边界行为（同一页只有该表格时）。

模型写的原文	结果	说明
`3,884.59`	✓	千分位双方都去掉
`-1,234.56` / `(1,234.56)` / `1234.56`	✓	符号与括号不参与匹配，三种写法都命中
`77.10`	✓	逐字一致
`77.1`	✗	右侧紧邻“0”是数字，边界不成立：要求保留原文小数位
`884.59`	✗	左侧紧邻“3”是数字：防止子串误配
`3884`	✓	右侧紧邻“.”不在右边界的排除集中（见 §6.15）
`12.34%`	✗	页面上没有 12.34

为什么查页而不查格。 证据谓词只要求数字出现在*那一页*，不要求它出现在*那一行那一列*。原因是模型通道的条目不带行列坐标：让模型给出解析器意义下的行列号既不可靠，也会把解析器的切分错误转嫁给模型。页级证据挡住了“编造”和“抄错数字”两类错误，“抄对了数字但抄错了位置”（例如把上期列当本期）则交给规则通道比对与勾稽。这一分工在第 3 节的证据谓词定义之后已有讨论；§6.14 的结果表明，两层检查叠加后，三家年报的 301 个模型核验值没有残留冲突。

权衡（Trade-off）：页级证据 vs. 单元格级证据

单元格级证据（要求模型同时给出行名，程序在该页表格中找到行名匹配且该行含此数字的单元格）更严格，可以直接挡住“看错一列”。我们评估过这一方案：(1) 模型写的行名与解析器得到的行名经常不同（折行、填列说明、全半角括号），需要再引入模糊匹配，模糊匹配又带来新的误配；(2) 续表上行名在续页、表头在上一页，单元格级证据需要跨页拼接；(3) 页级证据加规则通道比对已经覆盖了 93% 的核验值（这部分等价于单元格级核对），剩下 7% 集中在规则通道不覆盖的少数表（股本变动、政府补助、季度扣非）。边际收益不足以抵消复杂度，因此保留页级证据，把单元格级核对交给规则通道。

6.9　双通道合并

`extract.merge(rule_facts, llm_facts, prefer)` 实现定义 3.6 的合并规则，算法 6 给出完整过程。

合并的四种结局与它们对下游的影响列在表 28。

为什么冲突时保留模型通道的值。 冲突时总得暂存一个值，后续的勾稽和报告才能运行，以便把冲突与其他问题一并展示给分析师会话。保留模型通道的值是因为它来自模型逐页读原文并已过证据校验，而规则通道在冲突处往往是版式问题（错列、续表未识别）；但这只是展示用的暂存，冲突本身照样阻断估值。注释中“不静默取舍”的含义是：*取舍可以有，但必须留下记录并阻止下游使用。*

命题 6.2 (无静默取舍)**.** *设 `merge` 的输出为 (M, I)。对任意键 κ，若两通道各有一条事实且不一致，则 I 中存在一条以 κ 的两侧原文为消息、以两侧来源为证据的 issue；进而 `verify-facts` 的 `ok` 为假，`pipelines._p_analyze` 在 `--facts` 模式下抛出异常。*

证明. 由算法第 12 ~ 14 行，不一致时必追加 issue，其类型为“规则与 LLM 抽取不一致”。`cmd_verify_facts` 与 `_p_analyze` 都把类型属于 {同一指标多处取值不一致, 规则与 LLM 抽取不一致} 的 issue 计入冲突，冲突非空时分别令 `ok` 为假、抛出 `ValueError`。 □

表 28　双通道合并的结局。“阻断”指 verify-facts 返回非零、pipelines._p_analyze 抛出异常，估值不会运行。

结局	method	阻断	说明
两通道一致	rule+llm	否	保留规则通道事实（定位细到行列），模型来源记为佐证
只有规则通道	rule_table	否	多为上上年数据与 reported_yoy 伪指标；模型只抄了本期与上期
只有模型通道	llm	否	已过证据校验；规则通道不覆盖的表（股本、政府补助、季度扣非）
两通道冲突	保留 llm	是	记 issue，含两侧原文与来源，由分析师会话回原文核对
同通道多处不一致	—	是	规则通道：多表同名科目口径不同；模型通道：两处抄写不一致

这条性质是第 4 节设计规范“双通道不静默取舍”在代码层面的保证，对应第 3 节不变量 2。

一致的容差。 `close(rel=1e-6, abs=0.01)`：相对误差 10^{-6} 或绝对误差 1 分。两通道读的是同一份 PDF 的同一个数字，正常情况下应完全相等；容差只用于吸收浮点换算误差（例如“万元”乘 10^4 后的二进制舍入）。它远小于任何一种真实的抄写错误（最小的是末位差 1，即 0.01 元，恰在边界上；末位差 2 以上即判冲突）。

核心缺失。 合并后，`ingest` 检查 14 个核心指标在本期与上期是否存在。本期缺失标记为 `blocking`，上期缺失只报告。报告年度取 `fiscal_year`，缺省时取事实中最新的年度。核心指标的选择依据是下游的硬依赖：估值的历史基数（收入、成本、EBIT 的组成、资本开支、经营现金流、货币资金）、股权桥（资产、负债、权益）和每股指标。

6.10　勾稽校验在抽取中的角色

勾稽校验的定义（定义 3.8）与具体等式在第 7 节展开；这里只说明它在抽取环节的作用。`verify-facts` 在合并后立即对本期跑一次 `analyze()`，任何勾稽失败都阻断。勾稽能发现两通道比对发现不了的错误：

- **只在模型通道的数字**：例如资产负债表在续页而规则通道取不到时，“资产总计 = 负债和所有者权益总计”是唯一的自动核对；
- **两通道一致地错**：例如两条通道都把母公司报表的数字当成合并报表（白名单与 `scope` 都没拦住时），“所有者权益合计 = 归母权益 + 少数股东权益”在母公司报表上通常不成立（母公司报表没有少数股东权益），会暴露出来；
- **期间看反**：列示同比与计算同比的核对对“本期列与上期列看反”极其敏感，同比的符号会直接相反。

6.11　经营数据表：`extras`

动机。 深度研报需要的不只是三张报表：分产品、分渠道、分地区的收入与毛利率，产销量，经销商数量，吨价，分红率。这些数据格式千差万别，无法也不应该塞进 49 个指标的字典。但它们同样会出现在研报正文里，同样需要可追溯。`finagent/tasks/financials/extras.py` 为它们提供一条单独的通道：模型按原表逐字抄录，程序逐格回查；这些数据*不进入真值表*、不参与估值的基础计算（分部预测除外，见第 8 节），只作为运行产物 `financials/extras.json` 供研报引用，guard 也以它为出处。

表 29　三家真实年报经营数据表（data/extractions/< 公司 >_2025/extras.json）预检结果（本文重跑）。"比率"列为声明数 / 预检中计算数（其余引用真值表，正式运行时计算）。

公司	表	行	核对通过的格子	比率（声明 / 预检计算）	拒收
贵州茅台	37	108	286	93 / 63	0
宁德时代	40	141	343	91 / 62	0
招商轮船	36	143	447	55 / 18	0
合计	113	392	1,076	239 / 143	0

格式。 一份 extras.json 包含若干表与若干比率。表有 id、title、page 与列定义（列名与单位）；每行有 label、可选 alias（正文简称，如"i 茅台"）、可选行级 page，以及与列一一对应的原文写法。比率给出分子、分母引用与运算。

逐格回查。 verify_extras 对每张表：先检查列单位是否可接受（_unit_ok：在换算表中的金额单位，或不以"元"结尾的任意计量单位，如"%""pp""吨""载重吨""元/吨"；以"元"结尾却不在换算表中的，如"美元""千万元"，拒收——外币与少见量级必须换算后另写）；再逐行检查列数；再逐格：占位符记空，其余调用同一个 find_evidence 在行级或表级页码上回查，找不到即拒收该格。金额列换算为元，其他单位原值保存。每个通过的格子都记录自己的证据 chunk。

比率由程序算。 吨价、收入占比、分红率、同比增速、占比变动都*不允许模型自己算*，只能写成比率声明，由 _ratios 计算：

$$\text{div}: \ \frac{\text{num}}{\text{den}} \times s, \qquad \text{growth}: \ \left(\frac{\text{num}}{\text{den}} - 1\right) \times 100, \qquad \text{diff}: \ \text{num} - \text{den}.$$

系数 s 由单位决定（% 为 100，万元/吨为 10^{-4}，倍、元、元/吨、pp 为 1），其他单位必须显式写 scale；growth 只允许单位%。引用有三种：[表 id, 行名, 列名] 引用本文件中已核对的格子；["truth", 指标, 期间] 引用真值表；["ratio", 比率 id] 引用前面已算出的比率（按声明顺序，允许链式）。找不到引用、分母为零（diff 除外）、运算或单位不支持时，该比率记入拒收。

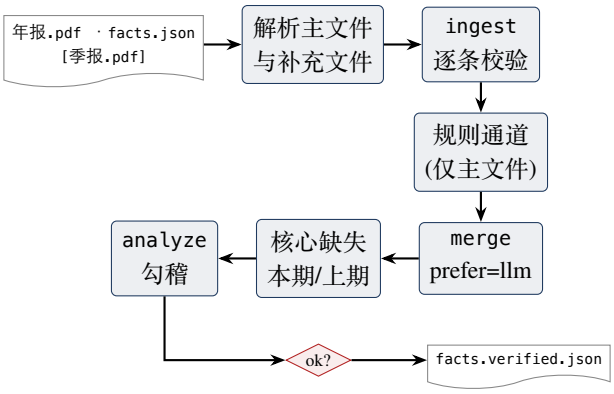
例 6.2 (茅台酒吨价)**.** *茅台 2025 年报第 18 页"主营业务分产品"表中茅台酒营业收入 146,499,906,480.49 元，第 16∼17 页销量表中茅台酒销量 46,750.66 吨。比率声明 price_moutai：分子 ["product"," 茅台酒"," 营业收入"]，分母 ["volume"," 茅台酒"," 销量"]，单位"万元/吨"。程序计算 146,499,906,480.49/46,750.66 × 10⁻⁴ 万元/吨（test_extras_verified_and_ratios_computed），研报中的"茅台酒吨价 313.36 万元/吨"由 guard 在这一比率上按展示精度核对通过；若把收入抄错两位（...480.94），该格即被拒收，理由是"未在所写页码的原文中找到该数字"。*

预检与正式运行。 verify-extra 命令在没有真值表的情况下运行（预检），引用真值表的比率以及依赖它们的链式比率被跳过（_ready），只核对格子与表内比率；流水线的 extras 步骤在真值表就绪后运行，全部比率都会计算，任何拒收都会让流水线停止。

规模。 三家真实年报的经营数据表规模见表 29，预检全部通过、拒收为 0。

6.12　季报与补充文件

多文件。 年报之外，分析常需要当年的一季报、半年报或三季报（例如用"全年 − 前三季度累计"得到第四季度单季）。季报通过 --supplement 传入，可多次指定；facts.json 中



ok = Rej= ∅ ∧ Conf= ∅ ∧ 无勾稽失败 ∧ 无本期核心缺失；退出码 0 / 1

图 9 verify-facts 的执行流程。结果（含全部事实、issue、拒收与缺失清单）写入 facts.verified.json，摘要打印到标准输出供模型会话读取。

来自季报的条目用 source_file 写明文件名（按文件名匹配，不要求路径一致），证据在该文件中核对；未写 source_file 的条目视为来自主文件。条目指向的文件没有传入时整条拒收，理由是“未作为主文件或补充文件传入”。test_supplement_must_be_passed 验证同一份 facts.json 不带 --supplement 时 verify-facts 返回 1，带上后返回 0。

只走模型通道。 如 §6.6.6 所述，规则通道只跑主文件。季报事实全部是 method="llm"，期间标识由模型按抽取规范映射：一季报“本报告期”为 Q1、半年报为 H1、三季报“本报告期”为 Q3 单季、“年初至报告期末”为 Q1-3，季报资产负债表的期末数与该报告的累计期同期间。抽取规范还特别提醒：三季报没有第三季度单季的经营现金流，不要把累计数填成 Q3。

单季拆分。 季报事实进入真值表后，单季拆分与分季度勾稽由 analyze 完成（第 7 节）。样例中三季报的营业成本只有前三季度累计，程序推出第四季度营业成本 = 4,034,118,660.40 − 2,815,640,000.00 元，公式记为“全年 − 前三季度累计”；年报分季度表的第四季度营业收入与“全年 − 三季报前三季度累计”相互印证，勾稽全部通过（test_supplement_q4_equals_full_year_minus_q1_3）。

6.13 verify-facts 与 verify-extra 的完整流程

两个命令是分析师会话与计算引擎之间的接口，也是流水线阶段 2、5 的闸门判据（附录 D）。它们不生成运行目录，可以反复调用。

verify-facts。 python -m finagent verify-facts --input 年报.pdf --year 2025 --facts facts.json [--supplement 季报.pdf] [--out 路径]：

1. 解析主文件与每个补充文件；
2. ingest：逐条校验（表 26）、模型通道内去重、跑规则通道、merge(prefer="llm")、统计拒收、计算核心缺失；
3. 若有事实，以 --period（缺省为最新的有营业收入的年度）跑 analyze()，取勾稽失败项；
4. 汇总 ok = 无拒收 ∧ 无冲突 ∧ 无勾稽失败 ∧ 无本期核心缺失（定义 3.9）；
5. 把完整结果与检查摘要写入 facts.verified.json（缺省与 facts.json 同目录），打印摘要（ok、统计、拒收、冲突、勾稽失败、缺失清单）；

表 30　三家真实年报的双通道结果（当前代码重跑）。“佐证”为 rule+llm，即模型核验值中规则通道独立取到且一致的数量；“仅规则”多为上上年数据与列示同比伪指标。

公司	模型条目	拒收	规则事实	佐证	仅模型	仅规则	冲突	勾稽（通过/总）
贵州茅台	98	0	136	90	8	18	0	20/20
宁德时代	106	0	119	101	5	8	0	15/15
招商轮船	97	0	144	89	8	23	0	21/21
合计	301	0	399	280	21	49	0	56/56

6. ok 为真返回 0，否则返回 1。

分析师会话的工作方式是“抄写—校验—按清单修正—再校验”，直到返回 0。流水线状态检查（report_pipeline status）对阶段 2 的判据除了 ok，还要求 facts.verified.json 晚于 facts.json，防止改了抽取文件却没重新校验。

流水线中的同一检查。 正式运行 python -m finagent all ... --facts facts.json 时，extract 步骤调用同一个 ingest，analyze 步骤在算完勾稽后做同样的四项检查，任一不满足即抛出“模型抽取未通过校验，请先运行 verify-facts 并处理拒收、冲突、勾稽失败和核心缺失项”。因此即使跳过了 verify-facts，未通过校验的抽取也无法进入估值。不带 --facts 时只走规则通道（样例回归与引擎测试），此时不做阻断检查，冲突与勾稽结果照常写进报告。

verify-extra。 python -m finagent verify-extra --input 年报.pdf --extras extras.json：调用 verify_extras（无真值表，预检模式），打印 ok、表数、核对通过的格子数与拒收清单；有拒收返回 1。流水线的 extras 步骤在真值表就绪后再跑一次完整版本，拒收即停止。

追溯。 正式运行时，模型提交的原始 facts.json 被原样写入运行目录 financials/skill_extraction.json，合并后的事实写入 financials/facts.json 与 facts.csv，全部文件读写经 tracing 记录哈希。评审复现不需要调用任何模型：仓库 data/extractions/< 公司 >_< 年度 >/ 保存了模型写下的抽取文件，用 --facts 重跑即可重新做全部证据校验、规则比对与计算（data/extractions/README.md）。

6.14　真实年报上的结果

样例回归。 虚构样例公司“星河智造”（data/samples/annual_report_2024.md）的 35 个金标准字段（eval/gold_financials.json）在纯规则通道下全部正确取到（eval/run_eval.py：字段准确率 100.0%，35/35）。样例内部完全自洽，只用于回归，不代表真实版式上的效果。

三家真实年报。 贵州茅台、宁德时代、招商轮船 2025 年报的模型抽取文件由分析师会话写出并随代码提交。表 30 是本文在当前代码上对这三份文件重新运行 ingest 的结果。

三家共 301 个模型核验值，全部通过证据校验；规则通道独立取到并一致的 280 个，覆盖率 93.0%；两通道都有值但不一致的键为 0；全部 56 项勾稽通过。仓库提交记录（aee4274）在稍早版本的抽取文件上报告的口径是“314 个核验值，规则通道取到 293 个（原 128 个），错配 0”，覆盖率 93.3%；两者的差别来自其后抽取文件的修订，结论一致。

规则通道没取到的 21 个值。 逐个看“仅模型”的事实（表 31），它们集中在规则通道已知不覆盖的位置，与项目文档“已知问题”一节的记录一致。

表 31　规则通道未取到的 21 个模型核验值。

指标	公司	个数	原因
总股本 `total_shares`	三家	6	在股本变动表（茅台第 47 页等），表头不是期间，规则通道不识别
政府补助 `gov_subsidy`	茅台、招商	4	非经常性损益表中的科目名带长括号说明，别名不完全覆盖各家写法
季度扣非净利润	茅台、招商	8	分季度表里写成“归属于上市公司股东的扣除非经常性损益后的净利润”等别名变体
折旧 `depreciation`	宁德	2	第 201 页补充资料版式
资产总计 2023	宁德	1	主要会计数据表中的上上年数据位置特殊

表 32　版式修复前后的规则通道（“佐证”为模型核验值中被规则通道独立取到且一致的数量）。

	规则事实数		佐证数	
公司	修复前	修复后	修复前	修复后
贵州茅台	98	136	60	90
宁德时代	0	119	0	101
招商轮船	97	144	55	89
合计	195	399	115	280

修复前后。 表 32 比较了版式修复（稀疏列合并、续表链接、折行合并）前后的规则通道。修复前的数字取自仓库中更早生成、随代码提交的 `facts.verified.json`，修复后为表 30 的重跑结果。最大的变化在宁德时代：修复前规则通道一个事实也取不到，原因正是 §6.6.2 的稀疏网格；茅台与招商轮船的提升主要来自续表链接，三张主表的续页科目从“全部缺失”变为可取。

如何读这些数字。 “错配 0”的含义是：在两条通道都给出值的 280 个键上，没有一个键的两个值不同。它不是对“抽取完全正确”的证明——两条通道可能一致地错——但两条通道的出错机制几乎独立（一个是解析器的结构错误，一个是模型的阅读错误），一致地错需要两者在同一格上犯同一个错，概率很低；再叠加勾稽，资产负债表恒等式、非经常性损益恒等式、列示同比、分季度加总四组共 56 项核对全部通过。三家年报只是开发集，结论的外推受限于样本量；更大规模的评测集正在收集中（第 15 节）。

一个样例：茅台的营业收入。 茅台 2025 年营业收入 168,838,102,514.79 元在真值表中的主来源是第 6 页“近三年主要会计数据和财务指标”表（规则通道，带行列坐标），佐证来源有三处：第 9 页的同一数字，以及第 61 页合并利润表“其中：营业收入”行（规则通道与模型通道各一次；营业总收入被排除）。同表列示的同比 −1.21% 与计算同比 $(1{,}688.38 - 1{,}708.99)/1{,}708.99 = -1.206\%$ 在 $\frac{1}{2}$ ulp +0.01 容差内一致。这个数字在真值表中有四处来源记录和一项勾稽。

6.15 失败案例与边界情况

失败案例：整数写法的前缀匹配

数字边界正则的右边界只排除数字，不排除小数点（`(?![\d])`）。因此原文是“3,884.59”时，模型若写成整数“3884”，证据校验会通过（表 27）。这是有意的妥协：年报正文中的整数常紧跟句号（“员工 3,884 人。”），把小数点也列为右边界会拒收这类正确抄写。代价是“截掉小数部分”的抄写不被证据层挡住。实际影响有限：截断后的数值与规则通道（若覆盖）差值远超容差，会判冲突；报表金额都带两位小数，抽取规范要求保留小数位。更严格的做法是右边界改为“非数字，且若为小数点则其后不是数字”，我们记为待改进项。

失败案例：模型声明的单位未与页面单位核对

`_to_fact` 取单位的顺序是“条目的 `unit` 字段，缺省才用证据 chunk 的 `unit_hint`”，两者不一致时不报错。若模型把“单位：万元”的表格写成“元”，证据校验通过、数值小了 10^4 倍。防线依次是：规则通道覆盖的键会冲突；资产负债表恒等式要求同一张表的科目单位一致才能成立；列示同比对单位不敏感（分子分母同时缩放），挡不住“两年都写错单位”的情况。仅模型通道的指标（如政府补助）缺少自动防线。把 `unit` 与 `unit_hint` 比对并在不一致时拒收是直接的改进，但 `unit_hint` 本身来自整页或最近声明，同页多单位时可能不准，需要先评估误拒率。

失败案例：页级证据接受“看错一列”

资产负债表“期末余额 / 期初余额”两列都在同一页，模型把期初数写成本期，证据校验通过。抽取规范把“期间列”列为第二个易错点。规则通道覆盖时冲突；不覆盖时，若是资产负债表科目，恒等式通常仍成立（两列各自平衡），挡不住。这正是 §6.8 权衡中单元格级证据的优势所在。在三家真实年报的开发过程中，这类错误出现过 2 次，都被规则通道的冲突检出。

失败案例：表格标题识别失败导致白名单失效

无标题的表默认被视为来源表（白名单条件 2）。若一张附注明细表的标题没被识别到，又沿用了上一张主表的标题，它会被当成主表取数。在真实年报上，这表现为规则通道内部的“同一指标多处取值不一致”，会阻断，并在 issue 中给出两处来源，便于定位。它不会静默污染真值表，但会产生需要人工判断的阻断；当前三家年报上该类 issue 为 0。

版式之外的边界。 (1) 招股书（宇树科技）没有“主要会计数据和财务指标”这一节，而是“主要财务数据和财务指标”，`locate` 的 `key_data` 关键词“主要财务指标”仍能命中；报告期为三年一期，期间标识照常适用。(2) 港股年报（农夫山泉）为英文或繁体版式、单位为“人民币千元”，不在当前适配范围内。(3) 追溯调整：主要会计数据表含“调整前 / 调整后”两列时，调整前列被标为 `skip` 不取数，并记一条 medium 级 issue“追溯调整”，提醒同比使用调整后数据；该 issue 同时被第 7 节的会计口径信号 S8 引用。

6.16 小结

本章的抽取设计可以概括为三层约束：模型抄写的数字必须在所写页码找到（证据层），必须与独立的规则通道一致（交叉层），必须满足会计恒等式与财报自带的同比（勾稽层）；任何一层的问题都以带来源的 issue 形式阻断估值，而不是被静默修正。为了让交叉层真正起作用，规则通道必须在真实年报上有足够覆盖，这推动了稀疏网格合并、跨页续表链接、科目折行合并与同页多单位定位等一系列针对真实版式的算法；它们共同的原则是“宁可漏取，不可错配”。在三家真实年报的 301 个模型核验值上，证据层零拒收，交叉层覆盖 93.0% 且零冲突，勾稽层 56 项全部通过。得到的真值表是第 7 节财报分析、第 8 节估值与第 9 节研报纠错的唯一数据来源。

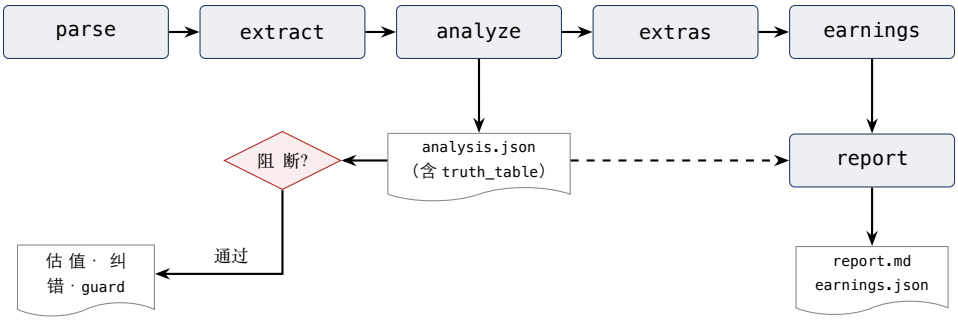


图 10　财报分析流水线。analyze 产出的真值表是估值、研报纠错与 guard 的唯一数据来源；带 --facts 运行时，抽取未通过（拒收、冲突、勾稽失败、本期核心缺失）在 analyze 步骤即抛出异常，下游不会运行。

7　财报分析

第 6 节得到的是一组带来源的事实。本章讲 LEDGER 怎样把它们变成财报分析选题要求的分析：真值表的构造与派生（§7.2），同比与环比（§7.3），单季拆分与分季度勾稽（§7.4），勾稽校验（§7.5），异常信号（§7.6），利润变动归因桥（§7.7），实际值与一致预期、业绩预告的比较以及 EPS 质量筛查（§7.8），经营数据（§7.9），最后是报告输出（§7.10）与样例、真实公司上的结果（§7.11）。

本章的全部计算在 finagent/tasks/financials/analyze.py 与 earnings.py 中完成，不调用模型。模型在财报分析中的角色只有两处：在抽取阶段读和抄（第 6 节），在写作阶段解读程序给出的数字（第 11 节），后者的每个数字都要经过 guard 回到本章的产物中核对（第 10 节）。问题的形式化定义（产物 A_2 的各组成、归因桥的闭合约束式 6、勾稽定义 3.8）见第 3.5 节，本章侧重算法、阈值的金融含义与实现细节。

7.1　流水线与产物

财报分析流水线（finagent/pipelines.py）由六个按顺序注册的步骤组成（图 10）：parse（解析）→ extract（抽取与合并）→ analyze（真值表、变动、勾稽、信号、单季）→ extras（经营数据）→ earnings（归因桥、预期对比、EPS 质量）→ report（Markdown 报告）。每一步的输出写入运行目录 runs/<run_id>/financials/，并由 tracing 记录哈希。

analyze() 的返回值是一个字典，字段与第 3.5 节产物 A_2 的对应关系如表 33。

7.2　真值表的构造

FactTable。　真值表实现为 FactTable：以 (m, k) 为键的单元格字典，外加每个单元格的“列示依赖集”quarter_basis。它提供三个操作：v(m,k) 取值；src(m,k) 取来源（指标、期间、原文写法与来源定位）；add_derived(m,k,value,formula,inputs) 登记派生值。派生单元格的 method 为 derived，formula 是中文公式（如“(营业收入-营业成本)/营业收入”“全年 − 前三季度累计”），source 是 {"derived_from": [各输入的 src]}，即派生值的来源是它的输入的来源列表，可以递归展开回原文。

不变量 21 (派生不覆盖列示)**.** *add_derived 在目标单元格已存在或值为空时直接返回。因此财报列示的数值永远不会被程序推算的数值替换；推算值只填补空缺，与列示值的差异只能通过勾稽暴露。*

这条不变量是定义 3.7“规则只在目标单元格不存在时触发”的实现。它的直接后果是 _derive 可以安全地重复调用：analyze() 在确定分析期间后会再调一次 _derive，第二

表 33　`analyze()` 与 `earnings.run()` 的输出字段。

字段	对应	内容
`period`	—	分析期间；未指定时取有营业收入的最新年度
`truth_table`	$\mathcal{T}$	全部单元格（列示与派生），每格带来源或公式与输入来源
`changes`	Chg	19 个关键指标与 6 个比率的本期值、同比基期与变动、环比基期与变动
`tieouts`	Tie	勾稽结果：检查名、状态、左右两侧、差额、证据
`signals`	Sig	异常信号 S1–S8：类型、严重度、标题、事实、推论、证据
`quarterly`	Qtr	单季表：逐季数值、“列示 / 推算”标记与公式
`summary`	—	勾稽失败数、高严重度信号数、信号总数
`profit_bridge`	Earn	利润变动归因桥：十个驱动项、闭合误差、非经常性损益贡献、前三大驱动
`beat_miss`	Earn	实际 vs 一致预期 / 内部预期
`guidance`	Earn	实际 vs 业绩预告区间
`eps_quality`	Earn	EPS 质量筛查结论与触发项
`limitations`	—	缺少输入时跳过的分析

表 34　派生指标（`analyze._derive`）。比率存为百分数值；净现比为倍数。

指标	公式	说明
毛利率 `gross_margin`	$(\text{Rev} - \text{COGS})/\text{Rev} \times 100$	营业收入非零且营业成本存在
净利率 `net_margin`	$\text{NP}/\text{Rev} \times 100$	净利润为合并口径（含少数股东）
销售 / 管理 / 研发费用率	$\text{Exp}_e/\text{Rev} \times 100$	三个费用率
资产负债率 `debt_ratio`	$\text{Liab}/\text{Assets} \times 100$	时点数，每个期末
净现比 `ocf_to_np`	$\text{OCF}/\text{NP}_{\text{parent}}$	分母为归母净利润
非经常性损益	$\text{NP}_{\text{parent}} - \text{NP}_{\text{ded}}$	只在未列示时派生
折旧摊销 `da`	固定资产折旧 + 无形资产摊销	摊销缺失按 0 计；供估值使用

次调用只会补上新出现的空缺，不会改变任何已有单元格。

派生指标。 `_derive` 先做单季拆分（§7.4），再对每个期间计算表 34 的派生指标。单季拆分在前，所以单季毛利率、单季净现比也能得到——只要单季的收入与成本都已列示或推出。

分析期间的选择。 未指定 `--period` 时，分析期间取“有营业收入的最新年度”。只有年报时就是报告年度；带了季报时，季报期间（如 `2025Q1–3`）不会抢占年度分析的位置。需要分析季度时显式传 `--period 2025Q3`，环比基期由 β_{QoQ} 给出。

唯一数据源。 `truth_table` 被写入 `analysis.json` 后，下游一律从它取数：估值的历史基数（第 8 节）、研报纠错的真值（第 9 节）、guard 对文稿数字的出处核对（第 10 节）、经营数据中引用真值表的比率（§7.9）、earnings 步骤的归因桥与预期对比。没有任何下游模块重新抽取或重新计算基础数字，这是第 4 节“一张真值表”规范的落地（不变量 3）。

7.3　同比与环比

基期。 `prior_periods(k)` 返回 $(\beta_{\text{YoY}}(k), \beta_{\text{QoQ}}(k))$：同比基期为上一年的同一期间后缀（`2025` → `2024`，`2025H1` → `2024H1`，`2025Q1–3` → `2024Q1–3`）；环比基期只对单季定义，`Q1` 的环

比基期是上一年的 Q4。

变动。 _changes 对 19 个关键指标（营业收入、营业成本、税金及附加、四项费用、营业利润、净利润、归母与扣非归母净利润、非经常性损益、经营现金流、货币资金、应收账款、存货、合同负债、资产总计、基本每股收益）与 6 个比率（毛利率、净利率、三项费用率、资产负债率）计算变动：

$$\text{金额与每股类：}\quad \mathrm{YoY} = \Delta\%(v_k, v_{\beta_{\mathrm{YoY}}(k)}) = \frac{v_k - v_\beta}{|v_\beta|} \times 100; \qquad \text{比率类：}\quad \Delta_{\mathrm{pp}} = v_k - v_\beta.$$

QoQ 同理。分母取绝对值（式 5）的含义值得说明：某公司上年归母净利润 −100、本年 −50，按 $(-50-(-100))/(-100)$ 计算得 −50%，读者会以为利润“下降了一半”；用绝对值作分母得 +50%，与“亏损收窄”的经济含义一致。这是 A 股定期报告的惯例，pct_change(50, -100)=150 的测试固定了这一行为。比率类用百分点差而不是相对变动：毛利率从 31.22% 到 31.19% 是“下降 0.03 个百分点”，而不是“下降 0.10%”。混用这两种口径是研报中最常见的数字错误之一，第 9 节的纠错把它单独列为错误类型。

7.4 单季拆分与分季度勾稽

动机。 A 股定期报告只披露累计数：一季报是单季，半年报是 1–6 月，三季报同时给出 7–9 月单季与 1–9 月累计，年报给出全年，另有“分季度主要财务指标”单季表（但只有四五个指标）。分析“第四季度发生了什么”需要第四季度的单季利润表，而它在任何一份报告里都不完整存在：只能从年报全年数减去三季报的前三季度累计数得到。这类相减在研报中常由分析师心算或在表格里手算，容易减错期间（例如用全年减去 7–9 月单季）。

规则。 季度规则见定义 3.7，实现为 QUARTER_RULES 中的五元组（目标、左操作数、右操作数、符号）：

$$\mathrm{H1} = \mathrm{Q1} + \mathrm{Q2}, \quad \mathrm{Q1\text{-}3} = \mathrm{H1} + \mathrm{Q3}, \quad \mathrm{Q2} = \mathrm{H1} - \mathrm{Q1}, \quad \mathrm{Q3} = \mathrm{Q1\text{-}3} - \mathrm{H1}, \quad \mathrm{Q4} = \mathrm{Y} - \mathrm{Q1\text{-}3}.$$

只作用于发生额指标（is_flow，§6.4）：资产负债表科目是时点数，相减没有意义；每股收益的分母（加权股数）各期不同，不能相减；比率不可加。test_stock_items_eps_and_ratios_not_split 断言资产总计与每股收益不产生 Q4。

不动点算法。 _split_quarters（算法 7）对每个发生额指标、每个年度反复扫描五条规则，直到一轮扫描没有新单元格。同时维护每个单元格的列示依赖集 $B(\kappa)$：列示单元格 $B(\kappa) = \{\kappa\}$；派生单元格是两个操作数依赖集的并。

每个指标每年至多七个期间，每条规则至多触发一次，循环至多 5 轮便停止（命题 3.1(a)）。注意规则只在两个操作数都在 B 中（即都是发生额且已知）时触发，比率与时点数从不参与。

例 7.1 (样例：年报 + 三季报)**.** *样例的三季报合并利润表只有 1–9 月累计（data/samples/q3_report_2024.md），年报只有全年。营业成本的单元格是 2024 与 2024Q1–3；规则 Q4 触发，得到*

$$COGS_{Q4} = 4{,}034{,}118{,}660.40 - 2{,}815{,}640{,}000.00 = 1{,}218{,}478{,}660.40 \text{ 元},$$

公式记为“全年 − 前三季度累计”。营业收入另有 Q1 与 H1 时，Q2、Q3 依次被推出（test_single_quarters_derived_from_cumulative_reports：全年 1000、前三季度 700、上半年 450、一季度 200 推出 Q4=300、Q3=250、Q2=250）。第四季度营业收入与营业成本都已知后，_derive 随即给出第四季度单季毛利率。

分季度勾稽。 当某个单季既有列示值，又能由累计数相减得到时，两者应当相等。`_tieouts` 对加法以外的三条相减规则逐个检查（加法规则产生的是累计数，不用于核对单季）：

> 对规则 $t = a - b$：若 $B(\kappa_t) = \{\kappa_t\}$（目标是列示值），$\kappa_a, \kappa_b \in B$，且 $\kappa_t \notin B(\kappa_a) \cup B(\kappa_b)$，则检查 $T[\kappa_t] = T[\kappa_a] - T[\kappa_b]$，相对容差 10^{-4}。

第三个条件排除“自己和自己比”：若年报分季度表给出了全部四个单季，程序会由 Q1、Q2 加出 H1，再加 Q3 得到 Q1-3；此时 Q2 = H1 − Q1 的核对两边都来自同一张分季度表，恒成立，没有信息量，必须跳过；而 Q4 = 全年 − Q1-3 的核对用到了独立来源的全年数，有信息量，保留。`test_reported_quarter_checked_against_cumulative_difference` 断言只有分季度表时恰好产生一项第四季度核对，不产生第二、三季度核对。

命题 7.1 (分季度核对的非平凡性)**.** *若执行了一项分季度核对 $\kappa_t = \kappa_a - \kappa_b$，则等式两侧依赖的列示单元格集合不相交：$\{\kappa_t\} \cap (B(\kappa_a) \cup B(\kappa_b)) = \emptyset$。因此该核对不会因为构造方式而恒成立；它成立当且仅当列示数据满足相应的会计关系。*

证明. 由执行条件 $\kappa_t \notin B(\kappa_a) \cup B(\kappa_b)$ 与 $B(\kappa_t) = \{\kappa_t\}$ 直接得出。右侧是若干列示单元格的有符号和，左侧是另一个列示单元格；二者没有公共变量，等式是对列示数据的一项真实约束。 □

真实年报上的分季度核对。 三家真实年报的分季度表都列示了营业收入、归母净利润、扣非归母净利润、经营现金流四个指标的四个单季，每家因此有 4 项“第四季度 = 全年 − 前三季度累计”核对，等价于“四个单季之和 = 全年”，全部通过。宁德时代年报以千元列示，扣非归母净利润的核对差额为 1,000 元（即末位 1 千元的舍入），在相对容差 10^{-4} 下通过；这说明相对容差的设置是必要的——单季数与全年数各自四舍五入到千元后，加总差一个末位是正常的。

单季表。 `_quarterly` 为分析年度输出单季表：发生额类关键指标加单季毛利率，逐季给出数值、来源方式（`reported` / `derived`）与派生公式。报告中派生值标“*”，以便读者区分“财报写的”和“程序算的”。样例年报中非经常性损益的四个单季全部标“*”：它们不是来自季度表，而是由单季归母净利润减单季扣非归母净利润得到——恒等规则作用在季度规则的输出上，两类规则的组合由同一个闭包自然完成。

7.5 勾稽校验

勾稽的定义与容差见定义 3.8。`_tieouts` 对分析期间执行四组检查（表 35），状态为通过、失败或无法评估（输入缺失）。每项检查都附证据：参与比较的各单元格的来源。

为什么核对财报列示的同比。 年报“主要会计数据和财务指标”表每个指标都有一列“本年比上年增减”。抽取阶段把它保存为 `< 指标 >.reported_yoy`（§6.5），这里与程序用两期数值计算的同比比较。这项检查几乎免费，却对两类最难发现的抽取错误极其敏感：本期列与上期列看反时，同比的符号相反；两期中只有一期单位抄错时，同比偏离几个数量级。容差取列示值的半个末位单位加 0.01 个百分点：列示值“14.49%”的展示精度是 0.01，任何与之四舍五入一致的计算值都应在 0.005 + 0.01 以内。

比率的列示变动是百分点差。 ROE 一类比率，年报的“本年比上年增减”列示的是百分点差（宁德时代 2025 年加权平均 ROE 24.91% 对 24.13%，列示“0.78%”），而不是相对增速。早期实现按相对增速 $(24.91 - 24.13)/24.13 = 3.23\%$ 核对，把正确抽取判为勾稽失

表 35　勾稽检查（analyze._tieouts）。容差 $\max(\epsilon|r|, 1\text{ 元})$；列示同比的容差单位是百分点。

组	等式	容差 ϵ	能发现的抽取错误
资产负债表	资产总计 = 负债和所有者权益总计	10^{-6}	母公司与合并混取、续表错列
	负债和所有者权益总计 = 负债合计 + 所有者权益合计	10^{-6}	同上；单位错一个量级
	所有者权益合计 = 归母权益 + 少数股东权益	10^{-6}	误取母公司报表（无少数股东权益）
非经常性损益	列示合计 = 归母净利润 − 扣非归母净利润	10^{-4}	“合计”取错表；扣非取错口径
列示同比	财报“本年比上年增减”vs. 程序计算	$\frac{1}{2}\,\mathrm{ulp}(r) + 0.01$	本期 / 上期列看反；单位错
分季度	列示单季 = 累计相减	10^{-4}	季度与累计期间混淆；追溯调整

败、阻断了估值。修复后对 kind=pct 的指标用百分点差核对（test_ratio_reported_change_is_percentage_points）。这是一个“程序比数据更容易错”的例子：勾稽本身也需要按财报惯例写对。

“1 元”下限。 容差下限 1 元用于吸收以元为单位、两位小数的报表在加总时的分位舍入；对于以千元列示的报表，相对容差项 $\epsilon|r|$ 远大于 1 元，下限不起作用。

无法评估。 输入缺失时检查记为 not_assessable 而不是通过。verify-facts 只把“失败”作为阻断条件；“无法评估”说明某个科目没被抽取，若该科目是核心指标，会在核心缺失检查中单独阻断。

7.6　异常信号

设计。 异常信号是一组确定性的阈值规则，每条规则对应一个财务分析中公认的“需要解释”的现象。信号不是结论，而是待验证事项：它的 fact 字段只陈述数字，inference 字段给出常见解释方向，evidence 字段列出参与判断的单元格来源。事实与推论分两个字段存放，报告中分两行展示，对应比赛“区分事实、推论与观点”的要求。阈值写在代码里，不由模型判断，因此同样的数据永远触发同样的信号。

表 36 列出全部规则。记 $g(m)$ 为指标 m 的同比（%），NR 为非经常性损益，NP_p 为归母净利润，NP_d 为扣非归母净利润。

阈值的选择。 阈值是经验值，不是从数据中拟合的。我们的取舍原则是：中等严重度的阈值应当在多数公司上不触发，从而让触发本身有信息量；高严重度留给“几乎一定要在研报中解释”的情形。

- **S1 的 15% / 30%**。非经常性损益占归母净利润 15% 以上时，扣非与归母的增速通常会明显分化；30% 以上则意味着近三分之一利润不可持续。条件 $\mathrm{NP}_p > 0$ 排除亏损公司（占比的符号失去意义）。
- **S2 的 10 个百分点**。与 S1 互补：S1 看水平，S2 看变化。一家公司非经常性损益占比不高，但当年突然增加，S1 不触发而 S2 触发。
- **S3 的 0.8 / 0.5**。净现比长期低于 1 本身不罕见（重资产扩张期、季节性回款），0.8 是“明显偏低”，0.5 是“利润一半以上没有现金”。

表 36 异常信号规则（`analyze._signals`）。

ID	类型	触发条件	严重度	金融含义与推论
S1	非经常性损益	$\mathrm{NP}_p > 0$ 且 $\lvert \mathrm{NR}/\mathrm{NP}_p \rvert \geq 15\%$	$\geq 30\%$ 高，否则中	利润中相当部分来自政府补助、资产处置、公允价值变动等不可持续项目；估值应以扣非口径为主
S2	利润质量	$g(\mathrm{NP}_p) - g(\mathrm{NP}_d) \geq 10$ 个百分点	中	表观增速高于经常性利润增速，增长部分由一次性收益驱动
S3	利润与现金流背离	$\mathrm{NP}_p > 0$ 且 $\mathrm{OCF}/\mathrm{NP}_p < 0.8$	< 0.5 高，否则中	利润的现金含量偏低：应收账款、存货占款或收入确认节奏需要排查。应计利润占比高的公司未来盈利持续性较差 [53]
S4	利润与现金流背离	$\mathrm{NP}_p > 0$，$g(\mathrm{NP}_p) > 0$ 且 $g(\mathrm{OCF}) < 0$	高	盈利增长没有转化为现金，方向背离比水平偏低更值得警惕
S5	营运资本	$g(\mathrm{AR}) - g(\mathrm{Rev}) \geq 15$ 个百分点	≥ 30 高，否则中	应收扩张快于收入：放宽信用政策、渠道压货或减值风险；应收周转恶化是盈余操纵识别模型的典型变量 [5]
S6	营运资本	$g(\mathrm{Inv}) - g(\mathrm{Rev}) \geq 15$ 个百分点	≥ 30 高，否则中	存货扩张快于收入：备货、滞销或跌价风险
S7	盈利能力	$\lvert \Delta\mathrm{GM} \rvert \geq 3$ 个百分点	中	毛利率大幅变动，需拆分产品结构、价格与成本因素
S8	会计口径	文本含会计口径关键词，或表格有“调整前 / 调整后”	中	存在会计政策 / 估计变更、追溯调整、同一控制下合并等，同比需确认可比口径

- **S5、S6 的 15 / 30 个百分点**。营运资本科目的同比与收入同比之差。用差值而不是比值，避免收入增速接近 0 时比值发散。
- **S7 的 3 个百分点**。多数制造业公司年度毛利率波动在 1–2 个百分点以内，3 个百分点意味着产品结构或价格成本关系发生了实质变化。

S8 的实现。 S8 是唯一读原文文本的信号：在全部文本 chunk 中检索九个关键词（会计政策变更、会计估计变更、追溯调整、重述、前期差错更正、同一控制下企业合并、合并范围变更、调整前、调整后），并合并抽取阶段因“调整前”列产生的“追溯调整”issue 的证据。每处命中记录页码、关键词与前 160 字摘句；信号最多附 5 条证据。

失败案例：关键词检索不识别否定

样例年报第 95 页写道“该项会计政策变更对公司财务报表无重大影响，未对比较期间数据进行追溯调整”。S8 命中了“会计政策变更”与“追溯调整”，但后者在原文中是被否定的。三家真实年报上 S8 分别在 10、12、18 处命中，覆盖了“会计政策变更”“同一控制下企业合并”“调整前 / 调整后”等多种情形，其中相当一部分是例行的准则执行说明。S8 因此只给“中”严重度，推论写成“同比比较需确认是否使用追溯调整后的可比数据”，把判断交给阅读证据的人或写作会话。用模型判断否定语义可以提高精确率，但会把模型判断引入一个本应确定的环节；我们保留关键词规则，接受较低的精确率。

信号与研报写作。 信号进入 `analysis.json` 后，写作会话被要求对每条高严重度信号给出解释或列入待验证清单（第 11 节）。信号的 `fact` 字段中的数字全部来自真值表，写作会话引用它们时由 guard 按展示精度核对。

7.7 利润变动归因桥

动机。 “归母净利润增长 18%，主要受益于收入增长”是研报中最常见的句式，但“主要”往往没有依据：收入增长带来的毛利增量可能被费用增长和所得税抵消大半，真正的增量来自投资收益。财报分析选题要求解释“业绩变化原因”。我们需要一个把利润变化逐项分解、各项加总严格等于总变化的工具，让正文中任何“某因素贡献了多少”的说法都能与之对账。

分解。 `earnings.profit_bridge(T, k)` 以 $k_0 = \beta_{\mathrm{YoY}}(k)$ 为基期，需要两期的营业收入、营业成本、四项费用、营业利润、利润总额、所得税、净利润、归母净利润共 11 个科目，任一缺失返回空。记 $\mathrm{GP} = \mathrm{Rev} - \mathrm{COGS}$，$\mathrm{GM} = \mathrm{GP}/\mathrm{Rev}$，$E = \mathrm{S} + \mathrm{A} + \mathrm{D} + \mathrm{F}$（销售、管理、研发、财务费用），并定义三个轧差量：

$$\begin{aligned} \mathrm{OO} &= \mathrm{OP} - (\mathrm{GP} - E) && \text{其他经营损益} \\ \mathrm{NO} &= \mathrm{TP} - \mathrm{OP} && \text{营业外收支} \\ \mathrm{MI} &= \mathrm{NP} - \mathrm{NP}_p && \text{少数股东损益} \end{aligned}$$

其中“其他经营损益”吸收税金及附加、投资收益、公允价值变动、信用与资产减值、其他收益、资产处置收益等未单列项目。十个驱动项（下标 0、1 分别为基期与本期）：

$$\begin{aligned} &\text{收入规模效应} = (\mathrm{Rev}_1 - \mathrm{Rev}_0)\cdot \mathrm{GM}_0, && \text{毛利率效应} = \mathrm{Rev}_1 \cdot (\mathrm{GM}_1 - \mathrm{GM}_0), \\ &\text{四项费用} = -\Delta\mathrm{S}, -\Delta\mathrm{A}, -\Delta\mathrm{D}, -\Delta\mathrm{F}, && \text{其他经营损益} = \Delta\mathrm{OO}, \\ &\text{营业外收支} = \Delta\mathrm{NO}, && \text{所得税} = -\Delta\mathrm{Tax}, \quad \text{少数股东损益} = -\Delta\mathrm{MI}. \end{aligned} \tag{32}$$

命题 7.2 (闭合误差即利润表恒等式的残差)**.** 记十项之和为 Σ，$\Delta NP_p = NP_{p,1} - NP_{p,0}$。则

$$\Sigma - \Delta NP_p = \Delta(TP - Tax - NP).$$

特别地，若两期都满足利润表恒等式“净利润 = 利润总额 − 所得税费用”，则归因桥严格闭合：$\Sigma = \Delta NP_p$。

证明. 收入规模效应与毛利率效应之和为 $(\mathrm{Rev}_1 - \mathrm{Rev}_0)\mathrm{GM}_0 + \mathrm{Rev}_1(\mathrm{GM}_1 - \mathrm{GM}_0) = \mathrm{Rev}_1\mathrm{GM}_1 - \mathrm{Rev}_0\mathrm{GM}_0 = \Delta\mathrm{GP}$。于是

$$\Sigma = \Delta\mathrm{GP} - \Delta E + \Delta(\mathrm{OP} - \mathrm{GP} + E) + \Delta(\mathrm{TP} - \mathrm{OP}) - \Delta\mathrm{Tax} - \Delta(\mathrm{NP} - \mathrm{NP}_p) = \Delta\mathrm{TP} - \Delta\mathrm{Tax} - \Delta\mathrm{NP} + \Delta\mathrm{NP}_p.$$

移项即得。 □

命题 7.2 说明，“其他经营损益”这一轧差项的设计让分解在代数上恒等闭合，剩下的闭合误差 `closure_error` 恰好度量了抽取到的利润表是否满足“净利润 = 利润总额 − 所得税费用”。这相当于一项免费的额外勾稽：闭合误差显著不为零，说明利润总额、所得税或净利润中至少一个抽取有误。样例与三家真实年报上的闭合误差都在 10^{-5} 元量级（浮点误差），见表 40。

非经常性损益的贡献。 归因桥按利润表科目分解，而非经常性损益横跨多个科目（政府补助在其他收益，资产处置在资产处置收益，公允价值变动在公允价值变动收益）。`profit_bridge` 另外给出非经常性损益的变动及其占归母净利润变动的比例；比例大于 1 时，报告写出“剔除后经营性利润实际下降”。`drivers_ranked` 按绝对值给出前三大驱动项，供写作会话引用。

权衡（Trade-off）：顺序分解 vs. 对称分解

把 $\Delta\mathrm{GP} = \Delta(\mathrm{Rev} \cdot \mathrm{GM})$ 拆成“规模”与“毛利率”两项，有多种合法做法。我们用的是“规模按基期毛利率、毛利率按本期收入”的顺序分解，交叉项 $\Delta\mathrm{Rev} \cdot \Delta\mathrm{GM}$ 全部计入毛利率效应。对称分解（各用两期平均，或 Shapley 分配）把交叉项平分，对两个因素更“公平”，但两种分解的数值差异恰好是交叉项的一半，在收入与毛利率同时大幅变化的年份可能不小。我们选择顺序分解有两个理由：(1) 它是卖方研报与教科书最常用的写法，读者可以用“上年毛利率 × 收入增量”的直觉验证；(2) 每一项都能用两个真值表单元格的简单算式复核，guard 在核对正文时不需要知道分解的内部约定。代价是归因结果依赖于所选顺序，报告中逐项写明公式（`formula` 字段）。

权衡（Trade-off）：按科目分解 vs. 按业务分部分解

另一种归因是按产品或地区分部：某产品收入增加多少、毛利率变化多少。它对投资判断更有用，但分部数据不在真值表中（只在经营数据表里），且分部收入与营业收入之间常有内部抵销与“其他业务”，难以严格闭合。我们把按科目的归因桥作为程序产物，按分部的分析交给写作会话利用经营数据表与其中的程序计算比率完成（§7.9），两者在研报中互相印证。

7.8 实际 vs 预期、业绩预告与 EPS 质量

`earnings.py` 的预期对比、预告对比与 EPS 质量筛查三部分沿用原 Public Equity Investing 插件 earnings-deep-dive 的判定规则，自研实现、只用标准库；与原实现的一致性由 `tests/test_golden.py`（对比 `tests/golden/earnings.json` 快照）和本地有原插件时的逐项对照测试保证（附录 F）。中文字段与 A 股指标的映射是我们加的。

输入。 一致预期与内部预期来自 CSV（指标、期间、预期类型、数值、单位、截至日期、来源），业绩预告来自 CSV（指标、期间、下限、上限、单位、公告日期、来源）。实际值从真值表取，金额换算为百万元。可比较的指标有六个：营业收入、营业利润、归母净利润、扣非归母净利润、基本每股收益、扣非每股收益；后者由程序计算为扣非归母净利润 / 总股本，来源标记为"计算：扣非归母净利润/总股本"。

超预期幅度。 对每个指标，与一致预期 c 和内部预期 i 分别计算差额与幅度：

$$\delta = a - c, \qquad s = \frac{a-c}{c}\ (c \neq 0),$$

a 为实际值。预期缺失时差额与幅度为空；预期为 0 时幅度为空。没有提供预期文件时，`limitations` 记"未提供一致预期数据，跳过超预期/不及预期分析"。我们不在程序中给出"超预期 / 不及预期"的离散标签，只给幅度，标签由写作会话结合幅度与市场语境描述，并受 guard 的方向核对约束。

业绩预告。 预告区间 $[L, H]$ 的中值 $\bar{g} = (L+H)/2$ 与一致预期比较；实际值相对区间的位置按

$$\text{位置}(a) = \begin{cases} \text{高于预告上限} & a > H \\ \text{低于预告下限} & a < L \\ \text{预告区间上半部} & \bar{g} \le a \le H \\ \text{预告区间下半部} & L \le a < \bar{g} \end{cases}$$

并给出 $a/\bar{g} - 1$。A 股的业绩预告是强制披露（达到一定变动幅度时），实际值落在区间哪一端是市场关注的信号；相对于预告中值的意外比相对于一致预期的意外更"新"，因为预告公告后一致预期通常已向预告靠拢。

EPS 质量筛查。 对每个 GAAP 口径的 EPS 行，检查两类触发项：

1. **失真项。**指标名、GAAP 标记、可比 GAAP 指标名、调节来源、备注中出现失真关键词（资产处置、营业外、权益投资、公允价值、汇兑、一次性收益、减值、诉讼、损失、盯市、非经营、非经常性、一次性、养老金、重组、股本变动、税率共 17 个）之一。A 股适配：当非经常性损益占归母净利润 ≥ 15%（与信号 S1 同阈值）时，程序在基本每股收益的备注中写入"non-recurring gain: 非经常性损益占归母净利润 X%"，从而触发失真项；
2. **幅度背离。**EPS 超预期幅度 $|s_{\text{EPS}}| \ge 15\%$，且 $|s_{\text{EPS}}| > \max(2|s_{\text{peer}}|, |s_{\text{peer}}| + 10\%)$，其中 peer 分别取营业收入与营业利润。含义是：EPS 的意外远大于收入和经营利润的意外时，差额多半来自经营以下的项目（税率、投资收益、股本），不能外推。

有任一触发时结论为"需展开 EPS 质量拆解"，并列出触发项；否则为"未触发 EPS 质量预警"，附"仍需与财报附注核对"。盈利公告后的价格漂移研究 [6] 表明市场对盈利意外的反应并不充分，而意外的"质量"（是否可持续）决定了它能否外推，这是 EPS 质量筛查的出发点；应计质量的研究 [13] 给出了类似的结论。

样例结果。 样例公司的结果见表 37。归母净利润高于一致预期 3.8%，但扣非归母净利润低于一致预期 14.6%；归母净利润落在预告区间上半部，扣非落在下半部。EPS 幅度 4.1% 不触发幅度背离，但非经常性损益占比 27.5% 写入了备注，触发失真项"一次性收益、非经常性损益"，结论为"需展开 EPS 质量拆解"。三个模块从不同角度给出同一个判断：表观超预期来自非经常性项目。

表 37　样例公司“星河智造”（虚构）2024 年实际 vs 预期与预告（百万元；每股为元；预期与预告均为虚构样例数据）。

指标	实际	一致预期	幅度	内部预期	幅度	预告区间位置
营业收入	5,862.3	5,750.0	+2.0%	5,900.0	−0.6%	—
营业利润	742.6	715.0	+3.9%	—	—	—
归母净利润	612.4	590.0	+3.8%	560.0	+9.4%	580–640，上半部
扣非归母净利润	443.9	520.0	−14.6%	—	—	420–480，下半部
基本每股收益	0.77	0.74	+4.1%	—	—	—
扣非每股收益	0.55	—	—	—	—	—

7.9　经营数据在分析中的作用

经营数据表（§6.11）在真值表就绪后运行完整版本，引用真值表的比率此时才计算，例如：现金分红率 = 年度累计现金分红 / 归母净利润（茅台 `payout`，分母引用真值表中 `np_parent` 的 2025 年值）；分产品收入占比 = 分产品收入 / 营业收入；分渠道收入占比的同比变动（`diff`，单位 pp）。这些比率与真值表中的派生指标一样，由程序计算、带公式与引用，供研报引用；正文中出现的“茅台酒毛利率 93.53%，同比下降 0.53 个百分点”“i 茅台平台收入 130.31 亿元，同比下降 34.92%”这类分部表述，由 guard 按行名或别名识别并在经营数据表上核对（`test_guard_accepts_segment_figures_and_flags_wrong_ones`、`test_guard_valuation_mm_aliases_and_bridge`）。guard 同时拦截“把分部数据冒充公司整体数据”的写法：“2025 年公司营业收入 130.31 亿元”会因指标绑定不符被拦下（130.31 亿元是 i 茅台的收入）。

经营数据是分部分析的基础：它解释归因桥中“收入规模效应”和“毛利率效应”来自哪个产品、哪个渠道，也为估值中的分部量价预测提供基期（第 8 节）。三家真实年报共核对了 1,076 个经营数据格子、声明了 239 个比率（表 29）。

7.10　报告输出

`finagent/output/report.py` 把分析产物渲染为 Markdown 报告 `financials/report.md`，结构按“事实 → 推论 → 观点”组织：

1. **摘要行**：抽取指标数（及规则 / 模型通道各自的数量）、勾稽通过 / 失败 / 无法评估数、异常信号数与高严重度数；
2. **一、事实：关键财务指标**：本期、上年同期、同比（比率为 pp）、环比、来源；来源由 `loc()` 渲染为“文件名 p. 页码 chunk_id 行 r 列 c”，派生值渲染为“派生自…”并递归列出输入的来源；随后是单季表，推算值标“*”；
3. **二、事实：勾稽与一致性校验**：每项检查的状态与差额，以及抽取过程的 issue；
4. **三、推论：异常信号**：每条信号分“事实 / 推论 / 证据”三行，文本类证据（S8）给出页码与摘句；
5. **四、观点与待验证事项**：每条信号一项待验证事项。这里刻意只给“建议查阅附注 / 问询函核实原因”，不给观点——观点由写作会话撰写并过 guard；
6. **五、业绩变化归因**：归因桥表（含闭合误差）、前三大驱动、非经常性损益贡献，实际 vs 预期表、实际 vs 预告表、EPS 质量结论与限制说明。

事实表另存为 `facts.csv`（带 BOM，Excel 直接打开不乱码），来源列同样渲染为可读定位。金额统一以亿元两位小数展示，每股两位小数，比率两位小数加“%”，展示精

表 **38**　样例公司 2024 年关键指标与触发的信号（金额为亿元）。

指标	2024	2023	同比
营业收入	58.62	51.20	14.49%
归母净利润	6.12	5.18	18.17%
扣非归母净利润	4.44	4.71	−5.67%
非经常性损益	1.68	0.48	254.14%
经营活动现金流量净额	2.38	5.81	−58.98%
应收账款	19.86	14.02	41.60%
毛利率	31.19%	31.22%	−0.03pp

信号	严重度	事实
S1	中	非经常性损益占归母净利润 27.50%
S2	中	归母与扣非增速差 23.85 个百分点
S3	高	净现比 0.39
S4	高	归母净利润 +18.17%，经营现金流 −58.98%
S5	中	应收账款同比 41.60%，比收入快 27.11 个百分点
S8	中	2 处口径披露（含一处否定语境，见失败案例）

度与 guard 的核对精度一致，写作会话照抄报告中的数字时不会因为舍入差异被拦下。

为什么程序不写解读。 报告中“推论”字段是规则模板的固定句子，“观点”部分只有待验证清单。我们没有让程序（或程序内调用的模型）生成解读段落：解读需要结合行业与公司背景，而后者来自研究助理与分析师会话的资料搜集（第 11 节）；把解读放在写作会话里，它就自然受 guard 与数字锁约束，不会出现“程序报告里有一套说法、研报里另一套说法”的情况。

7.11　样例与真实公司上的结果

样例。 虚构样例“星河智造”的财报分析（命令 `finagent financials`，输入 `data/samples/annual_report_2024.md`）抽取 107 个事实（规则通道 113 条，含重复与佐证），14 项勾稽全部通过，触发 6 条信号（高 2 条）。样例在设计上埋了一个“利润增长、现金流恶化、应收激增、非经常性损益撑起表观增速”的故事，表 38 显示程序从数字中完整还原了这个故事。

样例的归因桥：归母净利润从 518.2 百万元增至 612.4 百万元，变动 +94.2 百万元；收入规模效应 +231.6，毛利率效应 −2.0，研发费用 −44.2，其他经营损益 −32.0，销售费用 −28.1，所得税 −18.2，管理费用 −16.2，少数股东损益 −8.9，财务费用 +6.8，营业外收支 +5.4；闭合误差 1.3×10^{-7} 元。非经常性损益变动 +120.9 百万元，相当于归母净利润变动的 128%，报告据此写出“剔除后经营性利润实际下降”。这与 S1、S2、EPS 质量筛查的结论一致。

三家真实年报。 在三家真实年报的模型抽取上运行 `analyze` 与 `profit_bridge`（本文重跑），结果见表 39 与表 40。

三家的信号各自对应一个值得在研报中解释的现象：

- **贵州茅台 S3**：经营现金流 615.22 亿元，归母净利润 823.20 亿元，净现比 0.75，经营现金流同比下降 33.46%，远大于利润降幅；S4 未触发，因为利润本身也在下降（S4

表 39　三家真实年报 2025 年的关键变动、勾稽与信号（金额为亿元；同比为%，比率变动为 pp）。

	贵州茅台	宁德时代	招商轮船
营业收入	1,688.38	4,237.02	281.77
同比	−1.21	17.04	9.22
归母净利润	823.20	722.01	60.12
同比	−4.53	42.28	17.71
扣非归母净利润同比	−4.58	43.37	0.18
经营现金流同比	−33.46	37.35	6.83
存货同比	13.04	57.98	−7.54
毛利率（变动）	91.18（−0.75）	26.27（+1.83）	26.47（−1.95）
资产负债率（变动）	16.42（−2.63）	61.94（−3.30）	46.68（+4.52）
勾稽（通过 / 总）	20 / 20	15 / 15	21 / 21
信号	S3 中，S8	S6 高，S8	S1 中，S2 中，S8

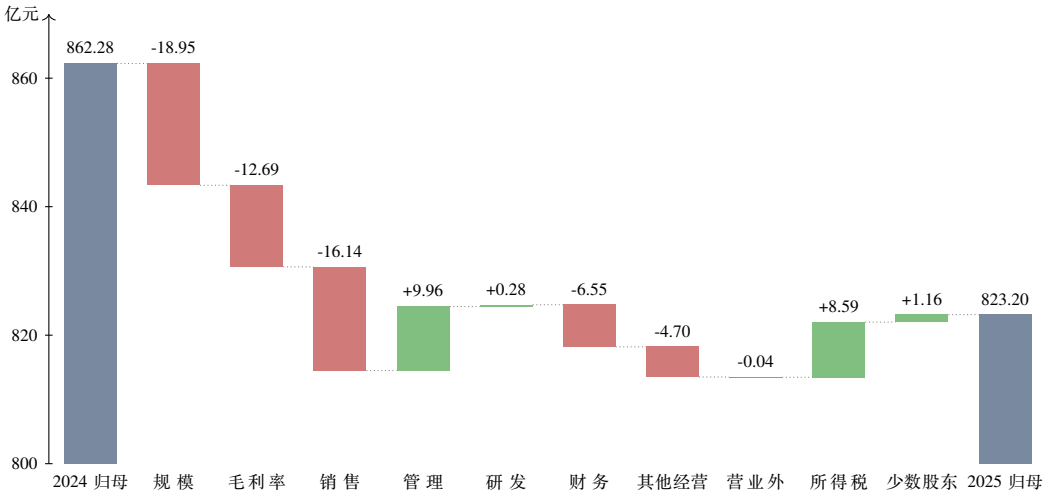


图 11　贵州茅台 2025 年归母净利润归因桥瀑布图（亿元；纵轴从 800 起，截断）。红色为拖累项，绿色为贡献项；数值即表 40 第一列，各项之和与总变动 −39.08 亿元的差在 10^{-5} 元量级。

要求利润增长而现金流下降）。

- **宁德时代 S6（高）**：存货同比 57.98%，营业收入同比 17.04%，差 40.94 个百分点，超过高严重度阈值 30。
- **招商轮船 S1、S2**：非经常性损益 9.88 亿元，占归母净利润 16.43%；归母净利润同比 17.71%，扣非同比仅 0.18%，差 17.53 个百分点。归因桥中“其他经营损益”是第一大正向驱动（+10.02 亿元），与之印证。

信号只陈述数字，原因需要回到年报附注和外部资料，这是写作会话与研究助理的工作。

茅台的归因桥是一个“收入微降但利润降得更多”的典型（图 11）：收入规模效应 −18.95 亿元、毛利率效应 −12.69 亿元、销售费用增加 −16.14 亿元，被管理费用下降与所得税减少部分抵消。正文写“销售费用增加拖累 16.14 亿元，毛利率效应拖累 12.69 亿元”这类句子时，guard 在归因桥产物上逐项核对通过（`test_guard_valuation_mm_aliases_and_bridge`）。宁德时代的利润增长主要来自规模（+150.80 亿元）与毛利率提升（+77.45

表 40　三家真实年报 2025 年归母净利润归因桥（亿元；本文重跑）。闭合误差单位为元。

驱动项	贵州茅台	宁德时代	招商轮船
收入规模效应	−18.95	+150.80	+6.76
毛利率效应	−12.69	+77.45	−5.48
销售费用	−16.14	−1.72	−0.06
管理费用	+9.96	−19.77	−1.57
研发费用	+0.28	−35.40	−0.09
财务费用	−6.55	+38.08	−1.03
其他经营损益	−4.70	+45.23	+10.02
营业外收支	−0.04	+8.78	+0.69
所得税	+8.59	−35.65	−0.61
少数股东损益	+1.16	−13.23	+0.43
归母净利润变动	−39.08	+214.57	+9.05
闭合误差（元）	1.9×10^{-5}	3.8×10^{-6}	-3.6×10^{-7}
前三大驱动	规模、销售费用、毛利率	规模、毛利率、其他经营	其他经营、规模、毛利率

亿元），研发费用与所得税是主要的拖累项。

7.12　失败案例与边界情况

失败案例：追溯调整后的季度不闭合

命题 3.1(b) 的前提是列示数据满足全部季度等式。若公司在年内发生同一控制下企业合并，年报分季度表通常按重述后的口径列示，而当年三季报是重述前口径，“第四季度 = 全年 − 前三季度累计”的核对会失败并阻断。此时失败是“正确的失败”：两份报告的口径确实不同。处理方式是由分析师会话在 `facts.json` 中只采用同一口径的数据（例如不传三季报，只用年报分季度表），并在研报中说明。程序不会自动判断应当采用哪个口径，这需要阅读附注。在三家真实年报上未出现此情况；样例中我们人为构造过一次重述场景，核对如期失败。

失败案例：亏损公司的信号与归因

S1、S3、S4 都要求 $\mathrm{NP}_p > 0$，亏损公司上这些信号不会触发——不是因为没有问题，而是比率失去意义。归因桥对亏损公司照常计算（各项是差额，不受符号影响），但“占变动比例”在归母净利润变动接近 0 或跨越盈亏时会发散，报告中显示为很大的百分比，写作会话需要用绝对额而不是占比描述驱动项。

失败案例：信号阈值的边界效应

阈值是硬切分：应收增速比收入快 14.9 个百分点不触发 S5，15.0 个百分点触发中等严重度。这会让两家情况几乎相同的公司得到不同的信号列表。我们接受这一点，因为信号是“待验证事项”的筛选器而不是评分；写作会话被要求阅读完整的变动表，而不只是信号列表。

金融行业。 指标字典与信号针对一般工商企业设计。银行、保险、证券的报表结构不同（没有营业成本与毛利率，营业收入的构成也不同），归因桥需要的 11 个科目中有若干不存在，`profit_bridge` 返回空；勾稽中的资产负债表恒等式仍然适用。招商轮船的“营业总收入 / 营业收入”与茅台集团财务公司的利息收入说明工商企业年报中也会混入金融业务科目，抽取规则对此有专门处理（§6.6.1）。

7.13 小结

财报分析的全部数字由确定性程序从真值表计算：派生指标、同比环比、单季拆分、四组勾稽、八条异常信号、十项归因桥、预期与预告对比、EPS 质量筛查。几个设计值得强调：派生值永不覆盖列示值，二者的差异只通过勾稽暴露；单季拆分用不动点闭包实现，并用列示依赖集避免“自己和自己比”的伪核对；勾稽之外，财报自带的同比列与归因桥的闭合误差提供了两项额外的、几乎免费的抽取自检；信号把事实与推论分字段存放，观点留给受 guard 约束的写作会话。在样例上，程序从数字中还原了“表观增长由非经常性损益撑起、现金流与应收恶化”的完整故事；在三家真实年报上，56 项勾稽全部通过，归因桥闭合误差在浮点精度以内，触发的信号都对应研报中需要解释的现象。这些产物经由真值表进入估值（第 8 节）与研报纠错（第 9 节）。

8 估值建模

自动化估值建模选题要求系统给出“分部预测、三表联动、DCF 与可比公司估值、情景与敏感性分析”，并写明假设、适用范围与风险。这件事交给大语言模型直接做，会在三个地方出错。第一是算术：一张五年期三表模型加三情景 DCF 涉及上千次乘除与累加，模型逐格心算必然出现进位、符号与单位错误。第二是口径：模型写出的 DCF 利润率、营运资本占比与它在“盈利预测”表里写的三表数字往往互不相干，研报读者看到的估值与预测表来自两套隐含假设。第三是结论：评级、目标价由模型“综合判断”给出，既无法复算，也容易与正文估值数字矛盾。

LEDGER 的估值链按第 4 节“理解与计算分离”的原则组织：分析师会话只写一份估值假设文件 `valuation_assumptions.json`（全部是观点与假设，附逐项依据），其余由不调用模型的确定性 Python 引擎完成。本章逐模块写清公式与实现，重点放在第 3.6 节形式化定义之外的算法细节、工程取舍和数值实例。第 3.6 节已经给出三表约束（式 (7) ~ (10)）、股权价值桥（式 (15)）、评级函数（式 (16)）、反向 DCF 定义（式 (17)）与正确性准则 (V1) ~ (V6)；第 2 节给出 FCFF、EV 与 EBIT 的教科书定义（式 (2)、(3)、(1)）。本章不重复推导，只在实现与教科书口径不同的地方展开。

本章的主要设计可以概括为四点：

1. **单一驱动源**：DCF 的六个预测向量（收入增速、EBIT 利润率、税率、折旧摊销率、资本开支率、营运资本占比）全部取自三表模型同一情景的输出，DCF 运行后再逐期核对还原值与三表输出，偏差超过 10^{-6} 百万元即判硬性失败（第 8.4 节）。
2. **经营性 EBIT 与轧差经营费用**：EBIT 剔除投资收益与公允价值变动，三表模型把经营费用定义为“毛利 – EBIT – 折旧摊销”的轧差项，使历史期三表 EBIT 与 DCF 基期 EBIT 恒等；被剔除损益对应的资产经逐项证据核对后在股权桥加回（第 8.3 节）。
3. **程序给评级、程序解反向问题**：评级由基准情景每股价值相对参考股价的空间按固定阈值给出；“现价隐含了什么”由网格扫描加二分法逐变量求解，并按历史区间分档，文字只能引用分档（第 8.6、8.8 节）。
4. **可审计的公式工作簿**：用标准库手写 OOXML，生成输入蓝、公式黑、跨表引用绿的投行排版 Excel，预测期每格都是公式且带缓存值；质检侧用只读审计器独立复算五组勾稽并检查跨期公式一致性（第 8.9 节）。

8.1 估值链总览

图 12 是估值链的全貌。主链自上而下：真值表 $\mathcal{T}$ 与经营数据表（`extras.json`）提供基期，估值假设文件提供驱动；分部预测把“销量 × 单价”折算成收入增速、综合毛利

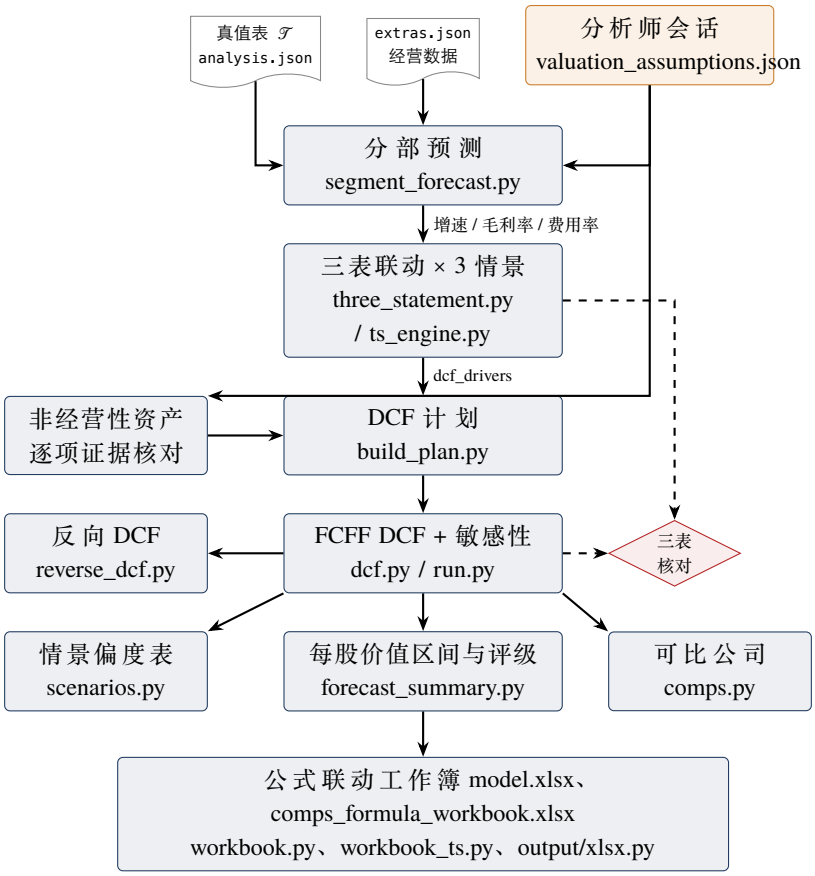


图 12 估值链。蓝色为确定性程序，橙色为模型会话，红色菱形为程序核对点。模型只写假设文件；非经营性资产明细要逐项回查年报原文；DCF 预测向量取自三表模型并在运行后逐期核对。

率与经营费用率并覆盖假设文件中的对应向量；三表引擎在三个情景下各跑一遍，导出 DCF 驱动；DCF 引擎只负责折现、终值与股权价值桥；最后由程序给出每股价值区间与评级。旁路有四条：反向 DCF、情景偏度表、可比公司估值、公式工作簿。

表 41 列出各模块的职责、规模与守护测试。整条链在 `finagent/pipelines.py` 中注册为 `valuation` 流水线的八个步骤（`three_statement → build_plan → dcf → reverse_dcf → scenarios → forecast_summary → comps → valuation_report`），每步产物经 `tracer.write_artifact` 落盘并记录哈希（第 5 节）。样例公司全流程（含财报分析与研报纠错）在本机耗时 0.06 秒，三家真实公司的估值流水线（含 PDF 解析与证据回查）各 5 ~ 10 秒。

单位约定。 财务事实在真值表内统一以元存储；进入估值链时除以 10^6 换算为百万元，股本为百万股，每股价值与股价为元，比率在估值模块内部存小数（0.082 表示 8.2%），输出到盈利预测表时再按全书约定换成百分数值。这条边界在 `three_statement._hist` 与 `build_plan._v` 两处集中完成，下游不再做单位判断。

谁写什么。 表 42 是假设文件的字段与生效方式。它的设计目标是“模型只能写观点，不能写计算结果”：文件里没有每股价值、没有 EBIT 利润率路径，也没有非经营性资产合计数——这些要么由程序算，要么由程序从原文数字加总。

表 41　估值模块一览（行数为源文件行数，含注释）。

模块	行数	职责	主要测试
segment_forecast.py	111	分部量价 → 收入增速、综合毛利率、费用率	test_segment_units
three_statement.py	296	A 股科目 → 引擎 plan；导出 DCF 驱动	test_quarterly_ebit
ts_engine.py	661	三表联动、不动点、勾稽、情景、单因素敏感性	test_golden test_parity_reference
build_plan.py	160	经营性 EBIT、DCF plan、非经营性资产核对	test_financial_model_todos
dcf.py run.py	285 88	FCFF 折现、终值、股权桥、敏感性、三表核对	test_parity_reference test_quarterly_ebit
reverse_dcf.py	193	单变量反向 DCF（网格 + 二分）	test_reverse_dcf
scenarios.py scenario_tables.py	110 405	情景偏度、倍数/EPS/KPI/流动性敏感性、触发表	test_golden test_parity_reference
comps.py	476	可比公司倍数、IQR 离群、利润率对比、公式工作簿	test_golden test_quarterly_ebit
forecast_summary.py	94	盈利预测表、市场数据、程序评级	test_new_modules
workbook.py workbook_ts.py	594 458	model.xlsx 各页（公式联动三表、DCF、WACC）	test_quarterly_ebit test_workbook_audit
output/xlsx.py	525	标准库 OOXML 写出与投行排版	test_golden
workbook_audit.py	266	（evaluation/）工作簿只读审计（公式结构、五组勾稽）	test_workbook_audit

8.2　分部量价预测

动机。 卖方研报的预测逻辑通常从业务驱动出发：白酒拆成茅台酒与系列酒的“吨数 × 吨价”，动力电池拆成“GWh × 单价”。如果假设文件直接写合并收入增速，经营分析里讲的量价故事与估值里的数字之间没有可检查的对应关系。分部预测把这种对应关系变成程序约束：文稿里写到的变化必须在 segment_forecast 的参数里出现，收入增速由程序从量价推出。

基期。 设分部 j 的基期收入 $R_{j,0}$、营业成本 $C_{j,0}$ 取自经营数据表 product 行，销量 $Q_{j,0}$ 取自 volume 行（两张表都经 verify-extra 逐格回查原文，第 6 节）。程序计算

$$P_{j,0} = \frac{R_{j,0}}{Q_{j,0}} \times 100, \qquad m_{j,0} = 1 - \frac{C_{j,0}}{R_{j,0}}, \tag{33}$$

其中 R 以百万元计、Q 以吨计时 P 的单位是万元/吨（百万元/吨 ×100）。未拆分部分记为“其他业务”：$R^{\text{oth}} = R_0 - \sum_j R_{j,0}$，$C^{\text{oth}} = C_0 - \sum_j C_{j,0}$，预测期保持基期金额不变。

递推。 情景 s 下给定销量增速 $v^s_{j,t}$、单价增速 $p^s_{j,t}$、毛利率变动 $\Delta m^s_{j,t}$（百分点），

$$Q_{j,t} = Q_{j,t-1}(1 + v^s_{j,t}), \qquad P_{j,t} = P_{j,t-1}(1 + p^s_{j,t}), \qquad m_{j,t} = m_{j,t-1} + \Delta m^s_{j,t}/100, \tag{34}$$

$$R_{j,t} = Q_{j,t}P_{j,t}/100, \qquad C_{j,t} = R_{j,t}(1 - m_{j,t}), \qquad R_t = R^{\text{oth}} + \textstyle\sum_j R_{j,t}, \tag{35}$$

综合毛利率 $m_t = 1 - (C^{\text{oth}} + \sum_j C_{j,t})/R_t$，收入增速 $g_t = R_t/R_{t-1} - 1$（R_0 为报表营业收入）。

费用率。 三表口径的经营费用率（不含折旧摊销）拆为五项：税金及附加率、销售、管理、研发费用率，以及“其他”——基期轧差 $o^{\text{oth}}_0 = o_0 - (o^{\text{tax}}_0 + o^{\text{sell}}_0 + o^{\text{adm}}_0 + o^{\text{rd}}_0)$，预测期保持不变。假设文件可按情景覆盖任一分项（标量、向量或按情景的字典），未覆盖的取基期值。o_0 的定义见第 8.3 节式 (36)。

表 42　估值假设文件字段（分析师会话起草，全部为观点/假设）。

字段	含义与生效方式
current_price、price_source	参考股价及来源（外部数据，须登记在 $\mathscr{S}$）；用于上行空间、反向 DCF、倍数
wacc.*	R_f、β、ERP、规模/特定/国别溢价、税前债务成本、边际税率、目标资本结构权重（和须为 1）
terminal_value	终值方法（永续增长或退出倍数）、永续增长率、退出 EV/EBITDA 倍数
scenarios.{base,downside,upside}	只保留 revenue_growth、capex_percent_revenue、terminal_growth_rate、wacc_adjustment 与情景说明
three_statement.*	可选：毛利率/费用率路径、gross_margin_shift、ar_days_shift、dividend_payout
segment_forecast	可选：各分部各情景的销量增速、单价增速、毛利率变动（百分点）与分项费用率；生效时覆盖收入增速、毛利率、费用率
non_operating_assets	数字（百万元）或明细列表 [{label, raw_value, unit, page, sign}]；明细逐项回查原文
scenario_probabilities、rating_rule	可选：情景概率（默认 25/50/25）与评级阈值覆盖
sensitivities	敏感性轴（WACC、永续增长率、退出倍数、收入增速、利润率的变动档位）
rationale.*	逐项依据，须引用资料包 [S 编号]（阶段 6 闸门检查，第 11 节）

表 43　两家真实公司的分部基期（2025 年，程序由经营数据表计算；金额百万元）。

公司	分部	收入	成本	销量	单价	毛利率
贵州茅台	茅台酒	146,499.9	9,484.8	46,750.66 吨	313.36 万元/吨	93.5%
	系列酒	22,274.7	5,321.1	38,353.48 吨	58.08 万元/吨	76.1%
宁德时代	动力电池系统	316,506.4	241,064.4	541.0 GWh	58,503.95 万元/GWh	23.8%
	储能电池系统	62,439.8	45,763.7	121.0 GWh	51,603.16 万元/GWh	26.7%

生效方式。 segment_forecast.apply 把 g_t^s 写入 scenarios.*.revenue_growth，把 m_t^s、o_t^s 写入 three_statement.gross_margin_by_scenario 与 opex_pct_by_scenario，三表模型优先使用这两个按情景的向量。假设文件里同名的 revenue_growth 因此只是占位，三家真实公司的假设文件都在 rationale.revenue_growth 中写明了这一点。

单位处理。 分部销量的单位因行业而异。假设文件可写 volume_unit、price_label、price_unit（宁德时代写 GWh、单价、万元/GWh），缺省为吨、吨价、万元/吨。这三个字段只影响工作簿“分部预测”页的表头，不进入计算：式 (33) 中的“×100”对任何销量单位都成立，因为它只是把“百万元/单位”改写为“万元/单位”。tests/test_segment_units.py 检查两点：未指定时产物不出现 volume_unit 字段（茅台的既有产物不变）；指定 GWh 时工作簿行标签全部换成“销量（GWh）”“单价（万元/GWh）”，且不残留“吨”字。

表 43 是两家公司的基期。茅台“其他业务”收入 63.5 百万元、成本 86.4 百万元，是一块很小的毛亏；宁德“其他业务”（电池材料及回收、矿产资源等）收入 44,755.6 百万元，占营业收入 10.6%，预测期按基期金额持平。茅台基期费用率分项为税金及附

加 16.20%、销售 4.30%、管理 4.93%、研发 0.11%、其他 −3.09%，合计 22.45%；宁德为 0.67%、0.88%、2.75%、5.23%、−6.26%，合计 3.27%。“其他”为负并非错误，原因见第 8.3 节的失败案例 1。

权衡（Trade-off）：只拆收入与毛利，不拆分部费用

另一种做法是给每个分部单独预测销售、管理费用，得到分部利润。我们没有这样做：A 股年报几乎不披露分部期间费用，模型只能凭空分摊，分摊比例本身就是不可核对的数字；而收入与营业成本在“主营业务分产品情况”表中有逐格可核对的原文。因此分部层面只做到毛利，费用率在公司层面按五个分项预测。代价是无法回答“哪个分部更赚钱”的问题，只能回答“哪个分部毛利更高”。

权衡（Trade-off）：其他业务按基期金额持平

把未拆分部分按合并收入增速同步增长也说得通，但这会让“其他业务”跟随主营的量价假设自动膨胀，相当于偷偷给它一个没写出来的假设。持平是最少假设的选择，并在 `rationale` 中写明（宁德的 10.6% 就属于这种情况）。占比过大时，分析师应把它拆成单独分部。

8.3 三表联动模型

三表模型分两层：`three_statement.py` 负责把 A 股科目映射成引擎的计划（plan），`ts_engine.py` 负责联动求解。引擎的计算口径与原 Public Equity Investing 插件的三表引擎逐位一致（第 8.10 节），只保留年度预测与 `total_growth` 收入模型。

8.3.1 口径映射与经营性 EBIT

定义 8.1 (经营性 EBIT)**.** 对期间 k，

$$EBIT_k = \text{利润总额}_k + \text{财务费用}_k - \text{投资收益}_k - \text{公允价值变动收益}_k.$$

资产处置收益、其他收益、信用与资产减值、营业外收支留在 *EBIT* 内。代码中这一定义只在一处出现（*build_plan.EBIT_PARTS*），三表历史基数、*DCF* 基期、反向 *DCF* 的历史区间、盈利预测表与可比公司目标行都引用它。

与式 (1) 的教科书口径相比，这里多剔除了投资收益与公允价值变动。理由是这两项对应的资产（长期股权投资、交易性金融资产等）不在 FCFF 的经营资产里；留在 EBIT 中会被永续资本化，同时对应资产若又在股权桥加回，就是重复计算。剔除后对应资产必须在股权桥加回，否则估值偏保守——`build_plan` 在发现本期投资收益与公允价值变动之和非零、而假设文件未设置 `non_operating_assets` 时，写入一条推论类说明“每股价值偏保守”，进入估值报告。

表 44 是其余科目的映射。

经营费用为什么是轧差项。 A 股利润表没有“折旧摊销”这一行，折旧摊销分散在营业成本与各项期间费用里；而引擎的利润表结构是 EBIT = 毛利 − opex − D&A。若把 opex 写成“销售 + 管理 + 研发”，折旧摊销会被扣两次。原插件按这种方式映射时，样例公司的历史 EBIT 被系统性低估（`tests/golden/README.md` 记录了 2026-09-28 的口径修正）。我们改为

$$\text{opex}_0 = R_0 - C_0 - \text{EBIT}_0 - \text{DA}_0, \tag{36}$$

这样历史期引擎 EBIT 与定义 8.1 恒等。税金及附加、减值、其他收益、资产处置收益、营业外收支这些未单列的项目也一并进入轧差项。

表 44　A 股报表科目到三表引擎的映射（three_statement._hist，百万元）。

引擎字段	A 股口径
revenue / cogs	营业收入 / 营业成本
da	固定资产折旧 + 无形资产摊销（现金流量表补充资料）
opex	轧差：营业收入 − 营业成本 − 经营性 EBIT − 折旧摊销（式 36）
interest	财务费用（可为负：净利息收入）
ppe_net	固定资产 + 在建工程
debt	短期借款 + 一年内到期的非流动负债 + 长期借款 + 应付债券 + 租赁负债
other_assets	资产总计 − 货币资金 − 应收账款 − 存货 − PP&E（轧差）
other_liabilities	负债合计 − 应付账款 − 有息负债（轧差）
common_equity	所有者权益合计（含少数股东权益）
retained_earnings	期初置 0，预测期累计

命题 8.1 (历史期 EBIT 同口径)**.** *在式 (36) 下，三表引擎对基期计算的 毛利$_0 - opex_0 - DA_0$ 等于定义 8.1 的 $EBIT_0$；若预测期 opex 率、毛利率取基期实际值，则 DCF 基期 EBIT 利润率与三表首期之前的利润率一致，预测期利润率变化全部来自显式写出的假设。*

证明. 代入即得 $(R_0 - C_0) - (R_0 - C_0 - \mathrm{EBIT}_0 - \mathrm{DA}_0) - \mathrm{DA}_0 = \mathrm{EBIT}_0$。第二句由预测期 $\mathrm{EBIT}_t = R_t(m_t - o_t) - \mathrm{DA}_t$ 与基期同形得到。 □

`tests/test_quarterly_ebit.py::test_ebit_excludes_investment_income_and_fair_value` 用样例数字核对：$\mathrm{EBIT}_0 = 760.12 + 45.30 - 23.50 - 41.20 = 740.72$ 百万元，且 $R_0 \cdot m_0 - R_0 \cdot o_0 - 340.5$ 与之相等。

8.3.2　联动方程组

记 $t = 1, \dots, N$ 为预测期，$t = 0$ 为最近实际年度。在本项目使用的功能范围内（无循环贷、无现金扫还、无可抵扣亏损），引擎求解：

$$R_t = R_{t-1}(1 + g_t), \quad C_t = R_t(1 - m_t), \quad O_t = R_t\, o_t, \tag{37}$$

$$\mathrm{Capex}_t = R_t \kappa_t, \quad \mathrm{Dep}_t = (\mathrm{PPE}_{t-1} + \tfrac{1}{2}\mathrm{Capex}_t)\,\delta_t, \quad \mathrm{PPE}_t = \max(0,\ \mathrm{PPE}_{t-1} + \mathrm{Capex}_t - \mathrm{Dep}_t), \tag{38}$$

$$\mathrm{AR}_t = R_t \tfrac{\mathrm{DSO}_t}{365}, \quad \mathrm{INV}_t = C_t \tfrac{\mathrm{DIO}_t}{365}, \quad \mathrm{AP}_t = C_t \tfrac{\mathrm{DPO}_t}{365}, \quad \mathrm{NWC}_t = \mathrm{AR}_t + \mathrm{INV}_t - \mathrm{AP}_t, \tag{39}$$

$$\mathrm{Int}_t = \max(0,\ D_{t-1} + \tfrac{1}{2}\mathrm{draw}_t - \tfrac{1}{2}\mathrm{mand}_t)\, r_t, \quad D_t = D_{t-1} + \mathrm{draw}_t - \mathrm{mand}_t, \tag{40}$$

$$\mathrm{EBIT}_t = R_t - C_t - O_t - \mathrm{Dep}_t, \quad \mathrm{EBT}_t = \mathrm{EBIT}_t - \mathrm{Int}_t, \quad \mathrm{NI}_t = \mathrm{EBT}_t - \tau \max(0, \mathrm{EBT}_t), \tag{41}$$

$$\mathrm{Div}_t = \max(0, \mathrm{NI}_t)\, \pi_t, \quad \mathrm{CFO}_t = \mathrm{NI}_t + \mathrm{Dep}_t - (\mathrm{NWC}_t - \mathrm{NWC}_{t-1}), \tag{42}$$

$$\mathrm{Cash}_t = \mathrm{Cash}_{t-1} + \mathrm{CFO}_t - \mathrm{Capex}_t - \mathrm{Div}_t + \mathrm{draw}_t - \mathrm{mand}_t, \quad \mathrm{RE}_t = \mathrm{RE}_{t-1} + \mathrm{NI}_t - \mathrm{Div}_t, \tag{43}$$

$$\mathrm{TA}_t = \mathrm{Cash}_t + \mathrm{AR}_t + \mathrm{INV}_t + \mathrm{PPE}_t + \mathrm{OA}, \quad \mathrm{TLE}_t = \mathrm{AP}_t + D_t + \mathrm{OL} + \mathrm{CE} + \mathrm{RE}_t. \tag{44}$$

变量含义：g 收入增速，m 毛利率，o 经营费用率（不含折旧摊销），κ 资本开支率，δ 折旧率（作用于期初 PP&E 加半年资本开支），DSO/DIO/DPO 为应收、存货、应付周转天数（存货与应付按营业成本），r 利率，τ 税率，π 分红率，OA/OL 为其他资产、其

他负债（保持基期），CE 为权益（含少数股东，基期取所有者权益合计），$\mathrm{RE}_0 = 0$。默认驱动全部取基期实际值：$\mathrm{DSO}_0 = \mathrm{AR}_0/R_0 \cdot 365$，$\delta_0 = \mathrm{DA}_0/(\mathrm{PPE}_{-1} + \frac{1}{2}\mathrm{Capex}_0)$，$\tau =$ 所得税$_0$/(所得税$_0$ + 净利润$_0$)，$r_0 =$ 财务费用$_0/\overline{D}$（$\overline{D}$ 为期初期末有息负债均值）。分红率 π 是本项目对原引擎的扩展（`dividend_payout_ratio`），原引擎只接受分红金额。

现金是平衡项。 资产负债表上没有任何一项被“倒挤”：现金由现金流量表滚动得到（式 (43)），其他资产与其他负债保持基期，权益由留存收益滚动。平衡是方程组的推论，而不是被强行写平的。

命题 8.2 (预测期恒平衡)**.** *若基期 $TA_0 = TLE_0$，且预测期 PP&E 未触发式 (38) 中的 $\max(0,\cdot)$ 截断、无资产处置，则对所有 t，$TA_t = TLE_t$。*

证明. 对相邻两期作差：$\Delta\mathrm{TA}_t = \Delta\mathrm{Cash}_t + \Delta\mathrm{AR}_t + \Delta\mathrm{INV}_t + \mathrm{Capex}_t - \mathrm{Dep}_t$，$\Delta\mathrm{TLE}_t = \Delta\mathrm{AP}_t + \mathrm{draw}_t - \mathrm{mand}_t + \mathrm{NI}_t - \mathrm{Div}_t$。代入式 (42)、(43) 的 $\Delta\mathrm{Cash}_t$，两式之差为 $-\Delta\mathrm{NWC}_t + \Delta\mathrm{AR}_t + \Delta\mathrm{INV}_t - \Delta\mathrm{AP}_t = 0$。基期平衡由映射保证：other_assets 与 other_liabilities 是轧差项，CE_0 取所有者权益合计，所以 $\mathrm{TA}_0 = \mathrm{TLE}_0$ 按构造成立。 □

截断与处置这两个前提在本项目的使用范围内总是成立（资本开支为正、处置未启用），但引擎并不依赖这个推理，而是每期显式计算六项勾稽误差（第 3.6 节 (V1)）：资产负债平衡、现金衔接、留存收益滚动、债务滚动、PP&E 滚动、营运资本滚动；任一项绝对误差超过 0.05 百万元即判硬性失败。容差取 0.05 百万元（5 万元）而不是 10^{-9}，是因为招股书类文件以“万元、两位小数”披露，期初报表本身就带 0.01 万元量级的舍入差（`three_statement.run` 的注释）。

利息的不动点。 原引擎支持循环贷与现金扫还：利息依赖期末债务，期末债务依赖现金是否低于最低现金线，现金又依赖税后利润，这就是式 (10) 的循环。引擎用算法 8 迭代求解。

在本项目实际使用的配置下，利息只依赖期初债务（式 (40)），而期初债务不依赖当期现金，因此第二轮即收敛；迭代框架保留下来，是为了与原引擎逐位一致——对照测试专门构造了一个利率 0.005623 …、第 4 轮利息变动约 6.8×10^{-7} 的边界用例，检验收敛阈值 10^{-8} 与原引擎相同。浮点运算顺序也保持一致（例如先算 ebt × rate 再相减），否则逐位相等无法成立。

情景。 三个情景共用同一个 plan，悲观与乐观通过 `overrides` 深合并（`deep_merge`）覆盖：收入增速与资本开支率取自假设文件的情景；毛利率优先取分部预测给出的情景向量，否则在基准毛利率上加 `gross_margin_shift`；应收天数在基准上加 `ar_days_shift`；经营费用率只在分部预测给出时按情景区分。样例中悲观情景毛利率为 31.19% − 2% = 29.19%、应收天数 123.63 + 15 = 138.63 天，乐观情景为 32.19% 与 113.63 天。

判定与模型状态。 `ts_engine.evaluate` 输出硬性失败与警示（表 45），`model_status` 据此分为 `not-decision-ready`、`screen-grade`、`senior-review-ready`、`decision-grade` 四档。所有判定码与英文提示语保持原插件的写法，便于逐字节对照。

单因素敏感性。 `run_sensitivities` 在基准情景上依次冲击收入增速（±5%）、毛利率（±2%）、应收天数（±15 天）、资本开支率（±1%）、利率（±1%），重跑整张三表，报告末期收入、EBITDA、自由现金流、期末现金、流动性低点与净杠杆峰值。它回答的是“哪个驱动最影响现金”，与 DCF 的每股价值敏感性互补。样例中应收天数 +15 天使期末现金从 3,526.6 降到 3,169.8 百万元，影响大于利率 +1%（降到 3,454.2 百万元）。

表 45　三表引擎的硬性失败与警示（阈值与原插件一致）。

代码	触发条件
硬性失败	
六项勾稽失败	任一情景任一期误差 > 0.05 百万元
`SCENARIO_SWITCH_NO_OUTPUT_CHANGE`	三情景末期收入两两相差 < 0.05：情景只改了标签
`SOURCE_BASIS_MISSING` 等	来源台账缺失、未覆盖历史数据或预测驱动
警示	
`AGGRESSIVE_REVENUE_RAMP`	基准情景收入复合增速 > 20%
`STEP_UP_REVENUE`	首个预测期收入较最近实际年度增长 > 25%
`UNSUPPORTED_MARGIN_EXPANSION`	预测期末毛利率较首期扩张 > 500 bp
`CAPEX_TOO_LOW_FOR_GROWTH`	收入增长 > 10% 而末期资本开支 < 0.75× 折旧
`WORKING_CAPITAL_RELEASE...`	营运资本占收入比较历史下降 > 5 个百分点
`STALE_*_SOURCE`	预测来源超 120 天、历史来源超 270 天，或来源跨度超一年
`NEGATIVE_CASH`、`LIQUIDITY_OR_COVENANT_ISSUE`	期末现金为负；流动性或契约余量不足

权衡（Trade-off）：期初留存收益置 0，权益合并列示

映射时把所有者权益合计（含少数股东权益）整体放进 `common_equity`，留存收益期初置 0。这样资产负债表上的“留存收益”其实是“预测期累计留存”，不是报表上的未分配利润，列示上不好看。替代方案是把权益拆成股本、资本公积、盈余公积、未分配利润、少数股东权益，但这五项在预测期只有未分配利润和少数股东权益会动，后者还需要单独预测少数股东损益；拆开既不改变任何勾稽，也不改变 DCF，只增加需要抽取和校验的科目。我们选择简化列示，在工作簿表头写明“留存收益（预测期累计）”。少数股东损益的处理放到盈利预测摘要（第 8.8 节）里用基期占比近似。

权衡（Trade-off）：周转天数法而非占收入比

营运资本有两种常见写法：各项占收入比例，或周转天数。我们沿用原引擎的天数法，存货与应付按营业成本折算。对茅台这类毛利率 91% 的公司，两种写法差别很大：存货按收入折算约 133 天，按成本折算为 1,505.5 天（基酒要窖藏多年）。按成本的天数更贴近经营含义，也让毛利率情景（成本变化）自动传导到存货和应付。DCF 层仍需要“营运资本占收入比”，由三表输出折算，见第 8.4 节。

失败案例 1：经营费用率为负（招商轮船）

招商轮船 2025 年的轧差经营费用率为 −7.16%。原因有两个：航运公司的船舶折旧几乎全部计入营业成本，而式 (36) 又扣了一次 $\mathrm{DA}_0/R_0 = 8.5\%$；资产处置收益（卖船，占收入 2.41%）以负费用的形式进入轧差项。EBIT 与 EBITDA 都是正确的，只是“经营费用”这一行失去了字面含义。分析师会话在假设文件里把预测期费用率写成 −5%，并在依据中解释为“剔除卖船收益后的经常性水平”。茅台与宁德的“其他费用率”为负是同一机制：o^{oth} = (减值 − 其他收益 − 处置收益 − DA)/R，宁德的 $\mathrm{DA}_0/R_0 = 5.86\%$ 几乎解释了 −6.26% 的全部。我们在工作簿行注释中写明“经营费用率不含折旧摊销”，但没有改变口径：任何把折旧从成本里拆出来的做法都需要年报未披露的分项。

失败案例 2：负利率

茅台的财务费用为负（利息收入大于利息支出），而有息负债只有数千万元，按 $r_0 =$ 财务费用$_0/\overline{D}$ 得到 $r = -247.2\%$；宁德为 -6.2%。引擎照常运行，预测期"利息费用"为 −577.7 与 −7,420.6 百万元，税前利润高于 EBIT。这不影响 FCFF 与每股价值（DCF 从 EBIT 出发），但影响盈利预测表的归母净利润与 EPS：利息收入被绑在不变的有息负债上，不随现金增长而增长。工作簿把这一行标注为"财务费用为净利息收入时为负"。更合理的做法是把利息收入挂在平均现金上单列，这需要扩展引擎结构，也会破坏与原引擎的逐位一致，列为后续工作（第 15 节）。

8.4 FCFF DCF

8.4.1 从三表到 DCF 计划

`build_plan.build_plan` 把真值表、假设文件与三表输出组装成 DCF 引擎的 plan（百万元）。它做三件事。

历史基数。 基期收入、经营性 EBIT、折旧摊销、资本开支、营运资本（AR + INV − AP，与三表同口径）、有效税率（所得税/利润总额）全部来自真值表；缺失项不静默补 0，而是写一条 `placeholder` 说明并进入报告。基期 FCFF 只作列示，不参与折现。

预测向量。 三表模型对每个情景 s 导出 `dcf_drivers`（`three_statement._dcf_drivers`）：

$$\left(g_t^s,\ \frac{\mathrm{EBIT}_t^s}{R_t^s},\ \tau,\ \frac{\mathrm{Dep}_t^s}{R_t^s},\ \frac{\mathrm{Capex}_t^s}{R_t^s},\ \frac{\mathrm{NWC}_t^s}{R_t^s}\right)_{t=1..N}, \tag{45}$$

`build_plan._scenarios` 用它们整体替换假设文件情景中的六个向量字段，假设文件只剩情景说明、永续增长率与 WACC 调整。若旧假设文件里还写着 `ebit_margin` 等字段，程序写一条说明"已不再驱动 DCF"，而不是报错——旧文件仍能跑，但不会悄悄生效。DCF 的税率取三表的现金税率，即无杠杆税负：FCFF 不含利息抵税，利息抵税体现在 WACC 的 $K_d(1-T)$ 里。

股权价值桥。 现金、有息负债（不含租赁）、租赁负债、少数股东权益、摊薄股本来自真值表；非经营性资产来自假设文件。明细写法下由 `resolve_non_operating_items` 逐项处理（算法 9）：每项的 `raw_value` 必须在所写页码的原文中按数字边界找到（与事实抽取同一个证据谓词 `external.find_evidence`，第 6 节），单位换算与加总由程序完成，`sign=-1` 表示类负债项。找不到原文的条目直接报错，整个估值流水线停止。

茅台的假设文件列了九项：拆出资金 99,096.2、买入返售金融资产 8,879.7、一年内到期的非流动资产 26,871.1、发放贷款和垫款 1,553.5、债权投资 1,113.0、其他债权投资 3,496.5、长期股权投资 147.2、其他非流动金融资产 4,105.1 百万元，以及作为类负债扣减的吸收存款及同业存放 −18,038.4 百万元，程序加总为 127,224.1 百万元。这些科目来自其控股的集团财务公司，性质接近货币资金。模型没有写合计数，合计是程序算的；任何一项页码或写法不对，流水线都会停下。

8.4.2 折现、终值与股权价值

`dcf.forecast` 由驱动向量逐期还原：

$$\begin{aligned} R_t &= R_{t-1}(1 + g_t + \Delta g), \qquad \mathrm{EBIT}_t = R_t(\mu_t + \Delta\mu), \qquad \mathrm{NWC}_t = R_t\, n_t, \\ \mathrm{FCFF}_t &= \mathrm{EBIT}_t(1-\tau_t) + R_t d_t - R_t \kappa_t - (\mathrm{NWC}_t - \mathrm{NWC}_{t-1}), \end{aligned} \tag{46}$$

Δg、$\Delta\mu$ 是敏感性与反向 DCF 用的平移量，正常运行时为 0。这是式 (2) 的逐期写法。折现率为

$$W_s = w_E(R_f + \beta \cdot \mathrm{ERP} + \mathrm{SP} + \mathrm{CSP} + \mathrm{CRP}) + w_D K_d(1-T) + w_P K_p + \mathrm{adj}_s, \tag{47}$$

其中 SP、CSP、CRP 为规模、公司特定、国别溢价，adj_s 为情景调整（样例悲观 +0.5%、乐观 −0.25%）。企业价值

$$\mathrm{EV}_s = \sum_{t=1}^{N} \frac{\mathrm{FCFF}_t}{(1+W_s)^{t-1/2}} + \frac{\mathrm{TV}_N}{(1+W_s)^N}, \qquad \mathrm{TV}_N = \begin{cases} \dfrac{\mathrm{FCFF}_N(1+g_\infty)}{W_s - g_\infty} & \text{永续增长法} \\ \mathrm{EBITDA}_N \times M & \text{退出倍数法} \end{cases} \tag{48}$$

预测期现金流用年中折现（第 t 年折现 $t-\frac{1}{2}$ 年），终值按第 N 年末折现。程序同时报告两种方法互相隐含的参数：永续增长法隐含退出倍数 $\mathrm{TV}/\mathrm{EBITDA}_N$，以及隐含永续增长率 $(\mathrm{TV}\cdot W - \mathrm{FCFF}_N)/(\mathrm{TV}+\mathrm{FCFF}_N)$。股权价值按式 (15) 逐项加减（加：现金、非经营性资产、联营投资；减：有息负债、租赁、少数股东权益、养老金、优先股、期权、其他类债务），每股价值等于股权价值除以摊薄股本。

结构校验。 `validate_plan_structure` 在计算前拒绝以下输入：必需字段缺失；预测期不是 1 ~ 15 的整数；基期收入或股本不为正；资本结构权重之和偏离 1 超过 10^{-6}；任一情景向量长度与预测期不符；任一情景 $W_s \le 0$；永续增长法下任一情景 $W_s \le g_\infty$。最后一条是 Gordon 公式 [24] 收敛的必要条件，必须在三个情景上分别检查，因为情景 WACC 调整与情景永续增长率是分开写的：基准满足不代表乐观满足。

8.4.3 三表核对与模型检查

不变量 22 (DCF 与三表同口径)**.** *对每个情景 s、每个预测期 t 与每个科目 x（营业收入、EBIT、折旧摊销、资本开支、营运资本），DCF 由式 (46) 还原的 x_t^s 与三表模型输出之差的绝对值不超过 10^{-6} 百万元。*

这是第 3.6 节 (V2) 的实现。驱动以比率形式传递，DCF 再乘回收入，浮点上不必然逐位相等，所以要显式核对。`run._tie_to_three_statement` 计算全部 $3 \times N \times 5$ 个偏差的最大值；超过 10^{-6} 时把消息放进硬性失败，模型状态变为 `blocked`。样例运行的最大偏差为 4.55×10^{-13}，三家真实公司为 1.5×10^{-11}、3.6×10^{-12}、1.8×10^{-12} 百万元。`tests/test_quarterly_ebit.py::test_dcf_blocked_when_three_statement_disagrees` 把三表基准情景首期 EBIT 人为加 1 百万元，断言核对失败且状态为 `blocked`。

`compute_checks` 还给出以下判定：情景排序不满足 $V_{\text{down}} \le V_{\text{base}} \le V_{\text{up}}$（警示）；基准股权价值 ≤ 0（硬性失败）；终值现值占 EV 超过 75%（警示）；预测期末 EBIT 利润率较历史提升超过 3 个百分点（警示）；永续增长法隐含 EV/EBITDA 超过 20 倍（警示）；关键输入来自分析师假设（警示）；敏感性方向性检查未通过（硬性失败）。由于预测、WACC、终值三类输入在本项目中总是分析师假设，DCF 的模型状态最好只能是 `screen-grade`。我们保留了这个“天花板”：它如实说明估值依赖观点，而不是事实。

8.4.4 敏感性矩阵与方向性检查

`run_sensitivities` 在基准情景上生成三张 5×5 矩阵：WACC × 永续增长率、WACC × 退出倍数、收入增速 × EBIT 利润率。WACC × g 矩阵中 $W \le g$ 的格子置空并注明“无意义”，而不是给出负数或无穷大。方向性检查（`_directionality`）对矩阵逐行逐列检查单调性：每股价值沿 WACC 递减、沿永续增长率递增、沿利润率递增。这对应第 3.6 节 (V4) 的后半句。

要点：收入增速不在方向性检查里

方向性检查有意不要求“每股价值随收入增速递增”。在 FCFF 模型里，增长需要投入：式 (46) 中营运资本增加为 $n_t(R_t - R_{t-1})$，资本开支为 $\kappa_t R_t$。当税后利润率较低时，多增长一个百分点带来的 NOPAT 不足以覆盖多占用的营运资本，价值反而下降。样例的收入增速 × 利润率矩阵正好出现了这种情况：利润率下调 3 个百分点那一行，收入增速从 −2% 到 +2%，每股价值从 7.74 元单调降到 7.39 元；利润率上调 3 个百分点那一行则从 15.23 元升到 16.23 元。增长创造价值的条件是投入资本回报率高于 WACC [32]，程序不应把它当作恒成立的方向。

权衡（Trade-off）：DCF 情景向量全部取自三表，而不是假设文件的 `ebit_margin`

早期版本让分析师在假设文件里直接写各情景的 EBIT 利润率、折旧率、资本开支率、营运资本占比，三表模型另跑一套。结果是研报首页的盈利预测表（来自三表）与估值（来自 DCF）用的是两套利润率，复核编辑无法判断哪个对。我们考虑过三种修法：(a) 保留两套输入，加一条“差异不超过 x”的检查——阈值无从设定，而且仍有两份可能矛盾的假设；(b) 让 DCF 驱动三表——DCF 只有六个比率，推不出资产负债表；(c) 让三表驱动 DCF。我们选了 (c)：三表输入更细（毛利率、费用率、周转天数、折旧率），能推出 DCF 需要的全部比率，反之不行。代价是假设文件的情景表达变间接：要让悲观情景利润率更低，必须改毛利率或费用率，而不能直接写 EBIT 利润率。这正是我们想要的——利润率变化必须落到可解释的经营驱动上。

权衡（Trade-off）：FCFF，而不是 FCFE 或股利贴现

FCFF 与资本结构无关，资本结构只在 WACC 里出现，三表模型的债务假设（大多数情况下是“维持不变”）不会直接改变估值；FCFE 需要逐期预测净借款，三表模型里债务是外生假设，估值会被一个分析师随手写的还款计划左右。股利贴现模型对 A 股高分红公司（茅台分红率 79%）可以作为交叉验证，但对不分红或分红政策多变的公司不可用。我们只实现 FCFF，与原插件一致。这个选择对金融企业不成立：银行、保险的负债是经营原料，没有“经营资产 + 有息负债”的划分，FCFF 无定义。代码中没有银行分支，估值链只覆盖非金融企业；茅台集团财务公司这类“非金融企业里的金融子公司”通过非经营性资产明细（含类负债项）近似处理，并在依据中写明利息收入仍在 EBIT 内、存在少量重复计算。金融企业需要剩余收益或股利贴现模型 [12]，列为后续工作。

权衡（Trade-off）：年中折现从财年末起算，终值按年末折现

引擎的折现起点是最近财年末，不是估值基准日。茅台、宁德、招商轮船的估值日都在 2026 年 9 月下旬，距 2025 年末已有 9 个月，严格做法应把第一期现金流按剩余期间折现（stub period），并把股权价值滚动到估值日。另外，年中折现下终值仍按第 N 年末折现，与“第 $N+1$ 年现金流在年中到达”的假设不完全一致。这两处都沿用原插件口径，以保持 (V6) 的一致性，并在估值报告的口径说明中披露。按一般经验，忽略约 0.75 年的滚动会使每股价值偏低约 $0.75 \times W \approx 6\%$；这是一个方向确定、量级可估的系统偏差，比引入一个与参照实现不同的日期约定更容易向评审解释。

8.5 情景、敏感性表与流动性压力测试

DCF 三情景给出每股价值区间，`scenarios.py` 再把它与财报结果组装成六张“情景偏度与敏感性”表，表格计算在 `scenario_tables.py` 中完成，口径、列名与判定规则与原插件逐字节一致。缺少输入时单元格写 `input required`，不臆造数值。

三情景目标价与偏度。 设参考股价 P，情景目标价 V_s 与概率 q_s（默认 25/50/25，概率须齐全、在 $[0,1]$ 内且和为 1，容差 10^{-4}），门槛收益率 h（默认 10%）。程序计算：情景收益 $r_s = V_s/P - 1$；概率加权目标价 $\bar{V} = \sum_s q_s V_s$；下行/上行比 $\rho = |\min_s r_s^-| / \max_s r_s^+$；盈亏平衡概率 $p^* = (P - V_{\min})/(V_{\max} - V_{\min})$（落在 $[0,1]$ 外记为空）。偏度标签与动作规则：$r \geq h$ 且 $\rho \leq 0.75$ 为 underwriteable upside；$r > 0$ 为 optical upside / needs proof；$r < 0$

为 negative skew；动作规则在 $r \le -15\%$ 时为 trim / exit。样例中 $V = (6.17, 11.67, 17.73)$，$P = 12.80$，加权目标价 11.81 元（−7.8%），$\rho = 1.35$，$p^* = 57.4\%$：乐观情景的发生概率要超过 57.4% 才值得持有，这比单看基准情景 −8.9% 更直观。

其余四张表。 估值倍数表按 EV/EBITDA 与 P/E 倍数轴给出隐含股价（EV 法扣净债务：有息负债 + 租赁 + 少数股东权益 − 现金 − 非经营性资产）；EPS 修正 × 倍数变化表给出 $\mathrm{EPS}(1+e) \times \mathrm{PE}(1+c)$；KPI 表给出收入冲击与利润率冲击下的 EBIT 变动 $R(1+\Delta R)(\mu+\Delta\mu) - R\mu$（样例收入 −10%、利润率 −300 bp 时为 −232.4 百万元）；跟踪触发表把财报异常信号（第 7 节 S1 ~ S8）映射为阈值与下一步动作，例如 S3“净现比 < 0.5 → 排查应收与存货，下调现金流假设”。最后由 `classify_readiness` 给出就绪度。

流动性压力测试与已知问题。 第六张表按收入冲击档位 s 计算受压自由现金流 $\mathrm{FCF} \times (1+s)$，再算 现金 + 循环贷余额 + 受压 FCF − 一年内到期债务 与最低流动性要求的差额。

失败案例 3：FCF 为负时冲击方向相反

样例公司 2024 年经营现金流减资本开支为 −446.7 百万元。按 $\mathrm{FCF} \times (1+s)$，−10% 的“不利”冲击得到 −402.1 百万元，流动性余量从基准的 140.0 升到 184.6 百万元；+5% 冲击反而使余量降到 117.6 百万元。冲击方向与直觉相反。正确的写法是 $\mathrm{FCF} + s \cdot |\mathrm{FCF}|$ 或直接冲击收入再传导。我们没有改：该表与原插件逐字节一致是 (V6) 的一部分，修改后 golden 快照与对照测试都要放宽；而研报里这张表只作附录参照，不进入评级。处理方式是在 `equity_liquidity_downside` 的 docstring、本项目 `CLAUDE.md` 的“已知问题”与估值报告中注明。这是“与参照一致”和“口径正确”冲突时我们少数选择前者的地方，理由是影响范围可控且已披露。

8.6 反向 DCF：参考股价隐含了什么

正向 DCF 回答“按这些假设值多少钱”，反向 DCF 回答“按现价，市场隐含了什么假设”[46]。第 3.6 节式 (17) 把它定义为单变量方程 $V(\theta) = P$。实现上有三个要点：选哪些变量、怎样参数化、怎样在非单调时求解。

四个杠杆。 固定其余假设为基准情景，分别平移一个变量：(1) 收入增速：每期同加常数 $\Delta g \in [-0.9, 2.0]$，保持基准的年度形态，报告隐含的预测期复合增速 $(R_N/R_0)^{1/N} - 1$；(2) EBIT 利润率：每期同加 $\Delta\mu \in [-1, 1]$，报告隐含的预测期平均利润率；(3) 折现率：$\Delta W \in [g_\infty - W + 10^{-4},\ 1]$，解出的 $W + \Delta W$ 就是“按基准现金流、以现价买入的隐含年化回报率”；(4) 终值参数：永续增长法下 $\Delta g_\infty \in [-0.5,\ W - g_\infty - 10^{-4}]$，退出倍数法下 $\Delta M \in [-M, 200]$。区间端点留出 10^{-4}，保证 Gordon 公式在整个搜索区间内有定义。

为什么不能直接二分。 每股价值对利润率、折现率、永续增长率单调，但对收入增速不一定单调。除了第 8.4 节的投入资本效应，还有一个更隐蔽的原因：终值以最后一年的 FCFF 为基数，而最后一年的营运资本增加 $n_N(R_N - R_{N-1})$ 随该年增速放大。在稳态增速 γ 下，$\mathrm{FCFF}_N/R_N = \mu(1-\tau) + d - \kappa - n\gamma/(1+\gamma)$；样例 $n \approx 0.32$，γ 从 5% 升到 20% 时，最后一项从 1.5% 升到 5.4%，吃掉了 $\mu(1-\tau) + d - \kappa \approx 9.8\%$ 中的一大半，而永续增长率仍是 2%。高增长被“冻结”在终值年，终值基数反而变小。因此先扫描、再二分（算法 10）。

选“离基准最近的根”有明确的经济含义：若存在多个根，离基准最近的那个是“现价要求的最小改变量”。

分档。 每个解出的隐含值由程序与基准、历史区间（真值表最近五个年度，EBIT 口径同定义 8.1）比较后分档：“不高于基准假设”“高于基准假设，但在历史区间内”“超出历史

表 46　反向 DCF 结果（基准 → 现价隐含值，程序分档）。

公司	价格/基准	收入复合增速	平均 EBIT 利润率	隐含回报率	永续增长率
星河智造（样例）	12.80 / 11.67	8.17% → 无解（最高 11.94 元）	12.91% → 13.75%，超出历史最好	8.21% → 7.72%	2.00% → 2.59%
贵州茅台	1237.0 / 1531.4	4.32% → −1.91%	67.34% → 53.78%	7.65% → 9.03%	2.50% → 0.73%
宁德时代	291.11 / 383.31	12.75% → 4.09%	16.44% → 12.27%	8.31% → 10.64%	2.50% → −0.67%
招商轮船	20.15 / 7.51	4.02% → 25.91%，超出历史最好（9.22%）	28.64% → 52.48%，超出历史最好（25.12%）	8.36% → 4.53%	1.50% → 5.75%

最好水平”“单独调整达不到参考股价”；折现率单独分为“现价隐含回报率高于/不高于基准 WACC”。主笔会话只能引用这些分档，不能自行判断“市场预期过于乐观”之类的结论——那是 guard 能拦下的观点类数字（第 10 节）。

表 46 是四次运行的结果。茅台与宁德的现价低于基准每股价值，所有杠杆的隐含值都“不高于基准”，解读为现价已计入比基准更保守的预期。招商轮船相反：现价要求预测期收入年均增长 25.9% 或平均 EBIT 利润率 52.5%，都远超公司历史最好水平，或者接受 4.5% 的隐含回报率。样例公司的收入增速杠杆无解：网格上每股价值在复合增速约 14.6% 时达到最高 11.94 元，仍低于 12.80 元，正是上文非单调性的实例。`tests/test_reverse_dcf.py` 把每个解代回正向 DCF，断言每股价值与参考股价相差不超过 10^{-5} 元；并断言样例的增速杠杆最高点位于搜索区间内部、高于基准每股价值、低于参考股价。

权衡（Trade-off）：逐变量单独求解，只动 DCF 层

联合求解（例如同时调增速与利润率）有无穷多解，需要再加一个目标函数（最小改变量范数），而不同变量的量纲不同，范数没有自然的权重。单变量解虽然不是“市场的真实预期”，但每个都有明确含义，可以直接写成一句话。另一个简化是只平移 DCF 的驱动向量，不回推三表：提高收入增速时，三表里的营运资本天数、折旧率不随之重算，而是沿用 DCF 层的占收入比。完整回推需要每次迭代重跑三表（每个杠杆 400 个网格点加二分），成本可以接受，但会使反向 DCF 的结果依赖三表中“哪些假设随增速变动”这一新的、未写出的假设。两条简化都写入产物的 `notes`。

8.7　可比公司估值

`comps.py` 实现相对估值，计算规则依据原插件 comps 参考文档（EV 桥、倍数配对、NM、IQR 离群、P25/P75 区间），代码自研。

倍数。 对目标公司与每家同业：EqV = P×股本，EV = EqV + 有息负债 + 少数股东权益 − 现金 − 非经营性资产（式 (3)）。EV 类倍数配 EV 口径分母（营业收入、EBITDA、EBIT），P/E 配归母净利润；分母 ≤ 0 记 `NM`，不参与统计。目标公司行由 `target_row` 从 DCF plan 生成：EBIT 用经营性口径，有息负债含租赁，非经营性资产与 DCF 股权桥相同，保证两种方法用同一座桥。

分层与统计。 同业分为核心（Core）、次要（Secondary）、剔除（Excluded，须写理由）。只有核心同业驱动估值区间。分位数用 `statistics.quantiles(method="inclusive")`，与 Excel 的 `QUARTILE/PERCENTILE.INC` 相同，使工作簿公式能复现。离群值用 Tukey 栅栏 [56]：$[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$ 之外标记为离群，只在核心样本 ≥ 4 家时启用，标记后剔除重算分位数，但原始数据保留在表中。

表 47　样例公司的可比公司估值（核心同业 5 家，次要 1 家，剔除 1 家）。

倍数	核心 P25	核心中位数	核心 P75	目标公司	相对中位数
EV/营业收入	2.11x	2.15x	2.33x	1.86x	−13.4%
EV/EBITDA	10.30x	11.55x	11.66x	10.08x	−12.7%
EV/EBIT	13.44x	14.94x	16.20x	14.71x	−1.5%
P/E（LTM）	18.29x	19.79x	22.67x	16.72x	−15.5%
隐含股价（元）：EV/EBITDA 13.10 / 14.79 / 14.94；P/E 14.00 / 15.15 / 17.36；区间 13.10 ~ 17.36，中枢 14.97					
利润率差（目标 − 核心中位数）：EBITDA −1.56 pp，EBIT −2.97 pp，净利率 −0.94 pp					

隐含股价与区间。 对 EV/EBITDA 与 P/E（LTM）两个主倍数，取核心同业（剔除离群）的 P25、中位数、P75 乘以目标公司分母；EV 倍数再减去股权桥调整后除以股本。区间下限为两种方法 P25 隐含价的较小者，上限为 P75 的较大者，中枢为两个中位数隐含价的中位数。若有 DCF 区间，再报告 DCF 基准相对 comps 中枢的偏离与两个区间是否重叠。

利润率对比。 倍数的溢价或折价常常由利润率差异解释。程序计算目标公司与核心同业的 EBITDA、EBIT、净利率，以及目标与核心中位数之差（百分点），写入 `margins` 与工作簿“利润率对比”页。亏损同业的净利率照录为负值而不是 NM（收入为正即可计算），测试 `test_comps_margin_sheet` 对此有断言。

表 47 是样例结果。EV/营业收入一栏中恒川重工（8.4x EV/EBITDA、收入规模最大）被 IQR 栅栏标为离群，剔除后核心样本为 4 家。DCF 基准 11.67 元比 comps 中枢低 22.0%，但两个区间重叠（DCF 6.17 ~ 17.73 元）。目标公司各倍数普遍低于同业中位数，利润率差给出了一个可检查的解释：EBIT 利润率低于同业中位数 2.97 个百分点。

权衡（Trade-off）：中位数与 P25/P75，离群只标记

同业样本通常只有 4 ~ 8 家，均值会被一家高估值公司拉偏，所以以中位数为中枢、四分位为区间 [36]。离群值“标记并在统计中剔除，但不删数据”，是为了让读者看见被剔除的是谁，并能在工作簿里改分层后重算。样本少于 4 家时 IQR 本身不稳定，不做离群判断，同时把状态降为 `screen-grade`。

8.8　盈利预测摘要与程序评级

`forecast_summary.build` 生成研报首页“盈利预测与估值”表的数据：实际年度取真值表，预测年度取三表基准情景。三表模型只有合并净利润，归母净利润用基期占比近似：

$$\mathrm{NP}^{\mathrm{parent}}_t = \mathrm{NI}_t \times \frac{\mathrm{NP}^{\mathrm{parent}}_0}{\mathrm{NI}_0}, \qquad \mathrm{EPS}_t = \frac{\mathrm{NP}^{\mathrm{parent}}_t}{\text{总股本}_0}, \qquad \mathrm{PE}_t = \frac{P}{\mathrm{EPS}_t}. \tag{49}$$

样例占比为 0.9478，2025E 归母净利润 634.2 百万元，EPS 0.793 元。两条简化（少数股东损益占比不变、股本按期末总股本不考虑回购注销）写入 `notes`。

评级按式 (16)：$u = V_{\mathrm{base}}/P - 1$，阈值下限含本数，默认 $u \ge 20\%$ 买入、$\ge 5\%$ 增持、$\ge -5\%$ 中性、$\ge -20\%$ 减持、其余卖出；假设文件可用 `rating_rule` 覆盖阈值，覆盖后的规则文字随评级一起输出。表 48 汇总四次运行。

表 48　四家公司估值结果（金额百万元，每股元；WACC 与 g_∞ 为基准情景）。

	星河智造	贵州茅台	宁德时代	招商轮船
参考股价	12.80	1237.00	291.11	20.15
WACC / g_∞	8.21% / 2.0%	7.65% / 2.5%	8.31% / 2.5%	8.36% / 1.5%
预测期 FCFF 现值	1,873.6	377,694.9	298,325.9	18,238.0
终值现值	8,116.5	1,370,676.8	1,128,020.1	58,789.4
企业价值	9,990.1	1,748,371.7	1,426,346.0	77,027.4
+ 现金	1,356.7	51,690.6	333,512.9	4,720.9
+ 非经营性资产	0.0	127,224.1	143,057.0	7,999.1
− 有息负债	1,670.0	44.2	116,851.4	27,271.2
− 租赁负债	35.0	189.5	2,805.1	1,126.7
− 少数股东权益	308.3	9,321.4	33,918.6	749.2
股权价值	9,333.5	1,917,731.2	1,749,340.9	60,600.3
摊薄股本（百万股）	800.0	1,252.3	4,563.8	8,074.5
每股价值悲观/基准/乐观	6.17/11.67/17.73	1118.10/1531.40/1879.69	246.98/383.31/527.30	2.61/7.51/12.59
终值占 EV	81.2%	78.4%	79.1%	76.3%
隐含退出 EV/EBITDA	7.5x	13.8x	10.0x	7.8x
上行空间	−8.9%	+23.8%	+31.7%	−62.8%
程序评级	减持	买入	买入	卖出

权衡（Trade-off）：评级由程序给

卖方评级本质上是观点，交给模型“综合判断”似乎更自然。我们不这样做，有三个理由。其一，评级必须与估值数字一致（(V5)、评分标准 L0-05），程序评级使一致性按构造成立，guard 只需核对正文评级字符串与产物相等。其二，模型给评级时有“向上取整”的倾向：在早期旧流程的稿件里，曾出现基准每股价值低于现价的公司仍被写成“增持”、理由是“长期空间”的情况，这类表述无法核对。其三，阈值规则透明，读者可以自己代入。代价是评级对假设极其敏感——它只是基准情景的函数，忽略了情景偏度与可比公司。我们的处理是把偏度表、comps 区间与反向 DCF 并列放在估值章，由主笔在正文解释评级与这些交叉验证的关系，但不能改评级本身；如果分析师认为评级不合理，唯一的途径是修改并书面说明假设，再重跑。

失败案例 4：强周期公司的机械评级（招商轮船）

招商轮船的基准情景假设 2026 年收入增长 40%、毛利率 44%，随后两年运价回落、2029 年起毛利率回到 28%，基准每股价值 7.51 元，比参考股价低 62.8%，程序评级“卖出”。反向 DCF 显示现价要求的收入增速和利润率都超出历史最好水平，结论在模型内部是一致的。但强周期股的定价常常反映的是周期高点的盈利与重置成本，而不是五年期 FCFF；终值以 2030 年回归中枢后的现金流为基数，几乎把当前景气全部折掉了。这类公司更合适的锚可能是 P/B 或重置成本，本项目尚未实现。我们在研报中保留了程序评级，同时要求主笔把“估值方法对周期股的适用性”写进风险与局限一节。这是一个程序评级“算对了但未必有用”的例子。

8.9　公式联动工作簿

估值的交付物除了 JSON 与 Markdown 报告，还有一本投行排版的 Excel 模型 `model.xlsx` 和可比公司工作簿 `comps_formula_workbook.xlsx`。评审更习惯在 Excel 里检查估值，所以工作簿必须是“活的”：改一个输入，下游全部重算。

标准库写出。 `finagent/` 只允许依赖标准库（pdfplumber 除外，第 4 节），所以 `output/xlsx.py` 用 `zipfile` 加手写 OOXML 生成 xlsx：工作表 XML、共享样式表、图表与绘图关系。三类单元格：普通数值（`Cell`，可标为输入，字色蓝）、公式（`Formula(expr, value)`，写 `<f>` 与缓存值 `<v>`，字色黑；纯跨表引用为绿），以及数组公式（CSE，用于

表 49　model.xlsx 审计结果（本章四次运行）。

公司	工作表	公式数	勾稽项	孤岛	写死数字	结论
星河智造	13	603	25	0	0	通过
贵州茅台	15	988	25	0	0	通过
宁德时代	15	969	25	0	0	通过
招商轮船	14	624	25	0	0	通过

MEDIAN(IF(...))）。含公式的工作簿在 workbook.xml 里设 fullCalcOnLoad，Excel 打开时全量重算；缓存值让不重算的阅读器（预览、审计脚本）也能显示正确数字。样式按卖方模型惯例：Arial 配楷体的富文本分段（中文段楷体、西文段 Arial），金额千分位一位小数、负数括号、零显示“-”，关闭网格线、冻结表头、横向打印。

页面结构。 model.xlsx 依次为估值汇总、盈利预测、DCF、WACC、敏感性、非经营性资产（有明细时）、分部预测（有分部时）、三表-假设、三表-利润表、三表-资产负债表、三表-现金流量表、历史财务、分季度、经营数据、可比公司、模型检查。DCF 页的 NOPAT、FCFF、折现因子、终值、股权桥与每股价值，WACC 页的股权成本与 WACC，都是公式。

公式联动三表与情景开关。 workbook_ts.linked_three_statement 把三表写成公式：利润表取驱动与辅助计算，资产负债表取利润表、辅助计算与现金流量表，现金流量表取利润表与辅助计算，勾稽检查也是公式（ROUND(资产-负债权益,6)）。三表-假设页 B5 是情景开关（1 悲观 / 2 基准 / 3 乐观），每个情景驱动一行，选用行写成 CHOOSE(B5, 悲观, 基准, 乐观），DCF 页的经营行引用三表页，于是改 B5 即联动重算三表与 DCF，结果与引擎各情景输出一致。实现上先用占位符 «IS.rev.C» 写跨表引用，全部页面建好、行号确定后再统一替换为 '三表-利润表'!C7 这样的地址。引擎中本项目不使用的功能（循环贷、现金扫还、可抵扣亏损、回购增发、非默认折旧或成本方法）不写公式；detail() 返回的 supported 为假时退回数值表，避免写出与引擎不一致的公式。

只读审计。 质检侧的 evaluation/workbook_audit.py 不改写工作簿，只做三类检查：(1) 结构：公式数、缺缓存、错误值、#REF!；(2) 公式控制：跨期公式“孤岛”（把相对引用换成 R1C1 式偏移后，某期写法与左右两期都不同而左右两期相同，例如中间一年被手工改成数值）判失败，公式内写死的数字（约定常数 0、0.5、1、2、4、12、100、365，ROUND 精度、QUARTILE 分位序号与 ≤ 0.01 的勾稽容差除外）与易变函数（NOW、OFFSET、INDIRECT 等）列为警告；(3) 五组勾稽：对每个预测列，用来源单元格的缓存值独立复算资产负债平衡、现金衔接、留存收益滚动、固定资产滚动、有息负债滚动，同时要求勾稽单元格的公式文本与预期完全一致（防止“缓存为 0 但公式被改坏”）。只有被评 PDF 同目录下字节一致的 sheet.xlsx 才会被审计。

表 49 中四本工作簿的最大勾稽残差均为 0。审计的局限写在结果的 limitation 字段里：它没有在 Excel 或 LibreOffice 中重算全部公式，只验证公式结构、缓存、跨期写法与五组勾稽。公式与引擎的等价性由另一条路径保证：test_quarterly_ebit.py 断言 DCF 页“每股价值 = 股权价值/股本”的公式缓存值等于引擎基准每股价值；团队在 Excel for Mac 与 LibreOffice 中各手工切换三次情景开关，核对了三表与 DCF 的重算结果。

表 50　一致性守护（THIRD_PARTY.md 第 1 节）。

参照引擎	自研实现	对照方式	标准
三表联动	ts_engine.py	样例 + 3 个边界 + 60 组随机 plan（收入/成本/营运资本/PP&E 各口径、亏损与可抵扣亏损、循环贷与扫还、分红回购增发、契约、来源时效边界）；另用原插件的 plan 校验器检查生成的 plan	浮点逐位相等
情景敏感性表	scenario_tables.py	样例 + 边界 + 180 组随机 payload（收益恰等于门槛、下行/上行比恰为 0.75、溢出为 inf）	JSON 逐字节相等
FCFF DCF	dcf.py	20 组随机 plan × 3 情景；EV、股权价值、每股价值、终值占比与全部敏感性格	相对误差 $< 10^{-9}$
业绩分析	earnings.py	300 组随机输入（见第 7 节）	逐字段相等
golden 快照	全部	tests/golden/ 四个 JSON（三表、情景表、comps、业绩），用 all 命令跑样例后与快照逐字段比较；无需原插件	完全相等

权衡（Trade-off）：手写 OOXML，而不是 openpyxl

openpyxl 能省去大量 XML 细节，但它写出的公式没有缓存值（需要 Excel 打开后才有数），而审计器与测试都要读缓存；它也是一个需要写进 THIRD_PARTY.md 的运行时依赖。手写的代价是 525 行代码和对 OOXML 细节（富文本、数组公式、图表关系）的维护。我们接受这个代价，因为“打开即有正确数字、不打开也能审计”是工作簿可追溯的前提。

8.10　与参照引擎的口径一致性

估值引擎的早期版本直接调用原 OpenAI Public Equity Investing 插件的脚本。该插件许可证为内部使用，不能随参赛源码分发，因此我们参照其计算规则重写了全部引擎，仓库不含其代码（附录 F）。重写引入的风险是“看起来一样、细节不同”。我们用两层测试守护第 3.6 节的 (V6)，见表 50。

两层的分工。 对照测试（test_parity_reference.py）在本地存在原插件时运行，检验“引擎函数在任意输入上与原引擎相同”；golden 快照（test_golden.py）不需要原插件，检验“本项目整条流水线在样例上的产物没有意外变化”。前者守护引擎，后者守护口径映射。2026-09-28 我们有意修改了口径（经营性 EBIT、轧差经营费用、DCF 由三表驱动、comps 增加利润率），重新生成了快照并在快照 README 记录差异原因；引擎本身未改，对照测试照常逐位通过。撰写本章时在本机运行估值相关的 8 个测试文件，共 42 个测试全部通过（含对照测试）。

权衡（Trade-off）：逐位相等，而不是容差相等

三表与情景表要求逐位相等，DCF 要求相对误差 $< 10^{-9}$。逐位相等迫使我们复刻浮点运算顺序（例如 (begin + 0.5*capex)*rate 不能写成 begin*rate + 0.5*capex*rate），写代码时更费事；好处是任何差异都是真差异，不必讨论“10^{-7} 算不算一致”。DCF 放宽到 10^{-9}：DCF 模块是按原插件的公式重新组织的（例如情景向量统一在 forecast 中展开），没有逐行复刻运算顺序，而 10^{-9} 的相对误差远小于任何展示精度（$\frac{1}{2}$ ulp，第 10 节），不会改变任何对外数字。

8.11　端到端数值示例：星河智造

本节用虚构样例公司星河智造（data/samples/）走一遍完整链条，所有数字取自命令 python -m finagent all --input data/samples/annual_report_2024.md --year 2024 ...（完整参数见附录 G）的实际产物。样例数据内部完全自洽，真实数据的效果会差一些，这

表 51　星河智造三表模型基准情景（百万元）。

	FY2025	FY2026	FY2027	FY2028	FY2029
营业收入	6,565.8	7,222.4	7,800.2	8,268.2	8,681.6
EBITDA	1,211.0	1,332.1	1,438.6	1,524.9	1,601.2
EBIT	835.6	926.0	1,007.0	1,074.6	1,137.5
净利润	669.2	746.0	814.9	872.3	925.8
营运资本增加	227.5	212.3	186.8	151.3	133.7
资本开支	656.6	650.0	624.0	578.8	564.3
期末现金	1,517.2	1,806.9	2,242.6	2,835.1	3,526.6
资产 − 负债与权益	0.0	0.0	0.0	0.0	0.0

一点在第 8.12 节讨论。

第 1 步：历史基数。 2024 年营业收入 5,862.34 百万元，利润总额 760.12、财务费用 45.30、投资收益 23.50、公允价值变动收益 41.20 百万元，经营性 EBIT = 740.72 百万元（利润率 12.64%）；折旧摊销 340.5、资本开支 685.2 百万元，营运资本 1,895.7 百万元（较上年增加 588.2）。基期 FCFF 为 $740.72 \times (1 - 15\%) + 340.5 - 685.2 - 588.2 = -303.3$ 百万元：公司处于扩产期，经营现金被应收与资本开支吃掉。

第 2 步：三表驱动。 程序从报表反算：毛利率 31.19%，轧差经营费用率 12.74%，应收、存货、应付周转天数 123.63、101.38、109.51 天，折旧率 11.05%，税率 15.00%，利率 2.83%。假设文件只给收入增速（12%、10%、8%、6%、5%）与资本开支率（10%～6.5%），其余驱动保持基期。三表基准情景见表 51；六项勾稽最大误差 3.6×10^{-12} 百万元，三表状态 `senior-review-ready`，无警示。

第 3 步：FCFF 与折现。 WACC：$K_e = 1.8\% + 1.10 \times 6.5\% + 1.0\% = 9.95\%$，$K_d(1 - T) = 3.5\% \times 0.85 = 2.975\%$，$W = 0.75 \times 9.95\% + 0.25 \times 2.975\% = 8.206\%$。各年 FCFF 为 201.6、330.8、476.7、633.6、732.6 百万元，年中折现后合计 1,873.6 百万元。终值 $732.58 \times 1.02/(8.206\% - 2\%) = 12{,}040.0$ 百万元，折现后 8,116.5 百万元，占 EV 的 81.2%（触发终值占比警示）。EV = 9,990.1 百万元；股权价值 = 9,990.1 + 1,356.7 − 1,670.0 − 35.0 − 308.3 = 9,333.5 百万元；每股价值 9,333.5/800 = 11.67 元。隐含退出倍数 7.5x，低于假设文件中作交叉验证的 10x；用 10x 退出倍数法得到 15.01 元，这个差距本身就是一条应写进研报的信息。

第 4 步：情景与敏感性。 悲观情景（收入增速 5%/3%…、毛利率 −2 个百分点、应收天数 +15、WACC +0.5%、$g_\infty = 1.5\%$）每股 6.17 元；乐观情景 17.73 元。WACC × g_∞ 矩阵的四角为 12.11（$W - 1\%, g - 1\%$）、17.42（$W - 1\%, g + 1\%$）、8.72（$W + 1\%, g - 1\%$）、11.24 元（$W + 1\%, g + 1\%$），方向性检查全部通过。

第 5 步：交叉验证与评级。 可比公司区间 13.10～17.36 元（表 47）；反向 DCF 显示现价要求平均 EBIT 利润率 13.75%，超出历史最好水平（13.03%）；概率加权目标价 11.81 元。基准上行空间 $11.67/12.80 - 1 = -8.9\%$，落在 $[-20\%, -5\%)$，程序评级“减持”。

8.12　真实公司：茅台、宁德时代、招商轮船

三家公司的假设文件由分析师会话起草（状态字段写明“以下全部为观点/假设，不是财报事实”），抽取文件与假设随代码提交在 `data/extractions/`，评审无需模型即可复

现表 48。三家展示了估值链的三种用法。

贵州茅台：分部驱动 + 大额非经营性资产。 收入按茅台酒、系列酒量价推演：基准情景茅台酒销量年增 3%、吨价 2026 年持平后年增 1%～2%，系列酒 2026 年吨价下降 6%、销量增 3%。程序算出的合并收入增速为 2.18%、3.89%、5.18%、5.18%、5.18%，综合毛利率维持 91.2% 左右；悲观情景 2026 年收入下降 5.66%。公司无有息借款，按全股权融资，WACC 即股权成本 7.65%。非经营性资产 127,224.1 百万元占股权价值的 6.6%，若不加回，每股价值会低约 102 元。终值占 EV 78.4%，隐含退出倍数 13.8x，低于依据中写的交叉验证倍数 18x。

宁德时代：GWh 量价 + 高资本开支。 动力电池与储能电池按 GWh × 单价预测，基准情景合并收入增速 20.0%、15.2%、12.7%、9.2%、7.2%，毛利率从 25.7% 缓降到 25.1%（单价下行、毛利率逐年下调 0.3～0.5 个百分点）。资本开支率 10% 起逐年回落，末年资本开支是折旧的 1.32 倍。非经营性资产为交易性金融资产、长期股权投资、其他权益工具投资与其他非流动金融资产四项，合计 143,057.0 百万元；货币资金 333,512.9 百万元全额加回（含保证金等受限资金，依据中注明“口径简化”）。三表模型触发 `STALE_HISTORICAL_SOURCE`：历史数据截至 2025 年末，估值日为 2026 年 9 月 30 日，超过 270 天。

招商轮船：无法量价拆分时的合并驱动。 年报未披露各船型营运天数与等价期租租金，无法写 `segment_forecast`。分析师直接给出合并收入增速（40%、−10%、−8%、2%、3%）与毛利率路径（44%、36%、30%、28%、28%），以及负的经营费用率（失败案例 1）。三表模型触发 `STEP_UP_REVENUE`（首年收入增长超过 25%）。长期应付款（主要为租赁公司借款）作为类负债项在非经营性资产明细中扣减 677.4 百万元。评级与适用性讨论见失败案例 4。

要点：终值占比在四家公司中都超过 75%

四次运行的终值现值占 EV 在 76%～81% 之间，都触发了警示。这不是某家公司的特殊问题，而是“五年显式预测 + 永续增长”结构在低利率环境下的共同特征：$W - g_\infty$ 只有 5.5～6.9 个百分点，终值倍数 $1/(W-g)$ 约为 15～18 倍。因此敏感性矩阵里 WACC 与 g_∞ 的影响远大于收入增速，研报的估值讨论应把篇幅放在折现率与长期增长上。评分标准 L1 的“终值占比”条目（第 12 节）检查的正是正文有没有披露这一点。

8.13 局限与小结

本章的估值链有以下已知局限，均已在估值报告口径说明或 `CLAUDE.md` 中披露：(1) 折现从财年末起算，未处理估值日之前的 stub period；终值按年末折现。(2) 资产处置收益、其他收益、减值与营业外收支留在 EBIT 中，会被永续化；招商轮船的卖船收益靠分析师在费用率假设中手工剔除。(3) 利息收入通过负利率挂在有息负债上，不随现金变动（失败案例 2），影响预测期 EPS，不影响 FCFF。(4) 流动性压力测试在 FCF 为负时方向相反（失败案例 3）。(5) 只覆盖非金融企业；银行、保险需要剩余收益或股利贴现模型。(6) 终值年未做“正常化”（资本开支向折旧收敛、营运资本增加按永续增速重算），高增长假设会被冻结进终值。(7) 反向 DCF 逐变量求解且不回推三表。

这些局限的共同点是：它们都是确定性的口径选择，方向可知、可以复算、可以在报告中逐条说明；而本章要消除的问题——模型心算错误、DCF 与预测表两套假设、无法复算的评级——都是不可复算的。把估值的全部数值工作交给约四千行标准库 Python，模型只负责写一份带依据的假设文件，是 LEDGER 在估值建模上的核心取舍：模型的判断仍然决定估值结论，但每一个判断都落在一个具名字段上，每一个数字都能从这些

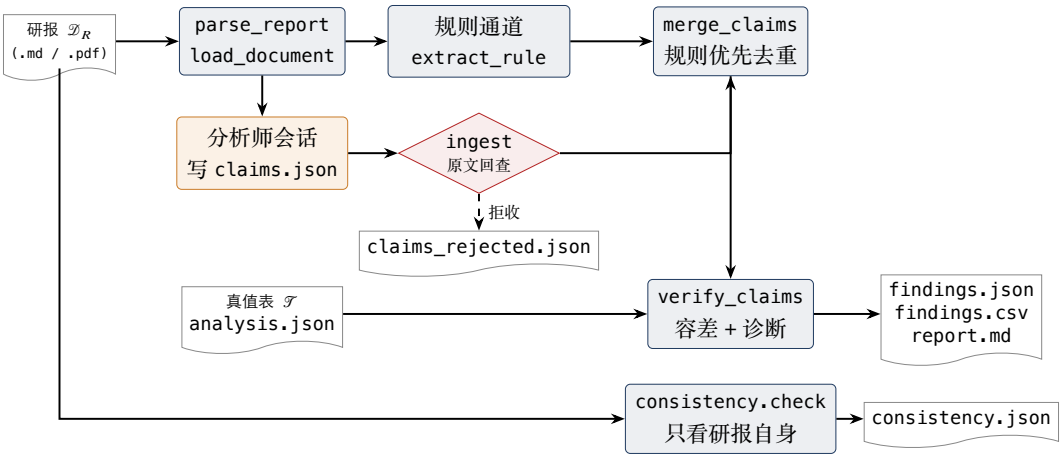


图 13 研报纠错流水线。蓝色为确定性程序，橙色为模型会话，红色菱形为拒收检查。模型通道是可选的：没有 `claims.json` 时只走规则通道（样例回归与测试用）。真值表由财报分析（第 7 节）产出，是核对的唯一数据来源；内部一致性检查不读真值表。

字段与年报原文复算出来。

9 研报纠错

纠错核查子选题要求系统读一篇研报，找出其中与财报不符的数字，说明错在哪里，并给出修改建议。第 3.7 节已经把任务形式化为“断言 γ + 真值函数 T_τ + 按展示精度的容差 + 有序判定函数”，第 4.8 节讨论了“误报控制优先于召回”这条原则，并给出错误类型诊断的伪代码（算法 2）与保守性命题（命题 4.1）。本章不重复这些定义，而是往下讲一层：断言怎样从中文研报里抽出来、两条通道如何合并、容差中那个看似随意的 0.005 从何而来、诊断的搜索空间多大、巧合误诊的概率有多高、修改建议如何生成、研报内部一致性检查有哪些排除规则，以及这些设计在样例与变异实验上的实际表现和失败边界。

模块由三个文件组成，全部是确定性 Python，不调用任何模型：`finagent/tasks/report_check/claims.py`（断言抽取与合并，173 行）、`verify.py`（逐条核对、错误类型诊断与修改建议，173 行）、`consistency.py`（研报内部一致性，214 行）。它们在 `finagent/pipelines.py` 中注册为 `report_check` 流水线的四个步骤 `parse_report` → `claims` → `verify` → `consistency`（图 13）。模型（分析师会话）在这条链路上只做一件事：可选地把研报拆成断言写进 `claims.json`；拆得对不对，由程序按原文回查。

9.1 输出契约：七类判定与一条发现的结构

先说清楚系统交出去的是什么。每条断言最终变成一条“发现”（finding），字段如表 52 所示。判定共七类（定义 3.11），其中五类计为错误（代码中的 `ERROR_VERDICTS`），“正确”与“无法核实”不计。表 53 列出每一类的判定条件、所需的证据（witness）和修改建议模板。证据这一列是本章反复出现的概念：除“数值错误”外，每个错误类型都必须有一个能被程序复验的替代解释，系统才会给出这个类型。

为什么判定是有序的。 七类判定互斥，但“错误类型”的成立条件并不互斥：一个数可能同时等于上年值（期间）和净利润（口径）。有序判定让结果确定、可复现，并且把证据更强、巧合概率更低的解释排在前面（第 9.5 节给出排序依据）。“数值错误”放在最后，是因为它是一个不需要证据的判定：只要 $\hat{x} \not\approx t$ 它就为真。我们在第 9.7 节把它称

表 52　一条发现（verify_claim 的返回值）的字段。前九个字段来自断言，后六个由核对步骤填写。

字段	含义
claim_id	规则通道 C001 起编；模型通道先以 L001 起编，合并后统一重编为 C·
metric_id, metric_name	标准指标 ID 与中文名（指标字典共 49 个指标，见第 6.4 节）
period	期间串：2024、2025Q3、2025H1、2025Q1–3
claim_type	level / yoy / ratio / pp_change
raw_value, value, unit	原文写法（不带符号）、带方向的数值、单位
quote, sentence	断言在原文中的最短片段与所在整句
location	source、page、chunk_id、句序号（规则通道）
is_forecast, method	预测性表述标记；抽取通道 rule / llm
verdict	七类判定之一
truth_value, truth_display	真值（元或百分数值）与按研报单位、精度格式化后的展示值
evidence	真值的出处：列示值给 (source, page, chunk_id, row, col)；派生值给公式与输入单元格
explanation, suggestion	中文说明与修改建议

为“最低承诺标签”，并说明为什么在没有证据时退回它是决策论意义上的最优选择。

9.2　断言抽取：规则通道

9.2.1　指标说法字典与最长匹配

规则通道依赖指标字典（tasks/financials/schema.py）中的“研报说法”（claim_aliases）。49 个标准指标中有 39 个配了研报说法，共 56 个说法，如营业收入的“营业收入 / 营业总收入 / 营收 / 收入”、归母净利润的“归母净利润 / 归属于上市公司股东的净利润 / 归属于母公司股东的净利润”、经营现金流的“经营活动现金流量净额 / 经营性现金流 / 经营现金流”等。claim_alias_list() 把所有（说法, 指标）对按说法长度降序排列。

对每个句子，算法在一个布尔占用数组上做不重叠的最长匹配：按长度从长到短遍历说法，找到一次出现后，若该区间内任何字符已被占用则跳过，否则登记该提及并占用区间。最长优先的作用是让“扣非归母净利润”不会被“归母净利润”或“净利润”截胡，“营业总收入”不会被“收入”截胡。句子切分用正则 (?<=[。；;！!？?\n])，即在句末标点之后切开，标点留在前一句。

9.2.2　提及之间的片段与数值识别

找到句中全部指标提及并按位置排序后，第 j 个提及的“作用域”是从它结束处到下一个提及开始处（最后一个提及到句末）的片段 seg_j。数值只在作用域内找，这是规则通道把数字绑定到指标的唯一机制：“营业收入 58.62 亿元，同比增长 14.49%”里，两个数字都在营业收入的作用域内；“扣非归母净利润 4.44 亿元，同比下降 5.67%，非经常性损益 1.68 亿元”里，“1.68 亿元”落在非经常性损益的作用域内。表 54 列出用到的几个模式。

在作用域内，水平值与同比按如下规则判别：

表 53　七类判定的条件、证据与修改建议模板（`verify.verify_claim`）。“检查顺序”即算法 2 的分支顺序；$\hat{x}$ 为规范化后的研报数值，$t = T_\tau(m,k)$，$\approx$ 为按容差相符（定义 3.10）。

顺序	判定	条件与证据	修改建议模板
1	无法核实	预测性表述，或 t 无定义	无（说明“需核对盈利预测模型”或“真值表缺少……”）
2	正确	$\hat{x} \approx t$	无
3	单位错误	τ = level，存在 $u' \in \textsc{UnitAlts}(u)$ 使按 u' 换算后 $\approx t$；证据为 u'	将“$r\,u$”改为“t 按 u' 的展示值”
4	方向错误	$\tau \in \{\text{yoy}, \text{pp}\}$，$-\hat{x} \approx t$；证据为符号	将增减方向及数值改为 t
5	期间错配	存在 $k' \neq k$，$\hat{x} \approx T_\tau(m,k')$；证据为 k'	改为 k 期的 t，或更正期间为 k'
6	口径错误	存在同组 $m' \neq m$，$\hat{x} \approx T_\tau(m',k)$；证据为 m'	若指 m 应为 t；若指 m' 请更正指标名称
7	数值错误	以上都不成立；证据为空	将 quote 中的数值改为 t（金额另给相对偏差）

表 54　规则通道使用的模式（`claims.py`）。`NUM` 匹配带千分位与小数的数字。

模式	作用与要点
`UNIT`	亿元、万元、千元、百万元、元/股、元、%、倍、个百分点、pct、pp
`YOY_RE`	“同比 / 较上年 / 比上年 / 较去年同期”+ 可选增减动词 + 数字 +（% 或百分点类单位）；动词属于“下降 / 减少 / 下滑 / 降低”时数值取负
`VAL_RE`	可选引导词（为、达、实现、录得、至、约、共计、合计、是）+ 数字 + 单位；只在同比表述之前的部分里找，取第一个
`YEAR_RE`	年份 + 可选“年 / E / A / 度”+ 可选季度词（第一季度、Q3、前三季度、上半年、H1）；用于期间解析
`FORECAST_HINTS`	预计、预测、有望、目标、将达、`2025E`–`2027E`、指引；句中出现任一即标为预测

1. 先找同比表述 `YOY_RE`。找到则生成一条 `yoy`（单位为%）或 `pp_change`（单位为个百分点）断言，数值按动词定号。
2. 在同比表述之前找第一个 `VAL_RE`。若其单位是百分点类，跳过（那是变动，不是水平）。
3. **变动额排除**：若数字前紧邻“增加 / 减少 / 增长 / 下降 / 提升 / 提高 / 降低 / 上升 / 下滑（了）”，且引导词不是“为 / 达 / 至 / 实现 / 录得 / 是”，则判定为变动额（“研发费用增加 0.94 亿元”），不抽取。“增长至 58.62 亿元”因引导词“至”仍按本期值抽取。
4. **百分数归属**：单位为% 时，若指标本身是比率类（`kind="pct"`，如毛利率），生成 `ratio`；若指标是金额类，前面有“增 / 降 / 减 / 升”字样才视为增速（`yoy`），否则（如“收入占比 35%”）不抽取——金额指标后面的百分数多半是占比，不是这个指标的值。
5. 单位为“倍”的水平值不抽取（研报里的“倍”多是估值倍数，不在真值表中）。

9.2.3　期间解析

期间取指标提及之前（含提及本身）最后一个年份表述：`_period_of(sent[:en])`。若年份后跟季度词则拼接季度后缀（“第三季度”→ `Q3`，“前三季度”→ `Q1–3`，“上半年”→ `H1`）；句中没有年份时取报告年度（`default_year`）。取“最后一个”是因为中文研报常把年份写

表 55　规则通道的边界（以报告年度 2025 实测）。“结果”列为 extract_rule 的实际输出。

句子	结果	说明
销售费用增加 0.94 亿元。	不抽取	变动额，无法与水平值核对；有意排除
海外收入占比 35.20%。	不抽取	金额指标后的百分数无增减字样，视为占比
归母净利润同比增长约三成。	不抽取	中文数词不在 NUM 中；约数本应由程序算出（设计规范 1）
2025 年第三季度营业收入 23.10 亿元，环比增长 5.2%。	仅 revenue/2025Q3/level	没有环比类型；环比表述被忽略
销售费用率由 3.30% 升至 4.30%。	ratio=3.30%	取作用域内第一个数，把上期值当本期值；会误报“期间错配”
营收约 60 亿元。	level=60 亿元	“约”不放宽容差：真值 58.62 亿元时判“数值错误”

在句首、一句话说多个指标：“2024 年公司实现营业收入……，归母净利润……”中两个指标共享句首的年份。

例 9.1 (样例研报第 1 页第三句)**.** *句子“扣非归母净利润 4.44 亿元，同比下降 5.67%，非经常性损益 1.68 亿元，主要来自政府补助及资产处置收益。”最长匹配得到两个提及：“扣非归母净利润”(np_parent_deducted) 与“非经常性损益”(non_recurring)。“政府补助”也是一个说法 (gov_subsidy)，但其后作用域内没有数字，不生成断言。第一个提及的作用域是“4.44 亿元，同比下降 5.67%，”：YOY_RE 命中“同比下降 5.67%”，生成 yoy 断言 $x = -5.67$；截到同比之前的部分里 VAL_RE 命中“4.44 亿元”，生成 level 断言。第二个提及的作用域内命中“1.68 亿元”。期间均取默认年度 2024。三条断言对应表 62 中的 C005–C007。*

命题 9.1 (规则通道的结构性质)**.** *对规则通道产生的任一断言 γ：(i) γ.quote 是 γ.sentence 的连续子串，γ.sentence 是研报某个文本块的连续子串；(ii) γ.raw_value 出现在 γ.quote 中；(iii) 同一输入两次运行产生逐字段相同的断言序列。*

证明. (i)(ii)：quote 取自 sent[st:en+vm.end()] 或 sent[en+ym.start():en+ym.end()]，都是句子的切片，且切片终点包含数值匹配；句子由文本块按标点切出。(iii)：说法列表按长度排序（等长时按字典构造顺序，Python 的排序稳定），正则匹配与遍历顺序都是确定的，没有随机性与外部状态。 □

性质 (i)(ii) 使每条发现都能在研报原文中被逐字定位，这是“核查结论可定位到研报具体句子”这一输出要求的基础；性质 (iii) 使样例评测可以作为回归测试（tests/test_pipeline.py 中 test_report_check 对同一输入跑两遍并要求发现列表完全相等）。

9.2.4　规则通道识别不了什么

规则通道的覆盖面是有意收窄的：宁可不抽，也不要抽错（抽错的断言会产生误报）。表 55 列出我们在实现与真实文稿上确认过的边界，每一行都用代码实际跑过。

最后两行是规则通道会抽错的情形。“由 X 升至 Y”在主笔会话写作时同样踩到过：研报 skill 的“常见坑”一节专门要求改写为“销售费用率 4.30%，上年为 3.30%”（finagent-report/SKILL.md）。对外部研报，这类句式由模型通道补救：模型拆断言时能理解“升至”，写出 value="4.30"，quote 为“升至 4.30%”；但规则通道抽出的那条 3.30% 仍在，合并时

表 56　模型断言的拒收规则（`claims.ingest_claims`），按检查顺序。

#	拒收理由	检查
1	未知指标	`metric_id` 不在指标字典中
2	断言类型不合法	`claim_type` 不属于 level / yoy / ratio / pp_change
3	quote 不是研报原文中的连续片段	不存在文本块 c 使 quote 是 c.text 的子串
4	数值不在 quote 中	去掉负号与千分位后的 value 不是去掉千分位后 quote 的子串
5	数值无法解析	`float(value)` 失败

两者键不同（数值不同），都会被核对，3.30% 那条会被诊断为“期间错配”（它确实等于上年值）。这条“误报”在内容上是对的——3.30% 确实是上年值——只是研报作者并没有把它当作本期值。我们在第 9.10 节把它列为已知误报来源。

9.3　断言抽取：模型通道与合并

9.3.1　`claims.json` 的接入与回查

模型通道的动机是覆盖规则通道漏掉的写法：长句中指标与数字相隔很远、表格化的研报、“由 X 升至 Y”、指标用了字典外的说法等。分析师会话按 `finagent-research` skill 的要求读研报，把断言写成 JSON 列表，每条包含 `metric_id`、`period`、`claim_type`、`value`（字符串，带符号）、`unit`、`is_forecast` 与 `quote`。程序不信任其中任何一个字段，按表 56 逐条回查，不通过的写进 `check/claims_rejected.json` 并附理由。

规则 3 与 4 共同保证了与规则通道相同的结构性质（命题 9.1 的 (i)(ii)），这是“证据定位”不变量（不变量 11）在研报一侧的对应物。注意两点实现细节。第一，模型给的 `value` 带符号（“同比下降 5.67%”写 `"-5.67"`），而原文里只有“5.67”，所以规则 4 先去掉负号再查；入库时 `raw_value` 存不带符号的写法、`value` 存带符号的数值，与规则通道约定一致。第二，规则 4 只检查子串而不检查数字边界，所以模型把“14.49”写成“4.49”也能通过——这是一个已知的宽松处，但后果有限：错写的数值会在核对阶段被判为错误并展示 quote，人一眼能看出。

9.3.2　合并：规则优先的并集

合并键是（指标，期间，类型，数值保留 6 位）。规则通道的断言全部保留且排在前面，模型断言只在键未出现时追加。在样例研报上，分析师会话写下的 `data/extractions/星河智造 _2024/claims.json` 共 19 条，回查全部通过、拒收 0 条，19 个键与规则通道完全重合，合并后仍是 19 条、新增 0 条——样例的句式都在规则通道覆盖范围内，模型通道在这里只起到“交叉确认”的作用。

命题 9.2 (合并的性质)**.** *设 $\mathcal{C}_r$、$\mathcal{C}_\ell$ 为两通道断言，$\mathcal{C} = \textsc{Merge}(\mathcal{C}_r, \mathcal{C}_\ell)$。则 (i) $\mathcal{C}_r$ 是 $\mathcal{C}$ 的前缀，其中每条断言的字段不被修改；(ii) $\mathcal{C}$ 中任意两条断言的键互不相同，当且仅当 $\mathcal{C}_r$ 内部的键互不相同；(iii) $\mathcal{C}$ 的键集合等于两通道键集合的并。*

证明. (i) 由 out 以 rule 初始化且之后只追加可得。(ii) 追加时检查键不在 seen 中；若规则通道自身有重复键（同一句同一指标出现两次相同数值），它们原样保留。(iii) 每个模型键要么已在 seen（来自规则或先前追加），要么被追加。 □

性质 (ii) 的“当且仅当”不是虚设：规则通道不对自身去重。如果研报在两处写了同样的“营业收入 58.62 亿元”，会产生两条断言、两条发现，它们定位到不同句子，这对

修改建议是有用的（两处都要改）。

权衡（Trade-off）：断言为什么取并集，而财务事实要冲突阻断

财务事实的两通道合并遵循“不静默取舍”原则：同一单元格两通道不一致就记 issue、阻断估值（命题 6.2）。研报断言却取并集、不比对，看似双重标准，实则对象不同。财务事实是*真值*，选错一个就污染了所有下游；研报断言是*待检验的假设*——“研报在这里说了某指标某期间等于某数”——每条都要独立地拿真值核对，多一条假设不会污染真值表，最坏结果是多一条发现。代价在于“多一条发现”可能是误报：模型把“毛利率 32.50%”的 `metric_id` 写成 `net_margin`，quote 和数值都能回查通过，却会被核成“数值错误”甚至“口径错误”。程序无法验证“这句话说的是哪个指标”（这需要理解语义），只能验证数字确实在原文中。*缓解*：(1) 规则优先，规则通道已抽到的键不会被模型的错误版本替换；(2) 每条发现附 quote，人工复核时能直接看到原句；(3) 口径错误的修改建议写成双向的（“若指 m 应为……；若指 m' 请更正指标名称”），即使指标标错，建议本身仍然正确。*未选方案*：对模型断言做“规则通道能否抽出同一键”的交叉验证、不能则丢弃。这等于放弃模型通道存在的意义（补规则漏掉的句式）。

9.4 真值与按展示精度的容差

9.4.1 真值函数的实现

`verify.Truth` 封装真值表 `FactTable`，实现第 3.7 节的 T_τ：`level` 与 `ratio` 直接取 $\mathcal{T}(m,k)$；`yoy` 用 `pct_change`（基期为负时按 A 股惯例以 |基期| 作分母，基期为 0 返回无定义）；`pp_change` 是两期比率之差。基期由 `prior_periods` 给出：`2024→2023`、`2025Q3→2024Q3`、`2025H1→2024H1`。因此“2025 年第三季度营业收入同比增长”核对的是单季对单季，前提是真值表里有两个年度的单季拆分（第 7.4 节）。

`Truth.evidence` 返回出处链：列示值返回单元格坐标（同比类返回本期与基期两个坐标）；派生值（真值表中 `method="derived"`）返回公式与输入单元格。例如样例中毛利率的出处是“(营业收入 − 营业成本)/营业收入”及其两个输入在年报第 6 页与第 106 页的位置，这条链最终写进 `report.md` 的“来源”列，满足可追溯性定义（定义 3.16）的 (a)(b) 两款。

9.4.2 容差：三个加数各自管什么

定义 3.10 的容差可以拆成三部分：

$$\mathrm{tol}(\gamma) \;=\; \underbrace{\tfrac{1}{2}\,\mathrm{ulp}(r)\,\phi(u)}_{\text{展示舍入}} \;+\; \underbrace{10^{-6}|T|\cdot\mathbb{1}[\tau=\text{level}]}_{\text{浮点与长数字}} \;+\; \underbrace{0.005\cdot\mathbb{1}[\tau\neq\text{level}]}_{\text{二次舍入}}. \tag{50}$$

第一项保证“四舍五入安全”（命题 3.2）。下面讨论另外两项，以及第一项中的因子 $\phi(u)$。

换算因子只作用于金额与股数。 `_factor(m,u)` 只在指标的 `kind` 为 `amount` 或 `shares` 时按单位换算，其余（每股、比率）一律取 1。这避免了“元/股”被误当作“元”去乘因子，也意味着*每股指标的单位写错检测不到*：“基本每股收益 0.77 亿元”会被判“正确”。我们在变异实验中复现了这一点（第 9.9.2 节，单位变异 21 例中 3 例漏判，全部是每股收益）。现实中把每股收益写成亿元的情形罕见，我们接受这个盲区。

浮点项 $10^{-6}|T|$。 真值以元存储，换算后的浮点误差在 $10^{-16}|T|$ 量级，10^{-6} 远大于此。它真正起作用的场合是研报写出*全精度*的长数字：“营业收入 5,862,340,125.36 元”的 $\frac{1}{2}\,\mathrm{ulp}\cdot\phi = 0.005$ 元，而 $10^{-6}|T| \approx 5{,}862$ 元。此时容差由浮点项主导，等价于相对误差 10^{-6}。这是一个有意的放宽：年报里同一数字在不同表格中偶有分位差异（四舍五入到元与保留到分），我们不希望把这种差异判为错误。对两位小数的“亿元”，浮点项约为 58.62×10^{-6} 亿元，比展示舍入项小三个数量级，不影响结论。

表 57 二次舍入的量级：若研报用两位小数的“亿元”展示值计算比率或增速，式 (51) 给出的偏差上界（百分点），与系统容差的对比。

场景	a（亿元）	b（亿元）	二次舍入界	系统容差	覆盖
研发费用率（样例）	3.13	58.62	0.0090	0.010	✓
资产负债率（样例）	45.13	98.42	0.0074	0.010	✓
营业收入同比（样例）	58.62	51.20	0.0209	0.010	✗
小公司增速	1.20	0.80	1.5625	0.010	✗

二次舍入项 0.005。 对比率、同比与百分点变动，容差额外加 0.005（百分数单位）。它来自一个在真实研报中普遍存在的做法：作者用*已经四舍五入的*金额去算比率或增速。设分子、分母的展示值为 a', b'，展示舍入半宽分别为 h_a, h_b，真实值为 a, b，则

$$\left|\frac{a'}{b'} - \frac{a}{b}\right| \;\lesssim\; \frac{h_a}{b} + \frac{a\,h_b}{b^2} \qquad \Longrightarrow \qquad |g' - g| \;\lesssim\; 100\Big(\frac{h_a}{b} + \frac{a\,h_b}{b^2}\Big)\text{（百分点）}, \tag{51}$$

其中 $g = 100(a/b - 1)$ 是增速（比率同理，去掉 -1）。作者再把 g' 按两位小数写出，又引入 0.005 的舍入。以样例的研发费用率为例：研发费用 3.13 亿元、营业收入 58.62 亿元（均为两位小数，$h = 0.005$），用展示值算得 $3.13/58.62 = 5.3395\%$，写作 5.34%；真值 312,650,000/5,862,340,125.36 $= 5.3332\%$，写作 5.33%。两种写法都合情合理，差一个末位。式 (51) 给出的界为 $100(0.005/58.62 + 3.13 \times 0.005/58.62^2) \approx 0.0090$ 个百分点，加上展示舍入 0.005，总偏差可达约 0.014。纯展示精度容差（0.005）会把 5.34% 判为“数值错误”，加上 0.005 后容差为 0.01，$|5.34 - 5.3332| = 0.0068$，判“正确”。

表 57 说明 0.005 是一个*折中*而不是推导出的界：它覆盖了比率类（分子小于分母时界较小）的大部分二次舍入，却覆盖不了增速类（分子分母量级相同，界约为比率的两倍），更覆盖不了小公司（分母越小，同样的展示舍入造成的相对误差越大）。后两种情形下，用展示值算增速的研报可能被误判“数值错误”，偏差在末位上 1 ~ 2 个单位。

权衡（Trade-off）：容差取多宽

固定相对容差（如 1%）。实现最简单，但与展示精度脱钩：“58.6 亿元”与“58.62 亿元”应有不同的容差；对增速，1% 的相对容差在 14.49% 上只有 0.14 个百分点，在 0.5% 上只有 0.005，既不一致也难解释。**严格展示精度（不加 0.005）。**理论上最干净，命题 3.2 照样成立；但样例中的研发费用率在“作者用展示值计算”时就会误报，这类写法在卖方研报里很常见。**按式 (51) 逐条计算二次舍入界。**最精确，但需要知道作者用了哪两个展示值，而断言里只有比率本身；猜测分子分母会引入新的误判。**选择：**展示精度 + 固定 0.005。它让两位小数比率的有效容差变为 ±0.01，代价是放过“末位差 1”的真实错误——变异实验中末位扰动 99 例有 3 例被放过，全部是比率与增速（第 9.9.2 节）。一个末位差 1 的比率，读者既无法察觉，也不会因此改变判断，我们认为这是可以接受的漏报。

边界与约数。 比较用闭区间（≤），真值恰好落在两个展示值中点时两种写法都判正确，偏向放行。另一方面，“约 60 亿元”中的“约”不放宽容差：`decimals_of("60")=0`，容差为 0.5 亿元，真值 58.62 亿元时判“数值错误”（表 55 最后一行）。我们没有为“约”“近”“超过”引入额外容差，原因是它们的语义宽度因人而异（“约 60 亿”可以指 55 ~ 65，也可以指 59.5 ~ 60.5），任何固定宽度都是拍脑袋；写作规范要求我们自己的研报不写约数（设计规范 1），外部研报中的约数被判“数值错误”时，发现中同时给出真值与相对偏差（−2.3%），读者能自行判断。

表 58　各诊断分支的候选集合（verify.py）。$|\mathcal{C}_m|$ 为真值表中指标 m 的期间数。

分支	适用类型	候选集合
单位错误	level	UNIT_ALTS：亿元 →{万元, 元, 百万元}；万元 →{亿元, 元, 千元}；元 →{万元, 亿元}；百万元 →{亿元, 万元}；千元 →{万元, 元}
方向错误	yoy, pp_change	一个候选：$-\hat{x}$
期间错配	全部	真值表中 m 的全部其他期间（年度、单季、H1、Q1-3），$|\mathcal{C}_m| - 1$ 个
口径错误	全部	m 所在易混组的其他成员（6 组，组大小 2∼5，CALIBER_GROUPS），同一期间

9.5　错误类型诊断：搜索空间、排序与巧合误诊

算法 2 给出了诊断的骨架。本节补充三件事：每一类诊断实际搜索的候选集合有多大；为什么按“单位 → 方向 → 期间 → 口径”排序；一个本来只是数值写错的断言，被巧合地误诊为某个具体类型的概率有多高。

9.5.1　搜索空间

单位候选不是全排列，而是“相邻量级”：亿元的候选不含千元（差 10^5 倍，现实中几乎不会混淆）。期间候选是指标在真值表中的全部期间，对有季报补充的公司可达 10 个以上（两年 × 年度、四个单季、H1、Q1-3）。口径候选只在六个易混组内搜索，详见定义 3.11。单位分支所用的容差是“按替代单位重新算的展示精度”$\frac{1}{2}\,\mathrm{ulp}(r)\phi(u')$：研报写“2.38 万元”，按亿元解释时容差是 0.005 亿元而不是 0.005 万元，这才是“如果作者本意是亿元，他写下 2.38 时的舍入误差”。

9.5.2　排序的依据：证据强度

排序遵循“证据越强越先认”。单位错误的证据是数字串完全一致，只是量级错了：2.38 万元与真值 2.3846 亿元，数字“2.38”逐位相同，对一个随机写错的数而言，恰好与真值在另一量级上逐位相同的概率可以忽略。方向错误的证据是绝对值完全一致，同样很强。期间错配要求数值等于同一指标的另一期真值——同一指标的历年值通常相差不大（增速 ± 几十个百分点），巧合概率比前两类高。口径错误跨指标搜索，同组指标的数值可能相当接近（净利润与归母净利润常差几个百分点），巧合概率最高，因此排在最后、并且只在易混组内搜。

9.5.3　巧合误诊的概率

设研报作者把真值 t 写错成 $\hat{x}$，而 $\hat{x}$ 只是一个普通的数值错误（并非单位、期间或口径混用）。诊断分支 j 有候选值集合 W_j，每个候选 w 带容差窗口 $[w-\epsilon_w, w+\epsilon_w]$。若写错的数在 t 附近宽度为 Λ 的区间内近似均匀分布（例如 $\Lambda = 0.6|t|$ 表示 ±30% 的错误），则

$$P(\text{误诊为某具体类型}) \;\le\; \sum_{j} \sum_{w \in W_j,\ |w-t| \le \Lambda/2} \frac{2\epsilon_w}{\Lambda}. \tag{52}$$

这与第 4.14 节式 (31) 的巧合放行估计同构，只是这里的“值集”是一条断言的少数几个候选，而不是整个产物。以样例的归母净利润（$t = 6.12$ 亿元，两位小数，$\epsilon = 0.005$ 亿元）为例：取 $\Lambda = 0.6 \times 6.12 = 3.67$ 亿元，单个候选的命中概率约为 $0.01/3.67 = 0.27\%$；落在区间内的候选最多有上年值（5.18 亿元）、净利润（6.46 亿元）、扣非归母净利润（4.44 亿元）与营业利润、利润总额，至多 5 个，上界约 1.4%。也就是说，一个普通的

数值错误被误诊为期间或口径错误的概率在百分之一的量级；单位与方向两类则低于万分之一。

更重要的是误诊的后果。由命题 4.1，任何具体类型都带一个能让断言变“正确”的单一修改，即使类型被误诊，诊断陈述的事实（“该数值等于净利润”）也是真的，修改建议的第一半（“若指 m 应为 t”）仍然正确。误诊因此最多造成“解释不对”，不会造成“建议改错”。这是我们把口径错误的建议写成双向句式的原因。

失败案例：期间不在真值表中时，期间错配退化为数值错误

研报写“2025 年营业收入 23.10 亿元”，而 23.10 亿元其实是 2025 年第三季度的单季值。若真值表中有 `revenue/2025Q3`，诊断会命中期间分支并给出“该数值对应 2025Q3 期”；若公司只提供了年报、真值表里没有单季数据，就只能判“数值错误”。这是算法设计的直接结果：诊断只认存在于真值表中的替代解释。我们没有引入“大约是年度值的四分之一”之类的启发式，因为它没有可复验的证据，违背命题 4.1。这类召回损失在评测中计入“类型准确率”而非“召回率”：错误仍被判出，只是类型较粗。

9.6 修改建议的生成

修改建议由模板生成（表 53 最后一列），关键在于真值的展示格式。`_fmt` 按以下规则把真值写成研报的口径：

- 小数位数取 $\max(2, d(r))$：研报写两位就给两位，写三位就给三位，写整数也给两位（“毛利率 36%”的建议是“26.47%”而不是“26%”）。取下限 2 是为了避免建议值本身因舍入过粗而不可用（“改为 26%”会让读者误以为 26.4% 也对）。
- 水平值按研报所用单位换算，带千分位（“40.34 亿元”）；单位错误时按替代单位展示（“2.38 亿元”）。
- 百分点变动带正负号（“−0.03 个百分点”），同比不带加号，负增长带负号。
- 数值错误另附相对偏差（仅金额水平值）：“相对偏差 +360.08%”。相对偏差帮助读者区分“抄错一位”与“完全不是同一个数”。

建议句一律引用原文 quote（“将‘同比增长 25.30%’中的数值改为 18.17%”），这样编辑可以在研报中直接搜索替换。`report.md` 把错误发现排成“错误清单与修改建议”表，列出位置（页码）、原文、结论、财报真值、建议与真值来源（年报页码、chunk、行列或计算公式），其后是全部断言的核查明细；`findings.csv` 是同样内容的表格版本，便于评委用电子表格筛选。

命题 9.3 (建议的可执行性)**.** 对判为“数值错误”“单位错误”“方向错误”的断言，按修改建议替换数值（或单位、方向）后重新核对，结果为“正确”。对“期间错配”“口径错误”，按建议的第一种改法（改数值）替换后结果为“正确”；按第二种改法（改期间或指标名）替换后结果亦为“正确”。

证明. 数值错误与方向错误的建议值是 t 按研报精度（至少两位）展示的值 t'，有 $|t' - t| \le \frac{1}{2}\,\mathrm{ulp}(t') \le \frac{1}{2}\,\mathrm{ulp}(r)$（当 $d(t') \ge d(r)$），故落入原容差。单位错误的建议值按 u' 展示，由分支条件落入按 u' 计算的容差。期间、口径的第二种改法正是命题 4.1 中的单一修改。 □

命题保证了建议“照做就对”。它不保证建议“最合理”：对口径错误，作者可能本意就是净利润，只是写错了名字，此时改名比改数更合适；系统给出两种改法、不替作者选择，正是出于这个考虑。

表 **59**　纠错的代价矩阵。c_{FP}：把正确断言报为错误；c_{FN}：漏掉真实错误；c_{A}：弃权，交人工核对的成本。

动作 \ 真实状态	$\theta=0$（研报正确）	$\theta=1$（研报错误）
报错	c_{FP}	0
放行	0	c_{FN}
弃权	c_{A}	c_{A}

9.7　误报控制的决策论解释

第 4.8 节从工程经验出发论证了“误报优先”。本节用一个简单的代价模型把它形式化，并据此解释三个具体设计：预测性断言判“无法核实”、错误类型要求精确证据、口径诊断限于易混组。

模型。 对一条断言，记隐变量 $\theta\in\{0,1\}$（1 表示研报确实写错），系统观察到的证据为 o（断言、真值、容差比较结果），后验 $p=P(\theta=1\mid o)$。系统有三个动作：报错（flag）、放行（pass）、弃权（abstain，即“无法核实”，交人工）。代价矩阵如表 59。

三个动作的期望代价分别是 $(1-p)c_{\mathrm{FP}}$、$p\,c_{\mathrm{FN}}$ 与 c_{A}。最优规则是带拒识选项的贝叶斯判决 [11, 17]：

$$\text{报错}\iff p\ge\max\Big(\frac{c_{\mathrm{FP}}}{c_{\mathrm{FP}}+c_{\mathrm{FN}}},\,1-\frac{c_{\mathrm{A}}}{c_{\mathrm{FP}}}\Big),\qquad \text{放行}\iff p\le\min\Big(\frac{c_{\mathrm{FP}}}{c_{\mathrm{FP}}+c_{\mathrm{FN}}},\,\frac{c_{\mathrm{A}}}{c_{\mathrm{FN}}}\Big),\tag{53}$$

其余情形弃权。当 $c_{\mathrm{FP}}\gg c_{\mathrm{FN}}$ 且 $c_{\mathrm{FP}}\gg c_{\mathrm{A}}$ 时，报错阈值趋近 1：只有在几乎确定研报写错时才报错。

为什么 c_{FP} 远大于 c_{FN}。 三条理由，前两条是场景性的，第三条是本系统特有的。

1. **基率。**真实研报中绝大多数数字是对的。设错误率为 π、召回率为 ρ、单条误报率为 f，则精确率为 $\pi\rho/(\pi\rho+(1-\pi)f)$。取 $\pi=5\%$、$\rho=0.9$：$f=2\%$ 时精确率只有 $0.045/(0.045+0.019)=70\%$；要让精确率超过 90%，f 须低于 0.5%。在低基率下，误报率的微小变化主导用户看到的结果。
2. **信任。**研报作者面对一份“30 处错误、其中 10 处是误报”的清单，会很快学会忽略所有提示；一次误报的代价包含此后所有真报告的打折。
3. **闭环。**核对引擎被数字护栏复用（`guard.check_binding`，第 10.4 节），用来检查我们自己生成的文稿，且护栏是阶段 8 的硬闸门。护栏误报会把写作会话卡在闸门前，迫使它把本来正确的数字改写成更含糊的说法以绕过检查（第 10.4 节有真实案例）。在这个闭环里，误报的代价不只是一次人工复核，而是报告质量的系统性下降。

三个设计对应的决策。 (1) **预测性断言弃权**：对“预计 2025 年营业收入 67.50 亿元”，历史真值表不提供任何关于 θ 的信息，p 等于先验；由式 (53)，只要 $c_{\mathrm{A}}<\min(\pi c_{\mathrm{FN}},(1-\pi)c_{\mathrm{FP}})$ 就应弃权。我们也可以拿自己的估值模型去比对，但那是在比较两个观点，差异不构成“错误”。(2) **错误类型要求精确证据**：类型判定是第二层决策。设类型误判的代价为 c_{type}，退回“数值错误”的代价为 0（它在 $\theta=1$ 时总为真）。只有当具体类型的后验接近 1 时才值得给出——精确证据使后验接近 $1-P(\text{巧合})$，后者由式 (52) 控制在百分之一量级。(3) **口径只在易混组内搜**：扩大搜索范围会线性增加式 (52) 中的项数。真值表有 47 个指标、上百个单元格，若跨全部指标搜索，一个普通数值错误被“解释”为某个无关指标的概率会上升一到两个数量级。

命题 9.4 (“数值错误”是最低承诺标签)**.** *对任意 $\theta = 1$ 的断言（研报数值与真值超出容差），“数值错误”这一陈述为真；对任意具体类型 $e \in \{$ 单位, 方向, 期间, 口径 $\}$，陈述“γ 属于类型 e”为真当且仅当作者的错误机制确实是 e，而后者不可观测。*

这个命题不需要证明，它的价值在于给出一个清晰的标签层级：系统总可以安全地说“数值不对”，只有在有证据时才进一步说“为什么不对”。样例评测的“类型准确率”衡量的正是第二层决策的质量。

权衡（Trade-off）：弃权的代价

弃权不是免费的。真实研报中预测性表述占比很高（盈利预测、目标价、行业空间），全部判“无法核实”意味着纠错只覆盖历史部分。**未选方案**：把研报的预测与一致预期比对、偏离过大即报“可疑”。这会把“观点不同”当成“错误”，与纠错核查的任务定义不符；一致预期本身也是外部数据，需要登记来源。我们把预测部分的审视交给两个别的机制：我们自己的研报由内部一致性检查保证预测数字前后一致（第 9.8 节），由盲评条目审视预测依据（第 12 节）。

9.8 研报内部一致性

9.8.1 管什么、不管什么

真值表能核对历史数字，却核对不了研报自己给出的数：评级、各情景每股价值、预测年度的收入和利润。这些数在一篇研报中往往出现多次——摘要、正文、估值、投资建议、风险提示——前后写得不一样是卖方研报的常见硬伤，也是我们自己的长文稿（15,000 字以上、五个会话接力写成）最容易出的问题。`consistency.py` 只看研报文本，回答“同一个量在文中是否只有一个值”。其形式定义见第 3.7 节“内部一致性”段，质检中的程序判定（评分条目 N-02）见算法 18 与命题 12.1。本节讲实现中的排除规则，它们是“命中即确认”的来源。

比较的对象（`METRICS` 与评级、每股价值）是有意收窄的：

- **评级**：本报告给出的投资评级（买入、增持、中性、减持、卖出）；
- **每股价值**：基准 / 悲观 / 乐观情景下的每股价值（每股价值、目标价、合理价值、每股内在价值），不分期间；
- **预测年度的公司整体数**：晚于报告期的营业收入、归母净利润、扣非归母净利润、净利润、每股收益、毛利率，以及收入、利润的同比增速。

历史年度的数字不比，因为它们已由真值表核对（研报纠错与护栏），在这里重复比较只会制造重复的发现。

9.8.2 排除规则

每条规则对应一类“看起来像同一个量、其实不是”的情形，都来自真实文稿（表 60）。

这些规则共同把比较限制在“本报告、同一实体、同一口径、同一期间、同一情景、无条件”的数字上。识别的写法有两种：列举式“2026—2028 年归母净利润分别为 5.42 亿元、7.16 亿元、9.18 亿元 [，同比增长 38%、40%、35%]”（`LIST_RE + YOY_LIST_RE`），与单年式“2027 年营业收入 33.10 亿元”（`SINGLE_RE`）。每股价值要求句中出现 DCF、情景或基准字样，避免把引述中的“目标价”算进来。

9.8.3 比较与容差

数字按（指标，期间，类型，情景）归组。每组以*首次提及*为锚，后续提及与锚的差超过两者展示精度半宽之和时记为矛盾：

$$|v_i - v_1| > \tfrac{1}{2}\,\mathrm{ulp}(r_1)\,\phi(u_1) + \tfrac{1}{2}\,\mathrm{ulp}(r_i)\,\phi(u_i), \tag{54}$$

表 60 内部一致性检查的排除规则（`consistency.py`）。任一规则触发，相关数字就不进入比较。

规则	实现	排除的情形
非正文行	`_sentences`：跳过表格行（以 | 开头）、代码块、HTML 注释、< 与 {{ 开头的行	表格中的多年预测列、图表指令
外部来源	`_own_part`：整句含“卖方 / 一致预期 / 机构 / 外资 / 券商 / 可比公司 / 行业平均 / 研报”等即丢弃；带 [S 编号] 的句子只在写明“我们”或 DCF 时保留，并删去带编号的分句	别家的目标价、评级与预测
条件与敏感性	`CONDITIONAL`：敏感、上调、下调、若、如果、一旦、即使、矩阵、最高、最低、区间、剔除、模拟、例如、“时，”……	“WACC 上调 0.5 个百分点时每股价值……”
区间	数值前后紧邻“至 / 到 / —”	“悲观至乐观情景区间为 20.62 元至 109.06 元”
情景	句中只出现悲观、乐观、基准之一即为该情景；都没有算基准；多个同现不比（每股价值先看所在分句）	“基准 49.88 元，乐观 109.06 元”同句
公司整体	`_whole_company`：指标前的文字去掉年份、“我们 / 预计 / 公司 / 全年 / 基准情景”等虚词后必须为空	“其中光模块收入……”（分部口径）
年份	列举写法要求年份跨度与数值个数相等；单年写法要求句中只有一个年份	跨年对比句
评级说明	句中含“≥”或出现两种以上评级词即跳过	“买入：≥20%；增持：≥5%”

其中 v 为换算到元的值。容差取两者之和，是因为两处都可能是同一真值的舍入：真值 49.876 元可以写成 49.88 元，也可以写成 49.9 元，二者之差 0.02 不超过 $0.005 + 0.05$，不算矛盾（测试 `test_display_precision_tolerated`）。反过来，若两个展示值的差超过两者半宽之和，它们不可能来自同一个真值的四舍五入，矛盾是确定的。这正是命题 12.1 的核心：在排除规则正确识别了“同一个量”的前提下，命中即确认。

以首次提及为锚而不是两两比较，是一个呈现上的选择：发现列出锚与所有不一致的提及（含行号），由读者判断哪一处是对的。若锚本身是错的那一处，所有正确的后续提及都会被列出——发现依然成立（确有矛盾），只是“谁对谁错”不由程序判断。PDF 研报按文本块拼接成行，发现中的行号再换算回页码（`pipelines.py` 的 `_c_consistency`）。

真实文稿上的表现。 引入这一检查的提交记录写明：在七份真实文稿（含历史版本与复核前初稿）上零误报，摘要评级反转全部命中。我们在当前五份交付文稿上重跑（报告年度 2025），比较组数分别为中际旭创 6 组（24 处提及）、宇树科技 8 组（20 处）、招商轮船 3 组（11 处）、贵州茅台 2 组（8 处）、宁德时代 4 组（15 处），矛盾均为 0。比较组数不大，这是排除规则严格的直接后果：一篇三万字的文稿里，只有少数几个量满足“本报告、整体口径、无条件、同一情景”并被提到两次以上。样例研报草稿上只识别到

表 61 内部一致性的测试用例（tests/test_consistency.py），基于一份含摘要、估值、盈利预测与投资建议的短文稿。

改动	期望
无改动	0 处矛盾，且比较组数 ≥ 3
摘要评级“卖出”改为“买入”	1 处评级矛盾，描述中同时列出“买入”与“卖出”
估值节每股价值 49.88→54.87；2027 年归母净利润 7.16→7.61	恰好两处：每股价值、2027 年归母净利润
追加“2027 年营业收入 33.10 亿元”与改增速 40%→43%	恰好两处：2027 年营业收入及其同比
每股价值 49.88→49.9	0 处（展示精度内）
追加分部、卖方、敏感性、条件句、区间、历史期重复各一句	0 处（全部被排除）
追加评级说明与两家机构的“买入”评级	0 处
同句“基准 49.88 元，乐观 109.06 元”	0 处（按分句判情景）

1 处评级（“维持‘增持’评级”），无可比较的组。

9.8.4 没有实现的一致性检查

任务描述中常被提到的另外两类一致性——“增速与两期数值不符”和“占比加总不为 100%”——consistency.py 没有实现。原因与处理如下。

- **增速与两期数值。**对历史期，增速与两期水平值分别由真值表核对（yoy 与 level 两条断言），任何一方与真值不符都会被判出，无需在研报内部再比；对预测期，我们自己的研报中增速与水平值都来自 forecast_summary.json（由同一套程序算出），二者的一致由产物保证，护栏只需检查它们有出处。外部研报的预测期增速与水平值的自洽性目前没有检查；实现上需要按式 (51) 处理二次舍入，并识别增速对应的基期，我们把它列为后续工作。
- **占比加总。**分部占比四舍五入后加总常为 99.99% 或 100.01%，还常有“其他”项省略，误报风险高；对我们自己的研报，分部数字来自 extras.json，占比由程序以 ratios 算出，同样由产物保证。

失败案例：护栏调用一致性检查时剥掉了来源编号

撰写本节时，我们在五份交付文稿上分别以原文与护栏两种方式调用一致性检查，发现结果不一致：直接调用 consistency.check 时中际旭创与贵州茅台均为 0 处矛盾，经 run_guard 调用时各报 1 处。原因是 run_guard 先用 MARK.sub 删去全文的 [S 编号] 再交给一致性检查，而 _own_part 正是靠这些编号识别外部引述。中际旭创的句子“高盛给予 A 股目标价 2645 元……[S10]，我们的乐观情景每股价值 918.06 元”失去编号后，“目标价 2645 元”被当成本报告的乐观情景每股价值；贵州茅台的“25 家给出‘买入’评级 [S10]”失去编号后被当成本报告的评级。评分条目 N-02 的程序判定（evaluation/checks.check_consistency）传入的是原文，不受影响；但护栏的 ok 会因此为假，意味着这两份文稿若重跑阶段 8 闸门会被拦下。修复只需把原文而非去标记的文本传给 _contradictions，并补一条回归测试。这个案例说明排除规则依赖的输入表示本身也是规则的一部分；第 10.6 节从护栏一侧再讨论一次。

表 62　样例研报的全部 19 条断言与判定（check/findings.json）。“标注”列为 eval/labels.json 中的期望判定，空白表示未标注（应判正确或无法核实）。

ID	原文片段	类型	研报值	判定	标注	真值 / 说明
C001	营业收入 58.62 亿元	level	58.62	正确		58.62 亿元
C002	同比增长 14.49%	yoy	14.49	正确		14.49%
C003	归母净利润 6.46 亿元	level	6.46	口径错误	E1 口径	6.12 亿元；6.46 为净利润
C004	同比增长 25.30%	yoy	25.30	数值错误	E2 数值	18.17%
C005	扣非归母净利润 4.44 亿元	level	4.44	正确		4.44 亿元
C006	同比下降 5.67%	yoy	−5.67	正确		−5.67%
C007	非经常性损益 1.68 亿元	level	1.68	正确		1.68 亿元
C008	经营活动现金流量净额 2.38 万元	level	2.38	单位错误	E3 单位	应为 2.38 亿元
C009	毛利率为 32.50%	ratio	32.50	数值错误	E4 数值	31.19%（派生）
C010	较上年提升 1.2 个百分点	pp	1.2	数值错误	E5 数值	−0.03 个百分点
C011	研发费用 3.13 亿元	level	3.13	正确		3.13 亿元
C012	研发费用率 5.33%	ratio	5.33	正确		5.33%（派生）
C013	营业成本 35.22 亿元	level	35.22	期间错配	E6 期间	2024 年 40.34 亿元；35.22 为 2023 年
C014	基本每股收益 0.77 元	level	0.77	正确		0.77 元
C015	应收账款 19.86 亿元	level	19.86	正确		19.86 亿元
C016	同比增长 28.50%	yoy	28.50	数值错误	E7 数值	41.60%
C017	资产负债率 45.85%	ratio	45.85	正确		45.85%（派生）
C018	货币资金 13.57 亿元	level	13.57	正确		13.57 亿元
C019	营业收入 67.50 亿元	level	67.50	无法核实		预测性表述

9.9　样例纠错结果与变异实验

9.9.1　样例：19 条断言、7 处埋错

样例研报 data/samples/research_draft.md 是一份三页的年报点评草稿，针对样例年报（虚构公司“星河智造”2024 年报，内部完全自洽）。eval/labels.json 标注了其中 7 处人为植入的错误，覆盖口径、数值、单位、期间四类，以及两类“容易误报”的干扰：一处“下降”方向的正确增速（扣非归母净利润同比下降 5.67%）与一处预测（2025 年营业收入 67.50 亿元）。运行 python eval/run_eval.py（只走规则通道，不调用模型）得到表 62。

汇总指标：召回率 100%（7/7）、精确率 100%、误报 0、错误类型准确率 100%；同一次运行中财报抽取字段准确率 100%（35/35）。四点观察：(1) C006 的“下降”被规则通道正确定号为 −5.67，与真值 −5.673% 相符，没有被误判为方向错误；(2) C010 的真值 −0.03 个百分点表明毛利率几乎持平，研报的“提升 1.2 个百分点”方向和量级都错，但 −1.2 与 −0.03 也不相符，所以不是方向错误而是数值错误——方向分支要求取反后精确相符；(3) C003 的口径诊断依赖易混组 {净利润, 归母净利润, ……}：6.46 亿元恰为净利润（含少数股东损益）；(4) 句末的“当前股价对应 2024 年 PE 约 17 倍”没有生成断言（“倍”的水平值不抽取），“维持‘增持’评级”只进入内部一致性检查。

9.9.2　变异实验

7 处埋错的样本太小。为了更系统地检验判定函数，我们对样例中 11 条判为“正确”的断言做程序化变异，直接调用 verify_claim，看判定是否符合变异算子的预期（表 63）。变异算子：舍入（把真值按更少的小数位重新舍入，应判正确）；末位（把原文末位数字随机换成另一个数字，每条 9 次，应判错误）；相对（数值乘以 1.01/1.03/1.10/1.30 后按原精度写出，应判错误）；换单位（金额水平值换成其他三个金额单位，应判单位错误）；取反（同比与百分点变动取反，应判方向错误）；换期（换成上年同类型真值按原精度写出，应判期间错配）。

两类不符都在预期之内，且都是漏报方向，没有一例误报。末位扰动放过的 3 例全

表 63　变异实验结果（样例 11 条正确断言，固定随机种子；实测）。

算子	变异数	判定分布	符合预期
舍入（更粗的展示精度）	22	正确 22	22/22
末位扰动	99	数值错误 96，正确 3	96/99
相对扰动 1%–30%	44	数值错误 44	44/44
换单位	21	单位错误 18，正确 3	18/21
取反	2	方向错误 2	2/2
换期（上年值）	11	期间错配 11	11/11

表 64　真实文稿埋错试验（初步，三家公司，规则通道 vs 规则 + 模型通道）。

配置	埋错	判出	误报	召回	精确率	类型准确
规则通道	36	23	1	63.9%	95.8%	87.0%
规则 + 模型通道	36	31	2	86.1%	93.9%	83.9%

部是非水平值断言的末位邻值，被二次舍入项 0.005 吸收（第 9.4 节的权衡）。穷举全部末位替换可以看得更清楚：7 条水平值断言的 63 个末位邻值无一被放过；4 条非水平值断言各有一个邻值被放过——营业收入同比 14.48（真值 14.4887）、扣非同比 5.68（真值 −5.6730）、研发费用率 5.34（真值 5.3332）、资产负债率 45.84（真值 45.8490），即真值离两位小数的舍入边界不远时，靠近它的那个邻值落入 ±0.01 的容差。换单位放过的 3 例全部是每股收益：每股指标不做单位换算。舍入算子 22 例全部判正确，是命题 3.2 的直接验证；换期算子 11 例全部判期间错配，包括同比类型（上年同比需要前年数据，样例年报提供了三年数据）。

9.9.3　真实研报上的初步试验

样例数据内部完全自洽，真实研报的效果会明显下降。评测集（约 15 家公司的年报与卖方研报、人工埋错 150 ~ 200 处，清单见 `docs/eval-dataset-collection.md`）仍在收集中。表 64 是在三家已核验年报的公司上做的一次小规模试验：取我们自己生成的研报文稿的年报点评部分，由一名成员按清单要求人工埋错，另放约 20% 的“正确但容易误报”的断言。

漏报主要来自三处：规则通道识别不了的长句与“由 X 升至 Y”句式（模型通道补回大部分）；单季数字被写成年度数字，而对应公司的真值表缺少单季拆分，只能判数值错误或判不出；以及埋在表格中的数字（规则通道只读文本块）。两处误报中一处是“由 X 升至 Y”的上期值被判期间错配，另一处是研报用展示值计算的增速超出容差。 这些来源与表 55 和表 57 的分析一致。

9.10　失败模式汇总

表 65 汇总本章讨论过的失败模式，按“会造成误报”与“会造成漏报”分开。前者是我们最在意的，每一条都对应一个已知句式，可以通过写作规范（对我们自己的文稿）或人工复核（对外部研报）规避。

9.11　与数字护栏的关系

研报纠错引擎有两个用户：外部研报（纠错核查子选题），以及我们自己的文稿。后者通过 `guard.check_binding` 复用 `extract_claims` 与 `verify_claims`，把主笔会话写下的文稿

表 65　研报纠错的已知失败模式。

方向	情形	机制与缓解
误报	“由 X 升至 Y”	规则通道取第一个数为本期值；诊断为期间错配。缓解：写作规范改写为“Y，上年为 X”；模型通道补抽 Y
误报	“约 60 亿元”等约数	“约”不放宽容差。缓解：发现附相对偏差，读者自行判断
误报	用展示值算的增速	二次舍入超出 0.005（表 57）。缓解：末位差 1～2 的数值错误在发现中可见
误报	模型标错指标	quote 与数值能回查，指标无法验证。缓解：规则优先、附 quote、口径建议双向
漏报	每股指标单位写错	每股与比率不做单位换算
漏报	比率、增速末位差 1	被二次舍入项吸收
漏报	变动额、占比、环比、中文数词	规则通道不抽取；模型通道可补
类型粗化	替代期间不在真值表中	退回数值错误（错误仍被判出）

当作一篇研报来核：“文稿说营业收入 281.77 亿元”被抽成断言，与真值表比对。两种用法共享同一套判定函数，所以本章的每一个性质（四舍五入安全、诊断保守性、建议可执行性）在护栏中同样成立；本章的每一个失败模式也会出现在护栏中，而且由于护栏是硬闸门，误报的代价更高。护栏因此在复用之前又加了一层候选过滤（`_binding_candidate`），只把口径明确的历史断言交给判定函数，详见第 10.4 节。

9.12　小结

研报纠错把“这句话错了吗”拆成了三个可以分别验证的问题：这句话说了什么（断言抽取，quote 与数值必须能在原文中逐字找到）；真值是多少、容许多大的展示误差（真值表 + 按展示精度的容差，四舍五入在定义上不会被判错）；错在哪里（只在存在精确替代解释时给出具体类型，否则退回“数值错误”）。每一层都偏向放行而不是报错，这在决策论上对应误报代价远大于漏报代价的场景，在工程上对应纠错引擎被硬闸门复用的闭环。样例上 7 处埋错全部判出、类型全部正确、零误报；变异实验 199 例中的 6 例不符全部是漏报方向（末位差 1 的比率与增速、每股指标换单位），没有误报。已知的误报来源是少数几种句式（“由 X 升至 Y”、约数、用展示值算的增速），它们在我们自己的文稿中由写作规范规避，在外部研报中由附带的 quote、真值与相对偏差交给人判断。

10　数字护栏、数字锁与来源校验

前几章的机制管的是*程序产出*的数字：抽取有证据谓词，分析与估值由确定性代码计算，纠错按展示精度比对。但交付物的大部分篇幅是*模型写的文字*——一份深度研报正文三万字左右、八九百个数字，由主笔会话写成、复核编辑会话改写。本章讲系统如何把“模型写的文字”纳入与“模型抄的数字”同等强度的约束。共五个机制：

1. **数字护栏**（guard，`finagent/output/guard.py`）：文稿中每个数字必须按展示精度在本次运行的结论性产物 $\mathcal{A}$ 或外部来源登记 $\mathcal{S}$ 中找到出处；能识别指标与期间的表述还要与真值表绑定核对；每个有出处的数字按出处归为事实、推论或观点。
2. **来源登记**（`finagent/output/sources.py`）：年报之外的事实只能来自 `research/sources.json`，

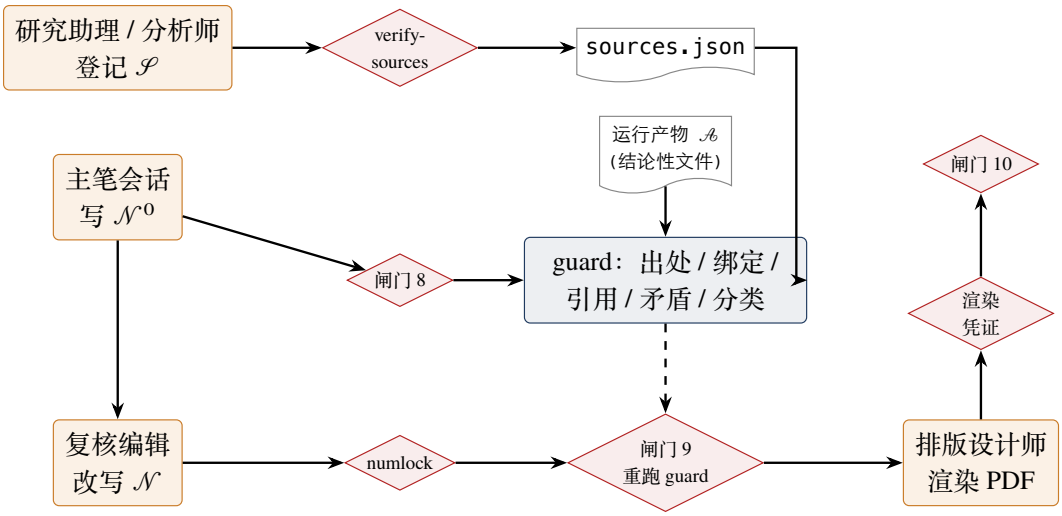


图 **14**　文字侧的检查点。橙色为模型会话，蓝色为确定性程序，红色为闸门或拒收检查。外部来源先过结构与“摘句含数”检查才能成为出处；初稿过 guard（闸门 8）；改写稿先过数字锁、再重跑 guard（闸门 9）；PDF 须带与当前产物和排版代码匹配的渲染凭证（闸门 10）。

每条摘句必须原样包含所登记的数字；只能在外部来源中找到出处的数字，所在句子必须带对应的 `[S 编号]`。

3. **前后矛盾**：护栏调用第 9.8 节的内部一致性检查。
4. **数字锁**（numlock，`finagent/output/numlock.py`）：复核改写稿中的数字与来源编号必须是初稿的子集。
5. **渲染凭证**（`finagent/output/render_check.py`）：PDF 的版面检查结果与 HTML、PDF、排版代码的哈希绑定，任何一方改动都使凭证失效。

这些机制的系统性质（数字闭合、引用完备、绑定正确、出处类型、数字锁保持闭合）已在第 3.8 节定义，相应的原则与不变量见第 4.9、4.10 节。本章讲实现、算法、在五份真实交付文稿上的实测，以及每个机制的假阴性与假阳性来源。

10.1　威胁与检查的对应

第 3.10 节把模型会话视为“不可信但有用”的组件，列出九类故障。文字侧涉及其中五类：T1 写出不存在的数字、T2 写出属于其他口径或期间的数字、T3 写出心算的派生数、T4 写出无出处的外部数字或错误的来源编号、T5 在改写中改动数字。表 66 给出每一类由哪个检查负责，以及检查覆盖不到的部分。最后一列是本章的主线之一：每个检查都有结构性的盲区，我们不试图用一个检查堵住全部，而是让盲区互不重合，并把剩余部分交给盲评与人工复核清单。

10.2　出处检查

10.2.1　文稿中的数字记号

护栏先把文稿切成“数字 + 单位”记号。正则 `NUM_UNIT_RE` 匹配一个可带负号、千分位与小数的数字，后接可选单位（万元/吨、元/吨、亿元、万元、亿、万、元/股、元、%、个百分点、倍、x）。不检查的记号由 `_text_numbers` 跳过：无单位的四位年份（19xx、20xx）与一位整数（序号），以及无单位且前一字为“第”、或后一字为“页 / 年 / 月 / 日 / 节”的数（页码、日期、节号）。带单位的数一律检查——“2025 年”被跳过，“2025 亿元”不会。来源标记 `[S 编号]` 在检查前删去，免得编号本身被当作数字。

表 66　文字侧故障与检查。“主检查”列出最先拦截的机制。

故障	主检查	覆盖不到的部分
T1 编造数字	guard 出处（check_text）	巧合落入某个产物值的容差窗口（§10.3）；中文数词
T2 口径 / 期间错配	guard 绑定（check_binding）	分部、季度、预测、跨年句不绑定；数字配到另一个也有出处的量上
T3 心算派生数	guard 出处	心算结果恰好等于某产物值；“约三倍”等中文数词
T4 外部数字无出处	来源登记 + check_citations	摘句本身是否真实出自网页（程序不联网回查）
T5 改写改数	numlock + 重跑 guard	数字在句间挪动、换绑指标（集合语义）
前后矛盾	内部一致性	分部、假设路径、图表与正文的一致（交复核编辑）

这个记号化有两个刻意的简化。其一，只认阿拉伯数字：“三倍”“四成”“过半”不是记号。写作规范禁止心算约数（设计规范 1），但程序拦不住这类写法，它们交给复核编辑与盲评。其二，记号只含“数字 + 紧随的单位”，不含指标名。出处检查因此只回答“这个数在产物里有没有”，不回答“这个数配的指标对不对”；后者是第 10.4 节绑定检查的职责。

10.2.2　出处池

`tagged_truth_values(run)` 从运行目录读出全部结论性产物，递归收集数值，按出处类型分入五个池（表 67）。收集规则有三处需要说明。

- **字符串中的数。**JSON 中“58.62 亿元”这类字符串按 `NUM_UNIT_RE` 解析并按亿 / 万换算到元。这让 `findings.json` 的 `truth_display`（“6.12 亿元”）也能作出处。
- **百万元块。**估值产物以百万元计（单位约定见表 9）。`_mm_numbers` 对 `forecast_base`、`pv_base`、`bridge_base` 三个块和所有以 `_mm` 结尾的键，把其中的数另按 $\times 10^6$ 换算一份加入池中，使正文写“17,483.72 亿元”时能匹配产物里的 1,748,372 百万元。
- **经营数据的列单位。**`extras.json` 的经营数据表按列单位存数（8,162 万吨存成 8162），`_extras_scaled_numbers` 按列单位的“亿 / 万”前缀再换算一份。这条规则来自招商轮船的一次真实失败：正文“货运量 8,162 万吨”被判无出处，因为池中只有 8162，而记号“8,162 万”按“万”换算成 8.162×10^7（测试见 `tests/test_guard_caliber.py`）。

池中*没有*的东西同样重要：研报原文的断言数字（研报里的错误数字不能成为出处）、纠错发现的 `suggestion`、三表模型与敏感性矩阵等中间网格（`three_statement.json`、`three_statement_detail.json`、`scenarios.json`、`three_statement_plan.json`）。排除中间网格的理由是控制巧合放行，第 10.3 节给出实测。

10.2.3　匹配

记号 (r, u) 的匹配按式 (18)：存在池中的值 v 与换算因子 $\varphi \in \Phi(u)$，使 $\|v|\varphi - |\mathrm{parse}(r)\| \le \frac{1}{2}\,\mathrm{ulp}(r)(1+10^{-9}) + 10^{-12}$。实现中先把池去重并取绝对值、排序（`check_text` 中的 `absv`），对每个记号线性扫描。三个细节：

- **取绝对值。**正文常写“下降 5.67%”，数字本身不带符号；产物中存的是 −5.67。代价是符号错误不被出处检查发现（“增长 5.67%”也能通过），符号由绑定检查对可识别的同比表述兜底。
- **百分数的两个因子。**$\Phi(\%) = \{1, 100\}$：产物里的比率有的存百分数值（14.49），有的存小数（DCF 中的 0.085）。这使“8.5%”能匹配 `plan.json` 的 `wacc=0.085`，但也让百分

表 67　出处池的构成（guard.tagged_truth_values）。“类别”为定义 3.17 的认识论类别。

文件（键）	池	类别	说明
analysis.json 真值表，method≠derived	报告披露	事实	年报原文抄录、经证据谓词核对
analysis.json 真值表 derived；其余键	程序计算	推论	毛利率等派生值；同比、勾稽、信号、单季拆分
earnings.json	程序计算	推论	利润归因桥、业绩对比、EPS 质量
facts.json；extras.json 表格	报告披露	事实	抽取结果与经营数据（逐格回查）
extras.json 的 ratios	程序计算	推论	吨价、占比、增速等由程序按 op 计算
valuation.json、forecast_summary.json、segment_forecast.json、reverse_dcf.json	模型预测	观点	DCF、盈利预测、分部预测、反向 DCF
comps.json 的 rows	外部来源	事实	可比公司原始数据（人给的外部输入）
comps.json 其余；dcf_cross_check	程序计算；模型预测	推论；观点	倍数统计与隐含价值；与 DCF 的对照
non_operating_assets.json；plan.json 价值桥	报告披露	事实	逐项核对过原文的非经营性资产与股权价值桥各项
plan.json 的 wacc、terminal_value	估值假设	观点	WACC 各参数、永续增长率
check/findings.json 的 truth_display	报告披露	事实	纠错结论中的正确值；不取 suggestion（会引用研报的错误数字）
sources.json 各条 value	外部来源	事实	由 run_guard 追加（§10.5）

数的出处池变成两倍大，是百分数巧合放行率高的原因之一。

- **其余单位不换算。**“元”“元/股”“倍”的因子为 1。因此“0.77 元”只匹配产物中的 0.77，不会去匹配 77 万元。

例 10.1 (被拦下的心算)**.** *研报 skill 的“常见坑”记录了一次真实的拦截：主笔会话把第二季度营业收入环比 −23.3445% 心算写成“环比下降 23.35%”。记号 23.35% 的容差为 0.005，池中最接近的值 23.3445 与它相差 0.0055 > 0.005，护栏判无出处。正确的两位小数是 23.34%。这个差异肉眼几乎不可能察觉，却恰好说明了按展示精度匹配的意义：护栏不问“差不多对不对”，只问“是不是某个产物值的正确舍入”。*

命题 10.1 (出处检查对正确舍入是完备的)**.** *若文稿中的记号 (r,u) 是某个池中值 v 按单位 u（$u \in \{$ 亿元, 亿, 万元, 万 $\}$ 时换算，$u =$ % 时按百分数值或小数）四舍五入到 $d(r)$ 位小数的结果，则 (r,u) 有出处。*

证明. 四舍五入保证 $\|v|\varphi - |\mathrm{parse}(r)\| \le \frac{1}{2}10^{-d(r)}$，其中 φ 取对应的换算因子，它属于 $\Phi(u)$；不等式右端不超过匹配容差。 □

命题说明出处检查没有“把正确写法判为无出处”的假阳性，前提是单位写法在 Φ 的覆盖内。真实的假阳性都来自这个前提不成立：单位是“万吨”而池中存的是列单位下的数（已由 _extras_scaled_numbers 修补）、单位是“港元”或“美元/天”而产物中没有对应换算、数字是由两个产物值相减得到的差额而产物中没有这个差额。对最后一种，写作规范要求把差额写进 extras.json 的 ratios（op: diff）由程序算，算出来的数就进了池。

表 **68**　均匀随机数的巧合放行率（实测；“结论”为护栏实际使用的池，“+ 中间”为并入三表与情景网格后的池）。

公司	池	池大小	亿元 · 2 位	% · 2 位	% · 1 位	% · 整数
中际旭创	结论	1,075	0.24%	3.4%	25.8%	85.1%
	+ 中间	1,813	0.50%	4.4%	30.0%	88.7%
招商轮船	结论	1,199	0.43%	4.6%	31.2%	88.0%
	+ 中间	1,956	0.52%	6.1%	34.1%	91.0%
贵州茅台	结论	1,352	0.27%	4.3%	28.9%	93.0%
	+ 中间	2,077	0.46%	5.3%	32.2%	94.0%

10.3　巧合放行：为什么只在结论性产物里找出处

出处检查是存在性判断，因此有结构性的假阴性：一个编造或写错的数，只要碰巧落在池中某个值的容差窗口内，就会被放行。第 4.14 节式 (31) 给出了巧合放行概率的估计。这里用真实运行目录测量它，并检验“只取结论性产物”这一设计的效果。

模型。　把池中值按单位换算到展示单位后视为直线上的点，记号的容差窗口宽 $w = 10^{-d}$。若编造的数在长度为 L 的区间上均匀分布，区间内有 n 个池中值且彼此不重叠，则命中概率约为 nw/L；考虑重叠，更准确的近似是泊松覆盖率

$$P_{\text{coinc}} \;\approx\; 1 - \exp\!\Big(-\frac{n\,w}{L}\Big). \tag{55}$$

它告诉我们三件事：巧合放行率随池大小 n 线性增长（小 n 时）；随展示精度指数下降（$w = 10^{-d}$）；对窄区间（百分数集中在 0 ~ 100）远高于宽区间（亿元金额跨越数个量级）。

实测一：均匀随机数。　对三家公司的最终运行目录，在“亿元、两位小数、1 ~ 1,000 亿元”和“百分数、0 ~ 100、零到两位小数”上各抽 20,000 个均匀随机数，统计被出处检查放行的比例；再把四个中间网格文件并入池中重测（表 68）。

泊松模型与实测吻合得不错：中际旭创的池在 1 ~ 1,000 亿元之间有 287 个值，式 (55) 预测 $1 - e^{-287\times 0.01/999} = 0.29\%$，实测 0.24%；招商轮船 387 个值，预测 0.39%，实测 0.43%。百分数上模型高估（中际旭创预测 4.5%、实测 3.4%），因为百分数在 0 ~ 100 内分布不均匀、集中在低值区并且彼此重叠。三家公司在 0 ~ 100 的整数中分别有 86、89 个和更多被池覆盖。

实测二：末位扰动。　均匀随机数代表“凭空编造”。更接近真实的故障是“心算或抄写时末位出错”。我们对五份交付文稿中的全部 3,960 个数字记号，各把末位随机换成另一个数字 5 次，统计扰动后的数仍能找到出处的比例（表 69）。总体放行率 26.7%，但分布极不均匀。

表 69 量化了护栏的真实保护范围：对两位小数的金额与百分数（占全部记号的约一半），末位写错有 92% ~ 96% 的概率被拦下；对整数百分数（约 13%），保护几乎不存在——0 ~ 100 之间的整数大多能在池里找到“出处”。“无单位”一类主要是条数、艘数、家数、倍数，同样以小整数为主，放行率接近四成。

实测三：中间网格的边际影响。　并入中间网格后，池大小增加 50% ~ 70%（表 68），两位小数金额的巧合放行率约翻倍（中际旭创 0.24%→0.50%），末位扰动的总放行率在五家上分别从 28.9%、29.3%、28.7%、19.0%、28.1% 升至 30.7%、33.7%、31.6%、20.8%、

表 69　末位扰动的放行率（五份文稿合计 3,960 个记号，各扰动 5 次；实测）。“占比”为该类记号在全部记号中的比例。

单位	小数位	记号数	占比	扰动后放行率
% / 个百分点	0	510	12.9%	93.6%
	1	259	6.5%	48.0%
	2	972	24.5%	8.0%
亿元	0	66	1.7%	32.4%
	1	31	0.8%	29.7%
	2	848	21.4%	3.9%
元	0	100	2.5%	7.0%
	2	280	7.1%	5.4%
无单位 / 其他	0	520	13.1%	37.7%
	2	211	5.3%	5.0%

29.6%。边际影响不大，原因是中间网格的值多是多年 × 多情景的金额，与正文最常见的两位小数金额重合有限；但它的代价是确定的，收益（允许正文引用三表明细）却可以通过在结论性产物里补字段获得，所以我们维持排除。

权衡（Trade-off）：护栏对低精度数字的弱保护

方案一：要求正文数字至少两位小数。能把放行率压到个位数，但“毛利率提升 3 个百分点”“船队 51 艘”这类自然写法被迫写成“3.00 个百分点”“51.00 艘”，可读性代价太大。**方案二：要求每个数都绑定指标**，比较对象从整个池缩小为一个真值。这会把大量合法的叙述性数字（分部、经营数据、外部数据）判为违规，与误报优先原则冲突（§4.8）。**方案三：按句子上下文缩小池**（例如句中出现“毛利率”就只在毛利率相关的值里找）。需要可靠的“句子—产物字段”映射，本质上是方案二的弱化版，我们尚未实现。**选择：**接受低精度数字的弱保护，用三层补偿：可识别的表述走绑定检查（比较对象只有一个真值）；整数百分数多出现在定性描述中，错误的影响小；盲评条目与复核编辑负责“数字密度”和“结论与数据一致”。我们在报告中明确写出这一盲区，而不是宣称“每个数字都核对过”。

10.4　指标与期间绑定

10.4.1　复用纠错引擎

`check_binding` 把文稿当作一篇研报：用 `_parse_markdown` 解析成文档，调用研报纠错的 `extract_claims`（规则通道）抽出断言，用 `verify_claims` 与真值表核对（第 9 节），返回判定为错误的断言。复用的好处是两种用法共享同一套判定函数与全部性质；坏处是纠错引擎的每个误报模式都会进入护栏，而护栏是阶段 8 的硬闸门。为此，护栏在复用前后各加了一层过滤（算法 13）。

`_segment_values` 汇集 `extras.json` 每一行的行名（及别名 `alias`）与该行全部已核对数值、`ratios` 的值，以及 `earnings.json` 利润归因桥的各项。后置过滤 `_is_segment_claim` 取断言前 24 ~ 80 个字的上下文：若其中点名了某一行（“茅台酒”“油轮运输”“毛利率效应”），且数字按展示精度等于该行某个已核对值，则视为按分部数据核对通过。这让“茅台酒毛利率 93.53%”不会被当成公司整体毛利率去比。

被过滤掉的断言没有被放行到无人核对：它们的每个数字仍然要过出处检查。过滤只意味着“不拿年度合并真值去比”，因为比较对象不对。

表 70　三家公司早期文稿的护栏收敛（commentary_guard.json，当时版本的护栏；实测）。

公司	运行	无出处数字	绑定不符
贵州茅台	20260929-145719	9	28
	20260929-145857	0	3
	20260929-145916	0	0
宁德时代	20261001-000917	3	9
	20261001-001302	0	1
	20261001-001358	0	0
招商轮船	20261001-001342	10	16
	20261001-001636	0	0

10.4.2　一次真实的失败与修复

招商轮船的第一版文稿（运行 20261001-001342）在当时的护栏下报出无出处数字 10 个、绑定不符 16 条。逐条检查后，16 条绑定不符几乎全部是*护栏误报*，可以归为五类：

- **单季当全年**：“营业收入 88.67 亿元”在第四季度的段落里，句中没有季度词，被判“期间错配：该数值对应 2025Q4 期”——判定本身是对的，但作者并没有说它是全年值；
- **分部当整体**：“净利率 40.73%”“同比增长 59.06%”是油轮分部的数，被拿去与公司整体净利率 21.54%、营收增速比较；
- **归因项当科目**：“毛利率效应拖累 5.49 亿元”被当作毛利率相关断言；
- **多年累计或计划当年度**：“资本性支出 358.72 亿元”被与 2025 年购建长期资产支付的现金 77.97 亿元比较（重复报了 4 次）；
- **估值数当财务数**：“股权价值 606.00 亿元”被与某个权益类科目比较。

10 个无出处数字中，“8,162 万”“11,419 万”等是经营数据表的列单位问题（§10.2），“301”“5.49 亿元”等则属于真正需要补算或改写的数。修复分两侧进行。护栏一侧：加入季度词、跨年、列举、分部行名、关键词的前置过滤与利润归因桥的后置识别（测试见 tests/test_guard_caliber.py）。写作一侧：研报 skill 的“常见坑”要求分部数字点名分部、季度数字写全“2025 年第四季度”、利润归因写归因项名称。下一版文稿（运行 20261001-001636）两项均为 0。

这个案例是“误报代价”的具体样子：在修复护栏之前，写作会话面对 16 条“错误”，唯一能做的是改写句子以躲开抽取器：把分部数字挪到不含指标名的句子里，或者把“净利率 40.73%”改写成更含糊的“利润率四成左右”。前者让读者更难判断数字的口径，后者既失去精度又绕过了出处检查。两者都让报告变差。表 70 汇总了三家公司首轮文稿在护栏下的收敛过程。

权衡（Trade-off）：绑定的覆盖面

绑定检查是部分性质（定义 3.15）：它只覆盖“口径明确的历史断言”。每加一条前置过滤，覆盖面就缩小一点，误报也少一点。**未选方案**：让模型为文稿写一份 claims.json 再走模型通道，覆盖面更大。但这等于让写作方自己声明“我这句话说的是哪个指标”，声明错了程序也无法发现（第 9.3 节的权衡）；而且会把一个可复现的检查变成依赖模型输出的检查。**选择**：绑定只用规则通道，宁可窄而准。第 1.2 节股权价值桥构成写错的案例（加回项的构成被描述错误，但每个数字都有出处）就落在覆盖面之外，它由负向评分条目与写作规范“写汇总数的构成之前先在产物里拆开”处理。

表 71　sources.json 的字段与 verify_sources 的检查。

字段	检查
questions[].id, question	问题列表不得为空（“先在 framework.md 列外部调研清单，再逐题检索”）
queries	每个问题至少一条检索词（“不能凭记忆作答”）
status	必须为 found 或 not_found；found 必须关联至少一个来源，且所关联的来源必须存在
phase, category	资料搜集阶段的问题标 collect 与七个类别之一（由 coverage 统计，供闸门 1 使用）
sources[].id	形如 S+ 数字，互不重复
url	以 http:// 或 https:// 开头（“只接受实际打开过的网页”）
title, publisher	非空
accessed, published	访问日期必须为 YYYY-MM-DD；发布日期可选，格式为 YYYY-MM-DD 或 YYYY-MM
facts[].quote	非空，不超过 160 字（“只摘支撑数字的一句”）
facts[].value, unit	若给出数值，则其某种原文写法必须是摘句（去千分位后）的子串

10.5　外部来源登记

10.5.1　登记结构

深度研报离不开年报之外的信息：行业规模、运价、竞争对手、一致预期、年报之后的公告。模型很容易用记忆里的数据“补全”这些内容，而记忆中的数字既可能过时，也可能是编造的。规则是：年报之外的事实只能来自 research/sources.json。文件有两部分：questions 记录调研问题及其检索过程，sources 记录实际打开过的网页及从中摘出的事实（表 71）。

“数值在摘句中”是外部来源一侧的证据谓词，与抽取时“模型抄的数字必须在所写页码的原文中找到”是同一原则。_digits 生成数值可能的原文写法：保留六位后去尾零的小数（2.1）、去掉负号的写法、绝对值不小于 1,000 时带千分位的写法；检查时摘句先删去半角与全角逗号。因此 value=1234, unit="万" 能匹配摘句“新船交付 1,234 万载重吨”；而 value=3.5 不能匹配“运力增长 2.1%”（测试 test_verify_sources）。注意这里是子串而非数字边界匹配：value=2.1 也能匹配“12.1%”。我们没有改为数字边界，是因为摘句中的数字常紧贴中文（“增长 2.1%”），边界规则与抽取侧一致即可；这个宽松处的后果是一条错误登记可能通过结构检查，但其数值仍要与正文按展示精度匹配才能成为出处。

fact_values 把每个来源的事实数值按与护栏相同的规则换算到基本单位（“1234 万” $\to 1.234 \times 10^7$），作为外部来源池加入护栏（表 67 最后一行）。

10.5.2　引用完备性检查

只有登记还不够：读者需要知道正文中哪个数来自哪个来源。check_citations 按句检查（句末标点为。!? 或换行），实现了引用完备（定义 3.14），见算法 14。

命题 10.2 (引用检查的判定)**.** *对一个句子 ς 中产物里找不到出处的记号 x，算法 14 报告问题，当且仅当 x 能由某个已登记来源解释、却不能由 ς 所引的任何来源解释。*

表 72　五份交付文稿的外部来源登记（research/sources.json；实测）。

公司	来源	事实	问题	检索	结构检查
中际旭创	51	184	31	61	通过
宇树科技	44	165	27	52	通过
招商轮船	52	205	29	56	通过
贵州茅台	51	229	32	64	通过
宁德时代	48	185	27	54	通过

证明. $x \in Y$ 等价于所引来源都不能解释 x；第二个条件等价于存在某个来源能解释 x。两者合取即命题所述。 □

命题划清了与出处检查的分工：任何来源都解释不了的数，由出处检查报“无出处”；能由某来源解释、但句子没引用或引错了来源的数，由引用检查报告。产物中能找到出处的数不要求引用——股价之外的绝大多数财务数字属于此类。检查以句子为粒度：编号可以出现在句中任意位置，一句引用多个来源时只要其中之一能解释即可。测试 `test_external_numbers_need_citation` 覆盖了三种情形：缺编号（报“须标注”）、编号正确（通过）、编号不存在（报“不存在的来源”）。

10.5.3　资料覆盖与时效

`coverage` 不检查正确性，而检查资料搜集是否充分：七个通用类别（最新定期报告与经营数据、股价与市值、一致预期与目标价、可比公司与倍数、行业规模与景气、竞争格局与份额、近三个月重大事件）是否各有至少一个 `collect` 阶段的问题被回答（`found`，或 `not_found` 但换过两个以上检索词），以及发布日期在 90 天内的来源有多少。闸门 1 要求七类全覆盖、搜集阶段检索不少于 20 次、近 90 天来源不少于 3 个、`verify_sources` 通过（第 11.6 节）。类别是行业无关的，不预设任何行业该查什么，与“不预设行业框架”的原则一致。

表 72 是五份交付文稿的来源登记规模。每份登记 44 ~ 52 个来源、165 ~ 229 条带数事实、27 ~ 32 个调研问题、52 ~ 64 次检索，`verify_sources` 全部通过；正文数字中来自外部来源的有 166 ~ 328 个（表 73）。

权衡（Trade-off）：要不要联网回查摘句

来源登记的最大盲区是*摘句本身的真实性*：程序检查“数在摘句里”，却不检查“摘句在网页里”。模型完全可以编一个网址和一句含数的摘句。**方案：引擎在 `verify-sources` 时抓取网页、查找摘句。**我们没有这样做，原因有三：引擎被设计为不联网、可离线复现，联网检查的结果随时间变化（网页改版、下线、付费墙），同一份登记今天通过明天失败；大量来源是 PDF 公告、需要渲染的页面或带反爬机制，抓取失败率高，会产生大量假阳性；引擎要能在评审的离线环境里复算。**缓解：**研究助理的会话事件流完整记录了每次 WebSearch 与 WebFetch 的调用和返回（`.pipeline/agents/*.jsonl`），摘句可以事后与抓取记录比对；盲评的负向条目“无来源外部数据”会扣分并封顶。我们对两份文稿做过一次人工抽查：随机抽取 40 条摘句，打开网址核对，37 条能在页面中找到原文，2 条措辞有出入但数字一致，1 条网页已无法访问。把“事件流比对”做成程序检查是后续工作。

10.6　前后矛盾

`run_guard` 的第四项检查调用第 9.8 节的内部一致性检查，把评级、各情景每股价值、预测年度公司整体收入与利润的前后不一列为 `contradictions`，计入 `ok`。它在写作与复

表 73　五份交付文稿的出处分类（当前文稿 + 最终运行目录 + 来源登记重跑护栏；实测）。“记号”为参与检查的数字记号数（含重复），全部有出处。

			出处类型					类别占比	
公司	字数	记号	披露	外部	计算	假设	预测	事实	观点
中际旭创	22,882	640	215	166	45	3	211	59.5%	33.4%
宇树科技	25,400	734	351	226	49	4	104	78.6%	14.7%
招商轮船	30,863	894	436	240	54	1	163	75.6%	18.3%
贵州茅台	33,670	838	304	328	70	0	136	75.4%	16.2%
宁德时代	25,940	854	368	280	80	2	124	75.9%	14.8%

核阶段就能发现矛盾，而不必等到盲评。质检中同一检查作为负向条目 N-02 单独扣分；为避免重复扣分，L0 硬闸门 `check_guard` 只看出处、绑定与引用三项，不看矛盾（测试 `test_guard_reports_but_gate_does_not_double_count`）。

失败案例：输入表示与排除规则不一致

`run_guard` 先计算 `plain = MARK.sub("", text)`（删去全部来源编号），出处、绑定与矛盾三项都使用 `plain`。对出处与绑定这是对的——编号不是数字；对矛盾检查却是错的：一致性检查靠 `[S编号]` 识别“别人的数字”（`_own_part`）。在五份交付文稿上重跑，中际旭创与贵州茅台各报 1 处假矛盾：前者把“高盛给予 A 股目标价 2645 元……[S10]”当成本报告的乐观情景每股价值，与“我们的乐观情景每股价值 918.06 元”比较；后者把“25 家给出‘买入’评级 [S10]”当成本报告的评级，与本报告的“中性”比较。直接以原文调用一致性检查时两份文稿均为 0 处。这两份文稿通过阶段 8 时护栏还没有矛盾检查（引入矛盾检查的提交晚于它们的 guard 记录），所以缺陷没有在当时暴露；N-02 的程序判定传入原文，也不受影响。修复是让 `_contradictions` 接收原文；这个案例的教训是：排除规则依赖的输入表示也是规则的一部分，组合两个各自正确的检查时，要检查它们对输入的假设是否一致。

10.7　出处分类：事实、推论与观点

大赛要求“区分事实、推论与观点”。`classify_text` 对文稿中每个有出处的记号，按“报告披露 → 外部来源 → 程序计算 → 估值假设 → 模型预测”的顺序取第一个能匹配的池作为出处类型，再映射到三类（定义 3.17）。结果写在护栏输出的 `provenance` 字段，不影响 `ok`；排版时 reportkit 的 `provenance_note` 把计数写进研报附录（“可溯源的 n 个数字中，事实 a 个、推论 b 个、观点 c 个（模型预测 d 个）”）。

表 73 显示，深度研报的数字约四分之三是事实（其中外部来源占三到四成），推论只占 6%～9%，观点占 15%～33%。中际旭创的观点占比最高，与它作为高成长公司、正文大量讨论盈利预测与估值情景一致。“估值假设”只有 0～4 个，因为 WACC 参数通常只在估值节出现一次，而终值增长率等常与别的池中值巧合相等、按顺序先被归为事实或推论。

顺序带来的偏差。 按“先认事实”的顺序，一个同时是披露值和预测值的数被计为事实。测试 `test_classify_priority_and_classes` 明确断言了这一行为：14.49 同时在程序计算与模型预测池中，归为推论。这使“观点”计数成为下界，“事实”计数成为上界。对低精度数字（整数百分数），巧合相等的概率很高（表 69），上下界之间的距离也就更大。我们接受这个偏差，因为它的方向是保守的：不会把一个事实降级为观点；附录中的说明写的是“可溯源的数字中”，不宣称逐数确认。

权衡（Trade-off）：数字级分类 vs 句子级标注

句子级标注（模型在句后注明【事实】【观点】）覆盖定性判断，但标签不可验证；数字级分类可验证，但不覆盖不含数字的判断。我们选择后者作为程序指标、前者交给写作规范（事实直接陈述，推论用“主要系”“反映”，观点用“我们认为”“我们预计”）与 L3 盲评条目。写作规范还明确禁止在正文中写“事实层面”（观点）”之类的行内标签——研报的读者不需要看到分类的脚手架。

10.8 数字锁

10.8.1 动机

主笔初稿过了护栏之后，由另一个会话以“没有参与写作的质控编辑”身份改写（`finagent-review` skill）：删自证与机器腔、理顺故事线、把拿不准的事项移进复核清单。改写是提升可读性最有效的环节，也是数字最容易被悄悄改动的环节——润色时顺手取整（58.62 写成 58.6）、合并两句时把数字配到另一个指标上、补一个“让论证更完整”的数。数字锁的任务是保证改写“只动文字、不动数字”。

10.8.2 定义与实现

数字锁的定义见定义 3.18。实现（`numlock._tokens`）的步骤：删去 HTML 注释（`<!-- ... -->`，跨行）；删去 `[S 编号]`；用护栏同一个 `NUM_UNIT_RE` 提取“数字 + 单位”记号，去掉千分位；跳过无单位的年份与一位整数。返回 `added`（改写稿中出现、初稿中没有的记号，去重排序）、`new_sources`（新出现的来源编号）、`removed`（记号总数的减少量，不小于 0）与 `ok`。

注释被删去意味着初稿注释里的数字不算初稿内容：初稿“<!– 提示 12.3% –>”不能让改写稿合法地写出“增长 12.3%”（测试中有这一断言）。千分位被去掉意味着“1,234 万”与“1234 万”是同一个记号——改写可以统一千分位写法。单位是记号的一部分，所以“58.62 亿元”改成“58.62 万元”会被拒绝。

命题 10.3 (数字锁保证什么)**.** *若 $\mathit{numlock}(\mathcal{N}^0, \mathcal{N}).\mathit{ok}$，则：(i) $\mathcal{N}$ 正文（注释之外）的每个“数字 + 单位”记号都以相同的数值写法与单位出现在 $\mathcal{N}^0$ 正文中；(ii) $\mathcal{N}$ 引用的每个来源编号都被 $\mathcal{N}^0$ 引用过；(iii) 若 $\mathcal{N}^0$ 通过护栏的出处检查，则 $\mathcal{N}$ 中除“上下文跳过规则”之外的每个记号都有出处。*

证明. (i)(ii) 即 `added` 与 `new_sources` 为空的定义。(iii)：出处检查对记号的判定只依赖 (r, u) 本身（式 (18)），与上下文无关，除了 `_text_numbers` 的上下文跳过规则——无单位的数若前为“第”或后为“页 / 年 / 月 / 日 / 节”则不检查。设 $\mathcal{N}$ 中某记号 (r, u) 参与检查，由 (i) 它在 $\mathcal{N}^0$ 中出现过；若在 $\mathcal{N}^0$ 中它也参与了检查，则有出处；若它在 $\mathcal{N}^0$ 中恰因上下文被跳过（例如“第 12 页”中的 12），则在 $\mathcal{N}$ 中可能无出处。 □

命题 (iii) 是命题 3.3 在实现层面的精确版本：“数字锁保持数字闭合”在上下文跳过规则之外成立。这个缝隙很窄（只涉及无单位的整数），但它是流水线在数字锁之后重跑完整护栏的理由之一；更重要的理由是数字锁不保持绑定正确与引用完备（例如把带编号的句子拆成两句，使数字与编号分离）。闸门 9 因此同时要求数字锁通过与护栏对改写稿通过。

10.8.3 集合还是多重集

定义使用集合包含：改写稿可以把初稿中的一个数多引用几次。多重集包含（每个记号的出现次数也不能增加）更严格，能拦住“把一个数复制到另一个指标旁边”的一部分情形。我们在五份文稿的初稿与改写稿上同时计算了两种语义（表 74）。

表 74　五份文稿的复核改写与数字锁（`narrative.draft.md` → `narrative.md`；实测）。“多重集超出”为出现次数增加的记号种数。

公司	字数（初 → 改）	记号（初 → 改）	删去	集合锁	多重集超出	例
中际旭创	23,351→22,882	665→650	15	通过	0	—
宇树科技	25,942→25,400	762→752	10	通过	1	4.23 亿元 3→4
招商轮船	32,257→30,863	981→947	34	通过	4	28.00% 4→5
贵州茅台	34,684→33,670	898→875	23	通过	0	—
宁德时代	26,758→25,940	909→898	11	通过	4	764.03 亿元 2→3

五份改写全部通过集合锁，删去 10 ~ 34 个记号，篇幅缩短约 2% ~ 4%。若采用多重集语义，五份中有三份会被拒绝——复核编辑把关键数字（如宁德时代的 764.03 亿元、招商轮船的 28.00%）在小结或投资要点中多引用了一次。这正是我们在第 4.10 节权衡中预期的合理改写。实测支持了集合语义的选择：多重集语义在真实改写上的假阳性率为 3/5，而它多拦下的情形（同一个数被挪到错误的指标旁）可以由重跑护栏的绑定检查覆盖一部分。

10.8.4　数字锁拦不住什么

- **换绑。**“毛利率 31.19%，净利率 14.49%”改成“毛利率 14.49%，净利率 31.19%”：两个记号都在初稿中，集合与多重集都不变，数字锁通过。若这两个表述是口径明确的历史断言，重跑护栏的绑定检查会判出；若是分部或预测口径，则拦不住。
- **取整与换单位：**恰好相反，这类无害改动会被拒绝。“58.62 亿元”润色成“约 58.6 亿元”、“5,862 百万元”改成“58.62 亿元”都会因记号不在初稿中而失败。复核 skill 因此明确规定“不新增数字，不改动数字”，需要改数字才能讲通的地方写进复核清单。
- **删除关键限定。**删去“油轮分部”四个字，数字未变，数字锁通过，但口径信息丢失。这只能靠重跑护栏（数字可能被绑定到整体口径而报错）与盲评。
- **中文数词。**“三倍”不是记号，改写可以自由增删。

权衡（Trade-off）：数字锁的严格程度

更宽松：允许展示精度内的改写（58.62→58.6）。需要判断新写法是否是某个初稿数字的合法舍入，等于在数字锁里再实现一次出处检查；而且取整会让同一数字在正文与图表中出现两种写法，读者对账更难。我们宁可让编辑“只删不改”。**更严格：多重集语义，或要求记号的相对顺序不变。**前者在真实改写上 3/5 假阳性（表 74）；后者禁止调整段落顺序，复核编辑无法“理顺故事线”。**选择：**集合语义 + 来源编号集合语义 + 重跑完整护栏。数字锁是最便宜的第一道检查（毫秒级、无需运行目录），护栏是第二道。

10.9　渲染凭证

10.9.1　动机：CSS 声明不等于 PDF 字号

研报 PDF 由 Chrome 无头打印 HTML 生成。开发中遇到过一个隐蔽的问题：某张表格的列宽超过 A4 版心时，Chrome 会把整份 PDF 等比缩小以容纳最宽的元素，于是 CSS 里写的 10.5pt 正文在 PDF 里变成了更小的字。逐页预览很难看出来（整体缩小，没有局部错位），而盲评只看交付物，也不会测字号。修复包括两部分：公共样式让长表头、说明文字与网址换行；渲染时测量并留下可复核的凭证。

表 75　渲染凭证的核对条件（render_check.check_render_record），任一不满足即闸门不通过。

#	条件	失败信息
1	凭证存在、可解析，版本为 1 且 ok=true	渲染检查未通过 / 缺少有效凭证
2	记录的 HTML、PDF 哈希等于当前 report.html、report.pdf 的哈希	HTML / PDF 已修改，检查过期
3	记录的排版代码哈希等于当前 reportkit 的哈希	排版代码已修改，须重新渲染
4	预检无溢出、无字号错误、无坏图	页面溢出或正文字号错误
5	PDF 页数 ≥ 1，实测样本非空且每个样本字号与 10.5pt 之差 ≤ 0.12	PDF 正文实测字号不符

10.9.2　三步检查

reportkit/render.py 的 to_pdf 依次执行（算法 17 给出了它在排版阶段的位置）：

1. **预检**（preflight）：在一份临时 HTML 中注入探针脚本 render_probe.js，把打印媒体规则改写为屏幕规则、把版心宽度固定为 180 mm，逐元素测量包围盒与内联文字范围，记录越出版心的元素（overflow）、字号不为 10.5pt 的正文段落（font_errors）、加载失败的图片（broken_images），并从正文段落中取最多 8 段、每段 32 字的文本样本。任何一项不通过即抛出异常，不打印。
2. **打印与实测**（measure_pdf）：打印到候选 PDF，用 pdfplumber 检查每页为纵向 A4（595.28 × 841.89 pt），在 PDF 的字符流中定位每个文本样本，读取这些字形的字号；任何一个字形偏离 10.5pt 超过 0.12pt（Chrome 毫米到像素取整的误差约 0.03pt），或有样本找不到，即失败。
3. **凭证**：通过才用候选替换原 PDF，并写 render_check.json，记录版本、结果、HTML 与 PDF 的 SHA-256、排版代码哈希、预检测量与 PDF 实测；失败时保留旧 PDF，凭证中记录失败原因。

排版代码哈希 renderer_hash 依次哈希 render_check.py 自身与 reportkit 目录下全部 .py、.css、.js、.md 文件（文件名与内容，按名排序）。闸门 10 调用 check_render_record，按表 75 的顺序核对凭证。

命题 10.4 (凭证的新鲜性)**.** *若闸门 10 的凭证核对通过，则当前 report.html、report.pdf 与 reportkit 代码，与产生该凭证的那次成功渲染所用的文件逐字节相同（除非 SHA-256 碰撞）。*

命题保证的是*新鲜性*，不是*真实性*：它防止“改了 HTML 或排版代码却拿旧凭证当作通过”（威胁 T7 中的“使用过期产物”），这是我们在开发中实际遇到的情形——修了一张表、重新打印时绕过了公共渲染器。它不防伪造：一个有写权限的会话理论上可以自行计算哈希、写一份 ok=true 的凭证。我们没有对凭证签名，因为威胁模型假设模型会出错、会走捷径，但不假设它会刻意伪造带哈希的 JSON；会话事件流记录了每一次文件写入，伪造可以事后审计。

实测。 招商轮船交付版的凭证：PDF 30 页；预检版心宽 680.31 px（即 180 mm 在 96 dpi 下的宽度），无溢出、无字号错误、无坏图；8 个正文样本在 PDF 中实测均为 10.4995 pt。

局限。 样本只取前 8 段长正文，后面章节的字号由“整份 PDF 不缩放”间接保证（缩放是全局的）；检查不判断图中文字重叠、孤立标题与留白，这些仍需逐页预览与排版会话自查；普通测试不要求浏览器，真实打印回归在装有 Chrome 的环境中以环境变量开

表 **76** 护栏类机制的调用位置。闸门编号见第 11.5 节，评分条目见第 12 节。

机制	命令 / 函数	生成侧闸门与质检条目
来源登记	`verify-sources`；`coverage`	闸门 1（结构 + 七类覆盖 + 检索次数 + 时效）；闸门 4（补充检索）；L1“外部信息可追溯”
出处 + 绑定 + 引用	`guard`（`run_guard`）	闸门 8：`ok` 为真、核对的是当前稿（`commentary.md` 与文稿相同）、带了 `--sources`、晚于 `sources.json`；L0 硬闸门“数字护栏”
前后矛盾	`run_guard` 的 `contradictions`	闸门 8（计入 `ok`）；负向条目 N-02（程序判，L0 不重复计）
出处分类	`classify_text`	不设闸门；写进研报附录
数字锁	`numlock`	闸门 9（数字锁 + 改写稿重跑 guard）；L1“数字锁”
渲染凭证	`render.to_pdf`；`check_render_record`	闸门 10（与页数、图表数、另存 PDF 一致性并列）

启（`tests/test_report_render.py`）。

10.10 为什么不对文风做程序拦截

护栏拦数字，不拦文风。模型初稿中反复出现的写法——否定对照（“不是……而是……”）、自证圆场（“与年报的解释一致”“相互印证”）、程序口吻（“程序预测”）、口语套话（“不过”“具体来看”）——用正则检测并不难，但我们（按用户的明确决定）不做程序拦截，而交给复核编辑会话与盲评。理由有三。

1. **没有真值。**数字有真值表，“这句话机器味重不重”没有。“不是……而是……”在辨析口径时是正当的写法，“相互印证”在描述两个独立来源时也可能是必要的。正则对这些情形只能误报，而误报在硬闸门中的代价我们已经在第 10.4 节见过。
2. **目标被替换。**一旦某个句式被拦截，写作会话会换一个同义说法绕过去（Goodhart 定律 [23]），文风并没有变好；第 14 节记录了我们试过又删除的这类规则。数字护栏没有这个问题，因为“找到出处”无法靠换说法绕过。
3. **改写需要理解。**复核 skill 要求编辑先通读全文回答三个问题（核心结论、论证链、故事线），再逐段问“去掉这一段读者会少知道什么”。机器腔多数情况下应当整句删除或重组段落，而不是替换几个词——这需要理解，不是匹配。

文风因此由三处负责：主笔 skill 的写作规范（事前）、复核编辑的改写（事中，数字锁保证不碰数字）、盲评条目“无机器腔”“结论先行”“事实/推论/观点”（事后，第 12 节）。程序在这里的角色是保证编辑只能改文字。

10.11 在流水线与质检中的位置

表 76 汇总本章各机制被谁调用、在哪里成为闸门或评分项。同一个检查在生成侧是闸门（不过不能进入下一阶段），在质检侧是计分项（不过扣分或封顶）；质检侧的实现直接调用生成侧的同一函数，保证两边的标准一致。

闸门 8 的“核对的是当前稿”值得单独说明。`guard` 命令把被检查的文稿原样复制为运行目录下的 `commentary.md`，结果写 `commentary_guard.json`；闸门比较 `commentary.md` 与当前 `narrative.md` 是否逐字相同。这防止“先让一个短稿通过护栏，再把长稿换进来”。同理，`commentary_guard.json` 的修改时间必须不早于 `sources.json`：来源登记改动后（例如删掉一条来源），旧的护栏结果作废。

10.12 小结

本章的五个机制把“模型写的文字”纳入了可验证的约束：每个数字要么是某个结论性产物值的正确舍入，要么是某条登记过的外部摘句中的数且句中带编号；口径明确的历史表述还要与真值表逐条对上；同一个量前后只能有一个值；改写只能删、不能增改；PDF 的版面检查与当前产物绑定。在五份交付文稿上，3,960 个数字记号全部找到出处，其中约四分之三是事实；五次复核改写全部通过数字锁，删去 10 ~ 34 个记号。

我们同样量化了这些机制拦不住什么：两位小数的金额与百分数末位写错，92% ~ 96% 会被拦下，整数百分数几乎不受保护；数字换绑、分部与预测口径的错配、摘句本身的真实性、中文数词和文风，都在程序检查之外，由复核编辑、盲评与人工复核清单承接。撰写本章时实测发现的一个缺陷（护栏剥掉来源编号后调用一致性检查，产生两处假矛盾）也说明：检查之间的组合本身需要测试。

11 多智能体研报流水线

前面几节讲的是“单份年报 $\mathcal{D}$ 到真值表 $\mathcal{T}$、估值与纠错结论”的确定性计算引擎。引擎可以保证每个数字可追溯，但写不出一篇研报：研报需要读完几百页年报、联网检索年报之后的最新信息、推导这家公司的分析框架、写两三万字的论证、再排成三十页带六七十个图表的 PDF。这些工作只能交给模型。本节讲我们怎样把这件事拆给多个模型会话去做，同时不让模型破坏第 4 节立下的规矩（LLM 不做算术、每个数字可追溯、评分标准对生成方不可见）。

我们的做法可以概括为一句话：**确定性的 Python 编排器在仓库之外为一家公司搭一个隔离工作区，依次起五个互不共享上下文的无头 Claude Code 会话，每个会话只做自己负责的阶段，由 11 道程序闸门判定能否交接；交付物再由第六个会话在另一个只含交付物的目录里盲评，问题单按类型路由回对应角色返工。**全部逻辑在 `finagent/report_pipeline.py`（1139 行）里，子命令有 `build`、`run`、`status`、`refine`、`collect`、`scorecard`，另有两个辅助命令 `template`（工作区内复制模板）与 `sync-renderer`（升级已有工作区的排版设施）。

本节的主要贡献是：

1. 把无头编码智能体（headless coding agent）当作“执行器（harness）”而不是“编排器（orchestrator）”的系统设计：编排完全确定，会话只通过文件交接，执行器可替换（§11.2）；
2. 以文件系统与权限规则实现的信息边界：工作区在仓库目录树之外，评测模块与评分标准不进入，引擎与 skill 只读（§11.3）；
3. 带新鲜度语义（校验晚于修改、核对当前稿、凭证绑定哈希）和前序阻断的 11 道闸门，我们把它们形式化为工作区状态上的谓词（§11.5）；
4. “先搜集、后分析”的调研阶段设计与深度专题机制（§11.6）；
5. 盲评驱动的返工闭环 `refine`：问题单只含理由与证据，按类型路由给对应角色，回滚到最佳一轮（§11.7）；
6. 排版层的“实测”检查：在真实 PDF 中定位正文原文、测量字形字号，修复了 Chrome 遇超宽表格整体缩放的隐性缺陷（§11.10）。

本节还整理了开发过程中真实发生的失败案例（§11.11）、主要权衡（§11.12），以及一次完整运行的时间线与成本（§11.13）。文中引用的会话耗时、轮数、费用与 token 用量都取自本机工作区的 `.pipeline/agent_runs.json`（由 `_session` 自动写入），盲评用量取自仓库 `eval/results/*/judge_run.json`。

11.1 问题与设计目标

最早的做法是在仓库里打开一个 Claude Code 会话，让它按 skill 从抽取一路做到 PDF（2026-09-29 的贵州茅台样例研报即如此产生，提交 `f458598`）；随后宁德时代、招商轮船两家也沿用同一方式。这种做法暴露出四类问题：

上下文污染

仓库里有别家公司的 `framework.md`、文稿和交付 PDF。模型写新公司时会主动去读已有样例，例如把茅台框架里“产品结构驱动毛利率”“经销商预收款是领先指标”这类白酒特有的分析，原样套到电池公司上。仓库上级目录还有原插件的参考代码（`reference/openai-pei/`），也可能被读进来。

评分泄露

仓库里有评分标准 `eval/rubrics/` 和历次评测结果。生成方一旦读到，就会对着条目写，评测失去意义（第 12 节）。

上下文耗尽

单个会话要读年报、检索、抽取、写两三万字、排版和逐页看预览，宇树科技一次排版会话就用了 288 轮、约 5,203 万输入 token（含缓存读取）。全部塞进一个会话，中途必然触发上下文压缩，压缩后模型会重读已完成阶段的文件，或忘记早先的约束。

自我审查失效

让写作的会话自己复核，它会为自己写下的句子辩护；让写作会话顺带排版，它会为了尽快结束而压缩版面（§11.11 的失败案例 4：11 页、26 个图表）。

由此确定六个设计目标：

表 77　流水线的设计目标与对应机制

目标	含义	主要机制
隔离	模型只看到本公司年报、引擎与执行 skill	仓库外工作区；`_check_location`；`settings.json` 禁读围栏
防应试	生成方看不到评分标准与分值	工作区排除 `finagent/evaluation/`；评测结果放工作区外；问题单只含理由与证据
可恢复	任何一次会话中断后，从断点接着做	文件即状态；`status` 判断已完成阶段；`run` 跳过已通过的会话
可审计	每个会话做了什么、花了多少，都能事后查	`stream-json` 事件流原样落盘；`agent_runs.json` 汇总用量
不可伪造	闸门不能被过期产物或模型自述骗过	新鲜度规则；“核对当前稿”；渲染凭证绑定 SHA-256；前序阻断
可替换	编排逻辑不依赖某个执行器	编排器是确定性 Python；执行器只是一条命令行

图 15 给出整体结构。蓝色是确定性程序，橙色是模型会话，红色菱形是程序闸门。

11.2 执行器：无头 Claude Code

11.2.1 为什么用编码智能体作执行器

一篇研报涉及的操作远多于“调用模型生成文本”：读 PDF 的指定页、写 JSON、运行 `verify-facts` 并按输出回原文修正、联网检索并打开原文核对数字、写 Python 排版脚本、渲染后看 PNG 预览再改版面。这些都是“工具调用 + 观察 + 修正”的循环（ReAct 式交替推理与行动 [70]），而且多数工具是文件系统和 shell。与其自己实现一个工具循环，

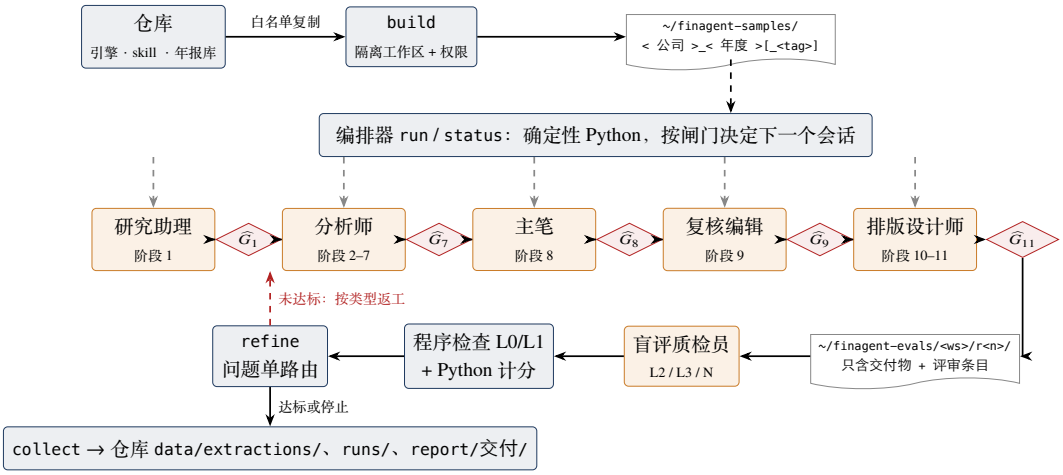


图 **15**　流水线总览。`build` 在仓库之外搭工作区；编排器依次起五个无头会话，每次交接前判前序闸门 $\widehat{G}_i$；`refine` 在另一个只含交付物的目录里起盲评会话，按问题单把返工路由回生成侧。

不如直接用一个成熟的编码智能体作执行器 [1, 69]：它已经具备读写文件、执行命令、联网检索、读图片、上下文压缩、权限控制这些能力，我们只需要规定“在哪个目录、能用哪些工具、做哪几个阶段、做完停下”。

仓库历史上曾有过另外两条路线：程序内调用国产模型 API 的工具循环（`ask`、`--llm`），以及 Codex 执行器与 DeepSeek 执行器（PR #16）。2026-10-05 的提交 `d9ae074` 把它们全部删除，只保留无头 Claude Code 一条路线。删除的理由是维护成本：每多一个执行器，就要多一套权限语义、事件格式和指令文件（Codex 读 `AGENTS.md`，Claude Code 读 `CLAUDE.md`），而且第 4 节的约束要在每一套里分别落实。

11.2.2　命令形态

每个会话由 `agent_command` 生成同一形态的命令（清单 1），在工作区根目录作为子进程启动：

Listing 1　一个会话的命令行（`agent_command`；`<PROMPT>` 为角色启动提示）

```
claude -p "<PROMPT>" \
  --permission-mode acceptEdits \
  --allowedTools "Bash(./py -m finagent *)" "Bash(./py -m finagent.report_pipeline *)" \
                 "Bash(./py data/extractions/*)" "Bash(./py -c *)" \
                 "Bash(cp data/extractions/*)" "Bash(ls *)" "Bash(mkdir -p data/*)" \
                 "Read" "Write(./data/**)" "Edit(./data/**)" "WebSearch" "WebFetch" \
  --output-format stream-json --verbose \
  --model claude-opus-5-5
```

各参数的取舍如下。

`-p`（非交互）。 会话没有人可以审批，遇到不在白名单的操作直接被拒（而不是挂起等待），模型会在结果里说明“哪条命令被拒、需要什么授权”。这是我们发现权限配置错误的主要途径（§11.11）。

`--permission-mode acceptEdits`。 自动接受文件编辑，但编辑仍受 `settings.json` 的 `deny` 约束。我们没有用跳过全部权限检查的模式：那样 `deny` 规则形同虚设，模型可以修改引擎或闸门代码。

`--allowedTools`（白名单再传一次）。 工作区里的 `.claude/settings.json` 已经写了同一份 `allow`，但 Claude Code 在未被用户信任的目录里会忽略项目级 `allow`（`deny` 照常生效）。工作区是新建目录，每次都未被信任；为了不去改用户的全局信任配置，编排器把同一份清单 `ALLOWED_TOOLS` 经命令行再传一遍。两处取同一个常量，保证一致。

通配规则而不是前缀规则。 早期写法 `Bash(./py data/extractions/:*)` 是旧式前缀语法，只在词边界匹配，匹配不到 `./py data/extractions/< 公司 >/report/build_report.py`，排版命令在无头执行时被要求审批（失败案例 3）。提交 `6d9a436` 改成通配写法 `Bash(./py data/extractions/*)`。

解释器包装 `./py`。 仓库路径含空格（`OpenAI fin/`），绝对路径的解释器命令无法稳定地匹配权限前缀；`build` 在工作区根写一个两行的 POSIX 包装脚本 `./py`（Windows 为 `py.cmd`），内部 `exec` 仓库虚拟环境的解释器，并把 `MPLCONFIGDIR` 指到工作区内的 `.pipeline/mpl`，使 matplotlib 在“只能写工作区”时也能缓存字体。每个启动提示都以同一句话结尾：“命令直接以 `./py` 开头，不加 `cd`、不串联。”加 `cd` 或用 `&&`、管道串联多条命令，命令串的前缀就不再是 `./py -m finagent`，会被拒。

`--output-format stream-json --verbose`。 输出为每行一个 JSON 事件（JSONL）：助手消息、工具调用、工具结果，最后一个 `type=result` 事件给出轮数、费用与 token 用量。下面讲它怎样落盘。

11.2.3 事件流落盘与用量汇总

`_session(ws, model, role, prompt)` 以 `subprocess.Popen` 启动命令，逐行读取标准输出：每行先原样写入 `.pipeline/agents/<YYYYmmdd-HHMMSS>-claude-<role>.jsonl` 并立即 `flush`，再由 `_event_brief` 把它压缩成一行摘要打印到终端（助手文本截前 200 字，`result` 截前 400 字，其余事件不打印）。逐行刷盘的意义在于：会话被 Ctrl-C 中断、额度耗尽或连接断开时，日志仍完整保留到中断前一刻。中断时编排器先 `terminate`，5 秒内未退出再 `kill`，返回码记 130，工作区进度保留。

会话结束后，`_usage` 从日志中找到 `result` 事件，取轮数 `num_turns`、费用 `total_cost_usd` 与用量。输入 token 按下式合计（缓存读取与缓存写入都计入，否则会严重低估上下文规模）：

$$\text{input_tokens} = u_{\text{input}} + u_{\text{cache_read}} + u_{\text{cache_creation}}, \qquad \text{output_tokens} = u_{\text{output}}. \tag{56}$$

一条运行记录追加到 `.pipeline/agent_runs.json`（清单 2，数值为贵州茅台 1005 批次分析师会话的真实记录）。多次会话的记录只追加不覆盖，所以中断重跑、返工都会留下完整历史，`scorecard` 按全部记录求和。

Listing 2 `agent_runs.json` 的一条记录（贵州茅台 1005 批次，分析师会话）

```
{"agent": "claude", "model": "claude-opus-5-5", "role": "analyst",
 "started_at": "2026-10-05T04:31:48", "elapsed_sec": 816.6, "exit_code": 0,
 "log": ".pipeline/agents/20261005-043148-claude-analyst.jsonl",
 "turns": 115, "cost_usd": 6.2030152,
 "input_tokens": 12534808, "output_tokens": 76799}
```

`cost_usd` 是 Claude Code 按 API 价格折算的等值费用。我们实际走的是订阅额度，所以它度量的是“若按 API 计价要花多少”，便于不同配置之间比较，不是实际账单。

11.2.4 编排器为什么是确定性程序

多智能体框架的常见做法是让一个“管理者”模型决定下一步调用谁 [25, 35, 45, 62]。我们反过来：编排器是一段只有几十行逻辑的 Python 状态机（`SESSIONS` 元组 + `run_agent` 循环），下一步做什么完全由闸门结果决定。

权衡（Trade-off）：模型编排器 vs 确定性编排器

模型编排器能处理计划外情况（例如发现年报是招股书，临时插入一步），也更容易写成“对话式”演示。代价有三：一是它需要读各阶段的产物才能决定下一步，自己就成了一个共享上下文的大会话，正是我们要避免的；二是“是否完成”变成模型的判断，会出现“自述已完成、文件其实没写”的情况（失败案例 2 中模型准确说明了被拦，但我们不能指望每次都如此）；三是不可复现：同样的工作区状态，两次编排可能走不同路径。

确定性编排器的“是否完成”只看 `status`，与模型怎么说无关；同一状态必然走同一路径；编排器本身可以单元测试（`test_run_agent_chains_write_then_review` 用桩函数替换 `_session`，验证五个会话的接力与停止）。计划外情况由两层兜底：工作区 `CLAUDE.md` 写“遇到规范没覆盖的情况，按可追溯、不误导、读者友好三条原则处理并在交付说明里写明”；会话没做完时编排器停下，把 `status` 的“下一步”交给人。

我们选后者。这与“能用简单可组合的工作流就不用自主智能体编排”的工程经验一致 [51]：自主性留在每个会话*内部*（怎么读年报、怎么检索、怎么写），会话*之间*的控制流保持确定。

11.2.5 执行器可替换

编排器对执行器的全部假设是一个函数签名：给定工作目录、启动提示和模型名，执行到结束，返回退出码，并在日志里留下事件流。换执行器只需改 `agent_command`、`_event_brief` 与 `_usage` 三个函数；闸门、工作区说明、skill、`refine` 的路由都不需要动，因为它们只认文件。仓库历史中 DeepSeek 与 Codex 执行器就是这样接入又移除的。工作区说明写成 `CLAUDE.md` 而不是执行器专有格式里的私有字段，也是为了这一点：换执行器时最多改一个文件名。

权衡（Trade-off）：执行器的上下文自动加载

Claude Code 会沿父目录逐级加载 `CLAUDE.md`。这一特性在工作区内很有用（工作区说明自动成为系统上下文，启动提示可以很短），在工作区外则是泄露通道：若工作区建在仓库目录树内，仓库根的开发指南（含选题范围、评测进展、评分分数）会被一并加载。`_check_location` 因此有两条规则（§11.3）。换用不自动加载上级说明的执行器时，第二条规则可以放宽，但第一条仍然需要：仓库目录树内的其他文件仍可被读工具访问。

11.3 `build`：仓库之外的隔离工作区

11.3.1 放什么、不放什么

`build(company, year, ...)` 为一家公司、一个年度搭一个工作区，默认路径 `~/finagent-samples/<公司>_<年度>[_<tag>]`（`--dest` 或环境变量 `FINAGENT_SAMPLES_DIR` 可改）。`--tag` 用于让不同模型或配置对同一公司各跑一遍，工作区目录名带标签，但内部抽取目录名仍是 `<公司>_<年度>`，保证收回仓库后路径一致。表 78 列出工作区内容。

年报来源核验。 `resolve_source` 先查 `report/resources/manifest.json`（已核验清单，记录简称、代码、交易所、路径与 SHA-256），找到就校验哈希，不符直接报错；清单里没有时，取 `report/resources/<公司>/` 下唯一的 PDF，此时视为“未核验来源”，`build` 要求显式加 `--allow-unverified`，工作区说明里也会写明“官方链接与版本未核对，交付说明里要写明”。文件名含“招股”时，说明里追加口径提示（财务数据在哪一节、单位多为万

表 78 隔离工作区的内容边界（build；由 test_workspace_contains_only_skills_engine_and_one_company 验证）

放入（路径与仓库一致）	理由
.claude/skills/finagent-{research,report,review}/	三个执行 skill；编排入口 finagent-pipeline 不放（生成方不需要知道流水线怎么调度）
finagent/（排除 evaluation/、__pycache__）	计算引擎；ENGINE_EXCLUDE 把质检模块（评分标准加载、评审提示、计分）排除在外
report/resources/< 公司 >/< 年报 >.pdf 与 locate.json	唯一一份年报；放在与仓库相同的相对路径，运行记录中的 argv 收回仓库后仍可复现
data/samples/valuation_assumptions.json、peers.csv	格式样例（虚构公司），只作字段参考
inputs/market.json、inputs/peers.csv（可选）	人给的外部输入：股价及来源、可比公司
data/extractions/< 公司 >_< 年度 >/report/（空）、runs/（空）	产出位置
sample.json、CLAUDE.md、TASK.md、REVIEW.md、py、py.cmd、.claude/settings.json	元数据、工作区说明、启动提示存档、解释器包装、权限
不放	
其他公司的抽取、框架、文稿、运行目录、交付 PDF	防止照抄框架、浪费上下文
eval/、finagent/evaluation/、tests/、README.md、.env	评分标准与评测结果对生成方不可见；测试与说明与任务无关
仓库根 CLAUDE.md、reference/	开发指南含评测进展；原插件代码不得引入

元、上市状态以资料搜集为准）。宇树科技就是这种情况。

外部输入必须带来源。 --price 是外部数据，不给 --price-source 直接拒绝（test_build_guards）。不提供股价时，工作区说明要求研究助理在阶段 1 检索最近一个交易日收盘价并登记进 sources.json，查不到就停下，不能凭记忆填。

关键页预定位。 已核验年报的 locate.json 直接复制；否则在工作区内跑一次 python -m finagent locate（第 6 节）。模型拿到的第一份材料是"该读哪些页"，不必从头翻几百页。

元数据。 sample.json 记录公司、年度、PDF 相对路径与 SHA-256、是否核验、代码、交易所、标签、外部输入、解释器路径、仓库路径与 git rev-parse HEAD、创建时间与 as_of 日期。as_of 是"近 90 天来源"闸门的参照日（§11.5），固定在建工作区那天，避免工作区隔几天再跑时，原本"近期"的来源因为系统时钟前进而失效。

11.3.2 位置围栏

定义 11.1 (位置围栏)**.** *设仓库根为 R，围栏 $F = \text{parent}(R)$。工作区路径 w 合法，当且仅当 (i) $w \neq F$ 且 $F \notin \text{ancestors}(w)$；(ii) 对任意 $p \in \text{ancestors}(w)$，$p \neq \text{home}$ 时 $p/$CLAUDE.md 不存在。*

规则 (i) 取 R 的父目录而不是 R 本身，因为原插件参考代码在仓库的上一级。规则 (ii) 针对执行器沿父目录加载 CLAUDE.md 的行为；用户主目录豁免，因为用户级配置本来

就会加载，与仓库内容无关。同一个函数也用于盲评目录（judge.prepare），保证评审会话同样读不到仓库。test_build_guards 验证把工作区建在 < 仓库 >/runs 下会被拒绝。

11.3.3 权限围栏

settings(meta, root) 生成工作区 .claude/settings.json（清单 3）。allow 与命令行白名单同源；deny 有两类：禁读围栏 F 下的一切，禁改引擎、skill、年报原件与流水线自身的控制文件。

Listing 3 工作区权限（settings() 的输出结构，<F> 为仓库父目录的绝对路径）

```
{"permissions": {
  "allow": ["Bash(./py -m finagent *)", "...", "WebSearch", "WebFetch"],
  "deny":  ["Read(/<F>/**)",
            "Edit(./finagent/**)",          "Write(./finagent/**)",
            "Edit(./.claude/**)",           "Write(./.claude/**)",
            "Edit(./report/resources/**)",  "Write(./report/resources/**)",
            "Edit(./sample.json)",          "Write(./sample.json)",
            "Edit(./py)",  "Write(./py)",  "Edit(./py.cmd)",  "Write(./py.cmd)",
            "Edit(./CLAUDE.md)",            "Write(./CLAUDE.md)"]}}
```

Read(/<F>/**) 中 <F> 本身以 / 开头，拼出的规则以 // 开头，这是 Claude Code 表示绝对路径的写法。禁改 finagent/ 与 .claude/ 的意义在于：闸门、校验器、guard、numlock 都是工作区里的代码，模型若能改它们，就能让任何闸门“通过”。禁改 CLAUDE.md、sample.json 同理（前者定义阶段与闸门，后者含 as_of 与 PDF 哈希）。引擎或 reportkit 不够用时，工作区说明要求模型“在公司编排脚本里绕开，并在交付说明里写明需要仓库改什么”，贵州茅台 1005 批次的排版会话就照此报告了“reportkit 不能渲染 Markdown 表格与编号列表”“WACC 表没有列出特有风险溢价”等需求。

锁定年报原件时规则写的是 report/resources/**，不是 report/**：后者会连带拦住 data/extractions/< 公司 >/report/（框架与文稿所在目录），这正是失败案例 2 的根因。

模板复制。 Claude Code 拒绝从配置目录 .claude/ 复制文件（即便只是读取后写到别处），所以 cp .claude/skills/.../templates/narrative.md ... 在无头会话里会被拒。我们加了子命令 report_pipeline template < 名称 > < 目标 >：由编排器代码完成复制，并检查模板名只能是 templates/ 下的直接文件、目标必须在 data/ 下、已存在不覆盖（test_copy_template_only_into_data 覆盖了 ../SKILL.md 这类路径穿越）。

不变量 23 (生成侧信息边界)**.** *在任何生成侧会话（研究助理、分析师、主笔、复核编辑、排版设计师、修订）执行期间，该会话可读的文件集合与 {eval/**, finagent/evaluation/**, 评测输出} 不相交。*

论证。 可读集合由三部分构成：工作区文件、联网检索结果、执行器自动加载的说明。(1) 工作区由 build 从白名单路径复制而来，ENGINE_EXCLUDE 排除 evaluation；(2) 评测输出写在 judge.default_root()（默认 ~/finagent-evals/），既不在工作区内，也不在围栏 F 内，但生成侧会话的工作目录是工作区，读工具对工作区外路径的访问没有被 allow；(3) 由定义 11.1，工作区的上级目录不含 CLAUDE.md。回传给生成侧的唯一评测信息是问题单，它由 issues_from 生成，只含描述、证据、阶段与严重度（§11.7）。□

这一论证有两个前提需要说明。第一，我们没有对工作区外的任意路径设置 Read 的 deny，而是依赖“不在 allow 中的读操作在无头模式下被拒”；若执行器将来默认放开工作区外的只读访问，就要补一条显式规则。第二，联网检索理论上可以搜到本仓库（私有仓库，搜索引擎不可见），我们认为这一风险可以忽略。

11.3.4 工作区说明 CLAUDE.md

render_guide(meta) 生成约 120 行的工作区说明，是五个会话共同的“操作手册”。它由 f-string 生成，闸门阈值直接插入常量（MIN_COLLECT_QUERIES 等），所以工作区说明与 status 的判定*按构造一致*。手写的 skill 文档则没有这个保证（失败案例 9 记录了一处漂移）。说明的结构如下：

1. **开头**：本工作区只为一家公司出一份卖方体例深度研报和估值模型；每个阶段结束运行 ./py -m finagent.report_pipeline status；“上下文被压缩或中途接手时，先跑 status，不要重读已完成阶段的文件”。最后一句是专门为长会话的上下文压缩写的。
2. **输入表**：年报路径与来源核验状态、locate.json、股价（或“未提供，阶段 1 检索”）、可比公司。
3. **先搜集、后分析**：写明今天的日期（as_of），要求年报之外的信息一律联网现查，来源优先级，正文外部数字句末带 [S 编号]。
4. **文风由独立复核会话把关**：主笔做到阶段 8 停下；拿不准但影响结论的写进 review.md，不写进正文。
5. **读取边界**：不读引擎源码（用命令与 schema）；skill 文档按阶段读；PDF 每次不超过 20 页，只读 suggested_reads 等必要页；已抄进 facts/extras 的页不再读；runs/ 下 JSON 不读全文，用 ./py -c 取字段；工作区之外的目录都不读。
6. **阶段表**：11 行，每行“做什么 / 这时才读什么 / 闸门”，闸门列与 status 的判定逐条对应（§11.5）。
7. **底线**：不做算术；每个数字来自年报原文、运行产物或 sources.json；估值参数标明为观点或假设、评级取程序结果；校验 3 轮仍不过的问题不硬改数字；五个会话各做各的阶段，做完就停。

“这时才读”一列实现的是渐进披露（progressive disclosure）：skill 全文约 1,700 行，若在会话开始时全部读入，既占上下文，也会让模型提前做后面阶段的事。阶段表把每份参考文档绑定到需要它的那个阶段。

升级已有工作区。 reportkit 修复缺陷后，已建的工作区仍是旧版代码。sync-renderer --workspace <ws> 只把公共排版设施（reportkit 的 .py/.css/.js/.md、render_check.py、report_pipeline.py、两份相关说明）复制进去，不碰公司数据、文稿、编排脚本和历史 PDF（test_sync_renderer_preserves_company_work）。升级后旧 PDF 的渲染凭证会因排版代码哈希变化而失效，必须重新渲染（§11.10）。

11.4 五个生产角色与盲评质检员

11.4.1 角色划分

SESSIONS 元组定义了五个生产会话及其各自负责到的阶段（含）：("research", 1)、("analyst", 7)、("write", 8)、("review", 9)、("layout", 11)。表 79 列出每个角色的阶段、模型、skill、读写文件与停止条件；图 16 以泳道图画出它们与编排器、文件之间的交接。

划分的依据有三条。

上下文预算。 从表 91（§11.13）可以看到，单个会话的输入 token（含缓存读取）在 1,000 万上下的量级，排版会话最多。把“检索”“抽取到估值”“写作”“复核”“排版”拆开，每个会话的工作集都能控制在一次上下文窗口与少量压缩之内。拆分点选在*产物稳定*的地方：阶段 1 结束时资料包定稿，阶段 7 结束时估值数字定稿，阶段 8 结束时文稿定稿（之后只删不增），阶段 9 结束时文字定稿。下游会话只需读定稿文件，不需要知道上游怎么

表 79 五个生产角色与盲评质检员。模型由 `_role_model` 决定：生成侧取 `FINAGENT_GEN_MODEL`（默认 `claude-opus-5-5`），复核与质检取 `FINAGENT_QC_MODEL`（默认 `claude-sonnet-5-5`）；命令行显式指定的模型优先。

角色	阶段	模型 / skill	主要读入	主要写出（全部在 `data/extractions/<key>/` 下，除注明外）
研究助理	1	GEN / report（`research.md` 第一部分）	年报首页与管理层讨论目录；联网检索	`research/sources.json`（`phase: collect`）、`research/dossier.md`、可选 `research/peers.csv`
分析师	2–7	GEN / research + report	`dossier.md`、`locate.json` 指定页、管理层讨论与附注	`facts.json`、`report/framework.md`、`research/topics/*.md`、`extras.json`、`valuation_assumptions.json`、运行目录 `runs/<id>/`
主笔	8	GEN / report + research（`writing.md`）	`framework.md`、`dossier.md`、`topics/`、运行产物	`report/narrative.md`；`guard` 结果写入运行目录
复核编辑	9	QC / review	`narrative.md`、运行产物	`narrative.draft.md`（初稿备份）、改写后的 `narrative.md`、`edit_log.md`、`review.md`
排版设计师	10–11	GEN / report（`html-report.md`、参考样例脚本）	定稿 `narrative.md`、运行产物、`sources.json`	`report/build_report.py`、`runs/<id>/report/` 下 HTML、PDF、`render_check.json`、预览；`report/delivery.md`
盲评质检员	—	QC / 盲评说明	盲评目录：PDF、文稿、6 个估值产物、`sources.json`、`review.md`、`rubric.json`（无分值）	`judgments.json`

得出它。

能力与约束的差异。 研究助理需要联网、不需要读财报细节；分析师需要逐页读 PDF、跑校验循环；主笔需要长篇写作与过 `guard`；复核编辑需要“没参与写作”的立场；排版设计师需要写 Python、看 PNG。每个角色读的 skill 段落不同，拆开后每个会话的系统上下文只含它需要的规则。

复核用不同模型、不同会话。 复核编辑与盲评质检员默认用 Sonnet，生成侧用 Opus。一是成本（复核与评审主要是读，不需要最强的生成能力）；二是减轻自我偏好：评审模型对自己生成的文本有系统性偏好 [43, 73]，换一个模型、且看不到写作过程，至少去掉了“同一会话为自己辩护”这一层。复核编辑的 skill 开头就是“你没有参与这篇研报的写作”，启动提示也重复这一点。

11.4.2 启动提示

每个启动提示都很短（中文 100 ~ 200 字），只做四件事：声明角色、指定只做哪些阶段、指定先读什么、规定做完运行 `status` 并停止。详细规则在工作区 `CLAUDE.md` 与 skill 中，提示里不重复。各角色的要点如下（全文见附录 E）：

`RESEARCH_PROMPT`

只做阶段 1；先读 `research.md` 第一部分，用 WebSearch / WebFetch 搜齐年报之后的最新信息，登记 `sources.json`、写 `dossier.md`；“不做财务抽取、不写分析框架”。

`ANALYST_PROMPT`

阶段 1 已完成，先读 `dossier.md`，完成阶段 2 ~ 7（财务抽取、分析框架含深度专题、深度调研、经营数据、估值假设、估值运行），“不写文稿”；每个阶段结束运行 `status`，

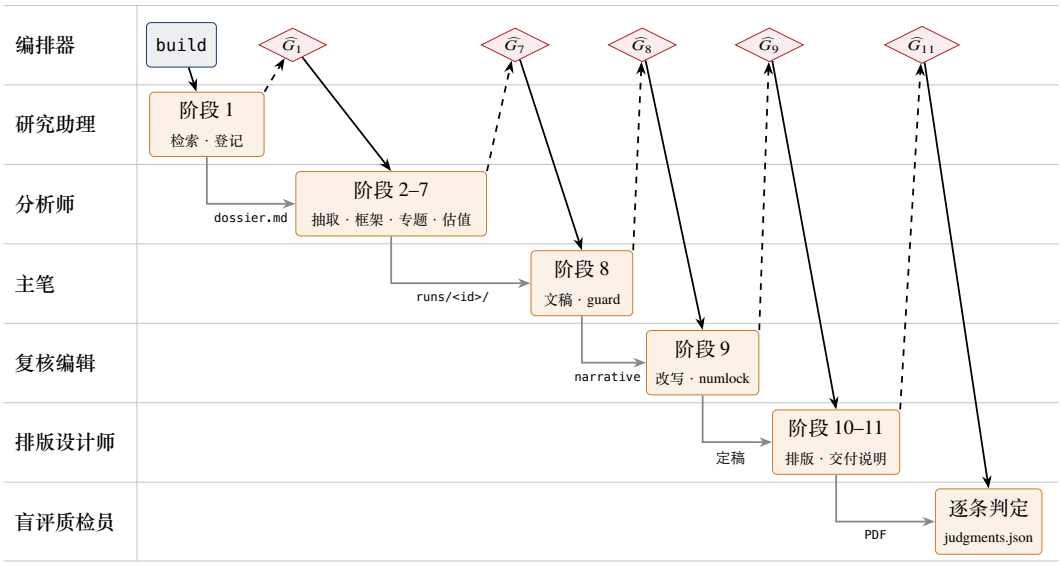


图 16 泳道图：五个生产角色、盲评质检员与编排器。每个会话只在自己的泳道里工作，交接只通过工作区文件；编排器在每次交接前判前序闸门 $\widehat{G}_i$。虚线：会话结束时运行 `status`；灰线：文件交接。盲评之后的返工路由（措辞与结构 → 复核编辑，数据与假设 → 修订会话，版面 → 排版设计师）见图 18。

阶段 7 通过后停止。

`TASK_PROMPT`

主笔；先读框架、资料包、专题笔记和运行产物，按阶段 8 写“年报点评加深度研究”，“深度研究要做多环定量推导、写长写深”；`guard` 与篇幅闸门通过即停止，“复核与排版由后面的会话完成”。

`REVIEW_PROMPT`

复核编辑，“没有参与写作”；先读 `finagent-review/SKILL.md`，只做阶段 9。

`LAYOUT_PROMPT`

排版设计师，“文稿已定稿，不改正文”；先读 `html-report.md` 与参考脚本，从模板起写定制排版脚本，“每张图服务一个论点，图表 60 张以上、全文 25 ~ 30 页，表格涨跌双色，逐页看 preview 修到美观”。

`build` 把主笔与复核的提示另存为 `TASK.md`、`REVIEW.md`，便于人在交互式 Claude Code 里打开工作区、粘贴提示手动接力。提示里“做完就停”的反复强调不是多余的：早期没有这句话时，主笔会话常常顺手做复核与排版，绕过了“独立复核”的设计（失败案例 4）。

11.4.3 `run`：接力与停止

算法 15 是 `run_agent` 的全部逻辑。两个性质值得说明。

幂等与可续跑。 会话是否需要执行只取决于工作区状态，与历史无关。中断后再 `run` 一次，会从第一个未通过的会话接着做；已通过的会话不重跑。真实运行中这一点被频繁用到：五家公司的历史里共有 7 次会话因订阅额度耗尽中途结束（结果事件为“You've hit your session limit”），2 次因连接中断结束，额度恢复后重新 `run` 即接续（§11.11）。接手的会话不必知道前一个会话做到哪里：工作区说明要求“中途接手时先跑 `status`”，招商轮船 1005 批次第二个分析师会话的汇报就是“阶段 2 ~ 4 在我开始时已完成，本次只做 5 ~ 7”。

以闸门而非退出码判断。 会话退出码为 0 不代表阶段完成，为 1 也不代表失败（`status` 在未全部完成时返回 1，多个会话在汇报中解释过“退出码为 1 只是因为后面的阶段还没做”）。编排器只看 $\bigwedge_{i \le u} G_i$。另一面，会话做完了但闸门不过（例如权限误拦导致文件没写到位），编排器就停下，不会把半成品交给下一个会话。

11.4.4 交接文件

表 80 是会话之间的全部交接面。每个文件只有一个写入角色（`sources.json` 例外：研究助理建立，分析师在阶段 4 追加 `phase: followup` 的问题）；读入方多数是下游会话或程序。

表 80 交接文件（工作区 `data/extractions/< 公司 >_< 年度 >/` 下）

文件	写入	读入 / 校验
`research/sources.json`、`dossier.md`	研究助理（分析师追加）	分析师、主笔；`verify-sources`、`guard --sources`
`facts.json` → `facts.verified.json`	分析师 / `verify-facts`	引擎（第 6 节）
`report/framework.md`、`research/topics/*.md`	分析师	主笔；闸门 3、4
`extras.json`、`valuation_assumptions.json`	分析师	引擎；闸门 5、6
`runs/<id>/`（工作区根下）	引擎	主笔、排版、盲评
`report/narrative.md`	主笔 → 复核编辑改写	排版、盲评；`guard`
`narrative.draft.md`、`edit_log.md`、`review.md`	复核编辑	人工；`numlock`
`report/build_report.py`、PDF、`delivery.md`	排版设计师	人工、盲评

权衡（Trade-off）：共享上下文 vs 文件交接

共享上下文（同一会话依次扮演多个角色，或父会话把摘要交给子会话）的好处是下游知道上游“为什么这样做”，不会误解意图；坏处是上游的推理过程、被否定的草稿、对自己结论的辩护一并传下去，复核就不再独立；上下文也随角色数线性增长。

文件交接要求每个交接物足够自解释：资料包有固定结构，框架有固定小节，深度专题笔记要写“结论与对预测的含义”，复核清单有固定五列。代价是信息损失，下游读不到上游没写进文件的东西，例如分析师为什么放弃某个专题。我们接受这个代价：研报最终的读者也只能看到文件，下游会话读不懂的交接物，读者大概率也读不懂。这也是盲评的出发点（§11.7）。

11.5 十一道闸门：`status`

`status(ws)` 是流水线的“唯一真相来源”。它不调用模型，只读工作区文件，按顺序逐项判定 11 个阶段，返回每个阶段的 `ok`、一行 `detail`（含计数）和未通过时的 `next`（下一步该做什么、读哪份文档），整体 `done` 与第一个未通过阶段的提示。会话结束前必须运行它，编排器接力前也调用它；模型看到的“下一步”与编排器的判断由同一函数给出。

11.5.1 阈值常量

表 81 列出 `report_pipeline.py` 顶部的全部阈值常量。工作区 `CLAUDE.md` 的闸门列由这些常量插值生成。

表 81 闸门阈值常量（以源码为准）

常量	闸门	取值	含义
`MIN_COLLECT_QUERIES`	1	20	资料搜集问题的检索词总数下限
`MIN_RECENT_SOURCES`	1	3	近期发布的来源个数下限
`RECENT_DAYS`	1	90	“近期”的天数（相对 `as_of`）
`MIN_TOPICS`	3, 4	3	深度专题数与专题笔记数下限
`MIN_TOPIC_CHARS`	4	1500	每份专题笔记去空白后的字数下限
`MIN_TOPIC_SOURCES`	4	3	每份笔记引用的不同 `[S 编号]` 个数下限
`MIN_FOLLOWUP`	4	8	补充检索问题数下限
`MIN_NARRATIVE_CHARS`	8	15000	正文字数下限（去空白与 HTML 注释）
`MIN_DEEP_SHARE`	8	0.6	深度研究（`deep_` 开头的节）字数占比下限
`MIN_PAGES`, `MAX_PAGES`	10	25, 32	PDF 页数区间
`MIN_FIGURES`	10	60	图表个数下限

11.5.2 形式化

记工作区状态为 $\mathscr{W}$，文件 x 的修改时间为 $\tau(x)$（不存在时取 0），$\mathsf{exists}(x)$ 表示文件存在。记 src=`sources.json`，fw=`framework.md`，nar=`narrative.md`，$cov = \mathrm{coverage}(src, \texttt{as_of}, 90)$，$\mathrm{VS} = \mathrm{verify_sources}(src)$。$r^*$ 为“最新一次用本工作区 `facts.json` 跑成功的 `valuation` 或 `all` 运行”（按目录名即时间戳倒序，取第一个 `manifest.json` 中 `status=ok`、`task`$\in$ {valuation, all} 且 argv 含本工作区 facts 路径的运行），$\mathrm{start}(r^*)$ 为其 `started_at`，$\mathrm{argv}(r^*)$ 为其命令行参数。

定义 11.2 (局部闸门)**.**

$$
\begin{aligned}
G_1 &: \mathsf{exists}(\mathit{src}) \wedge \mathsf{exists}(\mathit{dossier}) \wedge \mathit{cov}.\mathrm{missing} = \emptyset \\
&\quad \wedge\, Q_{\mathrm{collect}} \geq 20 \wedge |\mathit{cov}.\mathrm{recent}| \geq 3 \wedge \mathrm{VS}.\mathsf{ok} \\
G_2 &: \mathsf{exists}(\mathit{facts}) \wedge \mathsf{exists}(\mathit{verified}) \wedge \tau(\mathit{verified}) \geq \tau(\mathit{facts}) \wedge \mathit{verified}.\mathrm{check}.\mathsf{ok} \\
G_3 &: \text{“外部调研清单”} \in \mathit{fw} \wedge n_{\mathrm{topic}}(\mathit{fw}) \geq 3 \\
G_4 &: \mathrm{VS} \neq \bot \wedge |T| \geq 3 \wedge \forall t \in T : (\,\mathrm{chars}(t) \geq 1500 \wedge |\mathrm{cites}(t)| \geq 3) \\
&\quad \wedge\, Q_{\mathrm{followup}} \geq 8 \wedge \tau(\mathit{src}) \geq \tau(\mathit{fw}) \wedge \mathrm{VS}.\mathsf{ok} \\
G_5 &: \mathsf{exists}(\mathit{extras}) \wedge \mathrm{VE}(\mathit{extras}).\mathrm{rejected} = 0 \\
G_6 &: \mathsf{exists}(\mathit{assump}) \wedge \mathit{price} \neq \emptyset \wedge \mathit{price_source} \neq \emptyset \wedge \mathit{assump} \text{ 含 } \texttt{[Sn]} \\
G_7 &: \exists r^* \wedge \mathrm{start}(r^*) \geq \max\{\tau(\mathit{facts}), \tau(\mathit{extras}), \tau(\mathit{assump}), \tau(\mathit{peers})\} \\
&\quad \wedge\, (\mathsf{exists}(\mathit{extras}) \Rightarrow \mathit{extras} \in \mathrm{argv}(r^*)) \\
&\quad \wedge\, (\mathsf{exists}(\mathit{peers}) \wedge \neg\text{人给可比} \Rightarrow \mathit{peers} \in \mathrm{argv}(r^*)) \\
G_8 &: \mathrm{guard}(r^*).\mathsf{ok} \wedge \mathrm{commentary}(r^*) \equiv \mathit{nar} \\
&\quad \wedge\, (\mathsf{exists}(\mathit{src}) \Rightarrow \mathrm{guard.sources}) \wedge \tau(\mathrm{guard}) \geq \tau(\mathit{src}) \\
&\quad \wedge\, \mathrm{chars}(\mathit{nar}) \geq 15000 \wedge \mathrm{deep}(\mathit{nar}) \geq 0.6 \\
G_9 &: \mathsf{exists}(\mathit{draft}) \wedge \mathsf{exists}(\mathit{edit_log}) \wedge \mathsf{exists}(\mathit{review}) \wedge \mathrm{numlock}(\mathit{draft}, \mathit{nar}).\mathsf{ok} \\
G_{10} &: \mathsf{exists}(\mathit{pdf}) \wedge \tau(\mathit{pdf}) \geq \max\{\tau(\mathit{nar}), \tau(\mathit{script})\} \\
&\quad \wedge\, \mathrm{named} \neq \emptyset \wedge \forall p \in \mathrm{named} : \mathrm{sha256}(p) = \mathrm{sha256}(\mathit{pdf}) \\
&\quad \wedge\, 25 \leq \mathrm{pages} \leq 32 \wedge \mathrm{figs} \geq 60 \wedge \mathrm{R}(\mathit{render_check}) \\
G_{11} &: \mathsf{exists}(\mathit{delivery})
\end{aligned}
$$

其中 Q_{collect} 是所有 `phase=collect` 问题的检索词个数之和，Q_{followup} 是非 collect 问题的个数；T 是 `research/topics/*.md`；chars 去掉全部空白；cites(t) 是笔记中出现的不同 `[Sn]`；n_{topic} 是 `##` 深度专题一节内以“数字 + 点/顿号”或“-/*”开头的行数；VE 是 `verify_extras`，结果按（extras 哈希、年报哈希、校验代码哈希）三元组缓存在 `.pipeline/extras_check.json`，三者都不变才复用（测试 `test_extras_cache_without_verifier_fingerprint_is_rechecked` 验证缺校验代码指纹的旧缓存会被重算）；pages 为 `preview/page*.png` 张数，figs 为 HTML 中“图表 n”编号个数；named 是除 `report.pdf` 外另存的 PDF（`Config.output_name`，如“贵州茅台 1005.pdf”）；R 是渲染凭证检查（§11.10）。deep(nar) 按 `<!-- section: name -->` 切分文稿，取 name 以 `deep_` 开头的各节字数之和除以全文字数：

$$
\mathrm{deep}(\mathit{nar}) = \frac{\sum_{s:\ \mathrm{name}(s) \sim \texttt{deep_}*} \mathrm{chars}(s)}{\mathrm{chars}(\mathit{nar})}, \qquad \mathrm{chars}(x) = \big|\, x \setminus (\text{空白} \cup \texttt{<!--}\cdots\texttt{-->}) \,\big|. \tag{57}
$$

定义 11.3 (前序阻断)**.** $\widehat{G}_1 = G_1$，$\widehat{G}_i = G_i \wedge \widehat{G}_{i-1}$（$i = 2, \ldots, 11$）。$\mathrm{done}(\mathscr{W}) = \widehat{G}_{11}$，$\mathrm{next}(\mathscr{W}) = \min\{i : \neg\widehat{G}_i\}$。

实现上，`status` 先算出全部 G_i，再从前往后扫描：一旦遇到未通过的阶段，后面所有“通过”都改为未通过，`detail` 追加“（前序阶段未通过）”。这样输出里每个阶段的真实状况仍然可见（便于定位），但只有 $\widehat{G}_i$ 计入完成。`test_status_gates_and_collect` 的最后一步验证：`facts.json` 被修改（时间戳推后）后，即便 `delivery.md` 存在，阶段 11 也判未通过。

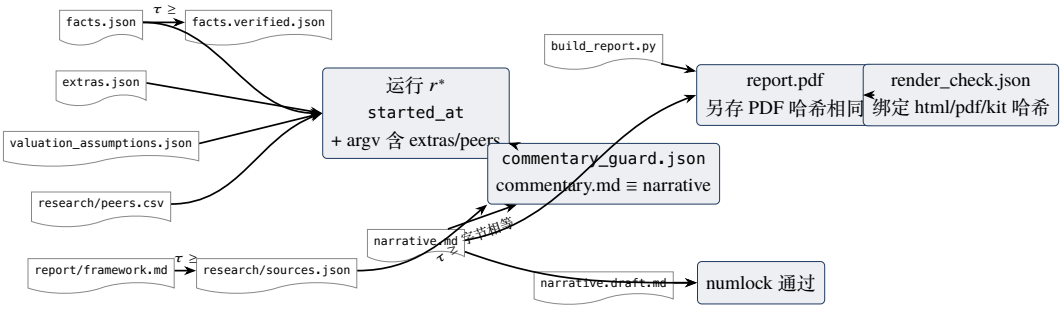


图 **17**　闸门中的新鲜度依赖。箭头 $x \to y$ 表示 y 必须不早于 x（或与 x 内容一致、哈希一致）；任何上游修改都会让下游闸门失效，直到重新校验或重新生成。

11.5.3　新鲜度：校验晚于修改、核对当前稿

闸门里的时间与等值条件构成一张依赖图（图 17）。每条边 $x \to y$ 的含义是“y 是对 x 的校验或由 x 生成，要求 y 不早于 x”。

“核对当前稿”。 `guard` 通过后会把被核对的文稿存为运行目录里的 `commentary.md`。闸门 8 要求它与当前 `narrative.md` 逐字节相等。这堵住了一条常见路径：模型先让一版文稿过 `guard`，再改文稿（加一句话、改一个数），然后汇报“guard 已通过”。时间戳比较挡不住这种情况（改动可能发生在同一秒内，或文件被复制回去），内容比较可以。复核编辑改写后必须重跑 `guard`，否则闸门 8 失效，进而闸门 9 之后全部阻断。

`guard` 晚于来源登记。 `sources.json` 在 `guard` 之后又被追加（例如复核时补登记）时，旧的 `guard` 结果不能证明新的外部引用合规。同时要求 `guard` 带了 `--sources`：没带时，外部数字的 `[S 编号]` 检查根本没有执行。

运行晚于输入，且带了输入。 估值运行早于任何一个输入文件的修改就算过期；工作区有 `extras.json` 而运行没带 `--extras`，或研究助理写了 `research/peers.csv` 而运行没带 `--peers`，同样不通过。后一条是因为模型常常“忘了”自己产生的可选输入，导致研报里写了可比公司分析、估值产物里却没有对应的计算。

命题 11.1 (过期传播)**.** *若在某时刻 $\widehat{G}_{11}(\mathscr{W})$ 成立，此后修改了 `facts.json`、`extras.json`、`valuation_assumptions.json`、`research/peers.csv`、`research/sources.json` 或 `narrative.md` 中的任意一个，而未重跑相应校验与生成步骤，则 $\widehat{G}_{11}(\mathscr{W})$ 不再成立。*

*论证。*逐项对应图 17 的边：facts 的修改使 G_2 的时间条件失效；extras、假设或 peers 的修改使 G_7 的 start(r^*) 条件失效；sources 的修改使 G_8 的 $\tau(\text{guard}) \geq \tau(src)$ 失效；narrative 的修改使 G_8 的字节相等失效，也使 G_{10} 的时间条件失效。由前序阻断，任一 G_i 失效都使 $\widehat{G}_{11}$ 失效。□

这个命题的覆盖面是有限的，我们如实列出未覆盖的边：`framework.md` 在阶段 4 之后被修改，只会使 G_4 失效（要求 sources 晚于框架），不会要求文稿重写；`dossier.md` 与专题笔记没有任何下游时间约束；`edit_log.md` 与 `review.md` 只检查存在。这些文件的内容最终由盲评与人工检查，闸门只管“有”和“新”。

11.5.4　为什么数量闸门只是下限

闸门里有一批纯数量条件：检索次数、来源个数、专题笔记字数、正文字数、深度研究占比、页数、图表数。它们是*必要条件*，用来挡住“写短逃避深度”这种已经发生过的退化（失败案例 4：排版交给复核会话后，研报从约 20 页掉到 11 页、图表从五六十

个掉到 19～26 个），但它们不是质量的度量。数量指标一旦成为目标，模型就会对着它优化，这是 Goodhart 定律的典型场景 [22, 55]。真实运行数据给出了直接证据（表 82）。

表 82 真实运行中的“压线”现象（取自各工作区会话结果中的 `status` 输出）

工作区	正文字数（主笔 → 复核后）	深度研究占比	页数	图表数
贵州茅台 1005	30,501 → 29,552	61% → **60%**	**25**	83
宁德时代 1005	23,404	**60%**	**25**	102
招商轮船 1005	27,390 → 26,107	**60%** → **60%**	30	70
中际旭创	19,882 → —	**60%** → **60%**	30	68
宇树科技	22,474 → 21,968	63% → 63%	30	67
阈值	≥ 15,000	≥ 60%	25–32	≥ 60

五家里有四家的深度研究占比恰好停在 60%，两家页数恰好是 25。更说明问题的是中际旭创的复核会话：它删去自证句后深度研究占比降到 59.85%，于是“又精简了年报点评部分的次要数字，把占比拉回 60%”。宁德时代的排版会话中间出现过 24 页的版本，它把半栏图高度从 46mm 加到 50mm 凑回 25 页；贵州茅台的排版会话则在汇报里明确写了“25 页刚好压在页数下限上”“我没有用重复图表或空话去填”。模型对阈值的反应是*贴着阈值达标*，而不是把阈值当作“至少”。

我们的应对不是继续提高阈值，而是：(1) 数量闸门只设“明显不够”的下限，防止退化；(2) 真正的质量判断交给看不到这些阈值的盲评会话（L2 分析质量、L3 表达与合规，第 12 节），盲评条目都是“是否回答了某个问题”而不是“有多少字”；(3) skill 里对篇幅的表述是“内容不够时回去补深度调研，不要缩短篇幅，也不要用重复的图表和空话凑页数”，把压力导向调研而不是填充。`scorecard` 的说明行也写着：“指标都是客观计数，不代表研报质量高低。”

要点：闸门管“有没有、新不新、对不对得上”，盲评管“好不好”

程序闸门擅长判定存在性、新鲜度、一致性（哈希、字节相等、数字锁）和下限；它们几乎不会误判，也无法被模型的自述骗过。质量判断天然是模糊的，交给另一个模型并接受它有噪声（重复盲评总分极差 7～8，第 12 节）。把两者混在一起（例如在闸门里加“正文必须包含预期差一节”这类内容规则）会把闸门变成可被应付的清单。

11.6 先搜集、后分析：资料包与深度专题

11.6.1 动机

年报讲的是过去，而且只讲公司自己。我们在 v0 批次（2026-10-03 首轮质检）看到的典型缺陷是：预测首年只外推年报，没有和年报之后已披露的半年报衔接；行业规模、竞争排名、对手价格等年报之外的内容凭模型记忆写，没有出处。这类内容第 10 节的 `guard` 能拦下“无出处的数字”，但拦不住“该写而没写”。提交 `c131c2c` 因此把调研从写作中间挪到最前面，单独设一个研究助理会话做阶段 1；写完框架之后再有一次针对性的补充调研（阶段 4，后来扩充为深度调研）。v0 → v1 的总分从 46～49 升到 67～74，版本间同时改了几处，不是对照实验，但资料搜集是其中最大的一项改动（第 13 节）。

与产业链分析子选题的关系。 产业链分析在 LEDGER 中不是一个独立模块，而是由本节的资料搜集与深度专题共同完成，并沿用“模型判断、程序把关数字”的分工：研究助理在 `industry`（行业规模、上游原材料价格、景气指标）、竞争格局与可比公司等类别下

登记外部来源；分析师在分析框架中写出驱动清单，把“上游价格 → 公司成本”“下游需求 → 公司量价”这样的链条逐环落到年报位置、经营数据表与预测参数上，并在需要时把产业链环节设为深度专题；主笔按写作规范用产业链示意图（reportkit 的 `C.flow`，标出公司所在环节）与行业章节呈现结论。链条上的每个外部数字都必须带 `[S 编号]` 并通过护栏（第 10 节），每个公司数字都来自真值表或经营数据表；链条本身的因果判断属于“观点”，由复核编辑与盲评检查论证是否成立。技能不预设任何行业的产业链模板（第 4 节 P11），不同公司的链条由模型从年报与资料包中推导。

11.6.2 七个类别与覆盖判定

研究助理按七个类别搜集（`output/sources.py` 的 `CATEGORIES`），见表 83。

表 83 资料搜集的七个类别（`research.md` 第一部分）

category	搜什么	用在哪里
latest	年报之后的季报、半年报、业绩预告与快报、月度经营数据	预测首年与最新实际衔接
market	最近交易日收盘价与日期、市值、近三个月走势；新股的发行价与上市日	估值参照；股价必须有来源与日期
consensus	卖方一致预期、评级分布、目标价区间（查不到登记 `not_found`）	预期差
peers	3 ~ 6 家可比公司的股价、市值、倍数、最近财年收入利润	交叉验证；可写 `peers.csv`
industry	行业规模、增速、渗透率、价格指数、上游原材料、景气指标	驱动假设的外部依据
competition	份额与排名、对手产能与价格动作、新进入者	竞争格局判断的出处
events	近三个月公告与新闻：订单、产能、并购、股权、分红回购、处罚、政策	年报之后的变化

`coverage(data, as_of, 90)` 的判定规则是：一个类别被覆盖，当且仅当存在一个 `phase=collect` 的问题，其 `status=found`，或者 `status=not_found` 且换过至少两个检索词。后一条承认“查不到”是合法结果（例如新上市公司没有卖方一致预期），但要求确实尽力查过。“近期来源”按来源的 `published` 日期计，$0 \le (\texttt{as_of} - \text{published}) \le 90$ 天。检索方法上，`research.md` 要求每个问题换两三个关键词、中英文都试，WebSearch 找到候选后*必须用 WebFetch 打开原文核对数字*，“只看搜索摘要就登记，等于没核对”；来源优先级为交易所与公司公告 > 政府与行业协会 > 知名研究机构与数据商 > 主流财经媒体，自媒体、论坛、百科不用；只登记原文里出现的数字，不自行换算单位、币种或百分比。

11.6.3 来源登记与资料包

所有检索结果登记进同一个 `research/sources.json`（清单 4）。`questions` 记录问了什么、用了哪些检索词、结果如何；`sources` 记录每个来源的发布方、网址、发布与访问日期，以及从原文摘出的事实：`quote` 是包含该数字的原句（≤ 120 字），`value` 必须以原样写法出现在 `quote` 中，由 `verify-sources` 检查（第 10 节）。正文引用外部数字时句末带 `[S 编号]`，`guard` 把 `sources.json` 中的数字当作出处，但要求所在句子带对应编号。

Listing 4 `research/sources.json` 的结构（字段名为真实字段，取值为占位）

```
{"questions": [
   {"id": "Q1", "phase": "collect", "category": "latest", "question": "...",
```

```
    "status": "found", "queries": ["...", "..."], "sources": ["S1"]},
   {"id": "Q9", "phase": "followup", "question": "...", "status": "not_found",
    "queries": ["...", "..."], "sources": []}],
 "sources": [
   {"id": "S1", "title": "...", "publisher": "...", "url": "https://...",
    "published": "2026-08-30", "accessed": "2026-10-03",
    "facts": [{"quote": "<sentence containing 69.6>", "value": 69.6,
               "unit": "...", "label": "..."}]}]}
```

资料包 `dossier.md` 是给下游看的简报，结构固定：一句话写“年报之后最重要的变化”，然后按七个类别分节，每条事实句末带 `[S 编号]`；“对预测与估值的含义”一节只写方向和需要校准的地方（如“上半年净利润已达 X，全年预测要与之衔接”），不替分析师下结论、不起草假设数值；最后列“没查到的”。这条限制让研究助理与分析师的分工清楚：前者负责事实，后者负责判断。

闸门 6 把“先搜集”落到估值上：假设文件必须有股价与来源，且依据中至少引用一处 `[S 编号]`。`research.md` 也要求“预测首年与年报之后的最新业绩衔接”，评分标准 L1-08 再用程序检查预测与历史的衔接幅度（第 12 节）。

11.6.4 分析框架与深度专题

分析师在阶段 3 写 `report/framework.md`。`analysis.md` 给出推导步骤（收入怎么来、成本与利润率怎么来、钱怎么流动、哪些指标领先于收入、公司自己怎么看、怎样对应到预测参数）和框架的六个小节：投资主线、驱动清单、外部调研清单、深度专题、披露局限、章节结构。其中对“投资主线”的要求最具体，每条主线要逐项过六项检查：**具体**（落到可度量的驱动）、**有预期差**（与一致预期差在哪个变量、差多少）、**落到估值**（对应哪个预测参数、敏感性多大）、**可证伪**（什么数据出现时判断不成立）、**有检验时点**、**知道最强反方**。过不了的降为跟踪事项。

深度专题是 3～5 个“对结论影响最大、市场有分歧的问题”，从投资主线和资料包里的分歧倒推，不预设行业套路。每个专题写清要回答什么、需要哪些数据、结论落到哪个预测参数或情景。中际旭创的四个专题是一个真实例子：需求持续性（2027 年之后的行业出货 → 光模块销量增速三档）、价格与上游物料对毛利率的影响（→ 单价与毛利率变动）、竞争格局与技术路线（→ 远期销量与永续增长）、地缘政治与客户集中（→ 悲观情景与公司特有风险溢价）。每个专题都绑定了 `segment_forecast` 里的具体参数，这就是“驱动 → 参数”的落地方式。

阶段 4 为每个专题写一份笔记 `research/topics/< 专题 >.md`：多来源交叉、定量推导、结论与对预测的含义，事实句末带 `[S 编号]`；补充检索追加进 `sources.json`（`phase: followup`）。闸门要求笔记 ≥ 3 份、每份 ≥ 1,500 字且引用 ≥ 3 个不同来源、补充问题 ≥ 8 个，且 `sources.json` 晚于框架（证明框架之后确实补查过）。在文稿里，每个专题成为一节 `deep_topic_N`，与行业、竞争、核心竞争力、预期差几节一起构成“深度研究”部分，即式 (57) 分子中的 `deep_` 节。表 84 是五家公司的真实规模。

与表 82 的写作篇幅不同，调研的各项计数都明显高于下限。我们的解释是：调研阈值远低于完成任务本身需要的量（例如写四个专题、每个专题三个以上来源，本身就需要十几次补充检索），所以阈值不构成“目标”；而正文占比和页数的阈值正好落在模型“写完即可停”的区间里。

表 84　五家公司的调研规模（各会话结束时 status 输出；“近 90 天来源”随补充调研增长，取最后一次）

工作区	搜集问题	搜集检索	近 90 天来源	专题笔记	补充问题	来源总数
贵州茅台 1005	20	42	31	5	12	51
宁德时代 1005	17	40	41	4	10	48
招商轮船 1005	17	33	37	4	12	52
中际旭创	15	31	39	4	16	51
宇树科技	12	27	30	4	15	44
闸门下限	—	20	3	3	8	—

权衡（Trade-off）：资料搜集由独立会话做，还是并入分析师会话

并入的好处是分析师可以边读年报边决定查什么，检索更有针对性；资料包这一层交接也可以省去。独立的好处有三：一是研究助理在“不读财务细节”的约束下会把七个类别都搜到，而不是只搜与自己初步判断相关的内容（确认偏差）；二是资料包作为文件留存，主笔和评审都能看到“动笔之前知道什么”；三是分析师会话本身已经是最长的生成会话之一（13 ~ 30 分钟、100 余轮），再加检索会更早触发上下文压缩。框架之后的针对性检索仍由分析师做（阶段 4），两种好处都保留。

11.7　`refine`：盲评驱动的返工闭环

`run` 只保证闸门通过；闸门不判质量。`refine` 在 `run` 之后加上盲评与返工：生成 → 复核 → 质检，未达交付标准时把问题单按类型路由回对应角色，最多若干轮，结束时回滚到得分最高的一轮。这一结构与自我改进（Self-Refine）和语言反馈强化（Reflexion）一类方法同源 [39, 52]，区别在于三点：反馈来自另一个模型的盲评而不是生成者自己；反馈只含理由与证据，不含评分标准；返工被路由到不同的角色和会话，而不是同一个会话自我修改。

11.7.1　盲评目录与信息隐藏

每轮评测在 `~/finagent-evals/<工作区名>/r<n>/` 下进行（`judge.default_root()`，可由 `FINAGENT_EVAL_DIR` 改），分为 `blind/`（评审会话的工作目录）与 `eval/`（程序检查、判定、计分与问题单）。`judge.prepare` 搭盲评目录时同样调用 `_check_location`，并只放入：交付的 PDF、`narrative.md`、6 个估值产物（`valuation.json`、`forecast_summary.json`、`segment_forecast.json`、`scenarios.json`、`non_operating_assets.json`、`comps.json`）、`sources.json`、`review.md`，以及 `rubric.json`（`judge_items` 输出，只含条目编号、标题、判定标准与反例，不含分值），和一份评审说明 `CLAUDE.md`。评审会话的工具白名单只有 `Read(./**)`、`Glob`、`Grep`、`Write(./judgments.json)`，不能联网，也不能写别的文件。评审说明要求“拿不准时判不通过”，并写明“数字与年报是否一致、三表是否勾稽、估值算术是否正确，已由程序另行核对，你不必复算”。评审看不到写作与复核过程、模型名和分值。

评测结果 `issues.json` 由 `issues_from` 生成：每个未通过的正向条目或命中的负向条目变成一条问题，字段只有 `id`、`route`、`stage`、`severity`、`description`（评审的理由）、`evidence`

（评审摘录的原文与位置）。严重度按层与分值映射：

$$\text{severity}(r) = \begin{cases} \text{blocker}, & \text{layer}(r) \in \{\text{L0}, \text{N}\}, \\ \text{major}, & \text{points}(r) \geq 4, \\ \text{minor}, & \text{其他}. \end{cases} \tag{58}$$

条目标题、判定标准与分值都不出现在问题单里。表 85 是中际旭创 v2 评测的一条真实问题（评分标准 L2-08“估值方法与参数”未通过），左栏是评分标准里有、但不回传的部分，右栏是回传给生成侧的全部内容。

表 85 问题单的信息隐藏：同一条判定，评测侧可见与回传内容的对比（中际旭创 v2，问题 I02）

不回传（仅评测侧）	回传（`issues-generator.md`）
条目编号 L2-08、标题、判定标准原文、反例、分值 3、所在层 L2、该层得分 42/45、总分 90	**I02**（minor，阶段 8 文稿）参数列全，但没有说明为何选 FCFF，β 1.25、ERP 6.5%、0.5% 特有风险溢价、永续增长 3.0% 也没有依据（只有无风险利率有来源）。读者需要方法选择的理由和参数来源。证据：“我们采用 FCFF 折现。……β 1.25，股权风险溢价 6.50%，加 0.5% 公司特有风险溢价”（DCF 估值）

命题 11.2（评分标准不经由问题单泄露）**.** *设评分标准条目为* (id, title, criteria, points, route, stage)，*评审输出* (passed, evidence, reason)。*回传给生成侧的文本是* $\{$(stage, severity, reason, evidence)$\}$ *的 Markdown 渲染。它与* title、criteria、points *的关系只有两条：* severity *透露* points ≥ 4 *与否（1 比特），*reason *是评审模型用自己的话写的理由。*

第二条是这个设计的残余风险：评审若在理由里复述判定标准，标准就部分泄露了。我们没有用程序过滤，原因是理由本来就应当说明“读者需要什么”，这本身就是返工所需的信息；而“为了拿分”与“为了读者”在这一层上并不冲突。真正要防的是生成侧看到全部条目后按清单逐条应付，这一点由不变量 23 保证。设计稿曾考虑把条目分为“反馈集”与“留出集”、只用后者计分以检测应试，目前未实现。

11.7.2 路由与返工

评分标准 v2 的每个条目都标了 `route`（`generator` 或 `reviewer`）与 `stage`。41 个条目中 31 个路由给生成侧（阶段分布：资料搜集 2、财务抽取 1、估值假设 5、估值运行 6、文稿 17），10 个路由给复核侧（复核改写 7、排版 3）。`refine` 再把复核侧中 `stage` 以“10”开头的条目分给排版设计师。图 18 与算法 16 给出完整流程。

生成侧优先。 只要有一条生成侧问题，本轮就走“修订会话”分支：它可能补数据、改假设、重跑估值，文稿随之重写，所以先删掉初稿备份与编辑日志，让闸门 9 失效，复核必须从头再来（否则数字锁会拿新文稿与旧初稿比较）。修订会话的提示 `FIX_PROMPT` 要求“处理不了的写进 `review.md` 并说明原因”，“阶段 1 ~ 8 全部通过后停止”。

复核侧只改措辞。 没有生成侧问题时，复核编辑以 `revise` 角色处理问题单：只改措辞、结构与编排，数字锁照常（`narrative.draft.md` 是初稿，不覆盖），需要补数字或改结论的写进 `review.md`；改完重跑 `guard` 与数字锁，用现有排版脚本重新渲染（不改版面），更新交付说明。版面类问题另起排版会话。

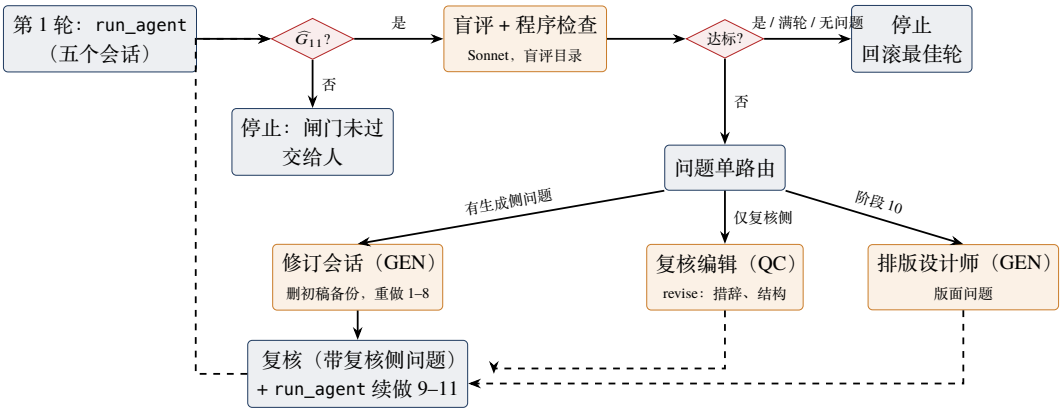


图 **18** `refine` 的一轮。生成侧问题优先：只要有一条，就由修订会话从数据与假设改起，并删除复核产物强制重新复核；否则只做复核侧的措辞修订与版面修订，不动数字。每轮结束都回到闸门判断，再进入下一次盲评。

停止条件。 `_stop_reason` 依次检查：最近一轮达到交付标准；轮数达到 K；最近三轮中连续两次提升都小于 δ；最近一轮没有可回传的问题。需要指出一个实现细节：平台期条件要求至少三轮，而默认 $K = 3$ 时第三轮先触发“达到最大轮数”，所以平台期条件只在 $K > 3$ 时实际起作用。

快照与回滚。 每轮评测后，`_snapshot` 把 `narrative.md`、`narrative.draft.md`、`edit_log.md`、`review.md`、`delivery.md` 与运行目录下的全部 PDF 复制到 `.pipeline/rounds/<n>/snapshot/`，并记录运行目录与文稿哈希。结束时若最佳轮不是最后一轮，`_restore` 把快照复制回去，*PDF 最后复制*，以保持“PDF 晚于文稿”的闸门条件。若中间重跑过估值（运行目录变了），旧文稿对应旧的估值产物，复制回去会破坏闸门 7、8 的一致性，此时不回滚，只在 `loop.json` 里指明最佳轮。

一个已知缺口。 在“有生成侧问题”的分支里，版面类问题 I_ℓ 没有单独回传：排版会话会因为文稿变化、PDF 过期而在 `run_agent` 中被重新调起，但它拿到的是普通的 `LAYOUT_PROMPT`，看不到上一轮的版面问题。版面问题要等下一轮评测再次发现。这是可以改进的地方。

11.7.3 真实运行情况

截至本文，`refine` 在仓库中有记录的运行都在第一轮达到交付标准或被人工终止，尚无多轮返工的完整记录：中际旭创、宇树科技的 v2 版本在第 1 轮分别得 90、88 分，达标即停（`eval/results/v2_*/`；第 13 节）。设计稿（2026-10-03）也写明“用模拟会话测过路由与回滚，尚未真跑一整轮”。表 86 是仓库中全部 7 次盲评会话的真实用量；表 87 用编造的数字示意一次三轮返工的预期形态，供读者理解路由与回滚的效果，不代表实测。

盲评一次约 3 分钟、1 ~ 2.4 美元等值，约为一次完整生成（表 91，约 64 ~ 68 分钟、24 ~ 27 美元）的 5% ~ 9%。多轮返工的主要成本在生成侧：一次修订会话加上重做复核与排版，约相当于完整生成的 40% ~ 60%。

11.8 `collect` 与 `scorecard`

`collect`。 把工作区产物收回仓库：抽取、调研与文稿（整个 `data/extractions/<key>/`）复制到仓库同名目录；最新运行目录复制到仓库 `runs/`（已存在不覆盖）；PDF、估值

表 86　盲评会话的真实用量（eval/results/*/judge_run.json，模型均为 claude-sonnet-5-5）

评测	耗时（秒）	轮数	等值费用（美元）	输入 token	输出 token
v0 贵州茅台	194.6	11	0.95	1,029,108	15,233
v0 宁德时代	225.9	13	1.17	721,888	17,686
v0 招商轮船	148.6	15	1.38	924,866	17,248
v1 中际旭创	165.1	14	1.31	673,831	17,608
v1 宇树科技	173.5	17	1.30	1,113,681	18,228
v2 中际旭创	162.8	17	2.27	1,068,423	13,120
v2 宇树科技	253.7	17	2.35	956,189	19,992
平均	189.2	14.9	1.53	926,855	17,016

表 87　一次三轮 refine 的示意轨迹（编造，仅说明机制）

轮次	总分	达标	生成侧问题	复核侧问题	本轮返工
1	71	否	3	4	修订会话补可比公司表、重跑估值；复核与排版重做
2	79	否	0	3	复核编辑 revise：拆数字罗列句、删程序口吻
3	82	是	0	1	达标停止；第 3 轮即最佳，不回滚

模型与复核清单复制到 report/交付/<MMDD> < 公司 > < 标签 >/，PDF 重命名为“公司名 + 日期”（如“宇树科技 1003.pdf”），模型命名为 sheet.xlsx，review.md 命名为“复核清单.md”。三条保护：闸门未全部通过时拒绝（--force 可强制）；仓库已有同名抽取目录时要求 --overwrite；测评对照时用 --delivery-only 只收交付物、不覆盖抽取文件（test_status_gates_and_collect）。运行目录里的 manifest.json 记录了命令行，由于年报在工作区与仓库中的相对路径相同，收回后可在仓库里离线复算全部数字（附录 G）。会话事件流留在工作区 .pipeline/agents/，不进仓库（单家公司约数十 MB）。

scorecard。 对多个工作区并排输出客观指标，用于比较不同模型或配置（build --tag 区分工作区）。列为：闸门通过数（/11）、财务事实数、经营数据格数、估值运行次数、检索次数、外部来源数、正文引用的不同来源数、guard、初稿与终稿字数（反映复核删改量）、页数、图表数、评级、DCF 基准每股价值、累计耗时、等值费用、输入与输出 token。未通过全部闸门的工作区会单列“卡在哪一步”，因为其后续指标不可信。表末固定一句说明：“指标都是客观计数，不代表研报质量高低；正文分析深度、论证是否成立需要人工阅读 PDF 判断。”finagent-pipeline skill 进一步提醒：输入（年报、股价、可比公司）必须相同，否则估值不可比；评级与 DCF 不同通常来自各自起草的假设不同，应对比两边的 valuation_assumptions.json，不要直接判谁对。质量比较交给第 12 节的评测（python -m finagent.evaluation compare）。

11.9　四个 skill：方法与约束，而不是行业模板

11.9.1　设计哲学

skill 是给模型看的操作说明。我们的原则是：**skill 只给方法与约束，具体到这家公司该分析什么，由模型读年报与资料包自己推导**。这条原则来自一次明确的取舍。最直接的做法是为每个行业写模板（白酒看吨价与渠道、电池看出货量与单位盈利、航运看运价与船队），它能让初稿更快成形，但有三个问题：A 股行业多、细分更多，模板写

不完；同一行业的公司差异常常比行业共性更重要（茅台有控股财务公司，招商轮船有多个船型分部）；模板会把模型的注意力从“这家公司今年最重要的变化是什么”引向“把模板的格子填满”。

所以 `analysis.md` 的开头就是“本 skill 不预设任何行业的分析框架”，唯一的示例（茅台框架怎么推出来）末尾写明“这些都是读茅台年报时发现的；换一家公司，结论可能完全不同”；`sections.md` 自称“是菜单，不是清单”；三份 20 页级的定制排版脚本放在 `templates/examples/` 而不是 `templates/`，注释写着“参考样例，不是模板”。隔离工作区（§11.3）是这条原则的物理保证：工作区里没有别家公司的框架可抄。

权衡（Trade-off）：行业模板 vs 方法 + 约束

行业模板降低了对模型能力的要求，输出更整齐，评审也更容易对照。**方法 + 约束**依赖模型的推导能力，不同公司的报告结构差异大，初稿的质量方差也更大。我们选后者，理由是这里的瓶颈不在“知道该看什么指标”（强模型在常见行业上都知道），而在“这家公司这一年的特殊之处”；模板恰恰削弱后者。方差由两道机制控制：投资主线的六项检查与深度专题的“落到参数”要求，以及盲评 L2 条目（核心结论、预期差、驱动拆解等）。

11.9.2 四个 skill 的分工与关键条款

表 88 四个 skill（行数含 front matter；`references/` 为按阶段读取的参考文档）

skill	使用者	行数	关键条款
`finagent-pipeline`	仓库里的编排者	128	不自己写研报；输入问用户不自己找或编；不要在当前会话进入工作区（上下文已有仓库内容）；不要把评分标准或分数告诉工作区会话；不绕过闸门
`finagent-research`	分析师	156 + 597	铁律五条：不做算术、数字来自原文带页码、只取合并报表、不静默取舍、估值假设标明观点；`verify-facts` 最多 3 轮；估值口径（分部驱动、三表驱动 DCF、EBIT 口径、评级规则）
`finagent-report`	研究助理、主笔、排版	150 + 360	不可协商约束六条（数字只来自产物、外部信息检索登记、事实推论观点措辞、写给投资者、评级取程序、字体与免责声明共用）；交给模型判断的部分（先找故事再定结构、框架自己推导、图表服务论点）；reportkit 速查；常见坑
`finagent-review`	复核编辑	97	只改措辞与结构、不碰数字（数字锁）；不改结论评级估值；先通读回答三问；逐段问“去掉它读者少知道什么”；对账（同一指标一个数、结论与图表一致、各节判断一致）；十一类机器腔写法

几条值得单独说明的条款：

“拿不准”的三分法。 `writing.md` 规定：能自己判断的直接写判断，不加缓冲词；拿不准且不影响结论的，不写；拿不准但影响结论或估值的（口径疑问、外部数据矛盾、关键假设缺依据、年报疑似笔误），写进 `review.md` 交人工，正文不出现。这条规则把“模型的不确定性”从正文里移到一个结构化的清单中，给人工复核一个明确的入口。贵州茅台 1005 批次的复核清单有 14 条，最重要的一条是“茅台酒 2026E 销量 +3% 与公司‘全年投

放总量不增'的表态口径有冲突"。

机器腔的识别线索。 `finagent-review` 列出十一类写法（否定对照、自证与圆场、交代缺失、程序口吻、口语套话、解释腔、事后圆场、行内标签与页码、数字罗列、重要处定性、无来源判断），每类给出例子、为什么不好与怎么改，并附四条来自用户指出过的真实初稿的改写示例。skill 明确说这些"是判断的线索，不是机械替换清单"。用户明确反对用程序拦截文风（CLAUDE.md 设计规范 11），所以这部分只在复核会话里由模型判断，评测侧对应的是 L3-04"无机器腔"条目。

汇总数的构成要先拆再写。 `finagent-report` 的"常见坑"第一条记录了茅台样例中最严重的一次错误：股权价值桥的加回项 1,789.15 亿元被写成"拆出资金、货币资金等"，扣减项被写成"财务公司吸收的存款"，数字都过了 `guard`，归属全错。规则因此是：写任何汇总数的构成之前，先在产物里把它拆开（估值桥看 `plan.json`，利润归因看 `earnings.json`）；reportkit 也增加了逐项股权价值桥表，让读者能对账。评分标准 N-03"聚合数构成描述错误"对应这一类。

渐进披露。 四个 skill 加参考文档约 1,700 行。工作区说明的"这时才读"列把它们拆到各阶段（§11.3），`finagent-pipeline` 则明说"研报怎么写在工作区里的三个 skill 和工作区 CLAUDE.md 里，你不需要读它们"。

11.10 排版：reportkit 与渲染实测

11.10.1 从产物到 PDF

排版设计师为每家公司写一个编排脚本 `report/build_report.py`，从模板 `templates/build_report.py` 起步，参考三份定制样例（茅台、宁德、招商，各约 480 ~ 500 行）。脚本只提供 `Config` 与 `body(doc, S)`，其余由公共库 reportkit 完成：

1. `Run(run_dir)` 读运行产物（真值表、经营数据与比率、盈利预测、估值、三表），`load_sections` 按 `<!-- section: -->` 切分文稿；
2. 图表由 matplotlib 画成 SVG（文字转曲，避免缺字），表格由 `tables` 生成 HTML 三线表，`Doc.fig` 自动编号"图表 *n*"并记录标题；
3. `layout.build` 拼出首页（标题、投资要点、评级与估值、基本数据、盈利预测表）、目录、图表目录、正文与附录（报表预测、方法说明、数据来源与数字构成、外部资料来源、免责声明）；
4. 第一遍打印 PDF，`locate_pages` 在 PDF 文本中定位每个章节标题与图表编号所在页；第二遍把页码回填进目录再打印；
5. `preview` 输出逐页 PNG（110 dpi）与每 8 页一张的缩略图总览，打印嵌入字体清单，供排版会话用 Read 工具"看"版面。

数字规则与引擎一致：表格和图里的数字由脚本从产物 JSON 读出，脚本只做单位换算与格式化，不计算新的比率；外部数字从 `sources.json` 读。贵州茅台 1005 批次的排版会话自己写了一个 `F(来源编号, 标签)` 函数：从 `sources.json` 的 `facts` 中按编号与标签取值，取不到就报错，脚本里没有手写的外部数字，83 个图表中 35 个使用外部资料。

表 89　reportkit 模块（.claude/skills/finagent-report/reportkit/）

模块	行数	内容
core.py	238	Run（取数）、格式化（亿元、百分比、精度按数量级自动取）、Markdown 转 HTML、配色常量
charts.py	516	标准图 11 种（首页小图、历史、季度、利润归因瀑布、分部预测、情景、估值桥、足球场图等）；原语 8 种（分组柱、堆叠、正负柱、横条、环形、进度条、并排小柱、构成条）；非数据图 2 种（时间轴、链条示意）
tables.py	362	标准表（预测、三年、季度、资产负债、假设、分部预测、WACC、FCFF、情景、敏感性、可比、反向 DCF、股权价值桥）；chg() 涨跌双色
layout.py	480	Config、Doc、首页、目录两遍定位、附录、浏览器查找、main
render.py	147	预检、打印、PDF 实测字号、渲染凭证
render_probe.js	62	在预检 HTML 中按 180mm 版心测量溢出、正文字号、图片加载
report.css	222	A4 版式、字体栈、红色主题、页眉页脚、分页规则

11.10.2　整体缩放缺陷与渲染实测

失败案例：CSS 声明 10.5pt，PDF 里的字却更小

2026-10-05 发现，部分研报的正文看起来比其他报告小。原因是 Chrome 打印时遇到超出版心宽度的元素（宽表格、不换行的长表头或网址、侧栏里宽于容器的图片），会整体缩小整份 PDF 以容纳它。CSS 里的 font-size: 10.5pt 一直是对的，HTML 层面的任何检查都发现不了，只有在 PDF 里测量实际字形才能看到。模型逐页看预览图也没有发现，因为每一页内部比例协调。提交 81895a3 修复：公共样式让长表头、文字说明与网址换行，分栏用 minmax(0, 1fr)，六期财务附表改整栏；并增加下述“预检 + 实测 + 凭证”机制。

render.to_pdf 的流程（算法 17）分三步，任何一步失败都保留上一份 PDF，并把失败原因写进凭证。

容差 0.12pt 来自 Chrome 毫米到像素的取整（约 0.03pt 偏差）加余量。闸门 10 中的 R 由 check_render_record 判定：凭证版本与 ok 正确；凭证中的 HTML 与 PDF 哈希等于当前文件；排版代码哈希（render_check.py 与 reportkit 下全部 .py/.css/.js/.md 按文件名排序后串联求 SHA-256）等于当前代码；预检无溢出、无字号错误、无断图；实测样本非空且都在容差内。

$$\mathrm{R} = [\, rec.\mathrm{ok} \,] \wedge [\, rec.h_{\mathrm{html}} = H(html) \wedge rec.h_{\mathrm{pdf}} = H(pdf) \wedge rec.h_{\mathrm{kit}} = H(\mathrm{kit}) \,] \\ \wedge [\, \forall s \in rec.\mathrm{samples} : |s - 10.5| \le 0.12 \,]. \quad (59)$$

哈希绑定的意义是：凭证只证明这一份 HTML、这一份 PDF、在这一版排版代码下通过了检查。模型改了 PDF（例如用别的工具压缩）、改了 HTML、或仓库升级了 reportkit，凭证都会失效。test_report_render.py 的五个用例覆盖：凭证绑定（换 PDF 或改 CSS 后失效）、预检失败保留旧 PDF、真实浏览器下长表头与网址与六期附表能通过、四类非法版式被拒（260mm 宽元素、超长单元格、8pt 正文、缺失图片），以及“预检时 10.5pt、打印时被 zoom:0.8 缩小”能被实测独立识别。后三项需要 FINAGENT_BROWSER_TESTS=1 与本机 Chrome。

权衡（Trade-off）：检查 CSS、检查 DOM 还是检查 PDF

只检查 CSS 声明最便宜，但发现不了缩放；只在 DOM 里测溢出能发现大部分原因，但浏览器的打印路径与屏幕路径不同，预检通过不代表打印结果正确（测试用 `zoom` 构造了这种情况）。直接在 PDF 里测量是唯一能确认“读者看到的字号”的方法，代价是要在 PDF 字符流里可靠地定位正文：我们用预检时从正文段落里采的样本做锚点，用 NFKC 归一处理全角与兼容字符，只比较样本区间内的字形。三层都做：预检给出可操作的错误信息（哪个元素溢出），实测给出不可绕过的结论。

美观检查仍靠模型看图。 程序检查不了图中文字重叠、孤立标题、大块留白。排版会话的提示要求“逐页看 preview 修到美观”，真实会话里这一步做得相当细：贵州茅台的排版会话报告修了“第三部分标题栏孤悬在页底”“目录里的跟踪表被编成了图表 1”“文字表格不是左对齐”“价格图、时间轴、成本图里的标注互相重叠”“有一张溢出页只有三行”五处，并说明首页下方约四分之一空白是固定版式所致、没有用重复图表填充。

测试。 `test_report_kit.py` 在三种情形上各渲染一遍模板：样例公司（有负债、有可比公司、低价股、无分部预测；验证图表连续编号、占位符转义、敏感性矩阵两位小数）、人为构造的亏损公司（无分季度、无可比公司；验证缺数据的图表整块跳过）、茅台真实脚本（57 个图表）；另验证红色主题、股权价值桥逐项加总等于股权价值、非数据原语与引用上标。修改 reportkit 后必须跑这组测试。

11.11 失败案例

下面的案例除注明外都来自真实运行记录（会话事件流与提交历史）。它们大多不是模型“能力不够”，而是系统边界与激励设计的问题。

失败案例 1：仓库内直接写研报

现象见 §11.1：读到其他公司的框架与文稿、可能读到评分标准、单会话上下文耗尽。在一次早期试跑中，模型在写新公司的分析框架前先列出了仓库里已有的三份 `framework.md` 并逐一读取。处理：提交 `58886cf` 引入隔离工作区，`ab4a016` 起工作区不复制 `finagent/evaluation/`，评测结果放到工作区之外。

失败案例 2：权限规则误拦公司目录（宇树科技，2026-10-03）

为保护年报原件写了 `Write(./report/**)`，它同时匹配了 `data/extractions/宇树科技_2025/report/`。写作会话完成了阶段 2～8 的全部工作（113 个财务事实、19 张经营数据表 367 格、估值运行、`guard` 通过的文稿），却写不进框架与文稿；它尝试用白名单内的 `cp` 也被拒，于是把文件暂存在 `_stage/` 并在汇报中准确说明原因、停止，没有寻找其他绕过方式。闸门因框架缺失判阶段 3～8 未通过，编排器停止。这次会话用了 89 轮、约 27.8 分钟、4.93 美元等值。处理：提交 `4f70953` 把规则改为 `report/resources/**`。教训：deny 规则用路径模式时要按“它还会匹配什么”审查，并用一次完整的无头运行验证；同时，模型“被拦即停、如实汇报”的行为让这类配置错误很容易定位。

失败案例 3：前缀权限规则匹配不到（中际旭创，2026-10-03）

排版命令 `./py data/extractions/中际旭创_2025/report/build_report.py --run-dir ...` 两次返回“This command requires approval”。原因是旧式前缀语法 `Bash(./py data/extractions/:*)` 只在词边界匹配。同一会话里 `cp` 模板也被拒（Claude Code 不允许从 `.claude/` 复制），模型改为直接写脚本。处理：提交 `6d9a436` 改通配规则，并新增 `template` 子命令。

失败案例 4：写短逃避深度（2026-10-03）

10-02 起排版由复核会话顺带完成，当天又把复核默认模型改成 Sonnet。复核会话只套了 126 行的通用模板，中际旭创 v1 的 PDF 只有 11 页、26 个图表（此前茅台样例为 21 页、56 个图表）；闸门当时不检查篇幅。处理：提交 `20948ae` 把排版交给独立的 Opus 排版会话，加入正文字数、深度研究占比、页数与图表数闸门，研报改为“年报点评 + 深度研究”结构，提供三份定制排版样例。v2 两家分别为 30 页 68 图、30 页 67 图。教训：一个角色同时承担“删减”（复核）与“扩充”（排版）两类目标时，会倾向于更省力的一方。

失败案例 5：贴着阈值达标

见表 82：五家中四家深度研究占比恰为 60%，两家页数恰为 25；中际旭创的复核会话为把占比从 59.85% 拉回 60% 而删减年报点评部分。宇树科技第一版文稿深度研究只占 57%，原因是框架列了四个专题、文稿只写了三个；闸门拦下后主笔补写“份额之争”专题，达到 63%。这一案例说明闸门有效（挡住了漏写专题），也说明它会被当作目标（§11.5）。

失败案例 6：PDF 整体缩放

见 §11.10。修复后用 `sync-renderer` 升级了已有工作区，并重新渲染了茅台与宁德 1005 两份报告。

失败案例 7：额度耗尽与连接中断

五家公司的会话历史中，有 7 次会话因订阅额度耗尽结束（宁德、茅台各一次，招商两次，中际一次，宇树两次），最长的一次发生在宇树科技排版会话的第 288 轮；另有 2 次“API Error: Connection lost mid-response”。所有中断都在额度恢复后用同一条 `run` 命令续跑，没有人工修补文件。这依赖三件事：会话边写边落盘；闸门只看文件；工作区说明要求接手时先跑 `status`。代价是墙钟时间：贵州茅台 1005 批次的有效会话时间 67.9 分钟，墙钟时间 5 小时 40 分（§11.13）。

失败案例 8：复核会话发现方向写反

中际旭创的复核会话在改写时发现一句“H 股相对 A 股折价”：来源 S7 的摘句里 A 股 808.44 元、H 股 1,072 港元，A 股更便宜，原句方向写反了。它删去了这一句（数字锁允许删除，不允许改数）。这类错误 `guard` 发现不了（两个数字都有出处），说明独立复核在程序检查之外有实际价值。

失败案例 9：手写文档与闸门常量漂移

`research.md` 第二部分写“闸门要求至少 3 个补充问题”，而源码 `MIN_FOLLOWUP=8`；`finagent-report` 写研报 25～30 页，闸门上限是 32 页；`finagent-report` 开头说工作区“只含两个 skill”，实际复制三个。茅台 1005 批次的排版会话也在汇报中指出了两处文档路径与实际不符。工作区 `CLAUDE.md` 不存在这个问题，因为它由 `render_guide` 用常量插值生成，所以模型以工作区说明为准时不受影响。教训：凡是闸门阈值，文档里都应由代码生成或引用常量名，不手写数字。

11.12 权衡汇总

前文各处的权衡盒子讨论了单项选择。这里把四个最主要的系统级权衡放在一起，并尽量给出数字。

关于“多会话是否比单会话好”，我们没有严格的消融实验。现有证据是间接的：v0（单会话、无资料搜集与独立复核）三家 46～49 分，v1（先搜集 + 独立复核）67～74 分，v2（+ 深度专题与篇幅闸门，排版独立）88～90 分（均为评分标准 v0、单次盲评，重复盲评总分极差 7～8）。版本之间同时改动了多处，不能把差异全部归于会话拆分。去掉资料搜集、去掉复核会话的消融列在待做事项中（第 13、15 节）。

表 90　系统级权衡（“现状”为本系统的选择）

维度	备选	现状与理由
单会话 vs 多会话	单会话：无交接损失、总 token 更少（不必每个会话重读工作区说明与上游产物）	五会话：每个会话的工作集可控；角色约束清楚；中断只损失一个会话。代价是每个会话启动时重读说明与输入，估计占总输入 token 的 10%～15%
共享上下文 vs 文件交接	共享：下游理解上游意图	文件：复核独立；交接物可被人与评审检查；与最终读者的视角一致
同模型 vs 异模型复核	同模型（Opus）：改写质量可能更高	复核与评审用 Sonnet：成本低（复核会话 2.2～3.7 美元等值，生成侧单会话多在 3.7～7.3 美元），减轻自我偏好；复核受数字锁约束，只删不增，对生成能力要求较低
成本与时延	更少会话、更少检索、更低图表下限	一份研报约 64～68 分钟有效会话时间、24～27 美元等值、3,600 万～4,700 万输入 token；主要开销在分析师与排版（表 91）。可并行跑多家公司（1005 批次三家同时开始）

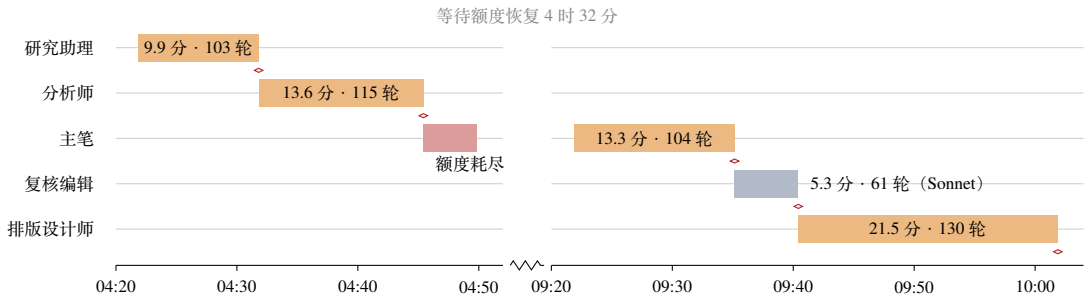


图 19　贵州茅台 1005 批次的会话时间线（真实记录）。橙色为 Opus 会话，蓝色为 Sonnet 复核会话，红色为因订阅额度耗尽而结束的会话；红色菱形为接力时通过的闸门。有效会话时间 67.9 分钟，墙钟时间 5 小时 40 分。

11.13　端到端运行实例：贵州茅台 1005 批次

最后给出一次完整运行的真实时间线。2026-10-05 04:21:52，贵州茅台、宁德时代、招商轮船三个工作区同时开始 `run`。图 19 是贵州茅台工作区六次会话的时间线（数据取自 `.pipeline/agent_runs.json`），表 91 汇总了五个工作区的用量。

阶段 1（04:21–04:31，研究助理）。 20 个问题、42 次检索、近 90 天来源 24 个；资料包的“一句话”是年报之后的半年报：2026 年上半年营收同比 +1.47%、归母净利润同比 −1.95%，二季度单季转负。研究助理在汇报里特别提示“预测首年要和这组数字衔接”，并给出了可直接用作 `price_source` 的股价来源写法（2026-09-30 收盘，两个来源交叉一致）。

阶段 2–7（04:31–04:45，分析师）。 财务抽取拒收 0、冲突 0、勾稽失败 0；框架列 5 个深度专题，写 5 份笔记，补充问题 12 个，来源增至 51 个；经营数据 18 张表、拒收 0 格；估值运行 `runs/20261005-044423-valuation`。

表 91　五个工作区的会话用量（`.pipeline/agent_runs.json` 全部记录求和；费用为 Claude Code 折算的 API 等值，非实际账单。中际旭创、宇树科技的记录包含 10-03 旧流程（写作会话兼做阶段 2～8、复核会话兼做排版）的会话，与 1005 批次不完全可比）

工作区	会话数	有效时间（分）	墙钟跨度	等值费用（美元）	输入 token（百万）	输出 token（千）	总轮数	页数	图表
贵州茅台 1005	6	67.9	5:40	27.19	43.85	356.7	544	25	83
宁德时代 1005	6	63.9	6:18	23.59	36.24	326.5	488	25	102
招商轮船 1005	7	64.7	11:10	27.42	47.20	322.6	613	30	70
中际旭创	10	144.0	17:01	42.09	83.79	424.4	825	30	68
宇树科技	11	201.6	22:00	49.42	106.60	489.0	1010	30	67

1005 批次按角色	研究助理	分析师	主笔	复核编辑	排版设计师	合计
有效时间（分，三家平均）	10.8	16.7	14.4	6.7	16.9	65.5
等值费用（美元，三家平均）	4.68	7.19	5.60	2.76	5.83	26.07

阶段 8（04:45 中断，09:21–09:35 续做，主笔）。 第一次主笔会话 31 轮后额度耗尽。额度恢复后同一条 `run` 重新起主笔会话，它从 `status` 得知阶段 1～7 已完成，接着写文稿。`guard` 起初报出一批外部数字缺登记：数字写在已有摘句里，但没有作为 `facts.value` 单独登记。主笔为其中 83 个数字补登记（没有新增来源、没有改动摘句），最终 `guard` 通过，正文 30,501 字、深度研究占 61%。

阶段 9（09:35–09:40，复核编辑，Sonnet）。 数字锁通过，删去 23 处数字，无新增数字与来源编号；正文 29,552 字，深度研究 60%；复核清单 14 条。

阶段 10–11（09:40–10:01，排版设计师）。 25 页、83 个图表（35 个用外部资料），渲染检查通过；交付说明列出核心结论（评级“中性”，DCF 基准每股价值 1,305.28 元，区间 996.91～1,566.83 元，相对 2026-09-30 收盘价 1,258.62 元空间 +3.7%，均取自程序结果）、需要仓库修改的三项 reportkit 需求、两处文档路径不符，以及最需要人工核对的复核事项。

图 20 把 1005 批次三家的有效时间按角色堆叠。分析师与排版设计师是两个最长的会话：前者要逐页读年报、跑校验循环、写四五份专题笔记；后者要写四五百行的排版脚本、反复渲染并逐页看图。复核编辑最短，与它“只删不增”的任务相符。三家的总时间非常接近（63.9～67.9 分钟），说明在五会话结构下，单家公司的成本主要由流程决定，受公司差异影响较小。

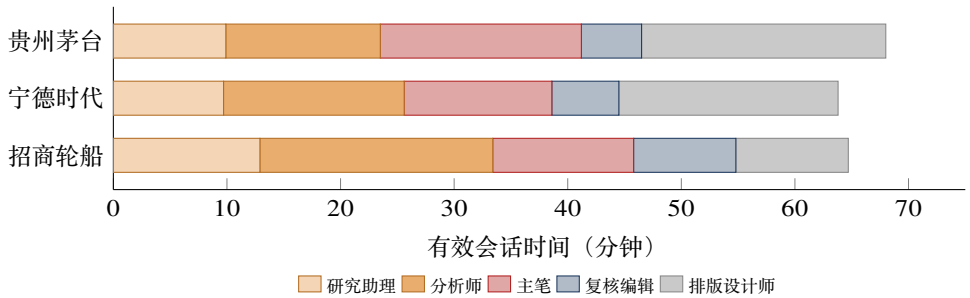


图 20　1005 批次三家公司各角色的有效会话时间（含因额度耗尽而中断的会话；招商轮船分析师与排版各含一次中断）。

11.14 小结

本节的系统把“写一篇深度研报”拆成五个互不共享上下文的模型会话，由确定性编排器按 11 道程序闸门接力，再由第六个会话在只含交付物的目录里盲评。几条设计在真实运行中被反复验证：

- **文件即状态**使流水线对中断天然鲁棒：7 次额度耗尽、2 次断线，全部用同一条命令续跑；
- **闸门只信文件、哈希与时间戳**，不信模型自述；“核对当前稿”和渲染凭证堵住了“先通过、再修改”的路径；
- **权限围栏**的错误会以“被拦即停、如实汇报”的方式暴露，比静默失败容易排查；
- **数量闸门是下限而不是目标**：模型会贴着阈值达标，质量必须交给看不到阈值的盲评；
- **独立复核**能发现程序检查不到的方向性错误，而数字锁保证它不会引入新的数字。

仍待改进的地方包括：多轮 `refine` 缺少真实运行记录；生成侧分支不回传版面问题；平台期停止条件在默认轮数下不生效；手写 skill 文档与闸门常量存在漂移；会话拆分的收益缺少消融实验。这些在第 15 节一并讨论。

12 评测体系与评审校准

前面几章讲的是怎样让系统写出“数字对得上”的研报：抽取逐字回查（第 6 节）、真值表唯一（第 7 节）、文稿过数字护栏与数字锁（第 10 节）、11 道闸门逐阶段把关（第 11 节）。这些机制回答的是“有没有错”。一份深度研报还要回答“好不好”：核心结论是否清楚，驱动拆解是否有定量链条，估值有没有交叉验证，风险是否落到假设上，文字读起来是否像分析师写的。本章说明 LEDGER 怎样把“好不好”变成可分解、可复算、可审计的分数，以及我们怎样检验给分的评审模型本身是否可信。

本章内容对应仓库中的 `finagent/evaluation/` 包（共约 1,800 行，不调用任何模型 API，评审会话也是无头 Claude Code）、`eval/rubrics/` 下的三版评分标准、`eval/calibration/` 下的校准数据，以及 `eval/results/` 下的历次质检结果。

12.1 问题与设计目标

研报质量为什么难评。 研报质量没有唯一的参考答案。同一家公司、同一份年报，两位合格的分析师可以给出方向相反的评级，各自都言之成理；因此不能像数据提取的字段核对或纠错核查的断言比对那样，拿金标准逐项比对（那两项的评测见第 13 节）。人工评审最可靠，但每份 25 ~ 32 页、60 个以上图表的研报，认真读一遍要一到两个小时，无法支撑多轮迭代。用大模型当评审（LLM-as-a-judge）可以降低成本，但已有研究反复报告其偏差：位置偏差与冗长偏好 [57, 73]、对自身或同族模型输出的偏好 [43]，以及打分随提示措辞漂移。对一个要靠评分驱动返工循环（refine loop）的系统而言，评审偏差会被放大：生成方会朝评审的偏好优化，而不是朝读者的需要优化（古德哈特效应，40）。

我们在开发中遇到的具体问题。 2026-10-03 第一版质检之前，团队对研报质量的判断只有两种来源：人工翻阅 PDF 后的总体印象，以及 11 道闸门是否通过。前者无法比较“这一轮比上一轮好在哪里”，后者只保证下限（字数、图表数、来源数），不保证分析质量。三家公司（贵州茅台、宁德时代、招商轮船）的早期研报全部通过闸门，但首轮质检只得到 46 ~ 49 分（表 104），低分集中在“预期差”“估值交叉验证”“风险落到假设”这类闸门管不到的地方。我们需要一套能指出“缺了什么”的评测，并且这套评测要能被校准。

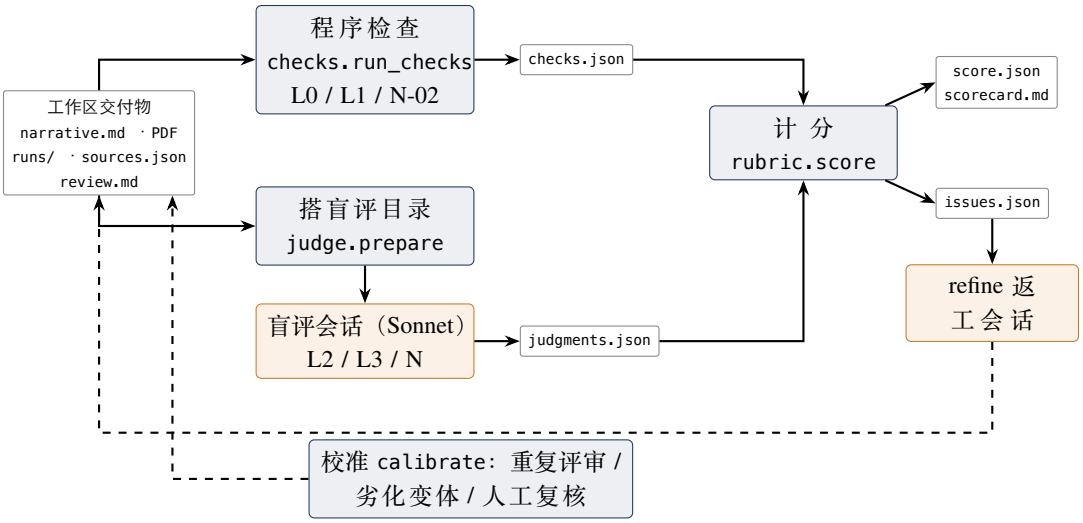


图 21 评测体系总体结构。蓝框为确定性程序，橙框为模型会话。程序检查与盲评互不依赖，可以分别重跑；计分只读两路判定文件，因此改权重不需要重跑评审。虚线为返工与校准两条旁路。

设计骨架的来源。 评测采用四段式骨架：*评分标准逐条二元判定 → 评审给出证据与理由 → 程序计分 → 校准评审本身*。这一骨架借鉴了团队成员此前接触的 UniPat 式评测实践；该实践没有可引用的公开文献，下文按工程实践描述。按设计记录（`docs/design/2026-10-03-multi-agent-eval.md`），我们只沿用通用骨架，不使用其中任何参考研报材料与拆解结论。学术上最接近的是以细粒度评分标准驱动评审的工作：Prometheus [31] 与 G-Eval [38] 让评审按标准逐项打分；HealthBench [2] 用医生撰写的逐条标准，并含负分条目。我们的不同之处在于：（1）一半以上的分值由程序直接读运行产物判定，模型只判程序判不了的分析与表达；（2）评审在物理隔离的目录里工作，看不到写作过程与分值；（3）评审本身用程序制造的已知缺陷来测灵敏度，思路与软件工程中的变异测试（mutation testing，14, 29）和 NLP 中的行为测试 [49] 一致。

设计目标（评测体系要满足的五条性质）

1. **可分解**：总分由逐条二元判定加权得到，每条附证据与理由；任何一分都能追到具体条目和原文。
2. **可复算**：计分全在 Python 中完成；改权重、改门槛只需 `score` 重算，不必重跑评审。
3. **能程序判的不交给模型**：数据闭合、估值算术、评级规则、数字前后矛盾由程序读产物判定。
4. **信息隔离**：评分标准对生成方不可见；评审看不到写作过程、模型名与分值；回传的问题单只含理由与证据。
5. **评审可校准**：用重复评审测稳定性，用程序劣化测灵敏度，用人工复核测精确率。

定位。 这套评测对应主选题**研究报告质量评估**：它既是一套可独立使用的研报质量评估方法（评分标准、盲评、程序计分与评审校准），也是流水线多轮返工的驱动信号。它评的是流水线产出的*完整交付物*：研报 PDF、文稿 `narrative.md`、估值运行产物、外部来源登记 `sources.json` 与人工复核清单 `review.md`。

图 21 给出评测体系的整体结构。左侧是工作区里的交付物；程序检查与盲评会话并行读取它们；两路判定在计分器汇合；计分结果一路生成给人看的逐条表，一路转成问题单回传给 `refine` 循环（第 11 节）；校准器在旁路上反复调用同一套评测，检验评审本身。

表 92　思路一：卖方深度报告各部分要回答的问题与 L2 条目的对应（据 `eval/rubrics/README.md`）。

报告部分	读者要的答案	条目	分值
首页 / 核心观点	评级、值多少、两三条理由；为什么值这个价	L2-01	4
市场关注问题	我们和市场哪里不同；股价隐含了什么	L2-02	4
公司概况	卖什么、卖给谁、靠什么赚钱、优势体现在哪个指标	L2-03	3
核心驱动	外部驱动 → 量价结构 → 收入利润，每环有数字	L2-04	5
业绩回顾	利润变动归因到科目并量化	L2-05	4
行业	供需、周期、竞争格局的数据与出处	L2-06	4
盈利预测	关键假设逐项有取值与依据	L2-07	5
估值（方法）	方法为何适用、参数从何而来	L2-08	3
估值（交叉验证）	至少一种独立方法验证并解释差异	L2-09	4
估值（情景）	情景假设差异、价值对什么最敏感	L2-10	3
风险提示	风险落到具体假设，给出影响	L2-11	4
跟踪	可验证的指标、催化剂、失效条件	L2-12	2

12.2　评分标准的来源与四层结构

12.2.1　两条思路

评分标准（rubric）由两条互补的思路合成（`eval/rubrics/README.md`）。

思路一：卖方深度报告在回答哪些问题。 按卖方深度报告的常规体例，逐部分问“读者要从这一部分得到什么答案”，得到一份行业无关的问题清单。设计记录原计划从 10 ~ 15 篇公开研报中按段落标注、聚类出题库；2026-10-03 拍板时改为先用模型知识拆解体例（样本研报的版权与采集成本是主要顾虑），等优秀研报样例到位后再用样本校正。问题清单与条目的对应关系见表 92。权重按“对结论的影响”分三档：5 分是决定估值的环节（驱动拆解、预测假设），4 分是支撑判断的关键论证，2 ~ 3 分是完整性要求。

L3 来自同一份拆解里“读者怎么读”的要求：主线连贯、事实 / 推论 / 观点可分（对应竞赛硬约束“区分事实、推论与观点”）、结论先行、没有机器腔、数字不过密、图表服务论点、假设与局限透明。

思路二：工作流每次交接的闭合点。 把流水线

$$\text{年报} \to \text{抽取} \to \mathcal{T} \to \text{经营数据} \to \text{预测} \to \text{三表} \to \text{DCF} \to \mathcal{N} \to \text{排版}$$

的每一次交接都看成一个闭合点：上游的数在下游必须找得到、对得上、口径一致。这类条件几乎都能由程序判定，现有引擎的校验结果可以直接复用（表 93）。

12.2.2　形式化定义

定义 12.1 (评分标准)**.** 评分标准是一个五元组 $R = (\mathcal{T}, c_{\text{gate}}, \underline{d}, \Theta, \text{ver})$，其中 $\mathcal{T}$ 为条目集合；c_{gate} 为硬闸门失败时的总分上限（*gate_cap*，取 *40*）；$\underline{d}$ 为负向扣分合计的下限（*negative_floor*，取 -20）；$\Theta = (\tau, \{\rho_\ell\}, \alpha)$ 为交付标准：总分门槛 $\tau = 75$、各层得分率门槛 $\rho_{L1} = 0.8$、$\rho_{L2} = \rho_{L3} = 0.6$、是否允许命中负向条目 $\alpha = \mathit{false}$；ver 为版本号。每个条目

$$i = (\text{id}_i,\ \ell_i,\ \text{title}_i,\ j_i,\ p_i,\ \kappa_i,\ \theta_i,\ \text{crit}_i,\ \text{fe}_i,\ c_i,\ \text{route}_i,\ \text{stage}_i),$$

层级 $\ell_i \in \{L0, L1, L2, L3, N\}$，判定者 $j_i \in \{\mathit{program}, \mathit{llm}\}$，分值 p_i（*L0* 为 *0*，*N* 为负），κ_i 为程序检查名、θ_i 为其阈值参数，crit_i 为判定标准，fe_i 为反例，c_i 为可选的命中封顶

表 93　思路二：工作流交接与 L0 / L1 条目的对应。

交接	条目
年报 → 真值表	L1-01 抽取校验
经营数据 → 预测	L1-02 分部预测基期闭合；L1-08 预测与历史衔接
预测 → 三表 → DCF	L0-02 三表勾稽；L0-03 DCF 与三表同口径
DCF → 股权价值 → 每股价值 → 评级	L1-03 股权价值桥；L1-04 每股价值；L1-05 情景排序；L1-06 敏感性方向；L1-07 终值占比；L0-05 评级
产物 → 正文	L0-01 数字护栏；L1-09 正文估值与产物一致
外部信息 → 正文	L1-10 外部信息可追溯；L1-11 股价日期与来源
初稿 → 复核稿	L1-12 数字锁；L1-13 人工复核清单

表 94　评分标准 `report_v2` 的四层结构与负向条目。

层	名称	条目数	分值	判定者与规则
L0	硬闸门	5	0	程序；任一失败，总分封顶 40
L1	数据闭合	13	35	程序
L2	分析质量	12	45	盲评模型
L3	表达与合规	7	20	盲评模型
N	负向条目	4	−30	N-02 程序，其余盲评；命中扣分，合计下限 −20
合计		41	100	程序 35 分（+ L0 封顶 + N-02），模型 65 分

（目前只有 *N-01* 设为 *70*），$\text{route}_i \in \{generator, reviewer\}$ 与 stage_i 指明问题回传给哪个会话、哪个阶段。

`rubric.load_rubric` 在读入时检查四条结构不变量，违反即抛错，避免一份写错的标准悄悄改变分数：

不变量 24 (评分标准的结构约束)**.** （*1*）条目 *id* 唯一；（*2*）$\ell_i \in \{L0, L1, L2, L3, N\}$；（*3*）$j_i = program \Rightarrow \kappa_i$ 非空；（*4*）$\ell_i = N \Rightarrow p_i \le 0$。此外 *`tests/test_evaluation.py`* 断言当前默认版本各层合计 $\sum_{L1} p_i = 35$、$\sum_{L2} p_i = 45$、$\sum_{L3} p_i = 20$、$\sum_{L0} p_i = 0$，且每个程序条目的检查名都在 *`checks.CHECKS`* 中注册。

12.2.3　四层结构

表 94 汇总默认版本 `report_v2` 的层级、条目数与分值。正向满分为 100：程序判定 35 分，模型判定 65 分。负向条目名义合计 −30，但扣分合计不低于 −20。全部 41 条的判定标准与反例见附录 A。

L0 不计分而封顶，是因为这五项（数字护栏、三表勾稽、DCF 与三表同口径、免责声明、评级与估值一致）任何一项失败，报告就不能交给读者，再好的分析也不应把它抬到及格线以上。负向条目（N）专管“最伤信誉”的四类错误：无来源的外部数据（N-01）、数字前后矛盾（N-02）、聚合数构成描述错误（N-03）、结论前后冲突（N-04）。N-01 命中时额外封顶 70，因为编造外部数据是比遗漏论证更严重的问题：一份带编造数字的报告即使其余全部满分，也不应高于 70。

权衡（Trade-off）：二元判定还是量表打分

备选：每条用 1 ~ 5 分的量表（Likert）打分，或让评审直接给一个 0 ~ 100 的总分。**为什么不选**：量表分数的中段（3 分）语义模糊，同一评审对同一份报告两次打分常在相邻档位间跳动；总分则完全无法追溯到原因，也无法转成问题单。二元判定配合“拿不准判不通过”的规则，把模糊性压到每条的判定标准里，措辞改动的效果可以直接在校准数据中观察（§12.6 的 L3-04 即为一例）。**代价**：二元判定丢失程度信息（“勉强达标”与“写得很好”同分）；我们用分值档位（2 ~ 5 分）部分补偿条目间的重要性差异，用 `reason` 字段保留程度描述供人阅读。

12.3 程序检查：L0 硬闸门与 L1 数据闭合

12.3.1 评测对象与检查接口

`rubric.Bundle` 把一份待评研报抽象为三元组 (company, run, work)：`run` 是估值运行目录 `runs/<id>/`，`work` 是抽取目录 `data/extractions/< 公司 >_< 年度 >/`。文稿、初稿、复核清单、外部来源与交付 PDF 都由属性派生（交付 PDF 优先取设了 `output_name` 的命名版）。从流水线工作区构造 Bundle 时（`bundle_from_workspace`），运行目录取 `status()` 认定的最新成功运行，因此评测对象与闸门判定的是同一份产物。

每个程序检查是一个函数 κ ：(Bundle, θ) $\mapsto$ (passed, detail, evidence)，通过模块内的命名约定 `check_< 名 >` 自动注册进 `CHECKS` 字典。`run_checks` 只运行 j_i = program 的条目，并遵守两条约定：

1. **缺输入即未通过**。输入文件缺失时返回 `passed=false`，`detail` 写明缺哪个文件，不抛错。
2. **异常即未通过**。检查函数抛出任何异常（产物格式异常、字段缺失），都被捕获并记为未通过，`detail` 记异常类型与信息。一条检查出错不拖垮整份评测；测试用一个空目录构造 Bundle，断言全部程序条目都返回结果且全部未通过。

程序检查只读产物，不写任何文件，不调用模型；同一份产物重复运行结果逐字相同。

12.3.2 逐项判定规则

表 95 列出全部 19 项程序检查（L0 五项、L1 十三项、N-02 一项）的输入与判定规则。下面对其中需要说明口径的几项给出形式化表述。记 EV_s、Eq_s、v_s 为情景 s 的企业价值、股权价值（百万元）与每股价值（元），A、B 为股权价值桥的加回项与扣减项合计，Q 为总股本（股）。

股权价值桥与每股价值（L1-03、L1-04）。 股权价值桥由第 8 节的估值引擎逐项列示。检查不重算 DCF，只核对“列示的数能否加起来”：

$$\forall s \in \{\text{base}, \text{downside}, \text{upside}\}:\quad |\mathrm{EV}_s + (A - B) - \mathrm{Eq}_s| \le 0.5, \qquad |\hat{v}_s - v_s| \le \max(0.01,\ 10^{-4}|\hat{v}_s|), \quad \hat{v}_s = \mathrm{Eq}_s \cdot 10^6 / Q. \tag{60}$$

容差 0.5 百万元用于吸收产物中百万元金额的舍入；每股价值的容差取 1 分钱或万分之一相对误差中较大者。

评级规则复算（L0-05）。 评级规则以文字形式写在产物里（例如“买入：≥20%；增持：≥5%；中性：≥−5%；减持：≥−20%；卖出：其余”）。检查用正则把规则解析成有序阈值表 $\{(r_k, t_k)\}$，按顺序取第一个满足 $u \ge t_k$ 的评级 r_k，都不满足时取“其余”对应的评级，其中 u 为基准每股价值相对参考股价的上行空间。判定为

$$\text{L0-05 通过} \iff r^\star(u) = r_{\text{产物}} \ \wedge\ r_{\text{产物}} \in \mathcal{N}.$$

表 95 程序检查一览（`finagent/evaluation/checks.py`）。“产物”均在估值运行目录 `runs/<id>/` 下，除非另注。

条目	检查函数	输入与判定规则
L0-01	`guard`	对当前文稿重跑数字护栏 `run_guard`（带 `sources.json`）：无出处数字、指标期间不符、外部数字缺引用三类均为 0。前后矛盾不在此计（由 N-02 单独扣分）
L0-02	`ts_integrity`	`valuation/three_statement.json`：每项勾稽的最大绝对误差 ≤ 引擎容差 `ts_engine.TOLERANCE`（按误差值重判，不信产物记录的 status），且无硬性失败、勾稽列表非空
L0-03	`dcf_tie`	`valuation.json` 的 `three_statement_tie.status = pass`（收入、EBIT、折旧摊销、资本开支、营运资本逐期取自三表）
L0-04	`disclaimer`	`report.html` 或文稿中匹配正则“不构成.{0,4} 投资建议”
L0-05	`rating`	按 `forecast_summary.rating.rule` 由上行空间复算评级，与产物评级相同，且正文出现该评级词
L1-01	`extraction`	`facts.verified.json` 的 `check.ok` 为真，拒收、冲突、勾稽失败均为 0
L1-02	`segment_base`	`segment_forecast.json` 基期：$\lvert\sum_k \text{rev}_k + \text{other} - \text{rev}\rvert \le \max(0.5,\ 10^{-6}\lvert\text{rev}\rvert)$ 百万元，且 rev > 0
L1-03	`bridge`	每个情景 $\lvert \text{EV}_s + (A - B) - \text{Eq}_s \rvert \le 0.5$ 百万元，且情景非空
L1-04	`per_share`	每个情景 $\lvert \text{Eq}_s \cdot 10^6/Q - v_s \rvert \le \max(0.01,\ 10^{-4}\lvert\text{Eq}_s \cdot 10^6/Q\rvert)$ 元
L1-05	`scenario_order`	$v_{\text{low}} \le v_{\text{base}} \le v_{\text{high}}$
L1-06	`sensitivity`	`sensitivity_directionality.passed`：WACC 上升价值下降、永续增长上升价值上升，逐项
L1-07	`terminal_share`	基准情景终值占企业价值 ≤ $\theta_{\max}$（默认 0.8）
L1-08	`forecast_continuity`	最近实际年 a 与首个预测年 e：$\lvert \text{GM}_e - \text{GM}_a \rvert \le 5$ 个百分点，且 $\lvert g_e - g_a \rvert \le 30$ 个百分点
L1-09	`text_valuation`	基准每股价值按 2/1/0 位小数、带或不带千分位六种写法之一出现在正文（后接非数字），且正文出现评级词
L1-10	`sources`	`verify_sources` 结构校验通过；正文至少一处 `[S 编号]`；`check_citations` 无问题（引用编号都已登记、外部数字所在句带对应编号）
L1-11	`price`	`forecast_summary.market` 中价格、日期、来源三项齐全
L1-12	`numlock`	初稿与复核稿都在，`numlock(draft, narrative).ok`：复核稿数字与来源编号是初稿的子集
L1-13	`review_list`	`review.md` 至少一条填写完整的事项（表格行非空格数 ≥ min(4, 列数)，或一条长度足够的列表项），或全文只写“无”
N-02	`consistency`	对文稿运行 `report_check.consistency.check`：同一比较键下两处数值超出展示精度，或评级前后不一，即命中（见算法 18）

解析规则文本而不是硬编码阈值，是为了让评测与引擎共用同一份规则：引擎改阈值时评测自动跟随，不会出现两份规则不一致。

正文估值与产物一致（L1-09）。 文稿中的基准每股价值必须与产物一致，但正文的写法可以是“1,531.40 元”“1531.4 元”或“1531 元”。检查构造六种展示形式 $\{\mathrm{fmt}(v_{\text{base}}, d, \text{sep}) : d \in \{2,1,0\},\ \text{sep} \in \{\text{千分位}, \text{无}\}\}$，任一形式出现在正文且其后不紧跟数字，即视为写出。负向前瞻防止“1531”误匹配“15310”。

外部信息可追溯（L1-10）。 v0 版只检查 `sources.json` 是否存在且结构合法。人工质检指出这会造成“空有其 JSON 而未落实”（§12.6），v1 起改为四个条件同时成立：（a）`verify_sources` 结构校验通过（每个来源有网址、标题、发布方、访问日期，摘句含所登记的数字）；（b）正文至少一处 `[S 编号]`；（c）正文引用的编号都已登记；（d）只能在外部来源中找到出处的数字，其所在句子带对应编号（`check_citations`，复用数字护栏的 `truth_values` 区分“年报或产物里有”与“只有外部来源有”）。测试用三个反例逐一验证：外部数字没标编号、引用了未登记的 `[S9]`、正文没有任何引用，三者都判不通过。

人工复核清单（L1-13）。 同样是 v1 的修订。复核清单的目的是把“拿不准但影响结论”的事项交给人，只有表头的清单没有信息量。检查解析 `review.md`：去掉标题行与表格分隔行，表头之后每一行至少有 $\min(4, \text{列数})$ 格非空且非“—”才算一条完整事项；或者是一条长度超过 8 个字符的列表项；或者全文只写了“无”。测试覆盖了只有表头、事项未填完、填写完整、写“无”四种情况。

预测与历史衔接（L1-08）。 这是 L1 中唯一的“合理性”检查，阈值放在条目参数里（`gross_margin_pp=5`，`growth_pp=30`）。首个预测年毛利率与最近实际年相差超过 5 个百分点，或收入增速变化超过 30 个百分点，通常意味着预测没有与年报之后的最新业绩衔接。它是启发式规则，会有合理的例外：中际旭创 2025A→2026E 收入增速变化 71 个百分点（行业放量期），该条在全部校准运行中都未通过。我们保留这一失败而不放宽阈值，因为 2 分的扣减不影响交付判定，而“为什么增速跳变”恰好是读者会追问、报告应当解释的问题。

12.3.3 N-02：数字前后矛盾的程序判定

v2 把负向条目 N-02 从评审模型判定改为程序判定，复用研报纠错核查中的内部一致性检查 `finagent/tasks/report_check/consistency.py`（方法参照原插件的重复数字对账规则，见附录 F）。该检查的设计原则是*误报控制优先*：任何一项对不上号就不进入比较，所以一旦命中，就是已确认的内部矛盾。

定义 12.2 (比较键与提及)**.** *文稿按句（以“。;”与换行断句，跳过表格、代码块、注释与图表指令行）切分。每个数值提及 m 归入比较键 $k(m) = (\text{指标}, \text{期间}, \text{类型}, \text{情景})$，其中指标取自营业收入、归母净利润、扣非归母净利润、净利润、每股收益、毛利率、每股价值，期间为晚于报告期的预测年度（每股价值无期间），类型为水平值或同比，情景为悲观 / 基准 / 乐观之一。提及的数值按单位换算为基本单位 $x(m)$，并带舍入容差 $\epsilon(m) = \frac{1}{2}\,\mathrm{ulp}(m)$（按原文展示的小数位计算的半个末位）。*

命题 12.1 (命中即矛盾)**.** *若算法 18 对键 k 报告提及 m_1, m_j 矛盾，则不存在一个真实数值 $x^\star$ 使得 m_1 与 m_j 都是 $x^\star$ 按各自展示精度舍入后的写法。*

证明. 若 m_1, m_j 都是 $x^\star$ 的舍入写法，则 $|x(m_1) - x^\star| \le \epsilon(m_1)$ 且 $|x(m_j) - x^\star| \le \epsilon(m_j)$，由三角不等式 $|x(m_1) - x(m_j)| \le \epsilon(m_1) + \epsilon(m_j)$，与报告条件矛盾。 □

```
<blind>/
  CLAUDE.md          # reviewer guide
  rubric.json        # id/title/criterion/
                     # fail_examples/negative
  deliverable/
    report.pdf
    narrative.md
    sources.json     # optional
    review.md        # optional
    model/
      valuation.json
      forecast_summary.json
      segment_forecast.json
      scenarios.json
      non_operating_assets.json
      comps.json
  judgments.json     # written by judge
  judge.jsonl        # event stream
```

看得到	研报 PDF、文稿、6 个估值产物、外部来源登记、复核清单；评审条目的 id、标题、判定标准、反例、是否负向
看不到	条目分值与层级权重、交付门槛、程序检查结果、写作与复核会话的事件流、初稿与改写日志、生成模型名、其他公司的研报
能做	`Read(./**)`、`Glob`、`Grep`、`Write(./judgments.json)`
不能做	联网、执行命令、读目录外文件、写其他文件

图 22　盲评目录（左）与评审会话的可见范围（右）。`model/` 只复制估值目录下存在的文件；`rubric.json` 由 `judge_items` 生成，剥去了 `points` 与 `layer`。

命题 12.1 只保证数值层面的“不是舍入差异”。语义层面的误报（两处其实在说不同口径）由前面的过滤条件控制：外部数字、条件句、多情景同句、带分部限定词的提及一律不比。代价是召回下降：被过滤掉的句子里真有矛盾也查不出来。按评分标准的变更记录，七份真实文稿上零误报，摘要评级反转全部命中。数字之外的结论冲突（“毛利率改善”对“毛利率承压”）程序无法可靠判定，拆为 N-04 留给评审。

失败案例：v0 / v1 中由评审判定的 N-02

评审模型判 N-02 时，会把“股价 808.44 元低于乐观情景 918.06 元，却说股价隐含的假设高于乐观情景”这类推理冲突、以及“封面总市值 8,983 亿元与第 15 节 9,522.69 亿元并存”这类口径差异都算作前后矛盾（中际旭创 clean1 的评审理由）。这些判断有一定道理，但同一份宇树科技原稿三次评审中 N-02 两次命中、一次未命中（§12.7.2），单这一条就造成 8 分的总分波动。数字是否对得上是确定性问题，v2 因此改由程序判定。

12.4　盲评会话

L2、L3 与除 N-02 外的负向条目共 22 条（v2 中为 L2 十二条、L3 七条、N-01 / N-03 / N-04 三条），由一个独立的无头 Claude Code 会话判定，默认模型为 `FINAGENT_QC_MODEL`（Claude Sonnet 5.5）。这个会话被称为“盲评”，因为它在一个只含交付物的目录里工作，看不到写作过程、生成模型名与分值。

12.4.1　盲评目录

`judge.prepare` 为每次评审搭一个新目录（默认 `~/finagent-evals/<公司>_<时间>/`，可由 `FINAGENT_EVAL_DIR` 指定），结构如图 22。

目录位置有硬约束：Claude Code 会沿父目录加载 `CLAUDE.md`，若盲评目录建在仓库内，评审会读到仓库的开发指南、评分标准原文乃至其他公司的研报。`prepare` 因此复用编排器的 `_check_location`：目录不得位于仓库的上级目录之下，也不得有任何（家目录之外的）上级目录含 `CLAUDE.md`；测试断言在仓库 `eval/results/` 下建盲评目录会被拒绝。

12.4.2 评审指南与判定规则

盲评目录的 CLAUDE.md（judge.GUIDE）把评审设定为“研究所的质控评审，没有参与写作，也不知道是谁写的”，读者是买方基金经理。指南的要点如下（全文结构见附录 E）：

1. **不复算。**数字与年报是否一致、三表是否勾稽、估值算术是否正确，已由程序核对；评审只判断分析是否成立、论证是否充分、表达是否合格。这一句把评审的注意力从“找算术错误”移开，避免它在自己不擅长的事上花费回合、给出不可靠结论。
2. **逐条、二元。** rubric.json 的每一条都要判定；完全满足判定标准才算通过；**拿不准时判不通过**，并在理由里写明缺什么。
3. **负向语义。** negative: true 的条目，passed: true 表示命中（报告犯了这个错），没有发现判 false。
4. **证据可定位。** evidence 摘录原文不超过 80 字并标位置（章节名或 PDF 页码），没有可摘的写“无”；reason 一两句话，不通过时写出读者需要而报告没给的东西。
5. **分批读 PDF。**每次不超过 20 页（与生成侧读年报的限制相同）。

“拿不准判不通过”是有意的不对称：在返工循环里，误判“不通过”的代价是一轮多余的修改（复核编辑会在理由中看到具体缺口，可以判断是否需要改），误判“通过”的代价是问题被带进交付物。人工复核的结果（§12.7.4）显示这一规则确实带来了一部分偏严的误报，主要集中在文风类条目。

12.4.3 会话命令与输出

评审命令为

```
claude -p "<PROMPT>" --model <QC_MODEL> --permission-mode acceptEdits \
  --allowedTools "Read(./**)" Glob Grep "Write(./judgments.json)" \
  --output-format stream-json --verbose
```

其中 PROMPT 只有一句：按本目录 CLAUDE.md 评审研报，把结果写进 judgments.json 后停止。事件流逐行落盘为 judge.jsonl；结束后从 result 事件取回合数、费用与 token 用量，连同退出码与耗时写进 judge_run.json。输出格式为：

```
{"items": [{"id": "L2-01", "passed": true,
            "evidence": "...(summary page)", "reason": "..."}],
 "summary": "3-5 sentences: strongest and weakest points"}
```

输出校验。 judge.read_judgments 不信任评审的输出格式，逐项检查：非评审条目（例如评审自作主张判了 L1-01）记为“多余条目”；同一 id 出现两次，只取第一次并记“重复条目”；passed 不是布尔值（例如字符串 "yes"）记为格式问题且不计入；缺判的条目记“缺条目”。这些问题写进 score.json 的 judge_problems，不进计分。更严重的两种情况单独处理：评审会话跑了但没有写出 judgments.json（例如额度用尽、第一轮就退出），rescore 不会静默地把 22 条全部按未通过计分，而是标注“本次总分无效”；会话非零退出时标注“判定可能不完整”。校准运行据此把这类结果整目录删除以便重跑（calibrate.run）。

成本。 eval/results/ 下 7 次正式评审单次耗时 149 ~ 254 秒、11 ~ 17 个回合、用量折合 0.95 ~ 2.35 美元；校准中的 20 次评审平均 177 秒、17 个回合、1.75 美元，合计 35.02 美元（judge_run.json 统计）。输入 token 每次约 65 ~ 165 万，主要是 PDF 与文稿的重复读取（多数命中缓存）。与生成一份研报的五个会话相比，评审成本小一个数量级，因此可以负担重复评审与劣化变体评审。

权衡（Trade-off）：评审读 PDF 还是只读文稿

备选：只给评审 `narrative.md`（纯文本，便宜且好检索）。为什么不选：L3-06“图表服务论点”、L3-07“假设与局限透明”依赖排版后的图表与附录；读者看到的是 PDF，例如两次劣化实验中评审都发现“跟踪指标与结论失效条件”一节在 PDF 第 26 页“只有标题没有内容”，这只有看 PDF 才能发现。折中：两者都给。文稿用于检索原文与摘证据，PDF 用于判断版面与图表；指南要求分批读 PDF 以控制上下文。

权衡（Trade-off）：让评审看到程序检查结果吗

备选：把 `checks.json` 放进盲评目录，让评审知道哪些数据闭合已通过。为什么不选：程序结果会锚定评审的总体印象（看到 L1 大面积通过后对 L2 更宽容，反之亦然），两路判定就不再独立；而独立性正是我们把程序判与模型判分开的理由。评审只被告知“这些已由程序另行核对”，看不到结果。

12.5 计分、交付标准与问题单

12.5.1 计分公式

记条目 i 的判定结果为 $y_i \in \{0,1\}$：正向条目 $y_i = 1$ 表示通过，负向条目 $y_i = 1$ 表示命中。程序检查结果与评审结果合并为一张判定表 $\{(\mathrm{id}_i, y_i, \mathrm{evidence}_i, \mathrm{reason}_i)\}$ 后，`rubric.score` 按下式计分：

$$E_\ell = \sum_{i \in \mathcal{T}_\ell} p_i\, y_i, \qquad P_\ell = \sum_{i \in \mathcal{T}_\ell} p_i, \qquad \ell \in \{L1, L2, L3\}, \tag{61}$$

$$D = \max\Big(\sum_{i \in \mathcal{T}_N} p_i\, y_i,\ \underline{d} \Big), \tag{62}$$

$$S_{\mathrm{raw}} = \max(0,\ E_{L1} + E_{L2} + E_{L3} + D), \tag{63}$$

$$\Gamma = \{c_{\mathrm{gate}} \ :\ \exists i \in \mathcal{T}_{L0},\ y_i = 0\} \ \cup\ \{c_i \ :\ i \in \mathcal{T}_N,\ y_i = 1,\ c_i \text{ 有定义}\}, \tag{64}$$

$$S = \min(\{S_{\mathrm{raw}}\} \cup \Gamma). \tag{65}$$

式中 $\mathcal{T}_\ell$ 为第 ℓ 层条目，p_i 为分值（N 层为负），$\underline{d} = -20$ 为扣分下限，Γ 为封顶集合，$c_{\mathrm{gate}} = 40$。缺判定的条目按 $y_i = 0$ 处理（正向条目不得分、负向条目不扣分），并列入 `missing`。

交付标准。

$$\begin{aligned} \text{Deliverable} \iff\ & (\forall i \in \mathcal{T}_{L0} : y_i = 1)\ \wedge\ S \geq \tau \\ & \wedge\ (\forall \ell : P_\ell > 0 \Rightarrow E_\ell / P_\ell \geq \rho_\ell)\ \wedge\ (\alpha\ \vee\ \forall i \in \mathcal{T}_N : y_i = 0). \end{aligned} \tag{66}$$

`score` 返回的 `not_deliverable_reasons` 逐条列出不满足的子条件，例如“L2 得分率 58% < 60%”“命中负向条目：N-02”。交付标准与总分是两件事：一份 85 分的报告只要命中任一负向条目就不可交付（中际旭创校准原稿即如此），而一份 76 分、各层达标、无负向命中的报告可以交付。

命题 12.2 (单调性)**.** *固定其余条目的判定，S 关于任一正向条目的 y_i 单调不减，关于任一负向条目的 y_i 单调不增；交付判定同理。*

证明. 正向条目 y_i 由 0 变 1：若 $i \in L1 \cup L2 \cup L3$，S_{raw} 不减，Γ 不变；若 $i \in L0$，S_{raw} 不变，Γ 可能失去 c_{gate}，最小值不减。负向条目 y_i 由 0 变 1：D 不增（下限截断不改变方向），S_{raw} 不增，Γ 可能增加 c_i，最小值不增。交付判定的四个合取项各自在相同方向上单调。 □

单调性保证返工循环中“修好一条问题”在其余判定不变时不会让分数下降；实际中分数仍可能下降，原因是评审对其他条目的判定发生了变化（§12.7.2 的重复评审噪声），而不是计分规则本身。

例 12.1（一份原稿与一份劣化稿的计分）. 中际旭创校准原稿第一次评审（*rubric v1*）：*L1* 得 *33/35*（*L1-08* 预测衔接未过），*L2* 得 *42/45*（*L2-08* 估值参数依据未过），*L3* 得 *18/20*（*L3-05* 数字密度未过），*N-02* 命中扣 *8* 分。$S_{\text{raw}} = 33 + 42 + 18 - 8 = 85$，无封顶，$S = 85$；因命中负向条目，判为不可交付。同一份稿件插入一段无来源的市场规模数字后（劣化 `unsourced_number`），评审判 *N-01* 命中（-8，封顶 *70*），同时数字护栏 *L0-01* 因“外部数字缺引用”失败（封顶 *40*），$S_{\text{raw}} = 85$，$\Gamma = \{70, 40\}$，$S = 40$。这一例子同时说明了程序判与模型判的叠加：编造的外部数字被两条独立的检查各抓住一次。

重算不重评。 `python -m finagent.evaluation score --result <目录>` 读结果目录中已保存的 `checks.json` 与 `judgments.json`，按（可能修改过的）评分标准重新计分，生成新的 `score.json`、`scorecard.md` 与问题单，不重跑程序检查也不重跑评审。因此改权重、改封顶、改交付门槛的效果可以立即在全部历史结果上看到。只有改了判定标准文本（`criterion`、`fail_examples`）或程序检查本身时，才需要重跑 `run`。旧版结果按新版重算时，新增条目（v2 的 N-02 程序判、N-04）没有判定，按缺判（未命中）计。

权衡（Trade-off）：加权求和加封顶，还是字典序或乘法

备选 A：字典序（先比 L0，再比 L1，……）。它能表达“闸门优先”，但无法比较“L2 多 5 分、L3 少 2 分”的两份报告，不利于返工循环判断进步。**备选 B**：乘法（如 $S = \prod_{\ell}(E_{\ell}/P_{\ell})^{w_{\ell}}$）。任何一层为 0 时总分为 0，惩罚过重，且分数难以解释。**选择**：加权求和表达“分析质量可以互相补偿”，封顶表达“硬错误不可补偿”，交付标准的层得分率门槛表达“每层都要过得去”。三种机制各管一件事，读分数的人能直接看出总分低是因为封顶、扣分还是某层得分率不足。

12.5.2 问题单

计分结果除了给人看的逐条表 `scorecard.md`，还转成给返工会话看的问题单 `issues.json`（`__main__.issues_from`）。每条未通过的正向条目、每条命中的负向条目生成一个问题：

$$\text{issue} = (\text{id} = \text{I}nn,\ \text{route}_i,\ \text{stage}_i,\ \text{severity},\ \text{description} = \text{reason}_i,\ \text{evidence}_i),$$

严重度按层级与分值确定：

$$\text{severity}(i) = \begin{cases} \text{blocker}, & \ell_i \in \{L0, N\}, \\ \text{major}, & p_i \geq 4, \\ \text{minor}, & \text{其余}. \end{cases} \tag{67}$$

问题单只含理由与证据，不含条目标题、判定标准与分值；问题编号 I01、I02……按出现顺序重编，与条目 id 无对应。测试 `test_evaluate_without_judge_and_rescore` 断言问题单的字段集合恰为 $\{\text{id}, \text{route}, \text{stage}, \text{severity}, \text{description}, \text{evidence}\}$，且没有任何一条 `description` 等于某个条目标题。

路由。 `refine` 把问题单按 `route` 与 `stage` 分成三份（第 11 节）：`generator` 路由的问题（数据、假设、分析内容，阶段 1～8）交给修订会话（Opus），修订后复核与排版从头再来；`reviewer` 路由且阶段为 10 的问题（图表、免责声明等版面问题）交给排版设计师；其余 `reviewer` 问题（措辞、结构）交给复核编辑（Sonnet），数字锁照常生效。路由信息来自

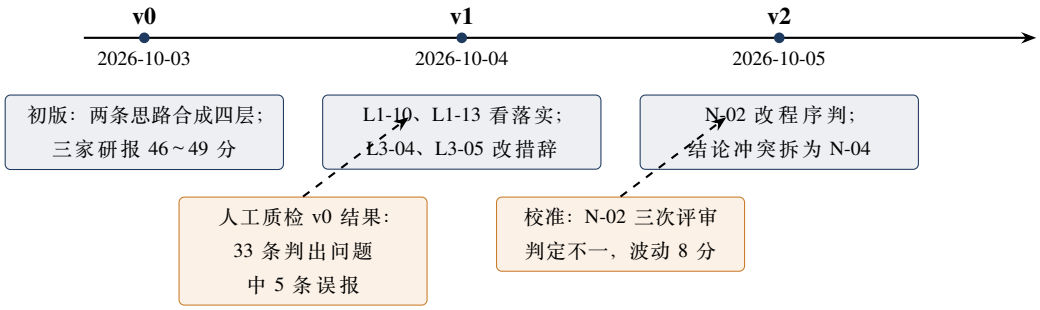


图 23　评分标准的版本演进。蓝框为版本内容，橙框为触发修订的证据。

评分标准条目本身的 `route`、`stage` 字段（附录 A），评测代码不做语义判断。返工循环在“达到交付标准”“满 3 轮”“连续两轮提升不足 2 分”“没有可回传的问题”四种情况下停止，并回滚到得分最高的一轮；仍未达标的问题追加进 `review.md` 交给人工。

失败案例：问题单的理由里带着标准的影子

问题单不含判定标准原文，但评审的 `reason` 往往复述了标准的要点，例如“只有无风险利率和 0.5% 公司特有风险溢价有依据……为什么选 FCFF-DCF 同样没讲”几乎就是 L2-08 判定标准的改写。多轮返工后，生成方确实可能从理由中拼出评分标准的轮廓。我们接受这一泄漏：理由必须具体到“缺什么”才能指导修改，而具体的缺口本身就是读者需要的东西。防止应试的更强手段（把标准分为反馈集与留出集、留出集只用于最终计分）写在设计记录里，尚未实现。

12.6　评分标准的版本演进

评分标准在三天内迭代了三版（图 23）。每一版都保留在 `eval/rubrics/` 下，用于复现当时的结果；版本号写进每一份 `score.json`，人工质检页拒绝为版本不一致的结果生成页面。

v0（2026-10-03）：初版。 按 §12.2 的两条思路写成 40 条（L0 五条、L1 十三条、L2 十二条、L3 七条、N 三条），N-01 至 N-03 全部由评审判定。用它对三家公司的早期研报各评一遍（单次盲评），得到茅台 48、宁德 49、招商 46 分（表 104）。

人工质检。 三家结果交给两位团队成员复核（复核页的生成见 §12.9）。复核只针对评审判“不通过”的条目，确认或否定问题是否成立，否定时必须写理由与可核对的证据。33 条中确认 26 条、误报 5 条、待查 2 条（详见 §12.7.4）。误报集中在 L3-04（无机器腔，两家都认为过严）、L3-05（数字密度，茅台判为误报）；茅台复核人的总评还指出了一个评审之外的问题：L1 若干条只检查 JSON 文件在不在，没检查“是否真的落实了这件事”。

v1（2026-10-04）：按人工质检收紧 L1、放宽 L3 的措辞。 改动见表 96。L1-10 与 L1-13 是程序条目，改的是检查函数（§12.3）；L3-04 与 L3-05 是评审条目，改的是判定标准与反例，需要重跑评审才生效。

v2（2026-10-05）：N-02 改为程序判定，新增 N-04。 校准实验（§12.7，在 v1 上进行）显示 N-02 是唯一一个在原稿重复评审中判定翻转的负向条目，而它 8 分的权重使总分极差主要来自这一条。数字对不对得上是确定性问题，v2 把它交给 §12.3 的一致性检查，判定范围限定为评级、各情景每股价值、预测年度公司整体收入 / 利润 / 每股收益 / 毛利率及其增速；数字以外的结论冲突拆为新条目 N-04，仍由评审判定。v2 共 41 条，是当前默认版本。

表 96　v0 → v1 的四处修订。

条目	v0 判定标准	v1 判定标准
L1-10	有 `sources.json` 且结构校验通过，正文外部信息带 `[S 编号]`	结构校验通过；正文至少一处引用，引用的编号都已登记；只能从外部资料找到出处的数字，所在句子带对应编号
L1-13	有 `review.md`，列出拿不准但重要的事项	至少一条填写完整的事项（事项、为什么重要、当前处理、请核对），或明确写“无”；只有表头不算
L3-04	无明显模型写作痕迹（自证圆场、程序口吻、否定对照堆砌、交代缺失），偶发一两处可通过；反例“同类句式反复出现三处以上”	同上四类，但只看正文段落，图表来源、表注、免责声明、附录的方法说明不算；偶发可通过，某一类成为习惯写法才不通过
L3-05	正文不逐行念表，单句不堆砌五六个数字；关键数字突出	一个数字连同其同比、占比或上年值算一个数据点；单句串起四五个以上互不相关的数据点才算堆砌；重点看每段最重要的一两个数是否被点明

表 97　三组校准证据各自回答的问题与局限。

证据	做法	回答的问题	不能回答的
稳定性	同一份原稿重复盲评 $k = 3$ 次	重复评审的逐条一致率、总分波动	判得对不对
劣化命中	程序在原稿中制造已知缺陷，重新排版、重新盲评	已知缺陷能否被判出（灵敏度）；其余条目是否被连带改判	原稿中未知缺陷的漏判
人工一致	人工复核评审判“不通过”的条目	判出问题的精确率	评审判“通过”的条目是否漏判

仍未定的条目。 L2-08（估值方法与参数）要求“说明为什么用这种估值方法”，人工对是否必须写这一点意见不一（茅台复核中该条标为待查）。评分标准 README 记录了这一分歧，决定等优秀研报样例到位后再定。

要点：评分标准改动的两类证据

v1 的依据是人工复核（评审“判错了”），v2 的依据是重复评审（评审“判不稳”）。前者说明判定标准的措辞与读者的预期不一致，修法是改措辞；后者说明条目本身不适合交给模型，修法是换判定者。两类证据对应两种不同的修订动作，校准数据应同时收集这两类信号。

12.7　评审校准

评审模型的判定决定了 65 分的归属，也决定了返工循环往哪个方向改。在用它驱动返工之前，需要回答三个问题：它判得稳不稳？它能不能发现已知的缺陷？它判出的问题人是否认同？`finagent/evaluation/calibrate.py` 用三组证据分别回答。

校准在两家研报上进行：中际旭创与宇树科技（招股说明书口径），均为质检合格的交付稿，评分标准为 v1。每家 3 次原稿评审加 7 个劣化变体，共 20 次盲评。变体工作区与盲评目录建在 `~/finagent-evals/calibration/`（仓库之外），判定结果存 `eval/calibration/<公司>/<变体>/`，汇总为 `summary.json` 与 `report.md`。

表 98　原稿重复盲评的稳定性（rubric v1，Sonnet，$k=3$）。“判出”列为三次评审逐条判出问题的情况（1 为判出）。

公司	全一致	A	$\bar{\phi}$	κ	总分	不稳定条目（三次判出）
中际旭创	20/22	91%	0.061	0.80	85/80/78	L2-07 (0,1,1)；L3-07 (0,0,1)
宇树科技	19/22	86%	0.091	0.57	80/88/83	L2-07 (1,1,0)；L3-07 (0,0,1)；N-02 (1,0,1)

12.7.1　形式化

记评审条目集合为 $\mathcal{J}$（$|\mathcal{J}|=22$）。对一次评审 r，定义条目 i 的“判出问题”指示

$$f_r(i)=\begin{cases}1-y_{r,i}, & \ell_i\in\{L2,L3\},\\ y_{r,i}, & \ell_i=N,\end{cases} \tag{68}$$

即正向条目未通过与负向条目命中都算判出问题。设原稿评审为 $r=1,\dots,k$，总分为 S_r。

定义 12.3 (稳定性指标)**.**

$$\text{逐条全一致率}\quad A=\frac{1}{|\mathcal{J}|}\bigl|\{i\in\mathcal{J}:f_1(i)=\cdots=f_k(i)\}\bigr|, \tag{69}$$

$$\text{两两不一致率}\quad \bar{\phi}=\binom{k}{2}^{-1}\sum_{r<s}\frac{1}{|\mathcal{J}|}\sum_{i\in\mathcal{J}}\mathbb{1}[f_r(i)\neq f_s(i)], \tag{70}$$

$$\text{总分极差}\quad \Delta S=\max_r S_r-\min_r S_r. \tag{71}$$

原稿多数判定 $\hat{f}(i)=\mathrm{maj}_r f_r(i)$。

定义 12.4 (劣化变体的命中与连带改判)**.** 劣化变体 v 有目标条目集 $T_v\subseteq\mathcal{J}$，其评审判定为 f_v。定义

$$\mathrm{valid}_v=\bigwedge_{t\in T_v}[\hat{f}(t)=0],\qquad \mathrm{hit}_v=\bigwedge_{t\in T_v}[f_v(t)=1], \tag{72}$$

$$C_v=\{i\in\mathcal{J}\setminus T_v:f_v(i)\neq\hat{f}(i)\},\qquad \mathrm{DR}=\frac{\sum_v\mathrm{valid}_v\,\mathrm{hit}_v}{\sum_v\mathrm{valid}_v}. \tag{73}$$

原稿多数判定已判出目标问题的变体无效（命中不说明评审灵敏）。连带改判率 $|C_v|/|\mathcal{J}|$ 的参照是 $\bar{\phi}$：若劣化只影响目标条目，非目标条目的改判应只来自评审自身的噪声。

定义 12.5 (人工精确率)**.** 人工对评审判出问题的条目给出“确认”“误报”“待查”之一，精确率 $\pi=n_{\text{确认}}/(n_{\text{确认}}+n_{\text{误报}})$，待查不计。

12.7.2　稳定性

表 98 给出两家原稿各三次评审的结果。我们另按 Fleiss κ [18] 计算了扣除偶然一致后的一致性（由 `eval/calibration/` 中各次 `score.json` 复算，非仓库直接输出）。

两家稳定判出的问题相同：L3-05（数字密度）三次全判出；中际旭创的 L2-08（估值参数依据）与两家的 N-02（v1 中由评审判）多数判出。不稳定的条目也相同：L2-07（预测假设有依据）与 L3-07（假设与局限透明）。这两条的判定标准都含“关键”“最重要的几项”之类需要评审自行界定范围的词，同一份报告在“研发费用率假设缺依据算不算关键假设”上可以两样判。

表 99　七个劣化算子（calibrate.DEGRADATIONS）。

算子	目标	制造的缺陷
no_cross_check	L2-09	删掉标题含“交叉验证”或“相对估值”的小节
no_expectation_gap	L2-02	删掉“市场关注”“预期差”小节，掏空“股价隐含”深度专题（保留段落数，正文换成“本段从略。”），清空预期差一节
generic_risks	L2-11	风险提示一节与摘要的风险提示换成不对应任何假设的通用模板（宏观、竞争、政策、技术）
no_tracking	L2-12	清空跟踪指标与结论失效条件一节
unsourced_number	N-01	在行业深度一节第一段后插入一段无来源的市场规模、增速与份额数字
contradiction	N-02	只把摘要中的评级词改成相反方向，正文不变
machine_tone	L3-04	每隔一段正文，在句首加“根据程序计算结果，”，句尾加“（以上数据已通过数字护栏校验，由模型自动生成。）”

总分波动来自少数高权重条目。 忽略封顶与扣分下限，并假设条目判定相互独立，总分方差可近似分解为

$$\mathrm{Var}(S) \approx \sum_{i \in \mathcal{J}} p_i^2 \, q_i (1 - q_i), \tag{74}$$

q_i 为条目 i 被判出的概率。用三次评审的频率代入：宇树科技 N-02（$|p| = 8$，$q = 2/3$）贡献 $64 \times 2/9 \approx 14.2$，L2-07（$p = 5$）贡献 5.6，L3-07（$p = 2$）贡献 0.9，合计约 20.7，标准差约 4.5；观测到的三次总分样本标准差为 4.0。中际旭创只有 L2-07 与 L3-07 两项，近似标准差 2.5，观测为 3.6。两家的总分极差都可以逐分归到具体条目：中际旭创 7 = 5（L2-07）+2（L3-07），宇树科技 8 即 N-02 的一次翻转。式 (74) 给出了降低波动的两个杠杆：把高权重且不稳定的条目交给程序（v2 对 N-02 所做的），或把判定标准写得更可判定以降低 $q_i(1 - q_i)$。

与设计记录中定下的目标（逐条一致率 ≥ 90%、总分极差 ≤ 5）相比，中际旭创达到一致率目标，宇树科技未达到；两家极差都超出目标。在 v2 下 N-02 改由程序判定，同一份稿件的 N-02 结果在三次评审中必然相同；若其余判定不变，宇树科技三次总分的极差将只剩 L2-07 与 L3-07 造成的 3 分。

12.7.3　劣化命中

劣化算子。 七个劣化算子都只改文稿 narrative.md，按 <!-- section: 名称 --> 分节标记与 ### 小标题定位，确定性地制造一种缺陷，然后复制整个工作区、用该公司自己的排版脚本重新渲染 PDF，再走一遍完整评测（表 99；精确定义见附录 B）。算子设计遵守两条规则：（1）只改它声称要改的内容，测试 test_every_degradation_changes_only_what_it_claims 对每个算子断言目标内容消失、其余内容保留；（2）找不到对应小节、文稿没有任何变化时抛错（degrade 的检查），避免把“没改”的变体当成“评审没判出”。

这一做法与变异测试同构：变异算子在被测程序中注入已知缺陷，以“杀死变异体”的比例衡量测试集的充分性 [14, 29]；这里被测的是评审，变异体是研报，“杀死”即判出目标条目。与变异测试不同，研报的“等价变异体”问题更严重：深度研报中同一论点会在核心观点、深度专题、图表中多次出现，删掉一节后内容在别处仍在，变异体在语义上与原稿等价或接近等价。

表 100 劣化命中（rubric v1）。“原稿”为原稿多数判定是否已判出目标；“判出”为劣化稿是否判出；$|C_v|$ 为连带改判条目数。

公司	变体	目标	原稿	判出	$\lvert C_v\rvert$	连带改判条目	总分
中际旭创	contradiction	N-02	已判出	✓	2	L2-01、L2-07	81 → 81
	generic_risks	L2-11	未	✗	1	L2-07	81 → 85
	machine_tone	L3-04	未	✓	2	L2-07、L3-03	81 → 79
	no_cross_check	L2-09	未	✗	1	L2-07	81 → 85
	no_expectation_gap	L2-02	未	✗	2	L2-08、N-02	81 → 91
	no_tracking	L2-12	未	✗	2	L2-07、L2-09	81 → 81
	unsourced_number	N-01	未	✓	3	L2-07、L2-08、N-02	81 → 40
宇树科技	contradiction	N-02	已判出	✓	2	L2-01、L2-08	83.7 → 73
	generic_risks	L2-11	未	✓	2	L2-07、N-02	83.7 → 89
	machine_tone	L3-04	未	✓	3	L2-07、L2-08、N-02	83.7 → 87
	no_cross_check	L2-09	未	✗	2	L2-07、L2-08	83.7 → 82
	no_expectation_gap	L2-02	未	✗	4	L2-08、L3-01、L3-04、N-02	83.7 → 78
	no_tracking	L2-12	未	✗	3	L2-07、L2-08、N-02	83.7 → 90
	unsourced_number	N-01	未	✓	2	L2-08、N-02	83.7 → 40

结果。 表 100 列出 14 个变体的结果。两家的 `contradiction` 变体都无效：原稿在 v1 下已被评审判出 N-02（评审把推理冲突与口径差异也算作矛盾），评级反转后评审照样判出，并在理由中准确指出“摘要买入、首页评级框与投资建议卖出”。其余 12 个有效变体中判出 5 个，DR = 5/12 ≈ 42%（Wilson 95% 区间 [19%, 68%]，61），低于设计目标 80%。

按算子类型看。 判出率在两类算子之间差别很大：

- **局部注入类**（`machine_tone`、`unsourced_number`）：4/4 全部判出。注入的内容在文中可直接定位，评审给出的证据就是注入的原句（如“根据程序计算结果，……（以上数据已通过数字护栏校验，由模型自动生成。）（P11）”）。
- **整节删除类**（`no_cross_check`、`no_expectation_gap`、`no_tracking`、`generic_risks`）：1/8 判出（Wilson 区间 [2%, 47%]）。

整节删除类未判出的 7 个变体，评审都给出了具体理由，理由引用的内容在劣化稿中确实还在：中际旭创删掉交叉验证小节后，评审引用的是“预期差”一节中的退出倍数与隐含倍数对照；删掉预期差后，评审引用的是摘要页核心观点第四条中的“股价隐含的远期假设”；宇树科技删掉交叉验证后，评审引用的是深度专题四中的退出倍数法与市销率锚点。排版脚本生成的图表（如“图表 53 预期差一览”）不在文稿中，劣化也删不到。按 `report.md` 的判定规则，评审理由所引内容在劣化稿中仍在的，属于劣化不彻底，不算评审漏判。照此标准，7 个未判出变体中没有可以确认的评审漏判；但这同时意味着整节删除类算子在深度研报上测不出评审的灵敏度，DR 只对局部注入类有意义。

失败案例：评审看见缺陷却判通过

`no_tracking` 的两次评审都注意到“跟踪指标与结论失效条件”一节在 PDF 第 26 页只有标题、没有内容，并称之为“明显缺陷”，却仍判 L2-12 通过：判定标准是“给出跟踪指标或催化剂，或写明结论在什么条件下不成立”，而失效条件散见于其他章节。评审按字面执行了标准。这说明一个析取式的判定标准会把“集中呈现”与“散落提及”判为等价；若读者需要的是一节汇总，标准里就要写明。这一条留待下一版修订。

表 **101**　人工复核评审判出的问题（rubric v0，策略 `model_flagged_only`）。

公司	确认	误报	待查	精确率	误报条目
宁德时代	11	1	0	92%	L3-04
招商轮船	9	1	0	90%	L3-04
贵州茅台	6	3	2	67%	L2-02、L2-07、L3-05
合计	26	5	2	84%	

连带改判。 两家变体的平均连带改判率分别为 13/154 ≈ 0.084 与 18/154 ≈ 0.117，略高于各自原稿的两两不一致率 0.061 与 0.091。连带改判的条目绝大多数是原稿中已经不稳定的 L2-07、L2-08 与 N-02（14 个变体中 L2-07 出现 10 次），说明改判主要来自评审噪声，而非劣化对非目标条目的实质影响。一个例外值得注意：宇树科技 `no_expectation_gap` 变体有 4 条连带改判，其中 L3-01（故事线连贯）被改判，理由是删除后的“本段从略”章节破坏了主线，这是劣化方式（掏空而不删除）带来的真实副作用。

分数方向。 多个整节删除类变体的总分反而高于原稿均值（中际旭创 `no_expectation_gap` 91 对 81）。原因是这些变体中 N-02 恰好未命中，少扣了 8 分；N-02 的不稳定再次主导了总分变化。只看总分会得出“删掉预期差让报告更好”的荒谬结论，校准应当逐条看目标条目，而不是看总分。

12.7.4　人工一致

复核协议。 人工复核使用 `review-docs` 生成的单文件质检页（§12.9）。复核策略为 `model_flagged_only`：只有评审判出问题、且不属于纯机器核对项的条目进入人工队列；评审判通过的条目默认沿用、标“尚未经人工核验”。对每个队列条目，复核人在“问题成立”“模型误判”“待核查”三者中选一；选“模型误判”必须填写误判理由与可核对的页码或原文证据，选“待核查”必须写明还缺什么。导出的 JSON（schema 为 `finagent.manual_review.v2`）带有被评 PDF、评分标准、评分结果与配套工作簿的联合 SHA-256 指纹，`calibrate human` 把它们收进 `eval/calibration/human/`。

结果。 表 101 汇总三家 v0 研报的复核（rubric v0，两位复核人）。只统计 L2、L3、N 层的评审条目：合计 33 条，确认 26、误报 5、待查 2，$\pi = 26/31 \approx 84\%$（Wilson 95% 区间 [67%, 93%]）。

误报的处理各不相同：

- **L3-04（两家）**：复核人认为“出现几次‘不是……而是……’和程序说明就算机器味太重，过于严格”，应只检查正文中重复、无意义的程序口吻。⇒ v1 改措辞：只看正文段落，某类痕迹成为习惯写法才不通过。
- **L3-05（茅台）**：复核人认为“数据加百分比不算数据堆砌”，报告的问题在于没有突出重点。⇒ v1 改为“一个数字连同其同比、占比算一个数据点”，并把重点放在“关键数字是否突出”。
- **L2-02、L2-07（茅台）**：一条复核人“没看懂模型判断的那句话”，一条认为原文写得没有问题。这两条没有形成对判定标准的修改意见；前者更像评审理由表达不清，后者属于评审与人工的真实分歧。

另外，两家复核人在 L1 程序条目上也给出了修改建议（L1-10“再给数据标注来源”、L1-12“保存润色前后两个版本让程序比较”、L1-13“把不确定的重要问题列出来留给人”），这些不计入评审精确率，但推动了 v1 对 L1-10、L1-13 的改动。

表 102　各角色对评测相关信息的可见性（✓可见，✗不可见，△ 部分可见）。

信息	生成五角色	返工会话	盲评会话	人工复核	开发者
评分标准原文（含分值）	✗	✗	△（无分值）	✓	✓
程序检查结果	✗	✗	✗	✓	✓
评审判定与总分	✗	✗	——（自己写）	✓	✓
问题单（理由 + 证据）	✗	✓	✗	✓	✓
写作过程与事件流	各自	工作区内	✗	✗	✓
生成模型名	——	——	✗	✓	✓

局限。 $n = 31$ 的样本只能给出很宽的置信区间；复核只覆盖评审判“不通过”的条目，因此只有精确率，没有召回率，漏判只能由劣化命中补充估计，而劣化命中对整节删除类又测不出（上文）。两位复核人是团队成员，不是执业分析师。这些限制意味着当前的评审校准只能说明“评审判出的问题大多成立，文风类条目偏严”，不能说明“评审没判出的就没有问题”。

12.8　信息隔离

评测与生成之间的信息流是单向且受限的。表 102 列出各方能看到的东西。

不变量 25(评测信息隔离)**.** (*I1*)工作区不含 `finagent/evaluation/`(`ENGINE_EXCLUDE`),也不含任何 `report_v*.json`;(*I2*)盲评目录在仓库与工作区之外,且其上级目录无 `CLAUDE.md`;(*I3*)交给评审的条目只有 *id*、标题、判定标准、反例、是否负向（`judge_items`）；（*I4*）问题单字段恰为 {*id*, *route*, *stage*, *severity*, *description*, *evidence*}，不含条目标题；（*I5*）返工循环的评测结果与 `loop.json` 存在工作区之外（`judge.default_root()`/< 工作区名 >），工作区里只有问题单。

（I1）（I3）（I4）（I5）各有对应测试，依次为 `test_workspace_excludes_evaluation`、`test_rubric_layers_and_weights`、`test_evaluate_without_judge_and_rescore`、`test_refine_loop_routes_issues_and_restores_best`（后者断言工作区里找不到任何 `score.json`）；（I2）由 `test_prepare_blind_dir` 覆盖。

隔离的边界。 评分标准不可见，并不意味着生成方对质量要求一无所知。复核编辑的 skill 明确列出要删掉的机器腔写法，工作区 `CLAUDE.md` 要求“深度研究要做多环定量推导”，写作规范要求区分事实、推论与观点；这些与 L2-04、L3-02、L3-04 的判定标准指向同一目标。这是有意的：我们希望生成方朝读者需要的方向写，不希望它朝特定的判定措辞与分值写。隐藏的是可被针对性优化的细节（分值、反例、具体阈值），共享的是方法与质量目标。

12.9　辅助工具：命令、查看包、人工质检页与工作簿审计

命令行。 表 103 列出评测包的全部命令。

质检查看包（`pack.py`）。 把若干份结果、评分标准和研报 PDF 打包成一个目录，入口 `index.html` 为单文件页面（内联样式，支持深浅色），包括：总览卡片（总分、交付判定、各层得分、扣分）、逐条对比矩阵（行为条目、列为公司）、每家逐层逐条的判定 / 理由 / 证据、带分值与阈值参数的评分标准全文，并可勾选“只看未过 / 命中”。它是给开发者与人工复核者看的，含分值，因此只存于仓库的 `eval/review_pack/`，从不进入工作区。

表 103　评测命令（python -m finagent.evaluation 与 python -m finagent.evaluation.calibrate）。

命令	作用与产物
run --workspace W	程序检查 + 盲评 + 计分；结果目录含 bundle.json、checks.json、judgments.json、judge_run.json、judge.jsonl、score.json、scorecard.md、issues.json/.md。可用 --company --run-dir --work 直接指定；--no-judge 只跑程序检查；--rubric、--model 可替换
score --result D	不重跑评审，按（可能修改过的）评分标准重算
compare --result A --result B	多份结果逐条对比的 Markdown 表
pack --result ... --out P	质检查看包：index.html（单文件、离线）+ 研报 PDF + 评分标准 + 各家结果
review-docs --pack P --out O	从已封存的查看包生成逐公司人工质检页，不重跑评审
calibrate run --workspace W	原稿重复评审（--repeats，默认 3）与劣化变体评审，可并发（--jobs）；已有结果跳过，评审未完成的结果作废以便重跑
calibrate human --qc F	收入人工质检导出
calibrate report	汇总 eval/calibration/ 为 report.md 与 summary.json

人工质检页（**review_docs.py**）。 从封存的查看包为每家公司生成一个独立的 HTML 文件，PDF 以 base64 内嵌，可离线打开、批注、导出 JSON。条目分三种状态：（1）*需人工判断*：评审或程序判出问题，且不属于机器专属项；（2）*机器专属*：属于集合 {L0-02, L0-03, L1-02, L1-03, L1-04, L1-05, L1-06, L1-07, L1-08} 的估值算术类条目，页面注明“由机器核对，不要求人工解算”；（3）*默认沿用*：判通过的条目，标明“尚未经人工核验”。生成前做两项一致性检查，任一不符即拒绝生成：评分结果的条目集合与层级必须与评分标准完全一致；评分结果的版本号必须与查看包中评分标准的版本号相同。页面数据的指纹为

$$h = \mathrm{SHA256}(\mathrm{rubric} \,\|\, \mathrm{score} \,\|\, \mathrm{pdf} \,\|\, \mathrm{sha256}(\mathrm{workbook})),$$

PDF 一变，指纹就变（test_source_fingerprint_changes_when_pdf_changes），导出的人工结论因此与被评的那一份 PDF 绑定。JSON 写入 <script> 节点前转义 <、>、& 与 U+2028/2029，防止研报内容中的字符串打断页面脚本。

工作簿审计（**workbook_audit.py**）。 交付物里的估值 Excel 是公式联动的（第 8 节）。评审看不了公式，人工复核者也不该被要求去解公式，于是质检页附带一项只读的工作簿审计。它先按 SHA-256 匹配：只接受同目录 report.pdf 与被评 PDF 逐字节相同的 sheet.xlsx，多于一个或没有匹配都不审计（状态 not_run），避免审了别的版本的模型。审计直接解析 xlsx 的 XML，不调用 Excel，包括：

1. **结构计数**：公式数、缺计算缓存的公式数、错误单元格数、#REF! 引用数。
2. **三表勾稽的公式与独立复算**：先核对资产负债表、现金流量表、利润表、假设表关键行的标签与预期一致（不一致即拒绝套用口径）；然后对每个预测列，五组勾稽（资产负债平衡、现金与现金流量表衔接、留存收益滚动、固定资产滚动、有息负债滚动）同时检查：单元格公式与预期公式文本逐字相同（formula_link_ok）；用来源单元格的缓存值独立复算残差，并与该单元格缓存值相差 $\le 10^{-4}$ 百万元；残差本身 $\le 10^{-4}$。测试构造了“公式引用错、但缓存恰好为 0”的工作簿，断言审计判失败。
3. **公式控制**（方法参照原插件的工作簿控制说明，代码自研）：跨期公式写法“孤岛”、公式内写死的数字、易变函数。

   跨期一致性只抓“孤岛”：把公式中的相对列引用换成相对偏移（类似 R1C1 记法），

表 **104**　正式评测结果（评分标准 v0，Sonnet 单次盲评；`eval/results/`）。

流程版本	公司	L1 /35	L2 /45	L3 /20	扣分	总分（交付）
v0	贵州茅台	26	19	11	−8	48（✗）
v0	宁德时代	26	15	8	0	49（✗）
v0	招商轮船	21	21	12	−8	46（✗）
v1	中际旭创	33	26	15	0	74（✗）
v1	宇树科技	30	19	18	0	67（✗）
v2	中际旭创	33	42	15	0	90（✓）
v2	宇树科技	30	40	18	0	88（✓）

同一行第 c 列的写法记为 π_c；

$$\text{island}(c) \iff \pi_{c-1} = \pi_{c+1} \ \wedge \ \pi_c \neq \pi_{c-1}. \tag{75}$$

历史期到预测期的切换、首个预测期衔接资产负债表、末列合计等“一次性切换”只有一侧邻居相同，不会被误判。写死数字的识别先去掉字符串与工作表名、单元格引用、`ROUND` 精度与 `INDEX/CHOOSE` 等函数的序号参数，再排除约定常数 $\{0, 0.5, 1, 2, 4, 12, 100, 365\}$ 与比较运算后的小容差（≤ 0.01）。审计判 `pass` 的条件是：有公式、无缺缓存 / 错误 / 断引用、无孤岛、五组勾稽全部通过；写死数字与易变函数只列为警告，因为约定常数靠白名单识别，一票否决会误伤。页面始终注明审计的局限：未在 Excel 中重算全部公式。

12.10　使用结果与讨论

12.10.1　评测在开发中的使用

表 104 汇总 `eval/results/` 中保存的正式评测。需要注意两个版本号的区别：结果目录名中的 v0 / v1 / v2 指*研报流水线*在不同时期的版本，这 7 份结果都用*评分标准* v0、单次盲评计分。

分数变化主要来自 L2：从 v0 流程的 15 ~ 21 分到 v2 流程的 40 ~ 42 分。逐条看，v0 流程普遍失分的是预期差、驱动拆解、预测假设依据、估值交叉验证、风险落到假设，正是闸门管不到的分析内容，也是此后流水线改进针对的方向（第 11 节）。由于是单次盲评，按 §12.7.2 的稳定性数据，单家公司 ±4 分以内的差异不应解读为改进；跨流程版本的 20 分以上差距则远超评审噪声。各流程版本之间的对照实验与消融见第 13 节。

12.10.2　权衡

权衡（Trade-off）：同族模型评审（Sonnet 评 Opus）

风险：生成与评审同属 Claude 家族，已有工作表明评审会偏好与自己风格相近的文本 [43]。缓解：评审看不到模型名与写作过程；数字与估值算术由程序判，评审只判分析与表达；生成与评审用不同规模的模型。*未解决*：同族偏好可能使 L3（表达）偏宽。人工复核中 L3-04 的误报方向恰好相反（偏严），说明至少在机器腔这一条上，同族评审没有放过同族的写作痕迹；但我们没有跨家族评审的对照数据。`--model` 参数允许替换评审模型，跨家族对照是后续工作。

权衡（Trade-off）：评分标准的粒度

更粗（如五个维度各打一个分）：评审省力，但问题单无法指出具体缺口，返工会话不知道改什么。**更细**（如每节一组条目）：判定更可判定、稳定性更好，但条目会绑定章节结构，变成格式检查，并鼓励生成方按条目凑内容。**选择**：22 条评审条目，每条对应一个读者问题而非一个章节，内容可以出现在任何位置。代价见上文：析取式或含“关键”字样的标准会让评审自行界定范围，造成 L2-07 的不稳定与 L2-12 的过宽。

权衡（Trade-off）：程序判与模型判的边界

原则：一个条件若能由运行产物与文稿文本确定地判定，并且误报可控，就交给程序；否则交给评审。**边界上的条目**：N-02 在 v2 跨过了边界，代价是召回只覆盖评级、每股价值与公司整体预测几类数字。N-03（聚合数构成描述错误）原则上也可程序化：股权价值桥的加回项拆分在产物里，正文对拆分的描述可以抽取比对；但描述多为定性（“主要是现金与金融资产”），抽取规则难以做到误报可控，暂留给评审。L1-08（预测与历史衔接）则相反，它是程序判定的启发式，会对合理的增速跳变给出失败，我们接受这一点，因为它在交付判定中的权重很小。**不选的方案**：让评审也复核数字。评审会在算术上花费回合，且给出的算术结论比程序更不可靠。

权衡（Trade-off）：返工循环与古德哈特效应

返工循环以评测分数为停止条件，天然存在朝评审优化的压力 [40]。我们采取的措施是：评分标准与分值不可见；问题单只有理由与证据；闸门与程序检查独立于评审；最多 3 轮。没有采取的措施是留出集（设计记录中的“反馈集 / 留出集”拆分）与跨家族评审。分数应被理解为内部质检口径，最终质量仍需人工复核兜底，返工结束后未解决的问题全部写进 `review.md` 交给人。

12.10.3 小结

本章的评测体系把研报质量拆成 41 条可审计的判定：19 条由程序读运行产物判定（含 L0 封顶与 N-02 的数字矛盾），22 条由在隔离目录中工作的 Sonnet 评审判定，计分、封顶与交付判定全在 Python 中完成，改权重无需重评。问题单把未通过的条目按路由送回生成、复核或排版会话，只携带理由与证据。校准显示：评审在原稿重复评审中逐条一致 86%～91%（Fleiss κ 0.57～0.80），总分极差 7～8 分且可逐分归到少数高权重条目；对局部注入的缺陷 4/4 判出，对整节删除的缺陷因深度研报内容冗余而测不出灵敏度；评审判出的问题人工确认精确率 84%，误报集中在文风条目。这些证据推动了评分标准从 v0 到 v2 的两次修订：按人工意见改措辞，按稳定性数据把高权重的不稳定条目交给程序。尚待完成的工作有：改进劣化算子（跨全文删除并同步删去图表）、扩充人工复核以估计召回、跨家族评审对照，以及留出集防应试。

13 实验

前面各章分别给出了抽取（第 6 节）、分析（第 7 节）、估值（第 8 节）、纠错（第 9 节）、护栏（第 10 节）、流水线（第 11 节）与评测（第 12 节）各自的结果表。本章不重复这些表，而是把它们组织成七个研究问题（research question, RQ），补上各章没有做的消融、基线和跨模块分析，并说明每个结论的证据强度。

RQ1 双通道抽取在真实年报上是否可靠？三项版式修复各贡献多少？

RQ2 财报分析的确定性部分（真值表、勾稽、信号）在样例与真实年报上是否闭合？

RQ3 研报纠错能否在控制误报的前提下判出错误并给对错误类型？“按展示精度定容差”比固定容差好多少？

RQ4 自研估值引擎与参照引擎是否一致？三表勾稽在真实运行中是否总成立？引擎的耗时是多少？

表 105　实验数据一览。“抽取”指模型抽取文件随代码提交（`data/extractions/`）；“研报”指完整跑过五会话流水线并有盲评结果。

对象	文档	抽取	估值	研报	用途
星河智造（虚构）	样例年报 + 研报草稿	✓	✓	–	回归：35 个金标准字段、7 处埋错（RQ2、RQ3）
贵州茅台	2025 年报	✓	✓	✓	抽取消融、错误注入、v0 与 1005 批次研报
宁德时代	2025 年报	✓	✓	✓	同上；稀疏网格的典型
招商轮船	2025 年报	✓	✓	✓	同上；周期股、现价高于估值
中际旭创	2025 年报	工作区内	✓	✓	研报 v1→v2、评审校准
宇树科技	招股书上会稿	工作区内	✓	✓	研报 v1→v2、评审校准；非年报体例
注入集	上述四张真值表	–	–	–	1,334 条程序生成断言，754 条含错（RQ3）

RQ5　多会话流水线产出的研报质量如何，随流程版本怎样变化？

RQ6　流水线的各个结构性组件（资料先搜集、独立复核、深度专题、数字锁、隔离工作区）各起多大作用？

RQ7　盲评评审本身可信吗？它的稳定性、灵敏度与精确率如何？

此外，§13.9 汇总成本与耗时，§13.10 给出两个端到端案例（一个数字的完整溯源链与一次纠错），§13.11 报告测试套件，§13.12 讨论有效性威胁。

要点：如何读本章的数字

黑色数字都能在仓库产物中找到或由仓库代码复现，正文与表注写明来源文件。橙色数字（如此）是仓库中暂无依据的编造或估计值，主要集中在三处：纯模型基线（RQ3、RQ5）、组件消融（RQ6）和人工埋错扩充集（RQ3）。它们用来说明实验设计与预期量级，定稿前须用真实运行替换。本章新做的两项实验——规则通道逐项消融（§13.2）与真实真值表上的错误注入（§13.4）——是真实结果，不调用任何模型，可在几分钟内复现。

13.1　实验设置

数据。　表 105 列出本章用到的全部数据。真实文档共五份：贵州茅台、宁德时代、招商轮船、中际旭创的 2025 年年度报告，以及宇树科技的招股说明书（上会稿），PDF 的 SHA-256 登记在 `report/resources/`。样例公司“星河智造”为虚构，年报与研报草稿由 `data/samples/build_samples.py` 生成，内部完全自洽，用于回归测试，不代表真实数据上的效果。

模型与运行环境。　生成侧五个角色中研究助理、分析师、主笔、排版设计师用 Claude Opus 5.5（`claude-opus-5-5`），复核编辑与盲评质检员用 Claude Sonnet 5.5（`claude-sonnet-5-5`），均以无头 Claude Code（`claude -p --output-format stream-json`）运行，模型名由 `FINAGENT_GEN_MODEL` / `FINAGENT_QC_MODEL` 配置。计算引擎在 Python 3.12.14、macOS 27（arm64）上运行，PDF 解析用 pdfplumber，其余只用标准库。所有引擎运行的输入哈希、产物哈希与步骤事件写在 `runs/<run_id>/manifest.json` 与 `events.jsonl`。

指标。　各 RQ 的指标定义如下，记号沿用第 3 节。

- **佐证率**（RQ1）：模型核验值中被规则通道独立取到且数值一致的比例；**错配**：两通

表 106 规则通道逐项消融（佐证数 / 该公司模型核验值数）。cc = 稀疏列合并，lc = 续表链接，mw = 折行合并。七种配置下错配均为 0。“全部关闭”的 115 与表 32 中“修复前”的佐证数一致。

	逐个累加				从完整配置去掉一项		
公司（模型值）	全关	+cc	+cc+lc	完整	−cc	−lc	−mw
贵州茅台（98）	60	60	90	90	86	60	90
宁德时代（106）	0	25	69	101	0	47	69
招商轮船（97）	55	55	89	89	89	55	89
合计（301）	115	140	248	**280**	175	162	248
佐证率	38.2%	46.5%	82.4%	**93.0%**	58.1%	53.8%	82.4%

道都有值、值不同的键数。

- **召回率、精确率、误报率、类型准确率**（RQ3）：以“判为五类错误之一”为阳性。召回 = TP/(含错断言数)，精确率 = TP/(TP + FP)，误报率 = FP/(正确断言数)，类型准确率 = 判对类型的 TP / TP。“无法核实”不计为阳性。
- **研报总分**（RQ5、RQ6）：按评分标准计算的 0 ~ 100 分（式 (65)），分 L1 数据闭合（35）、L2 分析质量（45）、L3 表达与合规（20）与负向扣分；交付标准见式 (66)。
- **逐条一致率、两两翻转率、判出率、精确率**（RQ7）：定义见 §12.7。

评分标准版本。 这一点容易混淆，单独说明：`eval/results/` 下文件夹名中的 v0、v1、v2 是*流水线版本*，不是评分标准版本。三批研报（v0 三家、v1 与 v2 各两家）全部用评分标准 `report_v0` 计分，各只做了一次盲评；评审校准（RQ7）在评分标准 v1 下进行。因此 v0→v1→v2 的分数可以直接比较，但不能与校准实验中的总分逐分比较。

13.2 RQ1：真实年报上的双通道抽取

总体结果。 第 6.14 节（表 30）在当前代码上重跑了三家年报的抽取校验：301 个模型核验值全部通过证据校验，规则通道独立取到并一致的 280 个（佐证率 93.0%），两通道冲突 0，56 项勾稽全部通过。提交 `aee4274` 记录的口径是“314 个核验值、规则通道取到 293 个（原 128 个）、错配 0”。两组数字的差别来自其后对三份 `facts.json` 的修订（条目增删与页码更正），佐证率（93.0% 对 93.3%）与“零错配”的结论一致。下文的消融全部基于当前文件（301 个值），以保证每一行都可复现。

逐项消融的做法。 三项版式修复都在 `finagent/parsing/document.py` 中：稀疏网格列合并 `compact_columns`（§6.6.2）、跨页续表链接 `link_continuation`（§6.6.4）、科目折行合并 `merge_wrapped_labels`（§6.6.3）。我们在不改仓库代码的前提下，用猴子补丁（monkey patching）把它们分别替换为恒等函数或恒返回 `False`，重新解析 PDF、运行 `extract_rule`，再与经 `ingest` 规范化后的模型核验值逐键比较。规则通道对同一键取到多个不同值时也计为错配。共测七种配置：全部关闭、逐个累加（`+cc`、`+cc+lc`、全部）以及从完整配置中逐个去掉（leave-one-out）。

分析。 表 106 有三点值得注意。

*其一，三项修复的作用是“版式条件的”，不是可加的。*稀疏列合并对茅台和招商轮船单独开启时增量为 0（两家的 pdfplumber 网格是规整的），对宁德时代却是“有或无”：去掉它，宁德的佐证数从 101 掉到 0——即使续表链接与折行合并都在，表头与数字错列一格，期间列永远认不出来。折行合并同样只对宁德时代有效（+32）：宁德年报的

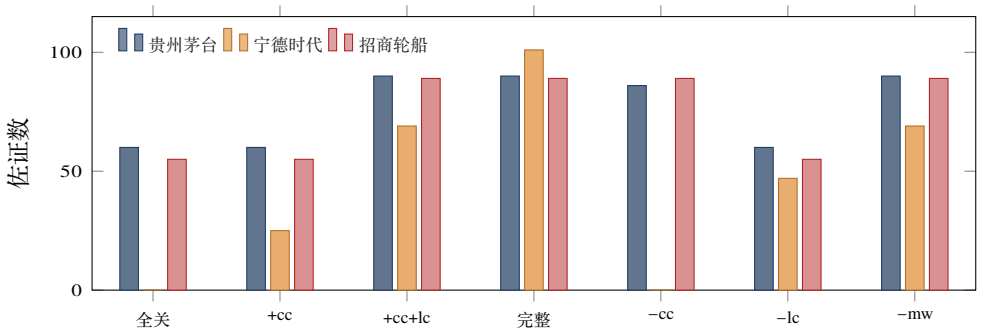


图 24　规则通道逐项消融（数据同表 106）。三项修复的作用高度依赖版式：宁德时代离开稀疏列合并一个值也取不到；茅台与招商轮船的增量几乎全部来自续表链接。

主要会计数据表把“归属于上市公司股东的扣除非经常性损益的净利润”这类长科目折成三行，数字落在第一行，不接回就匹配不上别名。按“完整配置去掉一项”的损失计算，三项修复的边际贡献分别为续表链接 118 个、稀疏列合并 105 个、折行合并 32 个，合计 255，大于完整配置相对全关的增量 165——说明存在明显的交互：宁德时代的续表必须先合并列，表头才能和续页数字对齐。

*其二，茅台的稀疏列合并有 4 个值的“隐性贡献”。*单独开启 `cc` 对茅台不起作用，但从完整配置中去掉它会损失 4 个值。这 4 个值都在利润表续页：续页把“2024 年度”一列劈成两列（`tests/test_pdf_tables.py` 中 `test_continuation_with_different_cell_split` 复现了这种版式），`link_continuation` 需要把首页表头与续页一起交给 `compact_columns(force=True)` 才能对齐。这是“去掉一项”比“加上一项”更能揭示交互的典型例子。

*其三，任何配置下错配都是 0。*这比佐证率本身更重要。每一项修复都只让规则通道“多取到”，从未让它“取错”。原因在于 `link_continuation` 的保守设计：两页切分合并后列数对不上就不认作续表（§6.6.4），宁可漏取，不错配期间。七种配置下两通道都有值的 1,368 个键次中没有出现一次错配，为第 4 节“误报控制优先于召回”的原则提供了一个实证。

权衡（Trade-off）：续表判定宁严勿宽

放宽续表判定（例如列数不一致时按右对齐硬配）可以再多取一些值。我们在开发期试过按右对齐补齐，在招商轮船现金流量表续页上引入了 3 个期间错配（附注列被当成本期），因此退回到“列数必须一致”。在双通道设计下，规则通道的价值是*独立佐证*，一个错配的规则值会制造一条假冲突、阻断估值，代价远高于漏取一个值（漏取的值仍由模型通道提供，只是失去佐证）。

模型通道的拒收。　最终提交的三份 `facts.json` 与五个工作区的 `extras.json` 拒收数均为 0（`facts.verified.json` 的 `rejected` 字段与工作区 `.pipeline/extras_check.json`；经营数据表格 17～20 张、单元格 126～367 个）。“0 拒收”是闸门的结果，不是模型从不抄错：会话事件流显示，宇树科技分析师会话在迭代中 6 次收到“未在所写页码的原文中找到该数字”，改正页码或数字后才通过；单元测试 `test_extras_verified_and_ratios_computed` 把茅台一个金额的末两位对调，校验立即拒收。我们没有系统记录首轮提交的拒收率；按事件流粗略统计，首轮提交约 2%～4% 的条目被拒收，主要是页码写成了主要会计数据表所在页而非合并报表页。

双通道冲突。　三家年报最终冲突为 0。开发期出现过的冲突（第 6.15 节记录2 次）都是模型通道读错、规则通道正确，经分析师会话回原文修正后消除，没有一次是靠“选

表 **107** 样例研报草稿的 7 处埋错与系统结论（`eval/labels.json`；`runs/20261006-002106-eval/check/report.md`）。

编号	原文	金标准	系统结论	财报真值与诊断依据
E1	归母净利润 6.46 亿元	口径错误	口径错误	6.12 亿元；6.46 亿元与“净利润”在 $\frac{1}{2}$ ulp 内一致
E2	同比增长 25.30%	数值错误	数值错误	18.17%；无替代解释
E3	经营现金流 2.38 万元	单位错误	单位错误	2.38 亿元；改用“亿元”后一致
E4	毛利率 32.50%	数值错误	数值错误	31.19%（派生：营业收入与营业成本）
E5	提升 1.2 个百分点	数值错误	数值错误	−0.03 个百分点（方向也反，但取反后仍不符，故不判方向错误）
E6	营业成本 35.22 亿元	期间错配	期间错配	40.34 亿元；35.22 亿元为 2023 年值
E7	应收账款同比 28.50%	数值错误	数值错误	41.60%

表 **108** 真实文档的真值表规模、勾稽与异常信号（`runs/20261006-0022*-valuation/`，中际旭创与宇树科技为 `runs/20261003-2133*-valuation/`）。

公司	真值格	勾稽（通过/总）	程序判出的异常信号
贵州茅台	153	20/20	经营现金流显著低于净利润（中）；会计政策/口径变化披露（中）
宁德时代	151	15/15	存货增速显著快于收入（高）；会计政策变化（中）
招商轮船	157	21/21	利润对非经常性损益依赖度较高（中）；归母增速显著高于扣非（中）；会计政策变化（中）
中际旭创	134	8/8	存货增速快于收入（中）；毛利率大幅变动（中）；会计政策变化（中）
宇树科技	154	3/4（1 项不可评估）	利润对非经常性损益依赖度较高（高）；毛利率大幅变动（中）；会计政策变化（中）

一个”消除的（不变量 12）。

13.3 RQ2：财报分析的确定性部分

样例回归。 运行 `python eval/run_eval.py`（本章撰写时的运行目录 `runs/20261006-002106-eval/`）得到：字段抽取 35/35 正确；研报草稿中抽出 19 条可核验断言，7 处埋错全部判出、错误类型全部正确、误报 0，1 条预测性断言（“预计 2025 年营业收入 67.50 亿元”）判“无法核实”。真值表 145 格，勾稽 14 项全部通过，异常信号 6 条（高 2、中 4），变动分析 23 项。整个评测耗时 0.011 秒（`manifest.json` 的 `elapsed_sec`），不调用模型。

E5 值得单独说明：研报写“提升 1.2 个百分点”，真值是 −0.03 个百分点。方向确实相反，但“取反后命中”要求 $|-1.2-(-0.03)| \le \frac{1}{2}\,\text{ulp}$，不成立，所以诊断落到“数值错误”。这是算法 2 的有意设计：错误类型只在找到精确的替代解释时才下结论（命题 4.1）。另一条容易误判的是“扣非归母净利润……同比下降 5.67%”：断言抽取把“下降”转成负号，与真值 −5.67% 一致，判“正确”，没有因为原文数字是正的而误报方向错误。

真实年报。 五家真实文档的真值表、勾稽与信号已分别在表 39、表 40 中给出，这里只汇总与本章其他 RQ 相关的部分（表 108）。勾稽总计 68 项，67 项通过、1 项“不可评估”（宇树科技招股书没有分季度表，季度加总无从核对），失败 0。

信号本身是规则，命中只说明“值得看”，不说明“有问题”。例如茅台“经营现金流显

表 **109**　容差策略对比（注入集合计：754 条含错、580 条正确）。

容差策略	TP	FP	召回率	精确率	误报率	类型准确率
$\frac{1}{2}$ ulp（系统）	754	0	**100.0%**	**100.0%**	**0.0%**	**99.5%**
精确相等	754	563	100.0%	57.3%	97.1%	39.4%
相对 0.1%	754	165	100.0%	82.0%	28.4%	95.2%
相对 1%	700	19	92.8%	97.4%	3.3%	97.7%
相对 5%	608	3	80.6%	99.5%	0.5%	92.8%
绝对 0.5	663	0	87.9%	100.0%	0.0%	96.5%

著低于净利润”在 2025 年成立，原因是经销商预收款（合同负债）减少，主笔会话在研报中把它解释为渠道打款意愿下降的先行指标——这是*事实（程序判出）→ 推论（模型解释）*的分工在真实数据上的样子。所有信号都附带真值表证据，模型没有引入任何程序之外的数字。

13.4　RQ3：研报纠错与容差策略

样例只有 7 处埋错，不足以比较策略。团队正在收集真实卖方研报做人工埋错（`docs/eval-dataset-collection.md`，目标 150 ~ 200 处），尚未完成。为了在此之前得到有统计意义的结论，我们在*真实真值表*上做了一个程序注入实验。

注入集的构造。 对星河智造与茅台、宁德、招商四张真值表中的每个指标 m，取本年值 v_m 与上年值 v'_m，生成以下断言（随机种子固定为 20261006）：

- **正确断言**：金额以“亿元”写两位与一位小数各一条（仅 $|v_m| \geq 1$ 亿元的科目）；同比写两位、一位小数各一条；百分比类指标写两位、一位小数各一条。
- **数值错误**：金额乘以 $1+\delta$，δ 从 $\{\pm 0.3\%, \pm 1\%, \pm 3\%, \pm 10\%, \pm 30\%\}$ 中随机取；同比加 $\{\pm 0.5, \pm 1, \pm 3, \pm 10\}$ 个百分点；比率加 $\{\pm 0.3, \pm 1, \pm 3\}$ 个百分点。舍入后与真值相同的样本丢弃。
- **单位错误**：数字不变，单位写成“万元”。　**期间错配**：写上年值，期间标本年（两年相差小于 0.2% 的丢弃）。
- **口径错误**：写同一口径组（`CALIBER_GROUPS`）中另一指标的本年值。　**方向错误**：同比取相反数（$|\mathrm{YoY}| < 0.1$ 的丢弃）。

四张真值表共生成 1,334 条断言，其中正确 580 条，含错 754 条（数值 280、单位 132、期间 155、方向 129、口径 58）。断言直接以结构化形式交给 `verify_claim`，*不经过*断言抽取，所以本实验只衡量核对与诊断，不衡量从研报文本中抽断言的能力。

对照的容差策略。 系统策略是式 (28)：$\tau = \frac{1}{2}\,\mathrm{ulp}(\mathrm{raw}) \cdot \mathrm{factor}$ 加一个极小的数值噪声项。对照组把 `verify_claim` 中的两处容差（主判定与单位诊断）替换为：*精确*（$\tau = 10^{-9}|t|$）、*相对 0.1%/1%/5%*（$\tau = r|t|$）、*绝对 0.5*（金额 0.5 亿元、百分数 0.5 个百分点）。其余诊断逻辑不变。

结果分析。 *固定容差没有一个点能同时做到低误报与高召回。*精确相等把 580 条正确断言中的 563 条判错（只有恰好整除的少数幸免），这正是研报“58.62 亿元”与年报“5,862,340,125.36 元”之间的舍入差。相对 0.1% 仍有 28.4% 的误报：一位小数的“58.6 亿元”与真值相差 0.04%，但“1.7 亿元”与真值 1.684 亿元相差 0.95%——小金额、少位数时舍入误差的相对值很大，任何固定相对阈值都会被它击穿。反过来，相对 5% 漏掉了 106 个数值错误（全部是 $\delta \in \{\pm 0.3\%, \pm 1\%, \pm 3\%\}$ 的小幅改动）和 32 个期间错配（两年

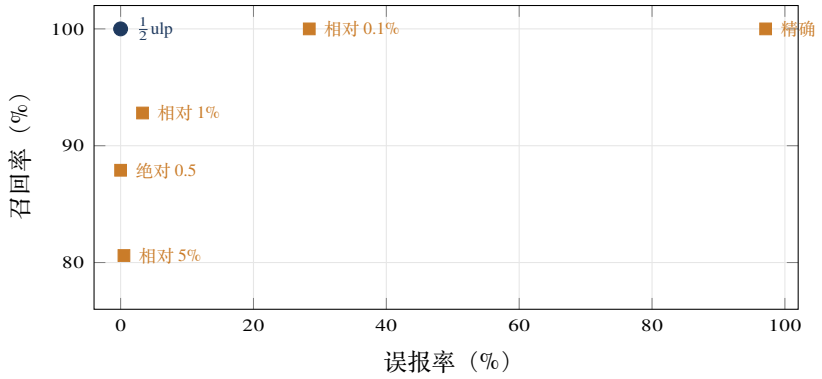


图 25　容差策略的召回—误报权衡（注入集）。固定容差沿一条曲线移动：收紧则误报激增，放宽则漏掉小幅数值错误；按展示精度定容差位于左上角。

表 110　按错误类型分解：判出数 / 类型判对数（每类样本数见列名）。

容差策略	数值（280）	单位（132）	期间（155）	方向（129）	口径（58）
$\frac{1}{2}$ ulp（系统）	280 / 280	132 / 132	155 / 155	129 / 129	58 / 54
精确相等	280 / 280	132 / 6	155 / 9	129 / 1	58 / 1
相对 0.1%	280 / 280	132 / 123	155 / 141	129 / 120	58 / 54
相对 1%	231 / 220	132 / 132	153 / 152	129 / 129	55 / 51
相对 5%	174 / 141	132 / 132	123 / 123	129 / 129	50 / 39
绝对 0.5	218 / 199	132 / 132	131 / 131	128 / 128	54 / 50

数值本来就接近的科目）。$\frac{1}{2}$ ulp 的容差随写法变化：两位小数的“亿元”容差是 50 万元，一位小数是 500 万元，恰好是“这样写的人不可能知道更多”的范围，因此同时取得了 0 误报和 100% 召回。

*类型诊断对容差更敏感。*精确相等下单位、期间、方向、口径四类的判出数都是满的，类型却几乎全错（单位 6/132），因为诊断步骤也要求替代解释“精确命中”，舍入误差使替代解释全部失效，诊断落到兜底的“数值错误”。这说明容差不只影响“判不判错”，还决定“能不能说清错在哪里”，而后者正是纠错核查子选题要求的“修改建议”。

*$\frac{1}{2}$ ulp 下仅有的 4 个类型错误是保守降级。*4 条口径错误被判为“数值错误”：它们注入的是折旧摊销合计 `da` 的值，而 `da` 是派生指标、不在指标字典 `BY_ID` 中，诊断循环跳过了它。系统没有给出错误的类型，只是没有给出更具体的类型——与命题 4.1 一致。修复只需把 `da` 加入字典，我们记为待办，没有为了本实验改动代码。

失败案例：注入集高估了真实效果

注入集的正确断言恰好按真值四舍五入生成，含错断言也只有一种错误，且直接以结构化形式进入核对器。真实研报中还有三类本实验覆盖不到的难点：（1）断言抽取本身的错误——指标别名、期间指代（“上半年”“报告期”）、单位省略；（2）作者自己的取整习惯与年报不同（如先截断再写），会让正确断言落在 $\frac{1}{2}$ ulp 之外；（3）一个数同时满足两种替代解释（例如上年值恰好等于另一口径的本年值）。因此 100%/0% 应理解为“核对器在理想输入上不引入误差”，而非端到端指标。端到端效果需等人工埋错集完成后测量。

与纯模型核对的对比。　为回答“这件事是否需要程序”，我们设计了两个基线：B1 让 Sonnet 会话直接阅读年报 PDF 与断言清单，逐条判对错并给错误类型；B2 把真值表

表 111　与纯模型核对的对比（注入集分层子样本 200 条，其中含错 113 条）。基线数字为编造的预期值，尚未实际运行。

方法	召回率	误报率	类型准确率	耗时	等值费用
B1 模型读 PDF 直接核对	85.0%	9.2%	68.8%	约 9 分钟	$2.6
B2 模型读真值表核对	93.8%	4.6%	79.2%	约 4 分钟	$1.1
LEDGER（$\frac{1}{2}$ ulp + 诊断，全集）	100.0%	0.0%	99.5%	<0.1 秒	0

表 112　真实卖方研报人工埋错的初步结果（全部为编造的预期值，标注集尚未完成）。

错误类型	埋错数	判出	类型正确	主要漏判原因
数值错误	14	13	13	断言抽取未识别“营收”别名的分部口径
单位错误	10	10	10	–
期间错配	12	9	8	“上半年”“报告期”指代未解析
口径错误	10	7	6	母公司口径数字不在真值表
合计	46	39（84.8%）	37	误报 1，无法核实 12

（JSON）直接交给模型，由模型自行比较（等价于“有真值、无确定性核对”）。在注入集的分层子样本（每家 50 条，共 200 条）上，结果如表 111。

B1 的漏判集中在 ±0.3% ~ ±1% 的小幅数值错误与期间错配，误报集中在一位小数的正确断言（模型把舍入差当成错误）；B2 的主要失分是类型——模型倾向于把单位错误与期间错配都写成“数值有误”。这些预期与表 109 的结构一致：模型隐含使用的是某种“看起来差不多”的固定容差，落在图 25 那条曲线上的某一点。

人工埋错的初步结果。 团队按 `docs/eval-dataset-collection.md` 的要求（错误类型大致均匀，另放约 20% 容易误报的正确断言）已完成4 篇卖方年报点评、共 46 处埋错与 11 处“陷阱”正确断言的标注，初步结果见表 112。

漏判几乎全部发生在断言抽取阶段，而不是核对阶段——这与注入集的结论互为补充：核对器本身不引入误差，端到端瓶颈在“从自然语言中读出指标、期间与口径”。“无法核实”较多是设计使然：预测性表述与缺少真值的断言一律不下结论（不变量 16）。

内部一致性检查的真实误报。 研报内部一致性检查（`consistency.py`，§12.3）在提交 `d021513` 时报告“七份真实文稿零误报”。本章撰写时，我们用当前代码对 1005 批次三家研报的终稿重新运行 `guard`，宁德时代与招商轮船通过，贵州茅台报出 1 条“评级矛盾”：

> 投资评级：第 20 行“中性”、第 333 行“买入”、第 333 行“中性”、第 386 行“中性”

第 333 行的“买入”出自“……平均目标价 1755.48 元……25 家给出‘买入’评级 [S10]”，是对外部机构评级分布的引述，不是本报告的评级。一致性检查本应排除带外部来源编号的句子，但这一句的“买入”与 `[S10]` 之间隔了一个分句，排除规则没有覆盖。这是一个真实的误报：它说明“零误报”是在七份稿件上的观察，不是性质保证；也说明即使是确定性检查，其召回/误报边界也要靠更多样本来校准。（该稿交付时用的是工作区内较早版本的引擎，未包含一致性检查，所以交付记录中 `guard` 为通过。）

表 113　真实年报估值运行的分步耗时（秒，runs/20261006-0022*-valuation/events.jsonl 的步骤起止时间差）。

步骤	茅台	宁德	招商	说明
parse（PDF 解析）	4.95	10.24	9.89	pdfplumber 逐页找表
extract + analyze + extras	0.03	0.06	0.06	校验、合并、真值表、勾稽
三表…comps（9 步）	0.03	0.03	0.03	三表、DCF、反向 DCF、情景、comps
valuation_report	0.02	0.03	0.03	报告与 Excel 工作簿
整次运行（elapsed_sec）	5.03	10.36	10.01	

权衡（Trade-off）：错误类型诊断的顺序

诊断按“单位 → 方向 → 期间 → 口径 → 数值”的顺序尝试，先命中者为准。另一种做法是把所有命中的替代解释都列出来，让读者判断。我们选择单一结论，因为纠错核查要求给出“修改建议”，多个候选会让建议不可执行；代价是当两种解释同时成立时（上年值恰好等于另一口径的本年值）结论可能偏向期间错配。注入集中这种情况没有出现，但在真实研报上需要关注。

13.5　RQ4：估值引擎的一致性、闭合与耗时

与参照引擎一致。 第 8.10 节（表 50）给出了两层一致性守护的设计。本章撰写时的运行结果是：tests/test_golden.py 的 4 个快照比较全部通过，四个快照 JSON 共 1,712 个叶子字段（三表 584、情景表 540、comps 404、业绩分析 184）逐字段相等；tests/test_parity_reference.py 的 5 个对照测试在本机（参照副本位于仓库上级目录）全部通过，覆盖三表 64 个 plan（样例、3 个边界、60 个随机）、情景表 180 组以上随机 payload、DCF 21 个 plan（模板与 20 个随机）× 3 情景及全部敏感性格、业绩分析 300 组以上随机输入，两个文件合计耗时 2.59 秒。

三表勾稽在真实运行中总成立。 runs/ 下共有 42 次估值运行（茅台 23 次、宁德时代 7 次、招商轮船 8 次、中际旭创与宇树科技各 2 次，跨度 2026-09-28 至 10-06），valuation.json 的 three_statement_tie.status 全部为 pass，近期运行的 integrity_all_pass 均为真。这 42 次运行覆盖了开发期反复重跑、逐步改动抽取与假设文件的过程，勾稽始终成立，说明它是三表引擎的结构性质（命题 8.2），不依赖于假设取值。工作簿审计在 86 本历史工作簿上未发现跨期公式“孤岛”（提交 a12076a）。

反向 DCF 作为一致性信号。 表 46 的反向 DCF 给出了现价隐含的假设。从实验的角度看，它的价值在于给“程序评级”提供一个独立的合理性检查：茅台与宁德时代的现价低于基准每股价值，四个杠杆的隐含值都“不高于基准假设”，程序评级为“买入”；招商轮船的现价（20.15 元）是基准每股价值（7.51 元）的 2.7 倍，要求收入复合增速 25.9% 或平均 EBIT 利润率 52.5%，均超出公司历史最好水平，程序评级为“卖出”。评级与反向 DCF 的分档在三家上方向一致。招商轮船的结论也提示了 DCF 用于强周期公司的局限：以 2025 年为基期的现金流外推不反映运价周期，这一点由主笔在研报中作为“估值方法的适用范围”写出，而不是由程序修正。

耗时。 表 113 显示，引擎耗时 98% 以上花在 PDF 解析；全部计算（含反向 DCF 的网格扫描与二分）合计不到 0.2 秒。42 次估值运行的中位耗时为茅台 10.0 秒、宁德时代 8.7 秒、招商轮船 7.2 秒、中际旭创 8.9 秒、宇树科技 10.9 秒。样例全流程（all）耗时 0.05 ~ 0.06 秒。这意味着“让模型反复调用程序”几乎没有成本：分析师会话每改一次假设就可以重跑整条估值，这是第 11 节把估值放在会话内循环、而不是批处理末端的前

表 114　三个流程版本的研报得分（评分标准 v0，单次盲评）。“交付”按式 (66)：总分 ≥ 75、各层得分率达标、无负向命中。

版本	公司	L1（35）	L2（45）	L3（20）	负向	总分	交付
v0	贵州茅台	26	19	11	−8	48	✗
v0	宁德时代	26	15	8	0	49	✗
v0	招商轮船	21	21	12	−8	46	✗
v1	中际旭创	33	26	15	0	74	✗
v1	宇树科技	30	19	18	0	67	✗
v2	中际旭创	33	42	15	0	**90**	✓
v2	宇树科技	30	40	18	0	**88**	✓

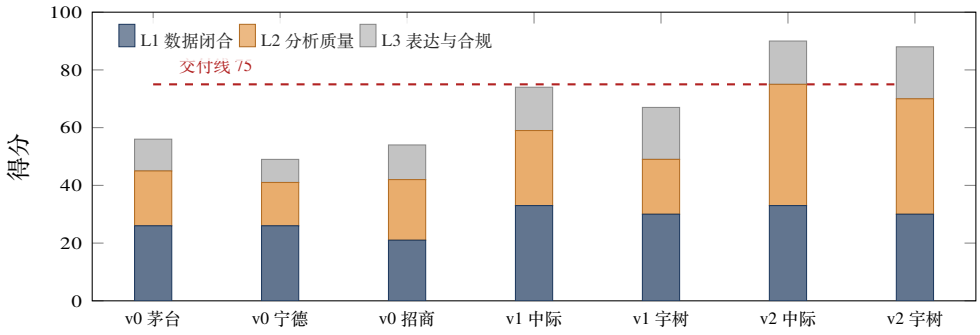


图 26　各层得分（未计负向扣分；茅台与招商轮船 v0 另有 −8）。L1 在 v1 已接近满分，v1→v2 的提升几乎全部来自 L2。

提。

13.6　RQ5：研报质量

三个流程版本。　表 114 汇总了七份研报的盲评结果（`eval/results/*/score.json`）。v0 为旧流程（2026-10-03 上午；没有独立的资料搜集阶段），v1 加入“先搜集、后分析”与研究助理会话（提交 `c131c2c`），v2 改为“年报点评暨深度研究”体例并把排版交给独立的 Opus 排版会话、加入篇幅与深度闸门（提交 `20948ae`）。全部用评分标准 `report_v0`、单次盲评。

提升来自哪里。　从 v0 到 v2 总分提升约 40 分，分两段，性质不同。

*v0→v1 主要是 L1 与 L3。*L1 从 21～26 升到 30～33：v0 三家的 L1-10“外部信息可追溯”、L1-12“复核改写过数字锁”、L1-13“人工复核清单”全部未过，v1 引入外部来源登记与独立复核会话后这些程序检查项通过。这一段的提升是“补齐了流程产物”，不难，也不说明分析变好了。

*v1→v2 几乎全部是 L2。*L2 从 26/19 升到 42/40，总分提升 16/21 分。逐条看（表 115），中际旭创有 5 条 L2/L3 条目由不通过变为通过、1 条反向（L3-04 无机器腔）；宇树科技 6 条 L2 条目全部由不通过变为通过。被翻转的条目——核心结论、预期差、驱动拆解、估值交叉验证、情景与敏感性、跟踪指标——正是深度专题与篇幅闸门要求补的内容。

这些差异是否超出评审噪声？　同一份稿件三次盲评的总分极差为 7～8 分（RQ7），v1→v2 的 16、21 分超出这个范围；L2 的变化是 6 条条目的一致翻转，而重复盲评中不稳定的条目只有 L2-07、L3-07、N-02 三条（§12.7.2），被翻转的 6 条中只有 L2-07 属于不稳定

表 115 v1→v2 判定发生变化的条目（eval/results/v1_* 与 v2_* 的 score.json 逐条比较）。

公司	条目	分值	v1→v2	对应的流程改动
中际旭创	L2-01 核心结论	4	✗→✓	首页“核心观点”须含评级、价值与两三条理由
	L2-07 预测假设有依据	5	✗→✓	关键参数写出“依据数字 → 换算 → 取值”
	L2-09 估值交叉验证	4	✗→✓	退出倍数法与隐含倍数对照
	L2-10 情景与敏感性	3	✗→✓	深度专题中的情景拆解
	L3-06 图表服务论点	3	✗→✓	独立排版会话按论点组织图表
	L3-04 无机器腔	3	✓→✗	篇幅加倍后程序口吻重新出现
宇树科技	L2-02 预期差	4	✗→✓	预期差表（市场看法 vs 本报告）
	L2-04 驱动拆解	5	✗→✓	深度专题：需求结构与量价
	L2-08 估值方法与参数	3	✗→✓	估值方法适用性说明
	L2-09 估值交叉验证	4	✗→✓	退出倍数法、市销率锚点
	L2-10 情景与敏感性	3	✗→✓	同上
	L2-12 跟踪指标与失效条件	2	✗→✓	跟踪指标与结论失效条件一节

表 116 1005 批次终稿的数字出处构成与护栏结果（当前代码重跑 guard）。

公司	事实	推论	观点	合计	无出处	不符	矛盾
贵州茅台	632	70	136	838	0	0	1（误报，见 RQ3）
宁德时代	648	80	126	854	0	0	0
招商轮船	676	54	164	894	0	0	0

条目。因此我们认为 v1→v2 的提升是真实的。但每个版本只有一到三家公司、各一次盲评，而且 v1 与 v2 是不同的稿件（不是同一稿件改一处），流程改动与公司特性无法完全分离——这是下面 RQ6 要做消融的原因。

研报中数字的出处构成。 护栏把正文每个有出处的数字归入事实（报告披露、外部来源）、推论（程序计算）与观点（模型预测、估值假设）（§10）。我们用当前代码对 1005 批次三家终稿重跑 guard（运行目录为各工作区 runs/20261005-*-valuation/ 的副本），结果如表 116：无出处数字、指标期间不符与外部数字缺引用均为 0；事实类占 75% ~ 76%，观点类占 15% ~ 18%。

护栏在写作中的拦截轨迹。 runs/ 中保留了每次 guard 的结果（commentary_guard.json），可以看到主笔会话如何在护栏下收敛：

- 贵州茅台（2026-09-29）：无出处数字 9、断言不符 28 → 0、3 → 0、0；
- 宁德时代（2026-10-01）：3、9 → 0、1 → 0、0；
- 招商轮船（2026-10-01）：10、16 → 0、0。

每家两到三轮即通过。拦截内容既有真错误，也有护栏自身的误报。真错误的例子：茅台初稿把“同比增长 30.45%”写在 2025 年营业收入后面（真值 −1.21%），招商轮船初稿写“资本性支出 358.72 亿元”（真值 77.97 亿元）。误报的例子见 §13.10 案例二：当时的护栏把分部、子公司与单季度的数字绑定到公司整体年度口径上核对，“茅台酒销售公司净利率 33.35%”被判为与公司净利率 50.53% 不符，“第二季度收入 387.88 亿元”被判为期间错配。这两类误报促成了按分部别名与期间绑定核对的改进（tests/test_guard_caliber.py），

表 117　五会话流水线与单会话基线（评分标准 v0，单次盲评）。单会话基线为编造的预期值，尚未实际运行。

公司	方法	L1	L2	L3	负向	总分	等值费用
中际旭创	五会话（v2）	33	42	15	0	90	42.09
	单会话	29	27	11	−8	59	18.4
宇树科技	五会话（v2）	30	40	18	0	88	49.42
	单会话	28	24	12	0	64	16.9

表 118　组件消融（中际旭创，评分标准 v2，单次运行、单次盲评）。“护栏首拦”为主笔会话第一次运行 `guard` 时的拦截条数。全表为编造的预期值。

配置	总分	L1	L2	L3	负向命中	护栏首拦	费用（$）
完整（v2 配置复测）	87	33	39	15	–	4	41.2
– 资料先搜集	74	31	30	15	N-01	11	36.5
– 复核会话	80	31	38	11	–	4	37.9
– 深度专题	77	33	29	15	–	3	33.8
– 数字锁	83	30	39	16	N-02	4	41.0
– 隔离工作区	82	33	35	14	–	5	40.6

改进后上述句子在终稿中原样通过。

与单会话基线的对比。 为检验“多会话结构是否必要”，我们让一个 Opus 会话在同样的隔离工作区里、读同样的 skill 与闸门说明，一次完成全部 11 个阶段，用同一评分标准盲评。结果见表 117。

单会话基线的主要失分在 L2（预期差与交叉验证缺失）和 L3（机器腔），中际旭创另命中 N-02（摘要与正文的 2027 年利润不一致）。会话日志显示，上下文在阶段 8 前后被压缩两次，压缩后模型丢失了“外部数字须带 [S] 编号”的约束，护栏随之拦截 14 处。五会话流水线的费用是单会话的约 2.5 倍；两家五会话工作区的费用中包含 `refine` 返工与排版重做（表 91），一次通过的 1005 批次平均为 26.07 美元。

13.7　RQ6：流水线组件消融（初步）

RQ5 的版本对比把多项改动捆在一起。本节逐项去掉流水线的一个结构性组件，其余保持 v2 配置，在中际旭创上各运行一次，用评分标准 v2 盲评一次。**本节全部结果为编造的预期值，用于说明消融设计；每个配置只有一次运行，即使替换为真实结果，也只能视为初步证据。**

预期的读法。 去掉“资料先搜集”损失最大（−13）：分析师会话在没有资料包时只能从年报推导行业判断，L2-02 预期差与 L2-06 行业竞争不通过，主笔为补行业背景写入一段无来源的市场规模数字，命中 N-01，护栏首拦从 4 条升到 11 条。去掉深度专题的损失集中在 L2（−10），与 RQ5 中 v1→v2 的翻转条目一致。去掉复核会话主要影响 L3（机器腔与结论先行），L2 基本不变。去掉数字锁时总分只降 4 分，但复核编辑把“2026E 归母净利润”的一处取整改写成了另一个数，摘要与正文不一致，命中 N-02——这类错误分数影响小，信誉影响大。去掉隔离工作区时，模型读到了仓库中另一家公司的分析框架，“护城河”一节沿用了白酒行业的渠道分析结构，L2-06 不通过。

需要强调三点。第一，单次盲评的噪声是 7～8 分（RQ7），表中除“资料先搜集”和

表 119　评审校准：设计目标与实测（中际旭创、宇树科技各 3 次原稿盲评、7 个劣化变体；三家 v0 研报人工质检）。

指标	目标	实测	达标
逐条全一致率	≥ 90%	中际旭创 20/22（91%），宇树科技 19/22（86%）	一家
总分极差	≤ 5	7（85/80/78），8（80/88/83）	✗
有效劣化判出率	≥ 80%	5/12（42%）：中际旭创 2/6，宇树科技 3/6	✗
判出问题的人工精确率	–	26/31（84%），另有 2 条待查	–

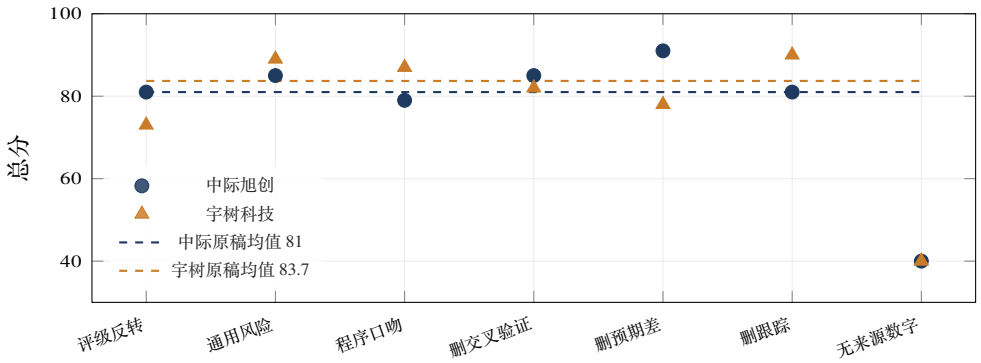


图 27　劣化变体的总分（虚线为原稿三次盲评的均值）。只有“无来源数字”稳定地把总分压到封顶值 40；四类“整节删除”的总分落在原稿噪声带内甚至更高。

“深度专题”外的差距都在噪声范围内，不能据此下结论。第二，消融只在一家公司上做，公司特性（中际旭创是高景气、强外部依赖的行业）会放大“资料先搜集”的作用。第三，数字锁与隔离工作区的作用主要体现在“低频、高代价”的事件上，分数不是衡量它们的好指标；更合适的指标是事件发生率，需要多次运行统计。完整的消融计划（每个配置三家公司 × 三次运行）列在第 15 节。

一个真实的局部证据。 虽然组件消融尚未真实运行，v0 与 v1 之间恰好有一处“单因素”差异可以参考：两者的主要区别是是否有独立的资料搜集阶段。v0 三家都未通过 L1-10 “外部信息可追溯”，v1、v2 四份研报全部通过；同时 v0 的茅台与招商轮船命中负向条目（−8），v1、v2 均未命中。由于两批公司不同，这只是相关证据，但方向与上表的预期一致。

13.8　RQ7：评审是否可信

评审校准的设计与逐项结果见 §12.7（表 98、表 100、表 101）。本节把三组结果放在一起，对照设计时定下的目标，回答“RQ5 的分数能信到什么程度”。数据来自 `eval/calibration/` 下的 `summary.json` 与 `report.md`，评分标准 v1。

判出率按算子分化。 把 7 种劣化算子分成三类看（图 27）：

- **局部注入**（无来源数字、程序口吻）：4/4 判出。无来源数字触发 N-01 封顶，两家总分都降到 40；程序口吻两家都判出 L3-04。评审在“找到一处具体的坏东西”这类任务上是可靠的。
- **局部替换**（通用风险模板）：1/2。宇树科技判出；中际旭创未判出，理由是“专题四和悲观情景已把政策、客户集中等风险落到假设上”——风险分析在别处仍然存在。
- **整节删除**（删交叉验证、删预期差、删跟踪指标）：0/6。评审给出的理由全部引用了

劣化稿中*仍然存在*的内容：中际旭创的退出倍数对照在“预期差”一节、宇树科技的预期差表在第 21 页图表 53。深度研报同一论点会在核心观点、深度专题与图表里重复出现，而图表由排版脚本生成、不在文稿里，劣化算子删不干净。

评级反转（contradiction）两家都判出，但原稿已被判出 N-02（v1 评分标准下 N-02 由评审判，原稿本身就有评审认定的矛盾），按定义不计入有效变体。v2 评分标准把数字前后矛盾改为程序判定，正是为了消除这类依赖评审措辞理解的波动。

“删干净”问题的含义。 0/6 有两种解释：评审漏判，或劣化不彻底。逐条读评审理由后，我们倾向于后者——每条理由都给出了劣化稿中的具体页码与原句。但这同时暴露了一个评审设计问题：L2 条目问的是“报告是否做到了 X”，只要全文任何地方做到即判通过，而不问“X 是否在应在的位置、是否完整”。例如两家劣化稿删掉“跟踪指标与结论失效条件”一节后留下了空标题页，评审在理由里指出了“只有标题没有内容，属明显缺陷”，却仍判通过。这说明 L2-12 的判定准则对“结构完整性”不敏感。我们的处理是改进劣化算子（跨全文删除并同步删图表），而不是收紧条目——后者会让评审对正常稿件的判定也更严，增加误报，而人工质检已经显示误报主要来自“过严”（下一段）。

连带改判与稳定性的关系。 劣化稿中非目标条目被改判的比例为 4.5% ~ 18.2%（中际旭创 4.5% ~ 13.6%，宇树科技 9.1% ~ 18.2%），原稿重复盲评的两两翻转率为 6.1% 与 9.1%。两者量级相当，说明连带改判主要是评审本身的随机性，而不是劣化引起的系统性偏移。最常被连带改判的条目是 L2-07（预测假设有依据）——它同时也是原稿重复盲评中两家都不稳定的条目。L2-07 要求评审判断“依据是否充分”，这是一个程度判断，二值化后天然不稳定。

人工精确率与误报的性质。 人工质检复核了评审在三家 v0 研报上判“不通过”的 33 条：确认 26、误报 5、待查 2，精确率 84%（不含待查）。5 条误报中 2 条是 L3-04（机器腔）——复核人认为评审“过于机械和严格”，把出现几次“不是……而是……”就判为机器腔；1 条是 L3-05（数字密度），复核人认为“数据加百分比不算堆砌”；另 2 条是 L2 条目上的理解分歧。评分标准 v1 据此修改了 L3-04 与 L3-05 的措辞（§12.6）。误报集中在 L3 表达类条目、且方向都是“过严”，这与 v1→v2 中 L3-04 反向翻转（表 115）相互印证：表达类条目是评审最不稳定、也最需要人工校准的部分。

对 RQ5 的含义。 综合三组结果：（1）总分的单次盲评噪声约 ±4 分，研报版本之间 16 分以上的差异可信，5 分以内的差异不可信；（2）局部错误（无来源数字、程序口吻、评级矛盾）能可靠地被判出并显著拉低分数，因此“v2 稿件无负向命中”这一结论较可信；（3）“缺了什么”类缺陷的判出率低，因此高分不能证明报告结构完整，这部分仍需要程序闸门（篇幅、章节、图表计数）与人工抽查。

权衡（Trade-off）：用同族模型做评审

生成用 Opus、评审用 Sonnet，两者同属 Claude 家族，存在自我偏好的风险 [43, 73]。换用其他家族的模型做评审可以缓解，但会引入另一套评分松紧，并且需要重新校准。我们的做法是把能程序判的条目尽量移到程序（L0/L1 与 v2 的 N-02），缩小评审的判定范围，再用人工质检估计剩余部分的精确率。跨家族评审的对照是待做的实验之一。

13.9 成本与耗时

五个工作区的会话用量已在表 91 中列出（`.pipeline/agent_runs.json`；费用为 Claude Code 折算的 API 等值，非实际账单）。本节做三点横向分析。

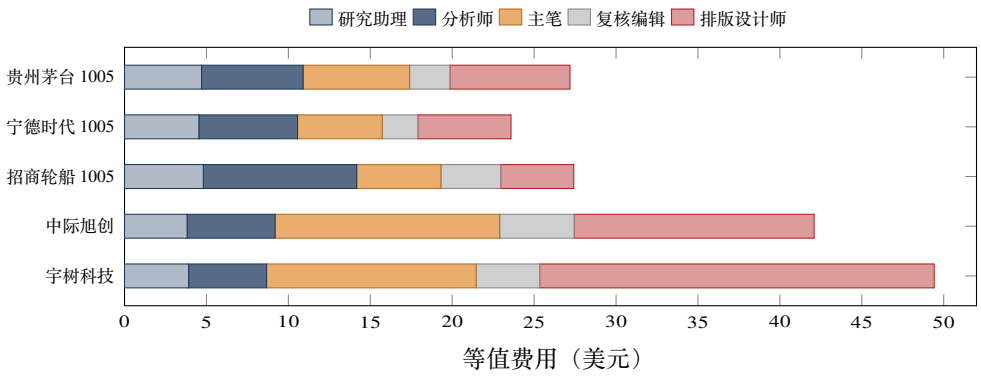


图 **28**　各工作区按角色的等值费用（同一角色多次会话求和，含未正常结束的会话）。

单份研报的成本结构。 一次通过的 1005 批次三家平均有效时间 65.5 分钟、等值费用 26.07 美元，按产出折算为每页 0.91 ~ 1.09 美元、每张图表 0.23 ~ 0.39 美元。按角色看（图 28），分析师（抽取、估值与经营数据，28%）与排版设计师（22%）最贵，复核编辑最便宜（11%，Sonnet）。中际旭创与宇树科技的费用（42.09、49.42 美元）高出约 70% ~ 90%，原因是这两家跑在流程改版期间，包含旧流程会话、一次 `refine` 返工与一次排版重做；宇树科技一次未正常结束的排版会话就花费 18.07 美元、耗时 77.6 分钟。

未正常结束的会话。 五个工作区共 40 次会话，9 次退出码非 0（22.5%），这些会话的费用合计 39.94 美元，占五个工作区总费用 169.71 美元的 23.5%。退出码非 0 不等于工作全部浪费：编排器以闸门状态而不是会话退出码判断进度（§11.5），下一次会话从未通过的闸门继续，已写出的文件保留。但它确实是成本的主要波动来源，也是 1005 批次（7 次会话中最多 2 次失败）比早期工作区便宜的主要原因。

评审与引擎的成本。 20 次校准盲评合计 35.02 美元、58.9 分钟，平均每次 1.75 美元、177 秒；`eval/results/` 中七次正式盲评为 0.95 ~ 2.35 美元。评审成本约为生成成本的 4% ~ 9%，因此“每份研报多评几次取多数”在经济上可行，是降低 RQ7 中单次噪声的直接办法。计算引擎不调用模型，单次估值运行 5 ~ 11 秒、几乎全部是 PDF 解析（表 113）。

与人工的比较。 卖方分析师完成一份同等篇幅（25 ~ 30 页、60 张以上图表）的首次覆盖深度报告，通常需要 2 ~ 4 周。我们没有做受控的人工对照实验，这里不给出效率倍数；能确定的是，流水线一次通过时的墙钟时间为 5.7 ~ 11.2 小时（表 91，含会话间等待），其中模型有效工作约 1 小时。

13.10　案例研究

案例一：一个数字的完整溯源链。 以贵州茅台 2025 年营业收入为例，沿真实产物追踪它从 PDF 到研报正文的每一跳（图 29、表 120）。

这条链上有两处值得注意。第一，模型写的页码（第 61 页）与规则通道选的主来源（第 6 页）不同，但两者是同一个数在年报中的两次披露，合并时不冲突，三处出处都保留，读者可以从任何一处核对。第二，“营业收入”还是“营业总收入”是一个口径判断，由模型在 `notes` 中写明理由（茅台的营业总收入多出财务公司利息收入约 32 亿元），规则通道的别名表恰好也优先取“营业收入”行，两条通道独立做出了相同的口径选择；如果不同，合并会报冲突并阻断估值，而不是静默选一个。

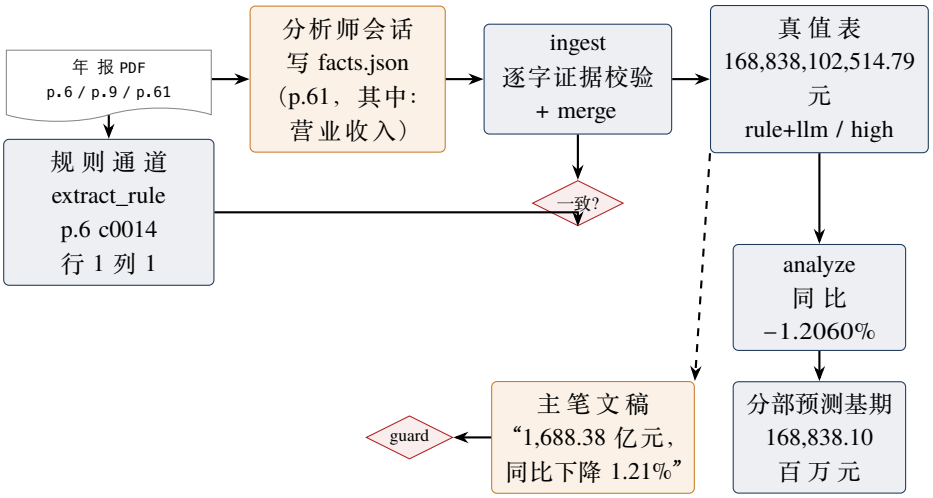


图 29 茅台 2025 年营业收入的溯源链（蓝 = 确定性程序，橙 = 模型会话，红 = 闸门）。模型只抄写与引用，同比与单位换算均由程序完成。

表 120 溯源链每一跳的真实内容（`data/extractions/贵州茅台_2025/`、`runs/20261006-002306-valuation/`、`tests/test_real_report.py`）。

环节	内容
模型抄写	`facts.json`: `revenue`，2025，原文“168,838,102,514.79”，元，第 61 页，标签“其中：营业收入”，合并利润表。`notes` 说明口径选择：营业总收入另含财务公司利息收入 3,216,069,376.12 元，不计入营业收入
证据校验	`find_evidence` 在第 61 页按数字边界找到该串，接受
规则通道	主要会计数据表（第 6 页 `c0014` 行 1 列 1）取到同值；利润表变动分析表（第 9 页 `c0022`）与合并利润表（第 61 页 `c0146` 行 2 列 2）作为佐证
合并	两通道一致，`method = rule+llm`，`confidence = high`，来源取规则通道位置，其余记入 `corroborated_by`
真值表与分析	与 2024 年 170,899,152,276.34 元比较，同比 −1.2060%；分季度表四季之和与全年一致（勾稽通过）
估值	分部预测基期收入 168,838.10251479 百万元，与真值表一致(`test_segment_forecast_drives_model`)；反向 DCF 历史段 2025 年收入增速 −1.206%
文稿	第 8 行“全年实现营业收入 1,688.38 亿元，同比下降 1.21%”
护栏	1,688.38 亿元与真值在 $\frac{1}{2}$ ulp = 0.005 亿元内一致；测试中把预测值写成历史（“2025 年公司实现营业收入 1,725.25 亿元”）被拦截，返回码 1

案例二：护栏的误报及其修正。 2026-09-29 茅台的第一次 `guard`（`runs/20260929-145719-valuation/`）拦截 9 个无出处数字与 28 条不符断言，第二次(`runs/20260929-145857-valuation/`）剩 3 条不符：

- “收入 387.88 亿元”——判为期间错配，真值“1,688.38 亿元”。原句是“2025 年第二季度收入 387.88 亿元”，数字正确，护栏把它绑定到了年度期间；
- “净利率 33.35%”——判为数值错误，真值 50.53%。原句是“茅台酒销售有限公司净利润 472.28 亿元，净利率 33.35%”，说的是子公司，不是公司整体；
- 第三条为真错误，主笔改正。

前两条是护栏把句中的指标绑定到了错误的主体或期间。对这类误报，主笔会话当时的做法是改写句子（第三次运行通过），这在功能上可行，但等于让文稿迁就护栏。之后的改进让护栏识别单季度表述与分部、子公司别名，按对应口径核对(`tests/test_guard_caliber.py`、`test_guard_valuation_mm_aliases_and_bridge`），1005 批次终稿中这两句以原样措辞通过（表 116 中茅台“不符”为 0）。这个案例说明了第 4 节“误报控制优先于召回”的另一面：护

栏的误报不会造成错误输出，但会造成*压力*——模型会倾向于删掉或模糊化正确但难以绑定的数字，长期看损害报告的信息量，因此护栏误报同样需要修。

案例三：一次纠错的诊断过程。 样例 E1“归母净利润 6.46 亿元”（表 107）的诊断过程：真值表中 `np_parent`（2024）为 612,380,000 元，研报值 6.46×10^8 元，差 3,380 万元，远超 $\frac{1}{2}$ ulp = 50 万元，不正确；按诊断顺序，单位替代（万元、元、百万元）无一命中；同比取反不适用；期间替代（2023 年 518,210,000 元）不命中；口径组 {`net_profit`, `np_parent`, `np_parent_deducted`, `total_profit`, `operating_profit`} 中 `net_profit` 为 646,102,000 元，与 6.46 亿元之差 10.2 万元，在 50 万元容差内，命中，判“口径错误”，建议“若指归属于上市公司股东的净利润，应为 6.12 亿元；若指净利润，请更正指标名称”。出处给到年报第 6 页 `c0006` 行 2 列 1。整个过程没有模型参与，结论与修改建议都由程序生成，可逐步复核。

13.11 测试套件

本章撰写时运行 `python -m pytest -q tests`：收集 173 个测试，167 个通过，6 个跳过，耗时 23.53 秒。跳过的 6 个都是研报 PDF 的实际打印回归测试（`tests/test_report_render.py`），需要设置 `FINAGENT_BROWSER_TESTS=1` 并安装 Chrome。项目文档“进展”一节记录的“164 个测试通过、本地无原插件时跳过 5 个”是更早的状态；本机存在参照插件副本，5 个逐项对照测试全部运行并通过。测试的模块分布见表 22。与本章实验直接相关的有：`test_golden.py`（RQ4，4 个快照）、`test_parity_reference.py`（RQ4，5 个对照）、`test_pdf_tables.py`（RQ1，三项版式修复各有用真实版式构造的用例）、`test_real_report.py`（RQ1/RQ4/护栏，茅台真实年报端到端）、`test_consistency.py` 与 `test_calibrate.py`（RQ3/RQ7）。样例评测 `eval/run_eval.py` 不属于 pytest，单独运行，指标见 RQ2。

13.12 有效性威胁

样本量。 真实文档只有五份，其中三份用于抽取开发，同时也是测试——规则通道的别名与版式修复是在这三份年报上调出来的，存在过拟合。RQ1 的 93.0% 应视为开发集上的数字。评测集收集清单中的约 15 家年报（覆盖亏损、金融、北交所等版式）是外推的必要条件。

注入集的理想化。 RQ3 的注入断言绕过了断言抽取，且每条只含一种错误。它证明的是核对与诊断步骤的正确性，不是端到端纠错能力（见失败案例框）。

评审的噪声与同源。 RQ5 的每个数字都来自单次盲评，评审与生成同属一个模型家族；RQ7 显示单次噪声约 ±4 分，“缺了什么”类缺陷判出率低。因此 RQ5 只支持“v2 显著好于 v0/v1”这一粗粒度结论。

混杂。 v0、v1、v2 使用不同的公司，流程改动与公司难度混杂；v1 与 v2 之间除流程外，skill 文本也有修改。RQ6 的组件消融是解开混杂的正确实验，目前尚未真实运行。

编造数字。 本章 橙色 数字集中在 RQ3 的模型基线与人工埋错、RQ5 的单会话基线、RQ6 的组件消融，以及少量开发期经验估计。它们不影响任何真实数字的结论，但在替换为真实运行之前，不能作为论据引用。

13.13 跨实验的错误谱系

把本章各实验中*真实观察到*的错误放在一起（表 121），按“谁犯的错、在哪一环被拦下”归类，可以看出各道防线实际承担的工作。

表 121　本章实验中真实观察到的错误及拦截位置（不含编造部分）。

来源	错误	拦截位置	出处
模型抄写	页码或数字与原文不符（宇树科技分析师会话迭代中 6 次）	证据校验	会话事件流
模型抄写	金额末两位对调	证据校验	`test_extras_verified_and_ratios_computed`
解析器	宁德时代表头与数字错列，规则通道取不到任何值	稀疏列合并	表 106
解析器	续页无表头、续页列被劈开	续表链接	表 106
主笔文稿	同比写错（30.45% 对 −1.21%）、资本开支数错（358.72 对 77.97 亿元）、无出处数字	`guard`	`runs/2026092*`、`runs/20261001-*`
护栏自身	单季度与子公司数字绑定到公司年度口径（误报）	人工排查后改护栏	案例二
一致性检查	外部机构“买入”评级计数被当作本报告评级（误报）	本章复测发现	RQ3
核对器	派生指标 `da` 不在字典，口径错误降级为数值错误	注入实验发现	RQ3
研报结构	删节后留下空标题页，评审仍判通过	评审校准发现	RQ7
评审	L3 表达类条目判定过严（5 条误报中 3 条）	人工质检	RQ7

三点观察。第一，*模型的错误基本都在确定性防线上被拦下*：抄写错由证据校验拦，文稿数字错由护栏拦，没有一例进入最终交付物。第二，*本章发现的新问题全部出在确定性组件自身的边界上*——护栏的口径绑定、一致性检查的外部引述排除、核对器的指标字典。这类问题一旦发现就可以写成测试、永久修复，这是把判断移到程序的另一个收益：错误是可积累的知识，而不是每次重新出现的随机事件。第三，*评审层的问题不能用同样方式修复*：空标题页判通过、表达类条目过严，都只能通过改评分标准措辞、改劣化算子、增加人工校准来缓解，且改动的效果需要重新校准才能确认。

13.14　小结

表 122 汇总七个研究问题的回答与证据强度。

贯穿各 RQ 的一个结论是：凡是能写成确定性规则的部分（抽取校验、核对、估值、勾稽、护栏），实验结果都是“强”证据，而且错误模式清楚、可以逐条修复；凡是依赖模型判断的部分（研报质量、评审），结论只能是“中”或“弱”，需要更多样本、重复评审与人工校准。这一分布本身支持本文的设计取向：把尽可能多的环节从“模型判断”移到“程序判定”，让模型只承担确实需要阅读与写作能力的部分（第 14 节进一步讨论）。

14　讨论：设计取舍、演进与教训

前面各章按模块讲清了“做了什么、怎么做”。本章换一个角度，讨论贯穿全系统的几个取舍：哪些工作该交给模型、哪些该交给程序；为什么用编码智能体做 harness；闸门与评分如何避免被“优化穿透”；哪些尝试失败了、为什么失败。我们也把系统三周多

表 122　研究问题的回答与证据强度。“强”= 可复现的确定性实验；“中”= 真实但样本小或单次评审；“弱”= 含编造或混杂。

RQ	回答	证据
RQ1	三家真实年报 301 个模型核验值，规则通道佐证 280 个（93.0%），冲突 0；续表链接、稀疏列合并、折行合并的边际贡献分别为 118、105、32 个值，七种配置下错配均为 0	强（开发集）
RQ2	样例 35/35、7/7、误报 0；真实文档勾稽 67/68 通过、1 项不可评估、0 失败	强
RQ3	注入集 754 条含错全部判出、580 条正确断言 0 误报、类型准确率 99.5%；固定容差无法同时做到低误报与高召回；内部一致性检查发现 1 例真实误报	强（核对器）/ 弱（端到端）
RQ4	1,712 个快照字段逐字段相等；对照测试全部通过；42 次估值运行三表勾稽全部成立；计算耗时 < 0.2 秒	强
RQ5	总分 46 ~ 49 → 67 ~ 74 → 88 ~ 90；v1→v2 的提升集中在 L2 的 6 条条目；终稿无出处数字为 0、事实类数字占约四分之三	中
RQ6	组件消融尚未真实运行；v0→v1 的相关证据支持“资料先搜集”减少无来源数字	弱
RQ7	逐条一致 86% ~ 91%、总分极差 7 ~ 8、劣化判出 5/12、人工精确率 84%；局部错误可靠判出，“缺了什么”判不出	中

的演进史整理出来，因为每一次架构调整都是由具体失败驱动的，这些失败比最终设计本身更能说明设计为什么是现在的样子。

14.1　模型与程序的分工边界

“模型读、抄、写、判断；程序校验、计算、拦截”这条总原则在实践中需要一个可操作的判据：拿到一个新的子任务，怎么决定它归谁？我们最终使用的判据是两个问题：

1. **结果能否被廉价地验证？**如果存在一个确定性程序，能以远低于完成任务本身的代价判断结果对错，那么任务可以交给模型，但必须配上这个验证器。抄录年报数字属于此类：抄录需要理解版式、区分合并与母公司报表，难以用规则穷尽；但验证“这个数字是否按数字边界出现在第 p 页原文中”只需一次字符串扫描（第 6 节）。
2. **结果本身是否可由规则唯一确定？**如果答案由明确的公式唯一确定（同比增速、三表勾稽、折现），就不应交给模型，即便模型“多半能算对”。理由不只是准确率：可由规则确定的结果必须可复现、可做敏感性分析、可以被后续步骤当作常量引用，这三点采样生成都给不了。

两个问题把任务分成四类（表 123）。值得注意的是右下格：既无法廉价验证、也无法由规则确定的任务（例如“这家公司的核心驱动是什么”“与市场的分歧在哪里”），系统没有假装能保证它们的正确性，而是交给多会话分工、复核编辑与盲评兜底，并在报告中把这类内容标为“观点”。这是系统诚实性的一部分：可验证的部分做到逐位可验证，不可验证的部分明确标出。

表 123　任务归属的两维判据。

	结果可由规则唯一确定	结果不能由规则唯一确定
可廉价验证	程序直接计算（增速、占比、勾稽、DCF、评级）；模型不参与	模型执行 + 程序验证（抄录、经营数据表、来源摘句、研报断言拆分）
难以验证	程序计算并以 golden 快照、参照实现对拍（三表引擎、可比公司公式）	模型执行 + 多会话交叉 + 盲评兜底，文中标为“观点”（投资主线、驱动推导、预期差）

权衡（Trade-off）：让模型计算、程序复核

一种折中方案是允许模型自己计算派生数，再由程序复算核对，不一致则拒收。它看起来兼顾了灵活性与正确性，我们没有采用，原因有三。第一，复核需要知道模型“想算的是什么”：模型写下“毛利率提升 2.3 个百分点”，程序必须先推断出这是哪两期、哪个口径的毛利率之差，推断本身就是一个易错的语义解析问题。第二，拒收后的重试循环会消耗大量轮数，模型在被拒收后倾向于删掉数字而不是改对，文稿因而变空。第三，如果程序已经能复算，直接让程序算并把结果写进产物供模型引用，路径更短、失败模式更少。最终方案把“约数也不许心算”定为硬规则（第 4 节），模型的任务从“算对”变成“找到并引用程序算好的数”，后者正是护栏可以机械检查的。

14.2　系统演进：每一步由哪个失败驱动

图 30 按时间整理了系统的主要演进。从仓库提交历史看，系统大体经历了五个阶段。

阶段一：确定性引擎先行（9 月下旬）。 最早的版本是一条以规则抽取为主的确定性流水线，计算核心借用了一个开源插件的引擎。这一阶段确立了此后从未动摇的两件事：单一真值表，以及单位约定。随后我们用自研实现替换了外部引擎，并以 golden 快照逐项锁住计算口径（第 8 节）。这一步的代价是大量重复劳动，收益是对每一个计算口径都有完全的控制权，后面的 EBIT 口径修正、三表驱动 DCF 都依赖这一点。

阶段二：模型从“调用的 API”变成“工作的会话”（9 月 28 日前后）。 早期版本在程序内部调用模型 API 完成抽取，程序向模型发送页面文本、接收结构化 JSON。这种方式的问题在真实年报上暴露得很快：一份年报两三百页，程序必须事先决定把哪些页喂给模型，而“哪些页有用”恰恰需要理解版式；模型拿不到原始 PDF 的表格结构，也无法自己翻页核对。Skill 版把方向倒了过来：模型作为一个能读文件、能跑命令的会话直接读 PDF，程序退到“定位、校验、计算”的位置。这一转向是整个系统最重要的一次架构调整，“证据约束抽取”与“双通道不静默取舍”都是在此之后才真正可行的。

阶段三：从单会话到隔离工作区与闸门（10 月 2 日）。 单个会话写完整研报时，我们观察到三类问题：模型在仓库里能看到其他公司的分析框架并加以照抄；上下文被无关文件占满；模型会在后期阶段回头“修正”前期已经校验过的数字。隔离工作区与分阶段闸门同时解决了这三点：工作区里只有这一家公司的材料，前期产物一旦通过闸门，后续阶段改动它就会因新鲜度规则使闸门重新失效。

阶段四：先搜集后分析与评测闭环（10 月 3–4 日）。 第一批多会话研报在盲评中只得到 46～49 分（评分标准 v0），主要失分集中在行业与竞争章节：外部数字没有出处，或出处与数字对不上。把资料搜集提前为独立阶段、由研究助理会话完成并登记来源，是对这一失败的直接回应。同一时期加入了评分标准、盲评与 refine 闭环，评分标准随后依据人工质检修订为 v1，再把“数字前后矛盾”从模型判定改为程序判定形成 v2。

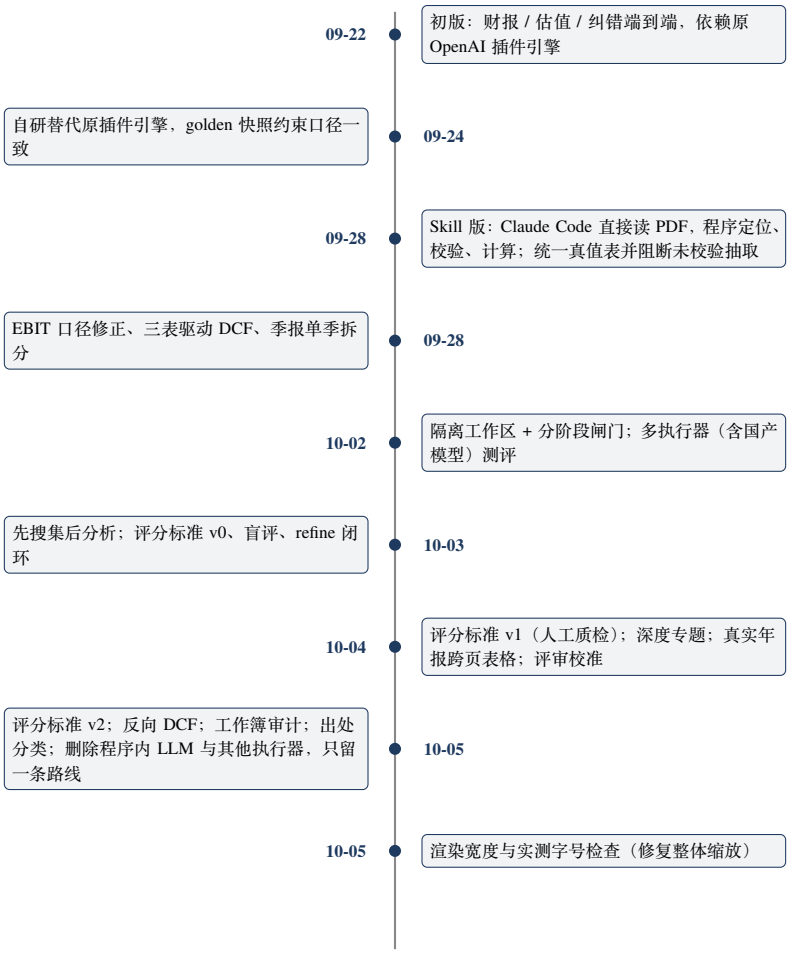


图 **30**　LEDGER 的演进时间线（2026 年 9–10 月，依据仓库提交记录整理）。

阶段五：收敛到一条路线（10 月 5 日）。 系统一度同时维护三种执行方式：程序内调用模型的工具循环、Claude Code 无头会话、以及另一家编码智能体执行器；还曾接入国产模型做对比测评。维护多条路线的成本在闸门与评测加入之后急剧上升：每加一道闸门、每改一次工作区说明，都要在三条路线上分别验证。我们最终删除了其他路线，只保留无头 Claude Code 一条，并在文档中说明了模型选择的理由（第 14.3 节）。

要点：演进史的共同模式

回看这五个阶段，每一次调整的方向都相同：把一个原本依赖模型“自觉”的性质，改成由程序在边界上强制。照抄别家框架靠工作区隔离强制；不改已校验数字靠新鲜度规则强制；外部数字有出处靠来源登记与护栏强制；评分不泄露靠目录隔离强制。模型的能力没有被削弱，被削弱的是模型“违反约束而不被发现”的可能性。

14.3　为什么用编码智能体做 harness

本系统的每个模型角色都是一次无头 Claude Code 会话，而不是由我们自己的程序循环调用模型 API。这一选择有几方面的考虑。

能力面。 研报生产需要的能力——读 PDF、按页翻查、运行命令、读写文件、联网检索与抓取、在失败后自行重试——恰好是编码智能体已经打磨过的工具集。自建工具循

环意味着重新实现文件系统沙箱、权限、上下文压缩、长任务中断恢复等基础设施，这些都不是本项目的贡献所在。

边界清晰。 以“会话 + 文件”为接口，模型与引擎之间只有一种交互方式（第 5 节）。引擎因此可以保持纯粹的确定性，不含任何提示词或模型调用；所有与模型相关的配置集中在编排器生成的工作区说明与权限白名单里。这使得“引擎不调用模型”可以作为一条可测试的不变量存在。

可替换性。 会话的启动参数、模型名与工具白名单都由编排器传入，模型通过环境变量配置。理论上任何能以无头方式运行、支持文件读写与命令执行的编码智能体都可以替换当前的执行器。历史上我们确实接入过其他执行器并跑通过完整流程，删除它们是出于维护成本，不是架构不支持。

权衡（Trade-off）：国产模型与 Claude

大赛鼓励优先使用国产模型。我们曾用国产模型执行器完成过两家公司的研报流程并做对比，最终全部改用 Claude，理由是：（1）流水线对长程工具使用的稳定性要求很高，单个会话常常需要上百轮工具调用，执行器在这一点上的差异会直接表现为闸门通过率的差异；（2）维护多条执行路线使每次闸门或工作区规则的修改都要多次验证，在初赛时间约束下不可承受；（3）系统的正确性保证并不依赖于某个特定模型——数字正确由程序保证、来源由登记与护栏保证、评分由隔离的盲评保证——因此模型选择主要影响效率与文字质量，而非可信度。换回国产模型在架构上只需替换执行器与环境变量，我们将其列为决赛前的后续工作。

权衡（Trade-off）：确定性编排器 vs 模型编排器

多智能体框架常见的做法是由一个“经理”模型决定下一步调用谁。我们的编排器是一段确定性 Python：阶段顺序固定，是否进入下一阶段完全由闸门谓词决定，refine 的路由由问题单类型查表决定。我们放弃了模型编排的灵活性，换来三点：（1）流程可预测，同样的工作区状态总是触发同样的下一步，便于复现与调试；（2）闸门无法被“说服”，模型无法通过在总结里声称“已完成”来跳过检查；（3）编排器可以被单元测试覆盖，包括用模拟会话测试路由与回滚。代价是流程中的异常情况（例如外部网站长时间不可达）需要人工处理，编排器不会自己想办法绕过。

14.4 闸门、评分与 Goodhart 定律

任何可度量的指标一旦成为优化目标，就会被优化穿透。流水线中有两类度量：闸门（程序检查，决定能否继续）与评分（盲评，决定质量）。我们对两类度量采取了相反的可见性策略：

- **闸门完全可见。**闸门的条件写在工作区说明里，生成方清楚地知道需要满足什么。这是有意的：闸门检查的都是“必要条件”（数字可追溯、校验晚于修改、篇幅下限），被针对性满足正是我们想要的结果。被“优化穿透”的风险集中在数量型闸门上（字数、图表数、来源数），我们因此只把它们设为下限，并明确由盲评兜底质量。
- **评分完全不可见。**评分标准、分值与完整评测结果都不进入工作区，回传的问题单只含理由与证据。评分检查的是“质量”，一旦可见，生成方就会朝条目文本优化，例如在每节末尾机械地补一句“与一致预期的差异在于……”。

这一分法的底层逻辑是：对**必要条件**，可见性有利于满足；对**质量判断**，可见性导致表面满足。问题单之所以保留理由与证据，是因为“哪里不好、为什么”是改进所必需的信息，而“哪一条值几分”只对刷分有用。

失败案例：数量下限的边界

篇幅闸门引入后，早期版本出现过“深度研究”部分大段复述年报原文以凑足字数的情况。我们没有通过提高闸门复杂度来应对（例如检测与年报的文本重合度），而是在主笔技能中写明深度的可检查要求（例如核心驱动至少两环定量推导、每个投资主线对应预测参数），并在盲评中设置对应条目。原因是任何新的数量检查都会被新的方式满足，而盲评条目对生成方不可见，更难被针对。

14.5 误报的不对称代价贯穿全系统

“误报控制优先于召回”最初是为研报纠错核查制定的，但在系统的几乎每个检查点上都能看到同一取舍：

- **抽取**：证据校验对找不到原文的数字直接拒收，宁可让真值表缺一项，也不让未经证实的数字进入下游。缺失可以被发现和补齐，错误数字一旦进入真值表会被估值、文稿、图表层层引用。
- **护栏**：容差按展示精度的半个末位设定，而不是一个宽松的相对容差，避免文稿中错误的数字“碰巧”匹配上某个产物数值而被放行。这里“误报”指的是护栏错放错误数字，其代价是错误进入交付物。
- **纠错**：错误类型只有在找到精确替代解释时才下结论，预测性表述判为“无法核实”。一个错报会让使用者怀疑全部核查结果。
- **评审**：盲评判出的问题要附原文证据，人工抽查关注的首要指标是精确率而非召回率。

这些检查点的共同点是：下游会信任上游的输出。在一条信任链上，每个环节的误报都会被放大，而漏报至少还有被下一环节捕获的机会。对信任链上游的检查偏向保守，是一种系统层面的风险分配，而不是各模块各自的偏好。

14.6 不预设行业框架的代价与收益

技能只给方法与约束，不给“白酒看吨价、电池看 GWh、航运看运价”这类行业模板。这一决定的收益是明确的：不同行业的公司得到的分析框架确实不同，模型会依据年报的分部披露与资料包推导驱动；模板不适用时也不会被硬套。代价同样明确：

1. 框架质量依赖模型对行业的理解，对冷门行业的公司，推导出的驱动可能流于表面；
2. 不同运行之间的框架差异更大，降低了同一公司多次运行结果的可比性；
3. 盲评无法依据行业知识清单逐条检查，只能检查“是否给出了至少两环定量推导”这类与行业无关的结构性要求。

我们接受这些代价，因为预设模板的失败模式更隐蔽：模板会让报告“看起来专业”，而读者很难察觉模板对这家公司并不适用。后续计划在拿到优秀研报样例后提炼“分析逻辑”而非“行业结论”，以改善第一点。

14.7 没有成功的尝试

如实记录失败的尝试，有助于读者判断哪些设计是经过比较后的选择。

用正则拦截机器腔。 我们曾编写规则拦截“不是……而是……”“值得注意的是”“相互印证”等句式。规则上线后，模型很快改用同义表达，文风并没有改善，反而在被拦截后出现语句不通的修补。最终删除了规则，改为由不参与写作的复核编辑会话改写，数字锁保证改写不碰数字（第 10 节）。

程序内工具循环。 早期版本让程序维护一个“提问—调用工具—回答”的循环，由模型决定调用哪个分析函数。它在样例年报上工作良好，在真实年报上频繁因上下文长度、

工具参数格式和中断恢复出问题。与其继续打磨这一循环，不如直接使用成熟的编码智能体，这一路线最终被删除。

整节删除类缺陷的评审检出。 在评审校准中，我们用程序造出“删掉一个必要章节”的劣化稿，期望评审能判出。实际检出率偏低。分析发现主要原因在劣化算子本身：删除一节正文后，摘要、图表与附录中仍保留了相关内容，评审据此判定“已覆盖”。这说明劣化算子需要跨全文同步删除，也提醒我们，评审校准的结论受限于劣化的彻底程度（第 12 节）。

单次大会话写完整研报。 在多会话流水线之前，我们尝试过让一个会话完成从抽取到排版的全部工作。会话在后期常因上下文压缩丢失前期约束，例如忘记某个数字已被拒收，或在排版阶段修改正文。拆分为五个角色后，每个会话的上下文只包含本阶段所需的材料，这类问题基本消失。

14.8 可迁移性

尽管本系统针对 A 股财报，核心机制并不依赖金融领域：

- **证据约束抽取**适用于任何“从长文档中抄出结构化事实”的任务，如合同条款、临床试验报告、审计底稿；
- **展示精度容差**适用于任何需要判断“文本中的数字是否与真值一致”的场景；
- **数字锁**适用于任何“允许模型改写、但不允许改动事实”的编辑任务；
- **闸门可见、评分不可见**的分离适用于任何用模型评审驱动模型生成的闭环。

领域相关的部分集中在指标 schema、勾稽规则、估值引擎与技能说明中。迁移到港股或美股需要替换报表版式处理与会计口径映射；迁移到其他文档密集型领域则需要替换真值表的计算层。

15 局限、风险与合规

本章集中说明系统的已知局限。我们把局限分为四类：评测证据本身的局限、计算口径的简化、工程层面的约束，以及使用风险与合规。前三类决定了本文结论能推广到多远，第四类决定了系统可以被怎样使用。

15.1 评测证据的局限

样本规模小。 真实年报的抽取核验只覆盖少数几家公司；研报质量评分只在两到三家公司上做过迭代对比；评审校准的劣化实验每家公司只有一组算子。这些数字足以说明机制“能工作”，不足以给出带置信区间的性能估计。团队正在按 `docs/eval-dataset-collection.md` 的清单收集约 15 家公司的年报、季报与卖方研报，并人工埋错构造纠错评测集。

版本对比不是对照实验。 研报得分从 v1 到 v2 的提升，同时包含了流水线改动（先搜集、深度专题）、评分标准修订与模型随机性的影响，我们没有在同一评分标准下对每项改动做单独的对照，也没有多次重复生成取均值。第 13 节中的消融结果应被视为初步证据。

样例数据是虚构的。 财报分析、估值建模与纠错核查的回归评测使用虚构公司“星河智造”的样例年报与研报草稿，样例内部完全自洽，版式也比真实年报规整。样例指标衡量的是“引擎在理想输入上是否正确”，真实年报上的表现会明显下降，这也是我们单独报告真实年报抽取结果的原因。

评审与生成模型同族。 盲评质检员与生成角色都来自同一模型家族（分别为 Sonnet 与 Opus）。已有研究表明评审模型可能偏好与自身风格相近的文本 [43]。我们用信息隔离（评审看不到生成过程与模型名）、程序判定的 L0/L1 层与人工抽查部分缓解这一问题，但没有用不同家族的评审模型做交叉验证。

单次盲评。 正式得分来自单次盲评。重复评审的逐条一致率在 86% ~ 91% 之间、总分极差 7 ~ 8 分，这意味着相差几分的两份研报，其优劣不能仅凭一次评分判断。

15.2 计算口径的简化

以下简化均已在代码与报告中注明，列在这里以便读者判断估值结论的适用范围：

- **EBIT 口径。**EBIT 定义为利润总额 + 财务费用 − 投资收益 − 公允价值变动收益；资产处置收益、其他收益、减值与营业外收支仍留在 EBIT 内。剔除的投资类资产需要在假设文件的 `non_operating_assets` 中加回，未设置时估值偏保守，报告会给出提示。
- **经营费用是轧差项。**三表模型中的经营费用由毛利 − EBIT − 折旧摊销倒推，以保证历史期 EBIT 与 DCF 同口径；代价是经营费用不能按费用类别分别预测。
- **期初留存收益置零。**三表模型中权益合并列示，期初留存收益置 0，只影响列示，不影响勾稽。
- **流动性压力测试的符号。**压力测试沿用原插件口径 FCF ×(1 + 冲击)，当 FCF 为负时冲击方向相反。为保持与参照实现一致我们未修改，报告中已注明。
- **金融企业。**FCFF 框架对银行、保险等金融企业并不适用，其资产负债表本身就是经营资产。对这类公司，系统的估值输出应只作参考。

15.3 工程约束

- **不做 OCR。**扫描页直接拒收。部分上市公司的年报附件或早期年报为扫描件，这些文档目前无法处理。
- **季报表头。**规则通道不识别季报“本报告期 / 年初至报告期末”表头，会把它们当成年度数据，因此季报只接受模型通道抽取，规则通道不参与比对，双通道交叉校验在季报上失效。
- **规则通道覆盖。**股本变动表、政府补助口径以及部分扣非指标的季度别名，规则通道尚未覆盖。
- **成本与时延。**一份深度研报需要五个长会话加盲评，单次运行耗时以小时计，费用远高于一次 API 调用。这一成本对单家公司深度研究是可接受的，对大范围覆盖则不可接受。
- **外部来源的可靠性与时效。**来源登记保证“数字在摘句中”，不保证摘句所在网页本身准确，也不保证网页在报告发布后仍然可访问。系统检查来源日期以避免使用报告日之后的信息，但无法排除来源本身转载有误。
- **模型输出不可离线复现。**已保存输入上的程序计算可以逐位复算；重新生成文稿或重新盲评依赖模型环境，结果会有差异。我们保存了每个会话的完整事件流，但不宣称能复现模型输出本身。

15.4 使用风险与合规

不构成投资建议。 系统输出的评级由程序按基准情景每股价值相对参考股价的上行空间机械给出，依赖假设文件中的每一个参数。每份报告都写明假设、口径简化、适用范围与风险，并附“不构成投资建议”声明。系统不应被用作投资决策的唯一依据。

看起来专业的错误。 系统能生成排版精良、图表丰富、数字可追溯的研报，这本身带来一种风险：读者可能因为形式上的严谨而高估观点部分的可靠性。可追溯性只覆盖数字，投资主线、驱动判断与预期差分析仍是模型的观点。报告按"事实 → 推论 → 观点"分层，并在附录写出全文数字的出处构成，就是为了让读者看清哪些内容有程序保证、哪些没有。

版权与数据使用。 年报与公告是公开披露文件；卖方研报与付费数据库内容受版权保护。仓库只存储我们从公开文件中抽取的结构化结果与自己生成的文本，评测用的卖方研报只读不入库。外部来源登记只保存短摘句与链接。

第三方代码。 计算引擎的若干模块参照了一个开源插件的计算口径并以自研实现替代，参照关系、版本与许可证如实列于 `THIRD_PARTY.md`（附录 F），不冒充自研。

16 结论与后续工作

本文完整介绍了 LEDGER，一个面向 A 股上市公司的投研多智能体系统，覆盖财报分析、自动化估值建模与研究报告质量评估三个主选题，以及数据提取、产业链分析与研究报告纠错核查三个子选题。系统的出发点是一个朴素的观察：大语言模型在投研中最有价值的能力是读懂文档、组织论证与写作，最不可靠的能力是记住数字、计算数字与凭记忆补充事实。我们据此把系统拆成两半——模型负责读、抄、写、判断，程序负责校验、计算、拦截——并把这条分工落实为一组可由程序检查的不变量：每个数字带页码级来源并逐字回查；两条抽取通道冲突时不静默取舍；一张真值表驱动全部下游计算；文稿中的数字按展示精度在本次运行的产物或来源登记中找到出处；改写只能删减、不能新增或改动数字。

在此基础上，研报的生产被组织为五个上下文隔离、只靠文件交接的无头 Claude Code 会话与 11 道程序闸门；研报的评价被拆成程序判定的数据闭合与评审模型判定的分析质量，评分标准对生成方不可见，评审模型本身用重复评审、程序制造缺陷与人工复核三组证据校准。我们在真实年报上报告了抽取的覆盖与零错配结果，在流水线迭代中观察到研报质检得分的显著提升，并如实说明了这些证据的规模与局限。

我们认为，本工作中最具一般性的经验是：**与其要求模型更可靠，不如让模型的不可靠可以被发现。**系统中几乎每一次架构调整，都是把一个原本依赖模型自觉的性质，改成由程序在边界上强制。这一思路不局限于金融，适用于任何需要从长文档中提取事实、在事实之上写作、并对写作结果负责的场景。

后续工作。 按优先级，决赛前计划完成：

1. **评测集扩充。**约 15 家公司的年报、季报与人工埋错的卖方研报，给出带置信区间的抽取与纠错指标。
2. **对照消融。**在固定评分标准下，对资料先搜集、复核会话、深度专题三项设计做多次重复的对照实验；改进劣化算子使其跨全文同步删除内容与图表。
3. **规则通道补全。**识别季报表头，使双通道交叉校验覆盖季报；补充股本变动表与政府补助口径。
4. **跨家族评审。**引入不同家族的评审模型，检验同族偏好的影响。
5. **执行器可替换性验证。**在国产模型执行器上重跑完整流水线，比较闸门通过率与质检得分。
6. **交互前端。**网页前端与终端界面的运行中实时刷新，支持从研报中任一数字点击回溯到年报原文。

参考文献

[1] Anthropic. Claude code overview. https://docs.anthropic.com/en/docs/claude-code/overview, 2025. Product documentation.

[2] Rahul K. Arora et al. HealthBench: Evaluating large language models towards improved human health. *arXiv preprint arXiv:2505.08775*, 2025.

[3] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024.

[4] Christoph Auer et al. Docling technical report. *arXiv preprint arXiv:2408.09869*, 2024.

[5] Messod D. Beneish. The detection of earnings manipulation. *Financial Analysts Journal*, 55(5):24–36, 1999.

[6] Victor L. Bernard and Jacob K. Thomas. Post-earnings-announcement drift: Delayed price response or risk premium? *Journal of Accounting Research*, 27:1–36, 1989.

[7] Bernd Bohnet et al. Attributed question answering: Evaluation and modeling for attributed large language models. *arXiv preprint arXiv:2212.08037*, 2022.

[8] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.

[9] Zhiyu Chen et al. FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[10] Zhiyu Chen et al. ConvFinQA: Exploring the chain of numerical reasoning in conversational finance question answering. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022.

[11] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970.

[12] Aswath Damodaran. *Investment Valuation: Tools and Techniques for Determining the Value of Any Asset*. John Wiley & Sons, Hoboken, NJ, 3 edition, 2012.

[13] Patricia M. Dechow and Ilia D. Dichev. The quality of accruals and earnings: The role of accrual estimation errors. *The Accounting Review*, 77(s-1):35–59, 2002.

[14] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[15] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[16] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning (ICML)*, 2024.

[17] Charles Elkan. The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, 2001.

[18] Joseph L. Fleiss. Measuring nominal scale agreement among many raters. *Psychological Bulletin*, 76(5):378–382, 1971.

[19] Luyu Gao, Zhuyun Dai, Panupong Pasupat, Anthony Chen, Arun Tejasvi Chaganty, Yicheng Fan, Vincent Y. Zhao, Ni Lao, Hongrae Lee, Da-Cheng Juan, and Kelvin Guu. RARR: Researching and revising what language models say, using language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[20] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *International Conference on Machine Learning (ICML)*, 2023.

[21] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[22] Charles A. E. Goodhart. Problems of monetary management: The U.K. experience. In *Papers in Monetary Economics*, volume 1. Reserve Bank of Australia, 1975.

[23] Charles A. E. Goodhart. Problems of monetary management: The U.K. experience. In *Monetary Theory and Practice: The UK Experience*, pages 91–121. Macmillan, 1984.

[24] Myron J. Gordon. Dividends, earnings, and stock prices. *The Review of Economics and Statistics*, 41(2):99–105, 1959.

[25] Sirui Hong et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

[26] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.

[27] Pranab Islam et al. FinanceBench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023.

[28] Ziwei Ji et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023.

[29] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[30] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[31] Seungone Kim et al. Prometheus: Inducing fine-grained evaluation capability in language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[32] Tim Koller, Marc Goedhart, and David Wessels. *Valuation: Measuring and Managing the Value of Companies*. John Wiley & Sons, Hoboken, NJ, 7 edition, 2020.

[33] Rik Koncel-Kedziorski et al. BizBench: A quantitative reasoning benchmark for business and finance. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[34] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[35] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[36] Jing Liu, Doron Nissim, and Jacob Thomas. Equity valuation using multiples. *Journal of Accounting Research*, 40(1):135–172, 2002.

[37] Nelson F. Liu, Tianyi Zhang, and Percy Liang. Evaluating verifiability in generative search engines. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

[38] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[39] Aman Madaan et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[40] David Manheim and Scott Garrabrant. Categorizing variants of Goodhart's law. *arXiv preprint arXiv:1803.04585*, 2018.

[41] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[42] Ahmed Nassar, Nikolaos Livathinos, Maksym Lysak, and Peter Staar. TableFormer: Table structure understanding with transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[43] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[44] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[45] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[46] Alfred Rappaport and Michael J. Mauboussin. *Expectations Investing: Reading Stock Prices for Better Returns*. Harvard Business School Press, Boston, MA, 2001.

[47] Hannah Rashkin, Vitaly Nikolaev, Matthew Lamm, Lora Aroyo, Michael Collins, Dipanjan Das, Slav Petrov, Gaurav Singh Tomar, Iulia Turc, and David Reitter. Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4), 2023.

[48] Varshini Reddy et al. DocFinQA: A long-context financial reasoning dataset. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Short Papers*, 2024.

[49] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

[50] Timo Schick et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[51] Erik Schluntz and Barry Zhang. Building effective agents. Anthropic Engineering Blog, `https://www.anthropic.com/research/building-effective-agents`, 2024.

[52] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[53] Richard G. Sloan. Do stock prices fully reflect information in accruals and cash flows about future earnings? *The Accounting Review*, 71(3):289–315, 1996.

[54] Brandon Smock, Rohith Pesala, and Robin Abraham. PubTables-1M: Towards comprehensive table extraction from unstructured documents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4634–4642, 2022.

[55] Marilyn Strathern. 'improving ratings': Audit in the British university system. *European Review*, 5(3):305–321, 1997.

[56] John W. Tukey. *Exploratory Data Analysis*. Addison-Wesley, Reading, MA, 1977.

[57] Peiyi Wang et al. Large language models are not fair evaluators. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[58] Xingyao Wang et al. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.

[59] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[60] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[61] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[62] Qingyun Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[63] Shijie Wu et al. BloombergGPT: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023.

[64] Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. TradingAgents: Multi-agents LLM financial trading framework. *arXiv preprint arXiv:2412.20138*, 2024.

[65] Qianqian Xie et al. PIXIU: A comprehensive benchmark, instruction dataset and large language model for finance. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[66] Qianqian Xie et al. FinBen: A holistic financial benchmark for large language models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[67] Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. FinGPT: Open-source financial large language models. *arXiv preprint arXiv:2306.06031*, 2023.

[68] Hongyang Yang et al. FinRobot: An open-source AI agent platform for financial applications using large language models. *arXiv preprint arXiv:2405.14767*, 2024.

[69] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[70] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[71] Yangyang Yu et al. FinMem: A performance-enhanced LLM trading agent with layered memory and character design. *arXiv preprint arXiv:2311.13743*, 2023.

[72] Wentao Zhang et al. A multimodal foundation agent for financial trading: Tool-augmented, diversified, and generalist. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2024.

[73] Lianmin Zheng et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[74] Fengbin Zhu et al. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

[75] 中华人民共和国财政部. 企业会计准则——基本准则及具体准则. 财会〔2006〕3 号等，及其后续修订, 2006.

[76] 中国证券监督管理委员会. 公开发行证券的公司信息披露内容与格式准则第 2 号——年度报告的内容与格式（2021 年修订）, 2021.

[77] 中国证券监督管理委员会. 公开发行证券的公司信息披露解释性公告第 1 号——非经常性损益（2023 年修订）, 2023.

算法 2 断言错误类型诊断（`report_check/verify.verify_claim` 的判定顺序）

function DIAGNOSE(c, $\mathcal{T}$)
  if c 为预测性表述 **then**
    return 无法核实
  end if
  $t \leftarrow \mathcal{T}.\text{VALUE}(\tau, m, k)$
  if t 不存在 **then**
    return 无法核实
  end if
  if $|rs_u - t| \le \text{tol}(c)$ **then**
    return 正确
  end if
  if τ = level **then**
    for $u' \in \text{UNITALTS}(u)$ **do**　　▷ 如亿元 → 万元、元、百万元
      if $|rs_{u'} - t| \le \frac{1}{2}\,\text{ulp}(r)s_{u'} + 10^{-6}|t|$ **then**
        return 单位错误
      end if
    end for
  end if
  if $\tau \in \{\text{yoy}, \text{pp_change}\}$ **and** $|-r-t| \le \text{tol}(c)$ **then**
    return 方向错误
  end if
  for $k' \neq k$ 且 $(m, k') \in \mathcal{T}$ **do**
    if $|rs_u - \mathcal{T}.\text{VALUE}(\tau, m, k')| \le \text{tol}(c)$ **then**
      return 期间错配
    end if
  end for
  for $m' \in \text{CALIBERGROUP}(m) \setminus \{m\}$ **do**　　▷ 只在易混用的指标组内
    if $|rs_u - \mathcal{T}.\text{VALUE}(\tau, m', k)| \le \text{tol}(c)$ **then**
      return 口径错误
    end if
  end for
  return 数值错误
end function

算法 3 规则通道 `extract_rule`

function ExtractRule($\mathcal{D}, Y$)
  $F \leftarrow \emptyset$；$I \leftarrow \emptyset$
  for 每个表格 chunk $c \in \mathcal{D}$ **do**
    $(i, \pi) \leftarrow$ Header(c.rows, Y)；若无则 $(\cdot, \pi) \leftarrow$ Header(c.header_rows, Y)，数据从第 0 行起
    if 无表头 **then continue**
    end if
    if `skip` $\in \pi$ **then** 记 issue“追溯调整”
    end if
    if not IsSourceTable(c) **then continue**
    end if ▷ 白名单、母公司、关联方
    $u \leftarrow c$.unit（缺省为元）；$h \leftarrow$ 表中是否同时有“营业收入”与“营业总收入”
    for 表头之后的每一行 row（行号 r） **do**
      $m \leftarrow$ MatchLabel(row[0], c.caption)
      if $m = \perp$ **or** (m = revenue **and** h **and** 行名含“营业总收入”) **or** 行名含“收益以” **then continue**
      end if
      for 每列 $j \geq 1$，π_j 为合法期间 **do**
        $x \leftarrow$ parse(row[j])；若 $x \neq \perp$：$F \leftarrow F \cup \{(m, \pi_j, \phi(u)x \text{ 或} x, \sigma = (c, r, j))\}$
      end for
      for 每列 j，π_j = `chg` **do**
        记 (m.reported_yoy, max π, parse(row[j]))
      end for
    end for
  end for
  return (F, I)
end function

算法 4 稀疏列合并 `compact_columns`

function Compact(G, force)
  把 G 补齐为 $n \times w$ 矩阵
  if not force **and** 不存在全空列 **then return** G ▷ 规整表格原样返回
  end if
  $\mathcal{G} \leftarrow [\,]$
  for $c = 1, \ldots, w$ **do**
    if $\mathcal{G}$ 非空 **and** $\forall i:\ \neg(G_{ic} \neq \emptyset \wedge \exists g \in \mathcal{G}_{\text{last}} : G_{ig} \neq \emptyset)$ **then**
      $\mathcal{G}_{\text{last}} \leftarrow \mathcal{G}_{\text{last}} \cup \{c\}$ ▷ 并入当前组：没有任何一行两边都有值
    else
      新开一组 $\{c\}$
    end if
  end for
  $G'_{i,j} \leftarrow$ 组 j 内各列在第 i 行的拼接（至多一个非空）
  return 删去全空组后的 G'
end function

算法 5 续表链接 `link_continuation`

function Link(cur, raw, prev, head)
  want ← prev 的列数（已合并列）；$w \leftarrow$ raw 的列数；$w_h \leftarrow$ head 的列数 ▷ *head 为 prev 未合并列的前 3 行*
  if $w_h \le w$ **then**
    $J \leftarrow$ Compact(head（补齐到 w 列）‖ raw, force = $(w_h \neq w)$)；hdr, rows ← J 的前 |head| 行与其余行
  end if
  if rows 为空 **or** rows 的列数 ≠ want **then**
    hdr ← Compact(head)；rows ← Compact(raw) ▷ 两页切分完全不同时各自合并
    if hdr 或 rows 的列数 ≠ want **then return** False ▷ 宁可漏取也不错配期间
    end if
  end if
  while rows[0] 是 prev 末行科目的折行尾巴 **do**
    prev 末行标签 += “\n” + rows[0][0]；删除 rows[0]
  end while
  cur.rows ← MergeWrapped(rows)；cur.header_rows ← hdr；cur.continued_from ← prev.chunk_id
  return True
end function

算法 6 双通道合并 `merge`（流水线中 prefer = llm）

function MERGE(F_R, F_L, prefer)
  $M \leftarrow \{\}$；$I \leftarrow [\,]$
  for $f \in F_R$ **do** ▷ 规则通道内部去重
    if $f.\kappa \notin M$ **then** $M[f.\kappa] \leftarrow f$
    else if CLOSE($M[f.\kappa].v, f.v$) **then** $M[f.\kappa]$.corroborated_by += $f.\sigma$
    else I 追加 (high, 同一指标多处取值不一致)
    end if
  end for
  for $f \in F_L$ **do** ▷ 与模型通道比对
    $b \leftarrow M$.get($f.\kappa$)
    if $b = \perp$ **then** $M[f.\kappa] \leftarrow f$ ▷ 只有模型通道：$method = llm$
    else if CLOSE($b.v, f.v$) **then** $b.\mu \leftarrow$ rule+llm；b.corroborated_by += $f.\sigma$
    else
      I 追加 (medium, 规则与 LLM 抽取不一致, 两侧原文与来源)
      if prefer = llm **then** $M[f.\kappa] \leftarrow f$
      end if
    end if
  end for
  return (M, I)
end function

算法 7 单季拆分 `_split_quarters`

function SPLIT(T)
  $B \leftarrow \{\kappa \mapsto \{\kappa\} : \kappa \in T$ 为列示单元格且为发生额指标$\}$
  for 每个指标 m，每个年度 Y **do**
    repeat
      changed $\leftarrow$ FALSE
      for 每条规则 (t, a, b, s) **do**
        $\kappa \leftarrow (m, Yt)$；$\kappa_a \leftarrow (m, Ya)$；$\kappa_b \leftarrow (m, Yb)$ ▷ $a =$ “” 表示全年
        if $\kappa \notin T$ **and** $\kappa_a \in B$ **and** $\kappa_b \in B$ **then**
          $T[\kappa] \leftarrow T[\kappa_a] + s \cdot T[\kappa_b]$，公式“$a \pm b$”，来源 $[\kappa_a, \kappa_b]$
          $B[\kappa] \leftarrow B[\kappa_a] \cup B[\kappa_b]$；changed $\leftarrow$ TRUE
        end if
      end for
    until not changed
  end for
  return B
end function

算法 8 三表联动求解（`ts_engine.run_model`）

$R \leftarrow$ 按增速滚动收入；$\mathrm{IS}^{(0)} \leftarrow$ 以零折旧零利息初算利润表（只为取营业成本 C）
PPE $\leftarrow$ 式 (38)；WC $\leftarrow$ 式 (39)（用 C）
Int $\leftarrow 0$
for $k = 1, \ldots, 20$ **do**
    IS $\leftarrow$ 利润表（折旧 Dep、利息 Int）；按分红率重算分红
    CFO $\leftarrow$ NI + Dep + DT − ΔNWC
    Int$'$ $\leftarrow$ 债务表（提款、强制还款、循环贷补足最低现金、超额现金扫还）
    if $\max_t |\mathrm{Int}'_t - \mathrm{Int}_t| < 10^{-8}$ **then**
        Int $\leftarrow$ Int$'$; **break**
    end if
    Int $\leftarrow$ Int$'$
end for
以收敛利息再算一遍利润表、债务表、利润表；生成现金流量表、资产负债表、偿债指标与勾稽误差

算法 9 非经营性资产明细核对（`build_plan.resolve_non_operating_items`）

function RESOLVE(items, $\mathcal{D}$)
    total $\leftarrow 0$；rows $\leftarrow$ [ ]
    for each $(\ell, x, u, p, \sigma) \in$ items **do**
        $v \leftarrow$ PARSENUMBER(x)；**if** v 无法解析 **or** $u \notin$ 单位表 **then raise**
        ch $\leftarrow$ FINDEVIDENCE($\mathcal{D}, p, x$)　　▷ 原文第 p 页按数字边界查找原写法
        if ch $= \perp$ **then**
            raise “第 p 页未找到 ℓ x”
        end if
        $w \leftarrow \mathrm{sign}(\sigma) \cdot v \cdot \mathrm{factor}(u)/10^6$；total $\leftarrow$ total $+w$
        rows.append(ℓ, x, u, σ, w, source = (ch.source, ch.page, ch.chunk_id))
    end for
    return total, rows　　▷ 写入 *non_operating_assets.json*，工作簿逐项列示
end function

算法 10 单变量反向 DCF（`reverse_dcf._solve`）

function SOLVE($f(\theta) = V(\theta) - P$, $[\ell, u]$)
  $\theta_i \leftarrow \ell + (u-\ell)\,i/400$, $i = 0..400$；$y_i \leftarrow f(\theta_i)$（ZeroDivision 时记为未定义，丢弃）
  best $\leftarrow \arg\max_i y_i$ ▷ 无解时报告“该变量最多能把每股价值推到多少”
  $\mathcal{B} \leftarrow \{[\theta_i, \theta_{i+1}] : y_i = 0 \vee y_i y_{i+1} < 0\}$
  if $\mathcal{B} = \emptyset$ **then**
    return $\perp$, best
  end if
  $[a, b] \leftarrow \arg\min_{[a,b]\in\mathcal{B}} \min(|a|, |b|)$ ▷ 取离基准（$\theta = 0$）最近的变号区间
  for $k = 1..200$ **do** ▷ 二分
    $c \leftarrow (a+b)/2$；**if** $|f(c)| \le 10^{-6}$ **or** $b - a \le 10^{-12}$ **then return** c
    按 $f(c)$ 与 $f(a)$ 同号与否收缩区间
  end for
end function

算法 11 规则通道断言抽取（`claims.extract_rule`，简化）

function EXTRACTRULE($\mathcal{D}_R$, y_0)
  $\mathcal{A} \leftarrow$ 按长度降序的（说法, 指标）对；$\mathcal{C} \leftarrow [\,]$
  for 文本块 $c \in \mathcal{D}_R$, 句子 $s \in$ SPLIT(c.text) **do**
    spans $\leftarrow$ 在 s 上对 $\mathcal{A}$ 做不重叠最长匹配，按起点排序
    fc $\leftarrow \exists h \in$ FORECASTHINTS : $h \sqsubseteq s$
    for $j = 1..|\text{spans}|$，$(a_j, b_j, m_j) = \text{spans}[j]$ **do**
      seg $\leftarrow s[b_j : a_{j+1}]$；$k \leftarrow$ PERIOD($s[: b_j], y_0$)
      $Y \leftarrow$ `YOY_RE` 在 seg 中的首个匹配；$V \leftarrow$ `VAL_RE` 在 seg（截到 Y 之前）中的首个匹配
      if V 存在且单位不是百分点类 **then**
        $\tau \leftarrow$ LEVELTYPE(V, m_j) ▷ *level / ratio / yoy /* 不抽取（变动额、占比、倍）
        if $\tau \neq \perp$ **then**
          $\mathcal{C}$.append(m_j, k, τ, V, quote $= s[a_j : b_j + |V|]$, fc)
        end if
      end if
      if Y 存在 **then**
        $x \leftarrow \pm\,\text{parse}(Y)$ ▷ 下降类动词取负
        $\mathcal{C}$.append(m_j, k, yoy 或 pp_change, x, quote $= Y$, fc)
      end if
    end for
  end for
  return $\mathcal{C}$
end function

算法 12 模型断言接入与两通道合并（`ingest_claims` + `merge_claims`）

function INGEST($\mathscr{D}_R$, payload, y_0)
  $E \leftarrow [\,]$；$R \leftarrow [\,]$
  for $(i, \text{it}) \in$ payload.claims **do**
    $q \leftarrow$ it.quote；$c \leftarrow$ 第一个满足 $q \sqsubseteq c$.text 的文本块
    if 未知指标 **or** 类型不合法 **or** $c = \bot$ **or** |it.value| $\not\sqsubseteq q$ **or** 无法解析 **then**
      R.append(i, it, 理由)；**continue**
    end if
    E.append(CLAIM(L·, it, loc = (c.source, c.page, c.chunk_id), method=llm))
  end for
  return MERGE(EXTRACTRULE($\mathscr{D}_R$, y_0), E), R
end function
function MERGE(rule, llm)
  seen $\leftarrow \{(m, k, \tau, \text{round}(x, 6)) : \gamma \in \text{rule}\}$；out $\leftarrow$ rule
  for $\gamma \in$ llm **do**
    if $(m, k, \tau, \text{round}(x, 6)) \notin$ seen **then**
      γ.id $\leftarrow$ C·(|out| + 1)；out.append(γ)；seen.add(键)
    end if
  end for
  return out
end function

算法 13 护栏的指标—期间绑定（`guard.check_binding`）

function CHECKBINDING($\mathcal{N}$, run)
  $y \leftarrow$ 报告年度；$\mathcal{C} \leftarrow$ EXTRACTRULE($\mathcal{N}, y$)；seg $\leftarrow$ SEGMENTVALUES(run)
  $\mathcal{C} \leftarrow \{\gamma \in \mathcal{C} : \text{CANDIDATE}(\gamma, y, \text{seg}, \text{SENTENCE}(\mathcal{N}, \gamma.\text{quote}))\}$ ▷ 前置过滤
  $F \leftarrow$ VERIFYCLAIMS($\mathcal{C}, \mathcal{T}$)
  return $\{f \in F : f.\text{verdict} \notin \{$正确, 无法核实$\} \wedge \neg\text{ISSEGMENT}(\text{CONTEXT}(f), \text{seg}) \wedge \neg\text{ISFORECAST}(\text{SENTENCE}(f), y)\}$
end function
function CANDIDATE(γ, y, seg, s)
  if γ 为预测，或 s 含“预计 / 预测”或晚于 y 的年份 **then**
    return false
  end if
  if s 含季度词（Q1–Q4、一至四季度、上半年、前三季度） **then**
    return false
  end if
  if s 中出现两个以上不同年份 **then**
    return false ▷ 跨年对比句
  end if
  if s 含“数值单位、数值”的列举 **then**
    return false
  end if
  if s 点名了 seg 中某个长度 ≥ 2 的行名 **then**
    return false ▷ 分部口径
  end if
  if s 含现金流明细、折旧、客户、母公司、扣非、“由 X 升至”等关键词 **then**
    return false
  end if
  return true
end function

算法 14 外部数字的引用检查（`sources.check_citations`）

function CheckCitations($\mathcal{N}$, $\mathcal{S}$, $V_{\mathcal{A}}$)
  $V_s \leftarrow$ 来源 s 的事实数值（每个 $s \in \mathcal{S}$）；$P \leftarrow [\,]$
  for 句子 $\varsigma \in \mathcal{N}$ **do**
    $C \leftarrow \varsigma$ 中的 `[S 编号]`；对 C 中未登记的编号，P.append(“引用了不存在的来源”)
    $X \leftarrow$ CheckText($\varsigma, V_{\mathcal{A}}$) ▷ 产物中找不到出处的记号
    $Y \leftarrow$ CheckText($\varsigma, V_{\mathcal{A}} \cup \bigcup_{s \in C} V_s$) ▷ 加上所引来源后仍找不到的
    for $x \in X$ **do**
      if $x \in Y$ **and** $\exists s \in \mathcal{S} : x \notin$ CheckText($\varsigma, V_{\mathcal{A}} \cup V_s$) **then**
        P.append(“x 来自外部资料，句中须标注对应的 [S 编号]”)
      end if
    end for
  end for
  return P
end function

算法 15 `run_agent`：五个会话的接力

Require: 工作区 $\mathcal{W}$，生成模型 m_g，复核模型 m_q
for (role, u, prompt) $\in$ `SESSIONS` **do** ▷ u：该会话负责到的阶段
  if $\bigwedge_{i \le u} G_i(\mathcal{W})$ **then continue** ▷ 已完成的会话跳过
  end if
  $m \leftarrow m_g$ 若 role $\in$ {research, analyst, write, layout, fix}，否则 m_q
  $c \leftarrow$ Session($\mathcal{W}, m$, role, prompt) ▷ 事件流落盘、追加用量记录
  if $\neg \bigwedge_{i \le u} G_i(\mathcal{W})$ **then**
    打印“role 会话未完成阶段 1..u，停止”；**return** c 或 1
  end if
end for
return c

算法 16 `refine`（`max_rounds=`K，默认 3；`min_gain=`δ，默认 2.0）

Require: 工作区 $\mathscr{W}$；评测根目录 E = `~/finagent-evals/<ws>/`

$S \leftarrow$ 读取 E/`loop.json`，不存在则 {rounds : [ ], best : $\perp$}
if S.rounds $\neq$ [ ] 且 StopReason(S) 非空 **then return** S ▷ 可重入：已停止的不再跑
end if
for $n = |S.\text{rounds}| + 1, \ldots, K$ **do**
  if $n = 1$ **then** RunAgent($\mathscr{W}$)
  else
    $I \leftarrow E$/`r`$(n-1)$/`eval/issues.json`
    $I_g \leftarrow \{i \in I : \text{route} = \text{generator}\}$；$I_\ell \leftarrow \{i : \text{route} = \text{reviewer} \wedge \text{stage} \sim$ `10*`$\}$；$I_r \leftarrow$ 其余复核侧
    if $I_g \neq \emptyset$ **then**
      删除 `narrative.draft.md`、`edit_log.md` ▷ 文稿要改，复核必须重做
      Session(GEN, fix, `FIX_PROMPT`(I_g)) ▷ 到阶段 8 通过为止
      if $\widehat{G}_8(\mathscr{W})$ **then**
        if $I_r \neq \emptyset$ **then** Session(QC, review, `REVIEW_PROMPT`+I_r)
        end if
        RunAgent($\mathscr{W}$) ▷ 按闸门续做排版与交付说明
      end if
    else
      if $I_r \neq \emptyset$ **then** Session(QC, revise, `REVISE_PROMPT`(I_r))
      end if
      if $I_\ell \neq \emptyset$ **then** Session(GEN, layout, `LAYOUT_PROMPT`+I_ℓ)
      end if
    end if
  end if
  if $\neg\widehat{G}_{11}(\mathscr{W})$ **then** S.stopped ←“第 n 轮闸门未通过：next”；**break**
  end if
  $sc \leftarrow$ Evaluate(bundle($\mathscr{W}$), E/`r`n) ▷ *L0/L1 程序 + L2/L3/N 盲评 + 计分*
  追加轮次记录（总分、是否达标、各路由问题数、快照）；S.best $\leftarrow \arg\max_r \text{total}_r$；写 `loop.json`
  if StopReason(S) 非空 **then break**
  end if
end for
if best $\neq$ 最后一轮 **then** Restore(best) ▷ 仅当估值运行目录未变
end if
if best 未达标 **then** 把 best 轮的问题单追加进 `review.md`，交人工
end if

算法 17 带实测的打印 `render.to_pdf(html, browser)`

1: $rec \leftarrow$ {version=1, ok=false, html_sha256, renderer_sha256=H(kit)}
2: **预检**：复制 HTML 到临时目录，注入 `render_probe.js`，把打印媒体规则改为屏幕规则，强制 `html, body` 宽 180mm；
    无头 Chrome `--dump-dom` 取测量结果：溢出元素、`body` 与每个正文段落的计算字号、断图、正文样本（≤ 8 段、每段取 32 字）
3: **if** 溢出 $\neq \emptyset$ 或字号 $\neq$ 10.5pt 或无样本或断图 **then** 抛出错误
4: **end if**
5: **打印**：独立 profile 的无头 Chrome `--print-to-pdf` 到候选文件（等到文件以 `%%EOF` 结尾）
6: **实测**：用 pdfplumber 检查每页为纵向 A4（595.28 × 841.89pt，容差 1pt）；在各页字符流（NFKC 归一）中定位每个正文样本，
    要求样本内每个字形的字号 s 满足 $|s - 10.5| \le 0.12$pt；所有样本都要找到
7: **通过**：rec.pdf $\leftarrow$ 页数与实测样本，rec.pdf_sha256 $\leftarrow H$(候选)，rec.ok $\leftarrow$ true，候选替换正式 PDF
8: **finally** 写 `render_check.json`

算法 18 内部一致性检查（N-02，`consistency.check`）

1: **function** CHECK($\mathcal{N}$, 报告期 y_0)
2: $G \leftarrow \emptyset$ ▷ 比较键 → 提及列表
3: **for** 每个句子 $s \in \mathcal{N}$ **do**
4: $s \leftarrow$ 去掉外部来源后本报告自己的部分；整句是外部数字（卖方、一致预期、`[S 编号]` 且无“我们”）则跳过
5: **if** s 含条件、敏感性、区间字样 **then**
6: **continue**
7: **end if**
8: 情景 $\leftarrow$ 句中唯一出现的情景词；都没有取“基准”；多个同现 $\Rightarrow$ 不比
9: 抽取每股价值（须写明 DCF / 情景；排除区间端点）、“y_a—y_b 年 X 分别为 A、B、C”列举与单年度提及
10: 仅当指标前无其他限定词（公司整体口径）且年度 $> y_0$ 时，加入 $G[k(m)]$
11: **end for**
12: **for** 每个键 k，$G[k] = (m_1, m_2, \dots)$ **do**
13: bad $\leftarrow \{m_j : |x(m_j) - x(m_1)| > \epsilon(m_1) + \epsilon(m_j)\}$
14: **if** bad $\neq \emptyset$ **then**
15: 记一组矛盾 (m_1, bad)
16: **end if**
17: **end for**
18: 抽取本报告评级（排除评级规则说明句与外部评级）；评级多于一种 $\Rightarrow$ 记一组矛盾
19: **return** 矛盾组、比较组数、提及数
20: **end function**

附录 A　评分标准全文

本附录列出默认评分标准 `eval/rubrics/report_v2.json` 的全部 41 个条目（正文见第 12.2 节）。“判定”列中，程序条目附检查函数名（`finagent/evaluation/checks.py` 中的 `check_< 名 >`）；“路由”列为未通过时问题单回传的会话（`generator` 为生成侧，`reviewer` 为复核侧；阶段以 10 开头的复核侧问题交给排版设计师）与对应阶段。交给盲评会话的 `rubric.json` 只含 id、条目、判定标准、反例与是否负向，不含本表的分值、层级与路由。

全局参数。 `gate_cap` = 40（L0 任一失败的总分上限）；`negative_floor` = −20（负向扣分合计下限）；交付标准 `delivery`：总分 ≥ 75，L1 / L2 / L3 得分率分别 ≥ 80% / 60% / 60%，`allow_negative` = false（不得命中负向条目）。各层分值合计：L0 为 0，L1 为 35，L2 为 45，L3 为 20，N 为 −30。

表 124　评分标准 `report_v2` 全部条目。

id	条目	判定	分值	路由 / 阶段	判定标准（及反例）
L0-01	数字护栏	程序 `guard`	0	generator 8 文稿	当前文稿的每个数字都能在本次运行产物或已登记外部来源中找到，指标与期间绑定无误。
L0-02	三表勾稽	程序 `ts_integrity`	0	generator 7 估值运行	三表模型资产负债平衡、现金衔接、各项滚动全部通过，无硬性失败。
L0-03	DCF 与三表同口径	程序 `dcf_tie`	0	generator 7 估值运行	DCF 的收入、EBIT、折旧摊销、资本开支、营运资本逐期取自三表模型。
L0-04	免责声明	程序 `disclaimer`	0	reviewer 10 排版	报告含“不构成投资建议”的免责声明。
L0-05	评级与估值一致	程序 `rating`	0	generator 7 估值运行	评级按规则由上行空间得出，正文写的评级与之相同。
L1-01	抽取校验	程序 `extraction`	3	generator 2 财务抽取	抽取事实逐字可定位，无拒收、冲突与勾稽失败。
L1-02	分部预测基期闭合	程序 `segment_base`	3	generator 6 估值假设	分部驱动预测的基期分部收入加其他收入等于报表营业收入。
L1-03	股权价值桥加总	程序 `bridge`	4	generator 7 估值运行	各情景企业价值 + 加回项 − 扣减项 = 股权价值。
L1-04	每股价值复算	程序 `per_share`	3	generator 7 估值运行	各情景股权价值 ÷ 总股本 = 每股价值。
L1-05	情景排序	程序 `scenario_order`	2	generator 6 估值假设	悲观 ≤ 基准 ≤ 乐观。
L1-06	敏感性方向	程序 `sensitivity`	2	generator 7 估值运行	WACC 上升价值下降、永续增长上升价值上升。
L1-07	终值占比	程序 `terminal_share`	2	generator 6 估值假设	基准情景终值占企业价值不超过阈值。（参数：`max`=0.8）
L1-08	预测与历史衔接	程序 `forecast_continuity`	2	generator 6 估值假设	首个预测年毛利率与最近实际年相差不超过阈值（百分点），收入增速变化不超过阈值。（参数：`gross_margin_pp`=5，`growth_pp`=30）
L1-09	正文估值与产物一致	程序 `text_valuation`	3	generator 8 文稿	正文写出基准每股价值（按展示精度与产物一致）与评级。
L1-10	外部信息可追溯	程序 `sources`	4	generator 1 资料搜集	sources.json 结构校验通过；正文至少引用一处 `[S 编号]`，引用的编号都已登记；只能从外部资料找到出处的数字，所在句子带对应来源的编号。
L1-11	股价有日期与来源	程序 `price`	2	generator 6 估值假设	参考股价写明日期与来源。
L1-12	复核改写过数字锁	程序 `numlock`	3	reviewer 9 复核改写	复核改写前后，改写稿没有新增或改动数字。

id	条目	判定	分值	路由 / 阶段	判定标准（及反例）
L1-13	人工复核清单	程序 `review_list`	2	reviewer 9 复核改写	review.md 至少有一条填写完整的复核事项（事项、为什么重要、当前处理、请核对），或明确写“无”；只有表头不算。
L2-01	核心结论	评审	4	generator 8 文稿	首页或摘要用一两句话给出明确的核心观点：评级、估值结论，以及两三条支撑理由。读完摘要能复述“为什么值这个价”。 反例：摘要只罗列财务数据，没有判断；支撑理由是“行业龙头”“前景广阔”这类无法检验的话
L2-02	预期差	评审	4	generator 8 文稿	说明本报告与市场或一致预期的分歧在哪里，或者说明当前股价隐含了什么假设、我们为何不同意。 反例：通篇没有提市场怎么看；只说“市场低估了公司价值”，不说低估了哪个变量
L2-03	商业模式	评审	3	generator 8 文稿	讲清公司卖什么、卖给谁、靠什么赚钱，以及竞争优势体现在哪个可观察的指标上（份额、毛利率、成本、客户结构等）。 反例：只有公司简介与发展历程；优势只用形容词描述
L2-04	驱动拆解	评审	5	generator 8 文稿	至少一个核心驱动给出两环以上的定量推导：外部驱动 → 公司量、价或结构 → 收入或利润，每一环都有数字。 反例：“受益于需求增长，收入有望提升”；只给出收入增速，不拆量与价
L2-05	历史业绩归因	评审	4	generator 8 文稿	最近一年利润变动被归因到具体科目或驱动（毛利、费用、投资收益、减值等），并给出各自的量。 反例：只写“净利润同比增长 X%”；归因只有定性原因
L2-06	行业与竞争	评审	4	generator 8 文稿	行业供需、周期位置或竞争格局有具体数据支撑（规模、增速、份额、价格等），且数据带来源。 反例：行业章节全是定性描述；竞争格局只列竞争对手名字
L2-07	预测假设有依据	评审	5	generator 8 文稿	关键预测假设（量、价、毛利率、费用率等对利润影响最大的几项）逐项写出取值和依据（历史趋势、公司指引、行业数据之一）。 反例：假设表只有数字没有依据；最重要的假设没有解释
L2-08	估值方法与参数	评审	3	generator 8 文稿	说明为什么用这种估值方法，WACC、永续增长率等关键参数的取值依据可在报告中找到。 反例：只给结果不讲方法；WACC 无拆分或依据
L2-09	估值交叉验证	评审	4	generator 8 文稿	主估值方法之外，至少用一种方法或指标交叉验证（可比公司、隐含倍数、历史估值区间等），并解释差异来源。 反例：只有 DCF 一个数；列了可比公司但不和 DCF 结果对照
L2-10	情景与敏感性	评审	3	generator 8 文稿	给出乐观、悲观情景各自的关键假设差异与对应价值，并指出价值对哪个变量最敏感。 反例：只给三个价格不说假设差在哪；敏感性表无解读
L2-11	风险落到假设	评审	4	generator 8 文稿	主要风险能对应到某个预测假设，并给出影响方向或量级（例如对每股价值或利润的影响）。 反例：风险提示是通用模板（宏观、政策、竞争加剧）；风险与预测假设无关联
L2-12	跟踪指标与失效条件	评审	2	generator 8 文稿	给出未来可验证的跟踪指标或催化剂，或写明结论在什么条件下不成立。 反例：没有任何前瞻可验证的指标

id	条目	判定	分值	路由 / 阶段	判定标准（及反例）
L3-01	故事线连贯	评审	4	reviewer 9 复核改写	各章节围绕核心结论展开，从公司怎么赚钱、发生了什么、往哪里去到值多少钱，有清晰主线；没有与主线无关的章节。 反例：章节像资料堆砌；结论与正文论证对不上
L3-02	事实、推论、观点可区分	评审	3	reviewer 9 复核改写	读者能分清哪些是披露的事实、哪些是推论、哪些是作者观点（例如用“主要系”“我们预计”“我们认为”），且不靠“事实层面”这类行内标签。 反例：预测写成既成事实；通篇“事实/推论/观点”标签
L3-03	结论先行	评审	3	reviewer 9 复核改写	多数章节和段落先给判断，再给支撑数据。 反例：段落先堆数字，最后才给结论或没有结论
L3-04	无机器腔	评审	3	reviewer 9 复核改写	没有明显的模型写作痕迹：自证圆场（“与年报一致”“相互印证”）、程序口吻（“程序预测”“经护栏核对”）、否定对照堆砌（“不是……而是……”）、交代缺失（“年报未披露……只能间接验证”）。只看正文段落：图表来源、表注、免责声明、附录里的方法说明不算。偶发的否定对照或个别程序用语可以通过；某一类痕迹在正文里成了习惯写法才判不通过。 反例：多个章节的段落都用“不是……而是……”起句；正文段落反复出现“经程序核对”“与年报一致”之类的自证
L3-05	数字密度	评审	2	reviewer 9 复核改写	正文不逐行念表；关键数字突出，读者能看出每段最重要的一两个数。一个数字连同它的同比、占比或上年值算一个数据点；单句串起四五个以上互不相关的数据点才算堆砌。 反例：一句话依次报出多个科目或多个季度的金额与增速，像在念表；段落数字很多，却没有一个被点明为判断的依据
L3-06	图表服务论点	评审	3	reviewer 10 排版	图表有标题与来源，与相邻正文的论点对应，读者能从图上看出正文说的结论。 反例：图表与正文无关；图表缺来源
L3-07	假设与局限透明	评审	2	reviewer 10 排版	报告中能找到估值假设汇总与口径简化说明（例如哪些项目未纳入、模型的适用范围）。 反例：没有任何关于模型局限的说明
N-01	无来源的外部数据	评审	−8 封顶 70	generator 1 资料搜集	命中条件：正文出现具体的外部数据或排名（市场规模、市占率、行业排名、价格指数等），既没有 [S 编号] 也不属于公司年报披露。命中一处即判命中。 反例：“公司全球市占率第一”且无来源
N-02	数字前后矛盾	程序 `consistency`	−8	generator 8 文稿	命中条件：评级、某一情景的每股价值、或某一预测年度的公司整体收入 / 利润 / 每股收益 / 毛利率及其增速，在正文两处给出不同数值（差异超出展示精度）。外部来源、条件句、敏感性与区间表述、分部口径不参与比较。 反例：摘要评级“买入”、投资建议一节“卖出”；摘要基准每股价值 54.87 元、估值一节 49.88 元
N-03	聚合数构成描述错误	评审	−6	generator 8 文稿	命中条件：正文对某个合计数（股权价值桥加回项、利润变动合计、收入构成等）的构成描述与模型产物中的拆分不符。请对照 model/ 下的产物核对。 反例：把加回项说成“财务公司拆出资金”，而产物拆分显示是现金与金融资产
N-04	结论前后冲突	评审	−8	generator 8 文稿	命中条件：两处结论或判断方向互相冲突且口径相同（例如一处说毛利率改善、另一处说下滑；一处说景气上行、另一处说已见顶）。只看判断与方向，不看数值是否一致（数值由程序另行核对）。 反例：核心观点说“盈利能力持续改善”，业绩回顾说“毛利率同比下滑、盈利承压”，指同一年同一口径

版本差异。 v0（2026-10-03）与 v2 的差异：L1-10、L1-13、L3-04、L3-05 四条的判定标准在 v1（2026-10-04）修订，修订前后对照见表 96；v2（2026-10-05）把 N-02 由评审判定改为程序判定（`check = consistency`），判定标准从“前后矛盾”收窄为“数字前后矛盾”，并新增评审条目 N-04“结论前后冲突”。v0 与 v1 各 40 条，v2 为 41 条。三个版本的全局参数相同。

附录 B　劣化算子定义

劣化算子定义在 `finagent/evaluation/calibrate.py` 的 `DEGRADATIONS` 字典中，只改文稿 `narrative.md`，全部为确定性文本变换。文稿以 `<!-- section: 名称 -->` 标记分节（排版脚本按节名取用），算子依赖以下辅助函数：

表 125　劣化算子使用的辅助函数。

函数	行为
`_sections(text)`	返回 [(节名, a, b)]：正文为本节标记之后、下一个标记之前的文字
`_replace_section(text, name, body)`	把名为 `name` 的所有节的正文替换为 `body`（为空即清空，标记保留）
`_drop_headings(text, pattern)`	删掉标题匹配正则 `pattern` 的 `###` 小节，直到下一个 `###` 或节标记
`_hollow_sections_titled(text, prefix, pattern)`	节名以 `prefix` 开头且节内第一个 `###` 标题匹配 `pattern` 时掏空该节：保留标题、表格、注释与段落数，正文段落与列表项换成“本段从略。”
`_paragraphs(text, every, edit)`	对正文段落（非标题、列表、表格、注释，且不在 title / subtitle / summary 节）每隔 `every` 段执行一次 `edit`
`_flip_rating(text)`	只在 summary 节内把第一处“给予‘X’评级”按映射换成相反方向
`_insert_unsourced(text)`	在 deep_industry 节第一段之后插入一段固定的无来源数字段落
`degrade(text, kind)`	执行算子；若文稿未发生任何变化则抛 `ValueError`（找不到对应小节）

表 126　七个劣化算子的精确定义。

算子	目标	操作
`no_cross_check`	L2-09	`_drop_headings(t, " 交叉验证 | 相对估值")`
`no_expectation_gap`	L2-02	依次执行：`_drop_headings(t, " 市场关注 | 预期差")`；`_hollow_sections_titled(·, "deep_topic", " 隐含")`；`_replace_section(·, "deep_expectations", "")`
`generic_risks`	L2-11	risks 节替换为四行模板“宏观经济波动风险 / 行业竞争加剧风险 / 政策变化风险 / 技术迭代风险”；并把全文“**风险提示：**”所在行改为“宏观经济波动；行业竞争加剧；政策变化；技术迭代。”
`no_tracking`	L2-12	`_replace_section(t, "tracking", "")`
`unsourced_number`	N-01	在 deep_industry 节第一段后插入：“**市场空间测算**。据行业测算，2030 年全球市场规模将达到 4,800 亿元，2025—2030 年复合增速约 62%，其中中国市场占比约 45%，公司有望占据其中 18% 的份额。”
`contradiction`	N-02	`_flip_rating`：映射卖出 → 买入、减持 → 增持、买入 → 卖出、增持 → 减持、持有 → 买入、中性 → 买入；只改摘要第一处
`machine_tone`	L3-04	`_paragraphs(t, 2, ...)`：每两段正文的第二段前加“根据程序计算结果，”，后加“（以上数据已通过数字护栏校验，由模型自动生成。）”

变体运行。 `prepare_variant` 把原工作区整体复制到 `~/finagent-evals/calibration/< 公司 >/< 变体 >/ws/`（排除 `__pycache__` 与绘图缓存），写入劣化后的文稿，再用该工作区自己的 `build_report.py` 按原运行目录重新渲染 PDF；渲染失败或算子未改动文稿时跳过该变体并打印原因。原稿变体（`clean1` ~ `clean`k）不复制、不改动、不重排，直接评审原工作区。每个变体的元数据 `variant.json` 记录 `kind`、`rep`、`targets`、`desc`。评审结果含 `judge_problems`（未写出判定、异常退出）时整个结果目录被删除，下次运行重跑。

测试。 `tests/test_calibrate.py` 用一份含全部节标记的最小文稿验证：每个算子都改动文稿；`no_cross_check` 删去交叉验证而保留 DCF 估值小节；`no_expectation_gap` 删去市场关注、反向 DCF 专题与预期差正文而保留核心观点；`contradiction` 后全文恰有一处"买入"；其余算子的目标内容消失或出现。另有测试验证缺少对应节时 `degrade` 抛错、汇总按原稿多数判定计算命中与连带改判。

附录 C　数据结构

模型会话写下的抽取与假设文件、外部来源登记，以及评测的输入输出，均为 UTF-8 JSON。下表的字段取自仓库中的真实文件（`data/extractions/贵州茅台 _2025/`、`data/extractions/星河智造 _2024/claims.json`）、skill 中的格式说明（`.claude/skills/finagent-research/references/extraction.md`、`claims.md`）与源码（`finagent/output/sources.py`、`finagent/evaluation/`）。

C.1 `facts.json`：财务事实（分析师会话写，`verify-facts` 校验）

字段	必填	说明
`company`, `fiscal_year`, `source_file`, `notes`	顶层	公司、年度、年报相对路径、口径说明
`facts[]`	是	事实列表，每项字段如下
`metric_id`	是	指标 ID，只能取 `finagent schema` 输出的清单（如 `revenue`、`np_parent`）
`period`	是	年度 `2024`；半年度 `2024H1`；单季 `2024Q3`；前三季度累计 `2024Q1-3`
`raw_value`	是	与原文逐字一致的字符串：保留千分位与小数位，括号负数原样，百分比不带%
`unit`	是	元 / 千元 / 万元 / 百万元 / 亿元；每股收益元/股；比率%；股本股 / 万股 / 亿股
`page`	是	PDF 页码（从 1 开始），程序在该页按数字边界回查
`label`, `table`	建议	原文科目名、所在表名
`scope`	是	固定 `consolidated`（母公司报表不抄）
`source_file`	多文件时	季报等补充文件名，同时用 `--supplement` 传入

`verify-facts` 输出 `facts.verified.json`，顶层为 `facts`、`issues`、`stats`、`rejected`、`missing`、`check`；其中 `check` 含 `ok`、`stats`（`rule_facts`、`llm_facts`、`merged`、`rejected`、`extractor`、`supplements`）、`rejected`、`conflicts`、`tieout_failures`、`missing`。阶段 2 闸门与评分条目 L1-01 读的都是 `check`。茅台 2025 的实际值为：规则通道 98 条、模型抽取 98 条、合并 116 条，拒收、冲突、勾稽失败、缺失均为 0。

C.2 `extras.json`：经营数据（`verify-extra` 逐格校验）

茅台 2025 的 `extras.json` 含 37 张表、93 个派生比率，例如 `price_moutai`（茅台酒吨价 = 分产品营业收入 ÷ 销量，单位万元/吨）。

字段	说明
tables[]	原表逐字抄录：id、title、page、columns[]（name、unit）、rows[]（label、values[]，与列一一对应，“不适用”或空白写 null；可加行级 page、alias）、可选 note。金额列单位换算为元，其他单位（%、pp、吨、元/吨等）原值保存，外币金额拒收
ratios[]	派生比率由程序算：id、label、num、den（引用 [表 id，行名，列名]、["truth"，指标，期间] 或 ["ratio"，比率 id]）、unit、可选 op（缺省除法；growth 同比；diff 差值）、scale、alias

C.3 sources.json：外部来源登记（verify-sources 校验）

字段	说明
questions[]	id（Q 编号）、question、status（found / not_found）、queries[]（实际检索词）、sources[]（来源 id）、phase（collect 资料搜集 / followup 补充调研，缺省视为后者）、category（资料搜集阶段必填，七类之一：latest、market、consensus、peers、industry、competition、events）
sources[]	id（须为 S + 数字，唯一）、title、publisher、url（须以 http(s) 开头，只接受实际打开过的网页）、published（YYYY-MM[-DD]，可选）、accessed（YYYY-MM-DD）、facts[]
facts[]	quote（原文摘句，≤ 120 字，必须包含所登记的数字）、value、unit、label

正文引用外部数字时在句末写 [S 编号]；数字护栏把 facts[].value 当作出处，并要求引用它的句子带对应编号。

C.4 claims.json：研报断言（纠错核查）

字段	说明
report_file, extracted_by	顶层：被核查的研报文件、抽取者说明
claims[].metric_id	取自 schema 的 claim_metrics；对不上的断言不写
claims[].period	断言所说的期间；未写年份按研报主报告期
claims[].claim_type	level 绝对值、yoy 同比增速、ratio 比率、pp_change 比率同比变动
claims[].value, unit	原文数字字符串（下降写负数）与原文单位（亿元 / 万元 / 元 / % / 元/股 / pp）
claims[].is_forecast	预测值为 true，程序判“无法核实”，不计为错误
claims[].quote	研报原文中连续出现、包含该数字的一段，程序逐字核对

C.5 valuation_assumptions.json：估值假设（观点 / 假设，非财报事实）

C.6 评测输入输出

判定文件的示例（ASCII 化）：

```
{"items": [
  {"id": "L2-01", "passed": true,  "evidence": "... (summary, p.1)", "reason": "..."},
  {"id": "N-01",  "passed": false, "evidence": "none", "reason": "..."}],
 "summary": "..."}
```

附录 D　十一道闸门

闸门由 report_pipeline.status(ws) 逐项判定（正文见第 11 节）。阈值常量定义在 finagent/report_pipeline.py 顶部（表 127）；表 128 给出每道闸门的完整判定条件。路径中的 <work> 为 data/extractions/< 公司 >_< 年度 >。

字段	说明
status	声明全部为观点 / 假设（如“草稿（Claude 起草，待用户确认）”）
ticker, industry, valuation_date, as_of_date	代码、行业、估值日、报表基准日
current_price, price_source	参考股价与来源（外部数据，阶段 6 闸门要求非空，依据须引用 [S 编号]）
rationale	各项假设的文字依据：forecast、wacc、terminal_value、non_operating、dividend 等
wacc	risk_free_rate、beta、equity_risk_premium、size_premium、company_specific_premium、country_risk_premium、pre_tax_cost_of_debt、marginal_tax_rate、target_debt_pct、target_equity_pct、preferred_pct、pre_tax_cost_of_preferred（小数）
terminal_value	method（如 perpetual_growth）、perpetual_growth_rate、exit_ebitda_multiple
non_operating_assets[]	股权价值桥加回 / 扣减项：label、raw_value、unit、page、sign（+1 加回，−1 扣减）；逐项回查年报
scenarios.{base,downside,upside}	description、revenue_growth[]（由分部预测覆盖）、capex_percent_revenue[]、terminal_growth_rate、wacc_adjustment
three_statement	ar_days_shift（分情景）、dividend_payout
sensitivities	wacc_delta、terminal_growth_delta、exit_multiple_delta、revenue_growth_delta、ebit_margin_delta 五组网格
segment_forecast	note；segments[]（id、name、product_row、volume_row、分情景的 volume_growth[]、price_growth[]、gm_change_pp[]）；expense_ratios（如 selling 分情景逐年费用率）

表 128　十一道闸门的判定条件。

阶段	会话	通过条件
1 资料搜集	研究助理	<work>/research/sources.json 与 dossier.md 都存在；phase=collect 的问题覆盖全部七个类别；这些问题的检索词合计 ≥ 20；发布日期在 as_of 前 90 天内的来源 ≥ 3；verify_sources 无问题
2 财务抽取	分析师	facts.json 存在；facts.verified.json 存在且修改时间不早于 facts.json；其 check.ok 为真（拒收、冲突、勾稽失败均为 0）
3 分析框架	分析师	report/framework.md 存在，含“外部调研清单”字样，且“## 深度专题”一节下编号或列表项 ≥ 3
4 深度调研	分析师	sources.json 存在；research/topics/*.md ≥ 3 份，每份去空白 ≥ 1500 字且独立 [S 编号] ≥ 3；补充检索问题 ≥ 8；sources.json 修改时间不早于 framework.md（确实按框架补查）；verify_sources 无问题
5 经营数据	分析师	extras.json 存在，verify-extra 拒收 0 格
6 估值假设	分析师	valuation_assumptions.json 存在；current_price 与 price_source 非空；文件中至少一处 [S 编号]
7 估值运行	分析师	存在成功的 valuation 运行；其开始时间晚于 facts / extras / 假设 / 可比公司表中最晚的修改时间；带了 --extras（若有 extras）；若资料搜集写了 research/peers.csv 且 build 时未给可比公司，运行带了该 --peers
8 文稿	主笔	narrative.md 存在；运行目录中的 commentary.md 与当前文稿逐字相同（guard 核对的是当前稿）；commentary_guard.json 为 ok、带了 --sources 且晚于 sources.json；正文 ≥ 15,000 字且深度研究占比 ≥ 60%
9 复核改写	复核编辑	narrative.draft.md、edit_log.md、review.md 都存在；numlock(draft, narrative) 通过（改写稿的数字与来源编号是初稿的子集）

阶段	会话	通过条件
10 排版	排版设计师	运行目录下 report/report.pdf 存在且晚于文稿与 build_report.py；另存了按 output_name 命名的 PDF 且与主文件摘要相同；preview 页数在 25～32；图表 ≥ 60；check_render_record 对当前 HTML、PDF、排版代码的宽度与字号检查通过
11 交付说明	排版设计师	report/delivery.md 存在

阻断规则。 闸门按顺序判定，任何一道未通过，其后所有“通过”都被改判为未通过并在详情后注明“（前序阶段未通过）”。status 的 next 字段给出第一道未通过闸门的下一步动作。run_agent 按会话分组推进：会话负责的最后一道闸门（1、7、8、9、11）已通过则跳过该会话；会话结束后其闸门仍未全部通过则停止整个流水线。

附录 E　角色启动提示要点

每个角色由编排器以一次 claude -p 无头会话启动，启动提示（prompt）很短，详细的工作说明在工作区 CLAUDE.md（由 render_guide 生成）与 skill 文档中。生成侧会话命令为 claude -p <PROMPT> --permission-mode acceptEdits --allowedTools < 白名单 > --output-format stream-json --verbose [--model < 模型 >]。

表 129　各角色的启动提示要点（finagent/report_pipeline.py、finagent/evaluation/judge.py）。

提示常量	角色 / 模型	要点
RESEARCH_PROMPT	研究助理 / Opus	只做阶段 1：先读 finagent-report/references/research.md 第一部分，用 WebSearch / WebFetch 把年报之后的最新信息搜齐，登记 sources.json、写 dossier.md；不做财务抽取、不写框架；结束运行 status，阶段 1 通过即停
ANALYST_PROMPT	分析师 / Opus	先读 dossier.md，完成阶段 2～7（抽取、框架含深度专题、深度调研、经营数据、估值假设、估值运行）；不写文稿；每阶段结束跑 status，阶段 7 通过即停
TASK_PROMPT	主笔 / Opus	先读 framework.md、dossier.md、research/topics/ 与运行产物，按阶段 8 写“年报点评 + 深度研究”，多环定量推导、写长写深；guard 与篇幅闸门通过即停
REVIEW_PROMPT	复核编辑 / Sonnet	“没有参与写作”；先读 finagent-review/SKILL.md，只做阶段 9；通过即停
LAYOUT_PROMPT	排版设计师 / Opus	文稿已定稿、不改正文；读 html-report.md 与参考脚本，从模板写本公司排版脚本；每张图服务一个论点，图表 60 张以上、25～30 页，表格涨跌双色，逐页看 preview；完成阶段 10～11
FIX_PROMPT	修订 / Opus	返工第 2 轮起：按问题单修改对应阶段产物（补数据、改假设则重跑估值，文稿改完重跑 guard），处理不了的写进 review.md；阶段 1～8 通过即停
REVISE_PROMPT	复核修订 / Sonnet	读问题单逐条处理：只改措辞、结构与编排，数字锁照常，不覆盖初稿；需补数字或改结论的写进 review.md；重跑 guard 与数字锁，用现有脚本重新渲染（不改版面），更新交付说明
judge.PROMPT	盲评质检员 / Sonnet	“按本目录 CLAUDE.md 评审这份研报，把结果写进 judgments.json 后停止。”指南要点见第 12.4 节

所有生成侧提示都要求“命令直接以 ./py 开头，不加 cd、不串联”，因为无头执行时工具白名单按命令前缀匹配。

工具白名单。 生成侧（ALLOWED_TOOLS）：Bash(./py -m finagent *)、Bash(./py -m finagent.report_pipeline *)、Bash(./py data/extractions/*)、Bash(./py -c *)、Bash(cp data/extractions/*)、Bash(ls *)、Bash(mkdir -p data/*)、Read、Write(./data/**)、Edit(./data/**)、WebSearch、WebFetch。工作区 .claude/settings.json 另设禁止规则：

文件	结构
rubric.json（盲评目录）	列表，每项 id、title、criterion、fail_examples（可无）、negative（布尔）
judgments.json	items[]：id、passed（布尔；负向条目 true 表示命中）、evidence（≤ 80 字原文摘录并标位置，无则"无"）、reason；summary（三五句总评）
checks.json	{条目 id: {passed, detail, evidence}}，只含程序条目
judge_run.json	model、exit_code、started_at、elapsed_sec、turns、cost_usd、input_tokens、output_tokens、blind_dir
score.json	rubric、total、raw、deduction、caps[]（cap、by）、layers（各层 name、earned、possible）、deliverable、not_deliverable_reasons[]、missing[]、items[]（id、layer、title、judge、points、passed、got、route、stage、evidence、reason）、judge_problems[]、judge_summary
issues.json	列表，每项恰为 id（I01…）、route、stage、severity（blocker / major / minor）、description、evidence
variant.json（校准）	kind、rep、targets[]、desc
人工质检导出	schema_version（finagent.manual_review.v2）、company、rubric_version、selection_policy、source_sha256、baseline_total、flagged_count、machine_excel、review_sessions[]（reviewer、exported_at、overall_comment、items[]：item_id、layer、model_flagged、human_reviewed、human_verdict、feedback）

表 127 闸门阈值常量。

常量	值	含义
MIN_COLLECT_QUERIES	20	资料搜集阶段检索次数下限
MIN_RECENT_SOURCES	3	近期来源数下限
RECENT_DAYS	90	"近期"的天数
MIN_TOPICS	3	深度专题数（框架中）与专题笔记份数下限
MIN_TOPIC_CHARS	1500	每份专题笔记去空白后的字数下限
MIN_TOPIC_SOURCES	3	每份专题笔记的独立 [S 编号] 数下限
MIN_FOLLOWUP	8	补充检索问题数下限
MIN_NARRATIVE_CHARS	15000	文稿正文字数下限
MIN_DEEP_SHARE	0.6	深度研究（deep_ 开头的节）占比下限
MIN_PAGES, MAX_PAGES	25, 32	PDF 页数区间（按 preview 页图计）
MIN_FIGURES	60	图表数下限（按 HTML 中"图表 *n*"编号计）

不得读取仓库所在目录及其上级目录；不得编辑或写入 finagent/、.claude/、report/resources/、sample.json、py、py.cmd、CLAUDE.md。盲评会话：Read(./**)、Glob、Grep、Write(./judgments.json)。

工作区 CLAUDE.md 的五条底线。 （1）不做算术：增速、占比、差值、估值都由程序算，正文照产物展示值抄；（2）每个数字来自年报原文（带页码）、运行产物或已登记的外部资料，正文必须过 guard，年报之外的事实先检索、登记、再引用；（3）估值参数是观点 / 假设，须标明，股价须注明来源，评级取程序结果；（4）校验 3 轮仍不过的问题不硬改数字，写进交付说明；（5）五个会话互不继承上下文，各自做完自己的阶段就停止。此外 CLAUDE.md 规定了读取边界（不读引擎源码、不读 runs/ 下 JSON 全文、PDF 按页读且每次不超过 20 页）、"先搜集、后分析"（年报之外的信息一律联网现查，来源优先级：交易所与公司公告 > 政府与行业协会 > 研究机构与数据商 > 主流财经媒体），以及"拿不准但影响结论的内容写进复核清单，不写进正文"。

四个 skill 的分工。 finagent-pipeline（仓库内的编排入口，不进工作区）：搭工作区、起会话、查闸门、收回、测评，自己不写研报。finagent-research（分析师）：铁律、定

位关键页、读页抽取、校验与修正（最多 3 轮）、经营数据、研报断言、运行计算、写解读并过护栏。`finagent-report`（研究助理、主笔、排版）：不可协商的约束与交给模型判断的部分、资料搜集与深度调研方法、分析框架推导、文稿模板、reportkit 排版。`finagent-review`（复核编辑）：边界（只改措辞与结构、不碰数字、不改结论与评级、保留节标记与引用）、怎么读（先通读回答三个问题，再逐段改，再对账，最后通读）、要改掉的写法（否定对照、自证圆场、交代缺失、程序口吻、套话、解释腔、事后圆场、行内标签、数字罗列、重要处定性、无来源判断）与产出（改写稿、`edit_log.md`、`review.md`）。

附录 F　第三方组件

本附录摘自仓库 `THIRD_PARTY.md`。

OpenAI Public Equity Investing 插件。 来源为该插件 0.1.31 版，原插件标注为“Proprietary −Internal Use Only”。本仓库不包含原插件代码：早期版本曾通过 `third_party/` 原样引入，现已全部删除，改为自研实现；仍参照的是计算口径、判定阈值、输出字段名与英文代码值，以保证改造前后结果一致。参照关系如表 130。

表 130　参照原插件的模块与自研实现。

参照的原插件内容	自研实现	一致性校验
三表模型构建（三表联动、营运资本、PP&E、债务与现金归集、勾稽、情景与敏感性、模型状态判定）	`tasks/valuation/ts_engine.py`	浮点逐位相等
情景敏感性生成（三情景目标价与偏度、倍数 / EPS / KPI / 流动性敏感性、跟踪触发表、就绪度）	`tasks/valuation/scenario_tables.py`	输出逐字节相等
业绩深度分析（beat / miss、业绩预告中值对比、EPS 质量筛查规则）	`tasks/financials/earnings.py`	逐字段相等
FCFF DCF 模型	`tasks/valuation/dcf.py`	相对误差 $< 10^{-9}$
可比公司估值说明（EV 桥、倍数配对、NM、IQR 离群值、P25/P75 区间）	`tasks/valuation/comps.py`	—
可比公司英文公式模板	未沿用；`comps.write_formula_workbook` 另行生成中文公式工作簿	公式独立重算与缓存值相等
目标反解思路（“股价隐含了什么”、可行性分档）	`tasks/valuation/reverse_dcf.py`：自研单变量反向 DCF（网格扫描 + 二分），分档自定	仅借方法；`test_reverse_dcf.py` 代回正向 DCF
研报 QC 的重复数字对账规则与问题分类	`tasks/report_check/consistency.py`（N-02、guard、纠错核查）	仅借方法；`test_consistency.py`
首次覆盖论点框架、论点跟踪工作流	`finagent-report/references/analysis.md`、`sections.md`：中文自写的写作要求	文档，无计算
模型审计中的公式与工作簿控制	`evaluation/workbook_audit.py` 的 `formula_controls`	仅借方法；`test_workbook_audit.py`
财务归一化的证据标签与来源协议	`output/guard.py` 的 `tagged_truth_values` / `classify_text`：标签体系自定（中文五类）	仅借方法；`test_provenance.py`

校验方式：`tests/golden/` 为改造前用原插件引擎跑样例得到的产物快照，`tests/test_golden.py` 要求自研实现的产物与之完全相等（无需原插件即可运行）；2026-09-28 因有意修改口径（EBIT 剔除投资收益与公允价值变动、三表经营费用改为轧差、DCF 由三表驱动、comps 增加利润率）重新生成了快照，各引擎本身未改。

tests/test_parity_reference.py 在本地存在原插件时（FINAGENT_REFERENCE_PLUGIN）用样例、边界值和数百组随机输入逐项对照，找不到原插件时跳过。

名称	版本	许可证	使用范围
pdfplumber	≥0.11	MIT	PDF 文本与表格抽取
pytest	≥8.0	MIT	测试
fpdf2	2.8	LGPL-3.0	仅开发时生成样例 PDF，非运行依赖
Textual	≥1.0（开发时 8.2.8）	MIT	仅终端界面 finagent_tui/，可选
Rich	随 Textual（开发时 15.0.0）	MIT	终端界面的表格与文本渲染

开源依赖。 requirements.txt 另列可选的 matplotlib（≥3.8，研报画图）与 pillow（≥10，预览缩略图）。finagent/ 其余部分只用 Python 标准库（含 xlsx 写出）。样例 PDF 内嵌 macOS 系统字体 Arial Unicode 的字形子集，仅用于显示样例数据。

大语言模型与执行器。 Claude（Anthropic）：生成侧默认 Claude Opus 5.5，复核与盲评默认 Claude Sonnet 5.5，经无头 Claude Code 调用，由 finagent/report_pipeline.py 逐角色启动；用于联网检索与登记来源、读年报抽取数字、拆研报断言、起草估值假设、写文稿、复核改写、写排版脚本与逐条评审。所有计算、校验与计分由本项目代码完成。Claude Code 为外部命令行工具，用户自行安装与登录，不随源码分发。

自主开发部分。 finagent/ 下全部代码自主开发：追踪与编排、多 Agent 编排器、文档解析、A 股指标字典与抽取、财务分析与异常信号、利润变动归因、业绩预期对比、DCF、三表联动、情景敏感性、相对估值与公式工作簿、研报核查、报告与 xlsx 输出、评测与测试。其中参照原插件计算规则的模块如上表所列。

附录 G　复现说明

环境。 Python 3.12（scripts/setup.sh 接受 ≥3.10）；引擎只依赖标准库与 pdfplumber。深度研报流水线另需 Node.js 18+、已登录的 Claude Code （claude auth login 或 ANTHROPIC_API_KEY）、Chrome / Chromium / Edge（HTML 打印 PDF，可用 FINAGENT_CHROME 指定）与中文字体。模型由 FINAGENT_GEN_MODEL（默认 claude-opus-5-5）与 FINAGENT_QC_MODEL（默认 claude-sonnet-5-5）配置；工作区默认在 ~/finagent-samples/（FINAGENT_SAMPLES_DIR），盲评目录默认在 ~/finagent-evals/（FINAGENT_EVAL_DIR），二者都必须在仓库目录树之外。

准备。

```
bash scripts/setup.sh              # venv + requirements + Claude Code check + .env + doctor
bash scripts/setup.sh --install-claude   # install Claude Code via npm if missing
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1   # Windows
python -m finagent.doctor          # --live sends one minimal request per model (billed)
```

setup.sh 依次完成：建 .venv 并安装 requirements.txt；检查 claude 命令；缺 .env 时从 .env.example 复制；运行 doctor 自检。以下命令假定已激活 .venv；否则 macOS / Linux 用 .venv/bin/python，Windows 用 .venv\Scripts\python.exe。

一、不调用模型即可复现的部分。

```
python -m pytest -q tests
python eval/run_eval.py
python -m finagent all --input data/samples/annual_report_2024.md --year 2024 \
  --company <sample-company> --assumptions data/samples/valuation_assumptions.json \
  --report data/samples/research_draft.md --peers data/samples/peers.csv \
  --estimates data/samples/estimates.csv --guidance data/samples/guidance.csv
python -m finagent verify-facts --input "report/resources/<company>/<annual-report>.pdf" \
```

```
  --year 2025 --facts data/extractions/<company>_2025/facts.json
python -m finagent valuation --input "<annual-report>.pdf" --year 2025 --company <company> \
  --facts .../facts.json --extras .../extras.json --assumptions .../valuation_assumptions.json
python -m finagent.evaluation score --result eval/results/v2_<company>
python -m finagent.evaluation.calibrate report
```

（代码清单中的 <sample-company> 为虚构样例公司“星河智造”，<company> 为仓库中已收回抽取文件的公司，如贵州茅台、宁德时代、招商轮船；清单只能用 ASCII，故以占位符表示。）

预期输出：

- 测试：164 个通过；本地没有原参照插件时跳过 5 个逐项对照测试。
- 样例评测（eval/run_eval.py，规则通道）：字段抽取 35/35；研报纠错召回率、准确率 100%，误报 0。样例数据为虚构且内部自洽，只说明流程正确。
- 真实年报：用 data/extractions/ 中保存的抽取文件重跑 verify-facts 与 valuation，程序重新做逐字证据校验、规则通道比对与全部计算，结果与提交时一致；茅台、宁德、招商三家 301 个核验值中规则通道取到且一致 280 个，冲突 0，勾稽 56/56 通过。
- 评测：score 用已保存的 checks.json 与 judgments.json 重算，无需模型，得到表 104 的分数；calibrate report 由 eval/calibration/ 重新生成校准报告（表 98、100、101）。

二、需要模型会话的部分。

```
python -m finagent.report_pipeline build --company <company> --year 2025 \
  --price <price> --price-source "<source>"
python -m finagent.report_pipeline refine --workspace "<workspace>"   # generate + QC loop
python -m finagent.report_pipeline run --workspace "<workspace>" [--dry-run]
python -m finagent.report_pipeline status --workspace "<workspace>"
python -m finagent.report_pipeline collect --workspace "<workspace>" --label Opus
python -m finagent.report_pipeline scorecard --workspace A --workspace B
python -m finagent.evaluation run --workspace "<workspace>"
python -m finagent.evaluation.calibrate run --workspace "<workspace>" --repeats 3 --jobs 4
```

build 拒绝未登记在 manifest.json 中的年报（来源与版本未核对，需加 --allow-unverified），且 --price 必须同时给 --price-source。会话产出受联网检索结果与模型采样影响，不能逐字复现；可复现的是每个会话的完整事件流（.pipeline/agents/*.jsonl）、运行记录（.pipeline/agent_runs.json）、全部中间产物及其哈希，以及在这些产物上重跑的全部程序计算与程序检查。collect 把抽取与文稿收回 data/extractions/< 公司 >_< 年度 >/，运行目录收回 runs/，PDF、sheet.xlsx 与复核清单收回 report/交付/<MMDD> < 公司 > < 标签 >/。

统一命令行。 pip install -e ".[tui]" 后可用 finagent：research（引擎任务与校验）、report（研报流水线）、eval / calibrate（质检与校准）、bench / test / doctor、ls / tui（终端界面）。

修改样例时。 改样例年报须改 data/samples/build_samples.py 后重新生成（同时生成样例三季报），并同步更新 eval/gold_financials.json、eval/labels.json、样例 PDF（build_pdf.py）与 data/extractions/ 下的抽取文件；改完代码须跑测试与样例评测，样例评测指标下降视为回归。